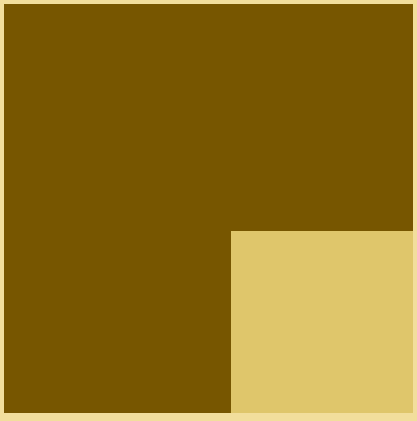
MATHEMATICS NOTES

数学笔记

卷三 · 图



施晓罡

2026 年 · 第一版

MATHEMATICS NOTES

数学笔记

卷三 · 图

施晓罡

2026 年 · 第一版

数学笔记

MATHEMATICS NOTES

作者：施晓罡

版本：第一版

年份：2026

Copyright © 2026 施晓罡

Licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (the ”License”);

You may not use this file except in compliance with the License.

You may obtain a copy of the License at

https://creativecommons.org/licenses/by-nc-nd/4.0/

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an ”AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

前言

本书目标

给定一个对象，我们怎样描述它的结构，判断它具有哪些性质，并据此进行计算？集合、函数、向量与图提供不同的描述方式；极限、测度与概率使这些描述能够处理逼近、连续变化与随机性；优化、微分方程和控制则进一步研究解怎样求得、状态怎样演化，以及行动怎样影响结果。本书围绕这些问题整理数学概念、推导与计算方法。

这份笔记从《机器学习笔记》的数学部分独立出来，供需要系统梳理相关数学工具的读者阅读。各章在具体对象和问题中引入定义，说明结论依赖的条件，并通过证明、算例和反例检查其适用范围。阅读时可以沿正文顺序建立基础，也可以从当前问题出发查阅相应章节，再补充所需的前置内容。

内容与阅读

全书按十卷组织。卷一《集合与一元微积分》建立集合、关系、映射与计数的语言，再研究实数上函数的极限、微分与积分；卷二《向量、矩阵与多元微积分》研究线性空间、谱分析、线性方程的求解与逼近，以及多元函数的微分与积分；卷三《图》研究图的结构与遍历、图上的组合优化、集合函数与子集选择，以及图上的动态规划。

此后各卷依次讨论度量、测度、函数空间与流形，优化，微分方程，概率，推断与因果识别，状态的演进与观测，以及决策、控制与博弈中的策略。目前已完成前三卷，其余各卷仍在整理；正文提到这些卷时，按全书计划的卷章位置给出可查阅之处。各卷独立构建，章节之间按需引用。

正文使用其他数学卷的结论时，会在当前问题中交代所需关系与条件，再给出可查阅的位置。少量机器学习应用作为延伸阅读，链接指向《机器学习笔记》的固定版本；本书的数学正文与编译过程独立维护。

数学符号

下表汇总全书反复使用的记号。各章若赋予符号更具体的含义，会在首次使用时另行说明。

对象与线性代数

对象	常用记号	含义
数集	$\mathbb{N}$, $\mathbb{Z}$, $\mathbb{R}$, $\mathbb{R}_+$	自然数集、整数集、实数集和非负实数集
集合与关系	$x \in A$, $A \subseteq B$, $A \cup B$, $A \cap B$, $\lvert A \rvert$	元素属于集合、子集、并集、交集和有限集合的基数
标量	a, x, $\lambda \in \mathbb{R}$	单个数值；小写斜体字母通常表示标量
向量	$\boldsymbol{x}$, $\boldsymbol{w} \in \mathbb{R}^d$	d 维向量；加粗小写字母表示向量
矩阵	$\boldsymbol{A} \in \mathbb{R}^{m \times n}$, $\boldsymbol{A}_{i,j}$	m 行 n 列矩阵及其第 i 行、第 j 列元素
数据与样本	$\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$	含 n 个输入标签对的数据集；下标 i 表示样本索引
切片与迭代	$\boldsymbol{A}_{i,:}$, $\boldsymbol{A}_{:,j}$, $\boldsymbol{w}^{(t)}$	矩阵的第 i 行、第 j 列，以及第 t 次迭代的向量
Kronecker delta	$\delta_{i,j}$	当 $i = j$ 时取 1，否则取 0
矩阵变换	$\boldsymbol{A}^\top$, $\boldsymbol{A}^{-1}$, $\boldsymbol{A}^\dagger$, $\boldsymbol{I}_d$, $\operatorname{tr}(\boldsymbol{A})$	转置、逆、伪逆、d 阶单位矩阵和矩阵的迹
代数运算	$\langle \boldsymbol{x}, \boldsymbol{y} \rangle$, $\boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x}$, $\sum_i a_i$, $\prod_i a_i$	内积、二次型、求和与连乘
函数与模型	$f : \mathcal{X} \to \mathcal{Y}$, $h_{\boldsymbol{w}}(\boldsymbol{x})$	从输入空间到输出空间的函数，以及参数为 $\boldsymbol{w}$ 的模型
范数与距离	$\lVert \boldsymbol{x} \rVert_p$, $d(\boldsymbol{x}, \boldsymbol{y})$	p 范数与两对象之间的距离；具体定义由上下文给出

微积分与优化

对象	常用记号	含义
导数与梯度	$\dfrac{\mathrm{d}f}{\mathrm{d}x}$, $\nabla_{\boldsymbol{w}} L(\boldsymbol{w})$	一元导数，以及标量值函数对参数向量的梯度
Jacobian 与 Hessian	$\nabla_{\boldsymbol{x}} \boldsymbol{f}$, $\nabla_{\boldsymbol{x}}^2 f$	向量值函数的一阶偏导矩阵，以及标量值函数的二阶偏导矩阵
渐近阶	$f(n) = \mathcal{O}(g(n))$	函数增长率的渐近上界
最优化	$\min_{\boldsymbol{w}} L(\boldsymbol{w})$, $\boldsymbol{w}^* \in \arg\min_{\boldsymbol{w}} L(\boldsymbol{w})$	最小目标值与取得最小值的参数；最大化使用 max 和 arg max

概率与信息论

对象	常用记号	含义
随机变量与实现	$X,\ \boldsymbol{X},\ x,\ \boldsymbol{x},\ p_X(x)$	随机标量、随机向量及其实现值；p_X 表示概率质量或密度函数
联合与条件概率	$p(x,y),\ p(x \mid y)$	随机变量实现值的联合概率与条件概率
边缘概率	$p(x)=\sum_y p(x,y),\ p(x)=\int p(x,y)\,\mathrm{d}y$	分别对离散变量求和或对连续变量积分得到的边缘概率
独立性与抽样	$X \perp Y,\ X \perp Y \mid Z,\ X_i \overset{\text{i.i.d.}}{\sim} p$	独立、给定 Z 条件独立，以及独立同分布抽样
矩与相关性	$\mathbb{E}[X],\ \mathrm{Var}(X),\ \mathrm{Cov}(X,Y)$	期望、方差和协方差
常用分布	$\mathrm{Bernoulli}(p),\ \mathrm{Cat}(\boldsymbol{\pi}),\ \mathcal{N}(\boldsymbol{\mu},\boldsymbol{\Sigma})$	伯努利分布、类别分布和多元高斯分布
事件指示函数	$\mathbb{I}\{A\}$	事件 A 成立时取 1，否则取 0
信息量	$H(p),\ H(p,q),\ \mathrm{KL}(p\|\|q)$	熵、交叉熵和分布 p 相对于 q 的 KL 散度
概率正比关系	$p(\boldsymbol{x}) \propto q(\boldsymbol{x})$	忽略归一化常数时的概率正比关系

目录

第一部分 **图结构与结构计算** 1

第 1 章 图与连接结构 3

1.1 图的基本概念 3

1.2 邻接与路径 8

1.3 连通与分隔 13

1.4 树与森林 17

1.5 图的表示 22

1.6 图同构 27

1.7 本章小结 31

第 2 章 图的遍历与结构计算 33

2.1 广度优先遍历 34

2.2 深度优先遍历 39

2.3 连通结构的计算 45

2.4 拓扑排序 56

2.5 基于分隔集的分块 61

2.6 顶点标记与结构比较 64

2.7 本章小结 68

第二部分 **图上的组合优化** 71

第 3 章 最短路 74

3.1 路径成本与最短路问题 74

3.2 距离标签与松弛 76

3.3 有向无环图上的最短路 78

3.4 Bellman–Ford 算法 81

3.5	Dijkstra 算法	86
3.6	方法比较与结果检验	89
3.7	本章小结	90
第 4 章	最小生成树	92
4.1	最小生成树问题	92
4.2	割性质与安全选边	93
4.3	Kruskal 算法	94
4.4	Prim 算法	97
4.5	方法比较与结果检验	99
4.6	本章小结	100
第 5 章	最大流与最小割	101
5.1	流、容量与割	101
5.2	残量网络与增广	102
5.3	Edmonds–Karp 算法	107
5.4	Dinic 算法	111
5.5	方法比较与结果检验	115
5.6	本章小结	115
第 6 章	二部图匹配与指派	117
6.1	二部图匹配	117
6.2	指派与带权二部图匹配	124
6.3	本章小结	136
第 7 章	最小费用流	138
7.1	最小费用流问题	138
7.2	费用残量网络与最优性条件	140
7.3	网络单纯形法	144
7.4	最大流、匹配与指派的统一表示	152
7.5	结果检验	153
7.6	其他改进路线	154
7.7	本章小结	155
第三部分	集合函数与子集选择	159
第 8 章	集合函数与子集累计	161

8.1 集合函数 161
8.2 集合函数的性质 165
8.3 子集累计与 Möbius 反演 172
8.4 本章小结 181

第 9 章 约束下的子集选择 183
9.1 基数约束下的子集选择 183
9.2 预算约束下的子集选择 191
9.3 本章小结 196

第 10 章 成对二值函数的最小化 197
10.1 问题与思路 197
10.2 次模条件与割表示 199
10.3 算法实现 202
10.4 算法分析 204
10.5 本章小结 205

第四部分 图上的动态规划 207

第 11 章 局部函数与树上动态规划 209
11.1 局部函数与计算目标 209
11.2 链与树上的接口递推 215
11.3 从树到一般图 226
11.4 本章小结 230

第 12 章 一般图上动态规划的构建 231
12.1 构建问题与输出要求 231
12.2 由变量消元构建动态规划 238
12.3 由树分解构建动态规划 244
12.4 两种构建的等价关系 249
12.5 构造策略 251
12.6 本章小结 263

第 13 章 一般图上动态规划的计算 265
13.1 局部项归属与求值顺序 265
13.2 最小成本 268
13.3 带保留变量的最小成本 277

13.4 计数和求和 280
13.5 计算代价 286
13.6 本章小结 289

参考文献 291

索引 293

第一部分

图结构与结构计算

Graph Structure and Computation

阅读指引

道路网、任务依赖、人员与岗位的配对，都涉及有限个对象之间的直接连接。图用顶点表示对象，用边记录直接连接；方向、边权和属性再分别补充先后、数值和附加数据。本卷围绕这类对象展开四个部分：本部分建立图的结构并把结构算出来，第 2部分在给定边权与约束下选择最优的路径、生成树、流和匹配，第 3部分把选择对象推广为一般有限集的子集，第 4部分在图的分解上组织局部函数的动态规划。

本部分的两章分别回答“结构是什么”和“结构怎样算”。第 1章定义行走与路径、连通分量、强连通与弱连通、分隔集、树与生成树，比较边列表、邻接表、邻接矩阵和关联矩阵四种表示，并用同构区分结构差异与单纯改名。这一章的例子都小到可以手工核对，每个概念都配有核对给定结果的方法。第 2章以邻接表为输入，用广度优先遍历和深度优先遍历在与顶点数加边数成正比的时间内求出最少边数、连通分量、强连通分量、拓扑序，以及删除给定分隔集后得到的局部块，再用顶点标记细化比较两张图的结构。

第 2章的每个算法都输出第 1章定义的对象，所以前一章的定义与核对方法也是检验算法输出的标准。阅读时要注意三组区分。第一，结构上的连接与边上的数值不同：本部分只问能否连接、按什么次序，最少边数不是最短长度，生成森林也不比较总权，带权的优化属于下一部分。第二，必要条件与充分判据不同：度数、分量数和标记计数等不变量可以证明两图不同构，却不能证明同构。第三，无向与有向不同：有向图的可达不对称，连通要分强弱两种，环与无环的判断也各有规则。

图上顶点和边的数值函数、拉普拉斯矩阵、图谱与图滤波不在本卷讨论。本部分只说明邻接矩阵与关联矩阵怎样记录连接，并用矩阵幂计数行走；以这些矩阵为基础的图上算子与谱方法，可参考卷九第三部分“图信号处理”。

第 1 章

图与连接结构

城市之间修了哪些道路，一项工程里哪项任务必须等另一项完成，这类信息都由“有限个对象”和“对象之间的直接连接”组成。我们关心的往往不是某一条连接本身，而是由连接串起来的整体：从一座城市能否开车到达另一座，拆掉一座桥以后交通网会不会断成两片，一组任务能否排出不违反依赖的先后次序。回答这些问题之前，先要说清图记录了什么、没有记录什么。

本章用集合、关系与映射的语言建立图这一对象，并沿“一步连接—多步连接—整体结构”的顺序展开。第 1.1节规定图记录哪些连接，方向、边权和子图分别附加或取出什么信息。一步的连接首尾相接，得到第 1.2节的行走、路径与可达关系。无向图中的可达关系是等价关系，第 1.3节据此把顶点划分为连通分量，并考察删除顶点或边怎样切断连接。

切断连接的研究说明，环上的边删去后图仍然连通。所以用最少的边保持连通的结构没有环，这就是第 1.4节的树：它的路径唯一，边数恰为顶点数减一。最后两节转向记录与比较。第 1.5节比较四种表示的访问代价，其中邻接矩阵把行走计数变成矩阵乘法；第 1.6节用同构判断两张图是否只差一套顶点名称。全章用同一张四个顶点、五条边的小图作贯穿例子。

本章只建立定义、基本性质和对给定结果的核对方法，所用的例子都小到可以手工完成。在大图上系统地求出可达集、连通分量、生成树和拓扑次序，需要广度优先遍历与深度优先遍历等算法，这是第 2章的任务。带权路径的最短成本和总成本最小的生成树，分别在第 3章和第 4章讨论。

1.1 图的基本概念

道路网记录哪些城市之间有直达道路，任务清单记录哪项任务直接依赖哪项任务。两者都由有限个对象和对象之间成对的直接连接组成，图就是描述这类信息的数学对象。本节先给出最简单的无向图，再根据实际需要依次加入方向和数值，最后讨论怎样从一张图中取出一部分。

1.1.1 图的定义

同一张道路网可以画成很多样子：城市的位置可以挪动，道路可以画直或画弯，两条道路在纸上还可能交叉。要判断两幅画描述的是不是同一张网，先得说清哪些内容是数据，哪些只是画法。对于“能否直达”这个问题，数据只有两样：有哪些城市，哪些城市之间有直达道路。

顶点集与边集　把每个对象记为一个顶点，把每一对有直接连接的对象记为一条边，就得到一张图。这里先考虑没有方向的连接，例如可以双向通行的道路。

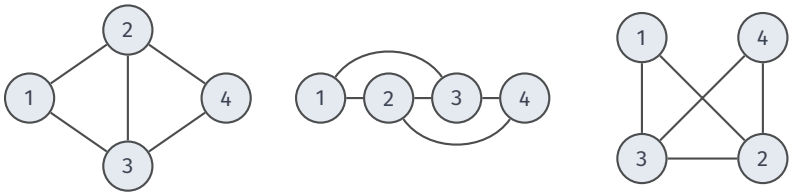
> **定义 1.1　图**　设 V 是有限集合，E 是由 V 中两个不同元素组成的无序对 $\{i,j\}$ 的集合，称 $\mathcal{G}=(V,E)$ 为一个（无向）图（graph）。V 中的元素称为顶点（vertex），E 中的元素称为边（edge）。若 $\{i,j\}\in E$，称 i,j 是这条边的两个端点。

记 $n=|V|$ 为顶点数，$m=|E|$ 为边数。全章使用同一个例子：

$$V=\{1,2,3,4\},\qquad E=\big\{\{1,2\},\{1,3\},\{2,3\},\{2,4\},\{3,4\}\big\},\tag{1.1}$$

它有 $n=4$ 个顶点、$m=5$ 条边，可以看成两个三角形 $\{1,2,3\}$ 与 $\{2,3,4\}$ 共用边 $\{2,3\}$。四个顶点之间最多有 $\binom{4}{2}=6$ 条边，这里只缺 $\{1,4\}$。

图相等的判断　既然图只由 V 和 E 确定，比较两幅图时只需读出各自的顶点集，把每条线段写成由两个端点组成的无序对，再比较两个顶点集和两个边集。图 1.1的三幅画顶点位置、线段形状和交叉情况都不同，读出的集合却都是式(1.1)，因此是同一张图。读边时只看端点，所以画面上的交叉点不是顶点，线段的长短也不是边的长度。问题若需要长度、坐标这样的数值，必须另外给出，见第 1.1.3小节。



(a) 菱形画法　(b) 排成一行　(c) 边在画面上交叉

图 1.1　贯穿全章的四点五边图(1.1)的三种画法。顶点位置、线段形状和交叉情况都不同，但边集都是 $\{1,2\},\{1,3\},\{2,3\},\{2,4\},\{3,4\}$。(c) 中两条对角边在画面上交叉，交叉处不是顶点。

简单图 定义 1.1要求边的两个端点不同，并且边集是普通集合，因此排除了一个顶点连到自身的**自环**（self-loop），也排除了同一对顶点之间的多条**平行边**（parallel edges）。满足这两条限制的图称为**简单图**（simple graph），本部分默认讨论有限简单图。简单图的每条边都是 V 的一个二元子集，而 n 元集合的二元子集共有 $\binom{n}{2}$ 个，所以 $m \le \binom{n}{2}$。

如果两条平行道路需要分别记录收费，就得给每条边一个独立的身份，这时上面的边数上限也不再成立；用到这类扩展时会明确指出。另一个边界情形是顶点集为空的图。它只在少数论证中出现；结论需要至少一个顶点时，我们写明“非空图”。

1.1.2 无向图与有向图

道路可以双向通行，任务之间的依赖却有先后：“任务 i 完成后才能开始任务 j”与“任务 j 完成后才能开始任务 i”是两回事。无序对满足 $\{i,j\}=\{j,i\}$，读不出谁在前。把每个依赖写成先做的任务在前的有序对 (i,j)，次序就成了数据的一部分。

> **定义 1.2　有向图**　设 V 是有限集合，$E \subseteq \{(i,j) \in V \times V : i \neq j\}$，称 $\mathcal{G}=(V,E)$ 为**有向图**（directed graph）。有序对 $(i,j) \in E$ 称为从 i 到 j 的**弧**（arc），也记作 $i \to j$；i 是它的起点，j 是它的终点。

为了区分，以无序对为边的图也称**无向图**（undirected graph）。有向图的弧集仍记作 E，弧数仍记作 m。弧 $i \to j$ 只允许从 i 一步到达 j，并不包含反向的 $j \to i$。

图 1.2对照了无向边 $\{u,v\}$ 与一对反向弧 $(u,v),(v,u)$。两者都允许在 u,v 之间来回，区别在于无向边是一条连接，删除它就同时失去两个方向；两条弧是两个独立的对象，可以只删除其中一条，也可以分别附上不同的数值，例如上坡和下坡的用时。所以用一对反向弧代替每条边来改写无向图时，m 条边变成了 $2m$ 条弧，边的个数和身份都变了。



(a) 无向边 $\{u,v\}$　　(b) 两条弧 (u,v) 与 (v,u)

图 1.2　一条无向边 (a) 与一对反向弧 (b) 都允许在 u,v 之间往返，但 (b) 中的两条弧是两个独立对象，可以分别删除或另附数值。

方向还决定了从哪里能走到哪里。图 1.3给同一条三点连接 1—2—3 赋予不同的方向。不定向时，三个顶点两两可以互相到达；定向为 $1 \to 2 \to 3$ 后，从 1 可以经 2 到达 3，从 3 却哪里也去不了；定向为 $1 \to 2 \leftarrow 3$ 后，1 和 3 都只能到达 2。



(a) 无向：两两可达　(b) 1 可到 3，3 不能回 1　(c) 1, 3 都只能到 2

图 1.3　同一条三点连接的无向形式 (a) 与两种定向 (b)(c)。箭头决定了从哪个顶点能走到哪个顶点。

底层无向图　有时只关心两个对象之间有没有直接联系，而不在乎谁先谁后。这时把每条弧 (i, j) 改写成无序对 $\{i, j\}$ 放进一个新的边集；(i, j) 与 (j, i) 同时出现时，集合中只留下一条边。得到的无向图称为原图的**底层无向图**（underlying undirected graph）。图 1.3中 (b)(c) 的底层无向图都是 (a)。这样做丢掉了先后：(a) 中 3 能到达 1，(b) 中却不能，因此不能根据底层无向图断言原来的有向图可以双向到达。第 1.3节定义弱连通时会用到底层无向图。

1.1.3　边权与属性

许多问题还需要每条连接上的一个数：道路有多长，管道每秒最多输送多少，两个样本有多相似。给定映射 $w : E \to \mathbb{R}$，它在一条边 $\{i, j\}$（或弧 (i, j)）上的值称为**边权**（edge weight）；没有边的顶点对没有边权。

同一组边权在不同问题中参与的运算可以完全不同。图 1.4在同一边集上附加同一组数，给出三种读法。读作长度 $\ell_{i,j}$ 时，数值沿路线相加，路线 $1, 2, 4$ 的总长是 $2 + 1 = 3$，越小越好；读作容量 $c_{i,j}$ 时，数值是单条边的流量上限，边 $\{1, 3\}$ 上至多通过 1 个单位，并不沿路线相加；读作相似度 $w_{i,j}$ 时，权重越大，后续计算越强地要求两端的取值接近。所以拿到一张带权图，要先弄清权重的用途。

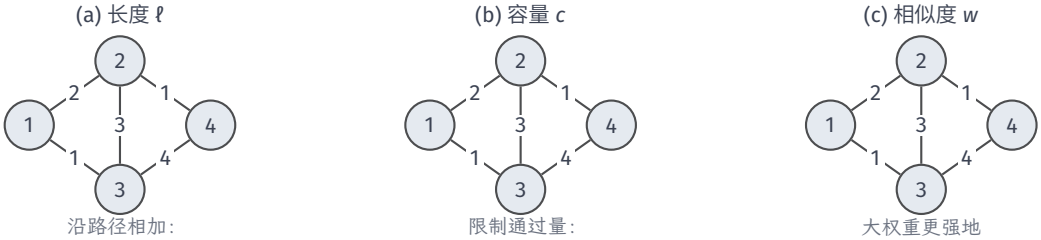


图 1.4　同一边集上的同一组边权在三种问题中的读法。(a) 长度沿路线相加，路线 $1, 2, 4$ 的总长为 3；(b) 容量限制单条边的通过量；(c) 相似度越大，越要求两端取值接近。

无边与零权边　边权为 0 不等于没有边。零长度的道路表示免费通行，零容量的管道不能输送正流量，零相似度在加权求和中不贡献数值，但这些边在结构上都还在。图 1.5中，两种情形在数值记录里都是 0，只有 (b) 中

顶点 1, 2 相邻。能否一步到达要查边集，沿边累计多少才查边权。第 1.5节用矩阵记录图时，这一区别会变成具体的存储问题。

边权能否为负同样由问题决定，定义本身不作限制。负长度会让“沿路线相加取最小”出现新的困难，第 3章专门处理；以相似度为权的计算则通常要求权重非负。使用带权图的结论时，要先确认它对边权符号的要求。



(a) 无边　　(b) 零权边

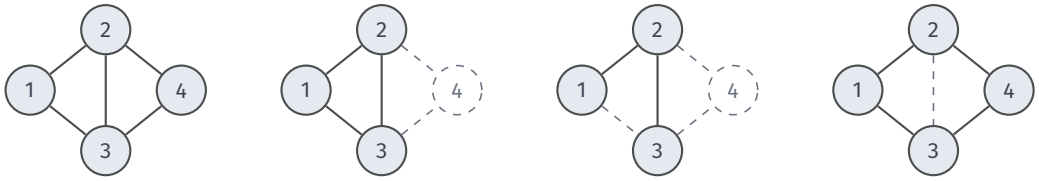
图 1.5　无边 (a) 与零权边 (b)。二者在数值上都记为 0，但只有 (b) 中顶点 1, 2 相邻。

顶点属性与边属性　边上还可以记录类别、时间等数据，顶点上也可以记录人口、坐标等数据，分别称为边属性和顶点属性。顶点的名称 1, 2, 3, 4 只用来区分顶点，本身不是属性。第 1.6节会看到，换一套名称不改变图的结构，而属性是否也必须保持，要由问题来规定。

1.1.4　子图与诱导子图

研究道路网时，我们常常只关心其中一部分城市，或者想知道封闭几条道路以后还剩下什么。取出的部分要仍是一张图，只需保证留下的每条边两个端点都还在。正式地说，若 $V' \subseteq V$，$E' \subseteq E$，并且 E' 中每条边的两个端点都在 V' 中，称 $\mathcal{G}' = (V', E')$ 是 $\mathcal{G}$ 的**子图**（subgraph）。

最常用的两种子图来自删除操作。删除一条边 e 只去掉这条边，记作 $\mathcal{G} - e$；删除一个顶点集 S 则去掉 S 中的顶点，连同所有至少有一个端点在 S 中的边，记作 $\mathcal{G} - S$。删除顶点时必须连带删边，否则留下的边会缺少端点，不再构成子图。图 1.6(d) 只删除了边 {2, 3}。(b)(c) 保留了同一组顶点 {1, 2, 3}，边集却不同，可见只给出顶点集合还不能确定一个子图。



(a) 原图 $\mathcal{G}$　(b) 删除顶点 4：$\mathcal{G} - \{4\} = \mathcal{G}[\{1,2,3\}]$　(c) 同样三个顶点，缺少 {1, 3}：非诱导　(d) 只删除边 {2, 3}，保留全部顶点

图 1.6　贯穿图的几种子图，灰色虚线表示被删除的顶点或边。(b) 删除顶点 4 及其两条边，恰好得到顶点集 {1, 2, 3} 上的诱导子图；(c) 保留同样三个顶点，却漏掉了原图中的边 {1, 3}，是子图但不是诱导子图；(d) 只删除一条边，顶点全部保留。

诱导子图　只研究一部分顶点时，通常不希望悄悄改变这些顶点之间的关系，于是把原图中两端都在其中的边全部保留下来。

定义 1.3　诱导子图　设 $S \subseteq V$。以 S 为顶点集、以 $\{\{i,j\} \in E : i,j \in S\}$ 为边集的子图，称为 $\mathcal{G}$ 在 S 上的**诱导子图**（induced subgraph），记作 $\mathcal{G}[S]$。有向图的诱导子图保留两端都在 S 中的全部弧。

与一般子图不同，诱导子图由顶点集唯一确定。构造 $\mathcal{G}[S]$ 时，标记 S 中的顶点，再逐条扫描边，两端都有标记的就保留。对贯穿图取 $S=\{1,2,3\}$，保留 $\{1,2\},\{1,3\},\{2,3\}$，舍去含 4 的两条边，得到图 1.6(b)。核对一个声称的诱导子图，还要反过来检查原图中两端都在 S 中的边是否一条不漏；图 (c) 漏掉了 $\{1,3\}$，所以不是诱导子图。

删除顶点得到的也是诱导子图。$\mathcal{G}-S$ 删去的恰好是至少一端在 S 中的边，留下的恰好是两端都在 $V\setminus S$ 中的边，所以 $\mathcal{G}-S=\mathcal{G}[V\setminus S]$；例如图 (b) 既是 $\mathcal{G}[\{1,2,3\}]$，也是 $\mathcal{G}-\{4\}$。第 1.3节讨论删除顶点能否切断连接，第 2章按分隔集把图分块，用到的都是这两种写法。

1.2　邻接与路径

贯穿图(1.1)中顶点 1 和 4 之间没有边，却可以先到 2 或 3，再到 4。一步能到谁，由边集直接读出；若干步能到谁，要把边首尾相接。本节从一个顶点的直接邻居出发，把一步的连接延长为行走和路径，由此得到环与可达关系。比较不同路线的总长度是第 3章的问题，这里只讨论哪些顶点序列允许通行。

1.2.1　邻接、邻域与度

站在一座城市，首先要知道哪些城市与它有直达道路，这样的城市有几座。在无向图中，若 $\{i,j\}\in E$，称 i 与 j**邻接**（adjacent），互为邻居。顶点 i 的全部邻居组成它的**邻域**（neighborhood）

$$N(i)=\{j\in V:\{i,j\}\in E\},$$

邻居的个数 $d_i=|N(i)|$ 称为 i 的**度**（degree）。贯穿图中 $N(1)=\{2,3\}$，$N(2)=\{1,3,4\}$，四个顶点的度依次为 $2,3,3,2$。度只数直接邻居，经两步才能到达的 4 不计入 $N(1)$。

在有向的依赖关系中，还要分清“我直接依赖谁”和“谁直接依赖我”：弧 $i\to j$ 对两端的意义不同。因此分别定义出邻域与入邻域

$$N^{+}(i)=\{j:(i,j)\in E\},\qquad N^{-}(i)=\{j:(j,i)\in E\},$$

二者的大小 d_i^{+} 与 d_i^{-} 分别称为**出度**（out-degree）与**入度**（in-degree）。图 1.7在贯穿图和一个小有向图上标出了这些量。

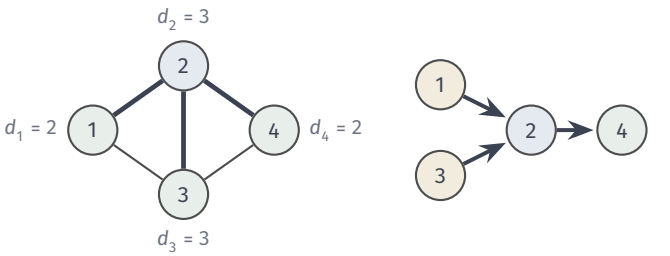


图 1.7 邻域与度。(a) 贯穿图中顶点 2 的邻域 $N(2) = \{1, 3, 4\}$（绿色），连接它们的三条边加粗，所以 $d_2 = 3$；顶点旁标出各点的度，四个度之和 10 恰为边数的两倍。(b) 有向图 $1 \to 2$、$3 \to 2$、$2 \to 4$ 中，顶点 2 的入邻域 $N^-(2) = \{1, 3\}$（黄色）与出邻域 $N^+(2) = \{4\}$（绿色），入度为 2，出度为 1。

度数和与边数 考虑所有“顶点—边”配对 (i, e)，其中 i 是边 e 的端点。按顶点分组，顶点 i 出现在 d_i 个配对中；按边分组，每条边恰有两个端点。两种分组数的是同一批配对，所以

$$\sum_{i \in V} d_i = 2m. \tag{1.2}$$

由此可知度为奇数的顶点个数必是偶数。有向图中每条弧恰好贡献一个出度和一个入度，所以 $\sum_i d_i^+ = \sum_i d_i^- = m$。

邻域与度的读取 给每个顶点准备一个空的邻居集合和一个初始为 0 的计数器，然后逐条读边：读到 $\{i, j\}$ 时，把 j 放进 $N(i)$、i 放进 $N(j)$，两个计数器各加 1。表 1.1是贯穿图上计数器的变化。每读一条边，度数和加 2，读完 m 条边就是 $2m$，这正是式(1.2)；最后一列若不等于两倍边数，说明某条边只记了一端。

有向图读到弧 (i, j) 时，把 j 放进 $N^+(i)$、i 放进 $N^-(j)$。图 1.3(c) 的 $1 \to 2 \leftarrow 3$ 读完后 $N^-(2) = \{1, 3\}$，$N^+(2) = \varnothing$；不说明方向，就可能把入邻居误当成下一步可以到达的顶点。这里的度只数结构上的边；边带权时把一个顶点所有邻边的权相加，得到的是另一个量，本部分不使用。

表 1.1 逐条读入贯穿图的边时，四个度计数器的变化。每读一条边，恰有两个计数器各加 1，度数和每次增加 2。

已读入的边	d_1	d_2	d_3	d_4	度数和
初始	0	0	0	0	0
$\{1, 2\}$	1	1	0	0	2
$\{1, 3\}$	2	1	1	0	4
$\{2, 3\}$	2	2	2	0	6
$\{2, 4\}$	2	3	2	1	8
$\{3, 4\}$	2	3	3	2	10

1.2.2　行走、路径与环

走一步就是从当前顶点走到它的一个邻居，走若干步就得到一串相邻两项都邻接的顶点。是否允许重复经过顶点要看用途：反复传播、来回巡视的过程允许重复，一条实际路线通常不允许。

定义 1.4　行走与路径　设 $G=(V,E)$ 是无向图。顶点序列 $(v_0,\dots,v_k)$ 若对每个 $r=1,\dots,k$ 都有 $\{v_{r-1},v_r\}\in E$，就称为从 v_0 到 v_k 的一条**行走**（walk），k 称为它的长度，即经过的边数。若行走中的顶点两两不同，就称为一条**简单路径**（path）。

单个顶点 (v_0) 是长度为零的行走，也是长度为零的路径。本部分不加修饰的“路径”都指简单路径。

绕一圈回到起点的路线还要与原路折返区分开。首尾相同（$v_0=v_k$）的行走称为**闭行走**（closed walk），但闭合不意味着绕成一圈：沿一条边去而复返的 $(2,4,2)$ 是闭行走，却没有提供第二条路线。

因此把长度 $k\ge 3$、且 $v_0,\dots,v_{k-1}$ 两两不同的闭行走称为**环**（cycle）；长度为 1 的闭行走需要自环，长度为 2 的只能沿同一条边往返，所以要求 $k\ge 3$。同一个环可以从不同顶点出发、沿两个方向书写，例如 $(1,2,3,1)$ 与 $(1,3,2,1)$，我们把经过同一组边的序列看成同一个环。

顶点序列的核对　判断一个顶点序列属于哪一类，按定义依次检查：每一步 $\{v_{r-1},v_r\}$ 是否为边，判断行走；从左到右维护“已出现顶点”的集合，看有无重复，判断路径；再要求 $k\ge 3$、首尾相同且前 k 个顶点不重复，判断环。图 1.8给出四个结果：$(1,2,3,1,2)$ 是行走，但第二次读到 1 时重复，不是路径；$(1,2,4)$ 是路径；$(2,4,2)$ 是闭行走而不是环；$(1,2,3,1)$ 是环。三者逐步加强，反过来都不成立：行走便于拼接与计数，路径对应实际路线，环表示存在替代路线。

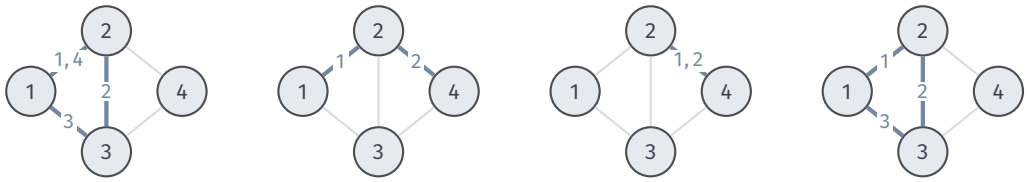


图 1.8　贯穿图上的四种顶点序列。蓝色粗线是经过的边，边旁数字是经过该边的步序，浅灰色是未经过的边。(a) 行走可以重复经过边 $\{1,2\}$；(b) 路径不重复顶点；(c) 沿一条边去而复返得到闭行走，但它不是环；(d) 绕三角形一周的环除首尾外不重复顶点。

1.2.3 有向路径与有向无环图

任务依赖要求每一步都顺着箭头走。若顺着箭头走一圈又回到出发的任务，例如 1 等 2、2 等 3、3 又等 1，这组任务就无法按任何次序全部完成。把定义 1.4中的条件换成 $(v_{r-1}, v_r) \in E$，就得到有向行走与**有向路径**（directed path）。除首尾外顶点两两不同、长度至少为 1 的有向闭行走称为**有向环**（directed cycle）；由于没有自环，其长度至少为 2。

> **定义 1.5　有向无环图**　不含有向环的有向图称为**有向无环图**（directed acyclic graph，DAG）。

有向环的最短长度与无向情形不同。图 1.9(a) 中两条反向弧 $i \to j$ 和 $j \to i$ 是两个不同的对象，(i, j, i) 构成长度为 2 的有向环；无向图中的 $(2, 4, 2)$ 两次经过的是同一条边。图 (b) 的 $(1, 2, 3, 1)$ 每一步都顺着箭头，是长度为 3 的有向环。

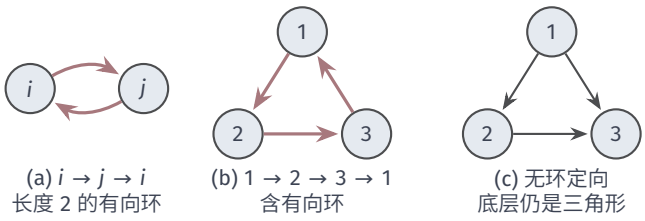


图 1.9　有向环与有向无环图。红色弧组成有向环。(a) 两条反向弧构成长度为 2 的有向环；(b) 三角形的循环定向含有向环；(c) 同一个三角形的另一种定向不含有向环，是有向无环图，尽管它的底层无向图是一个环。

无环的核对　有向环上的每个顶点都有环上的弧进出，所以入度或出度为 0 的顶点不在任何有向环上。图 1.9(c) 的弧为 $1 \to 2$、$1 \to 3$、$2 \to 3$，1 的入度与 3 的出度为 0，剩下的顶点 2 不足以构成至少含两个顶点的有向环，所以它是 DAG。大图上系统地判断有无有向环，要用第 2.2节的深度优先遍历或第 2.4节的拓扑排序。

图 (c) 是 DAG，它的底层无向图却是三角形。拓扑排序要求的正是有向无环；换成“底层无向图无环”这一更强的条件，会把 (c) 这样完全可以排出次序的依赖错误地排除。

1.2.4 可达性

很多问题只关心从 i 出发能不能到达 j，而不关心怎么到。若存在从 i 到 j 的行走，称 j 从 i**可达**（reachable）；有向图中要求有向行走。行走首尾相接仍是行走，路径则未必，所以用行走定义更方便。下面的引理说明改用路径定义并无区别。

引理 1.6　行走中含有路径　在无向图或有向图中，若存在从 i 到 j 的行走，则存在从 i 到 j 的路径，并且它的顶点和边都取自这条行走。

证明　在从 i 到 j、顶点和边都取自原行走的所有行走中，取长度最短的一条 $(v_0, \dots, v_k)$。若它不是路径，则有 $s < t$ 使 $v_s = v_t$。删去中间段 $v_{s+1}, \dots, v_t$，得到 $(v_0, \dots, v_s, v_{t+1}, \dots, v_k)$：原来从 v_t 出发的那一步现在从 $v_s = v_t$ 出发，每一步仍沿原有的边或弧，而长度减少了 $t - s > 0$，矛盾。 ■

从行走中取出路径　实际取出路径时，可以从左到右读一遍行走，随读随删。维护当前路径 P，初始为 (v_0)，并给 P 上的顶点做标记。读到 v_r 时，若它没有标记，就接到 P 末尾；若已有标记，说明 P 绕回了 v_r，就把 P 截到 v_r 所在的位置，并清除其后的标记。P 始终顶点互异、各边取自原行走，读完时就是所求的路径。行走 $(1, 2, 3, 1, 3, 4)$ 上，P 依次为 (1)、$(1, 2)$、$(1, 2, 3)$，读到第二个 1 时截回 (1)，再变为 $(1, 3)$、$(1, 3, 4)$。每个顶点至多接上、删去各一次，工作量与行走长度成正比。

可达关系的性质　零长度行走给出自反性，行走首尾相接给出传递性；无向图中把行走倒过来走仍是行走，所以还有对称性。同时满足自反、对称、传递三条的二元关系称为等价关系，所以无向图上的可达关系是等价关系。等价关系的等价类两两不交、并起来恰为全集，下一节据此划分连通分量；等价关系与划分的一般对应，可参考卷一第 1 章“集合、关系与映射”。

这里的性质只说明可达关系是什么。给出两个具体顶点、要判断是否可达，仍需要有组织地访问能走到的顶点，这是第 2.1节与第 2.2节的任务。

有向图中倒过来走需要反向弧，对称性不再成立：单向链 $1 \to 2 \to 3$ 中，3 从 1 可达，1 却不从 3 可达。为描述这种单向关系，$N^-(j)$ 中的顶点称为 j 的直接**前驱**（predecessor），$N^+(j)$ 中的顶点称为直接**后继**（successor）；能经一条或多条弧到达 j 的其他顶点称为 j 的**祖先**（ancestor），从 j 出发经一条或多条弧能到达的其他顶点称为 j 的**后代**（descendant）。本部分约定祖先与后代都不含 j 自身。单向链中 3 的直接前驱只有 2，祖先是 $\{1, 2\}$；1 的后代是 $\{2, 3\}$。

有向环上的两个不同顶点可以互为祖先，在 DAG 中这不可能发生：若 $i \neq j$ 互为祖先，两条有向路径拼成长度至少为 2 的有向闭行走 $(v_0, \dots, v_k)$；在所有满足 $s < t$、$v_s = v_t$ 的位置对中取 $t - s$ 最小的一对，$(v_s, \dots, v_t)$ 除首尾外不重复顶点，长度又不能为 1，所以是有向环，矛盾。可达的不对称使有向图的“连通”需要两种定义，这是下一节的内容。

1.3 连通与分隔

上一节的可达关系一次只回答两个顶点之间的问题。道路网规划还需要整体的回答：全部城市分成几片互相到达的区域，封闭哪些路口或道路会把区域切开。前一个问题由可达关系的等价类回答，有向图中则要区分强、弱两种连通；后一个问题要看删除顶点或边以后是否还有替代路径。这里只讨论结构上的路径是否存在；概率模型中的条件独立另有自己的分离判据，不在本卷讨论。

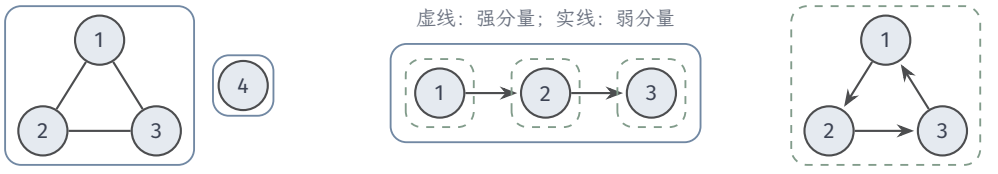
1.3.1 无向图的连通分量

我们想把全部城市分成若干片，同一片中的城市两两可以互相到达，不同片之间无路可通；有了这样的划分，“能否互相到达”就只需看是否在同一片中。第 1.2.4小节说明，无向图中的可达关系是等价关系；等价类两两不交、并起来恰为 V，所以它们恰好构成这样的划分。

> **定义 1.7　连通分量**　无向图 $\mathcal{G} = (V, E)$ 中可达关系的每个等价类 $C \subseteq V$ 称为 $\mathcal{G}$ 的一个**连通分量**（connected component）；有时也把诱导子图 $\mathcal{G}[C]$ 称为连通分量。若 $\mathcal{G}$ 非空且恰有一个连通分量，即任意两个顶点之间都有路径，就称 $\mathcal{G}$ 是**连通**（connected）的。

这个划分有三条直接的性质。连接同一分量中两点的路径不会离开该分量，因为路径上的每个顶点都与起点可达，所以 $\mathcal{G}[C]$ 本身连通。不同分量之间没有边，否则两端可达，应属同一分量。划分只取决于可达关系，由图唯一确定。

图 1.10(a) 是三角形 $\{1,2,3\}$ 加上孤立顶点 4，有两个分量 $\{1,2,3\}$ 与 $\{4\}$，不连通。空图的分量数为 0，按定义不算连通，用“恰有一个分量”判断连通时要先排除这个情形。



(a) 两个连通分量　(b) 单向链：三个强分量，一个弱分量　(c) 有向环：一个强分量

图 1.10　连通分量。蓝色实线框是无向图或底层无向图的连通分量，绿色虚线框是强连通分量。(a) 孤立点自成一个分量；(b) 单向链 $1 \to 2 \to 3$ 忽略方向后连成一片，但没有两个顶点能互相到达，每个顶点自成一个强分量；(c) 有向环上任意两个顶点都能互相到达，三个顶点属于同一个强分量。

分量划分的核对　核对别人给出的划分不必逐对找路径，上面的前两条性质反过来就够了。

> **引理 1.8　分量划分的核对**　设 $C_1, \dots, C_c$ 是无向图 G 的顶点集的一个划分。若每个 $G[C_k]$ 都连通，并且没有一条边的两端位于不同的块中，则 $C_1, \dots, C_c$ 恰好是 G 的全部连通分量。

证明　同一块中的两点由块内连通而可达。反过来，若 j 从 i 可达，取一条行走，它的每一步沿一条边，而没有边跨越不同的块，所以 i, j 在同一块中。于是各块就是可达关系的等价类。 ■

于是核对分两步：给顶点写上块编号，逐条扫描边，检查两端编号相同；再对每一块给出把块内顶点连在一起的边。图 1.10(a) 的划分 $(1,1,1,2)$ 两步都通过；若把贯穿图划分为 $\{1,2\}$ 与 $\{3,4\}$，第一步就在边 $\{1,3\}$ 处失败。第二步所需的块内连接在第 1.4节整理为生成森林；从零求出全部分量，由第 2.3节在与 $n+m$ 成正比的时间内完成。

一张图是否连通，取决于删去连接以后有没有替代路径。在贯穿图中删除边 $\{2,3\}$，2 与 3 仍可经 1 或 4 相连；在图 1.12(a) 中删除连接两个三角形的边 $\{3,4\}$，图就分成两个分量。两次都只删除一条边，结果不同是因为前者的边位于环上，环的另一侧提供了替代路径。第 1.3.4小节会把这一观察写成桥的判别条件。

1.3.2　强连通与弱连通

有向图中可达不是等价关系，不能直接用来划分顶点。修补的办法有两种：要求双向都可达，回答哪些顶点能“绕得回来”；或者忽略方向，只问是否连成一片。

若 j 从 i 可达且 i 从 j 可达，称 i, j 相互可达。相互可达关系是自反的（零长度行走）、对称的（定义对 i, j 对称）、传递的（两个方向分别首尾相接），因此是等价关系。它的等价类称为**强连通分量**（strongly connected component）；若有向图非空且只有一个强连通分量，就称它**强连通**（strongly connected）。

忽略方向的办法用底层无向图（第 1.1.2小节）的连通分量划分顶点，所得的分量称为原图的**弱连通分量**（weakly connected component）；底层无向图连通时，称原有向图**弱连通**（weakly connected）。

在小图上可以直接读出两种分量。记 $R^{+}(v)$ 与 $R^{-}(v)$ 分别为从 v 可达与能到达 v 的顶点集（都含 v），由定义，v 所在的强分量是 $R^{+}(v) \cap R^{-}(v)$；弱分量则在底层无向图上读出。图 1.10(b) 的单向链中，$R^{+}(2) = \{2,3\}$，

$R^{-}(2)=\{1,2\}$，强分量为 $\{2\}$，三个顶点各成一个强分量，却只有一个弱分量。(c) 的有向环中两个可达集都是 $\{1,2,3\}$，图强连通。在大图上求全部强分量的线性时间方法见第 2.3节。

有向路径去掉箭头后是底层无向图中的路径，所以每个强分量都包含在某个弱分量之内，强连通蕴含弱连通。单向链说明反过来不成立。需要回答“能否往返”时，必须使用强分量。

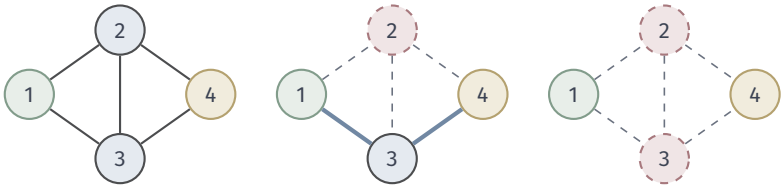
1.3.3 分隔集

规划者还关心脆弱性：封闭哪些路口能把两个区域隔开。被封闭的路口不能属于要隔开的两组本身；“隔开”的含义是，删掉这些路口以后，两组之间一条路径也不剩。

> **定义 1.9　分隔**　设 $A,B,S\subseteq V$ 两两不相交。若在 $G-S$ 中不存在从 A 中顶点到 B 中顶点的路径，就称 S**分隔**（separate）A 与 B，称 S 为 A 与 B 之间的一个**分隔集**（separator）。

等价地，G 中从 A 到 B 的每条路径都至少经过 S 中的一个顶点，因为 $G-S$ 中的路径就是 G 中避开 S 的路径。所以验证一个候选集要考虑全部路径，找到一条被阻断的路径并不够。

核对时却不必列举路径：在 $G-S$ 中从 A 出发求出可达集 R（第 2.1节的广度优先遍历允许以一组顶点为起点），S 分隔 A 与 B 当且仅当 $R\cap B=\varnothing$。在图 1.11中取 $A=\{1\}$、$B=\{4\}$：删除 $\{2\}$ 后 $R=\{1,3,4\}$，路径 $(1,3,4)$ 仍在，$\{2\}$ 不是分隔集；删除 $\{2,3\}$ 后 1 的邻居全被删去，$R=\{1\}$，$\{2,3\}$ 是分隔集。若 A 与 B 原本位于不同分量，空集就分隔它们。



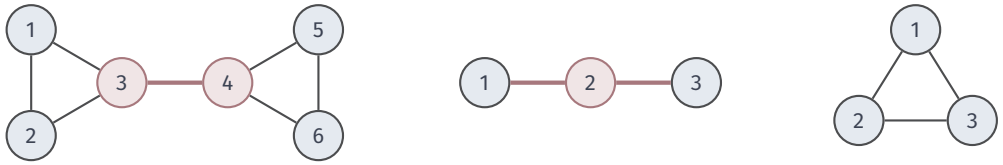
(a) $A=\{1\}$，$B=\{4\}$　(b) 删除 $\{2\}$：仍有 $1,3,4$　(c) 删除 $\{2,3\}$：已分隔

图 1.11　在贯穿图中检验分隔集，$A=\{1\}$ 为绿色，$B=\{4\}$ 为黄色，被删除的候选集 S 为红色虚线顶点。(b) 删除 $\{2\}$ 后蓝色路径 $(1,3,4)$ 仍然存在，$\{2\}$ 不是分隔集；(c) 删除 $\{2,3\}$ 后从 1 到 4 没有路径，$\{2,3\}$ 分隔 $\{1\}$ 与 $\{4\}$。

不指定两组顶点时，分隔集由删除后的分量给出。对连通图 G，若 $G-S$ 不连通，则 S 分隔 $G-S$ 中任意两个不同分量的顶点集。例如贯穿图删除 $\{2,3\}$ 后余下 $\{1\}$ 与 $\{4\}$。第 2.5节将沿这样的分隔集把图分成可以分别处理的块。

1.3.4　割点与桥

最脆弱的情形是只失去一个路口或一条道路，网络就断开。删除顶点 v 后 $\mathcal{G}-\{v\}$ 的分量数大于 $\mathcal{G}$ 的分量数，称 v 为**割点**（cut vertex）；删除边 e 后 $\mathcal{G}-e$ 的分量数大于 $\mathcal{G}$ 的分量数，称 e 为**桥**（bridge）。图 1.12(a) 由两个三角形 $\{1,2,3\}$、$\{4,5,6\}$ 和边 $\{3,4\}$ 组成，$3,4$ 是割点，$\{3,4\}$ 是桥，三角形内部的边和顶点 $1,2,5,6$ 都不是；(b) 的链中，中间顶点 2 是割点，两条边都是桥；(c) 的三角形中，删除任意一个顶点或一条边都不会断开。



(a) 割点 $3,4$；桥 $\{3,4\}$　　(b) 链：割点 2，两条边都是桥　　(c) 三角形：每条边都有替代路径

图 1.12　割点与桥，红色顶点为割点，红色粗边为桥。(a) 两个三角形由一条边相连，这条边是桥，它的两个端点是割点；(b) 链上每条边都是桥，中间顶点是割点，端点 $1,3$ 虽是桥的端点却不是割点；(c) 三角形的每条边都位于环上，删除任何一条边或一个顶点都不会断开。

桥的判别　按定义判断桥要比较删除前后的分量数。下面的定理把它换成只看这条边两端的条件，并说明环正是替代路径的来源。

定理 1.10　桥的判别　设 $e=\{i,j\}$ 是无向图 $\mathcal{G}$ 的一条边。以下三条等价：(1)e 是桥；(2)$\mathcal{G}-e$ 中没有从 i 到 j 的路径；(3)e 不在 $\mathcal{G}$ 的任何环上。并且删除任何一条边，分量数至多增加 1。

证明　删除 e 只可能拆开 e 所在的分量 C。C 中任一顶点 x 在原图中有到 i 的路径；若它经过 e，取它第一次到达 i 或 j 之前的部分。这部分不含 e，所以删除后 x 仍连到 i 或 j。因此 $\mathcal{G}-e$ 中 C 至多分成 i 所在和 j 所在两部分，分量数至多增加 1，并且增加当且仅当 i,j 不再连通，即 (1) 与 (2) 等价。若 e 在某个环上，环的另一侧就是 $\mathcal{G}-e$ 中从 i 到 j 的路径，所以 (2) 蕴含 (3)；反过来，$\mathcal{G}-e$ 中从 i 到 j 的路径长度至少为 2，接上 e 就是一个环，所以 (3) 蕴含 (2)。 ■

割点与桥的检验　由定理，检验 $e=\{i,j\}$ 是否为桥，只需在 $\mathcal{G}-e$ 中从 i 做一次可达性搜索，看能否到达 j：在图 1.12(a) 中删去 $\{3,4\}$ 后从 3 只能到达 $\{1,2,3\}$，它是桥；删去 $\{1,2\}$ 后仍可经 3 到达 2，它不是桥。

检验顶点 v 是否为割点，则删去 v 后从它的一个邻居出发，看能否到达它的其余全部邻居。只看邻居就够了：v 所在分量中的其他顶点，沿通往 v

的路径走到第一次遇到 v 的邻居为止，这一段避开了 v，所以它们在删去 v 后仍连到 v 的某个邻居。于是分量是否断开，只取决于 v 的邻居之间是否仍然连通。逐一检验全部顶点或全部边，每次都要做一次这样的搜索。

删点与删边的效果并不对称。删除一个顶点会同时删除它的全部邻边，可能把分量拆成很多块：星形图删除中心顶点后，其余每个顶点都自成一个分量；删除一条边却至多增加一个分量。桥的端点不一定是割点，例如图 1.12(b) 的端点 1；没有桥的图也可能有割点，例如两个三角形共用一个顶点。所以“删点能否断开”与“删边能否断开”需要分别检查。

1.4 树与森林

上一节看到，边位于环上时删掉它不会断开连接，环提供的是冗余的替代路径。若要用尽量少的道路把一组城市连起来，就不应保留任何环。本节研究这种连通而无环的结构。树的两个条件分别对应路径的存在与唯一，由此能数出树的边数，边数又反过来给出判断树的简便方法。

此后两小节讨论树的两种用法：在树上选根可以组织上下层次，在一般连通图中则可以抽出一棵覆盖全部顶点的树。这些结论是第 2章遍历结果、第 4章最小生成树和第 11.2节树上动态规划的共同基础。

1.4.1 树与森林的定义

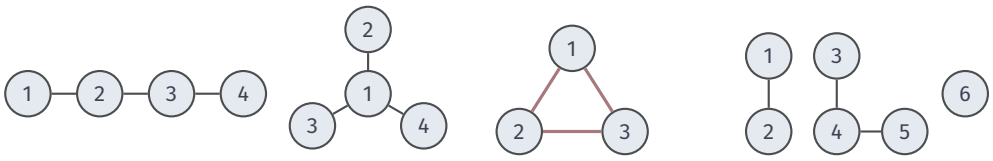
不修多余道路的道路网应当连通，以保证任意两点之间都有路线；还应当无环，因为由定理 1.10，环上的边删去后图仍然连通，可以省掉。同时要求这两点就得到树，只要求无环得到的是若干互不相连的树。

> **定义 1.11　树与森林**　非空、连通且不含环的无向图称为**树**（tree）。不含环的无向图称为**森林**（forest）。

森林的每个连通分量都连通且无环，是一棵树；只有恰好一个分量的森林才是树。图 1.13中，(a) 的链和 (b) 的星形都是树，树可以有分支；(c) 的三角形连通但有环，不是树；(d) 有三个分量，每个都是树，整体是森林。只有一个顶点、没有边的图是单点树，所以树中的顶点不必都有邻边。空图不含环，是有 0 个分量的空森林，但树要求非空，所以空图不是树。

1.4.2 树的基本性质

路径唯一性　树的连通保证任意两点之间有路径，无环则保证这样的路径只有一条。直观上，若两点之间有两条不同的路径，它们分开以后终究要会合，分开与会合之间的两段就围成一个环；反过来，环上相邻的两点既有一条边直接相连，又可以沿环的另一侧到达。



(a) 链：树　(b) 星形：树　(c) 三角形：有环　(d) 森林：三个分量

图 1.13　树、森林与非树的对照。(a) 链与 (b) 星形都是树，后者有分支；(c) 三角形连通但含有红色标出的环，不是树；(d) 由三个互不相连的树组成的森林，其中孤立顶点 6 是一棵单点树。

定理 1.12　树中路径唯一　设 $\mathcal{G}$ 是非空无向图。$\mathcal{G}$ 是树，当且仅当任意两个顶点之间恰有一条路径。

证明　先设 $\mathcal{G}$ 是树。连通性保证任意两点 u, v 之间至少有一条路径。假设有两条不同的路径 $P = (p_0, \dots, p_a)$ 与 $Q = (q_0, \dots, q_b)$，$p_0 = q_0 = u$，$p_a = q_b = v$。一条不能是另一条的真前段，否则较长的那条会两次经过 v；所以存在最小的 s 使 $p_{s+1} \neq q_{s+1}$，记 $x = p_s = q_s$。由于 v 同时在两条路径上，可以取 P 在 x 之后第一个也出现在 Q 中 x 之后的顶点 y。

P 从 x 到 y 的一段与 Q 从 x 到 y 的一段除端点外没有公共顶点：P 这一段的中间顶点按 y 的取法不在 Q 中 x 之后的部分，又因为 P 是路径，也不在两者共有的 x 之前的部分。沿 P 从 x 走到 y，再沿 Q 倒着走回 x，得到一个除首尾外不重复顶点的闭行走。两段不能都只有一条边，否则 $p_{s+1} = y = q_{s+1}$；所以它的长度至少为 3，是一个环，矛盾。图 1.14(a) 画出了这个构造。

反过来，设任意两点之间恰有一条路径，则 $\mathcal{G}$ 连通。若 $\mathcal{G}$ 含环 $(c_0, c_1, \dots, c_{k-1}, c_0)$，$k \geq 3$，则 (c_0, c_1) 与 $(c_0, c_{k-1}, \dots, c_1)$ 是从 c_0 到 c_1 的两条不同路径，矛盾。 ■

由这个定理，删除树的任意一条边 $e = \{u, v\}$ 后 u, v 之间不再有路径，由定理 1.10，树恰好分成两棵较小的树。树的每条边都是桥。

边数　数树的边数要用到度为 1 的顶点，称为**叶**（leaf）。叶只与一个顶点相连，删掉它不会断开其余部分，这使我们能对顶点数归纳。先说明叶总是存在。

引理 1.13　树有叶　至少有两个顶点的树至少有两个叶。

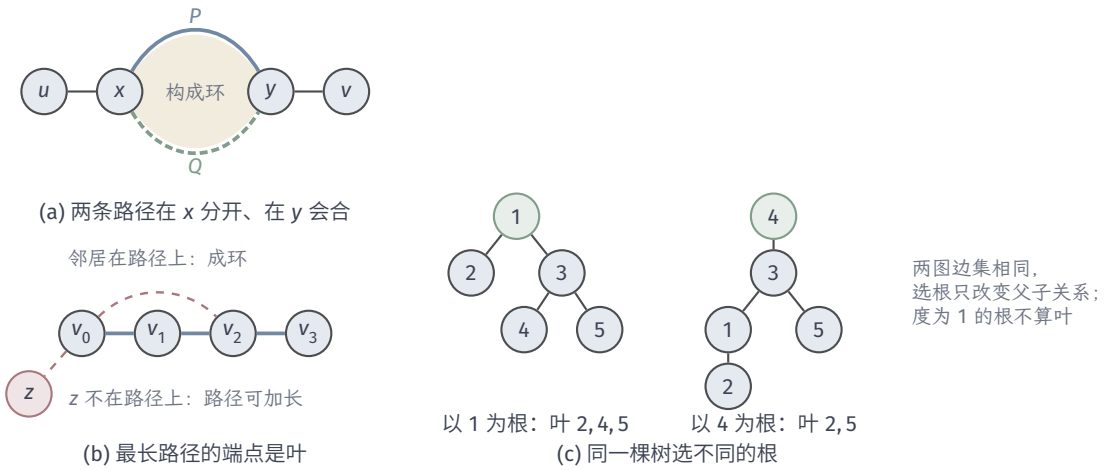


图 1.14　树的性质的证明示意与选根。(a) 两条不同的路径 P（蓝色实线）与 Q（绿色虚线）在 x 分开、在 y 第一次会合，二者之间的部分（黄色区域的边界）构成一个环。(b) 若最长路径的端点 v_0 还有别的邻居，这个邻居不在路径上时可以把路径加长，在路径上时则形成环，两种情形都不可能。(c) 同一棵树分别以 1 和 4 为根（绿色），边集不变，父子关系和叶的集合随根改变。

证明　取一条最长的路径 $P = (v_0, \dots, v_k)$，由于树连通且至少有两个顶点，$k \geq 1$。若 v_0 另有邻居 $z \neq v_1$：z 不在 P 上时，$(z, v_0, \dots, v_k)$ 是更长的路径；$z = v_t$ 在 P 上时 $t \geq 2$，$(v_0, v_1, \dots, v_t, v_0)$ 是环。两种情形都不可能（图 1.14(b)），所以 $d_{v_0} = 1$。同理 $d_{v_k} = 1$。 ■

定理 1.14　树的边数　有 n 个顶点的树恰有 $n - 1$ 条边。

证明　对 n 归纳，$n = 1$ 时单点树没有边。设 $n \geq 2$，由引理 1.13，树 $\mathcal{T}$ 有一个叶 v。$\mathcal{T} - \{v\}$ 无环；它也连通，因为路径的中间顶点要有两条路径上的邻边，度为 1 的 v 不会出现在连接其他两点的路径中间。所以 $\mathcal{T} - \{v\}$ 是 $n - 1$ 个顶点的树，有 $n - 2$ 条边，加上 v 的那条边得 $n - 1$ 条。 ■

1.4.3　树的判定

按定义判断一张图是不是树，要确认它没有环，而这是关于全部顶点序列的断言。边数定理提示了另一条途径：数一下边，再查一次连通。为此先说明任何连通图都能用 $n - 1$ 条边连成一棵树。

构造从一个顶点出发，每次向外多接一个顶点。任取顶点 r，令 $S = \{r\}$、$F = \varnothing$。只要 $S \neq V$，就取 S 外的顶点 t 和一条从 r 到 t 的路径，路径上必

有一步从 S 内走到 S 外，把这一步的边加入 F、它的 S 外端点加入 S。每一步增加一个顶点和一条边，$n-1$ 步后 $S=V$，$|F|=n-1$。

这样选出的 (V,F) 是树。它连通，因为每个新顶点都连到先前的顶点。它也无环：若 F 中有环，取环上最后加入 S 的顶点 t，它在环上的两个邻居都更早加入；而 F 中 t 与更早顶点之间的边只有 t 加入时选出的那一条，矛盾。

定理 1.15　连通且 $n-1$ 条边的图是树　设 $\mathcal{G}$ 是有 $n\ge 1$ 个顶点的连通无向图。若 $\mathcal{G}$ 恰有 $n-1$ 条边，则 $\mathcal{G}$ 是树。

证明　按上面的构造选出树 (V,F)。$F\subseteq E$ 且 $|F|=n-1=|E|$，所以 $F=E$。 ■

森林的边数　设森林有 n 个顶点、c 个分量，第 k 个分量有 n_k 个顶点。每个分量是树，有 n_k-1 条边，所以森林的边数是

$$\sum_{k=1}^{c}(n_k-1)=n-c. \tag{1.3}$$

于是无环且有 $n-1$ 条边的图只有一个分量，也是树。

三个条件的关系　对 $n\ge 1$ 个顶点的无向图，连通、无环、恰有 $n-1$ 条边三个条件中，前两个由定理 1.14推出第三个，连通与 $n-1$ 条边由定理 1.15推出无环，无环与 $n-1$ 条边由式(1.3)推出连通。只有一个条件不够：三角形加一个孤立顶点（图 1.10(a)）有 4 个顶点、3 条边，却既不连通又有环。

所以判断树只需两步：若 $m\neq n-1$，立即否定；否则查连通。边集为 $\{1,2\},\{1,3\},\{1,4\},\{2,5\},\{3,6\}$ 的六点图有 $5=n-1$ 条边，从 1 出发能到达全部顶点，所以是树；贯穿图有 $5\neq 3$ 条边，第一步就被排除。在大图上，第二步由第 2.3节的连通分量计算完成。

1.4.4　有根树

家谱、文件目录或分治计算中，每个顶点有一个上级、可能有若干下级，无向树的边却读不出谁是上级。路径唯一性使我们可以从任意一个顶点出发组织这样的层次。

在树 $\mathcal{T}$ 中选定一个顶点 r 作为根，得到**有根树**（rooted tree）。对每个 $v\neq r$，从 r 到 v 的唯一路径上紧挨在 v 之前的顶点称为 v 的**父节点**（parent），v 称为它的**子节点**（child）。从 r 到 u 的路径经过 v 的那些顶点

u（包括 v 本身）组成以 v 为根的**子树**（subtree）。在有根树中，没有子节点的顶点称为叶。

父子关系的确定 不必逐个找路径，可以一层层确定：根的邻居都以根为父节点；此后每确定一个顶点 u 的父节点，就把 u 除父节点以外的邻居都定为 u 的子节点。依据是：这样的邻居 w 不在从 r 到 u 的路径上，否则会与边 $\{u,w\}$ 构成环，所以那条路径接上 w 就是从 r 到 w 的唯一路径。

图 1.14(c) 的树有边 $\{1,2\},\{1,3\},\{3,4\},\{3,5\}$。以 1 为根时，2, 3 的父节点是 1，4, 5 的父节点是 3，叶是 2, 4, 5；以 4 为根时，3 的父节点是 4，1, 5 的父节点是 3，2 的父节点是 1，叶是 2, 5。根 4 在无向树中是叶，在以它为根的有根树中却不是。第 2章的遍历在树上给出的就是这样的父子关系。

选根没有增删任何边，只规定每条边上哪一端是父节点；记录 $n-1$ 个非根顶点的父节点就能还原边集，换一个根则全部重新确定。把每条边定向为从父节点指向子节点，得到一张有向图，但这一方向来自选根，而一般有向图的弧方向是原始数据，两者不应混同。第 11.2节在有根树上自下而上计算时，利用的正是“每个子树只通过它的根与外界相连”。

1.4.5 生成树与生成森林

道路网中常要选出一部分道路，使所有城市仍然连通且没有多余的环，例如只维护最少的道路。所选的边必须取自原图，顶点则一个都不能少。

> **定义 1.16 生成树与生成森林** 设 $G=(V,E)$ 是无向图。若子图 (V,F)，$F\subseteq E$，是一棵树，就称它为 G 的**生成树**（spanning tree）。若 (V,F) 是森林，并且它的连通分量与 G 的连通分量相同，就称它为 G 的**生成森林**（spanning forest）。

连通图一定有生成树，第 1.4.3小节的构造选出的 (V,F) 就是一棵；只有连通图才有生成树。在每个分量上各取一棵生成树就得到生成森林，由式(1.3)，它恰有 $n-c$ 条边，其中 c 是原图的分量数。

生成树的核对与列举 核对一个声称的生成树，只需检查 $F\subseteq E$、$|F|=n-1$、(V,F) 连通。贯穿图的 T_2 由 $\{1,2\},\{1,3\},\{3,4\}$ 组成，三项都通过；三角形 $\{1,2,3\}$ 的三条边也满足前两项，但没有连到顶点 4。同样的判定可以列举全部生成树：从 5 条边中选 3 条共 $\binom{5}{3}=10$ 种，有环的只有恰好选中三角形 $\{1,2,3\}$ 或 $\{2,3,4\}$ 的两种，其余 8 种都是生成树，见图 1.15。

生成树舍弃了环所提供的替代路径。以 T_2 为例：原图中 2, 4 相邻，在 T_2 中却要经过 $(2,1,3,4)$；原图没有割点，而 T_2 中 1 和 3 都成了割点。所以生成树上的距离和脆弱性一般不等于原图上的结果。生成森林恰好用最

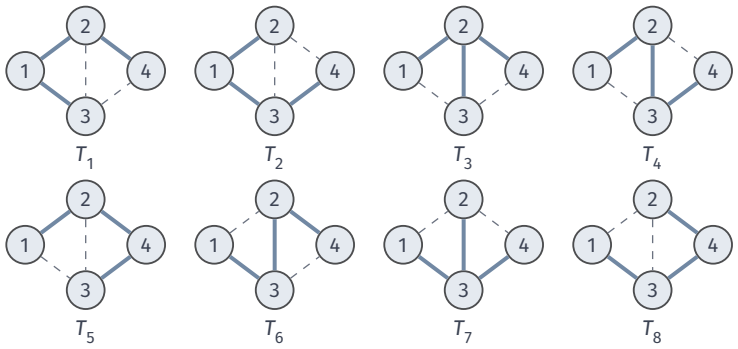


图 1.15　贯穿图的全部 8 棵生成树。蓝色粗线是树边，灰色虚线是被舍弃的原边。每棵生成树都有 3 条边并覆盖全部 4 个顶点；从 5 条边中选 3 条的 10 种选法里，只有恰好组成三角形 $\{1,2,3\}$ 或 $\{2,3,4\}$ 的两种不是树。

少的边给出第 1.3.1小节核对分量划分所需的块内连接。用遍历求生成森林是第 2.3节的内容，它只保证连通、不比较边权；带建设成本时选总成本最小的一棵，是第 4章的问题。

1.5　图的表示

要让计算机处理一张图，还得决定怎样存储它。不同的算法反复执行的操作不同：有的逐条扫描所有边，有的反复列出一个顶点的邻居，有的查询两个顶点之间是否有边，还有的做矩阵运算。选择记录方式时，要让这些反复执行的操作既正确又便宜。

本节介绍的四种常用表示各为一类操作而设计，图 1.16把贯穿图的四种表示放在一起。比较时既看存储与访问的代价，也计入相互转换的代价；邻接矩阵还把行走计数变成矩阵乘法。代价用记号 $O(\cdot)$ 表示：$O(g)$ 指操作次数或存储量不超过 g 的某个常数倍，常数与图的规模无关。这一记号描述的是规模增大时的增长速度，增长尺度的比较可参考卷一第 5 章“极限与连续”。

1.5.1　边列表

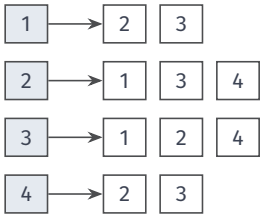
最直接的做法是把边一条条记下来。**边表**（edge list），也称边列表，为每条边保存一条记录：边的编号、两个端点，以及长度、容量等所需的属性。有向图的记录必须保存端点的次序。贯穿图的边列表见图 1.16(a)，编号为 $e_1=\{1,2\}$、$e_2=\{1,3\}$、$e_3=\{2,3\}$、$e_4=\{2,4\}$、$e_5=\{3,4\}$。有平行边时，两条边端点相同，只能靠编号区分。

边列表除顶点集外只需 $O(m)$ 的存储，适合逐条处理全部边的算法，例如按边权排序（第 4章），或每一轮把每条边都检查一遍（第 3章）；求全部顶点的度也只需扫描一遍，即表 1.1的做法。它的弱点是查询局部信息：

(a) 边列表

边	端点
e_1	$\{1,2\}$
e_2	$\{1,3\}$
e_3	$\{2,3\}$
e_4	$\{2,4\}$
e_5	$\{3,4\}$

(b) 邻接表



(c) 邻接矩阵 **A**

	1	2	3	4
1	0	1	1	0
2	1	0	1	1
3	1	1	0	1
4	0	1	1	0

(d) 关联矩阵 **B**

	e_1	e_2	e_3	e_4	e_5
1	1	1	0	0	0
2	−1	0	1	1	0
3	0	−1	−1	0	1
4	0	0	0	−1	−1

图 1.16 贯穿图的四种表示。(a) 边列表逐条记录边的编号和端点；(b) 邻接表为每个顶点（蓝色方框）列出邻居，每条无向边在两端各出现一次；(c) 邻接矩阵按顶点次序记录每对顶点是否相邻，蓝色为 1；(d) 关联矩阵的每列对应一条边，按“小编号指向大编号”的记录方向，起点处为 1（蓝色）、终点处为 −1（绿色）。

没有额外索引时，列出顶点 2 的邻居或判断 $\{1,4\}$ 是否为边，都要扫描全部 m 条记录。

1.5.2 邻接表

遍历一张图时最频繁的操作是“列出当前顶点的全部邻居”，在边列表上每次都要扫描全部记录。**邻接表**（adjacency list）预先给每个顶点保存一个列表，列出它的邻居，需要时连同边的属性。无向边 $\{i,j\}$ 在 i 和 j 的列表中各出现一次；有向图的邻接表默认只保存出邻居。

由边列表构造邻接表时，建立 n 个空列表，按编号扫描边列表：遇到无向边 $\{i,j\}$，在 i 与 j 的列表末尾各追加对方；遇到弧 (i,j)，只把 j 追加到 i 的列表。表 1.2是贯穿图上的构造过程，最后一行就是图 1.16(b)。建表用 $O(n)$ 时间，每条边追加两次，总共 $O(n+m)$。

列表中邻居的次序取决于读边次序，第 2章的遍历按列表次序读取邻居，所以执行过程会说明采用的次序。有向图若还要列出能一步到达 j 的顶点，就在读到 (i,j) 时再把 i 追加到 j 的入邻居列表，即反向图的出邻居表，存储增加 $O(m)$。

邻接表有 n 个列表头，无向图的列表项共 $\sum_i d_i = 2m$ 个（式(1.2)），有向图的出邻居表共 m 个，所以存储量为 $O(n+m)$。列出顶点 i 的邻居只需读它自己的列表，时间与 d_i 成正比；判断 $\{i,j\}$ 是否为边则要在列表中查

表 1.2　由贯穿图的边列表构造邻接表。每读一条边，在两个端点的列表末尾各追加一项；最后一行即邻接表，各列表的长度依次是度 2, 3, 3, 2。

已读入的边	顶点 1	顶点 2	顶点 3	顶点 4	列表项总数
初始	空	空	空	空	0
$e_1 = \{1,2\}$	2	1	空	空	2
$e_2 = \{1,3\}$	2, 3	1	1	空	4
$e_3 = \{2,3\}$	2, 3	1, 3	1, 2	空	6
$e_4 = \{2,4\}$	2, 3	1, 3, 4	1, 2	2	8
$e_5 = \{3,4\}$	2, 3	1, 3, 4	1, 2, 4	2, 3	10

找，最坏需要 $O(d_i)$ 时间。若一个算法对每个顶点恰好列出一次邻居，总工作量是 $\sum_i (1 + d_i) = n + 2m$；第 2章正是这样累计遍历的开销。

1.5.3　邻接矩阵

查询“i, j 是否相邻”时，邻接表要在列表里查找。若预先给每一对顶点留一个位置，这个查询就能一步完成，代价是更多的存储；把这些位置排成方阵，还能直接使用矩阵运算。

固定顶点次序 $1, \dots, n$。无权图的**邻接矩阵**（adjacency matrix）$\boldsymbol{A} \in \mathbb{R}^{n \times n}$ 定义为

$$a_{i,j} = \begin{cases} 1, & \{i,j\} \in E \text{（有向图中为 } (i,j) \in E\text{）}, \\ 0, & \text{其他情形.} \end{cases}$$

带权时把有边位置的 1 换成边权 $w_{i,j}$，记作 $\boldsymbol{W}$。有向图采用“行是起点、列是终点”的约定。无向图的 $\boldsymbol{A}$ 对称，有向图的邻接矩阵一般不对称。

构造时在全零矩阵上逐条填入边：无向边 $\{i,j\}$ 令 $a_{i,j} = a_{j,i} = 1$，弧 (i,j) 只令 $a_{i,j} = 1$。贯穿图得到

$$\boldsymbol{A} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}, \tag{1.4}$$

第 i 行中为 1 的列号就是 $N(i)$，行和就是度 2, 3, 3, 2，对角元为 0 对应没有自环。有向图中行和是出度、列和是入度：$1 \to 2 \leftarrow 3$ 的邻接矩阵只有 $a_{1,2} = a_{3,2} = 1$，第 2 列读出 $N^-(2) = \{1, 3\}$，第 2 行全为 0。

零权边与结构掩码　在权矩阵 $\boldsymbol{W}$ 中，零权边与无边都存成 0，单看 $\boldsymbol{W}$ 无法区分图 1.5(a) 与 (b)。允许零权边时要另存结构掩码 $\boldsymbol{M} \in \{0,1\}^{n \times n}$，有边处为 1。“是否相邻”查 $\boldsymbol{M}$，“沿边累计多少”查 $\boldsymbol{W}$；使用矩阵的算法应当说明它读的是哪一个。

稠密存储与稀疏存储 存下全部 n^2 个条目称为稠密存储，查询一对顶点只需 $O(1)$ 时间，列出一个顶点的邻居却要扫描整行，需要 $O(n)$ 时间。很多实际的图 m 远小于 n^2：一个有 10^5 个路口的道路网，边数只是几十万量级，稠密邻接矩阵却需要 10^{10} 个条目。稀疏存储只保存非零条目，存储量为 $O(n+m)$，按行访问时与邻接表相当。所以说一个算法的代价时必须同时说明存储方式：邻接表上为 $O(n+m)$ 的遍历，搬到稠密矩阵上就变成 $O(n^2)$。

1.5.4 邻接矩阵的幂与行走计数

从 1 到 4 走两步有几种走法？走三步回到原处有几种？这类计数在传播过程中反复出现，逐条列举很快变得繁琐。图写成矩阵后，它们可以用矩阵乘法回答；所用的只是乘积的逐项定义 $(\boldsymbol{XY})_{i,j}=\sum_{\ell} x_{i,\ell}y_{\ell,j}$，矩阵运算的一般规则可参考卷二第 1 章“向量、矩阵与张量的运算”。按这一定义，$(\boldsymbol{A}^2)_{i,j}=\sum_{\ell} a_{i,\ell}a_{\ell,j}$，而 $a_{i,\ell}a_{\ell,j}=1$ 当且仅当 (i,ℓ,j) 是一条长度为 2 的行走。所以对中间顶点 ℓ 求和，数出的正是从 i 到 j 长度为 2 的行走条数。这一观察可以推广到任意长度。

定理 1.17　邻接矩阵的幂计数行走 设 $\boldsymbol{A}$ 是无权图（无向或有向）的邻接矩阵，$k\ge 0$ 为整数。则 $(\boldsymbol{A}^k)_{i,j}$ 等于从 i 到 j 长度为 k 的行走条数。

证明 对 k 归纳。$k=0$ 时 $\boldsymbol{A}^0=\boldsymbol{I}$，长度为零的行走只有停在原地的 (i)。设结论对 k 成立。从 i 到 j 长度为 $k+1$ 的行走按倒数第二个顶点 ℓ 分类：它由一条从 i 到 ℓ 长度为 k 的行走接上一步 $\ell\to j$ 组成，这一步存在当且仅当 $a_{\ell,j}=1$。每条行走恰好落入一类，所以

$$\#\{\text{从 } i \text{ 到 } j \text{ 长度为 } k+1 \text{ 的行走}\}=\sum_{\ell=1}^{n}(\boldsymbol{A}^k)_{i,\ell}\,a_{\ell,j}=(\boldsymbol{A}^{k+1})_{i,j}.$$

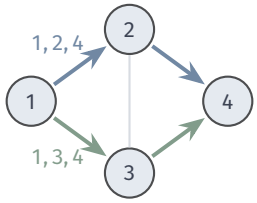
■

无向图中 $(\boldsymbol{A}^2)_{i,j}$ 就是 i 与 j 的公共邻居个数（$i=j$ 时是 i 的度）。贯穿图得到

$$\boldsymbol{A}^2=\begin{bmatrix}2&1&1&2\\1&3&2&1\\1&2&3&1\\2&1&1&2\end{bmatrix}.$$

图 1.17说明 $(\boldsymbol{A}^2)_{1,4}=a_{1,2}a_{2,4}+a_{1,3}a_{3,4}=2$，对应 $(1,2,4)$ 与 $(1,3,4)$。对角元 d_i 数的是走到一个邻居再立刻返回的闭行走，它们不是环。再乘一次 $\boldsymbol{A}$

可以数三角形：$(\boldsymbol{A}^3)_{i,i} = \sum_j (\boldsymbol{A}^2)_{i,j} a_{j,i}$，即对 i 的邻居 j 累加 $(\boldsymbol{A}^2)_{i,j}$，得对角元 $2,4,4,2$，迹为 12。长度为 3 的闭行走中三个顶点两两相邻，恰好组成一个三角形，而每个三角形可以从 3 个顶点出发、沿 2 个方向绕行，所以三角形个数为 $12/6 = 2$。



(a) 从 1 到 4 长度为 2 的行走

	1	2	3	4
1	2	1	1	2
2	1	3	2	1
3	1	2	3	1
4	2	1	1	2

$(\mathbf{A}^2)_{1,4} = \sum_\ell a_{1,\ell}\, a_{\ell,4}$
$= a_{1,2}a_{2,4} + a_{1,3}a_{3,4}$
$= 1 + 1 = 2$

(b) $\mathbf{A}^2$

图 1.17 邻接矩阵的平方计数长度为 2 的行走。(a) 从 1 到 4 的两条行走 $(1,2,4)$（蓝色）与 $(1,3,4)$（绿色），中间顶点分别是 2 和 3；(b) 贯穿图的 $\boldsymbol{A}^2$，黄色条目 $(\boldsymbol{A}^2)_{1,4} = 2$ 按中间顶点 ℓ 求和，只有 $\ell = 2,3$ 的两项非零。

计数的范围与代价 这个计数包含重复顶点的行走，不能直接当作路径数或环数；更长的闭行走会混入往返与重复绕行，不能像三角形那样简单换算。

矩阵幂也能判断可达，但代价很高。由引理 1.6，j 从 i 可达当且仅当 $\boldsymbol{I} + \boldsymbol{A} + \cdots + \boldsymbol{A}^{n-1}$ 的第 (i,j) 个条目为正，但每次稠密乘法要 n^3 次乘加，远高于遍历的 $O(n+m)$；矩阵幂的价值在于计数和代数性质。

带权时 $(\boldsymbol{W}^k)_{i,j}$ 是行走上边权乘积之和，既不是条数也不是长度之和；沿路线相加再取最小，需要把乘法换成加法、求和换成取最小，属于第 3章。

1.5.5 关联矩阵

邻接矩阵以顶点对为单位，看不出“第几条边”。以边为单位的计算，例如对每条边比较两个端点上的数值，需要行对应顶点、列对应边的矩阵。把边编号为 $e_1, \ldots, e_m$，给每条无向边任选一个记录方向，例如把 $e_k = \{i,j\}$ 记为从 i 到 j。**关联矩阵**（incidence matrix）$\boldsymbol{B} \in \mathbb{R}^{n \times m}$ 的第 k 列在起点 i 处为 1、终点 j 处为 -1，其余为 0；有向图直接用弧的方向。

构造时按编号读边列表，在第 k 列的起点行写 1、终点行写 -1。贯穿图取“小编号指向大编号”的记录方向，得到

$$\boldsymbol{B} = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ -1 & 0 & 1 & 1 & 0 \\ 0 & -1 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & -1 \end{bmatrix}, \tag{1.5}$$

即图 1.16(d)。每列恰有两个非零元且和为 0，第 i 行的非零元个数等于度 d_i；不满足这两条的列就是记错的边。

对无向图，记录方向只是一种约定。反转第 k 条边的记录方向，第 k 列整列变号，相当于右乘一个对角元为 ± 1 的对角矩阵，图本身没有改变，所以反映图性质的计算不应依赖这一选择。有向图的弧方向则是数据的一部分。

关联矩阵的列以边为单位，正适合以边为单位的计算。若给每个顶点赋一个数，组成向量 $\boldsymbol{x} \in \mathbb{R}^n$，则 $\boldsymbol{B}^{\top}\boldsymbol{x}$ 的第 k 个分量是 $\sum_v b_{v,k}x_v = x_i - x_j$，即第 k 条边记录的起点与终点上数值之差。反转这条边的记录方向只改变差的符号，不改变差的大小。顶点与边上的数值函数、由 $\boldsymbol{B}$ 构造的拉普拉斯矩阵及加权度，可参考卷九第 9 章“图上函数与图算子”。

1.5.6 四种表示的比较

表 1.3汇总了前面各小节的分析。没有一种表示在所有操作上都最好：逐边处理用边列表，逐点列邻居用邻接表，频繁查询顶点对或做矩阵运算用邻接矩阵，以边为单位的线性计算用关联矩阵。

表示	存储	扫描全部边	列出 i 的邻居	查询 $\{i, j\}$ 是否为边
边列表	$O(n+m)$	$O(m)$	$O(m)$	$O(m)$
邻接表	$O(n+m)$	$O(n+m)$	$O(d_i)$	$O(d_i)$
稠密邻接矩阵	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$
稠密关联矩阵	$O(nm)$	$O(nm)$	$O(m + d_i n)$	$O(m)$

表 1.3　四种表示的存储与三种基本访问的时间代价，n 为顶点数，m 为边数，d_i 为顶点 i 的度。边列表、邻接表不附加索引；稠密关联矩阵列出邻居时先扫描第 i 行找出 d_i 条邻边，再在每条边的列中找另一个端点。关联矩阵若改用按行、按列都可访问的稀疏存储，存储降为 $O(n+m)$，列出邻居的代价与邻接表相当。

算法所需的表示往往与输入不同，转换的代价也应计入：边列表与邻接表互转用 $O(n+m)$ 时间；由边列表建稠密邻接矩阵要 $O(n^2+m)$，由稠密邻接矩阵建邻接表要读遍 n^2 个条目；建稠密关联矩阵仅置零就要 $O(nm)$。

选择表示时，先列出算法反复执行的操作，再按表估计总代价并计入转换与存储：第 2章的遍历用邻接表，第 4章按边权排序用边列表，图上的矩阵算子则用邻接矩阵与关联矩阵。实际程序常同时保存两种表示，额外的存储也应计入。

1.6 图同构

顶点的名称只是为了区分顶点而取的。把贯穿图的顶点换一套编号，边集的写法会变，邻接矩阵的行列也会移动，连接结构本身却没有变。本节用保持邻接的一一对应把“只差一套名称”写成同构的定义。

有了定义，给定的对应容易核对，不同构也常能用简单的量排除，难的是在没有给出对应时断定同构。最后一小节用置换矩阵把重编号写成矩阵运算。系统比较两张图结构的算法是第 2.6节的内容。

1.6.1　图同构的定义

图 1.18(a) 是贯穿图 $\mathcal{G}$；(c) 的 $\mathcal{G}'$ 顶点集也是 $\{1,2,3,4\}$，边集由 $\{1,3\}$、$\{1,4\}$、$\{2,3\}$、$\{2,4\}$、$\{3,4\}$ 组成，两者的边集不相等。但若按 (b) 的规则 ρ 把 $\mathcal{G}$ 的顶点改名，$\mathcal{G}$ 的五条边依次变为

$$\{1,2\}\mapsto\{1,3\},\qquad \{1,3\}\mapsto\{1,4\},\qquad \{2,3\}\mapsto\{3,4\},$$
$$\{2,4\}\mapsto\{3,2\},\qquad \{3,4\}\mapsto\{4,2\},$$

恰好是 $\mathcal{G}'$ 的全部边；唯一不相邻的一对 $\{1,4\}$ 变为 $\{1,2\}$，在 $\mathcal{G}'$ 中也不相邻。可见这样的改名要一一对应，并且相邻的仍相邻，不相邻的仍不相邻。

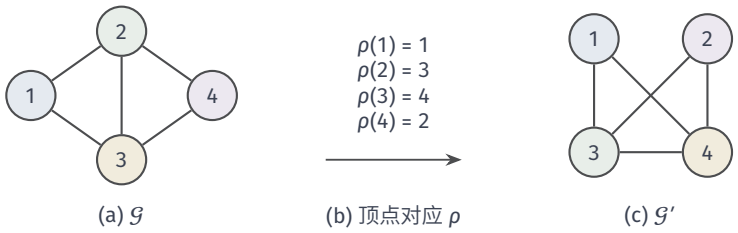


图 1.18　两张同构的图。(a) 贯穿图 $\mathcal{G}$；(c) 画法与编号都不同的 $\mathcal{G}'$；(b) 顶点对应 ρ。同色顶点互相对应：$\mathcal{G}$ 中顶点 i 与 $\mathcal{G}'$ 中顶点 $\rho(i)$ 颜色相同。逐条核对可见，$\mathcal{G}$ 中相邻的顶点对恰好对应 $\mathcal{G}'$ 中相邻的顶点对。

定义 1.18　图同构　设 $\mathcal{G}=(V,E)$ 与 $\mathcal{G}'=(V',E')$ 是无向图。若存在双射 $\rho: V\to V'$，使对任意 $i,j\in V$ 都有

$$\{i,j\}\in E \iff \{\rho(i),\rho(j)\}\in E',$$

就称 $\mathcal{G}$ 与 $\mathcal{G}'$**同构**（isomorphic），称 ρ 为一个同构映射。

双向条件不能减弱为单向：边集为 $\{1,2\},\{2,3\}$ 的路径按恒等映射放进三角形 $\{1,2,3\}$，边都映为边，但三角形多出了边 $\{1,3\}$。反方向的条件保证 $\mathcal{G}'$ 中不出现多余的连接。

有向图的同构要求 $(i,j)\in E \iff (\rho(i),\rho(j))\in E'$。$1\to 2\to 3$ 与 $1\to 2\leftarrow 3$ 的底层无向图相同，作为有向图却不同构：同构映射保持入度，而后者的顶点 2 入度为 2。加权图还要求 $w'_{\rho(i),\rho(j)}=w_{i,j}$；带属性的图要由问题声明哪些属性必须保持，例如比较分子时原子种类必须一致，原子编号无关紧要。

1.6.2 同构的核对与排除

若别人给出了一个对应 ρ，只需核对它是否满足定义；若只给出两张图，则要判断是否存在这样的对应，后者困难得多。

给定对应的核对 定义的条件要对全部 $\binom{n}{2}$ 个顶点对检查。下面的引理说明，只需检查边的像，再比较边数。

> **引理 1.19 同构映射的核对** 设 $\rho : V \to V'$ 是双射，$|E| = |E'|$，并且 $\mathcal{G}$ 的每条边 $\{i, j\}$ 都满足 $\{\rho(i), \rho(j)\} \in E'$。则 ρ 是 $\mathcal{G}$ 到 $\mathcal{G}'$ 的同构映射。

> **证明** 由于 ρ 是单射，$\{i, j\} \mapsto \{\rho(i), \rho(j)\}$ 是从 E 到 E' 的单射；两个有限集合元素个数相同，所以它是双射。于是若 $\{\rho(i), \rho(j)\} \in E'$，它是某条边 $\{i', j'\} \in E$ 的像，由 ρ 单射得 $\{i', j'\} = \{i, j\} \in E$。 ■

本节开头的例子正是这样核对的：ρ 是双射，两图都有 5 条边，五条边的像都在 E' 中，所以 ρ 是同构映射，“不相邻的一对仍不相邻”这一步可以省去。$\mathcal{G}'$ 以邻接矩阵存储时，核对用 $O(n + m)$ 时间。

图不变量 同构映射保持一切由连接结构定义的量：顶点数、边数、每个顶点的度（ρ 把 i 的邻居一一映为 $\rho(i)$ 的邻居）、度的多重集合、连通分量个数、三角形个数等。这些量称为**图不变量**（graph invariant），只要有一个不同，两张图就不同构；计算时宜从便宜的顶点数、边数和排序后的度开始。六个顶点排成的环与两个互不相连的三角形，顶点数、边数都是 6，每个顶点的度都是 2，但前者连通、后者有两个分量，所以不同构。

反过来，比较过的不变量都相同，也不足以断定同构。考虑两棵六个顶点的树，边集分别为

$$\{\{1,2\},\{1,3\},\{1,4\},\{2,5\},\{3,6\}\} \quad \text{与} \quad \{\{1,2\},\{1,3\},\{1,4\},\{2,5\},\{5,6\}\}.$$

二者都连通、没有三角形，度的多重集合都是 $\{3,2,2,1,1,1\}$。但前一棵树中度为 3 的顶点 1 与两个度为 2 的顶点 2, 3 相邻，后一棵树中顶点 1 只与一个度为 2 的顶点 2 相邻。同构映射必须保持度为 3 的顶点及其邻居的度，所以两棵树不同构：比较“邻居的度”能区分只看度区分不了的图。

要得出“同构”的肯定结论，必须真的找到一个对应。逐个尝试全部 $n!$ 个双射再逐个核对，虽然总能判定，但 $n = 20$ 时 $n!$ 已超过 2×10^{18}。不变量可以缩小搜索，例如度为 3 的顶点只能映到度为 3 的顶点。第 2.6节的

顶点标记细化把“比较邻居的度”反复执行，并会说明它仍然不能区分所有不同构的图。

1.6.3　重编号与置换表示

当两张图的顶点集都是 $\{1,\dots,n\}$ 时，同构映射 ρ 就是一个重编号。重编号以后，邻接矩阵的行和列都要按同样的方式重排，置换矩阵恰好执行这种重排，于是同构可以直接在矩阵上核对。

记 $\boldsymbol{e}_i$ 为第 i 个分量为 1、其余分量为 0 的标准基向量。取置换矩阵 $\boldsymbol{P}=\begin{bmatrix}\boldsymbol{e}_{\rho(1)} & \cdots & \boldsymbol{e}_{\rho(n)}\end{bmatrix}$，即第 i 列是 $\boldsymbol{e}_{\rho(i)}$，它满足 $\boldsymbol{P}\boldsymbol{e}_i=\boldsymbol{e}_{\rho(i)}$。按条目写，$p_{k,i}=1$ 当且仅当 $k=\rho(i)$，其余条目为 0；由于 ρ 是双射，$\boldsymbol{P}$ 的每行、每列都恰有一个 1。

计算 $\boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\top}$ 的条目时，按乘法定义展开

$$(\boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\top})_{k,l}=\sum_{i=1}^{n}\sum_{j=1}^{n}p_{k,i}\,a_{i,j}\,p_{l,j}.$$

求和中只有 $k=\rho(i)$ 且 $l=\rho(j)$ 的一项非零，所以左乘 $\boldsymbol{P}$ 把第 i 行移到第 $\rho(i)$ 行，右乘 $\boldsymbol{P}^{\top}$ 把第 j 列移到第 $\rho(j)$ 列。两者合起来，

$$\boldsymbol{A}'=\boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\top},\qquad a'_{\rho(i),\rho(j)}=a_{i,j}. \tag{1.6}$$

原来记录“i,j 是否相邻”的条目被搬到了新编号的位置。因此，ρ 是 $\mathcal{G}$ 到 $\mathcal{G}'$ 的同构映射，当且仅当 $\mathcal{G}'$ 的邻接矩阵等于 $\boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\top}$。

对图 1.18的 ρ，$\boldsymbol{P}$ 的各列依次为 $\boldsymbol{e}_1,\boldsymbol{e}_3,\boldsymbol{e}_4,\boldsymbol{e}_2$。由式(1.4)得

$$\boldsymbol{P}=\begin{bmatrix}1&0&0&0\\0&0&0&1\\0&1&0&0\\0&0&1&0\end{bmatrix},\qquad \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\top}=\begin{bmatrix}0&0&1&1\\0&0&1&1\\1&1&0&1\\1&1&1&0\end{bmatrix}.$$

右边矩阵的第 1 行读出 $N'(1)=\{3,4\}$，第 4 行读出 $N'(4)=\{1,2,3\}$，各行都与 $\mathcal{G}'$ 的边集一致；例如 $a'_{3,2}=a'_{\rho(2),\rho(4)}=a_{2,4}=1$。实际计算不必做矩阵乘法，按第二式把每个条目搬到 $(\rho(i),\rho(j))$ 即可，代价为 $O(n^2)$。加权图同样要求 $\boldsymbol{W}'=\boldsymbol{P}\boldsymbol{W}\boldsymbol{P}^{\top}$；允许零权边时还要检验结构掩码 $\boldsymbol{M}'=\boldsymbol{P}\boldsymbol{M}\boldsymbol{P}^{\top}$。

置换与一般换基　$\boldsymbol{P}$ 的各列是互不相同的标准基向量，所以 $\boldsymbol{P}^{\top}\boldsymbol{P}=\boldsymbol{I}$，即 $\boldsymbol{P}^{\top}=\boldsymbol{P}^{-1}$。于是式(1.6)可以写成 $\boldsymbol{A}'=\boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{-1}$，是一个相似变换，即在另一组基下表示同一个线性映射（可参考卷二第 2 章“线性空间、映射与坐

标”）。但反过来，一般的相似变换不能都解释为重编号。对两个顶点一条边的图，$\boldsymbol{A}=\left[\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right]$，取 $\boldsymbol{S}=\left[\begin{smallmatrix}1&1\\0&1\end{smallmatrix}\right]$，得到

$$\boldsymbol{S}\boldsymbol{A}\boldsymbol{S}^{-1}=\begin{bmatrix}1&1\\0&1\end{bmatrix}\begin{bmatrix}0&1\\1&0\end{bmatrix}\begin{bmatrix}1&-1\\0&1\end{bmatrix}=\begin{bmatrix}1&0\\1&-1\end{bmatrix},$$

它不对称，有非零对角元和负条目，不是任何简单无向图的邻接矩阵。一般换基混合不同顶点的坐标，重编号只搬动位置。

相似变换保持特征值，所以邻接矩阵的特征值也是图不变量：特征值不同的两张图一定不同构。它和其他不变量一样只能排除同构；图谱的用途与局限可参考卷九第 10 章“图谱与谱表示”。

1.7 本章小结

本章开头问：图记录了什么，沿连接能到哪里，删除连接会怎样，结构又怎样记录和比较。第一个问题的回答是，图 $\mathcal{G}=(V,E)$ 只记录有限个对象之间的直接连接，与纸上的画法无关，比较两幅画只需比较顶点集与边集。方向、边权和属性各自附加一类信息。方向决定哪些顶点序列可以通行，忽略方向得到的底层无向图丢掉了先后。边权的含义由问题决定：长度沿路线相加，容量限制单条边的通过量，相似度鼓励两端接近；零权边仍是边，“能否到达”查边集，“累计多少”查边权。子图可以任意保留一部分边，诱导子图则保留选定顶点之间的全部边，因而由顶点集唯一确定。

沿连接能到哪里，由行走和路径回答。行走允许重复，便于拼接与计数；路径不重复顶点，对应实际路线。任何行走都可以随读随删地化成一条同端点的路径，所以用两者定义的可达关系相同。无向图中可达关系是等价关系，它把顶点集唯一地划分为连通分量；核对一个声称的划分，只需检查没有边跨越不同的块、每块内部连通。有向图中可达不对称，要区分要求双向到达的强连通与忽略方向的弱连通，前者蕴含后者，反之不然。任务能否排出先后，取决于有没有顺着箭头绕回起点的有向环；不含有向环的有向无环图，底层无向图仍可以有环。

删除连接的影响由分隔集、割点和桥刻画，判断的依据始终是有没有替代路径。核对分隔集，就是在删除后的图中求出可达集；一条边是桥当且仅当它不在任何环上，删边至多使分量数增加一，删点却可能把分量拆成许多块。用最少的边保持连通，就得到树：树等价于任意两点之间路径唯一，n 个顶点的树恰有 $n-1$ 条边。连通、无环、恰有 $n-1$ 条边三个条件中任意两个推出第三个，所以判断树只需数边再查连通。选根把树组织成父子层次而不改变边集；生成森林在每个分量上保留一棵生成树，恰有 $n-c$ 条边，并且一般不唯一，它丢掉了原图的替代路径。

同一张图可以用边列表、邻接表、邻接矩阵和关联矩阵记录。四者保存的连接相同，支持的访问和代价不同，转换本身也有代价，选择时应先列出算法反复执行的操作。邻接矩阵还把组合计数变成矩阵运算：$\boldsymbol{A}^{k}$ 的条目计数长度为 k 的行走，其中包含重复顶点，不能直接当作路径数。

同构用保持邻接与不相邻的双射刻画“只差一套名称”，在矩阵上表现为置换共轭 $\boldsymbol{A}' = \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\mathsf{T}}$。给定对应时，核对边的像并比较边数就够了。度、分量数、特征值等不变量可以排除同构，却不足以确认同构，而逐个尝试 $n!$ 个对应并不可行。

本章说明了怎样在小图上核对连通分量、强连通分量、生成树、无环性和同构，却还没有给出在大图上高效求出它们的方法。第 2章在邻接表上用广度优先和深度优先遍历实际计算这些对象，用拓扑排序安排有向无环图的次序，并用顶点标记细化比较图的结构；那里的算法输出，仍要按本章的定义与核对方法检验。

第 2 章

图的遍历与结构计算

第 1章定义了可达、连通分量、强连通分量、有向无环图、分隔集和同构。这些定义说明了什么样的结构算数，却没有说明怎样在一张存放在计算机里的图上把它们找出来。例如，可达性要求“存在一条路径”，而路径数目可以随顶点数指数增长，逐条列举并不可行。本章只沿邻接关系有组织地访问顶点，由此得到距离、分量和依赖次序，并说明每项输出为什么正确、花费多少计算。

第 2.1节和第 2.2节给出两种基本的访问方式，后面各节都以它们为工具。广度优先遍历用队列由近及远逐层推进，得到最少边数和相应路线；深度优先遍历用栈沿一条分支走到底再退回，得到发现与完成两种次序，并能判断有没有环。

其余四节把两种遍历用于第 1章的各个结构。第 2.3节计算无向连通分量和生成森林，引入在不断加边时维护分量的并查集，再用两遍深度优先遍历求出有向图的强连通分量；把每个强分量缩成一点，得到的图没有有向环。第 2.4节在无环图上安排依赖次序。第 2.5节删除一个给定的顶点集，把图拆成若干共享接口的局部块。第 2.6节回到同构问题，用邻域信息反复细化顶点标记，得到不依赖编号的比较手段，并指出它的局限。

本章讨论有限图，输入以第 1.5.2小节的邻接表给出，默认无自环、无平行边，需要保留边身份时另行说明。记顶点数为 $n = |V|$，无向图的边数和有向图的弧数都记为 m；邻接表中无向边在两端各存一次，有向图默认只存出邻居。复杂度中的 $O(n + m)$ 表示所需基本操作次数不超过 $n + m$ 的常数倍。

本章的算法只使用结构上的邻接，不比较边权：生成森林只要求连通、不要求总权最小，广度优先遍历只数边数、不计长度。在给定结构中寻找最小成本或最大流量，是第 2部分的问题。

2.1 广度优先遍历

给定一张图和一个起点，最基本的问题是：从起点出发能走到哪些顶点，走到每个顶点最少要经过几条边，又该沿哪条路线走。广度优先遍历按离起点的远近逐层访问顶点，一次就回答这三个问题。

2.1.1 问题与思路

输入是图 $G = (V, E)$ 的邻接表和一个起点 $s \in V$。记 $\mathrm{dist}(s, v)$ 为从 s 到 v 的路径所含的最少边数，有向图中路径沿弧的方向；v 从 s 不可达时约定 $\mathrm{dist}(s, v) = +\infty$。对每个顶点 v，我们要知道它是否从 s 可达；若可达，$\mathrm{dist}(s, v)$ 是多少，以及一条恰好用 $\mathrm{dist}(s, v)$ 条边从 s 走到 v 的路线。

按定义逐条检查路径是行不通的。从 s 到 v 的路径可能极多，而且路径越长，种类增长越快；即使只想知道最少边数，也不能把所有路径列出来再比较。

不枚举路径，可以按距离把顶点分层。距离为 0 的只有 s 自己，距离为 1 的恰好是 s 的邻居。距离为 2 的顶点不在前两层，却与第 1 层某个顶点相邻：从 s 走两条边能到达它，一条边到不了。一般地，前 k 层找全以后，第 $k+1$ 层就是与第 k 层相邻、但还没出现过的顶点。于是从第 0 层开始，每次由当前层找出下一层，直到某一层为空。

一个顶点可能与上一层的多个顶点相邻，也可能与本层或更早的顶点相邻，所以它只能在第一次被找到时归入某一层，之后再遇到就跳过。找到它时顺手记下是从哪个顶点找到的，沿这些记录往回走，就能恢复一条路线。

2.1.2 算法实现

逐层推进不必真的维护一层层的列表，一个先进先出的队列就够了：先进入队列的顶点先展开，第 k 层的顶点全部展开完之前，第 $k+1$ 层的顶点只在队尾排队。算法维护四项状态：

- 发现标记：顶点是否已经被找到；
- 队列 Q：已经发现、但邻居尚未检查的顶点；
- 边数距离 $d(v)$：v 被找到时所在的层号；
- **父指针**（parent pointer）$p(v)$：v 是在展开哪个顶点时第一次被找到的。

初始时只有 s 被发现，$d(s) = 0$，$p(s)$ 为空，$Q = [s]$；其余顶点未发现，$d(v) = +\infty$。此后每一轮从队首取出一个顶点 u，依次检查它的邻居。未发现的邻居 v 属于 u 的下一层：立即把它标记为已发现，写入 $d(v) = d(u)+1$ 和 $p(v) = u$，放到队尾。已发现的邻居则跳过。队列变空时，从 s 可达的

顶点都已找到。这种按层访问的过程称为广度优先遍历，也常称**广度优先搜索**（breadth-first search，BFS）。

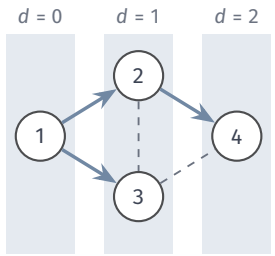
算法 2.1 广度优先遍历

输入：图 $\mathcal{G}=(V,E)$ 的邻接表，起点 $s\in V$。输出：距离 $d(v)$ 与父指针 $p(v)$，$v\in V$。

1. 标记 s 为已发现，置 $d(s)=0$、$p(s)$ 为空，令 $Q=[s]$；其余顶点未发现，$d(v)=+\infty$。
2. 当 Q 非空时，取出队首 u，按邻接表依次读取 u 的邻居 v（有向图只读出邻居 $v\in N^+(u)$）。若 v 未发现，立即标记 v 为已发现，置 $d(v)=d(u)+1$、$p(v)=u$，并把 v 放到队尾；若 v 已发现，跳过。
3. 队列为空时停止。

“只在第一次被找到时归层”要靠入队时就标记来落实。两个已发现的顶点可能共享一个尚未发现的邻居；若等到出队时才标记，这个邻居会被两个顶点各放入队列一次，距离和父指针也可能被改写。入队即标记，每个顶点就至多入队一次。

执行过程 在第 1.1节的贯穿四点图上，以 1 为起点，并约定按编号从小到大读取邻居。图 2.1(b) 逐行列出了每次出队后的队列。初始 $Q=[1]$。取出 1 时发现 $2,3$，写入 $d(2)=d(3)=1$、$p(2)=p(3)=1$，队列变为 $[2,3]$。取出 2 时，邻居 $1,3$ 都已发现，只有 4 是新的，写入 $d(4)=2$、$p(4)=2$，队列变为 $[3,4]$。取出 3 时，邻居 $1,2,4$ 全部已发现，4 不会第二次入队，队列变为 $[4]$。最后取出 4，没有新顶点，队列为空，算法停止。图 2.1(a) 把各顶点按 d 值分到三层，蓝色箭头就是父指针所指的边。



(a) 距离层与父边（蓝色箭头）

出队	新发现	出队后的队列
初始	1	1
1	2, 3	2 3 ← 入队时即标记
2	4	3 4
3	无	4
4	无	空，结束

(b) 队首在左，绿色为本步新入队

图 2.1 贯穿四点图上从顶点 1 出发的广度优先遍历，邻居按编号递增读取。(a) 顶点按边数距离 d 分为三层，蓝色箭头是父指针对应的边，虚线是检查过但没有带来新顶点的边。(b) 每次出队后的队列，队首在左，绿色格子是这一步新入队的顶点；取出 3 时 4 已在队中，不再重复入队。

路线的恢复 要得到从 s 到已发现顶点 t 的路线，从 t 出发反复取父指针，直到回到 s，再把得到的顶点序列反转。在上例中，4 的父指针是 2，2 的

父指针是 1，反转后得到路线 $1 \to 2 \to 4$，恰有 $d(4) = 2$ 条边。父指针总是指向更早发现的顶点，所以回溯不会绕圈，一定在有限步内回到 s。若 t 始终未被发现，即 $d(t) = +\infty$，算法应报告 t 从 s 不可达，而不能输出任何路线。

路线并不唯一。若改为先读 3 再读 2，取出 1 后队列为 $[3, 2]$，4 在展开 3 时被发现，父指针变成 $p(4) = 3$，路线变成 $1 \to 3 \to 4$。两条路线不同，边数却都是 2：改变同层顶点的处理次序可能改变父指针，但下面的定理表明它不改变距离。

2.1.3 算法分析

正确性　算法只在顶点第一次被发现时写入 $d(v)$，以后不再修改。正确性的关键在于，这个“第一次”恰好发生在最短的距离上。

定理 2.1　广度优先遍历的距离　设对有限图 G 和起点 s 执行算法 2.1。则对每个顶点 v，$d(v) = \mathrm{dist}(s, v)$；当 v 可达时，沿父指针从 v 回到 s 的路线恰有 $d(v)$ 条边。

证明　先说明 $d(v)$ 不会小于真实距离。若 $d(v)$ 有限且 $v \neq s$，则 $d(p(v)) = d(v) - 1$，沿父指针每退一步距离减一，$d(v)$ 步后回到 s。反转这串顶点就得到一条从 s 到 v、恰有 $d(v)$ 条边的路线，所以 $d(v) \geq \mathrm{dist}(s, v)$。

再说明队列中的距离值有一个固定的形状：从队首到队尾不减，且最大值与最小值至多相差 1。初始队列只有 s，满足这一点。设某时刻队列中的值为若干个 k 后接若干个 $k+1$，取出的队首 u 有 $d(u) = k$，新加入的顶点距离都是 $k+1$，并放在队尾，形状仍然保持。由此可知，顶点出队时的距离值按时间不减。

最后对 $\delta = \mathrm{dist}(s, v)$ 归纳，证明有限距离的顶点都会被发现且 $d(v) \leq \delta$。$\delta = 0$ 时 $v = s$。设 $\delta \geq 1$，并设距离小于 δ 的顶点都已满足结论。取一条从 s 到 v 的最短路径，记其倒数第二个顶点为 x，则 $\mathrm{dist}(s, x) = \delta - 1$，由归纳假设 x 被发现且 $d(x) = \delta - 1$。x 入过队，就一定在某一时刻出队并检查邻居 v。若此时 v 尚未发现，就得到 $d(v) = d(x) + 1 = \delta$；若 v 已经被发现，则发现它的顶点 u 出队不晚于 x，按上一段的单调性 $d(u) \leq d(x)$，于是 $d(v) = d(u) + 1 \leq \delta$。两个方向合起来得到 $d(v) = \mathrm{dist}(s, v)$。不可达的顶点没有任何从 s 出发的路线，第一段说明它不可能得到有限的 d 值。 ■

证明第二段的队列形状把“逐层推进”变成了可以检验的事实：处理第 k 层时只会首次发现第 $k+1$ 层的顶点，而它们全部排在第 k 层之后。最

短路径上的倒数第二个顶点 x 比 v 少一层，一定先被展开，所以 v 不会拖到更远的层才被发现。

复杂度 以邻接表存储时，每个顶点至多入队、出队各一次，出队时把它的邻接表读一遍，因此每条邻接记录只被检查常数次。无向图的邻接记录共 $2m$ 条，有向图的出邻接记录共 m 条，总时间为 $O(n+m)$。发现标记、距离、父指针和队列各占 $O(n)$ 额外空间。这一结论依赖“列出邻居的代价与度数成正比”。若改用邻接矩阵，找出一个顶点的邻居要扫描一整行，n 个顶点共扫描 n^2 个位置，时间变为 $O(n^2)$；对稀疏图，这比 $O(n+m)$ 大得多。

适用范围 广度优先遍历把每条边都算作一步，而边上带有长度时，边数最少的路线不一定总长度最小。图 2.2中，从 s 直接到 t 只需一条边，广度优先遍历就用这条边记录 t，但它的长度是 5；经过 a 的路线多走一条边，总长度却只有 $1+1=2$。所以广度优先遍历只解决所有边长相等时的最短路问题。一般边长下的最短路需要比较累计长度，留到第 3章。



图 2.2 带权反例。边旁数字是长度。广度优先遍历从 s 出发时经直接边一步发现 t（蓝色），这条路线总长 5；绿色虚线路线 s–a–t 多一条边，总长只有 2。广度优先遍历最小化的是边数，不是总长度。

一次广度优先遍历也只覆盖从 s 可达的部分。遍历结束时距离仍为 $+\infty$ 的顶点没有被访问，不能据此断言它们是孤立点，它们之间可能有很多边。要给全图的每个顶点划分连通分量，需要在外层不断换新的起点重新启动，这是第 2.3节的内容。

2.1.4 多源广度优先遍历

有时关心的不是离某一个起点多远，而是离一组起点中最近的那个多远。例如在网格上放置若干个种子，问每个格子离最近的种子几步。给定起点集合 $S \subseteq V$，要求的是

$$\operatorname{dist}(S, v) = \min_{s \in S} \operatorname{dist}(s, v),$$

空集上的最小值约定为 $+\infty$；同时还想知道 v 离哪个起点最近。

既然只关心最近的起点，就把 S 中的所有顶点一起当作第 0 层，从这一层开始逐层推进。实现上只改初始化：S 中所有顶点都标记为已发现、距离为 0、父指针为空，并按任意次序全部放入队列；此后的更新与算法 2.1完全相同。这称为**多源广度优先遍历**（multi-source breadth-first search）。

它的正确性可以归结到单源情形。设想添加一个新顶点 s^*，从它向 S 中每个顶点连一条边（有向图中连弧 $s^* \to s$）。新图中任一原顶点 v 满足 $\mathrm{dist}(s^*, v) = 1 + \mathrm{dist}(S, v)$，因为从 s^* 出发的路线第一步必然进入 S。从 s^* 做普通广度优先遍历，第一次出队就把 S 中所有顶点以距离 1 放入队列，此后的过程与多源版本完全相同，只是所有距离多了 1。由定理 2.1，多源版本输出 $\mathrm{dist}(S, v)$。沿父指针回溯会停在某个起点上，回溯路线恰有 $\mathrm{dist}(S, v)$ 条边，所以这个起点就是离 v 最近的起点之一。

图 2.3是一个 3×5 网格，相邻格子以上下左右相连，中间一个格子不属于图。两个起点放在左上角和右下角，每个格子中的数字是它到较近起点的边数。例如第一行第四个格子离左上角 3 步、离右下角也是 3 步，记为 3，两个起点都是它的最近起点；第一行最右的格子离右下角只有 2 步，记为 2。若只从左上角出发，右下角的格子距离为 6。

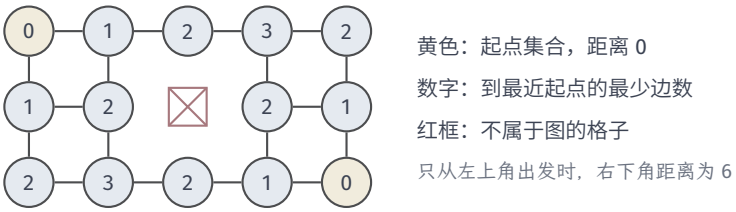


图 2.3　3×5 网格上的多源广度优先遍历。网格以上下左右相邻连边，红框格子不属于图；两个黄色起点同时作为第 0 层。每个顶点中的数字是到较近起点的最少边数，所有距离由一次遍历得到。

对每个起点各做一次广度优先遍历，回答的是另一个问题：它得到 $|S|$ 组距离，能说出 v 到每个起点各有多远，需要 $O(|S|(n+m))$ 时间。多源遍历只给出到最近起点的距离及对应的起点，一次只需 $O(n+m)$ 时间。若 S 为空，队列一开始就为空，所有顶点保持未发现，与 $\mathrm{dist}(\varnothing, v) = +\infty$ 的约定一致。

来源标签的同步传播　逐个回溯父指针才能知道来源，每次要花与距离成正比的时间。若需要全部顶点的最近来源，可以让来源随发现一起传下去：再维护一个数组 $\mathrm{src}(\cdot)$，初始化时对每个 $s \in S$ 置 $\mathrm{src}(s) = s$；展开 u 时每发现一个新顶点 v，除写入 $d(v)$ 和 $p(v)$ 外，再置 $\mathrm{src}(v) = \mathrm{src}(u)$。由于 $p(v) = u$，按父指针归纳可知 $\mathrm{src}(v)$ 就是从 v 回溯父指针到达的起点，所以它是 v 的一个最近来源。离两个起点一样近的顶点（例如上面网格第一行第四个格子）只记录先发现它的那一个；需要全部最近来源时，要在距离相等的情形下另行合并记录。

图像中从若干种子格子出发的区域生长，用的正是这一机制：来源标签说明每个格子归入哪颗种子。判断哪些像素属于前景、怎样选择邻接规则等具体处理不在本卷讨论。

2.2 深度优先遍历

广度优先遍历同时推进许多条路线，每条只走一步。深度优先遍历只盯住一条路线往前走，走到无路可走再退回来。它同样能找出全部可达顶点，还额外记录了当前正沿哪条分支前进、某个顶点之后的部分何时全部探完；后面判断有向环、计算强连通分量和安排依赖次序，都要用到这两项信息。

2.2.1 问题与思路

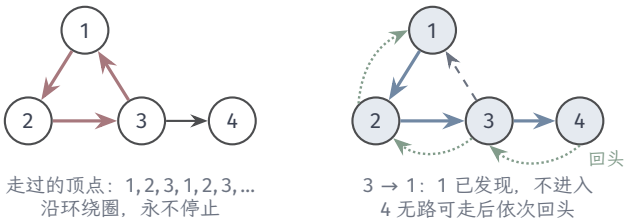
输入仍是图 $G = (V, E)$ 的邻接表。我们要访问全部顶点，并对每个顶点记录三项信息：它是从哪个顶点走过来的，即父指针 $p(v)$；它何时第一次被走到；它何时被彻底处理完，即从它出发能继续探索的部分何时全部探完。后两项用一个逐步递增的时钟记录，分别称为发现时刻 $d(v)$ 和完成时刻 $f(v)$。广度优先遍历给出的距离回答“离起点多远”，却不回答“谁的后续先探完”这类先后问题。

做法是一条路走到底再回头：从当前顶点挑一个还没去过的邻居走过去，再从那里继续往前；到了一个顶点，若它的邻居都已去过或者根本没有邻居，就退回上一个顶点，改走它的下一个邻居。退回到起点、起点也没有可走的邻居时，从起点能到达的部分就探完了。走迷宫时扶着一侧墙一路前进、碰到死路再折回，就是这种方式。

环的处理　如果不记录哪些顶点已经去过，“一直往前走”在有环的图上会停不下来。图 2.4(a) 中，弧为 $1 \to 2$、$2 \to 3$、$3 \to 1$、$3 \to 4$。从 1 出发沿弧前进，走到 3 后若选择 $3 \to 1$，就回到了起点，接着又走 $1 \to 2 \to 3$，如此绕圈，永远不会检查 4，也永远不会结束。

所以要给顶点加上发现标记，并规定只走向尚未发现的邻居：遇到已经发现的邻居就不进入，接着看下一个邻居；当前顶点的邻居都检查完了，就回头。图 2.4(b) 中，走到 3 时看到 1 已发现，不进入，改走 4；4 没有出邻居，回头到 3；3 的邻居检查完毕，再依次回头到 2、1。每个顶点只被走进一次，绕圈就不可能发生。

回头所需的记录　回头还带来两个要求。第一，退回到某个顶点时，要接着检查它的下一个邻居，而不是从头再来，所以每个顶点要记住自己的邻居检查到了哪里。



(a) 不记录已发现顶点　　(b) 只进入未发现顶点

图 2.4　深度优先遍历的回头规则。有向图的弧为 1 → 2、2 → 3、3 → 1、3 → 4。(a) 不记录已发现的顶点时，沿红色的环反复前进，永远到不了 4；(b) 只走向未发现的顶点：蓝色是实际走过的弧，虚线弧 3 → 1 因 1 已发现而不进入；4 无路可走后沿绿色点线依次回头。

第二，“已发现”要再细分：一个顶点被走到之后、邻居检查完之前，它位于从起点到当前位置的这条分支上；邻居全部检查完后，它就离开了分支，不会再从它出发。前者称为处理中，后者称为已完成，加上尚未被走到的未发现，顶点共有三种状态。处理中的顶点恰好构成当前分支，判断环时要用到这一点（第 2.2.4小节）；完成时刻则记录了某个顶点之后的部分何时全部探完。

2.2.2 算法实现

当前分支后进先出，最后走到的顶点最先退回，所以用一个栈保存它：栈底是本次起点，栈顶是当前顶点。算法维护五项状态：

- 顶点状态：每个顶点是未发现、处理中还是已完成；
- 栈 S：当前分支上的顶点，每一帧 $(u, \text{位置})$ 记录顶点 u 以及 u 的邻接表中下一条待检查记录的位置；
- 父指针 $p(v)$：v 是从哪个顶点第一次走到的；
- 发现时刻 $d(v)$ 与完成时刻 $f(v)$：v 进栈和出栈时的时钟读数；
- 时钟 t：到目前为止发生过的发现与完成事件的次数。

初始时所有顶点都是未发现，栈为空，$t = 0$。从起点 r 出发时，令 $p(r)$ 为空，时钟加一后记为 $d(r)$，r 改为处理中，压入帧 $(r, \text{邻接表起点})$。

此后每一步只看栈顶帧 $(u, \text{位置})$，按 u 的邻居是否已经检查完，分为前进和回头两种情形：

- 前进：u 还有未检查的邻居。读出下一个邻居 v，把位置后移一格。若 v 未发现，就走到 v：置 $p(v) = u$，时钟加一后记为 $d(v)$，v 改为处理中，压入帧 $(v, \text{邻接表起点})$。若 v 已发现，不进入，这一步结束。
- 回头：u 的邻居已全部检查。时钟加一后记为 $f(u)$，u 改为已完成，弹出栈顶帧，下一步回到它下面的顶点。

栈变空时，从 r 可达的部分已经探完。这种访问方式称为深度优先遍历，也常称**深度优先搜索**（depth-first search，DFS）。

算法 2.2 深度优先遍历（显式栈）

输入：图 $G=(V,E)$ 的邻接表。输出：父指针、每个顶点的发现时刻 $d(v)$ 与完成时刻 $f(v)$。

1. 所有顶点置为未发现，时钟 $t=0$。按某个固定次序扫描顶点；每遇到一个未发现的顶点 r，令 $p(r)$ 为空，执行下面两步。
2. 发现 r：$t \leftarrow t+1$，$d(r)=t$，r 改为处理中，把帧 $(r,$ 邻接表起点$)$ 压入栈。
3. 当栈非空时，看栈顶帧 $(u,$ 位置$)$。若 u 还有未检查的邻居，读取下一个邻居 v（有向图只读出邻居），并把位置后移一格：v 未发现时，置 $p(v)=u$，$t \leftarrow t+1$，$d(v)=t$，v 改为处理中并压入新帧；v 处于处理中或已完成时，不压栈，继续下一轮。若 u 的邻居已全部检查，则 $t \leftarrow t+1$，$f(u)=t$，u 改为已完成并弹出栈顶帧。

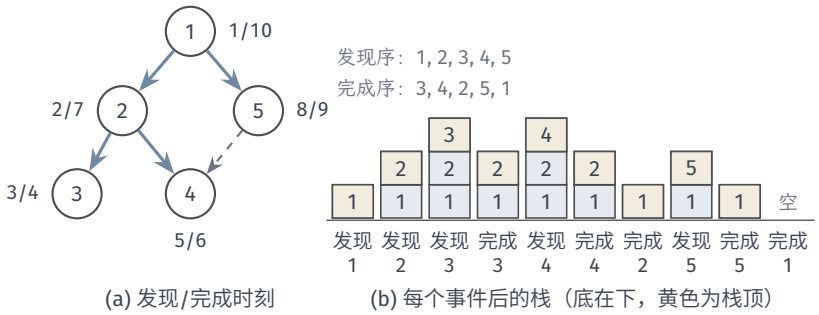
第一步的外层扫描保证覆盖全部顶点：栈空时若仍有未发现的顶点，就从下一个未发现顶点重新开始，已经完成的顶点不会被重新访问。

父指针把顶点组织成若干棵有根树，每次重新开始产生一棵，合起来称为深度优先森林。

执行过程 图 2.5(a) 是一个有分支的有向图，弧为 $1\to 2$、$1\to 5$、$2\to 3$、$2\to 4$、$5\to 4$。从 1 出发并按编号读取出邻居。发现 1，$d(1)=1$，栈为 $[1]$；1 的第一个出邻居 2 未发现，前进，$d(2)=2$，栈为 $[1,2]$；2 的第一个出邻居 3 未发现，前进，$d(3)=3$，栈为 $[1,2,3]$。3 没有出邻居，回头：$f(3)=4$，弹出 3。回到 2，它的下一个出邻居 4 未发现，前进，$d(4)=5$；4 也没有出邻居，$f(4)=6$，弹出。此时 2 的邻居已检查完，$f(2)=7$，弹出。回到 1，下一个出邻居 5 未发现，前进，$d(5)=8$；5 唯一的出邻居 4 已经完成，不进入，5 的邻居随即检查完，$f(5)=9$；最后 $f(1)=10$，栈变空。图 2.5(b) 画出了每个事件之后的栈，黄色格子是栈顶。

2.2.3 算法分析

发现次序与完成次序 按发现时刻从小到大排列顶点，得到**发现次序**（discovery order）；按完成时刻排列，得到**完成次序**（finishing order）。在图 2.5中，发现次序是 $1,2,3,4,5$，完成次序是 $3,4,2,5,1$。两者的差别来自栈的结构：一个顶点被发现之后、弹栈之前压入的所有顶点都位于它上方，必须先弹出。所以一个顶点的后代全部先于它完成；2 的后代 3,4 先完成，然后才轮到 2。下面两个引理把这一观察写准确，本节的正确性论证以及后面的强连通分量和拓扑排序都要用到它们。



(a) 发现/完成时刻　　(b) 每个事件后的栈（底在下，黄色为栈顶）

图 2.5　有向图 1 → 2、1 → 5、2 → 3、2 → 4、5 → 4 上从 1 出发的深度优先遍历，出邻居按编号读取。(a) 顶点旁的“d/f”是发现时刻与完成时刻，蓝色箭头是父指针对应的弧，虚线弧 5 → 4 检查时 4 已完成。(b) 十个事件后的栈，底在下、栈顶为黄色；栈高上升对应发现，下降对应完成。

引理 2.2　时刻区间的嵌套　在一次完整的深度优先遍历中，若顶点 v 在 u 处于处理中时被发现，即 $d(u) < d(v) < f(u)$，则 v 是 u 在深度优先森林中的后代，且 $f(v) < f(u)$。

证明　v 被发现时压入栈，位于 u 的上方，因为此时 u 仍在栈中。栈后进先出，v 必须在 u 之前弹出，所以 $f(v) < f(u)$。另外，v 的父指针指向发现它时的栈顶，这个栈顶要么是 u，要么是在 u 之后压入的顶点；对后者重复同样的推理，父指针链只会经过 u 上方的顶点，最终到达 u。因此 v 是 u 的后代。 ■

引理 2.3　未发现路径　若在时刻 $d(u)$ 存在一条从 u 到 v 的路径（有向图中沿弧方向），路径上除 u 以外的顶点此时都未发现，则 v 最终成为 u 的后代，从而 $f(v) < f(u)$。

证明　假设结论不成立，在路径上取第一个不成为 u 后代的顶点 w，记它在路径上的前一个顶点为 x。x 是 u 本身或 u 的后代，所以 $f(x) \le f(u)$。x 在完成之前要检查弧 $x \to w$，因此 w 在 $f(x)$ 之前已被发现；而按假设 w 在 $d(u)$ 时尚未发现，所以 $d(u) < d(w) < f(x) \le f(u)$。由引理 2.2，$w$ 是 u 的后代，矛盾。 ■

正确性　回头规则保证了不重复：只有未发现的顶点才会压栈，压栈的同时它就变为处理中，此后不可能再次压栈，所以每个顶点恰好被发现一次、完成一次，环不会让算法反复进入同一个顶点。每一帧的位置只向后移动，

每条邻接记录至多读一次，栈的每次变化不是压入一个新顶点就是弹出一个顶点，所以算法一定在有限步内停止。

也不会遗漏。设 r 是第一次扫描到的起点，在时刻 $d(r)$ 其余顶点都未发现，任何从 r 可达的顶点都通过一条全由未发现顶点组成的路径与 r 相连。由引理 2.3，它们都成为 r 的后代，即在从 r 出发的这一次遍历中被访问。外层扫描又保证每个顶点最终都会被发现。所以一次完整的遍历把全部顶点恰好访问一次，从单个起点出发时访问的恰是它的可达集。

复杂度 每个顶点恰好压栈、弹栈各一次，每条邻接记录恰好被读取一次，所以邻接表下总时间为 $O(n+m)$，状态、时刻、父指针和栈占 $O(n)$ 额外空间。栈的深度等于当前分支上的顶点数，最坏可达 n：在一条长链 $1 \to 2 \to \cdots \to n$ 上，栈会一直增长到装下全部顶点。

与广度优先遍历的比较 两种遍历访问的顶点集合相同,代价也同为 $O(n+m)$，记录的结构却不同，不能互相替代。

深度优先遍历的父指针也给出从根到每个顶点的路线，但它反映的是沿哪条分支先走到，与距离无关。在贯穿四点图上从 1 出发、按编号读取邻居：1 走到 2，2 的第一个未发现邻居是 3，3 的未发现邻居是 4，于是父路径为 $1 \to 2 \to 3 \to 4$，有三条边；而 $1 \to 2 \to 4$ 只需两条边，1 与 3 甚至直接相邻。需要最少边数时，应使用广度优先遍历。

反过来，完成次序是广度优先遍历给不出的信息，也不能用发现次序代替。设图 2.5中的弧表示依赖，$i \to j$ 意味着 i 要排在 j 之前。顶点 4 同时是 2 和 5 的后继。若按发现次序 $1,2,3,4,5$ 安排，5 排在 4 之后，违反了弧 $5 \to 4$：发现 4 时走的是经过 2 的分支，发现次序没有反映 5 也指向它。4 却在 5 之前就已完成，反转完成次序得到 $1,5,2,4,3$，五条弧都是起点在前。第 2.4.3小节会证明，在无环图中这一做法总是成立。

2.2.4 环的检测

给定一张图，我们要判断它是否含有环；若有，还要输出一个具体的环，使人可以逐条边核对。有向环意味着依赖关系自相矛盾，无法排出先后；无向图中的环意味着图不是森林。遍历中“遇到已发现的邻居就不进入”正是环留下的痕迹，但并非每次遇到已发现的邻居都说明有环，需要按顶点状态分清楚。

有向图 处理中的顶点构成当前分支，父指针已经把它们按先后串成一条有向路径。若检查栈顶 u 的出邻居时遇到弧 $u \to v$，而 v 也处于处理中，那么沿父指针从 u 往回走一定会到达 v，这段路径与弧 $u \to v$ 合起来就是一个有向环。这样的弧称为**回边**（back edge）。图 2.6(a) 中，栈内依次

是 $1,2,3$，检查 $3 \to 1$ 时 1 仍在栈中，沿父指针 $3 \to 2 \to 1$ 回溯，再补上 $3 \to 1$，得到有向环 $1 \to 2 \to 3 \to 1$。图 2.4(b) 中被跳过的弧 $3 \to 1$ 正是这样一条回边。

指向已完成顶点的弧则不同。图 2.6(b) 中先走 $1 \to 2$，2 没有出邻居而完成；回到 1 后走到 3，检查 $3 \to 2$ 时 2 已完成。2 已经不在当前分支上，弧 $3 \to 2$ 无法与当前分支拼成环。本例中从 2 出发也回不到 3（否则由引理 2.3，3 会在 2 完成之前就被发现），图中确实没有环。只看“是否已经访问过”会把这种情形误判为环，这就是要区分处理中与已完成的原因。

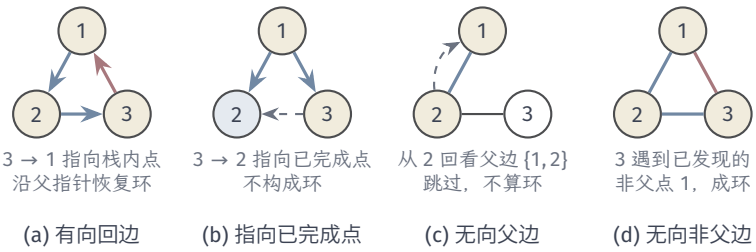


图 2.6　深度优先遍历中环的判断。黄色为处理中（在栈内）的顶点，蓝色填充为已完成顶点，蓝色粗线为父指针对应的边。(a) 有向图中 $3 \to 1$ 指向栈内顶点，构成回边，沿父指针恢复出环；(b)$3 \to 2$ 指向已完成顶点，不构成环；(c) 无向图中从 2 回看父边 $\{1,2\}$，这是同一条边的反向查看，应当跳过；(d) 无向三角形中 3 遇到已发现、但不经父边相连的顶点 1，构成环（红色）。

定理 2.4　有向环的判定　有限有向图含有向环，当且仅当对它的一次完整深度优先遍历中出现回边。出现回边 $u \to v$ 时，父指针链 $v \to \cdots \to u$ 加上弧 $u \to v$ 就是一个有向环。

证明　充分性已在上面说明：v 处于处理中且 u 是当前栈顶，u 在 v 上方，由引理 2.2u 是 v 的后代，父指针链给出从 v 到 u 的有向路径。

必要性。设图含有向环，在环上取最先被发现的顶点 v，记环上指向 v 的弧为 $u \to v$。在时刻 $d(v)$，环上其余顶点都未发现，沿环从 v 走到 u 的路径满足引理 2.3的条件，所以 u 成为 v 的后代，$d(v) < d(u) < f(u) < f(v)$。$u$ 在完成之前要检查弧 $u \to v$，那时 v 尚未完成，仍处于处理中，因此 $u \to v$ 是一条回边。 ■

因此判断有向环只需在算法 2.2的第三步多做一次检查：读到的邻居 v 处于处理中时，报告回边并沿父指针取出环。时间仍为 $O(n+m)$。

无向图　无向图中每条边在两端的邻接表里各出现一次。从 u 走到 v 之后，v 检查邻居时会再次看到 u，这只是同一条边的反向记录，不是新的路线。若把它当作“遇到已发现顶点”，任何一条边都会被误判为环，如图 2.6(c)。

因此规则是：检查 u 的一条邻接记录 $e=\{u,w\}$ 时，若 e 就是使 u 被发现的父边，跳过；否则只要 w 已经被发现，就报告存在环。

图 2.6(d) 中，3 经父边与 2 相连，它看到的 1 已被发现、且不经父边相连，于是 1–2–3–1 是一个环。允许平行边时，要按边的身份跳过，只跳过实际作为父边的那一条；两点之间的另一条平行边会与父边构成长度为 2 的环。

这一规则在两个方向上都可靠。先看没有报告的情形。若整次遍历从未报告，每条边在两端被检查时，要么发现了新顶点而成为父边，要么正是父边的反向记录，所以所有边都是父边，图就是深度优先森林本身，没有环。

再看报告的情形。考察第一次报告时的边 $e=\{u,w\}$，此时 u 在栈顶、处于处理中，下面说明 w 也在栈中。假设 w 已完成，那么 w 在完成之前已经从自己一端检查过 e。若那时 u 未发现，w 会经 e 发现 u，u 成为 w 的子节点并先于 w 完成，与 u 此刻仍在处理中相矛盾。若那时 u 已发现，那次检查本身就该报告，除非 e 是 w 的父边；而 e 是 w 的父边意味着 u 当初正是读 e 这条记录发现了 w，每条记录只读一次，u 不会现在再读到它。两种情形都不可能，所以 w 在栈中，是位于 u 下方的祖先。

于是父指针链从 w 到 u 再加上 e 构成一个环。在简单图中，若 w 是 u 的父节点，父边只能是 e 本身，而 e 不是 u 的父边；所以这条父指针链至少有两条边，环至少有三个顶点。算法因此既给出“有环”的结论，也给出可以直接核对的环本身。

2.3 连通结构的计算

第 1.3节按相互可达把顶点划分为连通分量或强连通分量，那是存在性的定义。本节把它们算出来，处理三个相关的问题：无向图的连通分量，边不断加入时无向图的连通分量，以及有向图的强连通分量。三个问题的任务相同，都是给每个顶点 v 打上一个分量标签 $\mathrm{comp}(v)$，使两个顶点标签相同当且仅当它们属于同一个分量。有了标签，“u 与 v 是否连通”就变成比较两个数是否相等，不必再去找路径。

三种做法都从遍历出发，差别来自图的形态。无向图上可达关系是对称的，一次遍历恰好找出一个分量，只需在外面套一层扫描；边逐条加入时，分量只会合并，需要一种能快速合并标签的数据结构；有向图上从一个顶点能到达的范围往往大于与它相互可达的范围，一次遍历不再恰好停在分量边界上，需要借助深度优先遍历的完成次序。

2.3.1 无向图的连通分量

2.3.1.1 问题与思路

输入是无向图 $G=(V,E)$ 的邻接表。要输出每个顶点的分量标签 $\mathrm{comp}(v)\in\{1,\dots,c\}$ 和分量个数 c，使得 $\mathrm{comp}(u)=\mathrm{comp}(v)$ 当且仅当 u 与 v 之间有路径。

从顶点 r 出发做一次广度或深度优先遍历，找到的是从 r 可达的全部顶点。无向图中可达关系是对称的，所以这些顶点恰好就是与 r 在同一分量的顶点，不多也不少。给它们打上同一个新标签，一个分量就处理完了。剩下的顶点里若还有没打标签的，它一定属于另一个分量，再从它出发遍历一次。于是在外面套一层扫描，依次从尚未打标签的顶点启动遍历，每启动一次就得到一个分量。

2.3.1.2 算法实现

算法在所用遍历的状态之外，只多维护两项：

- 标签数组 $\mathrm{comp}(\cdot)$：每个顶点所属分量的编号，0 表示尚未标记；它同时充当遍历的发现标记，$\mathrm{comp}(v)\neq 0$ 就表示 v 已发现；
- 已用的标签个数 c。

初始时 comp 全为 0，$c=0$。外层扫描每遇到一个 $\mathrm{comp}(r)=0$ 的顶点，就令 $c\leftarrow c+1$ 并从 r 启动遍历；遍历中每发现一个新顶点 v，就写入 $\mathrm{comp}(v)=c$。

算法 2.3　无向连通分量编号

输入：无向图的邻接表。输出：每个顶点的分量编号 $\mathrm{comp}(v)$，以及分量数 c。

1. 所有顶点置 $\mathrm{comp}(v)=0$，$c=0$。
2. 按任意固定次序扫描顶点。遇到 $\mathrm{comp}(r)=0$ 的顶点 r 时，令 $c\leftarrow c+1$，从 r 启动一次广度（或深度）优先遍历，把这次新发现的每个顶点 v 标记为 $\mathrm{comp}(v)=c$。已经标记的顶点不再作为起点，也不清除旧编号。
3. 扫描完全部顶点后输出。

执行过程　图 2.7中的图由贯穿四点图、一条边 $\{5,6\}$ 和孤立点 7 组成。按编号扫描、每次启动广度优先遍历，标签数组的变化如表 2.1所示。扫描到 1 时它尚未标记，令 $c=1$，从 1 出发的遍历标出 $\{1,2,3,4\}$；扫描到 $2,3,4$ 时它们已有标签，跳过；扫描到 5 时令 $c=2$，遍历标出 $\{5,6\}$；扫描到 6 跳过；最后 7 没有任何邻居，但外层扫描仍会遇到它，令 $c=3$，它自成一个分量。

表 2.1 按编号扫描时标签数组 comp 的变化。只列出启动遍历的三次扫描，其余扫描遇到已标记的顶点而跳过；“0”表示尚未标记。

扫描到的起点	c	1	2	3	4	5	6	7
初始	0	0	0	0	0	0	0	0
1	1	1	1	1	1	0	0	0
5	2	1	1	1	1	2	2	0
7	3	1	1	1	1	2	2	3

2.3.1.3 算法分析

正确性 每次启动的遍历恰好标出一个完整的分量。由第 2.1节的分析，从 r 出发的遍历找到从 r 可达的全部顶点；无向图中可达就是同在一个分量，所以这次遍历标出的恰是 r 所在的分量。它不会碰到先前已标记的顶点，因为那些顶点属于别的分量，与 r 之间没有路径。

外层扫描必须覆盖全部顶点，否则没有边的孤立点不会被任何遍历发现，也就得不到标签。改变扫描次序，例如先扫描 7，标签的数值会变，哪些顶点被分到一起却不会变，因为分量本身由图决定。

复杂度 各次遍历访问的顶点互不重叠，合起来每个顶点被发现一次，每条邻接记录被读常数次；外层扫描本身读 n 个标签。总时间为 $O(n+m)$，标签数组和遍历所需的队列或栈占 $O(n)$ 额外空间。给全图所有顶点打标签，与从一个起点做一次遍历同阶。

2.3.1.4 生成森林

标签只说明哪些顶点连在一起，遍历过程还顺带留下了它们连在一起的证据。每个新顶点都是经某条边第一次被发现的；把这些父边 $\{p(v), v\}$ 全部保留，遇到已发现顶点的边一律不加，就得到原图的一个生成森林，不需要额外计算。

定理 2.5 发现父边构成生成森林 设对无向图 $\mathcal{G}$ 执行算法 2.3，并保留所有父边。则这些父边与全部顶点组成 $\mathcal{G}$ 的一个生成森林，它在每个分量上是一棵生成树，边数为 $n-c$。

证明 每个非起点顶点恰有一条父边，父边的另一端是更早发现的顶点；起点没有父边。于是父边共 $n-c$ 条。

再说明父边不成环。假设某些父边构成一个环，取环上最晚发现的顶点 v。v 在环上有两条边，另两端都比 v 早发现。但与 v 相连的

父边中，只有 v 自己的父边通向更早发现的顶点，其余父边都是 v 的子顶点的父边，另一端比 v 晚发现。矛盾。

最后，每次遍历从起点出发，沿父指针可以从任何新发现的顶点回到起点，所以每个分量内的父边是连通的。连通、无环、覆盖分量的全部顶点，就是该分量的生成树。 ■

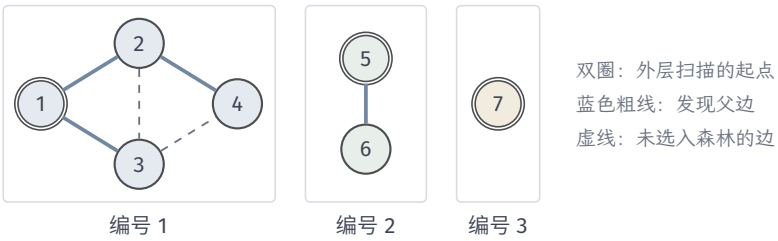


图 2.7　外层扫描与生成森林。图由贯穿四点图、边 $\{5,6\}$ 和孤立点 7 组成，按编号扫描、每次启动广度优先遍历。三个分量分别获得编号 $1,2,3$；双圈是每次遍历的起点，蓝色粗线是保留的父边，共 $4=7-3$ 条，虚线边被检查过但未选入森林。

生成森林可以用来检验输出：它应有 $n-c$ 条边、不含环，并且同一棵树上的顶点标签相同。它只保留了谁和谁连通，不保留其他结构。图 2.7中第一个分量保留了 $\{1,2\}$、$\{1,3\}$、$\{2,4\}$，舍弃了 $\{2,3\}$ 和 $\{3,4\}$。原图中 3 与 4 直接相邻，在森林中却要走 $3\to1\to2\to4$ 三步；原图没有割点，而在森林中删去 1 或 2 都会把树断开。边上若有权重，算法选出的边也没有任何总权最小的保证；在所有生成树中选总权最小的一棵，是第 4章的问题。

2.3.2　增量无向图的连通分量

2.3.2.1　问题与思路

现在顶点集固定，边一条一条地加入，每加入一条边之后都可能被问到“两个给定顶点现在是否连通”。用上一小节的方法，每次加边后重新给全图打标签需要 $O(n+m)$ 时间，加入 m 条边共要 $O(m(n+m))$，太浪费。我们希望每次加边和每次查询都只花很少的时间。

加入边 $\{u,v\}$ 对分量的影响很简单：若 u,v 原来已在同一分量，分量不变；否则把两个分量合并成一个。分量构成顶点集的一个划分，加边只会合并其中的两块，不会拆分。所以只需一种数据结构，维护若干互不相交的集合，支持两种操作：$\mathrm{find}(v)$ 返回 v 所在集合的代表元，它扮演分量标签的角色，两顶点连通当且仅当代表元相同；$\mathrm{union}(u,v)$ 把 u 和 v 所在的两个集合合并。这种数据结构称为**并查集**（union-find），也称不相交集合结构。

最直接的实现是让每个顶点直接存自己的标签。查询只需 $O(1)$，但合并两个分量时，要把其中一个分量的所有顶点逐个改写标签，一次合并可

能改写 $O(n)$ 个顶点。改进的办法是不立即改写，而把每个集合组织成一棵有根树：合并时只改一个指针，查询时再顺着指针找到代表元。

2.3.2.2 算法实现

并查集的数据结构 并查集维护两项状态：

- 父指针 parent(v)：每个元素 v 在所属集合的树中的父元素；根的父指针指向自己，根就是集合的代表元；
- 集合大小 size(r)：只对根 r 有意义，记录这棵树中的元素个数。

初始时没有任何边，每个元素自成一个集合：parent(v) = v，size(v) = 1。图 2.8画出了某一时刻的状态：两棵树对应两个集合，实际存储的只是右边两个长度为 n 的数组。find(4) 读 parent(4) = 3，再读 parent(3) = 1，到达指向自己的 1，于是返回根 1。合并两个集合只需把一个根的父指针改为另一个根，并更新新根的大小。

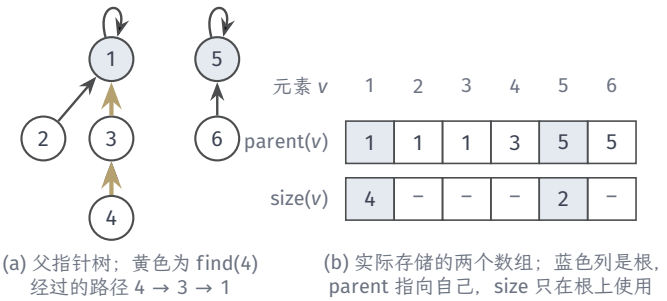


图 2.8 并查集的数据结构。元素 $1,\dots,6$ 分为集合 $\{1,2,3,4\}$ 与 $\{5,6\}$。(a) 每个集合是一棵父指针树，箭头由元素指向父指针，根（蓝色）指向自己；find(4) 沿黄色路径 $4 \to 3 \to 1$ 找到根。(b) 计算机中实际存储的是 parent 与 size 两个数组，size 只在根上使用。

合并时若随意选择谁接到谁下面，树可能退化成一条长链，查询要走很多步。下面的算法用两条规则避免这一点：合并时把小树接到大树下（按大小合并），查询时把经过的顶点直接接到根上（路径压缩）。

算法 2.4 并查集的两种操作

状态：每个元素 v 的父指针 parent(v)，每个根 r 所在集合的大小 size(r)。初始时每个元素自成一个集合，parent(v) = v，size(v) = 1。

1. find(v)：沿父指针从 v 走到根 r；然后把这条路上经过的每个元素的父指针都直接改为 r（路径压缩）；返回 r。
2. union(u, v)：求 r_u = find(u)、r_v = find(v)。若 $r_u = r_v$，不做任何事。否则把较小集合的根接到较大集合的根下，大小相同时把 r_v 接到 r_u 下（按大小合并），并把新根的大小更新为两者之和。

增量连通分量的实现　有了并查集，增量连通分量的维护就很直接。开始时顶点集上还没有边，为 V 中每个顶点建立一个单点集合，此时每个顶点自成一个分量。之后按到达次序处理两类请求：加入一条边 $\{u, v\}$ 时调用一次 union(u, v)；询问 u, v 是否连通时，比较 find(u) 与 find(v) 是否相等。需要给每个顶点输出分量标签时，取 comp(v) = find(v) 即可。

这里的父指针只表示“属于同一集合”，与原图的边没有关系：合并 $\{1, 3\}$ 时，可能是 3 的根被接到 1 的根下，而原图中这两个根之间未必有边。需要输出原图中的连接时，应另外保存实际加入的边。

执行过程　图 2.9按边序列 $\{1, 2\}$、$\{3, 4\}$、$\{1, 3\}$、$\{5, 6\}$、$\{4, 6\}$ 追踪这些操作，元素为 $1, \ldots, 6$。初始时六个元素各自为根。前两次合并各把一个单点接到另一个单点下：parent(2) = 1、parent(4) = 3，得到 $\{1, 2\}$ 与 $\{3, 4\}$，此时还有单点集合 $\{5\}$、$\{6\}$。

合并 $\{1, 3\}$ 时两个集合大小都是 2，按约定把根 3 接到根 1 下，size(1) = 4，4 的深度变为 2。之后若查询 find(4)，沿 $4 \to 3 \to 1$ 找到根 1，路径压缩把 4 的父指针直接改为 1，下次查询只需一步。

合并 $\{5, 6\}$ 得到大小为 2 的集合；处理 $\{4, 6\}$ 时，4 所在集合大小为 4、6 所在集合大小为 2，于是根 5 接到根 1 下，全部六个元素属于同一集合。每一步后的集合划分都与“已加入的边构成的图的连通分量”一致。

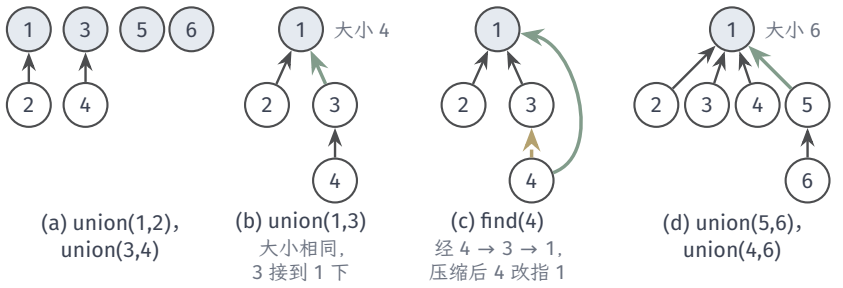


图 2.9　并查集的父指针树，箭头由元素指向其父指针，蓝色填充为根。(a) 合并 $\{1, 2\}$、$\{3, 4\}$ 后；(b) 合并两个大小为 2 的集合，按约定把根 3 接到根 1 下（绿色箭头）；(c) 查询 find(4) 经过 $4 \to 3 \to 1$（黄色虚线是原父指针），路径压缩后 4 直接指向根 1；(d) 合并 $\{5, 6\}$ 后再处理 $\{4, 6\}$，较小集合的根 5 接到较大集合的根 1 下。

2.3.2.3 算法分析

正确性　并查集的正确性来自一个保持不变的性质：在任何时刻，并查集中的集合恰好是已加入的边构成的图的连通分量。初始时没有边，每个顶点自成分量，集合也都是单点。加入边 $\{u, v\}$ 时，若 u, v 已在同一集合，分量不变，算法也不做任何事；否则两个分量合并，算法恰好把对应的两个集合合并。路径压缩只改变树的形状，不改变哪些元素在同一棵树中。

复杂度 查询的代价取决于树高，而按大小合并保证树不会太高。

> **定理 2.6 按大小合并的树高** 只使用按大小合并（无论是否路径压缩）时，任一元素到根的路径长度不超过 $\log_2 n$，其中 n 是元素总数。

> **证明** 先不考虑路径压缩。一个元素的深度只在它所在的根被接到另一个根下面时增加 1；按规则，被接的那一侧不大于另一侧，所以合并后该元素所在集合的大小至少是原来的两倍。集合大小从 1 开始，不超过 n，至多翻倍 $\log_2 n$ 次，深度也就至多为 $\log_2 n$。路径压缩只把元素的父指针改为它的某个祖先，只会让深度变小，不影响这个上界。 ■

由定理 2.6，每次 find 和 union 都只需 $O(\log n)$ 时间，存储为 $O(n)$。路径压缩使反复查询同一区域变得更快。可以证明，两条规则同时使用时，任意 k 次操作的总时间为 $O(k\,\alpha(n))$，其中 α 是增长极慢的反 Ackermann 函数，对任何实际规模的 n 都不超过 4；这一摊还分析较长，这里不展开，使用时只需记住每次操作的平均代价几乎是常数。与每次加边后重新遍历的 $O(n+m)$ 相比，差距随图的规模迅速拉大。

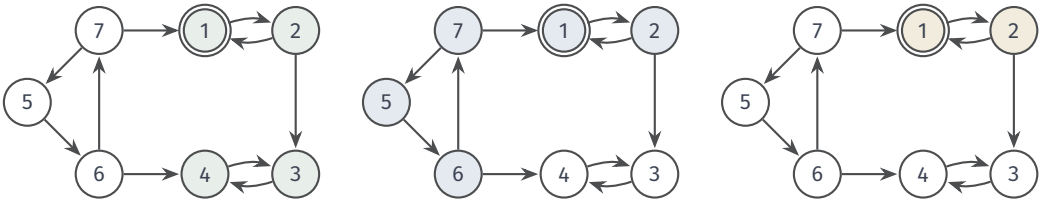
适用范围 并查集适合“只加边、不删边”的情形。第 4章的 Kruskal 算法按边权从小到大考虑每条边，需要判断加入这条边是否会成环，也就是判断两个端点是否已在同一分量，正好用并查集完成。删除一条边可能把一个分量拆成两个，而并查集的合并无法撤销，所以它不能直接维护任意删边后的分量。在图像中给相连的像素区域编号时，也常先给像素临时标号、再用并查集合并相遇的标号，具体扫描方式不在本卷讨论。

2.3.3 有向图的强连通分量

2.3.3.1 问题与思路

输入是有向图 $G=(V,E)$ 的出邻接表，要给每个顶点打上强分量标签 $\mathrm{comp}(v)$，使得 $\mathrm{comp}(u)=\mathrm{comp}(v)$ 当且仅当 u 与 v 相互可达。照搬无向图的做法，从 r 出发遍历一次，找到的是 r 能到达的全部顶点，而强分量要求双向可达。图 2.10(a) 中，从 1 出发能到达 $1,2,3,4$，但 $3,4$ 回不到 1，1 所在的强分量只是 $\{1,2\}$。

第 1.3.2小节已经说明，v 所在的强分量等于“从 v 可达的顶点”与“能到达 v 的顶点”的交集，图 2.10在本例上画出了这两个集合。后者可以在把所有弧反向后的图上从 v 遍历得到。但对每个顶点都这样做两次遍历，需要 $O(n(n+m))$ 时间。



(a) 前向可达集　(b) 后向可达集　(c) 1 所在的强连通分量

图 2.10　强连通分量是前向可达集与后向可达集的交。图中弧为 1 → 2、2 → 1、2 → 3、3 → 4、4 → 3、5 → 6、6 → 4、6 → 7、7 → 1、7 → 5，双圈为顶点 1。(a) 从 1 出发可达 $\{1,2,3,4\}$；(b) 能到达 1 的顶点为 $\{1,2,5,6,7\}$；(c) 两者的交 $\{1,2\}$ 就是 1 所在的强连通分量。

更快的办法只需两遍遍历，关键是在反向图上选对起点。把有向图 $\mathcal{G}$ 的每条弧 $u \to v$ 改为 $v \to u$，得到**反向图**（reverse graph）$\mathcal{G}^{\mathrm{rev}}$。$\mathcal{G}$ 与 $\mathcal{G}^{\mathrm{rev}}$ 的强连通分量相同，因为相互可达的两个顶点在弧全部反向后仍然相互可达。在反向图上从 v 遍历，找到的是原图中能到达 v 的顶点，它包含 v 的强分量，也可能越出去。越出去的部分都是原图中位于 v 的分量“上游”的分量。

关键在于起点的顺序。如果每次都从一个“没有未处理的上游分量”的分量中选起点，在反向图上的遍历就只能走到已经处理过的分量或者本分量内部；只要已处理的顶点不再进入，这次遍历就恰好停在本分量的边界上。这个顺序由第一遍深度优先遍历的完成时刻提供：完成时刻最大的顶点，位于最上游的分量中。这一方法称为**Kosaraju 算法**（Kosaraju's algorithm）。

2.3.3.2 算法实现

算法开始前，先扫描一遍出邻接表，构造反向图 $\mathcal{G}^{\mathrm{rev}}$ 的邻接表，也就是原图的入邻接表。它只是第二遍要读取的输入，在算法运行中不再改变。

此后分两遍进行，各自维护的状态是：

- 第一遍：深度优先遍历的全部状态（第 2.2.2小节），另加一张完成次序表，按完成的先后记下顶点；
- 第二遍：标签数组 $\mathrm{comp}(\cdot)$，初始全为 0；已用的标签个数 c，初始为 0。

第一遍在原图上做完整的深度优先遍历，填好完成次序表。第二遍按完成时刻从大到小扫描顶点，遇到未标记的顶点 r，令 $c \leftarrow c+1$，在反向图上从 r 遍历，只进入未标记的顶点，把到达的顶点标为 c。

算法 2.5 Kosaraju 强连通分量算法

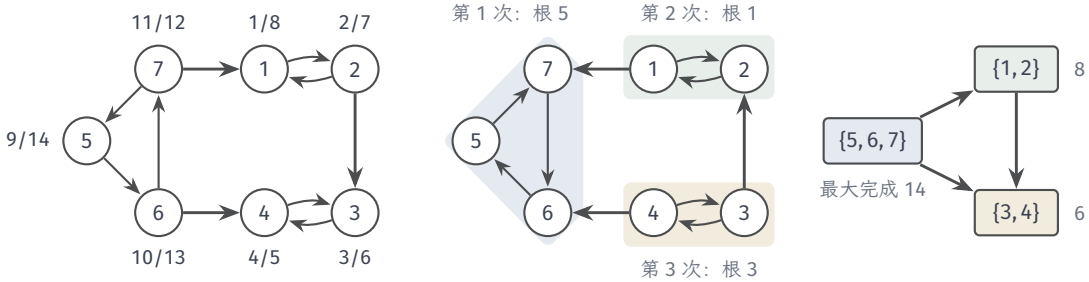
输入：有向图 $G=(V,E)$ 的出邻接表。输出：每个顶点所属强连通分量的编号。

1. 扫描出邻接表，构造反向图 G^{rev} 的出邻接表，即原图的入邻接表。
2. 在 G 上执行完整的深度优先遍历（算法 2.2），覆盖全部顶点，记录完成次序。
3. 所有顶点置为未标记。按完成时刻从大到小扫描顶点；遇到未标记的顶点 r 时，开始一个新分量，在 G^{rev} 上从 r 遍历（广度或深度优先均可），只进入未标记的顶点，把这次到达的顶点全部标为这个分量。

执行过程 仍用图 2.10中的有向图。它的强连通分量是 $\{5,6,7\}$、$\{1,2\}$ 和 $\{3,4\}$，分量之间只有单向的弧。

第一遍按编号选择起点、按编号读取出邻居。从 1 出发，依次发现 $1,2,3,4$；4 的出邻居 3 在栈中，4 完成，随后 $3,2,1$ 依次完成，完成时刻为 $f(4)=5$、$f(3)=6$、$f(2)=7$、$f(1)=8$。接着从 5 重新出发，发现 $5,6$；6 的出邻居 4 已完成，跳过，再发现 7；7 的出邻居 1 已完成、5 在栈中，于是 $7,6,5$ 依次完成，$f(7)=12$、$f(6)=13$、$f(5)=14$。完成次序为 $4,3,2,1,7,6,5$，见图 2.11(a)。

第二遍按完成时刻从大到小，即 $5,6,7,1,2,3,4$ 选择起点，在反向图上遍历，见图 2.11(b)。从 5 出发，令 $c=1$，反向弧把它引向 7，再引向 6；$\{5,6,7\}$ 中没有反向弧通往别处，于是 $\text{comp}(5)=\text{comp}(6)=\text{comp}(7)=1$。扫描到 $6,7$ 时已标记，跳过。下一个未标记的是 1，令 $c=2$：反向弧 $1\to 2$、$1\to 7$ 中 7 已标记，只进入 2，得到 $\{1,2\}$。最后从 3 出发，令 $c=3$，反向弧 $3\to 2$ 通往已标记的 2，只进入 4，得到 $\{3,4\}$。每次遍历恰好停在一个强分量的边界上。



(a) 第一遍：原图上的发现/完成时刻　(b) 第二遍：反向图，按 5,6,7,1,2,3,4 启动　(c) 缩点图

图 2.11 Kosaraju 算法的两遍过程与缩点图。(a) 原图上按编号重启的深度优先遍历，顶点旁为发现/完成时刻；粗箭头是连接不同强分量的弧。(b) 反向图上按完成时刻从大到小（$5,6,7,1,2,3,4$）选择起点，三次遍历分别得到蓝、绿、黄三个强连通分量，指向已标记分量的反向弧不再进入。(c) 把每个强分量缩成一点得到的缩点图，旁注为各分量中最大的完成时刻；缩点图无环，且最大完成时刻沿弧方向递减。

2.3.3.3 算法分析

正确性　第二遍之所以不会越界，是因为第一遍的完成时刻在分量之间有固定的先后。对强连通分量 C，记 $f(C) = \max_{v \in C} f(v)$ 为其中最大的完成时刻。在上例中，$f(\{5,6,7\}) = 14$、$f(\{1,2\}) = 8$、$f(\{3,4\}) = 6$，而分量之间的弧都从 f 值大的指向 f 值小的，即从上游指向下游。

引理 2.7　分量间的完成时刻　设 $C \neq C'$ 是有向图的两个强连通分量，且存在一条从 C 中顶点指向 C' 中顶点的弧。则在一次完整的深度优先遍历中，$f(C) > f(C')$。

证明　考察 $C \cup C'$ 中最先被发现的顶点 x。

若 $x \in C$。在时刻 $d(x)$，$C \cup C'$ 中其余顶点都未发现。C 中每个顶点都能从 x 经 C 内部的路径到达；C' 中每个顶点都能先在 C 内走到那条弧的起点，经弧进入 C'，再在 C' 内部到达。这些路径上的顶点此时都未发现，由引理 2.3，$C \cup C'$ 中其余顶点都成为 x 的后代并先于 x 完成，所以 $f(C) = f(x) > f(C')$。

若 $x \in C'$。同理，C' 中其余顶点都成为 x 的后代，$f(C') = f(x)$。另一方面，从 C' 中的任何顶点都不能到达 C：否则结合已有的从 C 到 C' 的弧，C 与 C' 中的顶点相互可达，它们就应属于同一个强分量。所以在 x 处于处理中期间，C 中没有顶点被发现（那样的顶点会是 x 的后代，从而从 x 可达）。C 中顶点都在 $f(x)$ 之后才被发现，也就在 $f(x)$ 之后完成，所以 $f(C) > f(x) = f(C')$。 ■

在反向图中，这个不等式的方向随弧一起反转：从分量 C 出发的反向弧只能通往 f 值更大的分量。第二遍按 f 值从大到小选择起点，这些分量在轮到 C 之前都已标记，于是遍历被挡在 C 之内。

定理 2.8　Kosaraju 算法的正确性　算法 2.5第三步的每一次遍历，恰好标记一个强连通分量的全部顶点。

证明　对第二遍已经完成的遍历次数归纳，假设此前每次遍历都恰好标记了一个完整的强分量。设下一个起点是 r，所在分量为 C。r 是未标记顶点中完成时刻最大的；由归纳假设，未标记的顶点恰好组成若干个完整的强分量，所以 $f(r) = f(C)$，且 C 在未标记的分量中 f 值最大。

> C 在反向图中仍然强连通，从 r 出发能到达 C 的全部顶点，而 C 中的顶点都未标记，因此都会被这次遍历标记。另一方面，设反向图中有一条弧从 C 中顶点指向另一个分量 C' 中的顶点，它对应原图中一条从 C' 指向 C 的弧，由引理 2.7有 $f(C') > f(C)$。而 C 是未标记分量中 f 值最大的，所以 C' 已经被标记，这次遍历不会进入。因此这次遍历标记的恰好是 C。 ■

这一论证依赖算法的三处细节，缺一不可：第一遍必须覆盖全部顶点，否则有些顶点没有完成时刻；第二遍必须在反向图上进行，若在原图上从 5 遍历，会经 $6 \to 4$、$7 \to 1$ 走进另外两个分量；起点必须按完成时刻从大到小选择，若改为从小到大，上例中第二遍会先从 4 出发，在反向图上经 $4 \to 6$ 进入 $\{5,6,7\}$，再经 $3 \to 2$ 进入 $\{1,2\}$，把三个分量全部混在一起。

复杂度 第一遍深度优先遍历用 $O(n+m)$ 时间。构造反向图需要扫描每条弧一次，时间与额外存储都是 $O(n+m)$。第二遍按完成时刻从大到小扫描，只需把第一遍的完成次序倒过来读，不必排序；遍历本身再用 $O(n+m)$ 时间。总时间为 $O(n+m)$，与无向图的分量编号同阶，远小于逐点求两个可达集的 $O(n(n+m))$。

2.3.3.4 缩点图

得到强连通分量后，可以把每个分量当作一个整体，压缩为一个顶点。若原图中有从分量 C 指向另一分量 C' 的弧，就在 C 与 C' 之间连一条弧 $C \to C'$（多条这样的弧只保留一条，分量内部的弧舍去），得到的有向图称为**缩点图**（condensation graph）。图 2.11(c) 中，缩点图有三个顶点和三条弧 $\{5,6,7\} \to \{1,2\}$、$\{1,2\} \to \{3,4\}$、$\{5,6,7\} \to \{3,4\}$。

定理 2.9 缩点图无环 任何有限有向图的缩点图都是有向无环图。

> **证明** 假设缩点图中有有向环 $C_1 \to C_2 \to \cdots \to C_k \to C_1$，$k \geq 2$，各 C_i 是不同的强分量。每条弧 $C_i \to C_{i+1}$ 对应原图中一条从 C_i 中某顶点到 C_{i+1} 中某顶点的弧，而每个分量内部任意两点相互可达。于是从 C_1 中任一顶点出发，可以依次经过 $C_2, \ldots, C_k$ 再回到 C_1，环上所有分量中的顶点两两相互可达，它们应属于同一个强分量，与 $C_1, \ldots, C_k$ 互不相同矛盾。 ■

由引理 2.7，缩点图中每条弧都从 f 值较大的分量指向 f 值较小的分量，所以 Kosaraju 算法第二遍得到分量的先后次序，恰好使缩点图的每条弧从先得到的分量指向后得到的分量。上例中得到的次序 $\{5,6,7\}$、$\{1,2\}$、$\{3,4\}$

就满足这一点。任何有向图都可以先缩点、再在无环的缩点图上安排先后，这正是下一节要研究的依赖排序。

2.4 拓扑排序

编译程序时，一个模块要等它引用的模块编译完才能编译；计算一组中间量时，一个量要等它依赖的量算出来才能计算。用有向图表示这类依赖，弧 $u \to v$ 表示 u 必须先于 v 处理，问题就是给所有顶点排出一个满足全部依赖的次序。

这样的次序只在无环图上存在；构造它有两种方法，Kahn 算法逐个取出没有依赖的顶点，基于深度优先遍历的方法反转完成次序。两种方法遇到有向环时都报告失败，而不是输出一个不完整的序列。

2.4.1 问题与存在条件

把依赖次序写成顶点的排列，要求就是每条弧的起点排在终点之前。

定义 2.10　拓扑序　有向图 $G = (V, E)$ 的**拓扑序**（topological order）是 V 中全部顶点的一个排列 $v_1, v_2, \ldots, v_n$，使得对每条弧 $v_i \to v_j$ 都有 $i < j$。求拓扑序的过程称为拓扑排序。

拓扑序只要求每条弧前后正确，没有弧相连的两个顶点谁先谁后都可以，因此拓扑序一般不唯一。检验一个输出是否正确，标准是逐条弧核对先后，而不是与某一个特定答案比较。

并非每个有向图都有拓扑序。若图中有有向环 $v_1 \to v_2 \to \cdots \to v_k \to v_1$，排列既要让 v_1 在 v_2 之前、v_2 在 v_3 之前……v_{k-1} 在 v_k 之前，又要让 v_k 在 v_1 之前，这不可能。所以有拓扑序的图一定没有有向环。

反过来，无环时可以先排没有依赖的顶点。排在第一位的顶点不能有任何前驱，也就是入度为 0。把它排好后，它对后继的约束就都满足了，可以把它和它的出弧从图中删去，在剩下的图上重复同样的事。这个想法能否一直进行下去，取决于每一步是否总能找到入度为 0 的顶点。

引理 2.11　有向无环图有源点　非空的有限有向无环图至少有一个入度为 0 的顶点。

证明 假设每个顶点的入度都至少为 1。从任一顶点 v_0 出发，取它的一个前驱 v_1，即有弧 $v_1 \to v_0$；再取 v_1 的一个前驱 v_2，如此继续。顶点只有有限个，这个序列中必有某个顶点第二次出现，设 $v_i = v_j$，$i < j$。则 $v_j \to v_{j-1} \to \cdots \to v_i$ 是一个有向环，与无环矛盾。 ■

定理 2.12 拓扑序的存在性 有限有向图存在拓扑序，当且仅当它是有向无环图。

证明 必要性已在上面说明。充分性对顶点数 n 归纳。$n = 0$ 时空排列即为拓扑序。设 $n \geq 1$，由引理 2.11取一个入度为 0 的顶点 v_1。删去 v_1 及其出弧，剩下的是原图的诱导子图，删去顶点不会产生新的环，它仍是有向无环图，由归纳假设有拓扑序 $v_2, \ldots, v_n$。在前面补上 v_1：以 v_1 为起点的弧终点都在后面；v_1 没有入弧；其余弧都在剩下的图中，已经满足要求。所以 $v_1, v_2, \ldots, v_n$ 是原图的拓扑序。 ■

这个证明本身就是一个构造：不断取出入度为 0 的顶点。

2.4.2 Kahn 算法

照上一小节的构造真的从图中删去顶点，代价很大，而且每一步还要重新寻找入度为 0 的顶点。实际上只需维护每个顶点在“尚未输出的顶点构成的子图”中的当前入度：输出一个顶点 u 时，它的每条出弧 $u \to v$ 不再计入，把 v 的当前入度减一。哪个顶点的当前入度刚降到 0，它就可以输出了，把它放进一个等待队列。这一方法称为**Kahn 算法**（Kahn's algorithm）。

算法维护三项状态：每个顶点的当前入度 $\mathrm{indeg}(v)$；一个队列 Q，存放当前入度为 0、尚未输出的顶点；输出序列 L。初始化时扫描全部弧统计入度，把所有入度为 0 的顶点（包括孤立点）放入 Q，L 为空。此后每一轮从 Q 取出一个顶点 u，追加到 L 的末尾，再检查 u 的每条出弧 $u \to v$，令 $\mathrm{indeg}(v) \leftarrow \mathrm{indeg}(v) - 1$，若它恰好降为 0，把 v 放入 Q。Q 为空时停止。

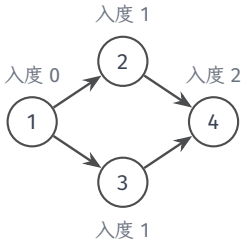
算法 2.6 Kahn 拓扑排序

输入：有向图 $G = (V, E)$ 的出邻接表。输出：一个拓扑序，或报告图中有有向环。

1. 扫描全部弧，统计每个顶点的入度 $\mathrm{indeg}(v)$。把所有入度为 0 的顶点（包括孤立点）放入队列 Q，输出序列 L 置空。
2. 当 Q 非空时，取出队首 u，追加到 L 末尾。对每条出弧 $u \to v$，令 $\mathrm{indeg}(v) \leftarrow \mathrm{indeg}(v) - 1$；若它恰好降为 0，把 v 放入 Q。
3. Q 为空时停止。若 L 含全部 n 个顶点，输出 L；否则报告有有向环。

一个顶点只在当前入度刚降到 0 的那一刻入队，因此不会重复入队。判断必须用当前入度，而不是初始入度：初始入度为 1 的顶点，在它唯一的前驱输出之前不能输出。

执行过程　考虑弧为 $1 \to 2$、$1 \to 3$、$2 \to 4$、$3 \to 4$ 的菱形依赖：任务 2, 3 都依赖 1，任务 4 同时依赖 2 和 3。初始入度为 $(0, 1, 1, 2)$，只有 1 入队，$Q = [1]$。图 2.12(b) 列出了每次输出后的变化。输出 1 后，2, 3 的入度都降为 0，两者入队，$Q = [2, 3]$；输出 2 后，4 的入度从 2 降为 1，还不能入队，$Q = [3]$；输出 3 后，4 的入度降为 0，入队，$Q = [4]$；最后输出 4，Q 为空。$L = 1, 2, 3, 4$ 含全部顶点，是一个拓扑序。



(a) 菱形依赖图

各顶点的当前入度

输出	1	2	3	4	零入度队列
初始	0	1	1	2	[1]
1	–	0	0	2	[2, 3]
2	–	–	0	1	[3]
3	–	–	–	0	[4]
4	–	–	–	–	[ ]

(b) 每次输出后的入度与队列

图 2.12　Kahn 算法在菱形依赖图上的执行。(a) 依赖图及各顶点的初始入度。(b) 每输出一个顶点后各顶点的当前入度（“–”表示已输出）与零入度队列；绿色底纹标出刚降为 0、随即入队的入度。顶点 4 要等两个前驱都输出后才入队。

输出 1 之后，2 和 3 同时位于队列中，这表示两项任务可以同时执行，或以任意先后执行。若先取 3，就得到同样合法的 $1, 3, 2, 4$。队列中同时有多个零入度顶点时选哪一个，决定了得到哪个拓扑序。

正确性　先看无环的情形。尚未输出的顶点构成原图的诱导子图，仍是有向无环图；只要它非空，由引理 2.11就有当前入度为 0 的顶点，而这样的顶点在入度降为 0 时已经入队。因此队列在全部顶点输出之前不会变空，L 最终含全部 n 个顶点。对每条弧 $u \to v$，v 的当前入度要等 u 输出后才扣除这条弧的贡献，所以 v 只能在 u 之后入队、之后输出，L 满足拓扑序的要求。

再看有环的情形。环上每个顶点在环上都有一个前驱，环上第一个被输出的顶点在输出之前当前入度至少为 1，不可能入队，所以环上的顶点一个也不会输出，L 少于 n 个顶点，算法报告有环。这时已经输出的部分只是一个前缀，不能当作完整的拓扑序。若需要给出环本身，可以在剩下的顶点中重复引理 2.11证明中的“不断取前驱”，必然在有限步内回到已经走过的顶点，找到一个有向环。

复杂度 统计入度要扫描每条弧一次；之后每个顶点入队、出队各一次，每条弧在其起点输出时被扣减一次。总时间为 $O(n+m)$，入度数组、队列与输出序列占 $O(n)$ 额外空间。若要求结果可复现，例如在多个零入度顶点中总是选编号最小的，就要把普通队列换成优先队列，每次入队、出队需要 $O(\log n)$ 时间，总时间变为 $O(m+n\log n)$。

2.4.3 基于 DFS 的拓扑排序

第 2.2.3小节看到，深度优先遍历中一个顶点的后代都先于它完成。在依赖图中，从 u 出发能走到的顶点都排在 u 之后；若它们都比 u 先完成，那么把完成次序反转，u 就排在它们前面。所以做一次深度优先遍历，按完成时刻从大到小输出顶点即可。被 u 的弧指向、却不是 u 后代的顶点同样先于 u 完成，这由后面的定理保证。

算法的状态就是深度优先遍历的状态（三种状态、栈、时钟），再加一个初始为空的输出序列 L。遍历按算法 2.2进行，更新规则只增加两处：每当一个顶点完成时，把它放到 L 的最前面，遍历结束时 L 就已是逆完成次序；每当读到的出邻居处于处理中，即遇到回边，就沿父指针取出一个有向环并停止，拒绝输出排序。若整次遍历没有遇到回边，输出 L。

执行过程 在同一个菱形依赖图上，从 1 出发、按编号读取出邻居：发现 $1,2,4$；4 没有出邻居，完成，$L=[4]$；回到 2，2 完成，$L=[2,4]$；回到 1 后发现 3，3 的出邻居 4 已完成，不进入，3 完成，$L=[3,2,4]$；最后 1 完成，$L=[1,3,2,4]$。完成次序为 $4,2,3,1$，L 正是它的反转。它与 Kahn 算法给出的 $1,2,3,4$ 不同，但两者都是合法拓扑序：如图 2.13(b)，把顶点按次序从左到右排开，四条弧全部指向右方。

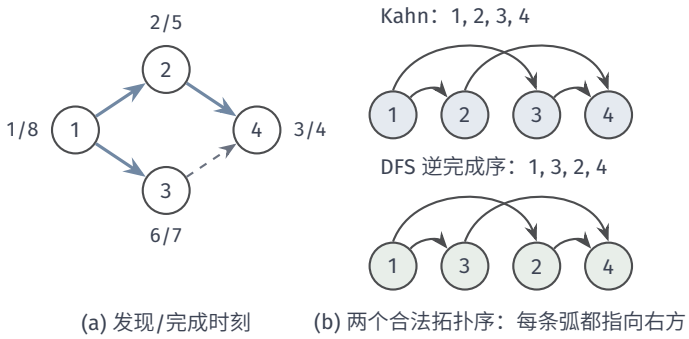


图 2.13 基于深度优先遍历的拓扑排序。(a) 菱形依赖图上从 1 出发的深度优先遍历，旁注为发现/完成时刻，蓝色箭头为父指针对应的弧，虚线弧 $3\to4$ 检查时 4 已完成。(b)Kahn 算法得到的 $1,2,3,4$ 与逆完成次序 $1,3,2,4$，两种排列下每条弧都从左指向右，都是合法拓扑序。

正确性　有环时，由定理 2.4，遍历一定会遇到回边，算法报告环而不输出排序。无环时，下面的定理说明逆完成次序总是拓扑序。

定理 2.13　逆完成次序是拓扑序　对有向无环图执行一次覆盖全部顶点的深度优先遍历，则对每条弧 $u \to v$ 都有 $f(v) < f(u)$。因此按完成时刻从大到小排列顶点，得到一个拓扑序。

证明　考察 u 检查弧 $u \to v$ 的时刻，此时 u 处于处理中，按 v 的状态分三种情况。若 v 未发现，它将以 u 为父指针被发现，由引理 2.2，$f(v) < f(u)$。若 v 已完成，$f(v)$ 早于当前时刻，而 u 尚未完成，同样 $f(v) < f(u)$。若 v 处于处理中，$u \to v$ 是一条回边，由定理 2.4图中有有向环，与假设矛盾，这种情况不会发生。 ■

定理要求遍历覆盖全部顶点。只从一个起点遍历时，得到的只是它可达部分的完成次序。在菱形图中若只从 2 出发，只能到达 $2, 4$，反转后是 $2, 4$，漏掉了 1 和 3。

也不能把各次重启的结果分别反转后简单拼接：先从 2 出发得到 $2, 4$，再从 1 出发得到 $1, 3$，拼成 $2, 4, 1, 3$，违反了弧 $1 \to 2$。正确做法是把整次遍历的完成次序 $4, 2, 3, 1$ 作为一个整体反转，得到 $1, 3, 2, 4$；定理 2.13的证明对跨越不同起点的弧同样成立。按上面“完成时放到最前面”的实现，各次重启共用同一个 L，自然就是整体反转。

把完成的顶点放到 L 最前面，每次只需常数时间（例如用链表，或先追加到末尾、最后整体反转一次），所以总时间就是一次深度优先遍历的 $O(n + m)$，额外空间为 $O(n)$。

2.4.4　两种方法的比较

两种方法的时间都是 $O(n + m)$，在无环图上都输出合法的拓扑序，在有环图上都报告失败，但维护的信息不同。Kahn 算法显式地维护“当前可以执行的任务”这一集合，即零入度队列，适合需要逐步列出可并行任务、或按某种优先规则选择任务的场合。基于深度优先遍历的方法不需要入度数组，在已经要做深度优先遍历、或需要在有环时直接给出环的场合可以复用同一次遍历。

对含环的一般有向图，可以先用第 2.3.3小节的方法求出强连通分量并缩点，再对无环的缩点图做拓扑排序，这样得到的是分量之间的先后。第 3章在有向无环图上计算最短路时，就是按拓扑序逐个处理顶点：处理到一个顶点时，它的所有前驱都已处理完毕。

2.5 基于分隔集的分块

大图上的计算常常希望“分而治之”：把图拆成几块分别处理，再在块与块的连接处汇总。第 1.3节定义了分隔集：删除一个顶点集 S 后，若某些顶点之间不再有路径，S 就把它们分隔开。

本节接受一个给定的候选集 S，检验它是否真的把图拆成了几侧，并把图分成共享接口的局部块。这里只做结构上的分块：怎样找到好的分隔集不在这里讨论，块上的数值怎样沿接口组合放在第 4部分。

2.5.1 问题与思路

输入是无向图 $\mathcal{G}=(V,E)$ 和一个候选集合 $S\subseteq V$。要回答两件事：删除 S 之后，其余顶点分成了几侧，每侧是哪些顶点；以及怎样把原图分成若干块，使每块对应一侧，块与块之间只通过 S 联系，并且原图的每条边都保存在某个块中。是否拆分成功只能由计算结果判断，不能因为输入被称作“分隔集”就默认成立。

办法是先删去 S 求分量，再把 S 加回。删除 S 中的顶点及与它们关联的边，得到 $\mathcal{G}-S$，即 $V\setminus S$ 上的诱导子图。它的连通分量 $C_1,\dots,C_r$ 就是“各侧”：不同的 C_j 之间在 $\mathcal{G}-S$ 中没有边，所以原图中从 C_j 到 C_k（$j\neq k$）的任何路径都必须经过 S。按 r 的取值有三种结果：$r\geq 2$ 时，S 把图拆成了 r 侧；$r=1$ 时，删除 S 后其余顶点仍然连通，S 没有形成多侧拆分；$r=0$ 时 $S=V$，余图为空，也谈不上拆分。

删除 S 的同时，各分量与 S 之间的边、S 内部的边也丢失了。要把原图完整地分成几块，还要把 S 加回每一侧，得到**局部块**（local block）

$$\mathcal{B}_j=\mathcal{G}[C_j\cup S],\qquad j=1,\dots,r,$$

其中 S 称为各块的公共接口。求各侧正是第 2.3.1小节的连通分量问题，只是要把 S 中的顶点当作不存在。

2.5.2 算法实现

实现时不必真的从邻接表中删去 S。算法开始前，用一个布尔数组把 S 中的顶点标记为“屏蔽”；这个数组只是对输入的标注，运行中不再改变。此后的计算就是在未屏蔽的顶点上做第 2.3.1小节的分量编号，维护的状态是：

- 标签数组 comp(·)：未屏蔽顶点所属的分量编号，0 表示尚未标记，初始全为 0；
- 分量个数 r：初始为 0；
- 所用遍历的队列或栈，初始为空。

更新规则与算法 2.3相同，只多了两处跳过：

- 外层扫描：遇到屏蔽顶点或已标记的顶点跳过；遇到未屏蔽且未标记的顶点 v，令 $r \leftarrow r+1$，置 $\mathrm{comp}(v)=r$，从 v 启动遍历。
- 遍历：读到邻居 w 时，若 w 被屏蔽或已标记就不进入；否则置 $\mathrm{comp}(w)=r$，把 w 放入队列或栈。

扫描结束后，标签为 j 的顶点组成 C_j。再按 r 报告结果，并为每个分量列出局部块的顶点 $C_j \cup S$。

算法 2.7　删除候选集后的分量与局部块

输入：无向图的邻接表，候选集合 $S \subseteq V$。输出：$\mathcal{G}-S$ 的连通分量 $C_1,\dots,C_r$，拆分是否成功（$r \geq 2$），以及局部块 $\mathcal{B}_1,\dots,\mathcal{B}_r$。

1. 用一个布尔数组标记 S 中的顶点为“屏蔽”，不修改原邻接表。
2. 对未屏蔽的顶点执行算法 2.3：外层扫描跳过屏蔽顶点，遍历时读到屏蔽的邻居一律跳过。
3. 输出各分量；若 $r \geq 2$ 报告拆分成功，$r=1$ 报告未拆分，$r=0$ 报告余图为空。
4. 对每个 j，以 $C_j \cup S$ 为顶点集输出局部块 $\mathcal{B}_j$；其中 S 的顶点和 S 内部的边只记录对原图的引用。

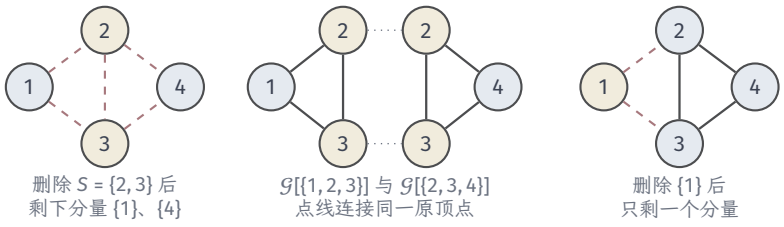
执行过程　在贯穿四点图上取 $S=\{2,3\}$。屏蔽 2, 3 后，外层扫描从 1 出发：1 的邻居 2, 3 都被屏蔽，遍历立即结束，得到 $C_1=\{1\}$；扫描到 2, 3 时跳过；从 4 出发同样只得到 $C_2=\{4\}$。余图有两个分量，因为 $\{1,4\}$ 不是原图的边，见图 2.14(a)。$S=\{2,3\}$ 成功地分隔了 1 和 4。把 S 加回两侧，得到 $\mathcal{B}_1=\mathcal{G}[\{1,2,3\}]$ 与 $\mathcal{B}_2=\mathcal{G}[\{2,3,4\}]$，都是三角形，共享顶点 2, 3 和边 $\{2,3\}$，见图 2.14(b)。

作为对照，取 $S=\{1\}$，删除后剩下的 2, 3, 4 两两相邻，构成一个三角形，余图只有一个分量，见图 2.14(c)，$\{1\}$ 不是有效的拆分。贯穿图中删去任何单个顶点都不会断开余图：删去 1 或 4 剩下三角形，删去 2 或 3 剩下一条路径，所以它没有割点，至少要删两个顶点才能拆开。

2.5.3　算法分析

正确性　屏蔽 S 后的分量编号就是在 $\mathcal{G}-S$ 上执行算法 2.3：读到屏蔽邻居不进入，等于删去了与 S 关联的边。由第 2.3.1小节的分析，得到的 $C_1,\dots,C_r$ 恰是 $\mathcal{G}-S$ 的连通分量，按 r 作出的判断也就正确。

局部块保存了原图的全部边。原图的一条边有三种可能：两端都在某个 C_j 中，或一端在 C_j、一端在 S 中，这两种边都属于 $\mathcal{B}_j$；两端都在 S 中，这



(a) 删除候选集　(b) 加回接口得到局部块　(c) 无效候选集

图 2.14　基于分隔集的分块。黄色顶点为候选删除集，红色虚线为随之删去的边。(a) 贯穿四点图删除 $S=\{2,3\}$ 后剩下两个分量 $\{1\}$ 与 $\{4\}$；(b) 把 S 加回各侧得到两个局部块，两块中的 2, 3 是同一批原顶点（点线相连），边 $\{2,3\}$ 被两块共同引用；(c) 删除 $\{1\}$ 后余下三角形仍然连通，没有形成拆分。

样的边属于每一个块。由于不同分量之间没有边，不存在第四种情况。所以原图的全部信息都保存在这些块中，块与块之间只通过 S 发生联系。

块与块之间的联系落在接口顶点上，而各块中的接口顶点是同一批原顶点，不是可以各自独立变化的副本。若以后要给顶点赋值，同一个接口顶点在各块中必须取相同的值；若块中记录了边的数据，S 内部的边被多个块同时看到，汇总时不能重复计数。因此算法第四步只记录接口顶点和接口边的引用，而不是把原图的一部分真正复制几份后各自修改。

复杂度　屏蔽 S 并计算余图分量需要 $O(n+m)$ 时间，输出局部块的存储则要另计：各块顶点数之和为 $\sum_j(|C_j|+|S|)=(n-|S|)+r|S|$，接口顶点被重复列出 r 次；S 内部的边也在每个块中出现一次。r 和 $|S|$ 都较小时，这部分额外存储可以忽略；接口很大或块很多时，共享引用就更为必要。

适用范围　以上计算回答的是“这个给定的 S 是否拆开了图”，用 $O(n+m)$ 时间就能完成。它不回答“最小的拆分集合是什么”。若想通过检验来找最小的分隔集，就要逐个尝试候选集合：大小为 k 的候选集有 $\binom{n}{k}$ 个，每个都要做一次连通分量计算，规模稍大就无法承受。寻找小的分隔集需要另外的方法，它们不是一次连通标记能自动给出的。

即使找到了小的接口，也只说明块与块之间的联系少，并不说明每个块内部简单。例如两个各有 100 个顶点的完全图只共用一个顶点，删除这个顶点就把图拆成两侧，接口大小为 1，可每个局部块仍是一个 100 个顶点的完全图。

分块的收益取决于后续计算在块内需要做什么。第 4部分会看到，当计算由许多只涉及少数顶点的局部项组成时，固定接口上的取值后各块可以分别处理，这时接口大小直接决定中间表的规模；第 12章再把一次分块推广为多层的树分解。

2.6　顶点标记与结构比较

第 1.6节把两个图同构定义为存在保持邻接的顶点双射，并指出顶点数、边数、度分布等必要条件一般不足以判定同构，而直接尝试所有 $n!$ 个双射又不可行。一种折中的办法是给每个顶点一个标记，反复根据“自己是什么、邻居都是什么”更新标记，使结构上不同的顶点逐渐得到不同的标记。这些标记不依赖顶点的编号，可以用来比较两个图，但它并不能区分所有不同构的图。

2.6.1　问题与思路

输入是一个无向图，顶点可以带有属性，也可以没有。我们要给每个顶点算出一个标记，满足两条要求：第一，标记只由图的结构（以及属性）决定，与顶点怎样编号无关，同构的图中对应的顶点得到相同的标记；第二，结构上不同的顶点尽可能得到不同的标记。有了这样的标记，比较两个图时就可以统计各种标记出现的次数：若次数不同，两图一定不同构。

无属性的图里，顶点一开始没有任何可区分的信息，可以给所有顶点同一个初始标记；带属性的图则可以把属性作为初始标记。

第一步自然的区分是看邻居个数，即度数：三点路径 1–2–3 中，两个端点各有一个邻居，中点有两个邻居，于是端点与中点被区分开。更进一步，不只数邻居有几个，还要看邻居分别属于哪一类、每类有几个。这样，“有两个端点邻居的顶点”和“有一个端点邻居、一个中点邻居的顶点”也能被区分。把这一步反复做下去，每做一次，顶点看到的范围就向外扩大一圈邻居。

多重集合　记录“邻居都属于哪些类、各几个”，普通集合不行：两个邻居都是 A 类时，集合 $\{A, A\}$ 就是 $\{A\}$，丢失了次数。

按顶点编号排出邻居标记的序列也不行：同一个顶点的邻居换一种编号，序列的次序就变了，而我们要的信息不应依赖编号。合适的对象是**多重集合**（multiset）：它像集合一样不计次序，又像序列一样记录每个元素出现的次数。

用双花括号表示多重集合，例如 $\{\!\{A, A, B\}\!\}$ 含两个 A、一个 B，它不等于 $\{\!\{A, B\}\!\}$，但等于 $\{\!\{A, B, A\}\!\}$。实际比较两个多重集合时，可以先把元素排序，再逐项比较。

把顶点自己的当前标记与邻居标记的多重集合放在一起，称为该顶点在第 t 轮的签名：

$$\sigma_i^{(t)} = \left(c_i^{(t)},\ \{\!\{ c_j^{(t)} : j \in N(i) \}\!\} \right).$$

签名完全由图的结构和当前标记决定，与顶点编号无关。签名相同的顶点给同一个新标记，不同的给不同的新标记。比较两个图时，两图必须使用同一种初始标记规则，并且把签名翻译成新标记时使用同一本“字典”：同一个签名在两个图中必须得到同一个新标记。否则即使两图结构相同，标记的名称也可能对不上。

同样出于第一条要求，不能为了“容易区分”而给每个顶点附上唯一的编号作为初始标记。那样所有顶点一开始就互不相同，比较的就不再是无属性图的结构，而是“编号也必须对应”的另一个问题。只有当属性本身是图的数据、并随顶点重编号一起移动时，才把它作为初始标记，这时比较的是带属性的图。

2.6.2 算法实现

把上一小节的思路写成精确的更新规则，就是下面的定义。

定义 2.14　1-WL 标记细化　给定无向图 $G=(V,E)$ 与初始标记 $c_i^{(0)}$，对 $t=0,1,2,\dots$ 同步计算

$$c_i^{(t+1)}=\operatorname{enc}\big(\sigma_i^{(t)}\big)=\operatorname{enc}\Big(c_i^{(t)},\ \{\!\{c_j^{(t)}:\ j\in N(i)\}\!\}\Big),\qquad i\in V,\tag{2.1}$$

其中 enc 是单射，即不同的签名映为不同的新标记。这一过程称为一维 Weisfeiler–Leman 标记细化（1-WL），也称**颜色细化**（color refinement）。第 t 轮的标记把 V 划分为若干类，记为 $\mathcal{P}^{(t)}$。

算法维护三项状态：

- 当前标记 $c_i^{(t)}$：每个顶点一个，初始时取同一个值，带属性时取顶点属性；
- 字典：把已经出现过的签名翻译为标记，初始为空；
- 当前划分 $\mathcal{P}^{(t)}$：由当前标记把顶点分成的类，只用于判断何时停止。

每一轮依次做三件事：

- 计算签名：对每个顶点 i 读出自己的标记和全部邻居的标记，把邻居标记排序成多重集合，得到 $\sigma_i^{(t)}$。
- 分配新标记：对每个签名查字典，已有就用对应的标记，没有就分配一个新标记并记入字典。全部顶点的新标记都算完后，才统一替换旧标记。
- 判断停止：比较新划分 $\mathcal{P}^{(t+1)}$ 与 $\mathcal{P}^{(t)}$，相同就停止并输出当前标记，否则进入下一轮。

统一替换是为了让第 $t+1$ 轮的所有签名都只读取第 t 轮的标记。若边算边替换，后处理的顶点会读到一部分新标记、一部分旧标记，结果就依赖处理次序，从而依赖编号。

enc 的单射要求由字典保证；实际程序常用哈希函数把签名压缩成整数，而不同签名可能碰巧得到相同的哈希值，所以应当比较完整的签名，或用字典给每个新出现的签名分配一个新整数，不能只凭哈希值判定两个签名相等。

标记的名称每一轮都可能换新，我们关心的却是顶点被分成了哪些类，所以停止的判断看划分而不看标记本身；这一规则的合理性在下一小节证明。

执行过程　图 2.15从三点路径 1–2–3 开始，初始标记全为 A。第 0 轮的签名为：端点 $1,3$ 是 $(A,\{\!\{A\}\!\})$，中点 2 是 $(A,\{\!\{A,A\}\!\})$。两种签名不同，分别编码为 B 与 C，得到 $c^{(1)}=(B,C,B)$，划分为 $\{1,3\}$、$\{2\}$。第 1 轮的签名为：端点是 $(B,\{\!\{C\}\!\})$，中点是 $(C,\{\!\{B,B\}\!\})$，编码为 D 与 E，得到 $c^{(2)}=(D,E,D)$。标记的名称换了，但划分仍是 $\{1,3\}$、$\{2\}$，与上一轮相同，细化到此停止。

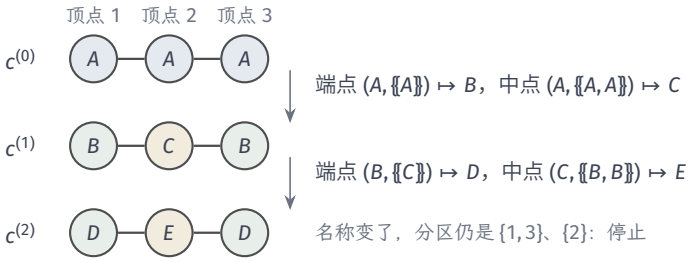


图 2.15　三点路径上的 1-WL 标记细化。每行是一轮标记，颜色表示所属的类；右侧是由签名得到新标记的编码。第一轮把端点与中点分开；第二轮标记名称改变，但划分仍为 $\{1,3\}$、$\{2\}$，细化停止。

2.6.3　算法分析

停止性　下面的引理说明划分只会越来越细，一旦某轮没有变化，以后也不会再变；因此按划分停止是合理的，算法一定结束。

> **引理 2.15　1-WL 的单调性与停止**　对定义 2.14中的划分：（1）$\mathcal{P}^{(t+1)}$ 是 $\mathcal{P}^{(t)}$ 的细分，即第 $t+1$ 轮同类的顶点在第 t 轮也同类；（2）若 $\mathcal{P}^{(t+1)}=\mathcal{P}^{(t)}$，则以后各轮划分都不再改变；（3）至多 $n-1$ 轮严格细分后，划分就停止变化。

证明 （1）若 $c_i^{(t+1)} = c_j^{(t+1)}$，由 enc 是单射得 $\sigma_i^{(t)} = \sigma_j^{(t)}$，签名的第一个分量相等，即 $c_i^{(t)} = c_j^{(t)}$。

（2）两轮划分相同，意味着存在标记之间的一一对应 φ，使得对每个顶点 $c_i^{(t+1)} = \varphi(c_i^{(t)})$。把 φ 作用到签名的每个分量上，得到 $\sigma_i^{(t+1)} = \big(\varphi(c_i^{(t)}), \{\!\{\varphi(c_j^{(t)}) : j \in N(i)\}\!\}\big)$。$\varphi$ 是一一对应，所以 $\sigma_i^{(t+1)} = \sigma_k^{(t+1)}$ 当且仅当 $\sigma_i^{(t)} = \sigma_k^{(t)}$，即 $c_i^{(t+2)} = c_k^{(t+2)}$ 当且仅当 $c_i^{(t+1)} = c_k^{(t+1)}$。因此 $\mathcal{P}^{(t+2)} = \mathcal{P}^{(t+1)}$，依此类推。

（3）每次严格细分至少使类的个数加一，类的个数从至少 1 开始、不超过 n，所以严格细分至多 $n-1$ 次。由（2），第一次没有变化之后就再也不会变化。 ■

对编号的不变性 问题的第一条要求是标记与编号无关。设 π 是从 $\mathcal{G}_1$ 到 $\mathcal{G}_2$ 的同构，且初始标记满足 $c_{\pi(i)}^{(0)} = c_i^{(0)}$（无属性时所有初始标记相同，自然满足；有属性时同构本来就要求保持属性），两图共用一本字典。π 把 i 的邻居一一映为 $\pi(i)$ 的邻居，对轮数归纳可得 $\sigma_{\pi(i)}^{(t)} = \sigma_i^{(t)}$，于是 $c_{\pi(i)}^{(t+1)} = c_i^{(t+1)}$。特别地，对同一个图重新编号顶点（用第 1.6节的置换表示），各顶点的标记随之移动，每种标记出现的次数不变：1-WL 的结果不依赖编号，也不依赖图的画法。

复杂度 每一轮要为每个顶点读取全部邻居的标记，共 $O(n+m)$ 次读取；把顶点 i 的邻居标记排序成多重集合的规范形式需要 $O(d_i \log d_i)$ 时间；再把全部签名排序或查字典以分配新标记。一轮的总代价为 $O((n+m)\log n)$ 量级。由引理 2.15，至多 n 轮就能确认划分稳定（最后一轮用于确认没有变化），所以总时间不超过 $O(n(n+m)\log n)$。实际图上细化往往在少数几轮内稳定，总开销远小于这个上界。

2.6.4 同构检验及其局限

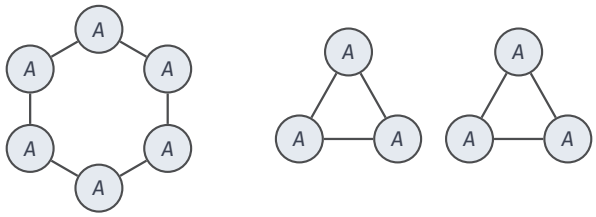
要用 1-WL 比较图 $\mathcal{G}_1$ 与 $\mathcal{G}_2$，把两图放在一起看成一个不交并，用同一个初始标记规则和同一本字典同步细化，然后在每一轮分别统计两图中每种标记出现的次数。这种比较方法称为**一维 Weisfeiler–Leman 检验**（1-dimensional Weisfeiler–Leman test）。若某一轮两图的计数不同，就可以断定两图不同构；若计数在细化稳定前始终相同，只能说这一方法没有区分它们。前一结论的依据是上一小节的不变性：同构的两图在每一轮都有相同的标记计数。

例如，四个顶点的路径与四个顶点的星形都有 4 个顶点、3 条边。初始标记全为 A 时，第一轮路径得到两个“一个邻居”的端点和两个“两个邻居”的内点，星形得到三个“一个邻居”的叶和一个“三个邻居”的中心，

计数不同，两图不同构。这一轮区分的其实就是度分布，更多轮次则会区分邻居的度、邻居的邻居的度，等等。

局限　1-WL 的判别能力是有限的。图 2.16中，六个顶点的环与两个不相连的三角形都有 6 个顶点、6 条边，每个顶点的度都是 2。

初始标记全为 A 时，每个顶点的签名都是 $(A, \{\!\{A, A\}\!\})$，所有顶点得到同一个新标记；下一轮又是同样的情形。标记从第一轮起就稳定为一类，两图的计数始终相同，1-WL 无法区分它们。可是它们并不同构：六环是连通的，而两个三角形构成两个连通分量。用第 2.3.1小节的方法数一下连通分量，就能立刻把它们分开。



(a) 六环：1 个分量　　(b) 两个三角形：2 个分量

两图每个顶点的签名都是 $(A, \{\!\{A, A\}\!\})$，各轮类别计数相同

图 2.16　1-WL 的局限。(a) 六顶点环与 (b) 两个不相连的三角形初始标记全为 A；每个顶点在每一轮都看到两个同类邻居，签名恒为 $(A, \{\!\{A, A\}\!\})$，两图各轮的标记计数完全相同。两图分量数不同，并不同构。

这个例子说明了 1-WL 的本质：每个顶点只能看到以自己为中心、逐轮向外扩大的邻域中“各类邻居有几个”，而六环和三角形上的每个顶点，无论看多少轮，看到的局部情形都一样。连通分量数这种整体性质，局部的计数无法反映。

因此 1-WL 计数不同可以作为不同构的证据，计数相同则不能作为同构的证明。实践中常把它与分量数、环长等其他检验结合使用；要确认同构，仍需给出一个实际的保持邻接的双射并逐边核对。

2.7　本章小结

本章的出发点是：第 1章定义的可达、分量、无环、分隔和同构，怎样在一张实际的图上算出来，而不去逐条列举路径。回答的共同方式是沿邻接表有组织地访问顶点，用少量状态记住已经知道的东西，而不去枚举路径或双射。表 2.2对照了本章各方法所维护的状态、输出和计算时间。

这些输出回答的是不同的问题。广度优先遍历的层数回答“离起点最少几步”，深度优先遍历的完成次序回答“谁的后续全部探完”，它在无环图上给出拓扑序，在一般有向图上把强连通分量按缩点图的先后分离出来。

表 2.2 本章方法维护的状态、输出与时间。时间均以邻接表存储计，n 为顶点数，m 为边数或弧数。

方法	维护的状态	输出	时间
广度优先遍历	队列、距离、父指针	最少边数与一条最短路线	$O(n+m)$
深度优先遍历	三色状态、栈帧、时刻	发现次序、完成次序、回边	$O(n+m)$
分量编号	外层扫描、分量号	连通分量、生成森林	$O(n+m)$
并查集	父指针树、集合大小	加边过程中的分量代表	每次 $O(\log n)$
Kosaraju 算法	完成次序、反向图	强连通分量、缩点图	$O(n+m)$
Kahn 算法	当前入度、零入度队列	拓扑序或有环报告	$O(n+m)$
分隔集分块	屏蔽标记、分量号	余图分量、局部块	$O(n+m)$
1-WL 细化	各轮标记、签名字典	稳定划分、标记计数	每轮 $O((n+m)\log n)$

并查集的代表元回答“在已加入的边下是否连通”，分隔集分块回答“删去给定接口后图分成几侧”，1-WL 的标记回答“局部邻域在计数意义下是否一样”。

同一张图上，这些量可以同时计算，但不能互相替代：深度优先的父路径不是最短路线，发现次序不是拓扑序，并查集的父指针不是原图的边，1-WL 计数相同也不是同构的证明。

判断一个算法的输出是否正确，标准是看它是否满足相应的结构条件，而不只是看算法是否停止。广度优先遍历的父路径应恰有 $d(v)$ 条边；生成森林应有 $n-c$ 条边且无环；拓扑序应使每条弧从前指向后；有环报告应附带一个可以逐弧核对的环；强连通分量在第二遍中不应越过已标记的分量；分块检验应报告余图实际有几个分量。各节的定理正是把这些条件与算法的状态联系起来的依据。

本章得到的可达区域、生成森林和依赖次序，都只描述了“能否连接、按什么次序”，没有比较不同路线或不同边集的代价：广度优先遍历只数边数，生成森林只保证连通。第 2部分在这些结构的基础上引入边上的数值，研究最短路、最小生成树、最大流、匹配与指派和最小费用流。本章的工具在那里继续使用：拓扑序用于第 3章有向无环图上的最短路递推，并查集用于第 4章的 Kruskal 算法，广度优先遍历用于第 5章在残量网络中寻找弧数最少的增广路径。

第 2.5节的接口思想则在第 4部分展开：固定接口上的取值后，各局部块上的计算可以分别进行，再沿接口汇总。

第二部分

图上的组合优化

Combinatorial Optimization on Graphs

阅读指引

第 1部分研究图上“有什么连接”和“怎样把连接找出来”：路径、树与分隔只问存在与否，遍历得到的生成森林也只保证连通、无环。本部分给边配上数值，在满足结构要求的众多候选中挑出数值最好的一个。同一张交通网络上，可以找一条总长度最短的路线，可以铺一组总造价最低、又连通全部站点的线路，也可以在每条道路的通行上限内安排尽可能多的输送量。三个问题选择的分别是一条路径、一组边和每条弧上的一个数，边旁的数字也分别表示长度、造价和容量，使用前须先说明它的含义。

前两章各选一种结构。第 3章的最短路算法共用距离标签与松弛，差别在于按什么次序松弛，而次序能否固定取决于图有无有向环、边长是否非负。第 4章的最小生成树比较整组边的总造价，Kruskal 算法与 Prim 算法都靠割性质的交换论证保证每次选边安全。

后三章研究流与配对。第 5章的最大流要同时决定每条弧上的流量，残量网络允许撤回旧流量，使“没有增广路径”恰好成为最优判据，由此得到最大流最小割定理。第 6章先把二部图匹配看成单位容量的最大流，再要求每个对象都完成配对并比较总成本，可行标号与价格分别支撑 KM 算法和 Auction 算法。第 7章在一般的容量与供需下最小化输送费用，最优性由残量网络中没有负费用环刻画，节点势推广了指派中的标号与价格；最大流、匹配与指派都能写成它的特例。

本部分依赖第 1章的路径、树与割，以及第 2章的广度优先遍历、拓扑序与并查集。各算法的正确性都直接由递推、交换和上下界论证给出，不调用一般的优化理论；卷五第 1 章“优化问题与解”起再用一般优化语言重新解释这些问题。全部分记顶点数为 $n = |V|$，无向边数为 m，有向弧数为 a；除非另有说明，图以邻接表存储，长度、容量、成本等数值另存在与边对应的数组中。

阅读时有三处要分清。其一，算法依赖的条件与它的运行时间是两回事：例如允许负边不等于允许负环，比较速度之前先要确认输入满足条件。其二，许多算法停止时附带一份可以独立核对的证书，如最短路的逐弧不等式、同值的流与割、成本等于标号之和的指派；各章的“方法比较与结果检验”说明怎样核对，第 7章的小结把它们并列比较。浮点计算中的核对只能在容差内进行，不能当作精确证书。其三，本部分的两种结构在后

面还会被调用：最大权生成树用于第 12章构造团树，最小割用于第 10章最小化成对二值函数；复用前都要先说明问题怎样转成当前的网络与目标。

第 3 章

最短路

第 2.1节的广度优先遍历给出从起点出发的最少边数。可是在道路网络中，每段路的长度不同；在任务调度中，从一个状态转到另一个状态的代价也各不相同。这时要比较的是路线上各段数值之和，经过的边数少不等于总成本低。本章回答：给每条弧一个长度以后，怎样求出从源点到每个顶点的最低成本，以及一条达到它的路线。

长度允许为负时，问题本身就变得微妙：绕一个总长为负的环，成本可以一直下降，“最短”可能根本不存在。所以第 3.1节先说清在哪些路线中比较，并把每个顶点的距离分成不可达、有限值和负无穷三类。第 3.2节建立三种算法共用的计算机制：每个顶点保存一个当前已知的最好成本，即距离标签，再用松弛把已有路线延长一条弧。

三种算法的差别只在按什么次序松弛、何时可以认定标签不再改变。第 3.3节在有向无环图上按第 2.4节的拓扑序一遍完成；第 3.4节允许任意有环图和负长度，反复修正标签并识别负环的影响；第 3.5节要求边长非负，换来每个顶点只需确定一次。第 3.6节按输入条件比较三者，并说明怎样不重跑算法而独立核对一份结果。

本章的图都是有向图，沿用本部分的记号：顶点数为 n，弧数为 a，图以出邻接表存储，弧长另存在与弧对应的数组中。

3.1 路径成本与最短路问题

输入是有限有向图 $G=(V,E)$、一个源点 $s\in V$，以及每条弧 $u\to v$ 上的一个实数长度 $\ell_{u,v}$。长度可以表示距离、时间或转换代价，也允许为负，例如某次转换带来收益。候选对象是从 s 出发的路线，比较的是路线的成本。对一条行走 $W=(v_0,v_1,\dots,v_k)$，规定它的成本为沿途弧长之和：

$$\ell(W)=\sum_{r=1}^{k}\ell_{v_{r-1},v_r}.$$

只含一个顶点的零步行走成本为零。所有长度都等于 1 时，成本就是边数，回到广度优先遍历的情形；一般长度下两者不再一致。例如 s 到 t 有一条长 5 的直达弧，又有经过 u、两段各长 1 的路线，两步路线反而更短。我们要对每个顶点 v 输出从 s 到 v 的最低成本，以及一条达到这个成本的路线。

“最短路线”还要先说清在哪些路线中比较。一种选择只允许简单路径，即不重复经过顶点的路径；另一种允许一般的行走，顶点和弧都可以重复。有限图中简单路径只有有限条，因此只要终点可达，最小值一定存在；行走可以无限地绕圈，候选有无穷多个，只能谈成本的下确界。

定义 3.1　最短路问题　给定有限有向图 $G=(V,E)$、弧长 $\ell_{u,v}\in\mathbb{R}$ 与源点 $s\in V$。对每个顶点 v，记 $\mathcal{W}_{s,v}$ 为从 s 到 v 的全部有限行走组成的集合，称

$$\delta(v)=\inf_{W\in\mathcal{W}_{s,v}}\ell(W)$$

为 s 到 v 的**最短路**距离，约定空集的下确界为 $+\infty$。若某条行走 $W\in\mathcal{W}_{s,v}$ 满足 $\ell(W)=\delta(v)$，就称 W 是一条 s 到 v 的最短路。求出全部 $\delta(v)$ 及相应路线的问题称为单源最短路问题。

本章的算法计算的都是行走下确界 $\delta(v)$。在多数情形中它与简单路径的最小值一致，原因是行走中的绕圈部分可以删掉。设行走 W 两次经过顶点 x，两次之间的一段是从 x 回到 x 的闭行走 C，删去 C 得到更短的行走 W'，并且 $\ell(W')=\ell(W)-\ell(C)$。只要 C 的成本非负，删去它就不会增加成本。反复删除，直到不再有重复顶点，便得到一条成本不超过 $\ell(W)$ 的简单路径。

负环与三类结果　删除闭合段的论证只在闭合段成本非负时有效。成本为负的有向环称为**负环**（negative cycle）。若能从 s 走到某个负环，绕若干圈后再走到 v，则每多绕一圈，成本就再降低一次，于是 $\delta(v)=-\infty$。这并不表示存在一条“长度为负无穷的路径”，而是说任何有限成本都能被另一条行走打破。

反过来，若没有这样“从 s 可达、又能到达 v”的负环，那么 s 到 v 的行走中出现的任一闭合段也不可能为负。原因是一条成本为负的闭行走，在第一个重复顶点处切开，总能分出一个简单环，剩下的部分仍是闭行走。重复这一切分，闭行走的成本就写成若干个简单环的成本之和，其中必有一个为负；这个环位于闭合段上，所以同样从 s 可达、又能到达 v，与假设矛盾。因此在这种情形下，所有闭合段都可以删除，$\delta(v)$ 等于简单路径的最小成本，并且由一条简单路径达到。

综上，一次单源查询中，每个顶点的距离属于三类之一：不可达时 $\delta(v) = +\infty$；可达且不受负环影响时 $\delta(v)$ 为有限值，并由一条简单路径达到；能从某个 s 可达的负环到达时 $\delta(v) = -\infty$。算法的输出也应按这三类给出。

三类顶点可以出现在同一张图中。图 3.1中，负环 $a \to b \to a$ 只影响它自己及其后继 t；旁支 u 不能从这个负环到达，距离仍是有限的 4；另一个负环 $x \to y \to x$ 从 s 出发到不了，对本次查询没有任何影响。所以不能因为图中某处有负环，就把全部顶点都标成负无穷。

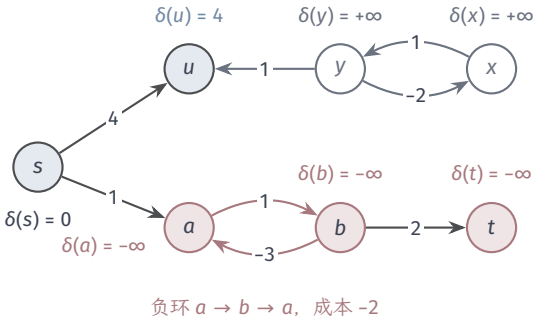


图 3.1　单源最短路的三类结果同处一图（源点为 s，弧旁数字为长度）。$a \to b \to a$ 的成本为 $1 - 3 = -2$，是从 s 可达的负环，环上的 a, b 及其后继 t 的距离为 $-\infty$（红色）；旁支 u 不能从该负环到达，距离为有限值 4（蓝色）；负环 $x \to y \to x$ 从 s 不可达，x, y 的距离为 $+\infty$（灰色），弧 $y \to u$ 也不会改变 u 的距离。

无向图中的负边需要特别注意。把无向边 $\{u, v\}$ 换成两条同长的反向弧 $u \to v$ 与 $v \to u$ 以后，一条长为负的无向边自己就构成负环：沿它走过去再走回来，成本是两倍的负长度。因此，含负边的无向图不能直接套用本章的有向算法；本章默认讨论有向图，无向边只在长度非负时换成一对反向弧。

3.2 距离标签与松弛

三种算法都不直接枚举行走，而是为每个顶点保存一个当前已知的最好成本，在发现更好的路线时更新它。设想已经知道一条从 s 到 u、成本为 $d(u)$ 的路线，又有弧 $u \to v$，那么把这条弧接在路线末尾，就得到一条到 v、成本为 $d(u) + \ell_{u,v}$ 的路线。如果它比 v 目前记录的成本更低，就应该改用它。

松弛操作　为每个顶点 v 保存一个**距离标签**（distance label）$d(v) \in \mathbb{R} \cup \{+\infty\}$ 和一个前驱 $p(v)$。初始时令 $d(s) = 0$，其余顶点 $d(v) = +\infty$，前驱为空。对一条弧 $u \to v$ 执行**松弛**（relaxation），是指比较当前标签 $d(v)$ 与候选值 $d(u) + \ell_{u,v}$：若 $d(u) < +\infty$ 且

$$d(u) + \ell_{u,v} < d(v),$$

就令 $d(v) \leftarrow d(u) + \ell_{u,v}$、$p(v) \leftarrow u$；否则什么也不做。只有严格改善时才更新，因而标签只会下降，不会上升。

图 3.2画出一次松弛的两种结果：候选值 5 小于原标签 8 时，标签与前驱一起改写；原标签已是 4 时，两者都不变。距离数组和前驱数组必须同步修改：如果只改距离而不改前驱，回溯得到的路线成本就不再等于标签。

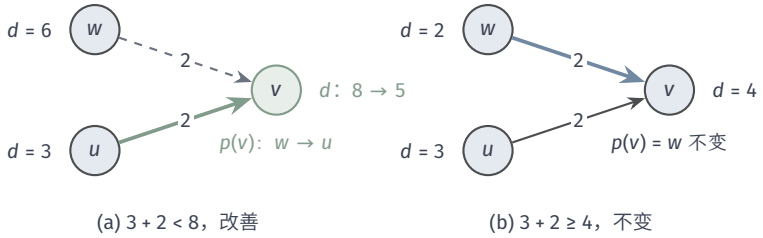


图 3.2　对弧 $u \to v$ 松弛（顶点旁为当前标签，弧旁为长度）。(a) 候选值 $d(u) + \ell_{u,v} = 3 + 2 = 5$ 小于原标签 8，标签改为 5，前驱由 w 改为 u（绿色表示被改善的标签与新的前驱弧，虚线为被替换的旧前驱弧）。(b) 原标签为 4，候选值 5 不能改善，标签与前驱都不变。

上界不变量　松弛只从有限标签出发延长路线，由此得到所有算法共用的一个不变量：每个有限标签 $d(v)$ 都等于某条从 s 到 v 的行走的成本。初始时只有 $d(s) = 0$ 有限，对应零步行走。若松弛把 $d(v)$ 改为 $d(u) + \ell_{u,v}$，则把 u 的那条行走接上弧 $u \to v$，就得到成本恰为新标签的行走。由此立即得到 $d(v) \geq \delta(v)$：标签永远是真实距离的上界。后面每种算法要做的，都是说明在适当的条件下，这个上界最终会降到 $\delta(v)$。

能否反过来，直接解一组方程得到距离？若没有可达负环，真实距离满足一组局部等式：$\delta(s) = 0$，并且对每个可达的 $v \neq s$，有 $\delta(v) = \min_{u \to v}\{\delta(u) + \ell_{u,v}\}$。这只是说最短路的最后一条弧总来自某个前驱。但满足这组等式的数组不一定就是距离。设 $s \to a$ 长 5，$a \to b$ 与 $b \to a$ 长都为 0，则对任意 $c \leq 5$，取 $D(s) = 0$、$D(a) = D(b) = c$ 都满足

$$D(a) = \min\{D(s) + 5,\ D(b) + 0\} = c, \qquad D(b) = D(a) + 0 = c,$$

而真实距离是 $\delta(a) = \delta(b) = 5$。这里的零成本环让 a, b 互相“支撑”着一个过低的值，却没有任何实际路线达到它。

所以算法不能只去求解局部等式，而要从“源为零、其余为无穷”的初值出发，只沿实际路线下调标签：初值保证标签不会无中生有，松弛保证每个标签都有路线见证。松弛按什么次序进行、何时可以停止，则由各算法分别规定。

三种算法都同时保存标签与前驱。前驱记录达到当前上界的最后一条弧，用于事后恢复路线，但只在标签不再变化以后才能放心回溯：在一般

有环图中，计算中途的前驱指针可能尚未形成从 s 出发的树；若存在可达负环，前驱指针甚至可能首尾相接成环。

3.3 有向无环图上的最短路

3.3.1 问题与思路

输入限定为有向无环图，弧长仍可为任意实数。在一般的图中，一个顶点的标签可能在它的出弧已被松弛之后才下降，这时它的后继也必须随之更新。有向无环图没有这个问题：第 2.4节说明，有限有向无环图存在拓扑序，使每条弧的起点排在终点之前。按拓扑序处理顶点，轮到 v 时，所有指向 v 的弧都已被松弛，v 的标签此后不会再变，于是可以放心地用它去松弛 v 的出弧。

3.3.2 算法实现

拓扑序在松弛开始前一次求出，只规定处理次序，运行中不变，不算状态。算法维护三项状态：

- 距离标签 $d(v)$：初值 $d(s)=0$，其余为 $+\infty$；
- 前驱 $p(v)$：初值为空；
- 位置 i：下一个待处理的顶点在拓扑序中的序号，初值为 1。

每一步处理拓扑序中的第 i 个顶点 v_i：若 $d(v_i)<+\infty$，对它的每条出弧执行松弛，否则它给不出有限候选，直接跳过；然后令 $i\leftarrow i+1$。处理完 v_n 后停止，再沿前驱恢复路线。

算法 3.1　有向无环图上的最短路

输入：有限有向无环图、弧长 $\ell_{u,v}\in\mathbb{R}$、源点 s。输出：全部距离 $\delta(v)$ 与前驱 $p(v)$。

1. 求一个拓扑序 $v_1,\dots,v_n$；令 $d(s)=0$，其余 $d(v)=+\infty$，前驱为空。
2. 按 $i=1,\dots,n$ 的顺序处理 v_i：若 $d(v_i)<+\infty$，对其每条出弧 $v_i\to w$ 执行松弛。
3. 对标签有限的顶点，沿前驱从终点退回 s 再反转，得到一条最短路；标签为 $+\infty$ 的顶点报告不可达。

执行过程　取弧 $s\to a$ 长 2、$s\to b$ 长 5、$a\to b$ 长 -1、$a\to t$ 长 5、$b\to t$ 长 2，它没有有向环，拓扑序可取 (s,a,b,t)。初始标签为 $(d(s),d(a),d(b),d(t))=(0,+\infty,+\infty,+\infty)$，前驱全为空。按拓扑序逐个处理顶点，松弛它的出弧：

- 处理 s：a 由 $+\infty$ 改为 $0+2=2$，b 由 $+\infty$ 改为 $0+5=5$，前驱都记为 s，标签变为 $(0,2,5,+\infty)$。

- 处理 a：候选 $2+(-1)=1<5$，b 改为 1、前驱改为 a；t 由 $+\infty$ 改为 $2+5=7$、前驱记为 a。标签变为 $(0,2,1,7)$。
- 处理 b：候选 $1+2=3<7$，t 改为 3、前驱改为 b，标签变为 $(0,2,1,3)$。
- 处理 t：没有出弧，标签不变。

扫描结束时前驱为 $p(a)=s$、$p(b)=a$、$p(t)=b$，从 t 回溯得到最短路 $s\to a\to b\to t$，成本 $2-1+2=3$。图 3.3记录了各标签的变化次序。

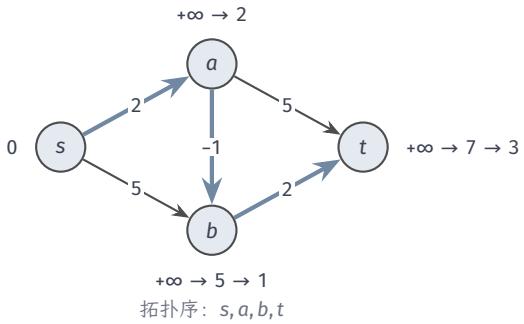


图 3.3　按拓扑序 (s,a,b,t) 在有向无环图上松弛。顶点旁列出标签的变化次序，例如 b 先由处理 s 得到 5，再由处理 a 改为 1；蓝色粗弧是最终前驱弧，组成最短路 $s\to a\to b\to t$（成本 3）。负长度弧 $a\to b$ 并不妨碍计算，因为正确性只依赖无环次序。

3.3.3 算法分析

正确性　一遍扫描之所以够用，是因为处理到某个顶点时，所有可能到达它的路线都已考虑过。

定理 3.2　有向无环图上最短路的正确性　设 $\mathcal{G}$ 是有限有向无环图，弧长为任意实数。算法 3.1结束时，对每个顶点 v 都有 $d(v)=\delta(v)$；其中 $\delta(v)=+\infty$ 当且仅当 v 从 s 不可达，其余 $\delta(v)$ 均为有限值。

证明　有向无环图中的行走不会重复顶点，因此每条行走都是简单路径，只有有限条，$\delta(v)$ 只可能是 $+\infty$ 或有限值。松弛只改变弧终点的标签，而指向 v 的弧都从拓扑序中排在 v 之前的顶点出发，所以轮到 v 时它的标签已经不再改变。下面对拓扑序归纳，证明轮到 v 时 $d(v)=\delta(v)$。

排在 s 之前的顶点从 s 不可达，它们的标签保持 $+\infty$，与距离一致。对 $v=s$，不存在回到 s 的非零步行走，故 $\delta(s)=0=d(s)$。设 v 排在 s 之后，并且此前的顶点都已满足结论。若 v 不可达，则它的标签不可能变为有限值，因为有限标签总对应一条实际行走。若 v 可达，任取一条 s 到 v 的路径，设其最后一条弧为 $u\to v$，则 u 排在 v

> 之前，路径前缀的成本不小于 $\delta(u)$，归纳假设给出 $\delta(u)=d(u)$。处理 u 时弧 $u\to v$ 已被松弛，因而
>
> $$d(v)\le d(u)+\ell_{u,v}=\delta(u)+\ell_{u,v}\le \text{该路径的成本}.$$
>
> 路径是任取的，所以 $d(v)\le\delta(v)$。结合标签不变量 $d(v)\ge\delta(v)$，得 $d(v)=\delta(v)$。 ■

证明中只用到“前驱先于后继处理”，没有用到边长的符号，所以负长度弧不会造成任何问题；反过来，图中只要有一个有向环就没有拓扑序，算法无法开始，此时要改用后面两种算法。

复杂度　求拓扑序需要 $O(n+a)$ 时间，之后每个顶点处理一次、每条弧松弛一次，总时间为 $O(n+a)$，标签、前驱与拓扑序共占 $O(n)$ 额外空间。

3.3.4 编辑距离

许多递推问题的状态之间天然形成有向无环图：每个状态的最优值，由能一步转来的前面状态加上这一步的代价取最小得到。这正是按拓扑序松弛的最短路问题，编辑距离是一个典型例子。

比较两段序列时，常问把序列 $\boldsymbol{a}=(a_1,\dots,a_p)$ 改写成 $\boldsymbol{b}=(b_1,\dots,b_q)$ 最少需要多少次单元素操作，允许的操作是插入一个元素、删除一个元素和把一个元素替换为另一个。

以 (i,j) 表示“已把 $\boldsymbol{a}$ 的前 i 个元素改写成 $\boldsymbol{b}$ 的前 j 个元素”这一状态，最后一步只有三种可能：删去 a_i，从 $(i-1,j)$ 而来，成本为 1；插入 b_j，从 $(i,j-1)$ 而来，成本为 1；让 a_i 对应 b_j，从 $(i-1,j-1)$ 而来，相同时成本为 0，不同时为一次替换，成本为 1。

于是最少操作数 $E(i,j)$ 满足

$$E(i,j)=\min\bigl\{E(i-1,j)+1,\ E(i,j-1)+1,\ E(i-1,j-1)+\mathbb{I}\{a_i\neq b_j\}\bigr\},\tag{3.1}$$

边界为 $E(i,0)=i$、$E(0,j)=j$，其中 $\mathbb{I}\{\cdot\}$ 在条件成立时取 1、否则取 0。

每条状态弧都使 $i+j$ 增加，所以状态图没有有向环，$E(p,q)$ 就是从 $(0,0)$ 到 (p,q) 的最短路距离，称为两段序列的**编辑距离**（edit distance）。

按 $i+j$ 递增或逐行的顺序计算，正是在按拓扑序松弛。例如 $\boldsymbol{a}=(A,B)$、$\boldsymbol{b}=(A,C,B)$ 时，逐行计算得到表 3.1，沿“匹配 A、插入 C、匹配 B”得到距离 $E(2,3)=1$。

全表连同前驱需要 $O(pq)$ 时间与空间；只要距离值时保留相邻两行即可。给插入、删除和替换配上不同成本的序列对齐，以及动态时间规整一

类的递推，也都可以照此写成有向无环图上的最短路，只是弧长和状态转移不同。

表 3.1 $\boldsymbol{a}=(A,B)$ 改写为 $\boldsymbol{b}=(A,C,B)$ 的编辑距离表。第 i 行第 j 列为 $E(i,j)$，按式 (3.1)逐行计算；右下角 $E(2,3)=1$。

	$j=0$	$j=1$（A）	$j=2$（C）	$j=3$（B）
$i=0$	0	1	2	3
$i=1$（A）	1	0	1	2
$i=2$（B）	2	1	1	1

3.4 Bellman–Ford 算法

3.4.1 问题与思路

一般的有向图可能有环，没有拓扑序可用，一遍扫描不够。最直接的补救是反复扫描全部弧，让后来的改善继续向后传递，直到标签不再变化。为了说清每一轮扫描得到了什么，考虑各轮彼此分开的同步版本：第 k 轮只读取第 $k-1$ 轮结束时的标签。这样，第 k 轮的标签恰好考虑了至多 k 条弧的路线；不受负环影响的顶点由至多 $n-1$ 条弧的简单路径达到，所以 $n-1$ 轮就够了。完成这些轮次后再扫描一次，仍能改善的弧就暴露了负环的影响。

3.4.2 算法实现

同步版本的第 k 轮对每个顶点比较“沿用旧值”与“在上一轮的某条路线后再接一条弧”：

$$d^{(k)}(v)=\min\Big\{d^{(k-1)}(v),\ \min_{u\to v}\big(d^{(k-1)}(u)+\ell_{u,v}\big)\Big\},\qquad k=1,2,\dots, \tag{3.2}$$

没有入弧时内层最小值取 $+\infty$。算法维护四项状态：

- 两份标签数组 $d^{(k-1)}$ 与 $d^{(k)}$：本轮只读前者、只写后者，初值 $d^{(0)}(s)=0$，其余为 $+\infty$；
- 前驱 $p(v)$：最近一次改善 $d(v)$ 时提供候选值的顶点，初值为空；
- 轮次 k：已完成的轮数，初值为 0；
- 改善标记：本轮有无标签改变。

每一轮依次做三件事：

- 复制：令 $k\leftarrow k+1$，把 $d^{(k-1)}$ 复制为 $d^{(k)}$，改善标记置为否。
- 逐弧更新：对每条起点标签有限的弧 $u\to v$，若 $d^{(k-1)}(u)+\ell_{u,v}<d^{(k)}(v)$，就令 $d^{(k)}(v)\leftarrow d^{(k-1)}(u)+\ell_{u,v}$、$p(v)\leftarrow u$，并记下有改善。扫描完全部弧，$d^{(k)}$ 恰为式 (3.2)的右端。

- 停止判断：若 $k = n-1$ 或本轮无改善，停止递推；否则进入下一轮。

递推停止后，固定此时的标签 d，分两步判别负环的影响：

- 找种子：起点标签有限、仍满足 $d(u)+\ell_{u,v} < d(v)$ 的弧 $u \to v$，其终点称为种子。递推提前停止时没有种子。
- 标记：从全部种子出发沿出弧遍历，把到达的顶点标记为 $-\infty$。

其余顶点中，标签为 $+\infty$ 的报告不可达，标签有限的以标签为距离。

算法 3.2　Bellman–Ford 算法

输入：有向图的出邻接表与弧长 ℓ，起点 s。输出：每个顶点的距离或“不可达”“$-\infty$”，以及前驱。

1. 置 $d^{(0)}(s)=0$，其余 $d^{(0)}(v)=+\infty$，前驱为空，$k=0$。
2. 令 $k \leftarrow k+1$，把 $d^{(k-1)}$ 复制为 $d^{(k)}$。对每条 $d^{(k-1)}(u)<+\infty$ 的弧 $u \to v$，若 $d^{(k-1)}(u)+\ell_{u,v} < d^{(k)}(v)$，置 $d^{(k)}(v)=d^{(k-1)}(u)+\ell_{u,v}$、$p(v)=u$。
3. 若本轮有标签改善且 $k<n-1$，回到第 2 步；否则记 $d=d^{(k)}$。
4. 把满足 $d(u)<+\infty$ 且 $d(u)+\ell_{u,v}<d(v)$ 的弧 $u\to v$ 的终点作为种子，从全部种子沿出弧遍历，到达的顶点标为 $-\infty$；其余顶点按 d 报告距离或不可达。

执行过程　上一节的四点图没有环，看不出反复扫描的必要。图 3.4换用一个有环的例子：五个顶点，弧为 $s\to t$ 长 6、$s\to b$ 长 2、$b\to c$ 长 3、$c\to a$ 长 -4、$a\to t$ 长 2、$t\to b$ 长 1。其中 $b\to c\to a\to t\to b$ 构成一个环，成本为 $3-4+2+1=2$，不是负环。按式 (3.2)逐轮计算，各轮标签见表 3.2。

- 第 1 轮只能用一条弧：t 由 s 得到 6，b 由 s 得到 2，其余仍为 $+\infty$。
- 第 2 轮：c 经 b 得到 $2+3=5$。$a\to t$ 与 $c\to a$ 的起点在上一轮还是 $+\infty$，不产生候选。
- 第 3 轮：a 经 c 得到 $5-4=1$，对应三弧路线 $s\to b\to c\to a$。
- 第 4 轮：t 经 a 降到 $1+2=3$，前驱由 s 改为 a，对应四弧路线 $s\to b\to c\to a\to t$，比直达的 $s\to t$ 便宜。

此时 $k=4=n-1$，递推停止。再检查全部弧，没有一条还能改善，因而没有种子，最终距离为 $(s,a,b,c,t)=(0,1,2,5,3)$。

这个例子恰好用满了 $n-1$ 轮，因为 t 的最短路含 4 条弧；每一行都是“至多 k 条弧”的最优值，例如 $d^{(3)}(t)=6$ 只计入了单弧路线 $s\to t$。改善沿环 $b\to c\to a\to t$ 传递了一圈，但回到 b 时，候选 $3+1=4$ 不小于 2，传递就此停止：环的成本非负，绕圈不会更便宜。

负环存在时，标签不会停下来。回到图 3.1。该图有 7 个顶点，同步计算 6 轮，各轮标签见表 3.3。u 在第 1 轮得到 4 后不再变化，x,y 始终为 $+\infty$；

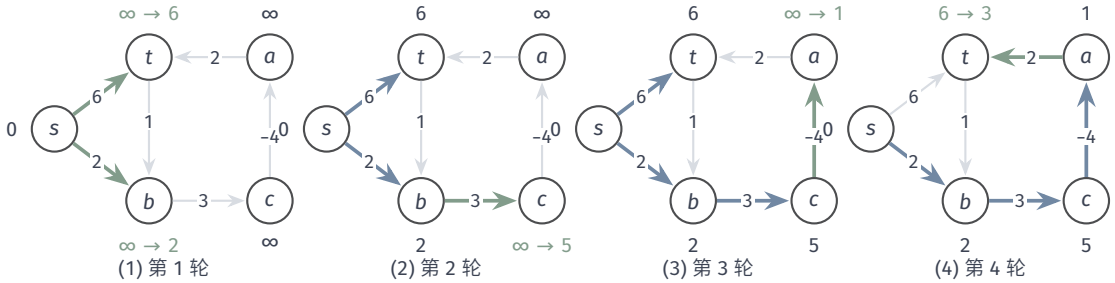


图 3.4 同步 Bellman–Ford 在含环图上的逐轮执行。弧为 $s \to t$ 长 6、$s \to b$ 长 2、$b \to c$ 长 3、$c \to a$ 长 -4、$a \to t$ 长 2、$t \to b$ 长 1，环 $b \to c \to a \to t \to b$ 的成本为 2。顶点旁为该轮结束时的标签，绿色数字是本轮改变的标签，绿色弧是提供改善的弧，蓝色弧是其余的当前前驱弧。改善每轮沿弧前进一步，第 4 轮 t 经 a 由 6 降到 3。

表 3.2 同步 Bellman–Ford 在图 3.4的含环图上的逐轮标签。第 k 行是至多 k 条弧的最小成本，括号内为该轮改变标签时的前驱。

轮次 k	$d^{(k)}(s)$	$d^{(k)}(a)$	$d^{(k)}(b)$	$d^{(k)}(c)$	$d^{(k)}(t)$
0	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	0	$+\infty$	2（s）	$+\infty$	6（s）
2	0	$+\infty$	2	5（b）	6
3	0	1（c）	2	5	6
4	0	1	2	5	3（a）

而 a, b, t 的标签每隔一两轮就再降一次，第 6 轮停在 $-3, -2, 2$。这些值只是绕圈到第 6 轮时恰好停留的数，本身没有意义。

此时检查全部弧：$b \to a$ 的候选 $-2 - 3 = -5 < -3$，$b \to t$ 的候选 $-2 + 2 = 0 < 2$，两条弧仍可改善，种子是 a 与 t；其余弧都不能改善。从种子沿出弧遍历，标记出 $\{a, b, t\}$。算法报告 u 的距离为 4，a, b, t 为 $-\infty$，x, y 不可达，与前面的分析一致。

表 3.3 同步 Bellman–Ford 在图 3.1上的逐轮标签（x, y 各轮均为 $+\infty$，未列出）。u 在第 1 轮后稳定；可达负环 $a \to b \to a$ 使 a, b, t 的标签持续下降。

轮次 k	$d^{(k)}(s)$	$d^{(k)}(u)$	$d^{(k)}(a)$	$d^{(k)}(b)$	$d^{(k)}(t)$
0	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	0	4	1	$+\infty$	$+\infty$
2	0	4	1	2	$+\infty$
3	0	4	-1	2	4
4	0	4	-1	0	4
5	0	4	-3	0	2
6	0	4	-3	-2	2

就地更新 常见的实现不复制数组，而是在同一个数组上逐弧松弛，一轮中刚改善的标签可以立即被同一轮后面的弧使用，称为就地更新。按

$s \to t$、$s \to b$、$b \to c$、$c \to a$、$a \to t$、$t \to b$ 的顺序扫描上例，第一轮就依次得到 $t = 6$、$b = 2$、$c = 5$、$a = 1$，并把 t 降到 3，即全部最终标签，第二轮无改善即可停止。若按相反的顺序扫描，每轮只能把改善推进一步，仍要四轮。

它的停止规则与负环检测方式和同步版本相同，只是“第 k 轮”的含义变了。

3.4.3 算法分析

正确性　分析从同步版本出发，因为它的标签有一个清楚的含义。

> **引理 3.3　同步标签的含义**　对每个 $k \ge 0$ 与每个顶点 v，$d^{(k)}(v)$ 等于从 s 到 v、至多含 k 条弧的全部行走的最小成本；没有这样的行走时为 $+\infty$。

> **证明**　对 k 归纳。$k = 0$ 时，至多零条弧的行走只有 s 处的零步行走，与 $d^{(0)}$ 一致。设结论对 $k-1$ 成立。一条至多含 k 条弧、到达 v 的行走，要么至多含 $k-1$ 条弧，其成本不小于 $d^{(k-1)}(v)$；要么恰含 k 条弧，去掉最后一条弧 $u \to v$ 后剩下至多 $k-1$ 条弧、到达 u 的前缀，其成本不小于 $d^{(k-1)}(u) + \ell_{u,v}$。所以全部候选的最小值不小于式 (3.2)的右端。反过来，右端每一项都由归纳假设对应一条至多 k 条弧的行走，最小值能够达到。故两者相等。 ■

由此先得到轮数上界。对不受可达负环影响的顶点 v，第 3.1节说明 $\delta(v)$ 由一条简单路径达到，而简单路径至多经过 n 个顶点、含 $n-1$ 条弧。由引理 3.3，第 $n-1$ 轮已经计入这条路径，所以 $d^{(n-1)}(v) = \delta(v)$。若某一整轮没有任何标签改变，则此后各轮的输入都相同，标签不会再变，提前停止不影响结果；此时由下面的定理，图中不存在可达负环。

就地更新得到的标签仍是真实行走的成本，又对每轮都不高于同步版本的标签（对轮数归纳即可），所以 $n-1$ 轮的上界照样成立。但“第 k 轮标签恰为至多 k 条弧的最优值”这一解释只属于同步版本，不能原样套在就地更新上：同一轮内的改善可能已经串起了多条弧。

负环的判别　$n-1$ 轮之后的种子与遍历恰好找出了全部距离为 $-\infty$ 的顶点。

定理 3.4　Bellman–Ford 算法对三类结果的判定　在同步递推的 $n-1$ 轮之后按上述方法标记，被标记的顶点恰好是 $\delta(v)=-\infty$ 的顶点；其余顶点中，$d^{(n-1)}(v)=+\infty$ 的不可达，$d^{(n-1)}(v)$ 有限的满足 $d^{(n-1)}(v)=\delta(v)$。

证明　先证每个种子 v 都满足 $\delta(v)=-\infty$。由引理 3.3，v 可改善说明 $d^{(n)}(v)<d^{(n-1)}(v)$，即存在一条恰含 n 条弧的行走，比所有至多 $n-1$ 条弧的行走都便宜。含 n 条弧的行走经过 $n+1$ 个顶点位置，必有顶点重复，从中可以切出一段闭行走 C。若 C 的成本非负，删去它会得到弧更少、成本不更高的行走，与上述“更便宜”矛盾；故 C 的成本为负，由第 3.1节的分解，C 中含有一个负环。这个负环位于一条从 s 出发、到达 v 的行走上，所以 $\delta(v)=-\infty$。从种子沿出弧能到达的顶点也能从同一个负环到达，因而被标记的顶点距离都是 $-\infty$。

再证每个 $\delta(v)=-\infty$ 的顶点都会被标记。此时存在从 s 可达的负环 $C=(c_0,c_1,\dots,c_k=c_0)$ 能到达 v。C 上的顶点从 s 可达，可达顶点至多经 $n-1$ 条弧到达，所以它们的标签有限。若 C 上没有一条弧可改善，则对每条环弧都有 $d(c_r)\le d(c_{r-1})+\ell_{c_{r-1},c_r}$，沿环相加，两侧的标签之和恰好相消，得到 $0\le \ell(C)<0$，矛盾。因此 C 上至少有一个种子，而 v 能从 C 到达，必然被遍历标记。

最后，未被标记且可达的顶点不受任何可达负环影响，前面已说明 $d^{(n-1)}(v)=\delta(v)$；不可达顶点的标签始终为 $+\infty$。 ■

复杂度　每轮逐弧扫描一次，用时 $O(a)$，至多 $n-1$ 轮，再加上一轮检测与一次遍历，总时间为 $O(na)$。同步版本每轮还要复制标签数组，另需 $O(n)$ 时间，合计至多 $O(n^2)$；就地更新省去这部分。只保存当前和上一轮的标签时，工作状态为 $O(n)$。

路线恢复　若要按引理 3.3的含义回溯“至多 k 条弧”的最优路线，就需要保存每一轮的前驱 $p^{(k)}(v)$，记录可达 $O(n^2)$ 条。只保存一份前驱数组时，也可以回溯有限距离的顶点，但要说明这份前驱与递推相容。

设 v 未被标记且标签有限，若 $p(v)=u$，则 $u\to v$ 是一条弧，u 也不会被标记，否则 v 能从负环到达。设置 $p(w)=u$ 时，$d(w)$ 等于 u 此前某一时刻的标签加上 $\ell_{u,w}$；标签只降不升，$d(w)$ 再下降时前驱又会改写，所以任一时刻每条前驱弧都满足 $d(p(w))+\ell_{p(w),w}\le d(w)$。

前驱指针在未标记顶点之间不会成环：若成环，考察环上最后被设置的那个指针，在它被设置之前，环上其余前驱弧都满足上述不等式，而这条弧满足严格不等式，沿环相加，标签相消，得到环的成本为负；这个负环

从 s 可达，又能沿前驱弧到达 v，与 v 未被标记矛盾。于是从 v 沿前驱必能回到 s。把沿途的不等式相加，所得路线的成本不超过 $d(v) = \delta(v)$，因而恰好是一条最短路。

对标为 $-\infty$ 的顶点，算法给出的不是一条“最短路”，而是负环的证据：一个种子、一条从 s 到负环的行走与一条从负环到该顶点的行走。

3.5 Dijkstra 算法

3.5.1 问题与思路

Bellman–Ford 算法要反复扫描全部弧，因为一个标签随时可能再被改小。若边长都非负，情况就不同了：在尚未确定的顶点中，标签最小的那个已经不可能再被改小，因为任何绕道而来的路线都要先经过某个标签不更小的顶点，后半程又不会让成本下降。Dijkstra 算法利用这一点，每次把这样一个顶点永久确定下来，只松弛它的出弧，于是每条弧只需松弛一次。

这里所需的条件是从源点可达部分的弧长都非负。全图非负是便于检查的充分条件；从 s 到达不了的区域即使有负边，也不影响本次查询。在这一条件下，从 s 可达的部分不含负环，所有距离都是 $+\infty$ 或有限值，输出只有两类。

3.5.2 算法实现

优先队列的数据结构　取出标签最小的未确定顶点、在标签下降时调整它的位置，都由以标签为键的优先队列完成，常见的实现有两种。一种是支持“降键”的索引堆，每个顶点在堆中只有一个条目，标签改善时原地下调它的键。另一种不降键，标签改善时直接插入一个新条目 $(d(v), v)$，旧条目留在堆中；出队时若发现该顶点已经确定，说明这是过期条目，直接丢弃。两种实现的输出相同，区别只在队列规模与操作次数。

Dijkstra 算法的实现　算法维护四项状态：

- 距离标签 $d(v)$：初值 $d(s) = 0$，其余为 $+\infty$；
- 前驱 $p(v)$：初值为空；
- 已确定集合 R：标签已永久确定的顶点，初值为空集；
- 优先队列：以标签为键存放标签有限的未确定顶点，初值只含 s。

每一步做两件事：

- 确定：取出队列中标签最小的顶点 u，把它加入 R；
- 松弛：对 u 通往 R 外的每条出弧执行松弛，标签改善时相应调整队列。

队列中不再有标签有限的未确定顶点时停止。

算法 3.3 Dijkstra 算法

输入：有限有向图、源点 s，从 s 可达部分的弧长非负。输出：全部距离 $\delta(v)$ 与前驱 $p(v)$。

1. 令 $d(s)=0$，其余 $d(v)=+\infty$，$R=\varnothing$。
2. 当存在标签有限的未确定顶点时，取出其中标签最小的 u，把 u 加入 R。
3. 对 u 的每条出弧 $u\to v$，若 $v\notin R$，执行松弛；标签改善时更新优先队列中 v 的键。
4. 结束时，R 中顶点的标签即为距离，其余顶点不可达。

执行过程 把四点图中 $a\to b$ 的长度由 -1 改为 1，全部弧长非负，Dijkstra 算法适用。表 3.4列出每次出队后的状态。

第 1 步取出 s，松弛得 $d(a)=2$、$d(b)=5$，前驱都是 s。

第 2 步取出标签最小的 a，候选 $2+1=3<5$ 使 $d(b)=3$、$p(b)=a$，候选 $2+5=7$ 使 $d(t)=7$、$p(t)=a$。

第 3 步取出 b，候选 $3+2=5<7$ 使 $d(t)=5$、$p(t)=b$。

第 4 步取出 t，它没有出弧，队列为空，停止。

最终 $\delta(a)=2$、$\delta(b)=3$、$\delta(t)=5$，从 t 沿前驱回溯得到 $s\to a\to b\to t$。

图 3.5画出同样的四个状态：已确定集合 R 逐个向外扩张，每个顶点在出队时的标签都不再改变。

表 3.4 Dijkstra 算法在非负四点图（$a\to b$ 长 1，其余弧长同前）上的状态。每行是一次出队并松弛之后的标签与已确定集合，括号内为本步改变标签时的新前驱。

步	出队	$d(s)$	$d(a)$	$d(b)$	$d(t)$	R
初始	—	0	$+\infty$	$+\infty$	$+\infty$	$\varnothing$
1	s	0	2（s）	5（s）	$+\infty$	$\{s\}$
2	a	0	2	3（a）	7（a）	$\{s,a\}$
3	b	0	2	3	5（b）	$\{s,a,b\}$
4	t	0	2	3	5	$\{s,a,b,t\}$

3.5.3 算法分析

正确性 关键论断是：顶点被加入 R 的那一刻，它的标签已经等于真实距离，此后不再需要修正。证明的切入点是一条假设更短的路线上第一个尚未确定的顶点。

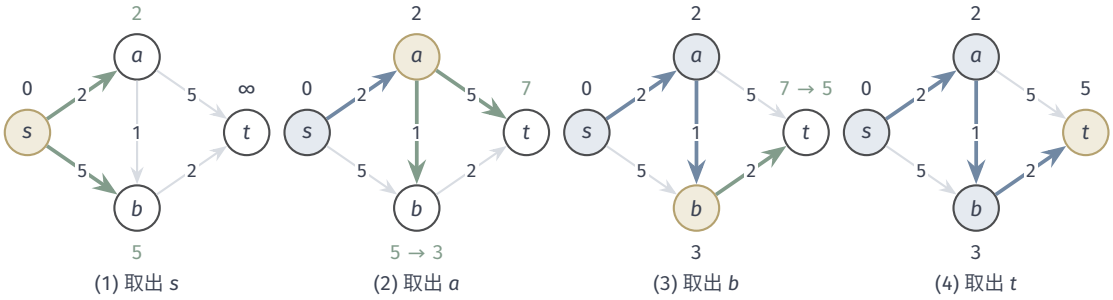


图 3.5 Dijkstra 算法在非负四点图上的出队过程（$a \to b$ 长 1，其余同前）。每个面板对应一次出队：蓝色为已确定顶点，黄色为本次出队的顶点，绿色标签与绿色弧表示本次松弛改善的标签及新前驱。四次出队依次确定 $s = 0$、$a = 2$、$b = 3$、$t = 5$，最终前驱弧组成路线 $s \to a \to b \to t$。

定理 3.5 Dijkstra 算法的正确性 若从 s 可达部分的弧长都非负，则算法 3.3中每个顶点 u 被加入 R 时都有 $d(u) = \delta(u)$；算法结束时未加入 R 的顶点从 s 不可达。

证明 对加入 R 的次序归纳。第一个加入的是 s，$d(s) = 0 = \delta(s)$，因为可达部分没有负环。设此前加入 R 的顶点都满足 $d = \delta$，现在 u 被选出，且 $d(u) < +\infty$。由标签不变量，$d(u) \geq \delta(u)$。假设 $d(u) > \delta(u)$，取一条 s 到 u、成本为 $\delta(u)$ 的简单路径 P。P 从 $s \in R$ 出发，到达 $u \notin R$，所以沿 P 存在第一个不在 R 中的顶点 y，记它在 P 上的前一个顶点为 $x \in R$。

x 加入 R 时弧 $x \to y$ 已被松弛，此后标签只会下降，所以 $d(y) \leq d(x) + \ell_{x,y}$。归纳假设给出 $d(x) = \delta(x)$；而最短路的前缀仍是最短路，故 $\delta(x) + \ell_{x,y} = \delta(y)$。于是 $d(y) \leq \delta(y)$。P 从 y 到 u 的后半段由可达部分的弧组成，长度非负，因而 $\delta(y) \leq \delta(u)$。合起来得到

$$d(y) \leq \delta(y) \leq \delta(u) < d(u).$$

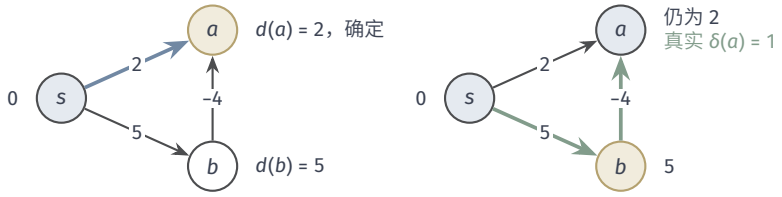
但 y 也是未确定顶点，而算法选出的 u 在未确定顶点中标签最小，应有 $d(u) \leq d(y)$，矛盾。所以 $d(u) = \delta(u)$。

若算法停止时某个可达顶点 v 不在 R 中，取一条 s 到 v 的路径，同样找出第一个不在 R 中的顶点 y 及其前驱 $x \in R$。松弛 $x \to y$ 已使 $d(y)$ 有限，算法便不会在此时停止，矛盾。 ■

适用范围 证明中只有一步用到非负性：“后半段长度非负，所以 $\delta(y) \leq \delta(u)$”。这一步失效时，算法确实会出错，即使图中没有负环。取弧 $s \to a$ 长 2、$s \to b$ 长 5、$b \to a$ 长 -4，如图 3.6。处理 s 后，a 的标签为 2、b 为 5；算法选出标签较小的 a 并永久确定 $d(a) = 2$。随后处理 b 时，候选

$5+(-4)=1$ 本可改善 a，但 a 已在 R 中，不再更新。真实距离是 $\delta(a)=1$，由路线 $s \to b \to a$ 达到。

对照证明，这里的 $y=b$、$u=a$，后半段 $b \to a$ 为负，使 $\delta(b)=5>\delta(a)=1$，“首个未确定顶点的标签不超过 $\delta(u)$”不再成立。



(a) 先确定标签较小的 a　　(b) 候选 $5-4=1$ 已无法改写 a

图 3.6　负边使 Dijkstra 算法过早确定标签。(a) 处理 s 后，a 的标签 2 小于 b 的标签 5，a 被永久确定（黄色为当前出队顶点）。(b) 处理 b 时候选值 $5-4=1$ 已无法改写 a；真实最短路是 $s \to b \to a$（绿色），成本 1。图中没有负环，出错的原因是后半段弧 $b \to a$ 为负。

复杂度　每个顶点至多出队一次，每条弧在其起点出队时松弛一次。用邻接表存储图、用支持降键的索引二叉堆存储未确定顶点时，取最小与降键各需 $O(\log n)$ 时间，总时间为 $O((n+a)\log n)$；标签、前驱、堆及其位置索引共占 $O(n)$ 额外空间，图本身的存储另计。

不降键的实现每次改善插入一个条目，堆中至多有 $a+1$ 个条目，操作时间为 $O(\log a)=O(\log n)$，总时间仍为 $O((n+a)\log n)$，但队列空间上界变为 $O(a)$。稠密图上也可以不用堆，每次扫描数组找最小标签，总时间为 $O(n^2+a)$。

3.6 方法比较与结果检验

三种算法解决同一个单源问题，但对输入的要求不同。比较它们时应先问：图有没有有向环？可达部分有没有负长度弧？是否需要检测负环？然后才比较运行时间。表 3.5汇总了这些区别。

表 3.5　三种单源最短路算法的比较（n 为顶点数，a 为弧数，邻接表存储）。

	有向无环图递推	Bellman–Ford	Dijkstra
图结构	无有向环	不限	不限
弧长	任意实数	任意实数	可达部分非负
标签修正	处理后不变	反复修正	出队后不变
负环	不会出现	检测并标记	条件排除
时间	$O(n+a)$	$O(na)$	$O((n+a)\log n)$
正确性依据	拓扑序归纳	至多 k 弧递推	首个未定顶点

“允许负边”与“存在负环”是两件不同的事。有向无环图递推与 Bellman–Ford 算法都允许负边；前者因为无环而不会遇到负环，后者则能在负环存在时正确报告受影响的顶点。Dijkstra 算法连负边都不允许，哪怕负边并不构成负环，图 3.6就是这样的例子。一种算法在某组数据上跑得更快、甚至答案碰巧正确，都不能替代对其条件的检查。

拿到一份声称的最短路结果后，可以不重跑算法而直接核对。对标签有限的顶点，先沿前驱恢复路线，检查起点为 s、终点为 v、每一步都是原图中的弧，并重算弧长之和是否等于 $d(v)$。这一步证明标签可以达到，即 $d(v) \geq \delta(v)$。

最优性需要反方向的下界。若输出满足 $d(s) = 0$，并且对每条起点标签有限的弧 $u \to v$ 都有

$$d(v) \leq d(u) + \ell_{u,v},$$

那么任取一条 s 到 v 的行走 $W = (s = w_0, w_1, \dots, w_k = v)$，沿 W 把这些不等式逐条相加，中间标签相消，得到 $d(v) \leq d(s) + \ell(W) = \ell(W)$。所以 $d(v) \leq \delta(v)$，与上一段的 $d(v) \geq \delta(v)$ 一起给出 $d(v) = \delta(v)$。

这组逐弧不等式只需 $O(n + a)$ 时间核对，它本身就是有限距离的最优性证书；不满足时，违反不等式的弧直接指出了一条更好的路线。例如在表 3.4的最终标签 $(0, 2, 3, 5)$ 下，五条弧依次给出 $2 \leq 0 + 2$、$3 \leq 0 + 5$、$3 \leq 2 + 1$、$5 \leq 2 + 5$、$5 \leq 3 + 2$，全部成立。

另外两类结果需要不同的证据。对报告为 $+\infty$ 的顶点，用一次广度优先遍历核对它确实不在 s 的可达集中。对报告为 $-\infty$ 的顶点，应给出三段证据：一条从 s 到某个有向环的行走、该环的成本为负，以及一条从环到该顶点的行走。三类结果各有各的验收方式，不能只核对其中一类就宣布整份结果正确。

3.7 本章小结

章首的问题是：给弧配上长度后，怎样求出从源点到每个顶点的最低成本及路线。回答的第一步是承认答案未必是一个有限数。按行走的下确界定义距离后，每个顶点属于三类之一：不可达时为 $+\infty$；可达且不受负环影响时为有限值，并由一条简单路径达到；能从某个 s 可达的负环到达时为 $-\infty$。图中别处有负环，并不影响到不了它的顶点。

三种算法共用同一个机制。距离标签从“源为零、其余为无穷”出发，只通过松弛下降，所以每个有限标签都对应一条实际行走，始终是真实距离的上界。只解局部等式不够：零成本环可以让一组过低的值满足全部等式，却没有路线达到它们。

差别在于何时可以认定一个标签不再改变，而这取决于输入条件。有向无环图按拓扑序处理，前驱的距离在使用前已经确定，弧长可以任意，总时间 $O(n+a)$；编辑距离一类的递推就是这种情形。Bellman–Ford 算法不要求无环，同步版本第 k 轮的标签恰是至多 k 条弧的最优值，$n-1$ 轮后仍能改善的弧暴露负环的影响，总时间 $O(na)$。Dijkstra 算法要求可达部分弧长非负，每次出队的顶点立即确定，用二叉堆时为 $O((n+a)\log n)$；哪怕没有负环，一条负边就可能使它出错。

结果可以不依赖算法过程而独立核对。有限距离用两份证据：沿前驱恢复的路线成本等于标签，给出可达性；逐弧不等式 $d(v) \le d(u) + \ell_{u,v}$ 给出下界。不可达用一次遍历核对，负无穷则要给出一条经过负环的行走。下一章转向另一种选择对象：最小生成树比较的是一整组边的总造价，而不是两点间路线的长度。逐弧不等式与由最短距离构造的势，则会在第 7章的最优性条件中再次出现。

第 4 章

最小生成树

要在若干城镇之间修路，使任意两个城镇都能互相到达，每条候选道路都有建造成本。这时关心的不是某两点之间怎样走最近，而是整张网络的总造价。第 1.4节说明，连通且无环的边集恰好连接全部顶点而没有多余的边，即生成树；多一条边便形成环，环上任意一条边都可以拆掉而不破坏连通。于是问题化为：在所有生成树中找总边权最小的一棵。

生成树的个数可以随顶点数指数增长，无法逐一比较。本章要回答的是：能否逐条选边，并且只看局部就能断定每一步没有选错？第 4.1节先定义问题，并用一个三角形说明它与上一章的最短路树不同。第 4.2节给出判断选边安全的割性质：与已选边相容的任何割上，最轻的跨割边都可以放心选入。

Kruskal 算法与 Prim 算法是同一条规则的两种组织方式。第 4.3节按全局边权次序合并分量，用第 2.3.2小节的并查集判断成环；第 4.4节只让一棵树生长，用与 Dijkstra 算法相同的优先队列维护每个外部顶点的最便宜接入边。第 4.5节比较两者，并给出一份不依赖算法过程的最优性检验。本章的图是无向图，顶点数为 n，边数为 m。

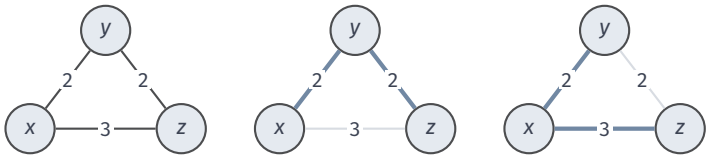
4.1 最小生成树问题

输入是有限无向连通图 $G=(V,E)$，每条边 e 带有实数权 w_e，表示建造成本。可行解是生成树 $T\subseteq E$，目标是它的总成本 $w(T)=\sum_{e\in T} w_e$，输出一棵总成本最小的生成树及其成本。

> **定义 4.1　最小生成树**　有限无向连通加权图的所有生成树中，总成本 $w(T)$ 最小者称为**最小生成树**（minimum spanning tree）。

生成树只有有限多棵，所以最小生成树总是存在。但生成树的个数可以随顶点数指数增长，逐一枚举并不可行。

与最短路树的区别 最小生成树最小化的是连接全部顶点的总成本，并不保证树上每对顶点之间的路线都是原图中的最短路。图 4.1中，三角形三条边的权为 $w_{x,y}=2$、$w_{y,z}=2$、$w_{x,z}=3$。最小生成树取 $\{xy, yz\}$，成本 4；但在树上 x 到 z 要绕经 y，长度为 4，而原图中的直达边只有 3。以 x 为源的**最短路树**（shortest-path tree），即由各顶点最短路的前驱弧组成的树，是 $\{xy, xz\}$，成本 5。两者回答不同的问题，一般不相同。同样，第 2.3节由遍历得到的生成森林只保证连通与无环，不带有任何成本上的最优性。



(a) 原图　(b) 最小生成树，成本 4 树上 x 到 z 长 4　(c) 以 x 为源的最短路树 成本 5，x 到 z 长 3

图 4.1 最小生成树与最短路树可以不同。(a) 原图，边旁为权。(b) 最小生成树 $\{xy, yz\}$ 总成本 4，但树上 x 到 z 的路线长 4。(c) 以 x 为源的最短路树 $\{xy, xz\}$ 保证每个顶点到 x 的距离最短，总成本却是 5。蓝色粗边为所选的边。

非连通、负权与平局 图不连通时不存在生成树，问题改为在每个连通分量内分别求最小生成树，合起来称为**最小生成森林**（minimum spanning forest）。

负权不会带来最短路那样的困难：每棵生成树恰有 $n-1$ 条边，把所有边权同时加上常数 c，每棵生成树的成本都增加 $(n-1)c$，比较结果不变，因此总可以设想权都为正。边权有相同值时，最小生成树可能不止一棵，但最小总成本是唯一的。

若要最大化总权，把全部边权取负即化为最小化问题，得到**最大生成树**（maximum spanning tree），下面的割性质与两种算法只需把“最小”改为“最大”即可照用。第 12章构造团树时会用到最大生成树。

4.2 割性质与安全选边

两种算法都是逐条选边，并希望每一步都不走错。“不走错”的确切含义是：当前已选的边集 A 总能扩充成某一棵最小生成树。只要保持这个不变量，选满 $n-1$ 条边时，A 本身就是一棵最小生成树。问题在于，下一条边选哪条才能保持不变量。

回答这个问题要用到割。把顶点集分成两个非空部分 S 与 $V \setminus S$，称为图的一个**割**（cut），记作 $(S, V \setminus S)$。一条边的两个端点分别在两侧时，称它跨越这个割。若已选边集 A 中没有边跨越该割，就称这个割与 A 相容。跨越某个割的边中权最小的称为该割的**轻边**（light edge），权相同时轻边可以不止一条。与当前选边相容的任何一个割上，轻边都可以放心选入。

引理 4.2　割性质　设 G 是有限无向连通加权图，边集 A 包含在某棵最小生成树中，割 $(S, V \setminus S)$ 与 A 相容，e 是该割的一条轻边。则 $A \cup \{e\}$ 也包含在某棵最小生成树中。

证明　取一棵包含 A 的最小生成树 T。若 $e \in T$，结论已成立。否则把 e 加入 T，由第 1.4节，树加一条边恰好形成一个环。设 $e = \{u, v\}$，$u \in S$、$v \notin S$，这个环由 e 与 T 中 u 到 v 的路径组成。这条路径从 S 出发，最终到达 $V \setminus S$，所以其中至少有一条边 f 跨越该割。

从 $T + e$ 中删去 f，环被打断，所得边集 $T' = T + e - f$ 仍连通且有 $n - 1$ 条边，因而是生成树。由于 e 是轻边，$w_e \le w_f$，所以 $w(T') = w(T) + w_e - w_f \le w(T)$，$T'$ 也是最小生成树。最后，割与 A 相容，跨越割的 f 不在 A 中，删去 f 不影响 A；于是 $A \cup \{e\} \subseteq T'$。 ■

图 4.2画出这一交换：轻边 e 加入后与树中路径形成环，环上另一条跨割边 f 被删去，成本不增。

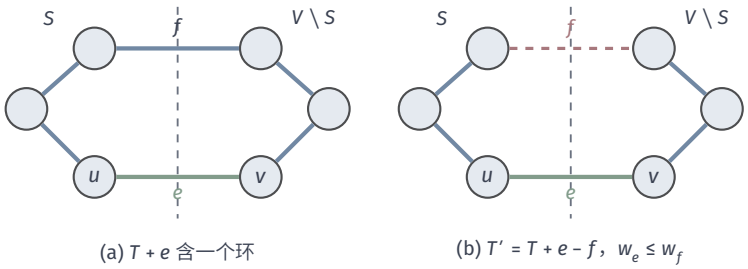


图 4.2　割性质的交换论证。虚线把顶点分成 S 与 $V \setminus S$，蓝色粗边为最优树 T，其中不跨割的边包含当前选边 A。(a) 轻边 e（绿色）不在 T 中，加入后与 T 中 u 到 v 的路径形成环。(b) 环上另一条跨割边 f（红色虚线）被删去，得到 $T' = T + e - f$，因 $w_e \le w_f$，成本不增。

安全不等于必选。割性质只保证轻边能留在某一棵最优树中，并不保证它出现在每一棵最优树中。设三角形三条边的权都是 1，取割 $(\{x\}, \{y, z\})$，xy 与 xz 都是轻边。最小生成树 $\{xy, yz\}$ 不含 xz，但按割性质选入 xz 仍然安全，因为 $\{xz, yz\}$ 同样是最小生成树。平局时不同的选择通向不同的最优树，最优成本却相同。

两种算法的区别只在于选取哪个割：Kruskal 算法以当前森林中某个分量作为 S，Prim 算法以当前唯一一棵树的顶点集作为 S。两者的正确性证明都归结为说明：被选入的边确实是所取割的轻边。

4.3　Kruskal 算法

4.3.1 问题与思路

一种自然的想法是从最便宜的边开始逐条考虑，只要不形成环就选入。开始时每个顶点自成一个分量；一条边的两端若属于不同分量，选入它会把两个分量连起来，不会成环；若两端已在同一分量，选入它就会成环，应当跳过。每选一条边，分量数减一且不产生环，所以选边数加分量数始终等于 n。判断两端是否同属一个分量，正是第 2.3.2小节并查集的用途。

4.3.2 算法实现

按权排好序的边序列预先构造，运行中不变。算法维护两项状态：

- 已选边集 A：初值为空；
- 并查集：其中的集合就是森林 (V, A) 的分量，初始时每个顶点自成一个集合。

按排序依次取边 $\{u, v\}$，比较 $\mathrm{find}(u)$ 与 $\mathrm{find}(v)$：

- 代表不同：把这条边加入 A，执行 $\mathrm{union}(u, v)$；
- 代表相同：选入会成环，跳过。

选满 $n-1$ 条边时可提前停止，否则扫完全部边。

算法 4.1　Kruskal 算法

输入：有限无向加权图。输出：最小生成森林的边集 A。

1. 把全部边按权从小到大排序，平局按固定的边序；令 $A=\varnothing$，为每个顶点建立一个单元素集合。
2. 按排序依次取边 $\{u, v\}$：若 u, v 的代表不同，把这条边加入 A 并合并两个集合；若代表相同，跳过。
3. 选满 $n-1$ 条边时可提前停止；否则扫描完全部边，输出的 A 是最小生成森林，集合个数即连通分量数。

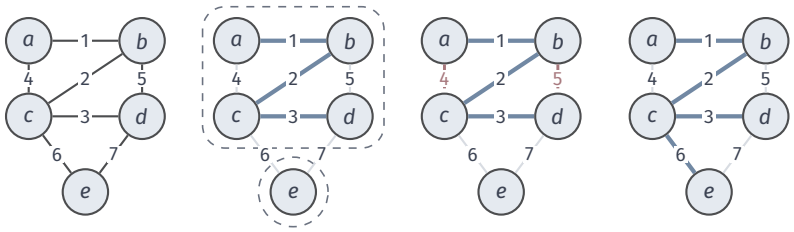
执行过程　取五个顶点 a, b, c, d, e 与七条边，权为 $ab=1$、$bc=2$、$cd=3$、$ac=4$、$bd=5$、$ce=6$、$de=7$，它们已按权排好序。表 4.1逐边列出判断与分量的变化。ab、bc、cd 的两端都属于不同分量，依次选入，此时分量为 $\{a, b, c, d\}$ 与 $\{e\}$，累计成本 6；ac 与 bd 的两端都已在 $\{a, b, c, d\}$ 中，选入会成环，跳过；ce 连接两个分量，选入后只剩一个分量，已选满 $4=n-1$ 条边，停止，de 不必再检查。结果为 $\{ab, bc, cd, ce\}$，总成本 $1+2+3+6=12$。图 4.3画出了这一过程。

4.3.3 算法分析

正确性　选入的边是否安全，由割性质判断。

表 4.1 Kruskal 算法在五点七边图上的状态变化。“两端分量”指处理该边之前两个端点所在的并查集集合；选满 4 条边后停止，de 未被检查。

边	权	两端分量	动作	处理后的分量与累计成本
ab	1	$\{a\}$，$\{b\}$	选入	$\{a,b\},\{c\},\{d\},\{e\}$；1
bc	2	$\{a,b\}$，$\{c\}$	选入	$\{a,b,c\},\{d\},\{e\}$；3
cd	3	$\{a,b,c\}$，$\{d\}$	选入	$\{a,b,c,d\},\{e\}$；6
ac	4	同为 $\{a,b,c,d\}$	跳过	不变；6
bd	5	同为 $\{a,b,c,d\}$	跳过	不变；6
ce	6	$\{a,b,c,d\}$，$\{e\}$	选入	$\{a,b,c,d,e\}$；12



(a) 原图　(b) 选 ab, bc, cd　(c) 跳过 ac, bd　(d) 选 ce，成本 12

图 4.3 Kruskal 算法在五点七边图上的过程。(a) 原图，边旁为权。(b) 依次选入 ab, bc, cd（蓝色粗边），虚线框标出当前分量 $\{a,b,c,d\}$ 与 $\{e\}$。(c) ac 与 bd 的两端已在同一分量，选入会成环，跳过（红色虚线）。(d) 选入 ce，得到最小生成树 $\{ab, bc, cd, ce\}$，总成本 12。

定理 4.3 Kruskal 算法的正确性 对有限无向加权图，算法 4.1输出一个最小生成森林；图连通时输出最小生成树。

证明 先设图连通，证明不变量“A 包含在某棵最小生成树中”。初始 $A = \varnothing$，结论成立。设算法即将选入边 $e = \{u, v\}$，取 S 为森林 (V, A) 中 u 所在的分量，则 $v \notin S$，割 $(S, V \setminus S)$ 与 A 相容。考察任意一条跨越这个割、权小于 w_e 的边 g。它排在 e 之前，已被处理过：若当时被选入，它就在 A 中，而 A 中没有跨割的边；若当时被跳过，它的两端当时已在同一分量，分量只会合并而不会拆开，现在仍在同一分量，也不能跨割。所以跨越该割的边权都不小于 w_e，e 是轻边，由引理 4.2，选入 e 后不变量仍成立。

扫描结束时，若原图中某条边连接了 (V, A) 的两个不同分量，处理到它时两端必属不同分量，它就会被选入，矛盾。所以 (V, A) 的分量与原图的分量相同，图连通时 A 是生成树。它包含在某棵最小生成树中，又与之边数相同，因而本身就是最小生成树。图不连通时，对每个分量分别应用上面的论证，结论对最小生成森林同样成立。 ■

复杂度 排序需要 $O(m\log m)$ 时间。之后至多处理 m 条边，每条边做两次 find，选入时再做一次 union，共 $O(m)$ 次并查集操作。只用按大小合并时，由定理 2.6，每次操作 $O(\log n)$，这部分为 $O(m\log n)$；由于 $m \le n^2$，$\log m \le 2\log n$，总时间为 $O(m\log n)$，空间为边表与并查集的 $O(n+m)$。若再配合路径压缩，并查集操作的摊还开销几乎为常数，此时排序成为主要开销。算法的正确性只依赖割性质，与并查集采用哪种实现无关。

4.4 Prim 算法

4.4.1 问题与思路

Kruskal 算法同时维护许多分量。另一种做法是只让一棵树生长：从一个顶点出发，每次把一个外部顶点用最便宜的边接到树上。这时取的割就是“树内与树外”，只要能随时知道每个外部顶点离树最近的那条边，就能找到这个割的轻边。

4.4.2 算法实现

输入是图的邻接表与起点 r。算法维护三项状态：

- 树的顶点集 S：初值为 $\{r\}$；
- 键 $\mathrm{key}(v)$ 与父点 $p(v)$：外部顶点 v 与当前树之间最便宜的边的权及这条边在树内的端点，初始时 r 的邻居 v 取 $w_{r,v}$ 与 r，其余键为 $+\infty$。键是一条连接边的权，不是 Dijkstra 算法中的累计长度；
- 优先队列：按键存放外部顶点，支持取最小与降键，实现与第 3.5节相同。

当 $S \neq V$ 时，每一轮做两件事：

- 取出：把键最小的外部顶点 u 移出队列并加入 S，把边 $\{p(u),u\}$ 加入树；
- 更新：对 u 的每个外部邻居 v，若 $w_{u,v} < \mathrm{key}(v)$，令 $\mathrm{key}(v) = w_{u,v}$、$p(v) = u$，并在队列中降键。

$S = V$ 时停止。

算法 4.2 Prim 算法

输入：有限无向连通加权图、起点 r。输出：最小生成树的边集。

1. 令 $S = \{r\}$；对 r 的每个邻居 v，令 $\mathrm{key}(v) = w_{r,v}$、$p(v) = r$，其余外部顶点的键为 $+\infty$。
2. 当 $S \neq V$ 时，取出键最小的外部顶点 u，把 u 加入 S，把边 $\{p(u),u\}$ 加入树。
3. 对 u 的每个外部邻居 v，若 $w_{u,v} < \mathrm{key}(v)$，令 $\mathrm{key}(v) = w_{u,v}$、$p(v) = u$。

执行过程 在与 Kruskal 算法相同的五点图上从 a 出发，各步状态见表 4.2与图 4.4。开始时 key(b) = 1、key(c) = 4。取出 b，选入 ab；b 的邻边把 c 的键从 4 改为 2，并给 d 设键 5。取出 c，选入 bc；d 的键从 5 改为 3，e 得到键 6。取出 d，选入 cd；$de = 7$ 不优于 e 现有的键 6，不更新。最后取出 e，选入 ce。所得的树与 Kruskal 算法相同，成本 12。

表 4.2 Prim 算法从 a 出发的状态。每行是取出一个顶点并更新键之后的情形，括号内为键对应的父点，“—”表示该顶点已入树。

取出	选入的边	key(b)	key(c)	key(d)	key(e)
初始（$S = \{a\}$）	—	1（a）	4（a）	$+\infty$	$+\infty$
b	ab	—	2（b）	5（b）	$+\infty$
c	bc	—	—	3（c）	6（c）
d	cd	—	—	—	6（c）
e	ce	—	—	—	—

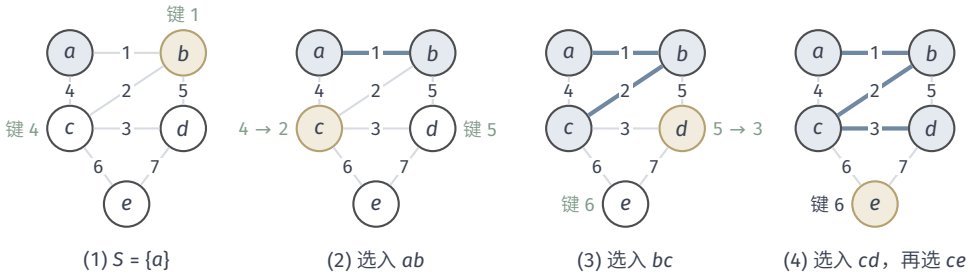


图 4.4 Prim 算法从 a 出发的过程。每个面板中蓝色顶点与蓝色粗边是当前树，外部顶点旁标出键，绿色表示刚被改善的键，黄色为下一个取出的顶点。依次选入 ab, bc, cd, ce，各步加入的边恰是当前树顶点集所形成割的轻边；最终的树与 Kruskal 算法相同，成本 12。

换一个起点，选边次序会变。从 e 出发时，c 与 d 的键分别为 6 与 7；先选入 $ec = 6$，c 的邻边使 b、a、d 的键变为 2、4、3；再选入 $cb = 2$，a 的键降为 1；随后选入 $ba = 1$ 与 $cd = 3$。所选边权依次为 $6, 2, 1, 3$，并不递增。这与 Kruskal 算法的全局排序不同：Prim 算法每次只在当前割上取最轻的边，不同时刻面对的是不同的割。

4.4.3 算法分析

正确性 每个外部顶点的键始终等于它与 S 之间最便宜的边权：初始时成立；u 入树后，新增的跨割边只有从 u 出发的那些，更新一步恰好逐一比较它们。于是取出的最小键对应割 $(S, V \setminus S)$ 的一条轻边。令 A 为当前树的边集，A 的边都在 S 内部，割与 A 相容；由引理 4.2，若 A 包含在某棵最小生成树中，加入新边后仍然如此。从 $A = \varnothing$ 出发归纳，选满 $n - 1$ 条边时得到最小生成树。

复杂度 用数组保存键、每次线性扫描找最小时，n 次取最小共 $O(n^2)$，每条边至多引起一次键的比较与更新，总时间为 $O(n^2 + m)$，适合稠密图。用支持降键的索引二叉堆时，取最小与降键各需 $O(\log n)$，总时间为 $O((n + m)\log n)$，适合稀疏图。两种实现的额外空间都是 $O(n)$。

非连通情形 图不连通时，一棵树长满所在分量后，外部顶点的键全为 $+\infty$，这时从任一尚未访问的顶点重新开始，得到最小生成森林，各分量的父边与成本分别保留。

4.5 方法比较与结果检验

两种算法读取边与维护状态的方式不同。Kruskal 算法对全部边排序，按全局次序检查，用并查集维护多个分量，适合边以列表给出的稀疏图。Prim 算法只读取新入树顶点的邻边，用键数组或堆维护局部信息；在邻接矩阵存储的稠密图上，数组版本的 $O(n^2)$ 开销与读入图同阶。孰优孰劣取决于图的稠密程度、边的组织方式和所用数据结构，不能只凭算法名称判断。

检验一份声称的最小生成森林时，先确认它是一个可行的生成森林：所选的每条边都来自原图；每个原连通分量的顶点都被覆盖；所选边无环，可以用并查集逐条加入核对；选边数等于 n 减去分量数。然后重算总成本。这些检查只证明结果是一个生成森林，不能证明它最优。

最优性另有一份不依赖算法过程的判据。对每条未被选中的边 $\{u, v\}$，考察树上 u 到 v 的唯一路径。若路径上有一条边的权大于 $w_{u,v}$，用这条非树边替换它，就得到成本更低的生成树，结果必不最优。反过来，没有这样的边也足以保证最优。

定理 4.4　非树边条件 设 T 是有限无向连通加权图的一棵生成树。若对每条非树边 $f = \{u, v\}$，T 中 u 到 v 路径上的每条边 g 都满足 $w_g \le w_f$，则 T 是最小生成树。

证明 在全部最小生成树中取一棵与 T 公共边最多的 $T^\star$。若 $T^\star = T$，结论成立。否则取 $f \in T^\star \setminus T$。从 $T^\star$ 中删去 f，顶点分成两块，f 是 $T^\star$ 中唯一跨越这两块的边。f 不在 T 中，是 T 的非树边，T 中连接 f 两端的路径从一块出发到达另一块，必含一条跨越两块的边 g；$g \ne f$，所以 $g \notin T^\star$。由条件 $w_g \le w_f$，$T^\star - f + g$ 仍是生成树，成本不超过 $w(T^\star)$，因而也是最小生成树，而它与 T 的公共边比 $T^\star$ 多一条，与 $T^\star$ 的取法矛盾。 ■

在五点图上核对：非树边 $ac = 4$ 在树上对应路径 a–b–c，边权 $1, 2$；$bd = 5$ 对应 b–c–d，边权 $2, 3$；$de = 7$ 对应 d–c–e，边权 $3, 6$。每条非树边

都不轻于对应路径上的边，所以 $\{ab, bc, cd, ce\}$ 最优。这一检查对每条非树边沿树找一次路径，朴素实现为 $O(nm)$。

小规模的例子还可以枚举全部生成树作对照：这个图共有 21 棵生成树，成本 12 的只有上面这一棵，次优的成本为 13。但两种算法在某个例子上给出相同的树，只是一致性的旁证，不能当作一般正确性的证明。

4.6 本章小结

章首的问题是：能否逐条选边，只看局部就断定没有选错。割性质给出肯定的回答：若已选边集包含在某棵最小生成树中，那么在任何与它相容的割上，选入一条轻边后仍包含在某棵最小生成树中。证明靠交换论证，即加入轻边、删去环上另一条跨割边，成本不增。安全不等于必选：权相同时轻边可能有多条，不同选择通向不同的最优树，最优成本却相同。

两种算法只是选取的割不同。Kruskal 算法按全局边权次序检查，以当前森林的一个分量为割的一侧，用并查集判断成环，总时间为 $O(m \log n)$，配合路径压缩时主要开销是排序；它自然地给出不连通图的最小生成森林。Prim 算法只让一棵树生长，以树内与树外为割，键记录每个外部顶点的最便宜接入边。数组实现为 $O(n^2 + m)$，适合稠密图；二叉堆实现为 $O((n + m) \log n)$，适合稀疏图。

最小生成树最小化的是整组边的总造价，不是两点间路线的长度，树上两点间的路径未必是最短路，键也不同于 Dijkstra 算法的累计距离。负权不造成困难，因为每棵生成树的边数都是 $n - 1$，所有边权同加一个常数不改变比较结果。结果的检验分两步：先核对所选边构成生成森林，再核对每条非树边都不轻于它在树上所对应路径中的边，后者足以保证最优。

把选边次序改为从大到小，同样的论证就求出最大生成树；第 12章以团的交集大小为边权构造团树时，调用的正是它。下一章转向另一种选择对象：不再挑出一组边，而是给每条弧分配一个流量。

第 5 章

最大流与最小割

前两章各选出一个结构：一条路线或一棵树。现在设想从水厂向用户输水，每段管道有输送上限，水可以并行、汇合、分流，问单位时间内最多能送出多少水。这次选择的对象是每条弧上的流量，它们要同时满足容量上限和中途不积存的条件。本章要回答两个问题：怎样逐步改进一个输送方案，以及怎样确认一个方案已经不能再改进。

第 5.1节先给出上界：任何一组把源与汇隔开的管道，其容量之和都限制了输送量，这组管道称为割。第 5.2节处理改进：先选的路线可能占用别的路线需要的容量，所以要允许把已送出的流量改道。记录“还能追加多少、能撤回多少”的残量网络把改进统一为找一条增广路径，找不到时恰好能读出一个与流值相等的割，这就是最大流最小割定理的内容。

具体算法的差别在于怎样找路。第 5.3节的 Edmonds–Karp 算法每次用广度优先遍历找一条弧数最少的路，增广次数因此与容量数值无关；第 5.4节的 Dinic 算法一次分层后在层内把能走的最短路都堵住，再重建层次。第 5.5节比较两者，并说明怎样用流与割核对结果。本章记顶点数为 n、原弧数为 a，残量网络沿用第 2.1节的广度优先遍历与邻接表。

5.1 流、容量与割

设 $G=(V,E)$ 是有限有向图，指定两个不同的顶点：源点 s 与汇点 t。每条弧 $u\to v$ 带有有限非负实数容量 $c_{u,v}$，表示该弧单位时间内最多能通过的量。这样的四元组 (G,c,s,t) 称为**流网络**（flow network）。

定义 5.1　流与流值　流网络上的一个流（flow）是给每条弧 $u\to v$ 赋予实数 $x_{u,v}$ 的一组数 $\boldsymbol{x}=(x_{u,v})$，满足

$$0\le x_{u,v}\le c_{u,v}\quad \text{对每条弧 } u\to v, \tag{5.1}$$

$$\sum_{u:\,u\to v} x_{u,v}=\sum_{w:\,v\to w} x_{v,w}\quad \text{对每个 } v\in V\setminus\{s,t\}. \tag{5.2}$$

前者称为容量约束，后者称为**流守恒**（flow conservation）。源点的净流出

$$\mathrm{val}(\boldsymbol{x}) = \sum_{w\,:\,s\to w} x_{s,w} - \sum_{u\,:\,u\to s} x_{u,s}$$

称为**流值**（flow value）。流值最大的流称为**最大流**（maximum flow）。

中间顶点只转运、不积存。把各顶点的净流出（流出减流入）全部相加，每条弧的流量一正一负，总和为零；中间顶点的净流出都是零，于是源点的净流出等于汇点的净流入，$\mathrm{val}(\boldsymbol{x})$ 也是 t 收到的净量。流值不是全图各弧流量的总和：一单位流量经过三条弧到达汇点，只算一单位流值。

为了估计流值能有多大，把顶点分成包含 s 的一侧 S 与包含 t 的一侧 $\bar{S} = V \setminus S$，称 $(S, \bar{S})$ 为一个 s–t 割。这沿用第 4.2节割的含义，只是要求源、汇分居两侧。割的**割容量**（cut capacity）只累计从源侧跨向汇侧的弧：

$$c(S, \bar{S}) = \sum_{u\in S,\ v\in\bar{S},\ u\to v} c_{u,v}.$$

从汇侧指回的弧不计入。直观上，从 s 到 t 的输送都要穿过这个割，总量不能超过这些弧的容量之和。

引理 5.2　流值不超过割容量　对任意流 $\boldsymbol{x}$ 与任意 s–t 割 $(S, \bar{S})$，有

$$\mathrm{val}(\boldsymbol{x}) = \sum_{u\in S,\,v\in\bar{S}} x_{u,v} - \sum_{v\in\bar{S},\,u\in S} x_{v,u} \le c(S, \bar{S}). \tag{5.3}$$

证明　对 S 中的全部顶点把净流出相加。s 贡献 $\mathrm{val}(\boldsymbol{x})$，其余顶点都是中间顶点（$t \notin S$），贡献为零，所以和等于 $\mathrm{val}(\boldsymbol{x})$。另一方面，两端都在 S 内的弧在起点计正、在终点计负，相互抵消；剩下的只有跨割的弧：从 S 指向 $\bar{S}$ 的弧计正，从 $\bar{S}$ 指回 S 的弧计负。这就得到式 (5.3)中的等式。再由容量约束，第一个和不超过 $c(S, \bar{S})$，第二个和不小于零，得到不等式。 ■

于是流值等于跨过任意一个割的净流量，每个割的容量都是流值的上界。若某个流与某个割数值相等，这个流就不可能再增大，这个割也不可能更小。图 5.1给出一例：取 $S = \{s, b\}$，净流量 $3 + 3 - 1 = 5$ 恰为流值，不超过割容量 6。

5.2　残量网络与增广

候选的流有无穷多个，只能从一个可行流出发逐步改进。最朴素的办法是找一条每条弧都有空余的 s–t 路径多送一些，但先选的路线可能占用别的

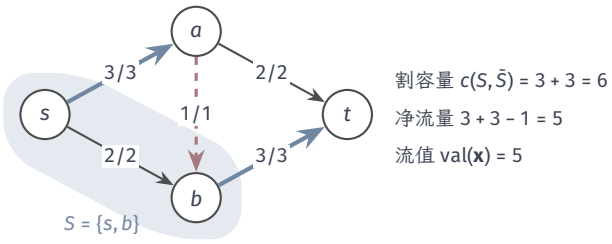


图 5.1 流网络、流与割。弧旁“x/c”为流量与容量，流值为 5。浅色底为源侧 $S=\{s,b\}$：跨出弧 $s\to a$、$b\to t$（蓝色粗弧）计入割容量 $3+3=6$；指回弧 $a\to b$（红色虚线）不计容量，其流量从净流量中扣除。

路线需要的容量。取弧 $s\to a$、$s\to b$、$a\to b$、$a\to t$、$b\to t$，容量都为 1。若先沿 $s\to a\to b\to t$ 送一单位，三条弧都满了，剩下的 $s\to b$ 与 $a\to t$ 连不成路径，只会追加的办法停在流值 1；而 $s\to a\to t$ 与 $s\to b\to t$ 本可送出 2。补救的办法是允许撤回旧流量，把撤回也当作一种可走的“通道”。

给定流 $\boldsymbol{x}$，对每条原弧 $u\to v$ 建立两条残量弧：正向残量弧 $u\to v$，容量 $c_{u,v}-x_{u,v}$，表示还能追加多少；反向残量弧 $v\to u$，容量 $x_{u,v}$，表示能撤回多少。只保留容量为正的残量弧，得到的有向图称为**残量网络**（residual network），记作 $G_{\boldsymbol{x}}$，残量弧的容量记作 $r_{\boldsymbol{x}}$。例如原弧容量为 3、当前流量为 2 时，正向残量为 1、反向残量为 2（图 5.2）：沿 $u\to v$ 再送 1 表示追加，沿 $v\to u$ 送 2 表示把原弧上的流量全部撤回，并不要求原网络中真有一条从 v 到 u 的管道。

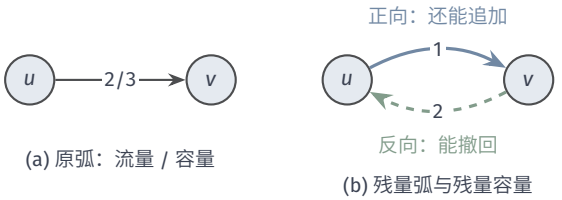
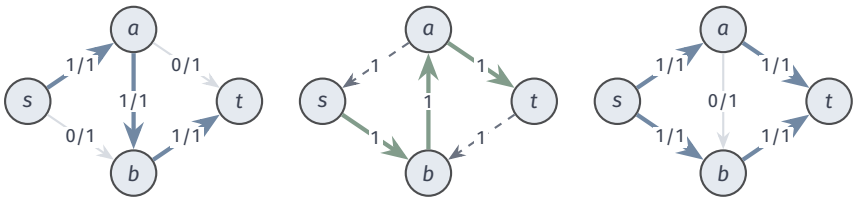


图 5.2 一条原弧对应两条残量弧。(a) 原弧 $u\to v$ 容量 3、流量 2。(b) 正向残量弧（蓝色）容量 1，可追加；反向残量弧（绿色虚线）容量 2，可撤回。流量为 0 时无反向弧，满流时无正向弧。

回到单位容量的例子。第一次送流后，残量网络中出现反向残量弧 $b\to a$，于是有路径 $s\to b\to a\to t$：在 $s\to b$ 与 $a\to t$ 上各追加一单位，同时把 $a\to b$ 上的流量撤回一单位。更新后 $a\to b$ 为 0，其余四条弧各为 1，流值为 2（图 5.3）：第一次经过 $a\to b$ 的那一单位被“改道”到了 $a\to t$。



(a) 第一次：$s \to a \to b \to t$ 流值 1　(b) 残量网络中的路径 $s \to b \to a \to t$　(c) 撤回 $a \to b$ 上的流量 流值 2

图 5.3　单位容量网络中撤回旧流量。(a) 沿 $s \to a \to b \to t$ 送一单位（蓝色粗弧）后，只用正向残量已无 s–t 路径。(b) 残量网络中绿色路径 $s \to b \to a \to t$ 经过反向残量弧 $b \to a$（灰色为其余反向弧）。(c) 更新后 $a \to b$ 的流量被撤回，流值为 2。

有了反向残量弧，改进一个流就统一成了找路。残量网络 $G_{\boldsymbol{x}}$ 中从 s 到 t 的简单路径 P 称为关于 $\boldsymbol{x}$ 的**增广路径**（augmenting path）。P 上残量容量的最小值

$$\Delta = \min_{(u\to v)\in P} r_{\boldsymbol{x}}(u,v) > 0$$

称为 P 的**瓶颈容量**（bottleneck capacity），即这条路一次最多能多送的量。沿 P 增广 Δ，是指对 P 上的每条残量弧：若它是原弧 $u \to v$ 的正向残量，令 $x_{u,v} \leftarrow x_{u,v} + \Delta$；若它是原弧 $v \to u$ 的反向残量，令 $x_{v,u} \leftarrow x_{v,u} - \Delta$。

引理 5.3　沿增广路径更新保持可行　设 $\boldsymbol{x}$ 是流，P 是增广路径，瓶颈容量为 Δ。沿 P 增广 Δ 后得到的 $\boldsymbol{x}'$ 仍是流，并且 $\mathrm{val}(\boldsymbol{x}') = \mathrm{val}(\boldsymbol{x}) + \Delta$。

证明　容量约束：正向残量弧满足 $\Delta \le c_{u,v} - x_{u,v}$，追加后不超过容量；反向残量弧满足 $\Delta \le x_{v,u}$，撤回后不小于零。流守恒：P 经过每个中间顶点 v 恰好一次，一条残量弧进入 v，一条离开 v。进入的那条若是正向残量，v 的流入增加 Δ；若是反向残量，即撤回一条从 v 出发的原弧上的流量，v 的流出减少 Δ。两种情形都使 v 的净流出减少 Δ。同理，离开的那条使 v 的净流出增加 Δ，两者相抵。最后，P 的第一条残量弧离开 s，使 s 的净流出增加 Δ，而 P 不再回到 s，所以流值增加 Δ。 ■

引理直接给出一个改进框架：从零流出发，只要残量网络中还有增广路径就沿它增广。这个框架称为**Ford–Fulkerson 方法**（Ford–Fulkerson method）。称它为“方法”，是因为它没有规定选哪条路，后面两种算法给出不同的规定。

算法 5.1 Ford–Fulkerson 增广框架

输入：流网络 $(\mathcal{G}, c, s, t)$。输出：流 $\boldsymbol{x}$ 与割 $(S, \bar{S})$。

1. 令 $\boldsymbol{x} = \mathbf{0}$。
2. 在 $\mathcal{G}_{\boldsymbol{x}}$ 中从 s 搜索；若到达不了 t，令 S 为搜索到的顶点集，输出 $\boldsymbol{x}$ 与 $(S, \bar{S})$，停止。
3. 取一条增广路径 P，求瓶颈 Δ，按引理 5.3更新 $\boldsymbol{x}$，回到第 2 步。

框架能否停下来，取决于每次能送多少。

定理 5.4 整数容量下的有限终止 若所有容量都是非负整数，则无论怎样选择增广路径，算法 5.1至多经过 $C_s = \sum_{w:\, s\to w} c_{s,w}$ 次增广就停止，并且全程每条弧上的流量都是整数。

证明 零流是整数的；若当前流量都是整数，则残量 $c_{u,v} - x_{u,v}$ 与 $x_{u,v}$ 都是整数，瓶颈 Δ 是正整数，更新后流量仍为整数。于是每次增广使流值至少增加 1。另一方面，取 $S = \{s\}$，由引理 5.2，流值不超过 C_s，所以增广至多 C_s 次。 ■

容量为有理数时，乘以公分母即化为整数情形。对一般实容量，这个论证不能照搬：增加量可能越来越小。已知存在含无理容量的网络，任意选路的框架永不停止，流值甚至收敛到小于最大流值的数。

框架停止时没有增广路径。这时的流是最大的，而且可以从残量网络中直接读出一个与流值相等的割。

定理 5.5 增广路径判据 设 $\boldsymbol{x}$ 是流网络上的一个流，则下列三条等价：

1. $\boldsymbol{x}$ 是最大流；
2. 残量网络 $\mathcal{G}_{\boldsymbol{x}}$ 中不存在增广路径；
3. 存在某个 s–t 割 $(S, \bar{S})$，使 $\mathrm{val}(\boldsymbol{x}) = c(S, \bar{S})$。

此时 $(S, \bar{S})$ 是割容量最小的 s–t 割，可取 S 为 $\mathcal{G}_{\boldsymbol{x}}$ 中从 s 可达的顶点集。

证明 由 1 推 2：若存在增广路径，由引理 5.3可得流值更大的流，与 $\boldsymbol{x}$ 最大矛盾。

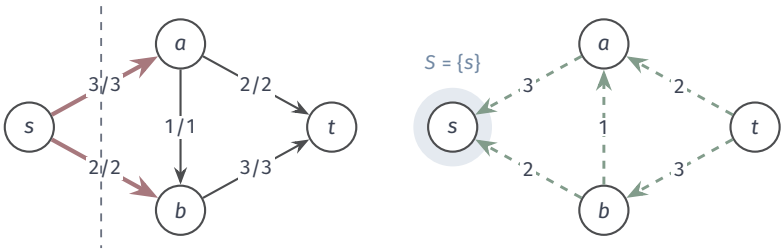
由 2 推 3：令 S 为 $\mathcal{G}_{\boldsymbol{x}}$ 中从 s 可达的顶点集。$s \in S$，而没有增广路径说明 $t \notin S$。若 $u \in S$、$v \in \bar{S}$ 且 $u \to v$ 是原弧，则必有 $x_{u,v} = c_{u,v}$，

否则正向残量弧 $u \to v$ 存在，v 就从 s 可达了。若 $v \in \bar{S}$、$u \in S$ 且 $v \to u$ 是原弧，则必有 $x_{v,u} = 0$，否则反向残量弧 $u \to v$ 存在，同样使 v 可达。代入式 (5.3)，第一个和等于 $c(S,\bar{S})$，第二个和为零，故 $\mathrm{val}(\boldsymbol{x}) = c(S,\bar{S})$。

由 3 推 1：对任意流 $\boldsymbol{y}$，引理 5.2给出 $\mathrm{val}(\boldsymbol{y}) \le c(S,\bar{S}) = \mathrm{val}(\boldsymbol{x})$，所以 $\boldsymbol{x}$ 最大。同一不等式对任意割 $(S',\bar{S}')$ 给出 $c(S',\bar{S}') \ge \mathrm{val}(\boldsymbol{x}) = c(S,\bar{S})$，所以 $(S,\bar{S})$ 的割容量最小。 ■

割容量最小的 s–t 割称为**最小割**（minimum cut）。定理把“是否最优”化成在残量网络中做一次遍历就能回答的问题；停止时的流与割互为证书，可以逐弧核对，不必相信算法过程。

结合定理 5.4，整数容量的网络存在每条弧流量都为整数的最大流，第 6.1节会用到这一点。图 5.4是同一网络的最大流及其证书。需要注意，判据只检验给定的流；一般实容量下能否有限步到达这样的流，要由具体的选路规则保证。



(a) 最大流，流值 5；割 $(\{s\},\{a,b,t\})$ 跨出弧满流，割容量 3 + 2 = 5

(b) 残量网络只剩反向残量弧 s 无出弧，没有增广路径

图 5.4　最大流与最小割互为证书。(a) 最大流，弧旁为“流量/容量”，流值为 5；虚线为割 $(\{s\},\{a,b,t\})$，跨出弧（红色）均满流，割容量 $3+2=5$。(b) 残量网络只剩反向残量弧（绿色虚线），s 没有出弧，可达集为 $S=\{s\}$（浅色底）。

实现时，后面两种算法共用同一种表示。每条原弧 $u \to v$ 登记一对残量边，分别挂在 u、v 的邻接表上，互相指向对方。每条残量边 e 记录终点、当前残量 r_e 和同伴边 $\bar{e}$ 的编号；正向边初值 $r_e = c_{u,v}$，反向边初值为 0。沿 e 送 Δ 时令 $r_e \leftarrow r_e - \Delta$、$r_{\bar{e}} \leftarrow r_{\bar{e}} + \Delta$，两条边总是成对更新，原弧流量随时可读作其反向边的残量。

真实的反向弧与平行弧各有自己的边对，“真实反向输送”与“撤回流量”不会混为一谈。因此搜索时要记住到达每个顶点所经的边编号，只记前一个顶点无法确定改哪条边。

5.3 Edmonds–Karp 算法

Edmonds–Karp 算法保留残量网络与增广更新，只规定每轮用广度优先遍历选出弧数最少的增广路径。这条规则决定了增广次数能否摆脱容量数值的影响。

5.3.1 主要思想

把 s 看作供水处、t 看作收水处，a, b 为中转点。四条外弧 $s \to a$、$s \to b$、$a \to t$、$b \to t$ 容量都为 4，中间弧 $a \to b$ 容量为 1，最大流值为 8。先选 $s \to a \to b \to t$，瓶颈是中间弧，只能送 1；再选 $s \to b \to a \to t$，经反向残量弧 $b \to a$ 撤回那一单位，又只增加 1。两步后中间弧恢复空闲，同样两条路可以再选。一直交替，送满 8 要增广 8 次（图 5.5(a)）。若外弧容量改为 10^6，就要 2×10^6 次。

回到零流，一层一层看：一步到 a, b，两步到 t。这样首先找到两弧路 $s \to a \to t$，送 4；再找到 $s \to b \to t$，又送 4，两次就达到 8（图 5.5(b)）。这就是最短增广路规则：在残量网络中用广度优先遍历找弧数最少的 s–t 路径。遍历决定走哪条路，瓶颈决定这次送多少；路径长短只数弧的条数，容量不参与比较。

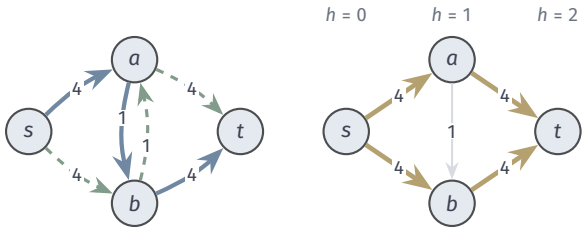


图 5.5　选路规则决定增广次数。外弧容量 4，中间弧 $a \to b$ 容量 1。(a) 交替选蓝色路径 $s \to a \to b \to t$ 与绿色虚线路径 $s \to b \to a \to t$（经反向残量 $b \to a$），每次送 1，共 8 次。(b) 按层数 h 先找到两条两弧路径（黄色），两次各送 4。

一般情况下，弧数最少的路未必瓶颈最大；规则的保证来自另一处：增广不会让任何顶点离 s 更近，于是同一条弧不能频繁地成为瓶颈，这在后面证明。Sedgewick 与 Wayne 的最大流实现正是用广度优先遍历选路[1]。

[1] R. Sedgewick, K. Wayne, *Maximum Flow*, June 14, 2017, URL: https://algs4.cs.princeton.edu/64maxflow/

5.3.2 算法流程

算法状态与初始化　手工找路要记住：哪些管道还能追加或撤回、本轮找到了哪些点、各点经哪条边到达、共送出多少。残量边对的结构（邻接表、终点与同伴编号）建图后不变，不算状态。算法维护：

- 残量 r_e：每条残量边一个，正向边初值为容量，反向边初值为 0，跨轮保留；
- 流值 F：初值为 0，跨轮累加；
- 访问标记与队列：每轮开始时清空，只标记 s、队列只含 s；
- 前驱边 $\pi(v)$：到达 v 所经的残量边编号，每轮开始时清空。

一次增广与停止　每一轮做三件事：

- 搜索：按广度优先次序取出顶点 u，扫描 u 的残量边 e；若 $r_e > 0$ 且终点 v 未标记，就标记 v、记 $\pi(v) = e$、让 v 入队。t 被标记即可结束搜索。
- 求瓶颈：从 t 沿 π 回溯到 s，取路径上残量的最小值 Δ。
- 更新：再回溯一次，对路径上每条边 e 令 $r_e \leftarrow r_e - \Delta$、$r_{\bar{e}} \leftarrow r_{\bar{e}} + \Delta$，并令 $F \leftarrow F + \Delta$。

某一轮标记不到 t 时停止，被标记的顶点集就是割的源侧 S。增广会让一些边耗尽、一些反向边出现，所以每轮都在更新后的残量上重新搜索。

算法 5.2　Edmonds–Karp 算法

输入：用残量边对存储的流网络，源点 s，汇点 t。输出：流值 F、各原弧流量与割 $(S, \bar{S})$。

1. 每对残量边中，正向边 $r_e = c_{u,v}$，反向边 $r_{\bar{e}} = 0$；$F = 0$。
2. 清空标记与前驱边，标记 s，令队列 $Q = [s]$。
3. 当 Q 非空且 t 未标记时，取出队首 u；对 u 的每条残量边 e（终点 v），若 $r_e > 0$ 且 v 未标记，则标记 v，置 $\pi(v) = e$，v 入队。
4. 若 t 未标记，令 S 为已标记顶点集；每条原弧的流量取其反向边的残量；输出 F、流量与 $(S, \bar{S})$，停止。
5. 从 t 沿 π 回溯到 s，求 $\Delta = \min r_e$；再回溯一次，对路径上每条 e 置 $r_e \leftarrow r_e - \Delta$、$r_{\bar{e}} \leftarrow r_{\bar{e}} + \Delta$；$F \leftarrow F + \Delta$，回到第 2 步。

运行示例　继续主要思想中外弧容量 4、中间弧容量 1 的网络。边对按 $s \to a$、$s \to b$、$a \to b$、$a \to t$、$b \to t$ 的次序登记，s 的邻接表先列 a 后列 b；表 5.1按出队逐行记录。

- 第 1 轮：s 出队，发现 a, b；a 出队，反向边 $a \to s$ 残量为 0，b 已标记，经 $a \to t$ 发现 t。回溯得 $s \to a \to t$，瓶颈 4，两条边耗尽，反向边残量变为 4。
- 第 2 轮：$s \to a$ 已耗尽，s 只发现 b；b 出队，经 $b \to t$ 发现 t。路径 $s \to b \to t$，瓶颈 4，流值为 8。
- 第 3 轮：s 的两条出边残量都为 0，只标记了 s，停止，$S = \{s\}$。

割 $(\{s\}, \{a, b, t\})$ 的容量 $4 + 4 = 8$ 等于流值，这是最大流。

表 5.1 Edmonds–Karp 算法在外弧容量 4、中间弧容量 1 的网络上的运行。每行是一次出队，“前驱边”为新记录的 π。

轮	出队	出队后队列	前驱边	更新	F
1	s	$[a,b]$	$\pi(a)=s\to a$，$\pi(b)=s\to b$	—	0
1	a	$[b,t]$	$\pi(t)=a\to t$	$\Delta=4$；$s\to a$、$a\to t$ 耗尽	4
2	s	$[b]$	$\pi(b)=s\to b$	—	4
2	b	$[t]$	$\pi(t)=b\to t$	$\Delta=4$；$s\to b$、$b\to t$ 耗尽	8
3	s	$[\,]$	—	t 不可达，$S=\{s\}$	8

图 5.3的单位容量例子在共同框架下先走三弧路、再撤回，用来说明边对的语义；Edmonds–Karp 算法在该网络上直接选两条两弧路，不需撤回。撤回在本算法中仍然必要，只是要在别的网络上才会出现。

5.3.3 正确性与复杂度

例中每次推送后中转点进出相等、各边不超容量；送满的 8 等于源点出弧容量和，不可能再超过。一般网络上，可行性由引理 5.3保证，停止时由定理 5.5得到最大流与最小割。需要另证的是任意有限实容量下的终止。

在主要思想的交替选路中，中间弧 $a\to b$ 被填满、撤空、再填满，反复成为瓶颈。要反复用 $a\to b$，就要反复用 $b\to a$；而在弧数最少的路上，$a\to b$ 要求 b 比 a 深一层，$b\to a$ 却要求 a 比 b 深一层。下面证明层数不会回退，于是每这样来回一次，a 的层数至少上升 2。记 $h_{\boldsymbol{x}}(v)$ 为 v 在 $\mathcal{G}_{\boldsymbol{x}}$ 中从 s 出发的最少弧数，不可达时为 $+\infty$。

引理 5.6　残量层数单调不减　设 $\boldsymbol{x}'$ 由 $\boldsymbol{x}$ 沿 $\mathcal{G}_{\boldsymbol{x}}$ 中一条弧数最少的增广路径增广得到，则对每个顶点 v 都有 $h_{\boldsymbol{x}'}(v)\ge h_{\boldsymbol{x}}(v)$。

证明　假设结论不成立，在所有满足 $h_{\boldsymbol{x}'}(v)<h_{\boldsymbol{x}}(v)$ 的顶点中取 $h_{\boldsymbol{x}'}(v)$ 最小的一个 v，显然 $v\neq s$。取 $\mathcal{G}_{\boldsymbol{x}'}$ 中一条 s 到 v 的最短路径，设其最后一条弧为 $u\to v$，则 $h_{\boldsymbol{x}'}(u)=h_{\boldsymbol{x}'}(v)-1$；由 v 的最小性，$h_{\boldsymbol{x}'}(u)\ge h_{\boldsymbol{x}}(u)$。

若 $u\to v$ 在增广前已在 $\mathcal{G}_{\boldsymbol{x}}$ 中，则 $h_{\boldsymbol{x}}(v)\le h_{\boldsymbol{x}}(u)+1\le h_{\boldsymbol{x}'}(u)+1=h_{\boldsymbol{x}'}(v)$，与假设矛盾。

否则 $u\to v$ 是增广新产生的残量弧。增广只会让路径上残量弧的反向弧获得正容量，所以 $v\to u$ 在增广路径上；这条路弧数最少，相邻顶点层数相差一，故 $h_{\boldsymbol{x}}(u)=h_{\boldsymbol{x}}(v)+1$。于是 $h_{\boldsymbol{x}}(v)=h_{\boldsymbol{x}}(u)-1\le h_{\boldsymbol{x}'}(u)-1=h_{\boldsymbol{x}'}(v)-2$，同样矛盾。 ■

每次增广中，瓶颈弧的残量降为零，称它在这次增广中饱和。

定理 5.7　Edmonds–Karp 算法的增广次数　在有限实容量的流网络上，Edmonds–Karp 算法至多进行 na 次增广就停止。

证明　残量弧按其对应的原弧与方向区分，至多有 $2a$ 条。固定其中一条 $u \to v$。它在某次增广中饱和时位于最短增广路径上，所以 $h_{\boldsymbol{x}}(v) = h_{\boldsymbol{x}}(u) + 1$，随后从残量网络中消失。它要再次出现，之后某次增广的路径必须经过反向弧 $v \to u$，设当时的流为 $\boldsymbol{x}''$，则 $h_{\boldsymbol{x}''}(u) = h_{\boldsymbol{x}''}(v) + 1$。由引理 5.6，

$$h_{\boldsymbol{x}''}(u) = h_{\boldsymbol{x}''}(v) + 1 \geq h_{\boldsymbol{x}}(v) + 1 = h_{\boldsymbol{x}}(u) + 2.$$

所以 $u \to v$ 相邻两次饱和之间，u 的层数至少增加 2。u 作为增广路径上的非汇点顶点，层数在 0 到 $n-2$ 之间，因此 $u \to v$ 至多饱和 $n/2$ 次。每次增广至少饱和一条残量弧，总增广次数不超过 $2a \cdot n/2 = na$。 ■

这个界只依赖顶点数与弧数，与容量的数值无关。由此得到本章的核心结论。

定理 5.8　最大流最小割定理　在有限非负实容量的流网络中，最大流存在，并且最大流值等于最小 s–t 割的割容量。

证明　由定理 5.7，Edmonds–Karp 算法有限步停止，输出流 $\boldsymbol{x}$ 在残量网络中没有增广路径。由定理 5.5，$\boldsymbol{x}$ 是最大流，且存在割 $(S, \bar{S})$ 使 $\mathrm{val}(\boldsymbol{x}) = c(S, \bar{S})$，这个割的割容量最小。 ■

这一结论通常称为**最大流最小割定理**（max-flow min-cut theorem）。

开销方面，先在线性时间内删去不与任何弧关联的非源汇孤立点，此后 $O(n + a) = O(a)$。每轮广度优先遍历与两次回溯用时 $O(a)$，乘以至多 na 轮，总时间为 $O(na^2)$ 次算术与比较操作，另加线性预处理，存储为 $O(n + a)$。

这个界不依赖整数容量，但它把每次加减比较算作一步，不等同于按输入位长计的复杂度。容量都是整数时，每次更新仍保持整数流，输出的最大流是整数流。

5.4 Dinic 算法

Edmonds–Karp 算法每送一条路就重做一次遍历，而相邻两次的层次常常相同。Dinic 算法在同一份层次内持续增广，直到这一层次下的最短路全都走不通才重建层次；关键是在层内找路时不重复试探已失败的方向。

5.4.1 主要思想

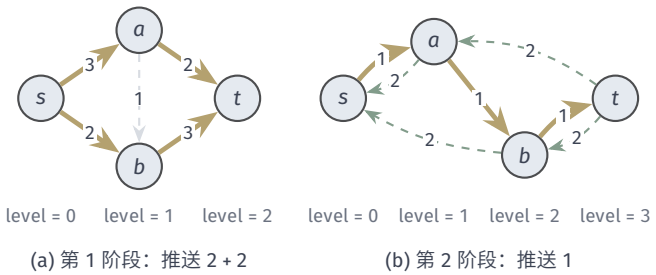
把 $s \to a$、$s \to b$、$a \to b$、$a \to t$、$b \to t$ 的容量依次设为 $3, 2, 1, 2, 3$，即图 5.1的网络。从零流分层，s 在第 0 层，a, b 在第 1 层，t 在第 2 层。沿 $s \to a \to t$ 送完 2 后，$s \to b \to t$ 还能送 2；两条路都只走向下一层，原层次足以指导第二次找路。

送完 4 后，$a \to t$ 与 $s \to b$ 都满了，只沿向下一层的边已到不了 t。但残量网络中还有 $s \to a \to b \to t$ 可送 1：它用到的 $a \to b$ 连接两个第 1 层的点，在旧层次里不可走。重新分层后 s, a, b, t 各在第 $0, 1, 2, 3$ 层，$a \to b$ 成了向下一层的边（图 5.6）。“这一层次下走不通”只说明最短的路用完了，不等于整个网络不能再增加流量。

把这两个观察写成定义。给定流 $\boldsymbol{x}$，令 $\mathrm{level}(v) = h_{\boldsymbol{x}}(v)$ 为广度优先层数。残量弧 $u \to v$ 若满足 $r_{\boldsymbol{x}}(u, v) > 0$ 且 $\mathrm{level}(v) = \mathrm{level}(u) + 1$，就称为可用边；全部可用边构成的子图称为**分层网络**（level graph），记作 $L_{\boldsymbol{x}}$。$L_{\boldsymbol{x}}$ 中的 s–t 路径恰好是 $\mathcal{G}_{\boldsymbol{x}}$ 中弧数最少的增广路径。

> **定义 5.9　阻塞流**　以残量为容量，$L_{\boldsymbol{x}}$ 上满足容量约束与内部守恒的流 $\boldsymbol{g}$ 若使 $L_{\boldsymbol{x}}$ 中每条 s–t 路径都至少有一条边满足 $g_e = r_{\boldsymbol{x}}(e)$，就称为 $L_{\boldsymbol{x}}$ 的**阻塞流**（blocking flow）。

阻塞流把本轮所有最短路至少堵住一处，但不要求是分层网络上的最大流：借反向弧还能多送的部分留给下一轮。Dinic 算法就是循环两步：建立分层网络，求它的一个阻塞流并加到当前流上；汇点不可达时停止[2]。



(a) 第 1 阶段：推送 2 + 2　　(b) 第 2 阶段：推送 1

图 5.6　Dinic 算法在 $3, 2, 1, 2, 3$ 网络上的两个阶段，顶点按层数从左到右排列，弧旁为残量。(a) 第 1 阶段：同层边 $a \to b$（灰色虚线）不可用，沿两条两弧路径（黄色）各推送 2。(b) 第 2 阶段：绿色虚线为反向残量，沿 $s \to a \to b \to t$ 推送 1。

[2] R. Kyng, *Classical Algorithms for Maximum Flow II, Advanced Graph Algorithms and Optimization, Lecture 11*, ETH Zürich, May 6, 2020, URL: https://kyng.inf.ethz.ch/courses/AGAO20/lectures/lecture11_maxflow-contd.pdf

5.4.2 算法流程

算法状态与初始化　例中给每个点标了层数，又依次尝试它的出边。算法要记住的正是这两件事：每个点本轮处于第几层，邻接表试到了哪里。层数约束推进方向，进度让失败的方向不再重试；重新分层后可用边变了，进度只在阶段内保留。算法维护：

- 残量 r_e 与流值 F：初值同 Edmonds–Karp 算法，跨阶段保留；
- 层数 level(v)：每个阶段开始时由广度优先遍历重算，不可达的顶点没有层数；
- **当前弧**（current arc）指针 ptr(u)：指向 u 的邻接表中下一条待试的残量边，每个阶段开始时复位到表头，阶段内只向后移。

分层与阻塞流计算　外层循环每个阶段先做广度优先遍历求 level，t 不可达就结束。随后反复从 s 做一次深度优先推送 Push($s, +\infty$)。Push(u, f) 表示“从 u 出发、手里最多带着 f，试着送一条路到 t”，返回实际送达的量：$u = t$ 时返回 f；否则从 ptr(u) 起依次检查边 $e = (u \to v)$，满足 $r_e > 0$ 与 level(v) = level(u) + 1 时递归调用 Push($v, \min\{f, r_e\}$)。递归送达 $d > 0$ 时，回程中成对更新 r_e 与 $r_{\bar{e}}$ 并把 d 逐层返回。Push($s, +\infty$) 返回 0 时，本阶段结束。

指针何时前移，决定搜索既不漏、也不重复：

- 跳过：$r_e = 0$，或 level(v) $\neq$ level(u) + 1，或递归 Push($v, \cdot$) 返回 0，说明本阶段从这条边已到不了 t，令 ptr(u) 前移；
- 保留：递归成功时不前移，因为这条边可能还有残量，下一次推送还能用；若它这次被用尽，下次来到 u 时会因 $r_e = 0$ 而跳过。

指针若在每次推送时清零，失败的分支会被反复搜索；若成功后无条件前移，又会丢掉仍有残量的边。

算法 5.3　Dinic 算法

输入：用残量边对存储的流网络，源点 s，汇点 t。输出：流值 F、各原弧流量与割 $(S, \bar{S})$。

1. 正向边 $r_e = c_{u,v}$，反向边 $r_{\bar{e}} = 0$；$F = 0$。
2. 从 s 只经 $r_e > 0$ 的边做广度优先遍历，求 level。若 t 不可达，令 S 为可达顶点集，原弧流量取其反向边的残量，输出并停止。
3. 对每个顶点 u 令 ptr(u) 指向其邻接表表头。
4. 令 d = Push($s, +\infty$)。若 $d > 0$，置 $F \leftarrow F + d$，重复本步；否则回到第 2 步。

Push(u, f)：

1. 若 $u = t$，返回 f。

2. 当 ptr(u) 尚未越过表尾时，取其所指的边 e，终点为 v：若 $r_e > 0$ 且 level(v) = level(u)+1，令 $d = \mathrm{Push}(v, \min\{f, r_e\})$；若 $d > 0$，置 $r_e \leftarrow r_e - d$、$r_{\bar{e}} \leftarrow r_{\bar{e}} + d$，返回 d。其余情形令 ptr(u) 前移一位。
3. 返回 0。

递归深度不超过 $n-1$；必要时可用显式栈按同样规则前进与回退。

运行示例 继续 3, 2, 1, 2, 3 的网络，边对按同样次序登记，于是 a 的邻接表依次是 $a \to s$（反向）、$a \to b$、$a \to t$，b 的依次是 $b \to s$、$b \to a$（均为反向）、$b \to t$。

- 第 1 阶段分层：level$(s, a, b, t) = (0, 1, 1, 2)$，全部指针在表头。
- 第 1 次推送：s 试 $s \to a$ 可用，进入 a。$a \to s$ 残量为 0，跳过；$a \to b$ 通向同层，跳过；$a \to t$ 可用，到达 t。送 $\min\{3, 2\} = 2$，ptr(a) 停在 $a \to t$，$F = 2$。
- 第 2 次推送：$s \to a$ 仍有残量 1，进入 a；$a \to t$ 已耗尽，ptr(a) 越过表尾，返回 0。s 因此跳过 $s \to a$，本阶段不再尝试；改试 $s \to b$，b 跳过两条反向边，经 $b \to t$ 送 $\min\{2, 3\} = 2$，$F = 4$。
- 第 3 次推送：$s \to b$ 已耗尽，ptr(s) 越过表尾，返回 0，阻塞流为 4。
- 第 2 阶段分层：残量网络中 $s \to a$ 剩 1，$a \to b$ 剩 1，$b \to t$ 剩 1，层数变为 $(0, 1, 2, 3)$；指针全部复位。上一阶段放弃的 $s \to a$ 又被尝试。
- 第 1 次推送：$s \to a$，a 跳过 $a \to s$ 后经 $a \to b$ 进入 b，b 跳过两条反向边后经 $b \to t$ 到达 t，送 1，$F = 5$。第 2 次推送返回 0，阶段结束。
- 第 3 次分层：s 的出边都已耗尽，t 不可达，停止，$S = \{s\}$，割容量 $3+2 = 5$ 等于流值。

同一输入上，Edmonds–Karp 算法要做三次成功的遍历（$s \to a \to t$、$s \to b \to t$、$s \to a \to b \to t$），Dinic 算法只用两个成功的阶段；两者都另有一次失败的遍历，给出同一个最小割。

5.4.3 正确性与复杂度

例中第 1 阶段后只能走三弧的路。一般地，完成一个阻塞流后汇点的残量距离严格增加；这以“阶段内的推送确实形成阻塞流”为前提。

每次成功的 $\mathrm{Push}(s, +\infty)$ 沿 L_x 中的一条 s–t 路径送出该路径的残量最小值（递归中逐层取 min），按引理 5.3，流保持可行，流值增加送出的量。阶段内，送流使路径上可用边的残量减少，同时让反向边获得残量；但反向边从第 $i+1$ 层指向第 i 层，不是可用边。所以阶段内可用边只会减少，不会新增。由此，从 v 一旦到不了 t，整个阶段都到不了，被跳过的边不会重新有用。图 5.6(b) 中的绿色反向残量都是第 1 阶段产生的，按该阶段的层数全指向低一层，不会为失败的分支开出新路。

当 Push$(s,+\infty)$ 返回 0 时，s 的每条边都已被跳过，从 s 沿剩余可用边到不了 t，即阶段开始时 $L_{\boldsymbol{x}}$ 中每条 s–t 路径都含一条已被用尽的边，本阶段送出的总流是阻塞流。

引理 5.10　阶段距离严格增加　设 $\boldsymbol{x}'$ 由 $\boldsymbol{x}$ 加上 $L_{\boldsymbol{x}}$ 的一个阻塞流得到。若 t 在 $\mathcal{G}_{\boldsymbol{x}'}$ 中可达，则 $h_{\boldsymbol{x}'}(t) > h_{\boldsymbol{x}}(t)$。

证明　记 $\ell = h_{\boldsymbol{x}}$ 为阶段开始时的层数。$\mathcal{G}_{\boldsymbol{x}'}$ 中的残量弧 $u \to v$ 只有两种来源：阶段开始时已在 $\mathcal{G}_{\boldsymbol{x}}$ 中，由层数定义有 $\ell(v) \le \ell(u)+1$；或者是阶段内送流产生的反向弧，其反向是可用边，有 $\ell(v) = \ell(u)-1$。总之每条残量弧都满足 $\ell(v) \le \ell(u)+1$。于是 $\mathcal{G}_{\boldsymbol{x}'}$ 中任一条 s–t 路径每走一步层数至多加一，弧数至少为 $\ell(t)-\ell(s) = h_{\boldsymbol{x}}(t)$。若恰好相等，路径上每一步都必须使层数加一，因而每条弧都是阶段开始时的可用边，并且送流后仍有残量；这条路就是 $L_{\boldsymbol{x}}$ 中一条未被堵住的 s–t 路径，与阻塞流的定义矛盾。所以 $h_{\boldsymbol{x}'}(t) > h_{\boldsymbol{x}}(t)$。 ■

定理 5.11　Dinic 算法的正确性与开销　在有限实容量的流网络上，Dinic 算法至多经过 $n-1$ 个阶段就停止，输出最大流与最小割。使用当前弧指针时，每个阶段用时 $O(na)$，总时间为 $O(n^2a)$，存储为 $O(n+a)$。

证明　汇点可达时距离在 1 到 $n-1$ 之间，由引理 5.10每个阶段至少加一，所以至多 $n-1$ 个阶段后 t 不可达，算法停止。停止时没有增广路径，由定理 5.5输出最大流与最小割。

一个阶段内，广度优先遍历用时 $O(n+a)$。每次成功推送至少用尽一条可用边，而可用边在阶段内只减不增，所以成功推送至多 $2a$ 次。考察一次 Push$(s,+\infty)$ 检查的边：每条被检查的边要么使某个指针前移，要么位于本次成功送达的路径上。指针在一个阶段内总共前移至多 $2a$ 次；成功路径至多 $n-1$ 条弧，共至多 $2a$ 条路径。加上每次调用的常数开销，一个阶段为 $O(2a + 2a(n-1) + n + a) = O(na)$。乘以至多 $n-1$ 个阶段，总时间 $O(n^2a)$；另计线性预处理。除残量边对外只多存 level 与 ptr，存储为 $O(n+a)$。 ■

这也给出最大流最小割定理的另一种构造性证明。在单位容量的二部网络上，阶段数还能进一步压低，第 6.1.3.4小节说明这一更强的界。

5.5 方法比较与结果检验

两种算法共用残量边对与增广更新，差别在于一次遍历之后做多少工作：Edmonds–Karp 算法只确定一条最短路，状态只需前驱边；Dinic 算法建立整个分层网络，用当前弧指针连续推送到所有最短路被堵住，多存层数与指针。理论界分别为 $O(na^2)$ 与 $O(n^2a)$，弧多于顶点时后者较小；但这是最坏情形的上界，不能直接当成实测排名。

算法应输出流 $\boldsymbol{x}$、残量可达集确定的割 $(S,\bar{S})$ 及共同的数值。检验时逐项核对：每条弧满足 $0 \le x_{u,v} \le c_{u,v}$；每个中间顶点流入等于流出；源点净流出与汇点净流入相等；割容量按跨出弧重新求和，并与流值比较。两者相等时，由引理 5.2，流与割同时最优，检验不依赖算法过程。

最小割不一定唯一：在 $3,2,1,2,3$ 的网络中，$S=\{s\}$、$\{s,a\}$、$\{s,a,b\}$ 的割容量都是 5，只有 $S=\{s,b\}$ 为 6；残量可达集给出的是源侧最小的一个。

采用浮点数时，应分别报告守恒误差、容量违反量以及流值与割容量之差，不能把近似满足称为精确证书。

后面的问题都要先转成某种网络再使用这里的结论。第 6.1节的二部图匹配对应单位容量网络，可直接维护匹配；第 7章再把各点的供需与每条弧的费用纳入一般模型；第 10章把一类二值变量的成对目标写成割容量，再用最小割求解。这些转换关系必须逐一建立，不能因都画在图上就共享最优性结论。

5.6 本章小结

章首提出两个问题：怎样逐步改进一个输送方案，怎样确认它已不能再改进。第二个问题由割回答。流值等于跨过任意 s–t 割的净流量，所以不超过割容量；一个流与一个割数值相等时，两者同时最优，检查者只需逐弧核对容量、守恒与割容量。

第一个问题由残量网络回答。每条原弧对应一条可追加的正向残量弧和一条可撤回的反向残量弧，改进统一为沿增广路径推送瓶颈容量。没有增广路径时，残量网络中从 s 可达的顶点集给出一个与流值相等的割，这就是增广路径判据。由此得到最大流最小割定理：在有限非负实容量的网络中，最大流存在，最大流值等于最小割容量。

能否有限步停下来，取决于选路规则。整数容量下任意选路都会停止，并给出整数最大流；一般实容量下任意选路可能永不停止。Edmonds–Karp 算法每次选弧数最少的增广路径，残量层数不减，于是同一条弧不能频繁成为瓶颈，增广至多 na 次，总时间 $O(na^2)$。Dinic 算法在分层网络中求阻塞流，每个阶段使汇点距离严格增加，配合当前弧指针得到 $O(n^2a)$。这些都是最坏情形的上界，不能直接当成实测排名。

本章的网络与证书会在后面反复使用。下一章的二部图匹配是单位容量的最大流，Hopcroft–Karp 算法把 Dinic 算法的分层思想用到配对上；第 7章再给残量弧配上费用；第 10章则把一类二值变量的成对目标写成割容量，用最小割求出精确的最优配置。

第 6 章

二部图匹配与指派

把若干任务分给若干名人员，只有具备资格的人才能接相应的任务，每人至多接一项、每项至多一人负责。这里有两种要求。一种只问最多能配上多少对；另一种要求每个人都分到任务，而不同配对的花费不同，要在全部完整配对中选总成本最低的一个。本章依次回答这两个问题，并给出不必相信求解过程、就能核对结果最优的证据。

第 6.1节处理配对数量。每人、每项任务至多用一次，与每条弧至多通过一单位流量是同一种限制，所以最大匹配就是第 5章单位容量网络上的最大流。残量网络中的增广路径落到二部图上，成为新旧配对交替的路径；匈牙利算法只记录配对伙伴，逐个为人员寻找这样的路径。由于逐条贪心加边可能停在“不能再加、却不是最多”的结果上，这一节要特别区分极大匹配与最大匹配。

第 6.2节加入成本，研究指派问题。给每行和每列各配一个数，可以得到全部完整指派成本的共同下界；只用恰好达到下界的边完成的指派就是最优的。KM 算法在这些边上扩大匹配、受阻时抬高下界，结束时给出精确的证书；Auction 算法让行为列出价，用价格协调争抢，结果的误差由一个参数控制。两种算法的保证不同，比较时须先固定条件。

本章的二部图记为 $G = (L \cup R, E)$。第 6.1节记 $n = |L| + |R|$、$m = |E|$，第 6.2节记指派两侧的规模为 r。

6.1 二部图匹配

本节只数配对，不考虑成本：在允许的“人员—任务”配对中选出一组，任何人员或任务都不出现在两个配对里，使配对数量最多。第 6.1.1小节给出二部图与匹配的定义，并用一个小例子说明“不能再加边”不等于“配对最多”。第 6.1.2小节把匹配写成单位容量网络上的整数流，由此得到交替路径形式的最优判据。

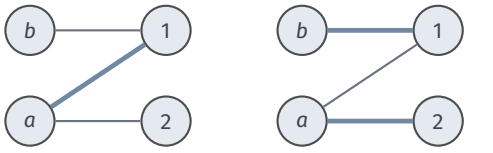
第 6.1.3小节的匈牙利算法直接利用这一判据，只维护伙伴数组，逐个为左侧顶点寻找增广路径；第 6.1.4小节说明怎样核对一个声称的最大匹配。

6.1.1 二部图匹配问题

人员与任务分属两类，资格关系只跨类连接。若无向图的顶点集能分成不相交的两部分 L 与 R，使每条边恰有一个端点在 L、另一个端点在 R，就称它为**二部图**（bipartite graph），记作 $G = (L \cup R, E)$，并称 L 为左侧、R 为右侧。边 $\{u, w\}$（$u \in L$，$w \in R$）简记为 (u, w)，称为允许边。一组配对合法，是指选出的边两两没有公共端点。

> **定义 6.1　匹配**　图 $G = (V, E)$ 中两两没有公共端点的边组成的集合 $M \subseteq E$ 称为一个**匹配**（matching）。若 $(u, w) \in M$，称 u 与 w 被 M 匹配，互为伙伴；不被任何匹配边关联的顶点称为**自由顶点**。边数最多的匹配称为**最大匹配**（maximum matching），没有自由顶点的匹配称为**完美匹配**（perfect matching）。不能再加入任何一条边而仍为匹配的 M 称为**极大匹配**（maximal matching）。

最大匹配一定是极大匹配，反之不然。设左侧为 a, b，右侧为任务 $1, 2$，允许边为 $(a,1)$、$(a,2)$、$(b,1)$，如图 6.1。若先选 $(a,1)$，另两条边都与它共享端点，无法再加，所以 $\{(a,1)\}$ 是极大匹配；可是 $\{(a,2),(b,1)\}$ 有两条边，是最大匹配，也是完美匹配。从前者到后者不能继续加边，只能把 a 改派到任务 2，腾出任务 1 给 b。逐条贪心加边可能停在这样的结果上，改进必须允许重排已有的配对。



(a) 极大匹配 $\{(a,1)\}$　　(b) 最大匹配 $\{(a,2),(b,1)\}$

图 6.1　极大匹配与最大匹配。细线为允许边，粗蓝线为匹配边。(a) 极大匹配 $\{(a,1)\}$；(b) 最大匹配 $\{(a,2),(b,1)\}$，由 (a) 到 (b) 须改动已有配对。

本节只最大化配对数 $|M|$，边上没有成本；成本问题留到第 6.2节。两侧大小可以不同，最大匹配至多有 $\min\{|L|, |R|\}$ 条边，但未必达到：若两名人员都只会同一项任务，至多一人能配上。后面的构造利用给定的两侧划分；含奇数长环（如三角形）的一般图无法这样划分，不能直接套用。记 $n = |L| + |R|$，$m = |E|$。

6.1.2 匹配与最大流的对应

由二部图构造流网络：增加源点 s 与汇点 t，从 s 向每个 $u \in L$ 连弧，把每条允许边 (u, w) 改为弧 $u \to w$，再从每个 $w \in R$ 向 t 连弧，所有容量都为 1，如图 6.2。

给定匹配 M，沿每条匹配边对应的路线 $s \to u \to w \to t$ 各送一单位，得到流值为 $|M|$ 的整数流。反过来，整数流 $\boldsymbol{x}$ 在每条弧上只取 0 或 1。左侧顶点 u 只有一条容量为 1 的入弧，由流守恒，它至多有一条流量为 1 的出弧；同理，右侧顶点至多有一条流量为 1 的入弧。所以流量为 1 的中间弧两两不共享端点，构成一个匹配，边数等于流值。

由定理 5.4，整数容量的网络有整数最大流，因此最大匹配的边数等于这个网络的最大流值。这里只从整数流恢复匹配，分数流量不对应任何一组配对。

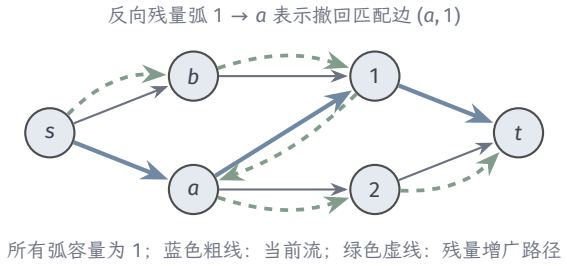


图 6.2　图 6.1的二部图对应的单位容量网络。蓝色粗弧是匹配 $\{(a,1)\}$ 对应的流；绿色虚线是一条残量 s–t 路径，反向残量弧 $1 \to a$ 表示撤回 $(a,1)$，去掉 s,t 即交替路径 $b,1,a,2$。

设 $\boldsymbol{x}$ 对应匹配 M。残量网络中，弧 $s \to u$ 存在当且仅当 u 自由；左侧顶点只能沿未匹配边走到右侧；右侧顶点 w 若自由就走 $w \to t$ 结束，否则只能沿反向残量弧回到伙伴，相当于准备撤回这条匹配边。去掉两端的 s 与 t，剩下的路径从左到右走未匹配边、从右到左走匹配边。图 6.2中的残量路径就是 $b,1,a,2$：b 要任务 1，请 a 改做任务 2。

一般地，给定匹配 M，边交替地不属于 M 与属于 M 的路径称为**交替路径**（alternating path）；两个端点都自由的交替路径称为关于 M 的增广路径。在二部图中，增广路径的一端是自由左侧顶点，另一端是自由右侧顶点；首尾两条边都不在 M 中，未匹配边比匹配边多一条。

引理 6.2　沿增广路径翻转　设 P 是关于匹配 M 的增广路径。把 P 上属于 M 的边移出、不属于 M 的边加入，得到的边集 M' 仍是匹配，且 $|M'| = |M| + 1$。

证明　P 外的顶点不受影响。P 的内部顶点恰关联 P 上一条匹配边和一条未匹配边，且唯一的匹配边就在 P 上，翻转后两者互换，仍恰有一个伙伴。P 的两个端点原来自由，翻转后各得到一个伙伴。所以 M' 中没有两条边共享端点。P 上未匹配边比匹配边多一条，故 $|M'| = |M| + 1$。 ■

沿残量路径增广一单位，正是沿对应的增广路径翻转，所以最大流的最优性判据可以搬到匹配上。

定理 6.3　增广路径判据　二部图中的匹配 M 是最大匹配，当且仅当不存在关于 M 的增广路径。

证明　若存在增广路径，由引理 6.2可得更大的匹配。反过来，设不存在增广路径，令 $\boldsymbol{x}$ 是 M 对应的流。由上面的对应，残量网络中没有 s–t 路径，由定理 5.5，$\boldsymbol{x}$ 是最大流。任何匹配 M'' 都对应一个流值为 $|M''|$ 的流，所以 $|M''| \le |M|$。 ■

这个判据比“不能再加边”强。图 6.1(a) 中，$\{(a,1)\}$ 已不能直接加边，却有增广路径 $b, 1, a, 2$；翻转后得到最大匹配 $\{(a,2),(b,1)\}$。整体重排能多配一对，是因为中间的顶点只换伙伴，两端的自由顶点各得到一个伙伴。

6.1.3　匈牙利算法

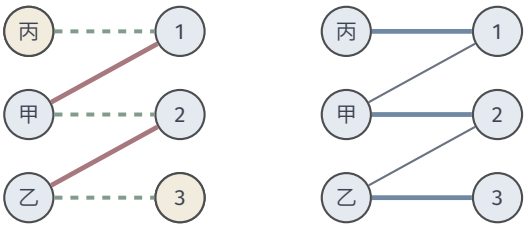
定理 6.3提示了一种算法：找到增广路径就翻转，找不到就停止。中文教学常把逐个左侧顶点用深度优先搜索找增广路径的做法称为**匈牙利算法**（Hungarian algorithm），也称 Kuhn 算法；英文文献中的 Hungarian algorithm 通常指第 6.2节处理成本的指派算法。它沿用残量增广的思想，但只维护配对关系。

6.1.3.1　主要思想

把图 6.1中的 a, b 看成员工甲、乙：甲能做任务 1 和 2，乙只会任务 1。甲先接下任务 1；轮到乙时任务 1 已有人负责，直接找空任务失败。换个问法：先问甲能否改做别的。任务 2 空着，甲改做 2，空出的任务 1 交给乙。想要的任务已有负责人时，不放弃也不抢走，先替原负责人找去处。

这次调整可以写成 乙 → 1 → 甲 → 2：候选分配、已有分配、候选分配依次交替，终点任务 2 空闲。找到空任务后再逆序落实：甲改做 2，乙接手 1。

再增加一名员工和一项任务：甲会 1, 2，乙会 2, 3，丙只会 1，当前已有甲—1、乙—2。丙想要任务 1，就请甲另找；甲想要任务 2，又请乙另找；乙找到空任务 3。沿 丙 → 1 → 甲 → 2 → 乙 → 3 连续让位，三人都有了任务，如图 6.3。替甲、替乙找去处是同一个问题，所以可以递归：为某人找任务时，逐个询问他会的任务，任务空着就成功，否则递归地为该任务的负责人另找任务。



(a) 匹配 {甲1, 乙2} 与让位链　　(b) 翻转后 {丙1, 甲2, 乙3}

红线：将撤去的匹配边；绿色虚线：将加入的边；粗蓝线：匹配边；黄色：自由端点

图 6.3　三人三任务的让位链。(a) 当前匹配为甲—1、乙—2，丙与任务 3 自由（黄色），让位链交替经过将加入的边（绿色虚线）与将撤去的匹配边（红线）；(b) 翻转后配对数由 2 增为 3。

把员工和任务还原为左右顶点，成功的让位链就是从自由左侧顶点到自由右侧顶点的增广路径，逆序落实就是引理 6.2的翻转；某个分支失败，原负责人找不到去处，原匹配应保持不变。与最大流归约相比，这里只用伙伴数组，省去源汇、容量和残量弧，但这只减少状态，不保证渐近界更优[1]。

[1] R. Sedgewick, K. Wayne, *BipartiteMatching.java*, 2025, URL: https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/BipartiteMatching.java.html

6.1.3.2 算法流程

算法状态与初始化　手工调整要记两件事：谁现在负责哪项任务；本轮哪些任务已经问过。前者成功改配后改变并跨轮保留；后者只属于本轮，重问只会重复同样的结果。邻接表在运行前给定、运行中不变，不算状态。取较小的一侧作为左侧，记其顶点数为 p，算法维护：

- 伙伴数组 $\mu_L(u)$ 与 $\mu_R(w)$：左右顶点的当前伙伴，初值全为空，即 $M=\varnothing$；
- 访问时间戳 $\tau(w)$：右侧顶点 w 最近一次被询问时的轮次，初值全为 0；
- 当前轮次 k：初值为 0。

每处理一个左侧顶点就令 $k \leftarrow k+1$，本轮已问过的恰是 $\tau(w)=k$ 的右侧顶点，旧标记自动失效，不必清空。新处理的左侧顶点从未被询问，总是自由的。

搜索与改配　为左侧顶点 u 寻找增广路径的过程 $\mathrm{Try}(u)$ 按邻接表扫描 u 的邻居 w：

- 若 $\tau(w)=k$：本轮已问过，跳过；
- 否则令 $\tau(w) \leftarrow k$。若 w 自由，或递归调用 $\mathrm{Try}(\mu_R(w))$ 成功，就令 $\mu_R(w) \leftarrow u$、$\mu_L(u) \leftarrow w$，返回成功；否则继续扫描下一个邻居。

邻居扫描完仍未成功，返回失败。伙伴只在成功分支上改写，所以整轮失败时原匹配完整保留。外层对每个左侧顶点依次调用 Try，处理完即停止。

成功时改写从链尾向链头逐层进行：内层先为靠后的员工记下新任务，外层才把前一项任务改记给新负责人。两步之间，该任务在 μ_R 中仍记着原负责人，而原负责人在 μ_L 中已记着新任务；回到链头时两数组重新一致，所以匹配只在一轮结束后对外使用。

算法 6.1　匈牙利算法

输入：二部图的左侧邻接表，左侧为较小的一侧。输出：最大匹配的伙伴数组。

1. 令全部伙伴为空，全部 $\tau(w)=0$，$k=0$。
2. 对左侧顶点 u 依次执行：令 $k\leftarrow k+1$，调用 Try(u)。
3. 输出 μ_L,μ_R。

过程 Try(u)：对 u 的每个邻居 w，若 $\tau(w)=k$ 则跳过；否则令 $\tau(w)\leftarrow k$，若 $\mu_R(w)$ 为空或 Try$(\mu_R(w))$ 返回成功，令 $\mu_R(w)\leftarrow u$、$\mu_L(u)\leftarrow w$ 并返回成功。全部邻居扫描完后返回失败。

运行示例　表 6.1按“询问—递归—成功返回—改配”逐行列出两个例子，邻居按编号读取。三任务例多递归一层，做法不变；其下半第 3 行 μ_R 把任务 2, 3 都记在乙名下，正是上述暂时不对应。

表 6.1　匈牙利算法的运行。上半为两任务例，下半为三任务例的第 3 轮（前两轮已得甲—1、乙—2）。τ 与 μ_R 按任务编号列出，“—”表示空。

轮	调用栈	动作	τ	μ_R
1	甲	问 1：空闲，记甲—1，成功	(1, 0)	(甲,—)
2	乙	问 1：甲负责，为甲递归	(2, 0)	(甲,—)
2	乙，甲	跳过 1；问 2：空闲，记甲—2，成功	(2, 2)	(甲, 甲)
2	乙	收到成功，记乙—1	(2, 2)	(乙, 甲)
3	丙	问 1：甲负责，为甲递归	(3, 2, 0)	(甲, 乙,—)
3	丙，甲	跳过 1；问 2：乙负责，为乙递归	(3, 3, 0)	(甲, 乙,—)
3	丙，甲，乙	跳过 2；问 3：空闲，记乙—3，成功	(3, 3, 3)	(甲, 乙, 乙)
3	丙，甲	收到成功，记甲—2	(3, 3, 3)	(甲, 甲, 乙)
3	丙	收到成功，记丙—1	(3, 3, 3)	(丙, 甲, 乙)

失败的搜索要保留原匹配。在三任务例中删去允许边甲—2，前两轮仍得到甲—1、乙—2，第 3 轮为：

- 丙询问任务 1，令 $\tau(1)=3$，任务 1 由甲负责，为甲递归；
- 甲只会任务 1，而它本轮已问过，甲的搜索失败，不改任何记录；
- 丙再无别的任务可问，整轮失败。

伙伴数组仍是甲—1、乙—2。若丙询问时就先撤去甲—1，甲、丙都将没有任务，匹配反而少一对。

6.1.3.3 正确性与复杂度

在三人让位链上数配对：甲、乙只换了任务，丙和任务 3 各得到伙伴，恰好多 1 对，一般情形即引理 6.2。还需说明搜索失败时当前匹配已不能改进。

定理 6.4　匈牙利算法的正确性　设左侧顶点按 $u_1, \dots, u_p$ 的顺序处理，$\mathcal{G}_k$ 为由 $u_1, \dots, u_k$ 与全部右侧顶点导出的子图。处理完 u_k 后，伙伴数组给出 $\mathcal{G}_k$ 的一个最大匹配；特别地，算法输出 $\mathcal{G}$ 的最大匹配。

证明　对 k 归纳，$k = 0$ 时匹配为空，结论成立。设处理 u_k 前的匹配 M 是 $\mathcal{G}_{k-1}$ 的最大匹配，u_k 是自由顶点。$\mathcal{G}_k$ 只比 $\mathcal{G}_{k-1}$ 多一个顶点 u_k，任何匹配中至多一条边含 u_k，所以 $\mathcal{G}_k$ 的最大匹配数至多比 $\mathcal{G}_{k-1}$ 的多 1。

若 $\mathrm{Try}(u_k)$ 成功，递归链依次经过 u_k、右侧顶点 w_1、w_1 的伙伴、右侧顶点 w_2、……，止于一个自由右侧顶点。本轮每个右侧顶点至多问一次，链上的左侧顶点互不相同，所以这条链是增广路径，逐层改写就是翻转。由引理 6.2，新匹配有 $|M| + 1$ 条边，达到上述上界，是 $\mathcal{G}_k$ 的最大匹配。

若 $\mathrm{Try}(u_k)$ 失败，伙伴数组未变。记本轮被询问过的右侧顶点集为 R^*，L^* 由 u_k 与 R^* 中顶点的伙伴组成。被询问的右侧顶点都不自由，本轮对其伙伴调用过 Try，失败的 Try 扫描了全部邻居，所以 L^* 中每个顶点的邻居都在 R^* 中。于是从 u_k 出发的交替路径始终留在 $L^* \cup R^*$ 中，到不了自由右侧顶点。假如 M 不是 $\mathcal{G}_k$ 的最大匹配，由定理 6.3存在增广路径 P。P 的内部顶点都已匹配，不会是自由的 u_k；若 u_k 也不是 P 的端点，P 就整条位于 $\mathcal{G}_{k-1}$ 中，与 M 在 $\mathcal{G}_{k-1}$ 中最大矛盾。所以 P 从 u_k 出发，而这刚被排除。故 M 仍是 $\mathcal{G}_k$ 的最大匹配。 ■

同一轮中每个右侧顶点至多被询问一次，每个左侧顶点至多调用一次 Try（它是 u_k，或某个被询问右侧顶点的伙伴），所以每条允许边至多扫描一次，一轮为 $O(1+m)$。p 轮共 $O(p+pm)$，加上伙伴数组与时间戳的初始化 $O(n)$，总时间为 $O(n+pm)$；邻接表、伙伴数组与时间戳共占 $O(n+m)$ 空间。若每轮清空访问数组，另需 $O(|R|)$；用邻接矩阵时每次扫描要读整行，也须另算代价。

6.1.3.4 分层增广的进一步改进

逐点搜索每轮至多找到一条增广路径。Hopcroft–Karp 算法把第 5.4节 Dinic 算法的分层思想用到单位容量网络上，每个阶段做两步：

- 分层：从全部自由左侧顶点同时沿交替路径做广度优先遍历，求出最短增广路径长度与各顶点层号；
- 增广：只沿层号逐层加一的边做深度优先搜索，找出一组极大的、顶点互不相交的最短增广路径，一起翻转。

这组路径就是分层残量网络中的阻塞流（定义 5.9），与 Dinic 算法同理，每个阶段之后最短增广路径的长度严格增加。

阶段数很少，理由简述如下：$\sqrt{n}$ 个阶段之后，最短增广路径已有超过 $2\sqrt{n}$ 条边；当前匹配与任一最大匹配的差可以分解为顶点互不相交的增广路径，每条都占去超过 $\sqrt{n}$ 个顶点，所以至多还差 $\sqrt{n}$ 条，至多再需 $\sqrt{n}$ 个阶段。合计 $O(\sqrt{n})$ 个阶段，每阶段为 $O(m)$，主计算为 $O(m\sqrt{n})$；孤立顶点不在任何增广路径上，先用 $O(n)$ 时间删去，总时间写作 $O(n + m\sqrt{n})$[2]。这个界不适用于前面逐点搜索的实现。

[2] R. Sedgewick, K. Wayne, *HopcroftKarp.java*, 2025, URL: https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/HopcroftKarp.java.html

6.1.4 结果检验

先确认可行性：所选边都是允许边，两两没有公共端点，再重数边数；伙伴数组还应两两对应，即 $\mu_L(u) = w$ 当且仅当 $\mu_R(w) = u$。

最大性另需证据：可以从全部自由左侧顶点重做一次交替搜索，确认到不了自由右侧顶点；更独立的证据是单位容量网络中容量等于 $|M|$ 的割。

令 Z 为从自由左侧顶点沿交替路径能到达的顶点集，若其中没有自由右侧顶点，取源侧 $S = \{s\} \cup Z$。这时 Z 中左侧顶点的邻居都在 Z 中，Z 中的右侧顶点都已匹配且伙伴也在 Z 中，所以跨出 S 的弧只有 $s \to u$（$u \in L \setminus Z$）与 $w \to t$（$w \in R \cap Z$）。对每条匹配边 (u, w)，$u \in Z$ 时恰有 $w \to t$ 跨出，$u \notin Z$ 时 $w \notin Z$，恰有 $s \to u$ 跨出；自由顶点不贡献跨出的弧。故割容量为 $|M|$，由引理 5.2，任何匹配对应的流值都不超过 $|M|$，M 最大。

在删去甲—2 的例子中，$Z = \{$丙, 1, 甲$\}$，跨出 S 的弧是 $s \to$ 乙 与 $1 \to t$，割容量为 2，等于匹配的边数。

不能因为自由顶点之间没有允许边就宣布最大：图 6.1(a) 中自由的 b 与 2 不相邻，却仍有增广路径。

完整配对可能有多种，数量无法区分优劣。第 6.2节给配对加上成本，选出总成本最小的完整配对。

6.2 指派与带权二部图匹配

上一节只关心能配上多少对。若每名员工都必须分到一项任务、不同配对花费不同，就要在全部完整配对中挑总成本最低的一个。逐人挑最便宜的任务会让几个人挤向同一项任务，所以需要判断“谁让出偏爱的任务更划算”，并证明得到的方案已经最省。

给每名员工和每项任务各配一个数，可以得到任何完整配对成本的下界；只用恰好达到下界的边完成的配对就是最优的。KM 算法保持下界有效，在这些边上用交替路径扩大匹配，受阻时抬高下界；Auction 算法让员工为任务出价，用价格协调争抢，得到误差可控的方案。

6.2.1 指派问题

把员工看成二部图左侧的行，任务看成右侧的列。行 i 与列 j 允许配对时，两者之间有一条边，成本 $c_{i,j}$ 是有限实数，可以为负；不允许的配对没有边。

> **定义 6.5　指派问题**　设二部图两侧各有 r 个顶点，左侧为行 $1,\dots,r$，右侧为列 $1,\dots,r$，允许的行列对组成边集 E，每条边 $(i,j)\in E$ 有有限实成本 $c_{i,j}$。**指派问题**（assignment problem）要求选出一个只用允许边的完美匹配，即给每行 i 指定一个不同的列 $\sigma(i)$，使总成本 $C(\sigma)=\sum_{i=1}^{r} c_{i,\sigma(i)}$ 最小。

指派问题也称为最小权完美匹配，输出一个最优指派，或“不存在完美匹配”的结论。若要使收益 $w_{i,j}$ 之和最大，取 $c_{i,j}=-w_{i,j}$ 即可。全部行列对都允许时，σ 可以是任意置换，成本写成 r 阶矩阵 $\boldsymbol{C}=(c_{i,j})$。

“每个顶点恰配一次”不能随意放宽。允许匹配任意多对、只求收益最大的是另一个问题，即最大权匹配：负收益的边宁可不选，结果可能只配一部分顶点；在最小成本形式下，成本全为正时空匹配反而“最便宜”。行列数不等、禁配、可以不配但要付惩罚等情形，在第 6.2.3.4小节归结为本定义；下文先在方阵上讲算法。

6.2.2 可行标号与最优性条件

判断一个指派是否最优，不能把 $r!$ 个指派逐一比较，需要一个能对全部指派同时成立的下界。给每行 i 配实数 u_i，每列 j 配实数 v_j。若对每条允许边都有

$$u_i+v_j\le c_{i,j},\tag{6.1}$$

就称 $(\boldsymbol{u},\boldsymbol{v})$ 为**可行标号**（feasible labeling）：把成本分摊给行与列，两份之和不超过实际成本。对任意完整指派 σ，把它用到的 r 条边的不等式相加；每行恰出现一次，每列也恰出现一次，于是

$$C(\sigma)=\sum_{i=1}^{r}c_{i,\sigma(i)}\ge\sum_{i=1}^{r}\bigl(u_i+v_{\sigma(i)}\bigr)=\sum_{i=1}^{r}u_i+\sum_{j=1}^{r}v_j.$$

右边与 σ 无关，是全部完整指派成本的共同下界。这个论证只用到“每行每列恰计一次”，不需要一般的线性规划对偶理论。从线性规划看，可行

标号正是指派问题对偶问题的可行解，下面的紧边条件就是互补松弛；这一解释可参考卷五第 2 章“线性规划”。

称 $c_{i,j}-u_i-v_j\ge 0$ 为边 (i,j) 的剩余量。剩余量为零、使式 (6.1)取等号的允许边称为**紧边**（tight edge），全部顶点与全部紧边组成的子图称为**相等子图**（equality subgraph）。

定理 6.6　紧边指派的最优性　设 $(\boldsymbol{u},\boldsymbol{v})$ 是可行标号。若完整指派 σ 只使用紧边，即 σ 是相等子图中的完美匹配，则 $C(\sigma)=\sum_i u_i+\sum_j v_j$，因而 σ 是最优指派。

证明　σ 用到的每条边都取等号，上面的求和不等式也取等号，所以 σ 的成本等于下界。任何完整指派的成本都不低于这个下界，所以 σ 最优。 ■

可行标号本身只提供下界；相等子图里可能没有完美匹配，不能据此宣布最优。求解因此变成两件同时进行的事：保持标号可行，同时在紧边中凑出完美匹配。KM 算法严格按此工作；Auction 算法允许所选的边留下一点剩余量，误差在第 6.2.4小节给出。

6.2.3　KM 算法

KM 算法（Kuhn–Munkres algorithm）以 Kuhn 和 Munkres 命名，英文文献通常称为 Hungarian algorithm，即带权的匈牙利算法；上一节的匈牙利算法只求最大匹配，没有成本。它从可行标号和空匹配出发，只在紧边上寻找增广路径，受阻时调整标号产生新的紧边；各列的最小剩余量作为状态维护，使复杂度与实际操作一致。

6.2.3.1　主要思想

沿用上一节的两名员工甲、乙和两项任务 1、2，现在每个配对都允许，但成本不同：

$$\boldsymbol{C}=\begin{bmatrix}1 & 2\\ 1 & 4\end{bmatrix},$$

行依次为甲、乙，第 j 列是任务 j。完整方案只有两个：甲—1、乙—2 成本为 5；甲—2、乙—1 成本为 3。两人最便宜的选择都是任务 1，“每行选最小”拼不成指派；要比较的是让步的代价：甲改做任务 2 多花 1，乙则多花 3。

把上一小节的标号放回成本表。从每行最小值出发取 $\boldsymbol{u}=(1,1)$、$\boldsymbol{v}=(0,0)$，标号可行，各格剩余量 $c_{i,j}-u_i-v_j$ 为

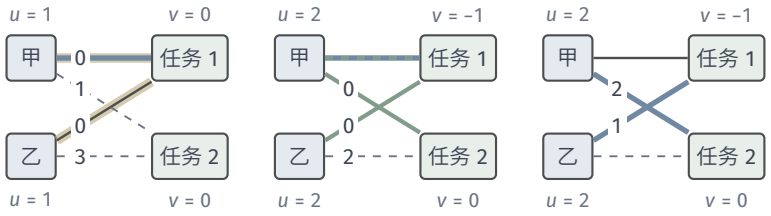
$$\begin{bmatrix} 0 & 1 \\ 0 & 3 \end{bmatrix}.$$

零格就是紧边。两个零格都在第一列，相等子图暂时没有完美匹配。

让甲先占任务 1，再从自由的乙出发找增广路径。乙沿紧边到任务 1，任务 1 已归甲，于是转去问甲；甲除任务 1 外没有紧边，搜索受阻，如图 6.4(a)。

要接到空闲的任务 2，甲还差 1、乙还差 3，所以只把标号推进较小的差额 1：甲、乙的标号各加 1，使通往树外的剩余量减 1；任务 1 的标号减 1，使树内两条紧边保持为零。新标号为 $\boldsymbol{u}=(2,2)$、$\boldsymbol{v}=(-1,0)$，甲到任务 2 变紧，得到增广路径乙—任务 1—甲—任务 2，如图 6.4(b)。

翻转后甲做任务 2、乙做任务 1，成本为 3，等于标号之和 $2+2-1+0$，由定理 6.6最优。成本表没有被改写，改变的只是下界的表示。



(a) 交替树受阻，$\delta=1$　(b) 调标后出现增广路径　(c) 最优指派，成本 3

图 6.4　KM 算法在成本矩阵 $\left[\begin{smallmatrix}1&2\\1&4\end{smallmatrix}\right]$ 上的运行。顶点旁为标号，(a)(b) 边旁为剩余量，(c) 边旁为成本；实线为紧边，虚线为非紧边，粗蓝线为匹配。(a) 从乙出发的交替树（黄色衬底）受阻，$\delta=1$。(b) 调标后甲到任务 2 变紧，出现增广路径（绿色）。(c) 指派成本 3 等于标号之和。

这个例子里的做法可以一般化。从一个自由行出发，只沿紧边生长**交替树**（alternating tree）：从行走未匹配的紧边到列，到达已匹配的列就沿匹配边回到它的行。记树中的行集为 S、列集为 T。

到达自由列就沿树中路径增广；所有紧边都通向 T 时，在 S 到树外列的边中取最小剩余量 δ，令 S 中行标号加 δ、T 中列标号减 δ。差额更大会违反式 (6.1)，更小则没有新的紧边。

为了不在每次受阻时重新比较全部边，算法为每个树外列记住“从树中哪一行过来差得最少、差多少”[3]。

[3] N. Zeh, *The Hungarian Algorithm*, Dalhousie University, URL: https://web.cs.dal.ca/~nzeh/Teaching/4113/book/matching/bipartite_perfect_matching/hungarian_initial.html

6.2.3.2 算法流程

算法状态与初始化　输入是允许边集与成本，运行中不变，不算状态。对照手算，算法维护以下状态：

- 伙伴数组：每行配给的列、每列配给的行，初值全为空。
- 标号 $\boldsymbol{u}, \boldsymbol{v}$：先检查每行至少有一条允许边，否则直接报告不可行；取 $u_i = \min_{j:(i,j)\in E} c_{i,j}$，$v_j = 0$。每条允许边满足 $u_i + 0 \le c_{i,j}$，标号可行，且每行至少有一条紧边。
- 交替树：行集 S、列集 T，以及每个入树列的前驱行，即把它带进树的那一行。为一个新的自由行建树时，S 只含该行，T 为空。
- 松弛量与来源：对每个树外列 j，

$$\mathrm{slack}(j) = \min\{c_{i,j} - u_i - v_j \,:\, i \in S,\ (i,j) \in E\},$$

  没有候选时记为 $+\infty$；$\mathrm{from}(j)$ 记录取到最小值的行。建树时按根行的允许边初始化。

调标后交替树和来源记录都保留，只有增广成功后，才为下一个自由行重建这些状态。

扩树、调标与增广　为一个自由行建树后，按情形反复执行：

- 扩树：若有树外列 j 满足 $\mathrm{slack}(j) = 0$，就把 j 加入 T，前驱记为 $\mathrm{from}(j)$。若 j 已匹配，把它的行 i' 加入 S，再只扫描 i' 的允许边，用 $c_{i',j'} - u_{i'} - v_{j'}$ 更新各树外列的 $\mathrm{slack}(j')$ 与 $\mathrm{from}(j')$。
- 增广：若加入 T 的 j 是自由列，从 j 沿前驱和匹配边回溯到根，得到一条全由紧边组成的增广路径；翻转路径上的配对，匹配增加一对，本轮结束。
- 调标：若树外列的松弛量都为正，取

$$\delta = \min_{j \notin T} \mathrm{slack}(j), \tag{6.2}$$

  令 S 中各行 $u_i \leftarrow u_i + \delta$、$T$ 中各列 $v_j \leftarrow v_j - \delta$，各树外列 $\mathrm{slack}(j) \leftarrow \mathrm{slack}(j) - \delta$，来源不变。取到最小值的列松弛量变为零，随即可以扩树。

若需要调标时全部树外列的松弛量都为 $+\infty$，报告不可行。全部行都匹配时停止，输出匹配与标号。

调标时行在 S 外、列在 T 中的边剩余量增加 δ，原来的紧边可能不再紧，相等子图并非只增不减；保留下来的是树中的边和匹配边，下一小节核对。

算法 6.2 KM 算法

输入：两侧各 r 个顶点的允许边集与成本。输出：最优指派与可行标号，或“不可行”。

1. 若某行没有允许边，报告不可行；否则置 u_i 为第 i 行最小允许成本、$v_j=0$，伙伴全为空。
2. 若所有行都已匹配，转第 6 步。否则取自由行 i_0，置 $S=\{i_0\}$、$T=\varnothing$，对每列 j 置 $\mathrm{slack}(j)=c_{i_0,j}-u_{i_0}-v_j$（不允许时为 $+\infty$）、$\mathrm{from}(j)=i_0$。
3. 若有 $j\notin T$ 使 $\mathrm{slack}(j)=0$：把 j 加入 T，前驱置为 $\mathrm{from}(j)$。若 j 自由，转第 5 步；否则把 j 的伙伴行 i' 加入 S，对每个 $j'\notin T$，若 $c_{i',j'}-u_{i'}-v_{j'}<\mathrm{slack}(j')$，就更新 $\mathrm{slack}(j')$ 并置 $\mathrm{from}(j')=i'$；重复第 3 步。
4. 若所有 $j\notin T$ 都有 $\mathrm{slack}(j)=+\infty$，报告不可行；否则按式 (6.2)求 δ，S 中行标号加 δ、T 中列标号减 δ、树外列松弛量减 δ，回到第 3 步。
5. 从自由列 j 开始，令 i 为 j 的前驱、j'' 为 i 原来的伙伴，把 i 改配给 j，再令 $j\leftarrow j''$，直到 $i=i_0$；回到第 2 步。
6. 输出伙伴数组与标号 $(\boldsymbol{u},\boldsymbol{v})$。

运行示例 表 6.2把上面的手算翻译成状态：

- 以甲为根，任务 1 的松弛量为 0 且空闲，直接增广，甲配任务 1。
- 以乙为根，任务 1 入树；它已归甲，甲入树，只扫描甲的边，把任务 2 的松弛量从 3（来自乙）降为 1（来自甲）。
- 调标 $\delta=1$，任务 2 的松弛量变为 0，来源仍是甲。
- 任务 2 入树，前驱为甲；它空闲，回溯：甲改配任务 2，任务 1 交给其前驱乙。

成本 3 等于标号之和，逐行贪心的成本 5 则不然。加入一行只扫描这一行、已有最小值继续保留，正是复杂度分析依赖的增量维护。

表 6.2 KM 算法在成本矩阵 $\left[\begin{smallmatrix}1&2\\1&4\end{smallmatrix}\right]$ 上的状态。每行为动作后的状态；松弛量依次为任务 1、2，括号内为来源，“—”表示已入树。

动作	S；T	$\boldsymbol{u}$；$\boldsymbol{v}$	松弛量（来源）	匹配
以甲为根建树	{甲}；∅	$(1,1)$；$(0,0)$	0（甲），1（甲）	空
任务 1 入树，增广	—	$(1,1)$；$(0,0)$	—	甲—1
以乙为根建树	{乙}；∅	$(1,1)$；$(0,0)$	0（乙），3（乙）	甲—1
任务 1、甲入树	{乙, 甲}；{1}	$(1,1)$；$(0,0)$	—，1（甲）	甲—1
调标 $\delta=1$	{乙, 甲}；{1}	$(2,2)$；$(-1,0)$	—，0（甲）	甲—1
任务 2 入树，增广	—	$(2,2)$；$(-1,0)$	—	甲—2，乙—1

6.2.3.3 正确性与复杂度

两人例中下界从 2 升到 3，与方案成本相等。一般情形依赖四件事：初始标号可行；扩树与调标保持标号可行、匹配紧和树中紧；调标后树能继续

生长；每次增广增加一对。扩树和增广只沿紧边进行、不改标号，剩下的是调标。

引理 6.7　调标的效果　设标号可行，匹配边与交替树中的边都是紧边，且 δ 按式 (6.2)取有限值。调标后标号仍可行，匹配边与树中的边仍是紧边，至少有一个树外列的松弛量变为零，标号之和增加 δ。

证明　按端点归属逐类检查允许边 (i,j) 的剩余量 $c_{i,j}-u_i-v_j$。$i\in S$、$j\in T$ 时，u_i 加 δ、v_j 减 δ，剩余量不变，树中的紧边都属于这一类。$i\in S$、$j\notin T$ 时，剩余量减 δ；由 δ 是这类边剩余量的最小值，减后仍非负，取到最小值的边变为零。$i\notin S$、$j\in T$ 时剩余量增加 δ，其余边不变，所以标号可行。树中每个列都与树中某一行匹配，每个非根行都经匹配边进树，匹配边要么两端都在树中，要么两端都在树外，剩余量都不变，仍为零。树外列的松弛量正是第二类边剩余量的最小值，统一减 δ 后与新标号一致。最后，树中每个列都带进一个已匹配的行，根行自由，所以 $|S|=|T|+1$，标号之和增加 $|S|\delta-|T|\delta=\delta$。 ■

由此，标号可行、匹配只用紧边两条性质在整个运行中保持。算法停止且全部行都匹配时，匹配是相等子图中的完美匹配，由定理 6.6，它是最优指派。报告不可行也是正确的：若树外列的松弛量全为 $+\infty$，S 中各行的允许列全部落在 T 中，$|S|$ 行只能使用 $|T|=|S|-1$ 个列，不可能各配一个不同的列。

有限终止由计数得到。每次调标后至少有一列入树；若它自由就立即增广，否则它的伙伴行入树。树中的列至多 r 个，所以一个自由行至多经过 r 次调标就增广。每次增广使匹配增加一对，至多 r 次。

对 r 阶稠密成本表，一轮建树中至多 r 行加入 S，每行扫描并更新松弛量 $O(r)$；至多 r 次扩树或调标，每次在树外列中找零或求 δ 并统一减 δ 为 $O(r)$。一轮为 $O(r^2)$，r 轮共 $O(r^3)$ 次实数运算。

这个界依赖于调标后保留交替树和已有的最小值；若每次受阻都从根重新搜索、重新比较全部边，就不能直接引用它[4]。成本表占 $O(r^2)$，是输入本身；伙伴、标号、前驱、松弛量等工作数组为 $O(r)$。

[4] N. Zeh, *A Faster Implementation of the Hungarian Algorithm*, Dalhousie University, URL: https://web.cs.dal.ca/~nzeh/Teaching/4113/book/matching/bipartite_perfect_matching/hungarian_fast.html

6.2.3.4　矩形指派与不完整配对

行数少于列数时，补充到每列成本都为零的虚拟行。虚拟行只占用真实行没用到的列，完整指派的成本等于真实部分的成本，删去虚拟行的配对即得原问题的解。

允许对象不配对、但要付惩罚时，不能只把“恰配一次”放宽成“至多配一次”：成本非负时，什么都不配就成了最优解。正确的做法是把惩罚写进模型。给每个左侧对象 i 配一个专属虚拟列，配给它的成本是 i 不配对的惩罚 ρ_i；给每个右侧对象 j 配一个专属虚拟行，成本是 j 的惩罚 η_j；真实对象与非专属虚拟对象之间禁止配对，虚拟行与虚拟列之间成本为零。这样补成的方阵中，完整指派的成本恰为真实配对成本加上未配对对象的惩罚。

禁止的配对就是边集 E 中没有的边，在 KM 算法中不参与行最小值、扩树和松弛量。不要用随手取的“大成本”代替禁止：不够大时算法可能真的选它，够大也把不可行掩盖成高成本的“解”。KM 算法在松弛量全为 $+\infty$ 时报告的不可行，对应原模型中某组行能用的列不够。

成本可以为负，行最小值初始化照样成立。所有边成本同加一个常数，完整指派的总成本都变化 r 倍的这个常数，最优解不变；这依赖配对数量固定，在允许不配对的模型中则会改变取舍。算法结束后，先删去或解释虚拟配对，再按原始成本重算真实成本和惩罚。

6.2.4 Auction 算法

Auction 算法（auction algorithm）又称拍卖算法，把行看作竞标者、列看作物品，用物品的价格协调多名竞标者对同一物品的争抢。本节介绍串行前向版本，主线采用 r 阶完整有限成本矩阵，即每个配对都允许。$r = 0$ 时返回空指派，$r = 1$ 时直接返回唯一配对；以下竞价过程假定 $r \geq 2$。

6.2.4.1 主要思想

仍用同一成本矩阵，甲、乙是竞标者，任务是物品。不加价格时两人都偏好任务 1；若只是“后来者抢走”，同一物品可以无休止地转手。需要让争抢付出代价：物品越受争抢，继续选它越不划算。为此给每个物品 j 配一个价格 p_j，竞标者 i 按调整成本 $c_{i,j} + p_j$ 比较物品。

先看甲的第一次出价。零价格下任务 1、2 的调整成本是 1 和 2，相差 1。若把任务 1 的价格恰好提高 1，两项就打平；并列时涨价为零，价格不再上涨，争抢可能无休止地继续。所以再多加一个小正数 ε。取 $\varepsilon = 1/4$，任务 1 涨价 $5/4$，甲所持任务的调整成本变为 $9/4$，比另一项选择的 2 只贵 $1/4$。甲接受的物品至多比最好选择差 ε，而每次出价至少涨价 ε。

接着乙看到 $9/4$ 和 4，仍选任务 1，把价格再提高 $4 - 9/4 + 1/4 = 2$，接管任务 1，如图 6.5(b)。甲重新比较 $17/4$ 与 2，转向任务 2，如图 6.5(c)。一般地，竞标者 i 在调整成本 $q_j = c_{i,j} + p_j$ 中找最小值 $q_{(1)}$ 及其物品 $j^\star$，再取排除 $j^\star$ 后的最小值 $q_{(2)}$，把 $p_{j^\star}$ 提高 $q_{(2)} - q_{(1)} + \varepsilon$。涨价后 $j^\star$ 的调整

成本恰为 $q_{(2)}+\varepsilon$，即“次好选择加 ε”。价格只用于协调选择，方案仍按原成本评价，为 3。

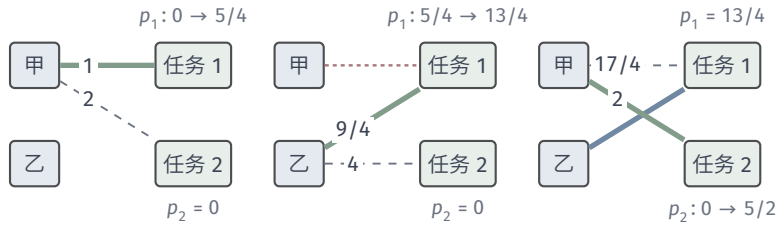


图 6.5 Auction 算法在成本矩阵 $\left[\begin{smallmatrix}1&2\\1&4\end{smallmatrix}\right]$ 上的三次出价，$\varepsilon = 1/4$。边旁为出价时的调整成本 $c_{i,j}+p_j$，物品旁为价格变化；绿色为本次所得，蓝色为保留的持有，红色点线为被抢走的持有。

与 KM 算法相比，Auction 算法不要求所选的边恰好紧，只要求每人手中的物品至多比最好选择差 ε。这换来了简单的局部操作，但要回答误差在 r 个人身上怎样累计、何时仍精确最优[5]。

[5] D. P. Bertsekas, *Linear Network Optimization: Algorithms and Codes*, MIT Press, 1991

6.2.4.2 算法流程

算法状态与初始化 输入是成本矩阵和 $\varepsilon > 0$，不算状态。手工竞价的四份记录对应以下状态：

- 价格 p_j：初值全为 0。
- 物品持有者 owner(j) 与竞标者所得 $\sigma(i)$：两套互相对应的记录，初值全为空。
- 未指派者队列 Q：初值含全部竞标者。

一次出价可能只是把物品从一人转给另一人，已指派的人数不变，所以停止条件看队列是否为空，而不看已经处理了多少人。每次只处理一名未指派者；最小值并列时取编号最小的物品。

竞价、易主与停止 从队列取出竞标者 i 后，一次出价分三步：

- 比较：扫描第 i 行，计算 $q_j = c_{i,j} + p_j$，求最小值 $q_{(1)}$ 及物品 $j^\star$，再求 $q_{(2)} = \min_{j \neq j^\star} q_j$。不同物品并列时 $q_{(2)} = q_{(1)}$，涨价恰为 ε。
- 涨价：$p_{j^\star} \leftarrow p_{j^\star} + q_{(2)} - q_{(1)} + \varepsilon$。
- 易主：若 $j^\star$ 已有持有者 k，置 $\sigma(k)$ 为空并把 k 放回队列；再置 owner($j^\star$) $= i$、$\sigma(i) = j^\star$。物品一旦有了持有者，此后始终有持有者。

队列为空时，每个竞标者都持有不同的物品，停止并输出指派 σ、原始成本 $C(\sigma)$ 和最终价格。

算法 6.3 Auction 算法

输入：$r \ge 2$ 阶有限成本矩阵 $\boldsymbol{C}$，$\varepsilon > 0$。输出：完整指派 σ、原始成本与价格 $\boldsymbol{p}$。

1. 置 $p_j = 0$，owner(j) 与 $\sigma(i)$ 全为空，全部竞标者入队 Q。
2. 若 Q 为空，转第 5 步；否则取出 i，对每个 j 计算 $q_j = c_{i,j} + p_j$，求 $q_{(1)} = \min_j q_j$ 及取到它的物品 $j^\star$，以及 $q_{(2)} = \min_{j \ne j^\star} q_j$。
3. 置 $p_{j^\star} \leftarrow p_{j^\star} + q_{(2)} - q_{(1)} + \varepsilon$。
4. 若 $k = \text{owner}(j^\star)$ 非空，置 $\sigma(k)$ 为空并把 k 加入 Q。置 owner$(j^\star) = i$、$\sigma(i) = j^\star$，回到第 2 步。
5. 输出 σ、$C(\sigma) = \sum_i c_{i,\sigma(i)}$ 与 $\boldsymbol{p}$。

运行示例 取 $\varepsilon = 1/4$，初始队列为甲、乙。三次出价都是伪代码第 2—4 步的同一循环，见表 6.3：

- 甲：选任务 1，涨价 $2 - 1 + 1/4 = 5/4$。
- 乙：选任务 1，涨价 2，价格变为 $13/4$；甲失去任务 1，重新入队。
- 甲：选任务 2，涨价 $17/4 - 2 + 1/4 = 5/2$，队列变空。

最终甲配任务 2、乙配任务 1，原始成本为 $2 + 1 = 3$。逐人核对：甲手中物品的调整成本是 $2 + 5/2 = 9/2$，当前最好选择是 $\min\{17/4, 9/2\} = 17/4$，恰好差 $1/4$；乙手中的任务 1 调整成本为 $17/4$，另一项为 $4 + 5/2 = 13/2$，乙持有的就是最好选择。最终价格不是原问题的额外支出。

表 6.3 Auction 算法在成本矩阵 $\left[\begin{smallmatrix} 1 & 2 \\ 1 & 4 \end{smallmatrix}\right]$ 上的出价，$\varepsilon = 1/4$。比较值为出价时的 $c_{i,j} + p_j$；价格与持有者为出价后的状态。

出价者	比较值	所选	涨价	价格 $\boldsymbol{p}$	持有者
甲	1，2	任务 1	5/4	$(5/4, 0)$	甲，—
乙	9/4，4	任务 1	2	$(13/4, 0)$	乙，—
甲	17/4，2	任务 2	5/2	$(13/4, 5/2)$	乙，甲

6.2.4.3 正确性与复杂度

运行示例结束时，甲的选择比最好选择差 $1/4$，乙没有差额。把这项性质一般化，就是算法维护的条件。

定义 6.8 ε 互补松弛 设 $\varepsilon \ge 0$，$\boldsymbol{p}$ 为价格，σ 为部分或完整指派。若每个已指派的竞标者 i 都满足

$$c_{i,\sigma(i)} + p_{\sigma(i)} \le \min_j \bigl(c_{i,j} + p_j\bigr) + \varepsilon,$$

就称 $(\sigma, \boldsymbol{p})$ 满足ε 互补松弛（ε-complementary slackness）。

$\varepsilon = 0$ 时，它就是“匹配边是紧边”的价格写法，下面的定理说明这一对应。

引理 6.9　ε 互补松弛的保持　初始状态满足 ε 互补松弛，此后每次出价都保持它。

证明　初始时没有人被指派，条件自动成立。考虑竞标者 i 对 $j^\star$ 的一次出价。对新持有者 i，涨价后 $c_{i,j^\star} + p_{j^\star} = q_{(2)} + \varepsilon$，而其他物品的调整成本都不小于 $q_{(2)}$，$j^\star$ 本身的调整成本也不小于 $q_{(2)} + \varepsilon$，所以条件成立。对其他仍持有物品的竞标者 k，只有 $p_{j^\star}$ 变了，而 k 持有的不是 $j^\star$：k 手中物品的调整成本不变，右端的最小值不会下降，条件继续成立。失去 $j^\star$ 的人回到未指派状态，不受条件约束。 ■

定理 6.10　Auction 算法的误差界　设成本矩阵完整有限，Auction 算法以完整指派 σ 和价格 $\boldsymbol{p}$ 停止，$C^\star$ 为最优指派成本。则

$$0 \le C(\sigma) - C^\star \le r\varepsilon.$$

若全部成本为整数且 $0 < \varepsilon < 1/r$，则 σ 是最优指派。

证明　令 $u_i = \min_j (c_{i,j} + p_j)$、$v_j = -p_j$。对任意 i, j 有 $u_i \le c_{i,j} + p_j$，即 $u_i + v_j \le c_{i,j}$，所以 $(\boldsymbol{u}, \boldsymbol{v})$ 是可行标号，由第 6.2.2小节，$\sum_i u_i + \sum_j v_j \le C^\star$。由 ε 互补松弛，$c_{i,\sigma(i)} \le u_i - p_{\sigma(i)} + \varepsilon = u_i + v_{\sigma(i)} + \varepsilon$，即每条匹配边的剩余量至多为 ε。对 i 求和，σ 是置换，每个 v_j 恰出现一次，

$$C(\sigma) \le \sum_i u_i + \sum_j v_j + r\varepsilon \le C^\star + r\varepsilon.$$

$C(\sigma) \ge C^\star$ 是最优值的定义。成本为整数时，$C(\sigma) - C^\star$ 是小于 $r\varepsilon < 1$ 的非负整数，只能为 0。 ■

一般的实数成本在固定的正 ε 下只有这个误差界。两人例中 $r\varepsilon = 1/2 < 1$，成本为整数，所以得到的成本 3 精确最优。

有限终止要用到“完整有限成本”和“零初始价格”。记 $\Delta = \max_{i,j} c_{i,j} - \min_{i,j} c_{i,j}$。物品一旦被出价就始终有持有者，已持有物品的数目等于已指派的人数；只要还没停止，就至少有一个物品从未被出价，价格仍为 0。竞

标者 i 对 $j^\star$ 出价时，若另有从未被出价的物品 $j_0 \neq j^\star$，则 $q_{(2)} \le c_{i,j_0}$，涨价后

$$p_{j^\star} = q_{(2)} + \varepsilon - c_{i,j^\star} \le c_{i,j_0} - c_{i,j^\star} + \varepsilon \le \Delta + \varepsilon.$$

若没有这样的 j_0，$j^\star$ 就是最后一个未被出价的物品，这次出价后人人都有物品，算法停止。每次出价至少涨价 ε，同一物品被出价 k 次后价格至少为 $k\varepsilon$，所以除停止前的最后一次外，每个物品至多被出价 $\Delta/\varepsilon + 1$ 次，总出价次数

$$B \le r\big(\Delta/\varepsilon + 1\big) + 1.$$

这个论证依赖每个物品都可选：禁配实例若不可行，价格可以无限上涨而永不停止。

按总出价次数 B 分析稠密实现：每次出价扫描一行为 $O(r)$，读入矩阵为 $O(r^2)$，总时间 $O(r^2 + rB)$；价格、持有者、指派与队列为 $O(r)$，成本表另占 $O(r^2)$。

由上面的界，整数成本且取 $\varepsilon = 1/(r+1)$ 时，$B = O(r^2\Delta)$，总时间为 $O(r^3\Delta)$，随成本范围增长，不能把 KM 算法的 $O(r^3)$ 直接套用到 Auction 算法上。ε 越小，误差界越紧，但需要的出价可能越多；更强的复杂度界依赖下一小节的缩放策略[6]。

6.2.4.4 误差缩放与适用边界

固定一个很小的 ε，开始阶段价格从零慢慢上涨，许多出价只是一点点地挪动价格。ε **缩放**（ε-scaling）先用较大的 ε 完成一个较粗的指派，得到大致合理的价格，再逐阶段缩小 ε，并把上一阶段的价格作为下一阶段的初值。

教学上最简单的做法是每阶段保留价格、清空指派、全部竞标者重新入队竞价；若想保留部分配对，须先解除不满足新 ε 条件的配对再继续。只缩小参数而保留完整指派是错误的：队列已空，算法会直接停止，新的 ε 条件根本没有被检查。最后一阶段的 ε 按定理 6.10的要求选取[7]。

非方阵、未分配惩罚等情形，先按第 6.2.3.4小节补成方阵。允许边不完整时，竞标者只在自己的允许物品中比较，这时要先用第 6.1.3小节的方法确认存在完美匹配，因为基本算法不会自行识别不可行。

只有一个允许物品的行，排除最好选择后没有次好值，涨价公式无法使用；这样的行在任何完整指派中都只能配这个物品，可以先把这一对固定下来，删去相应的行和列。不能把不存在的次好值当成 $+\infty$ 代入公式，那会让价格变成无穷。

浮点实现中，涨价、并列判断和 ε 条件都受舍入影响，ε 很小时涨价可能被舍入吞掉一部分。整数成本下“$\varepsilon < 1/r$ 即精确最优”的结论，要求算

[6] D. P. Bertsekas, *Auction Algorithms for the Assignment Problem, Chapter 2 of Auction Algorithms for Path Planning, Network Transport, and Reinforcement Learning*, Mar. 15, 2025, URL: https://www.mit.edu/~dimitrib/Ch2_AUCTION.pdf

[7] D. P. Bertsekas, *New Auction Algorithms for the Assignment Problem and Extensions* In Results in Control and Optimization, 2024, vol. 14, pp. 100383

术精度足以区分相差 ε 的值。并行和异步竞价要处理同时争抢的冲突，可作为后续阅读，其复杂度结论不能搬到本节的串行版本。

6.2.5 方法比较与结果检验

两种算法解同一个方阵指派。KM 算法保持可行标号、只在紧边上增广，结束时给出精确证书，可直接处理禁配，稠密方阵上为 $O(r^3)$。Auction 算法维护价格和满足 ε 互补松弛的部分指派，每步一次出价和易主；其保证随 ε、成本整数性和算术精度而定，出价次数随成本范围与 ε 变化，比较前须先说明这些条件。

检验指派先看原始问题的可行性：每个真实对象按模型要求配对或计入惩罚，所用的列互不相同，全部配对都是允许边；再按原始成本重算总成本。

KM 算法的结果还要检查证书：对全部允许边验证 $u_i + v_j \le c_{i,j}$，对所选的边验证取等号，并核对总成本等于 $\sum_i u_i + \sum_j v_j$。在两人例中，四条边的剩余量依次为 $0, 0, 0, 2$，所选的甲—2、乙—1 剩余量为零，成本与下界同为 3。

Auction 算法的结果检查最终价格下每人的 ε 互补松弛，再由定理 6.10给出目标误差界；只有在整数成本与 $\varepsilon < 1/r$ 等条件下才能称为精确最优。浮点实现中，可行性、紧性和出价比较都只能在容差内判断，应说明所用容差，不把近似相等当成精确证书。

标号和价格都在为配对成本提供可以核查的界：可行标号直接给出下界，价格经 $u_i = \min_j (c_{i,j} + p_j)$、$v_j = -p_j$ 转成同样的下界。第 7章把这种给顶点配数的做法推广到一般容量与供需网络上的节点势；指派也能写成那里的特例，但统一的模型并不意味着必须选用相同的算法。

6.3 本章小结

本章开头区分了两种要求：只求配对最多，或要求每个对象都配上并使总成本最低。对第一种，二部图匹配与单位容量网络上的整数流一一对应，残量增广路径落到二部图上就是交替路径，于是匹配最大当且仅当不存在增广路径。这个判据比“不能再加边”强：极大匹配可能仍有增广路径，改进要允许中间顶点换伙伴。匈牙利算法只维护伙伴数组，逐个左侧顶点用深度优先搜索找增广路径，总时间 $O(n + pm)$，其中 p 为较小一侧的顶点数；Hopcroft–Karp 算法按层成批增广，得到 $O(n + m\sqrt{n})$。这些构造都依赖给定的两侧划分，不能直接用于含奇数长环的一般图。

对第二种，可行标号给出全部完整指派的共同下界，只用紧边的完美匹配达到这个下界，因而最优。KM 算法在紧边上生长交替树，受阻时按最小松弛量调标，使下界升高、树继续生长；结束时匹配与标号构成精确证

书，禁配也能直接处理，稠密方阵上为 $O(r^3)$。Auction 算法用价格协调争抢，维护 ε 互补松弛：最终指派的成本至多比最优值高 $r\varepsilon$，只有在整数成本且 $\varepsilon < 1/r$ 时才保证精确最优；出价次数随成本范围与 ε 变化，总时间为 $O(r^2 + rB)$。

两类结果的核对方式也不同。最大匹配用单位容量网络中与配对数同值的割核对；KM 算法的结果逐边核对标号可行与所选边取等号；Auction 算法的结果只能核对最终价格下的 ε 条件，再据此给出误差界。矩形、禁配与允许不配对的情形，都要先补成方阵并把惩罚写进成本，求解后按原始成本重算。

标号和价格都是给顶点配上的数，用来为配对成本提供可核查的界：价格经 $u_i = \min_j(c_{i,j} + p_j)$、$v_j = -p_j$ 就转成可行标号。下一章把这种做法推广为一般容量与供需网络上的节点势，最大流、匹配与指派也都能写成那里的最小费用流。

第 7 章

最小费用流

第 5章只问能送多少。实际调运中，各处的产量与需求事先给定，各条线路运价不同，要比较的是满足供需的方案中哪个总费用最低。本章回答三个问题：怎样把容量、供需和单位费用写进同一个模型；怎样判断一个方案已经最省；怎样从一个可行方案出发逐步改进到最优。

第 7.1节给出模型，并说明下界、供需平衡与可行性怎样处理。第 7.2节回答最优性：对方案的任何改动都可以看成沿一个环把货物改道，所以有省钱的环，当前方案就不是最优；没有，就是最优。在第 5.2节的残量网络上给每条残量弧配上费用，这一观察就成为负费用环判据；再借第 3章的最短距离构造节点势，判据化为逐弧可查的证书。

第 7.3节的**网络单纯形法**（network simplex method，亦称网络单形法）把方案记在一棵生成树上，每次只沿一个环改道。它是线性规划单纯形法在网络上的特例，这里只用树、环和守恒式建立它，不依赖一般的线性规划理论。第 7.4节把最大流、二部图匹配和指派写成最小费用流，第 7.5节说明怎样回到原始问题核对结果，第 7.6节简述沿同一残量结构组织改进的其他算法。

本章为了把字母 c 留给费用，把容量改记为上界 $u_{i,j}$，弧写作 (i,j) 或 $i \to j$；顶点数仍为 n，弧数仍为 a。

7.1 最小费用流问题

设想几个仓库经中转站向几家商店供货：库存要全部运出，需求要恰好满足；每条线路有运量上限，有些按合同还有最低运量，每运一单位付相应的运价。把各处看成顶点、线路看成有向弧，就得到下面的问题。

定义 7.1　最小费用流问题　设 $\mathcal{G}=(V,E)$ 是有限有向图，每个顶点 i 有供需量 $b_i \in \mathbb{R}$，每条弧 $(i,j)\in E$ 有有限的下界 $\ell_{i,j}$、上界 $u_{i,j}$

（$\ell_{i,j} \le u_{i,j}$）与单位费用 $c_{i,j} \in \mathbb{R}$。给每条弧赋一个流量 $x_{i,j}$，若向量 $\boldsymbol{x}$ 满足

$$\begin{aligned} \sum_{j:\,(i,j)\in E} x_{i,j} - \sum_{j:\,(j,i)\in E} x_{j,i} &= b_i \quad (i \in V), \\ \ell_{i,j} \le x_{i,j} &\le u_{i,j} \quad ((i,j) \in E), \end{aligned} \tag{7.1}$$

则称 $\boldsymbol{x}$ 为可行流。**最小费用流**（minimum-cost flow）问题是在全部可行流中最小化总费用 $\sum_{(i,j)\in E} c_{i,j} x_{i,j}$。

守恒式左边是顶点 i 的净流出，所以 $b_i > 0$ 表示 i 是供给点，要净送出 b_i 单位；$b_i < 0$ 表示需求点，要净收进 $-b_i$ 单位；$b_i = 0$ 的顶点只做中转。把全部守恒式相加，每条弧的流量正负各计一次，左边为零，所以 $\sum_{i\in V} b_i = 0$ 是可行流存在的必要条件。

它不充分：平衡的供需不保证有路可走、容量够用。图 7.1的网络中，若把 $s \to a$ 与 $s \to t$ 的容量都改成 1，从 s 出发的总容量只有 2，供需虽然平衡，问题却没有可行流。

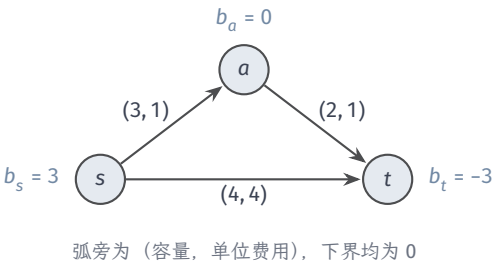


图 7.1　三点最小费用流算例。s 供给 3 单位，t 需求 3 单位，a 只做中转；弧旁括号依次为容量与单位费用，下界均为 0。绕行 $s \to a \to t$ 的单位费用为 2，低于直达弧的 4，但绕行量受 $a \to t$ 的容量 2 限制。

按弧序 $s \to a$、$a \to t$、$s \to t$ 把流量写成向量：3 单位全走直达弧是 $(0, 0, 3)$，费用 $3 \times 4 = 12$；让 2 单位绕行、1 单位直达是 $(2, 2, 1)$，费用 $2 \times (1 + 1) + 1 \times 4 = 8$。后面将证明 8 就是最小费用。

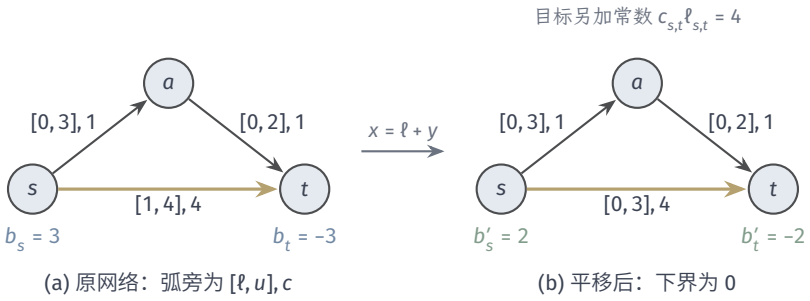
正下界（如合同规定的最低运量）的处理办法是先把下界部分“预先送出”，再对剩余部分求解。令 $x_{i,j} = \ell_{i,j} + y_{i,j}$，新变量满足 $0 \le y_{i,j} \le u_{i,j} - \ell_{i,j}$。预先送出的下界流量已经使顶点 i 净流出 $\sum_j \ell_{i,j} - \sum_j \ell_{j,i}$，所以 $\boldsymbol{y}$ 应满足的剩余供需为

$$b'_i = b_i - \sum_{j:\,(i,j)\in E} \ell_{i,j} + \sum_{j:\,(j,i)\in E} \ell_{j,i}.$$

目标变为 $\sum c_{i,j} y_{i,j} + \boldsymbol{c}^{\top}\boldsymbol{\ell}$，后一项是常数。两个问题的可行解与最优解通过 $\boldsymbol{x} = \boldsymbol{\ell} + \boldsymbol{y}$ 一一对应。

图 7.2把三点网络中 $s \to t$ 的下界改为 1：这条弧的容量从 4 变为 3，s 的剩余供给从 3 降为 2，t 的剩余需求从 3 降为 2，目标另加常数 $c_{s,t}\ell_{s,t} = 4$。只改容量而不改供需，下界那一单位就被重复计算。新问题的最优解是 $\boldsymbol{y} = (2,2,0)$，费用 4；加回下界与常数，得到原问题的最优流 $\boldsymbol{x} = (2,2,1)$ 与费用 8。

上下界相等的固定弧平移后容量为零，可以先删去，只保留它对供需和常数的贡献。以下除非特别说明，都假定下界已平移为零、固定弧已删去。



(a) 原网络：弧旁为 $[\ell, u], c$　　(b) 平移后：下界为 0

图 7.2　下界平移。(a) 中黄色弧 $s \to t$ 的流量须在 $[1,4]$ 内，弧旁依次为流量区间与单位费用。(b) 令 $x = \ell + y$ 后，该弧的区间变为 $[0,3]$，s 与 t 的供需同时各减少 1 单位（绿色），目标另加常数 4。

从源点 s 向汇点 t 输送固定量 F 时，令 $b_s = F$、$b_t = -F$，其余 $b_i = 0$。全部 $b_i = 0$ 时的可行流称为**循环流**（circulation）；第 7.4节会用它表达最大流。先尽量多送、再在全部最大流中挑费用最低的，称为**最小费用最大流**（minimum-cost maximum flow）：先用第 5章的方法求出最大流值 $v^\star$，再取 $F = v^\star$。

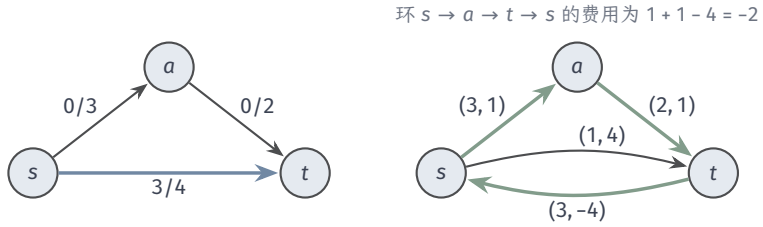
在有限上下界下，可行流集合是 $\mathbb{R}^a$ 中的闭有界集（$a = |E|$），总费用是线性函数，而闭有界集上的连续函数必能取到最小值（多元极值定理，可参考卷二第 3 章“赋范空间与内积空间”），所以只要可行流存在，最小费用一定能取到；可行性由后文网络单纯形法的第一阶段判断。若允许无穷容量，可无限推送的负费用环使费用没有下界；本章主线只讨论有限容量，无穷容量放在第 7.3.4小节。

7.2　费用残量网络与最优性条件

判断最优不能逐一比较全部可行流。这里沿用第 5.2节的残量网络描述流量还能怎样改，只是每条残量弧多带一个费用。

给定可行流 $\boldsymbol{x}$。对每条原弧 (i,j)，若 $x_{i,j} < u_{i,j}$，建立正向残量弧 $i \to j$，剩余容量 $u_{i,j} - x_{i,j}$，费用 $c_{i,j}$；若 $x_{i,j} > \ell_{i,j}$，建立反向残量弧 $j \to i$，剩余容量 $x_{i,j} - \ell_{i,j}$，费用 $-c_{i,j}$，表示撤回一单位省下 $c_{i,j}$。原网络同时有 $i \to j$ 与 $j \to i$ 两条弧时，各自的残量弧要记住来自哪条原弧，不能合并。

改道在残量网络中就是一个环。图 7.3从全走直达的流 $(0,0,3)$ 出发：把一单位改走中转，要在 $s\to a$、$a\to t$ 各加一单位，并从直达弧撤回一单位，三步首尾相接成残量环 $s\to a\to t\to s$，费用 $1+1-4=-2$。逐点看，s、a、t 各自少走一条弧、多走一条弧，净流出不变。逐边看，$s\to a$ 还能加 3，$a\to t$ 还能加 2，直达弧还能撤回 3，所以最多改道 $\min\{3,2,3\}=2$ 单位，费用从 12 降为 8。



(a) 可行流：弧旁为 x/u，费用 12　　(b) 残量网络：弧旁为（剩余容量，费用）

图 7.3　残量网络中的负费用环。(a) 全部 3 单位走直达弧的可行流，弧旁为流量与容量。(b) 它的残量网络，弧旁为剩余容量与费用；直达弧产生可追加 1 单位的正向弧和可撤回 3 单位、费用 -4 的反向弧 $t\to s$。绿色环费用 -2，推送 2 单位使费用降 4。

一般地，沿残量网络中的有向环 C 推送 $\theta>0$ 单位，是指正向残量弧对应的原弧流量加 θ，反向残量弧对应的原弧流量减 θ。环上每个顶点恰有一条环弧进入、一条离开，净流出不变；θ 不超过环上的最小剩余容量时，上下界仍然满足。费用改变 $\theta\,c(C)$，其中 $c(C)$ 是环上残量费用之和。所以有 $c(C)<0$ 的环时当前流不是最优。反过来也成立，关键是任何可行流与当前流之差都能拆成残量环的叠加。

定理 7.2　负费用环判据　可行流 $\boldsymbol{x}$ 是最小费用流，当且仅当它的残量网络中不存在费用为负的有向环。

证明　必要性已由上面的推送论证给出，下证充分性。任取可行流 $\boldsymbol{x}'$，令 $\boldsymbol{d}=\boldsymbol{x}'-\boldsymbol{x}$。两个流满足同一组供需，所以 $\boldsymbol{d}$ 在每个顶点的净流出为零。逐条考察原弧 (i,j)：若 $d_{i,j}>0$，则 $x_{i,j}<x'_{i,j}\le u_{i,j}$，正向残量弧存在且剩余容量至少为 $d_{i,j}$，在它上面放 $d_{i,j}$；若 $d_{i,j}<0$，则 $x_{i,j}>x'_{i,j}\ge \ell_{i,j}$，反向残量弧存在，在它上面放 $-d_{i,j}$。这样得到残量弧上的一组非负数 $\boldsymbol{g}$，它在每个顶点流入等于流出，按残量费用加权的总和等于 $\boldsymbol{c}^{\top}\boldsymbol{x}'-\boldsymbol{c}^{\top}\boldsymbol{x}$。

若 $\boldsymbol{g}\neq\boldsymbol{0}$，从任一取正值的残量弧出发，沿正值弧前进：每到达一个顶点，它的流入为正，必有一条正值出弧可走。顶点有限，前进中必然回到某个走过的顶点，截出一个有向环 C_1。令 $\lambda_1>0$ 为 $\boldsymbol{g}$ 在 C_1

上的最小值，从环上各弧减去 λ_1，剩余部分仍非负且守恒，而取正值的弧至少少了一条。重复有限次，得到

$$\boldsymbol{c}^{\top}\boldsymbol{x}' - \boldsymbol{c}^{\top}\boldsymbol{x} = \sum_k \lambda_k\, c(C_k), \qquad \lambda_k > 0.$$

每个 C_k 都由残量弧组成。残量网络中没有负费用环时，右边每项非负，$\boldsymbol{x}'$ 不比 $\boldsymbol{x}$ 便宜。 ■

判据不是完整的算法，也不断言最优流唯一；而且环的个数可能随规模指数增长。为了改为逐弧检查，给每个顶点 i 配一个实数 π_i，称为**节点势**（node potential），并定义弧 (i,j) 的**约化费用**（reduced cost）

$$\bar{c}_{i,j} = c_{i,j} - \pi_i + \pi_j. \tag{7.2}$$

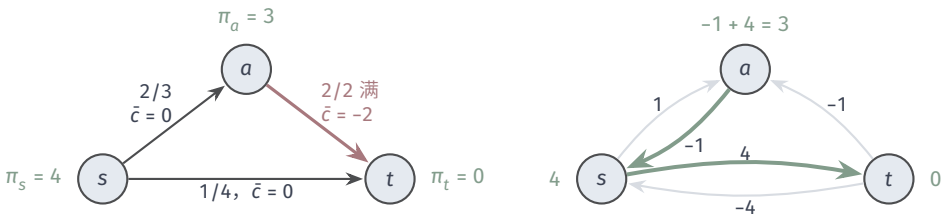
残量弧使用同一公式：反向残量弧 $j \to i$ 的费用为 $-c_{i,j}$，约化费用为 $-c_{i,j} - \pi_j + \pi_i = -\bar{c}_{i,j}$。沿有向环相加时，每个顶点的势在出弧上以 $-\pi$、在入弧上以 $+\pi$ 各出现一次，相消，所以环的约化费用等于原费用。因此，若能找到一组势，使所有残量弧的约化费用都非负，残量网络就没有负费用环。

反过来，残量网络没有负费用环时，这样的势一定存在。加入虚拟顶点 r，向每个顶点连一条零费用弧；新图的环不经过 r，仍无负环。记 d_i 为 r 到 i 的最短路径费用，绕环不会更便宜，只需比较有限条简单路径，所以 d_i 有限。

对残量弧 $i \to j$ 及其费用 c'，到 i 的最短路径接上这条弧是一条到 j 的行走，去掉其中的环不增加费用，所以 $d_j \le d_i + c'$。取 $\pi_i = -d_i$，即得 $c' - \pi_i + \pi_j = c' + d_i - d_j \ge 0$。实际可用第 3.4节的 Bellman–Ford 算法求 d_i，它报告负环时就找到了可推送的负费用环。

势可以理解为各顶点上货物的“边际运费”。把 π_i 看成从 i 再送一单位到需求点 t 的最低费用（$\pi_t = 0$），允许借助残量弧撤回已有流量。图 7.4以流 $(2,2,1)$ 为例：从 s 再送一单位，绕行已被 $a \to t$ 卡住，只能直达，$\pi_s = 4$；从 a 再送一单位，最便宜的是让一单位原本经 a 绕行的货改走直达，a 处这单位顶替它前往 t，省 1 付 4，$\pi_a = 3$。这正是由最短距离构造的势。

按这种理解，约化费用 $\bar{c}_{i,j} = c_{i,j} + \pi_j - \pi_i$ 比较两种送货方式：经弧 (i,j) 送出一单位，付 $c_{i,j}$ 再付 π_j；或按 i 现行的边际方式送出，付 π_i。$\bar{c}_{i,j} < 0$ 表示这条弧更便宜，应当多用；$\bar{c}_{i,j} > 0$ 表示更贵，应当少用。最优流中能加量的弧不能更便宜，能减量的弧不能更贵。



(a) 最优流与节点势

弧旁为流量 / 容量与约化费用

(b) 残量网络中再送一单位到 t

弧旁为残量费用；绿色为最便宜的路线

图 7.4　节点势的直观含义。(a) 流 $(2,2,1)$ 与势 $(4,3,0)$；界内弧约化费用为零，满载弧 $a \to t$ 为 -2（红色）。(b) 残量网络，弧旁为残量费用；绿色为从各顶点再送一单位到 t 的最便宜路线，费用 4 与 3 即 π_s, π_a。

“全部残量弧约化费用非负”于是逐弧翻译为：弧在下界时只有正向残量弧，要求 $\bar{c}_{i,j} \geq 0$；在上界时只有反向残量弧，要求 $\bar{c}_{i,j} \leq 0$；严格在两界之间时两条都存在，要求 $\bar{c}_{i,j} = 0$。

定理 7.3　节点势最优性证书　设固定弧已删去，$\boldsymbol{x}$ 是可行流，$\boldsymbol{\pi}$ 是一组节点势。若对每条弧 (i,j) 都有

$$\bar{c}_{i,j} \begin{cases} \geq 0, & x_{i,j} = \ell_{i,j}, \\ \leq 0, & x_{i,j} = u_{i,j}, \\ = 0, & \ell_{i,j} < x_{i,j} < u_{i,j}, \end{cases} \tag{7.3}$$

则 $\boldsymbol{x}$ 是最小费用流。

证明　任取可行流 $\boldsymbol{x}'$。把 $c_{i,j} = \bar{c}_{i,j} + \pi_i - \pi_j$ 代入费用差，

$$\boldsymbol{c}^{\top}(\boldsymbol{x}' - \boldsymbol{x}) = \sum_{(i,j)\in E} \bar{c}_{i,j}(x'_{i,j} - x_{i,j}) + \sum_{(i,j)\in E} (\pi_i - \pi_j)(x'_{i,j} - x_{i,j}).$$

第二个和按顶点归组，等于对每个 i 把 π_i 乘以顶点 i 在 $\boldsymbol{x}'$ 与 $\boldsymbol{x}$ 下净流出之差再求和；两个流满足同一组供需，这些差全为零。第一个和逐项非负：若 $x_{i,j} = \ell_{i,j}$，则 $x'_{i,j} - x_{i,j} \geq 0$ 且 $\bar{c}_{i,j} \geq 0$；若 $x_{i,j} = u_{i,j}$，则 $x'_{i,j} - x_{i,j} \leq 0$ 且 $\bar{c}_{i,j} \leq 0$；其余情形 $\bar{c}_{i,j} = 0$。所以 $\boldsymbol{c}^{\top}\boldsymbol{x}' \geq \boldsymbol{c}^{\top}\boldsymbol{x}$。 ■

删去固定弧是为了避免 $\ell_{i,j} = u_{i,j}$ 时前两个条件同时出现。这组条件在线性规划中称为互补松弛条件。

流量与势合起来构成证书：核对可行性和每条弧上的一个符号，就能确认最优，不必相信求解过程。在三点网络中取流 $(2,2,1)$ 与势 $(4,3,0)$，界

内的两条弧 $s \to a$、$s \to t$ 的约化费用都是零，上界弧 $a \to t$ 的约化费用为 $-2 \le 0$，所以费用 8 最小。

7.3 网络单纯形法

本算法用可行树基记录流量，用非树弧的约化费用寻找改善方向，再沿基本环调整。

7.3.1 主要思想

第 7.2节把三点网络的改道写成了负费用环 $s \to a \to t \to s$：每改道一单位省 2，最多改 2 单位，费用 $12 \to 8$。负环指明了改进方向，剩下的问题是有组织地找到它们：记录当前流的方式应使候选的环容易列出，环的费用和改道量容易算出。

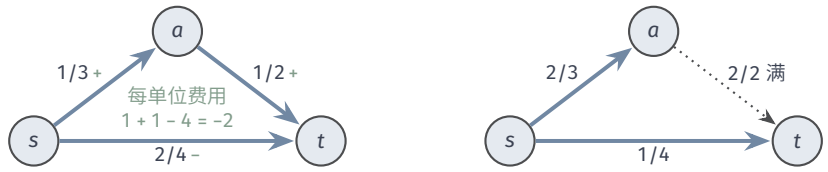
办法是把流记在一棵树上。规定 $s \to a$ 的流量固定为 0，只让 $s \to t$ 与 $a \to t$ 待定：中转点 a 没有外部供需，所以 $a \to t = 0$；t 要收的 3 单位只能来自 $s \to t$，所以 $s \to t = 3$。固定一部分弧后，其余流量顺着树由守恒式算出。一般地，忽略方向后构成生成树的弧集记为 T，称为树弧；其余弧的流量固定在下界或上界，分别组成 L 与 U。三元组 (T, L, U) 称为一个**树基**（spanning tree basis）。

叶子只关联一条树弧，它的守恒式中其余各项已知，于是解出这条树弧的流量；删去叶子再重复，就求出全部树弧流量。若它们都在上下界内，称 (T, L, U) 为可行树基。

把 $s \to a$ 加进树，恰好多出一个环，正是上面的改道环。一般地，非树弧与树上连接其两端的唯一路径合成的环称为**基本环**（fundamental cycle）。沿这个环推送 2 单位后，$a \to t$ 到达上界，它退出树、进入 U，$s \to a$ 进入树。这一步称为换基：它同时更新了表示当前流的树，使新流 $(2, 2, 1)$ 仍由新树确定。

只在树形方案中寻找，不会错过最优解。若严格位于两界之间的弧含有一个环，沿环的两个方向都能推送少量流量，两方向单位费用变化互为相反数，至少一个方向不增加费用；沿它推到某条弧碰界，环就被打断。反复这样做，界内弧不再含环，补上一些在界上的弧即成生成树。图 7.5中，流 $(1, 1, 2)$ 的三条弧都在界内而成环，推送 1 单位后 $a \to t$ 满载，得到 $(2, 2, 1)$，环被打断。树形方案只有有限多个。

同一棵树还给出节点势。令 $\pi_t = 0$，要求两条树弧约化费用为零，得 $\pi_s = 4$、$\pi_a = 1$；候选弧 $s \to a$ 的约化费用 $1 - 4 + 1 = -2$ 恰是每改道一单位的费用变化。这不是巧合：把基本环上每个 c 换成 $\bar{c} + \pi_i - \pi_j$，势相消，树弧约化费用为零，只剩非树弧自己的约化费用。所以定价，即检查非树弧约化费用的符号，就是逐个评估全部基本环。



(a) 界内弧成环：流 (1, 1, 2)，费用 10　　(b) 推送 1 单位后：流 (2, 2, 1)，费用 8

蓝色粗弧严格在上下界之间，点线弧在界上；弧旁为流量 / 容量

图 7.5　打断界内弧的环。(a) 流 (1, 1, 2) 的三条弧都在界内，按绿色符号沿环推送，每单位费用变化 −2。(b) 推送 1 单位后 $a \to t$ 到上界，界内弧不再成环，费用由 10 降为 8。

树上的路径可能逆着原弧方向走，$\pi_i - \pi_j$ 是沿树从 i 到 j 的有符号费用，顺弧计正、逆弧计负。它一般不是全图中的最短距离：第 7.2节把势解释为“边际运费”，是针对最优流说的。

概括起来，网络单纯形法只做三件事：只在树形方案中寻找；用一棵树同时决定流和势；每次找一条约化费用符号不对的非树弧，沿它的基本环改道，改到某条弧碰界为止，再让这条弧离开树。图 7.6画出了这个循环，系统的讲述可参考 Bertsekas 的网络优化教材[1]。

[1] D. P. Bertsekas, *Network Optimization: Continuous and Discrete Models*, Athena Scientific, 1998

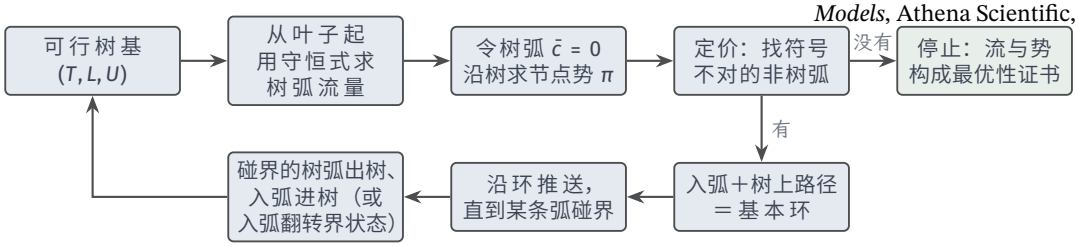


图 7.6　网络单纯形法的一轮。定价后若无符号不对的非树弧就停止，否则沿基本环推送到某弧碰界：碰界的树弧出树、入弧进树，或入弧自身翻转界状态。

7.3.2 算法流程

算法状态　手算用到的记录——每条弧送多少、哪些弧成树、其余弧停在哪个界、各点势多少——列在表 7.1中，并以 t 为根记下每个顶点的父顶点、父边与深度。父顶点和父边用于沿树找路，深度帮助入弧两端回溯到公共祖先。

表 7.1　三点网络在树 $T = \{s \to t, a \to t\}$ 下的状态，根为 t。左表为弧记录，右表为顶点记录。

弧	流量	界	费用	状态
$s \to a$	0	$[0, 3]$	1	L
$a \to t$	0	$[0, 2]$	1	T
$s \to t$	3	$[0, 4]$	4	T

顶点	势	父顶点	父边	深度
s	4	t	$s \to t$	1
a	1	t	$a \to t$	1
t	0	—	—	0

平移下界、删去固定弧后的网络及其邻接结构在开始前构造，运行中不变，不算状态。算法维护以下状态：

- 树基 (T, L, U)：初值为第一阶段的星形树基，第二阶段开始时换成清理人工结构后得到的树基；
- 各弧流量 $x_{i,j}$：初值由初始树基逐叶解出，连同防循环规则所需的扰动系数一起存储；
- 节点势 π_i：根的势为零，其余由树弧约化费用为零解出；
- 树的记录 parent(i)、parent_edge(i)、depth(i)：取一个顶点为根，初值由一次树遍历得到。

树弧的流量允许恰好在界上：一条弧是否属于 T 取决于它在树中的位置，而不是流量是否在界内。所以同一个流可能对应不同的树基，后面的退化现象正来自这种区别。

势沿树求解：根的势为零，沿树弧方向从已知的 π_i 走到 j 时取 $\pi_j = \pi_i - c_{i,j}$，逆着树弧方向从已知的 π_j 走到 i 时取 $\pi_i = \pi_j + c_{i,j}$。换一个根只让全部势同加一个常数，约化费用不变。

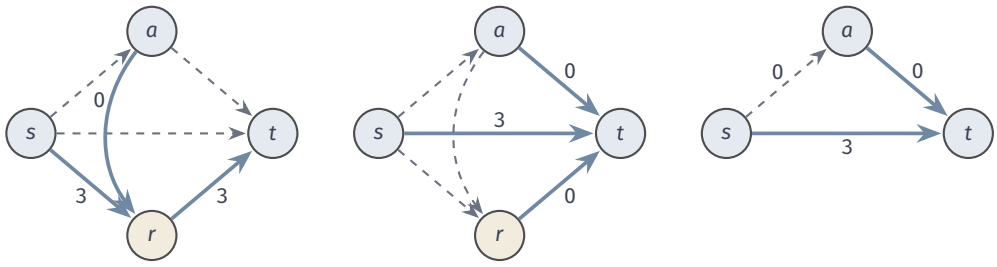
生成树要求底层无向图连通。消去固定弧后若原图不连通，就在各弱连通分量中分别处理：每个分量内的剩余供需之和必须为零，否则问题不可行；孤立顶点没有任何弧，它的供需必须为零。

初始化与第一阶段　随手选的树未必可行：在三点网络中取 $T = \{s \to a, a \to t\}$、令 $s \to t$ 取下界，叶子 s 与 t 迫使 $x_{s,a} = x_{a,t} = 3$，超过 $a \to t$ 的容量 2。办法是暂时“借用”虚拟线路：加一个只供记账的人工中转点 r，s 把 3 单位送进 r，r 再送 3 单位给 t，a 用零流量的人工弧 $a \to r$ 接入。账目先平衡了，但这不代表原网络可行。

一般地，在下界已平移为零的网络中令全部原弧流量为零，供给点连人工弧 $i \to r$、流量 b_i，需求点连人工弧 $r \to i$、流量 $-b_i$，零供需点连人工弧 $i \to r$、流量 0；人工弧容量取总供给 $\sum_{b_i>0} b_i$。人工弧构成以 r 为中心的星形生成树，是显式的可行树基。

借用的线路要尽量还回去。第一阶段令人工弧每单位记费用 1、原弧记 0，目标就是人工流总量，用下面同一套选弧、环调整与防循环操作求解。最优值为零即原问题可行，为正即不可行（定理 7.4）。

在三点网络中，星形树为 $s \to r$、$r \to t$、$a \to r$，流量 $3, 3, 0$，人工流总量 6。以 r 为根，势为 $(\pi_s, \pi_a, \pi_t) = (1, 1, -1)$，先选约化费用为 -2 的 $s \to t$ 入树，沿环 $s \to t \to r \to s$ 推送 3 单位，$s \to r$ 与 $r \to t$ 同时降为零，防循环规则选 $s \to r$ 出树；再选 $a \to t$ 入树，环上的 $r \to t$ 与 $a \to r$ 都已为零，步长为零，规则选 $a \to r$ 出树。此时人工流为零，见图 7.7。



(a) 星形初始树，人工流 6　　(b) 第一阶段结束，人工流 0　　(c) 删去人工根后的可行树基

粗蓝线：树弧（旁注流量）；虚线：取下界的非树弧；黄色顶点为人工根

图 7.7　两阶段法的第一阶段。(a) 人工根 r（黄色）与人工弧构成星形可行树，人工流为 6。(b) 两次换基后人工流为零，$s \to t$ 承担 3 单位，空载的 $r \to t$ 仍在树中。(c) 删去人工结构，剩下的树弧构成原网络的生成树，作为第二阶段的初始树基。

人工流归零后删去人工根和全部人工弧。原来的树弧去掉人工弧后留下一个森林；在原图的每个弱连通分量内，用连接不同森林分量的原非树弧把它补成生成树。补入的弧保持原流量，作为树弧恰在界上是允许的。随后恢复原费用、重新计算势，并从所得的普通可行树基重新设置防循环数据，进入第二阶段。三点网络中删去 r 后，$s \to t$ 与 $a \to t$ 已经构成生成树，$s \to a$ 在下界。

选入弧与沿环调整　每一轮按以下步骤进行：

- 定价：用当前势计算非树弧的约化费用。L 中 $\bar{c}_{i,j} < 0$ 的弧应沿正向增加流量，U 中 $\bar{c}_{i,j} > 0$ 的弧应沿反向减少流量；从这些违反条件的弧中选一条作为入弧。
- 找环：从入弧两端沿父指针上溯，深度大的一端先走，深度相同后一起走，相遇处是最近公共祖先；两段路径与入弧合成基本环。按入弧的调整方向给环定向，顺着定向经过的弧增加流量，逆着定向经过的弧减少流量。
- 求步长：增加的弧计算 $u - x$，减少的弧计算 $x - \ell$，入弧自身也参加比较，取最小值作为步长 θ。达到最小值的弧称为阻塞弧，可能不止一条，按防循环规则从中选出一条作为出弧。
- 更新流量：增加的弧加 θ、减少的弧减 θ，费用改变 θ 乘以入弧在调整方向上的约化费用，$\theta > 0$ 时严格下降。
- 更新树基：出弧是树弧时，它离开树，按到达的是下界还是上界归入 L 或 U，入弧加入 T，再更新父顶点、深度与势。出弧是入弧自身时，入弧在 L 与 U 之间翻转，树和势都不变。

退化、停止与输出　步长为零的换基称为**退化换基**（degenerate pivot）。它发生在基本环上某条树弧已在阻塞方向的界上时：流量和费用不变，树基却变了。连续的退化换基可能在若干树基之间兜圈子，所以阻塞弧有多

条时选哪一条必须写进算法。主流程采用适用于有界变量的**词典序规则**（lexicographic rule）：每个阶段开始时，在当时的可行树基上建立一致的符号扰动，比较阻塞量时连同扰动一起比较，并随换基更新扰动，完整构造见第 7.3.4小节。任意选择或只取编号最小的阻塞弧都不能排除循环。

全部非树弧都满足符号条件，即 L 中 $\bar{c} \geq 0$、U 中 $\bar{c} \leq 0$ 时，本阶段结束：第一阶段据人工流总量判定可行性，第二阶段返回原流、费用与势。

算法 7.1　网络单纯形法

输入：有限有向网络，供需 b_i，有限上下界与单位费用。输出：最优流 $\boldsymbol{x}$、费用与节点势 $\boldsymbol{\pi}$，或“不可行”。

1. 平移下界、删去固定弧，记下常数 $\boldsymbol{c}^{\mathsf{T}}\boldsymbol{\ell}$；在每个弱连通分量内检查剩余供需之和为零，否则报告不可行。
2. 第一阶段：增加人工根与人工弧，取星形可行树基，原弧费用为零、人工弧费用为一，按当前树基设置词典序扰动。
3. 以根为起点遍历树，求父顶点、深度与节点势，计算非树弧的约化费用；若没有违反符号条件的非树弧，转第 6 步。
4. 选一条违反条件的入弧，构造基本环，比较环上各弧（含入弧）的剩余量得到步长 θ，按词典序规则选出出弧。
5. 沿环更新流量；出弧是树弧时作树弧交换，否则作界翻转。回到第 3 步。
6. 第一阶段停止时，若人工流总量为正，报告不可行；否则删去人工结构，在各分量内补成原网络的可行树基，恢复原费用并重新设置扰动，以第二阶段身份回到第 3 步。第二阶段停止时，把下界加回流量、常数加回费用，合并各分量的结果，返回流量、费用与节点势。

运行示例　第一阶段结束后，三点网络的可行树基为 $T = \{s \to t, a \to t\}$、$L = \{s \to a\}$，流量 $(0, 0, 3)$，费用 12，势 $(\pi_s, \pi_a, \pi_t) = (4, 1, 0)$，即表 7.1的状态。第二阶段的一轮如下，见图 7.8(a)(b)：

- 定价：$s \to a$ 在下界，$\bar{c}_{s,a} = 1 - 4 + 1 = -2 < 0$，正向入弧。
- 找环：s 与 a 的父顶点都是 t，基本环为 $s \to a \to t$ 再逆着 $s \to t$ 回到 s；$s \to a$、$a \to t$ 增加，$s \to t$ 减少，正是“改走中转、减少直达”。
- 求步长：三条弧的剩余量依次为 3、2、3，$\theta = 2$，由 $a \to t$ 唯一达到。
- 更新：流量变为 $(2, 2, 1)$，费用下降 $2 \times 2 = 4$，为 8。$a \to t$ 到达上界，出树进入 U；$s \to a$ 入树，a 的父顶点改为 s、深度变为 2，顺着 $s \to a$ 得 $\pi_a = 4 - 1 = 3$，势为 $(4, 3, 0)$。
- 停止：唯一的非树弧 $a \to t$ 在上界，$\bar{c}_{a,t} = -2 \leq 0$，即第 7.2节的证书。

把 $s \to a$ 的容量改为 1，从同一状态出发，$s \to a$ 仍正向入弧，剩余量变为 1、2、3，步长 1 由入弧自身达到。于是作界翻转：$s \to a$ 从 L 转入 U，树和势不变，流量 $(1, 1, 2)$，费用 10，见图 7.8(c)。$s \to a$ 在上界而

$\bar{c}_{s,a} = -2 \le 0$，算法停止。两例只因入弧容量不同，阻塞弧一次是树弧、一次是入弧自身。

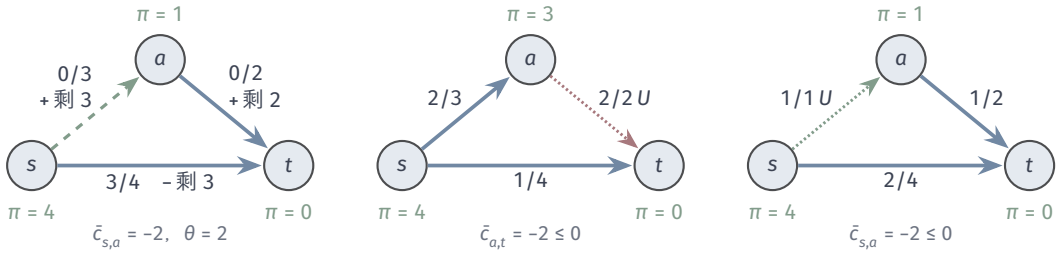


图 7.8　第二阶段的两种迭代。弧旁为流量/容量，“± 剩”为沿基本环的剩余量。(a)$\bar{c}_{s,a} = -2$，最多推送 2 单位。(b) 树弧 $a \to t$ 出基进入 U，费用 $12 \to 8$。(c)$u_{s,a} = 1$ 时入弧自身先到上界，树不变，费用 10。

图 7.9在原网络上构造一次退化换基：把 $s \to t$ 的容量改为 1，从 $T = \{s \to a, a \to t\}$、$U = \{s \to t\}$ 出发，流量 $(2,2,1)$，树弧 $a \to t$ 已在上界。以 t 为根，势为 $(2,1,0)$，$s \to t$ 在上界而 $\bar{c}_{s,t} = 4 - 2 + 0 = 2 > 0$，应反向进入；基本环上 $a \to t$ 要增加流量，剩余量为 0，所以 $\theta = 0$。$a \to t$ 出树进入 U，$s \to t$ 入树，流量和费用不变而树基变了；新势 $(4,3,0)$ 下 $\bar{c}_{a,t} = -2 \le 0$，算法停止。这次退化换基换来了证书，一般情形下连续退化则可能循环。

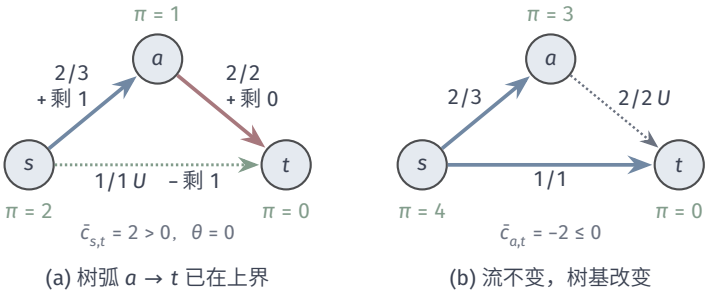


图 7.9　退化换基（$u_{s,t} = 1$）。(a) 入弧 $s \to t$（绿色）在上界且约化费用为正，反向进入；树弧 $a \to t$（红色）剩余量为零，步长为零。(b)$a \to t$ 出基后流量与费用不变，新势满足最优性条件。

7.3.3 正确性与复杂度

三点改道可分三件事检查：仍送到 3 单位，不超容量，每单位费用降 2。一般情形逐项对应：环上每个顶点一进一出，净流出不变；步长取环上剩余量的最小值，上下界仍满足；费用变化等于步长乘以入弧在调整方向上的负约化费用。表示也保持合法：树弧交换删去的出弧在基本环上，删去后树仍连通无环，新的 T 仍是生成树，出弧落在它到达的界上，被正确归入 L 或 U；界翻转只改变入弧的界状态。

树流由树基唯一确定，这可以用关联矩阵说明。按第 1.5节的约定，关联矩阵 $\boldsymbol{B}$ 中弧 (i,j) 对应的列在第 i 行为 1、第 j 行为 -1，守恒式就是 $\boldsymbol{B}\boldsymbol{x}=\boldsymbol{b}$。$\boldsymbol{B}$ 各行之和为零，删去根所在的一行不丢失信息，记剩下的矩阵为 $\tilde{\boldsymbol{B}}$，树弧对应的列组成 $(n-1)$ 阶方阵 $\tilde{\boldsymbol{B}}_T$。按消去叶子的次序排列行与列，每消去一个叶子，它所在的行在剩余树弧列中只有一个非零元，所以 $\tilde{\boldsymbol{B}}_T$ 经重排后是对角元为 ±1 的三角矩阵，从而可逆。于是 $\boldsymbol{x}_T=\tilde{\boldsymbol{B}}_T^{-1}(\tilde{\boldsymbol{b}}-\tilde{\boldsymbol{B}}_N\boldsymbol{x}_N)$ 唯一确定，其中 $N=L\cup U$。

定理 7.4　网络单纯形法的正确性　在精确运算下，采用词典序规则的网络单纯形法在有限轮后停止。第一阶段最优人工流为零当且仅当原问题可行；可行时第二阶段输出的流是最小费用流，停止时的势与之构成最优性证书。若供需与上下界都是整数，输出的流为整数流。

证明　有限终止：在一个阶段内，扰动后的供需固定不变，扰动后的流由树基唯一确定，所以扰动后的费用是树基的函数。词典序规则使扰动后的问题保持非退化，每轮扰动后的步长严格为正，扰动后的费用严格下降（第 7.3.4小节证明），因此已出现过的树基不会再出现。T 是有限多棵生成树之一，每条非树弧只有两种界状态，树基只有有限个，所以每个阶段在有限轮后停止。

可行性判定：若原问题有可行流，配上全零人工流就是第一阶段的可行解，目标为零，所以最优值为零；反之，最优值为零时人工弧全部空载，原弧流量满足原问题的全部约束。最优性：每个阶段停止时树弧约化费用为零，非树弧满足所在界对应的符号条件，由定理 7.3，当前流在本阶段的费用下最优。整数性：初始星形树的流量是供需本身；树流由三角方程组逐叶解出，每步只做整数加减；步长是若干个整数剩余量的最小值，沿环更新仍是整数加减。所以每轮的流都是整数流。 ■

整数性为第 7.4节的归约提供依据：只要可行，就存在整数最优流。第二阶段从清理后的树基重新设置扰动，有限终止的论证在两个阶段分别成立。

设顶点数为 n，弧数为 a。每轮扫描全部非树弧定价需要 $O(a)$；沿父指针找基本环、比较剩余量、更新流量都是 $O(n)$；沿新树重新求父顶点、深度与势也是 $O(n)$。所以每轮主要操作为 $O(n+a)$。

流量、界与费用的数组占 $O(a)$，树记录与势占 $O(n)$；词典序规则另为每条树弧存储 $n-1$ 个扰动系数，共 $O(n^2)$，每轮比较和更新这些系数也需

$O(n^2)$。总开销还取决于迭代轮数，其中包括退化换基；有限性论证只给出与树基个数相当的上界，不能从单轮开销推出统一的多项式总时间界。

7.3.4 实现改进与数值处理

词典序规则的完整构造作为补充阅读。把上界写成等式 $x_{i,j}+s_{i,j}=u_{i,j}$，$s_{i,j}\ge 0$ 是上界松弛量。每条弧有两个离界距离 $x_{i,j}-\ell_{i,j}$ 与 $s_{i,j}$；只有每条树弧的两个离界距离都为正，步长才一定为正。词典序规则用符号扰动人为地制造这种非退化情形。

在一个阶段开始时，把当前可行树基的树弧编号为 $e_1,\dots,e_{n-1}$，把 e_k 的流量改为 $x_{e_k}+\sigma_k\varepsilon^k$：若 e_k 在上界取 $\sigma_k=-1$，否则取 $\sigma_k=+1$。供需随之改为扰动后流量对应的净流出，并在整个阶段内保持不变；非树弧仍恰在界上。ε 是充分小的正数，每个量记为 $(\text{常数},\varepsilon\text{的系数},\dots,\varepsilon^{n-1}\text{的系数})$，从常数项起逐分量比较，即词典序比较。这样每条树弧的两个离界距离在词典序下都严格为正。

每轮比较步长时，增加流量的树弧以 $u-x(\varepsilon)$ 为候选，减少流量的树弧以 $x(\varepsilon)-\ell$ 为候选，入弧以 $(u-\ell,0,\dots,0)$ 为候选，取词典序最小者作为步长 $\theta(\varepsilon)$ 并让它出基，所有树弧的扰动流量按 $\theta(\varepsilon)$ 一同更新。

还原到基本环上，这个规则只在实际步长并列时起作用：比较并列阻塞弧的 ε 系数，选词典序最小者出基，实际流量仍按常数项更新。例如三点网络的第一阶段按 $s\to r$、$r\to t$、$a\to r$ 的次序设置扰动，流量为 $3-\varepsilon$、$3-\varepsilon^2$、ε^3；$s\to t$ 入树时 $s\to r$ 与 $r\to t$ 的候选向量为 $(3,-1,0,0)$ 与 $(3,0,-1,0)$，前者更小，所以 $s\to r$ 出基。

> **词典序规则保持非退化的证明（补充阅读）** 初始时各树弧的 ε 系数行是 $\pm$ 单位向量，线性无关。每次树弧交换，留下的树弧系数行加减出弧的系数行，入弧取出弧系数行的正负，这是可逆的行变换；界翻转不改变系数行。所以树弧的系数行始终线性无关，两条树弧的候选向量不会相等，入弧的候选向量只有常数项，也不会与树弧的相等，词典序最小者唯一。更新后出弧恰好落在界上，其余树弧仍在词典序下严格离界，扰动后的问题始终非退化，每轮的 $\theta(\varepsilon)$ 在词典序下严格为正。扰动后的费用改变量为 $\theta(\varepsilon)$ 乘以负的约化费用，因而严格下降。 ■

另一类常用规则维护所谓强可行树，有自己的方向约定与初始化要求；第一阶段清理只得到普通的可行树基，两类规则不能混用。

常见的实现改进只改变维护代价，不改变可行性与最优性条件。一是局部更新势：删去出弧后树分成两块，只有不含根的一块的势同加或同减入弧的约化费用，只需遍历这棵子树。二是按先序后继与子树大小存储树，

换基时只重排被移动的子树。三是分块定价：每轮只扫描一部分非树弧，找到违反条件的弧就入树。

找环、界翻转和子树更新的写法可以对照 NetworkX 库中的网络单纯形实现[2]。，但它的初始化与防循环规则另有前提，不能与本章的两阶段词典序版本混用。

[2] NetworkX Developers, *Source Code for networkx.algorithms.flow.networksimplex*, 2026, URL: https://networkx.org/documentation/stable/_modules/networkx/algorithms/flow/networksimplex.html

精确整数数据下，上述判断都能精确进行。浮点实现中，守恒、弧界、约化费用的符号与步长比较都只能在容差内判断，例如把绝对值小于容差的约化费用当作零；容差过小会因舍入反复选弧，过大会漏掉改进方向。

有限容量的主线不会出现费用无界。若扩展到无穷容量，还须处理增加方向剩余量全为无穷的基本环：这时步长无穷而入弧约化费用为负，存在可以无限推送的负费用环，应报告费用无下界。

7.4 最大流、匹配与指派的统一表示

第 5章与第 6章的问题都能写成最小费用流。最大流的流值不固定，不能直接写成供需，办法是令全部 $b_i = 0$，原弧费用为零，再加一条从 t 回到 s 的回边，容量取源点原出弧容量之和，费用为 -1。对任一循环流，由源点守恒，s 经原弧的净流出等于回边流量 $x_{t,s}$，即原网络的源汇流值，总费用为它的相反数。反过来，值为 v 的流配上 $x_{t,s} = v$ 就是循环流，v 不超过源点出容量，回边容量够用。于是最小费用循环流对应最大流，最小费用就是负的最大流值。只设零费用而不加回边则不够：全零流已是最优。

二部最大匹配沿用第 6.1.2小节的单位容量网络：s 连向左侧各顶点，允许边从左指向右，右侧各顶点连向 t，容量都是 1、费用为零，再加入上述费用为 -1 的回边，容量取两侧顶点数的较小者。由定理 7.4的整数性，存在整数最优循环流，其中间弧上流量为 1 的弧两两不共享端点，构成匹配，配对数等于回边流量；反之每个匹配都给出这样的循环流，所以费用最小等同于配对数最多。

r 阶指派在同一个单位容量配对网络中固定输送量 r：令 $b_s = r$、$b_t = -r$，中间弧的费用为配对成本 $c_{i,j}$，其余弧费用为零，不加回边。整数可行流使每个左侧顶点恰送出一单位、每个右侧顶点恰收到一单位，对应一个完美匹配，总费用就是指派成本。

由整数性，只要可行就存在整数最优流，它给出最优指派；若送不出 r 单位，说明不存在完美匹配，指派不可行。若要最大化收益，费用取收益的相反数；矩形问题按第 6.2.3.4小节补成方阵后写入网络，求解后删去虚拟配对。

图 7.10汇总了这些关系。最小费用流保留了一般的容量与供需；最大匹配只求配对数量，指派在完整配对的条件下比较成本。

专门算法利用各自的结构：最大流算法只沿源汇方向增广，匈牙利算法与 Hopcroft–Karp 算法只记录配对伙伴，KM 算法的标号与 Auction 算法的价格是二部图上的节点势。统一表示说明问题可以转化，并不意味着网络单纯形法在所有输入上都优于专门算法。

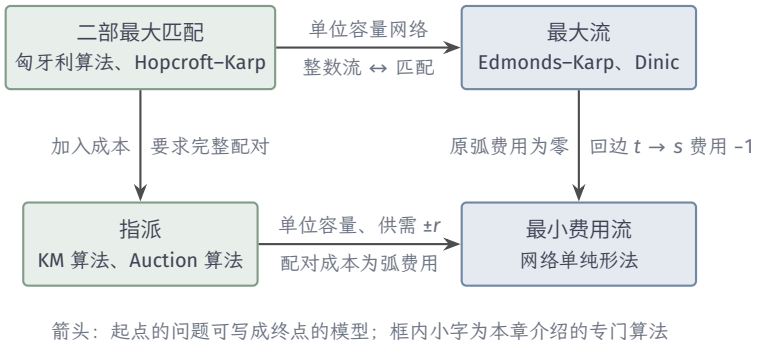


图 7.10　本章四种流与配对问题之间的模型关系。箭头表示起点的问题可以写成终点的模型：二部最大匹配是单位容量的最大流，加入成本并要求完整配对就成为指派；最大流加入费用为 -1 的回边后成为最小费用循环流；指派是供需为 $\pm r$ 的单位容量最小费用流。框内小字是本章介绍的专门算法。

7.5 结果检验

输出须回到原始问题检查，因为算法求解的是平移下界、删去固定弧之后的问题。先把下界平移量加回流量，核对每个顶点的净流出等于原始 b_i、每条弧满足原始上下界；再按原始费用重算目标值，确认它等于平移后问题的费用加上常数 $\boldsymbol{c}^{\top}\boldsymbol{\ell}$。被删去的固定弧要按其固定流量补回，它们的费用已计入这个常数。

最优性用输出的势逐弧核对条件 (7.3)，核对时使用原始上下界。平移不改变约化费用，也不改变一条弧是否在界上：$y_{i,j}=0$ 当且仅当 $x_{i,j}=\ell_{i,j}$，$y_{i,j}=u_{i,j}-\ell_{i,j}$ 当且仅当 $x_{i,j}=u_{i,j}$。以图 7.2的下界版本为例，平移后问题的最优解 $\boldsymbol{y}=(2,2,0)$ 加回下界得 $\boldsymbol{x}=(2,2,1)$。s 净流出 $2+1=3$，a 净流出 $2-2=0$，t 净收进 $2+1=3$，各弧都在原始界内；原始费用 $2+2+4=8$，等于平移后的费用 4 加常数 4。

再用势 $(4,3,0)$ 核对符号条件。$s\to a$ 严格在界内，$\bar{c}_{s,a}=1-4+3=0$；$a\to t$ 在上界，$\bar{c}_{a,t}=1-3+0=-2\le 0$；$s\to t$ 的流量 1 恰为下界，$\bar{c}_{s,t}=4-4+0=0\ge 0$。与无下界的原例相比，$s\to t$ 从“界内”变成了“在下界”，要求的符号随之改变，所以核对必须按原始上下界逐弧判断。

可行性与最优性分别验收：前者只看流，后者还要看势。浮点结果应分别报告守恒误差、弧界违反量与约化费用符号的误差；在容差内近似满足的条件不能当作精确证书。

7.6 其他改进路线

网络单纯形法不是求最小费用流的唯一方法。各种方法都以第 7.2节的条件为终点：残量网络中没有负费用环，或者等价地，存在一组使全部残量弧约化费用非负的势。它们的差别在于运行中始终保持哪一半条件。一类方法始终保持流可行，逐步消除负费用环；另一类始终保持约化费用非负，逐步满足供需。下面简述三条路线的思路、所需条件与复杂度量级，系统的讲述可参考 Bertsekas 的网络优化教材[3]。本章正文完整展开的仍是网络单纯形法。

[3] D. P. Bertsekas, *Network Optimization: Continuous and Discrete Models*, Athena Scientific, 1998

负环消去 负环消去直接执行定理 7.2。先求一个可行流：加入超级源 s^* 与超级汇 t^*，从 s^* 向每个供给点 i 连容量为 b_i 的弧，从每个需求点 i 向 t^* 连容量为 $-b_i$ 的弧，用第 5章的方法求最大流；最大流把这些新弧全部送满时，原弧上的流量就是可行流，否则问题不可行。

随后反复在费用残量网络中寻找负费用环，沿环推送环上的最小剩余容量 $\theta>0$，费用下降 $\theta\,|c(C)|$。为了不漏掉任何负环，先像第 7.2节那样加入一个向全部顶点连零费用弧的虚拟源点，再用第 3.4节的 Bellman–Ford 算法在 $O(na)$ 时间内判断有无负环。报告负环时，对一条仍可改善的弧再松弛一次，从其终点沿前驱指针回退 n 步，就落在前驱指针形成的一个环上；第 3.4节的路线恢复论证说明，这样的环费用为负。找不到负环时，由定理 7.2，当前流最优。

费用与容量都是整数时，每次推送使费用至少下降 1。记 $C=\max|c_{i,j}|$、$U=\max u_{i,j}$，任一可行流的费用绝对值不超过 aCU，所以消环次数不超过 $2aCU$，总时间为 $O(na^2CU)$。这个界随数值增长，是伪多项式的；任意选环时实际次数也可能很多。网络单纯形法可以看成只在当前树的基本环中找负环：定价只需 $O(a)$，代价是可能出现步长为零的退化换基，而负环消去在残量网络中推送的步长总为正。

逐次最短路 逐次最短路从另一半条件出发。它维护一个满足上下界、但未必满足供需的流 $\boldsymbol{x}$，以及一组使全部残量弧约化费用非负的势 $\boldsymbol{\pi}$；顶点 i 的剩余供给记为 $e_i=b_i-(i\text{ 在 }\boldsymbol{x}\text{ 下的净流出})$，$e_i>0$ 表示还要送出，$e_i<0$ 表示还要收进。初始取 $\boldsymbol{x}=\mathbf{0}$。若全部费用非负，取 $\boldsymbol{\pi}=\mathbf{0}$ 即可；若有负费用弧但没有负费用环，可按第 7.2节的办法用最短距离构造势；一般情形可以先把负费用弧送满，使它们只留下费用为正的反向残量弧，再相应修改 e_i。

每一轮取一个 $e_k>0$ 的顶点 k，在残量网络中以约化费用为长度求从 k 出发的最短距离 d_i。约化费用非负，所以可以用第 3.5节的 Dijkstra 算法。若从 k 到达不了任何 $e_i<0$ 的顶点，问题不可行：从 k 可达的顶点集 R 没有离开它的残量弧，即离开 R 的原弧都已满载、进入 R 的原弧都为零，当

前流已从 R 送出了可能的最大净量。R 中没有剩余需求，各顶点的 e_i 都非负，而 $e_k > 0$，所以任何满足供需的流都要从 R 送出比这更多的净量，不可能做到。

否则取一个可达的 $e_\ell < 0$ 的顶点 ℓ，令 $\pi_i \leftarrow \pi_i - d_i$；不可达顶点的 d_i 取为可达顶点距离中的最大值。对残量弧 $i \to j$，新约化费用为 $\bar{c}_{i,j} + d_i - d_j$：两端都可达时由最短距离的三角不等式非负；$i$ 不可达而 j 可达时，d_i 取最大值，同样非负；两端都不可达时不变；i 可达而 j 不可达的残量弧不存在。于是新势仍使全部约化费用非负，并且 k 到 ℓ 的最短路径上每条弧的约化费用都变为零。

沿这条路径推送 $\theta = \min\{e_k, -e_\ell, \text{路径上的最小剩余容量}\}$。推送产生的反向残量弧，约化费用是路径弧约化费用的相反数，也是零，所以条件继续保持。全部 $e_i = 0$ 时，流可行且全部残量弧约化费用非负，由定理 7.2最优。

整数数据下每轮至少推送一单位，轮数不超过初始剩余供给之和 $B = \sum_{e_i>0} e_i$；用二叉堆实现 Dijkstra 算法时，总时间为 $O(B(n+a)\log n)$，同样随数值增长。按容量缩放、每个阶段只沿剩余容量足够大的路径成批推送的变体，可以把对 B 的线性依赖换成对容量上界的对数依赖。在指派网络上，$B = r$，逐次最短路每轮为一行完成配对，并用势保持约化费用非负；这与 KM 算法逐行增广、用可行标号保持剩余量非负的做法相对应。

费用缩放　第三条路线放宽最优性条件。若全部残量弧的约化费用都不小于 $-\varepsilon$，称流 $\boldsymbol{x}$ 与势 $\boldsymbol{\pi}$ 满足 ε 最优性。费用为整数且 $0 < \varepsilon < 1/n$ 时，ε 最优已经推出最优：任一残量环至多含 n 条弧，环的费用等于约化费用之和，不小于 $-n\varepsilon > -1$，而它又是整数，只能非负，于是没有负费用环。

费用缩放从 $\varepsilon = C$ 开始，每个阶段把 ε 减半，并用沿残量弧的局部推送与调整势恢复 ε 最优性，直到 $\varepsilon < 1/n$。这样的算法可以得到关于 n、a 和 $\log(nC)$ 的多项式时间界，总时间只以对数形式依赖费用范围。第 6.2.4小节的 ε 互补松弛与 ε 缩放正是这一思想在指派问题上的形式：那里的误差界 $r\varepsilon$ 与这里的 $-n\varepsilon$ 下界出自同一个求和论证。

三条路线与网络单纯形法使用同一个残量结构和同一份最优性证书，差别在于维护什么、多久完成。比较时应说明数据是否为整数、复杂度依赖哪些数值，以及界是最坏情形还是经验表现；任何一种方法停止后，都可以用第 7.5节的检验核对结果。

7.7 本章小结

章首的第一个问题是怎样建模。最小费用流给每个顶点一个供需量，给每条弧上下界与单位费用，在全部满足守恒与上下界的流中最小化总费用。

供需之和为零只是可行的必要条件；正下界通过平移化为零，平移带来的常数费用必须在最后恢复。容量有限时，可行流集合闭有界，只要可行就一定取到最小费用。

第二个问题是怎样判断最优。任何两个可行流之差都能拆成残量环的叠加，所以可行流最优当且仅当费用残量网络中没有负费用环。环的个数可能很多，节点势把这一条件改写为逐弧可查的证书：界内弧约化费用为零，下界弧非负，上界弧非正。没有负环时，由虚拟源点出发的最短距离总能给出这样的势。

第三个问题是怎样逐步改进。网络单纯形法把可行流记在树基上，用树弧约化费用为零确定势，以非树弧的约化费用定价，沿基本环推送到某条弧碰界再换基。两阶段法用人工根判定可行性，词典序规则排除退化换基造成的循环；数据为整数时输出整数流，这为把最大流、二部图匹配和指派写成最小费用流提供了依据。每轮开销为 $O(n+a)$，但总轮数没有统一的多项式界。负环消去、逐次最短路与费用缩放用同一份证书组织不同的改进，统一表示也不意味着网络单纯形法在所有输入上都优于专门算法。

回看整个部分，各问题选择的对象、依靠的论证与停止时留下的证书各不相同，表 7.2把它们并列。证书的价值在于检查者不必相信求解过程，只需核对可行性和若干不等式；但证书只在其条件下成立，例如 Auction 算法的价格只给出误差界，整数成本且 $\varepsilon < 1/r$ 时才是精确最优。浮点计算中的核对都要带容差，近似满足不能直接当作精确证书。

本部分选择的始终是路径、树、流、匹配这类具有特定组合结构的对象，算法的正确性也来自这些结构本身的性质。下一部分转而用集合函数评价一般的子集，候选对象可以是顶点、边、特征或传感器。那里将区分两种情形：一种是评价函数的一般性质，如收益递减，给出贪心选择的近似保证；另一种是特殊的表示，如可以写成图割，带来精确的求解。

表 7.2 本部分六个问题的对比（n 为顶点数，m 为无向边数，a 为有向弧数，p 为二部图较小一侧的顶点数，r 为指派方阵的阶数，B 为 Auction 算法的竞价次数）。开销均为本部分所述实现的运算量。

问题	选择对象与目标	方法与成立条件	结果证书	主要开销
最短路	一条路径，弧长之和最小	DAG 递推（无有向环）；Bellman–Ford（允许负边，检测负环）；Dijkstra（可达部分边长非负）	逐弧距离不等式与前驱路线	$O(n+a)$；$O(na)$；$O((n+a)\log n)$
最小生成树	连接全部顶点的边集，成本之和最小	Kruskal、Prim，依靠割性质	无环、覆盖各分量；非树边不轻于树上对应路径的边	$O(m\log n)$；$O(n^2+m)$ 或 $O((n+m)\log n)$
最大流	各弧流量，源汇净输送量最大	残量增广；Edmonds–Karp 按最少弧数选路，Dinic 分层求阻塞流	同值的流与割	$O(na^2)$；$O(n^2a)$
二部最大匹配	互不共享端点的边，边数最多	交替路径增广（匈牙利算法），Hopcroft–Karp 分层成批增广；限于二部图	无增广路径，或单位容量网络中同值的割	$O(n+pm)$；$O(n+m\sqrt{n})$
指派	完美匹配，成本之和最小	KM：可行标号、紧边交替树与调标；Auction：价格与竞价，完整有限成本矩阵	成本等于标号之和；ε 条件与误差界 $r\varepsilon$	$O(r^3)$；$O(r^2+rB)$
最小费用流	各弧流量，供需固定时费用最小	网络单纯形法：可行树基、基本环调整、两阶段初始化、词典序防循环	流量与节点势的逐弧符号条件	每轮 $O(n+a)$，总轮数无统一多项式界

第三部分

集合函数与子集选择

Set Functions and Subset Selection

阅读指引

前一部分选择的是路径、生成树、流和匹配，候选对象都带有特定的图结构。许多选择问题只要求从有限个候选中挑出一组：安装哪些传感器、保留哪些特征、把哪些顶点分到一侧。这时被评价的是整组选择，而组内元素的重叠、替代或配合会改变总分。本部分把候选放进一个有限的底集，研究怎样评价它的子集，以及怎样利用评价的结构完成选择；底集可以是图的顶点或边，也可以与图无关。

第一章建立共同语言。集合函数给每个子集一个值，边际增益记录加入一个元素前后的变化；单调性、模性和次模性分别用边际增益回答三个不同的问题：新增元素会不会降低评价，各元素的贡献是否固定，收益是否随已选集合增大而递减。同一章还处理一个与选择无关的问题：已知每个子集的累计值，怎样由 Möbius 反演恢复各子集自身的贡献。这个问题只用有限求和与包含关系，不需要单调或次模假设。

后两章分别处理两类选择。第二章在“至多选 k 个”或“总成本不超过预算”的约束下最大化目标，研究逐轮加入元素的贪心规则；目标归一化、单调且次模时，基数约束下的贪心有固定比例的近似保证，预算约束下的简单比值规则却没有。第三章转向最小化：目标由一元项和成对项组成、每个成对项满足次模条件时，全部配置的成本可以写成割容量，再用第 5章的最大流最小割精确求解。

阅读时要分清三组东西。第一，函数性质各管一件事，次模不蕴含单调，单调也不蕴含次模。第二，贪心规则本身、它的惰性实现和它的近似保证是三件事：规则在任何集合函数上都能执行，惰性实现与普通贪心选出同样元素要靠次模性，比例保证还要加上归一化、单调和基数约束。第三，最大化得到的是有保证的近似解，成对二值最小化在条件满足时得到精确解；条件不满足或附加了其他约束时，两种方法都不能直接沿用。

第 8 章

集合函数与子集累计

在一片区域里安装传感器时，每个候选位置覆盖一块范围，相邻位置的覆盖范围往往重叠。选定一组位置后，有意义的是这组位置合起来覆盖了多少，而不是各自覆盖面积的简单相加。从一批特征里挑出一组、从一张图里挑出一部分顶点，评价的也都是整组选择。本章要回答两个问题：怎样描述这种对整组的评价，以及评价具有哪些结构可以供后面的选择算法利用；另外，已知每个子集的累计评价时，怎样恢复各个子集自身的贡献。

前两节建立描述子集评价的语言。第 8.1节把“给子集赋值”写成集合函数，用指示向量换一种方式索引子集，并用加入一个元素前后的评价差定义边际增益。第 8.2节用边际增益区分单调性、模性和次模性，并说明它们互不蕴含。

第 8.3节处理另一项任务：由累计值恢复自身贡献。它只用有限求和与包含关系，不依赖前一节的任何性质，也不是后两章选择算法的前提。本章用到卷一的幂集计数、裂项求和、偏序关系、有限和的交换次序与组合数的交错和，用到时在当地说明所需的结论。

8.1 集合函数

给每个元素赋一个数，只需要一张长度为元素个数的表。对整组选择打分就不同了：元素之间的重叠、替代或配合都会改变总分，分数只能直接对子集给出。所以本节以子集为自变量的函数为基本对象。选择又往往是一次加入一个元素，所以还要看加入一个元素前后分数怎样变化。

8.1.1 集合函数的定义

图 8.1是一块由上下两行、左右八列组成的区域，共 16 个格子。有四个候选传感器：a 覆盖第 2 至 6 列的 10 个格子，b 覆盖第 1 至 4 列的 8 个格子，c 覆盖第 5 至 8 列的 8 个格子，d 只覆盖左上角的 1 个格子。

选定一组传感器后，我们关心它们合起来覆盖了多少个不同的格子。记传感器 v 覆盖的格子集合为 C_v，选定的传感器集合为 S，覆盖量就是

$$F(S) = \Big|\bigcup_{v\in S} C_v\Big|.$$

单独看，$F(\{a\}) = 10$，$F(\{b\}) = 8$；两者一起时，第 2 至 4 列被重复覆盖，$F(\{a,b\}) = 12$，而不是 $10+8=18$。整组的价值不能由各成员的价值相加得到，因此评价必须直接以“哪些传感器被选中”为输入。本章和下一章都用这个实例做算例。

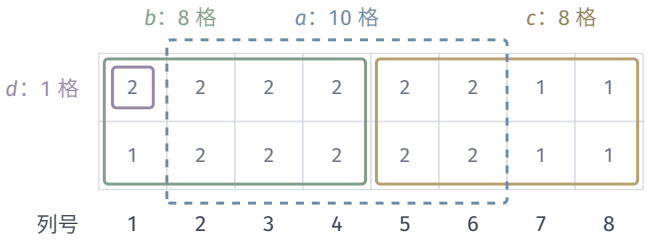


图 8.1　贯穿本章和下一章的传感器覆盖实例。区域由 2×8 个格子组成；绿色框为 b（第 1 至 4 列），黄色框为 c（第 5 至 8 列），蓝色虚线框为 a（第 2 至 6 列），紫色小框为 d（左上角一格）。格内数字是覆盖该格的传感器个数，数字 2 所在的格子都被重复覆盖，因而各传感器覆盖格数之和大于它们合起来覆盖的格数。

覆盖量的输入是一个子集，输出是一个实数。把这种对应一般化，就得到下面的定义。

定义 8.1　集合函数　设 V 是有限集合，称为**底集**（ground set）。把 V 的每个子集对应到一个实数的映射

$$F : \mathcal{P}(V) \to \mathbb{R}$$

称为 V 上的**集合函数**（set function），其中 $\mathcal{P}(V)$ 是 V 的幂集。

定义要求 F 在每个子集上都有值，包括空集。空集表示“什么都不选”，它的值是比较其他选择的起点：覆盖量取 $F(\varnothing)=0$，一个带固定开销的评价则可能取非零值。底集中的元素可以是传感器、特征、顶点或边；集合函数本身并不要求底集带有图结构。

总成本与割权　同一个子集可以按不同目的评价。给四个传感器依次指定安装成本 $4,3,4,1$（对应 a,b,c,d），总成本

$$c(S) = \sum_{v\in S} c_v$$

是一个集合函数，例如 $c(\{a,b\}) = 7$；覆盖量 F 是另一个，$F(\{a,b\}) = 12$。

第三个例子来自图。设 $G=(V,E)$ 是无向图，每条边 $\{i,j\}$ 带非负权 $w_{i,j}$。顶点子集 S 把顶点分成 S 与 $V\setminus S$ 两侧，这就是第 4.2节所说的割；那里要求两侧都非空，这里允许 $S=\varnothing$ 或 $S=V$，这时没有边跨越两侧。两端分居两侧的边的权之和

$$\operatorname{cut}(S)=\sum_{\{i,j\}\in E:\ |\{i,j\}\cap S|=1} w_{i,j}$$

称为 S 的割权，它以顶点子集为自变量。

三个函数分别累计单项成本、合并覆盖范围、统计跨越两侧的边，元素之间的相互影响各不相同。下一节据此区分它们的性质。

取值表与求值规则 n 元底集的每个元素有选与不选两种可能，按乘法规则共有 2^n 个子集，所以一个集合函数的完整取值表有 2^n 项（幂集计数可参考卷一第 2 章“计数”）。传感器实例只有 16 项。除上面的单元素值外，二元子集的覆盖量为 $F(\{a,b\})=12$、$F(\{a,c\})=14$、$F(\{a,d\})=11$、$F(\{b,c\})=16$、$F(\{b,d\})=8$、$F(\{c,d\})=9$，三元子集中 $\{a,b,c\}$ 与 $\{b,c,d\}$ 覆盖全部 16 格；后面各节的枚举基准都从这些值读出。底集有 50 个元素时，取值表已超过 10^{15} 项，无法存储。

因此实际问题中的集合函数通常由一条规则给出：给定 S，按规则算出 $F(S)$，这样的一次计算称为一次求值。覆盖量的规则是维护一个格子标记数组，初始全为未覆盖，依次标记 S 中各传感器覆盖的格子，最后计数。求 $F(\{a,b,c\})$ 时，读入 b 标记第 1 至 4 列，读入 a 新增第 5、6 列，读入 c 新增第 7、8 列，共 16 格，工作量与 $\sum_{v\in S}|C_v|$ 成正比。割权的规则是扫描全部边，工作量与边数成正比。

两种给出方式的代价不同。查表读值便宜，但表随 n 指数增长；规则不必存表，却每次读值都要计算。所以后面分析算法时，我们把求值次数和每次求值的代价分开计算，不把求值视为免费。

8.1.2 子集与二值向量

给底集的元素固定编号 $V=\{1,\dots,n\}$。前面计算子集个数时用到，子集 S 与长度为 n 的零一记录一一对应。把这条记录写成向量 $\boldsymbol{x}=(x_1,\dots,x_n)\in\{0,1\}^n$，其中

$$x_i=\mathbb{I}\{i\in S\},$$

称 $\boldsymbol{x}$ 为 S 的**指示向量**（indicator vector）。这里 $\mathbb{I}\{\cdot\}$ 是指示函数，条件成立时取 1，否则取 0。反过来，收集取值为 1 的坐标就恢复出子集：$S=\{i:x_i=1\}$。

集合运算在指示向量上逐坐标进行。设 S,T 的指示向量为 $\boldsymbol{x},\boldsymbol{y}$，则 $|S|=\sum_i x_i$；并集的第 i 个坐标是 $\max\{x_i,y_i\}$，交集的第 i 个坐标是 $\min\{x_i,y_i\}=$

$x_i y_i$，补集的第 i 个坐标是 $1 - x_i$；包含关系 $S \subseteq T$ 对应逐坐标的 $x_i \le y_i$。每种运算都只需扫描一遍 n 个坐标。

于是集合函数也可以写成二值向量的函数：令 $f(\boldsymbol{x}) = F(\{i : x_i = 1\})$，$f$ 与 F 只是同一组数值的两种索引方式。图 8.2以三元底集为例画出这一对应：八个子集恰好是立方体的八个顶点，立方体的每条边连接只差一个元素的两个子集，也就是只差一个坐标的两个向量。

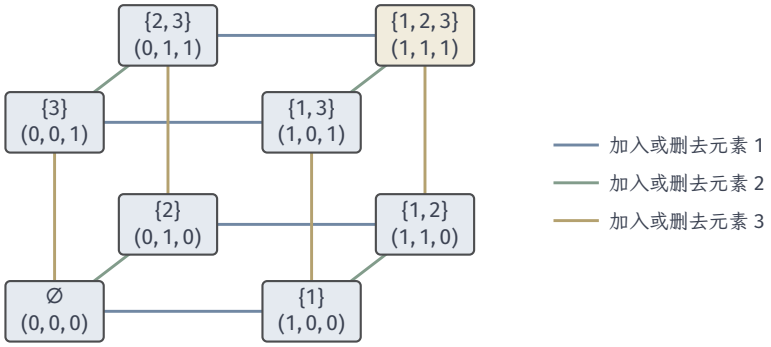


图 8.2　三元底集 $V = \{1, 2, 3\}$ 的八个子集与指示向量 (x_1, x_2, x_3) 排成立方体的顶点。每个框上行是子集，下行是对应的向量；边的颜色表示沿这条边加入或删去哪个元素，也就是改变哪个坐标。沿任一条边向上走一步，就是在当前子集中加入一个元素。

这一改写只换了表示方式，并没有带来额外结构。写成 $f(\boldsymbol{x})$ 之后，f 不一定是各坐标的函数之和，也不一定能分解成只读取少数坐标的局部项；在 $\{0, 1\}^n$ 上谈连续性或求导也没有意义。第 10章和第 4部分讨论的，正是函数确实具有某种局部分解时可以怎样利用它。

8.1.3　边际增益

选择子集时，最直接的问题是：在已经选好的集合上再加一个元素，评价会改变多少。

定义 8.2　边际增益　设 F 是 V 上的集合函数，$S \subseteq V$，$v \in V \setminus S$。称

$$\Delta(v \mid S) = F(S \cup \{v\}) - F(S)$$

为元素 v 相对于集合 S 的**边际增益**（marginal gain）。

边际增益同时依赖新元素和已选集合。图 8.3比较同一个传感器 c 在两种情况下的增益。从空集加入时，c 的 8 个格子都是新覆盖的，$\Delta(c \mid \varnothing) = 8$；已经选了 a 之后，c 的第 5、6 列已被 a 覆盖，只有第 7、8 列的 4 个格子是新的，$\Delta(c \mid \{a\}) = 4$。所以谈论一个元素“值多少”时，必须说明是相对于哪个已选集合。



图 8.3　同一传感器 c（黄色框）的边际增益随已选集合变化。(a) 已选集合为空，c 覆盖的 8 个格子（绿色，标“+”）全部是新覆盖；(b) 已选 $\{a\}$（蓝色底与蓝色虚线框），c 与 a 重叠的第 5、6 列（标“重”）不再计数，只有第 7、8 列的 4 个格子是新覆盖。

逐个加入的累计　边际增益只描述一步。两个有包含关系的集合相差若干元素时，把多出的元素逐个加入，函数的总变化就是一串边际增益之和。设 $A \subseteq B \subseteq V$，把 $B \setminus A$ 中的元素按任意次序排成 $v_1, \dots, v_r$，记 $A_0 = A$，$A_k = A \cup \{v_1, \dots, v_k\}$。于是 $A_r = B$，并且

$$F(B) - F(A) = \sum_{k=1}^{r} \big(F(A_k) - F(A_{k-1})\big) = \sum_{k=1}^{r} \Delta(v_k \mid A_{k-1}). \tag{8.1}$$

第一个等号是裂项求和：中间的 $F(A_1), \dots, F(A_{r-1})$ 各出现一次正号、一次负号而抵消，只剩 $F(A_r) - F(A_0)$（有限和的裂项可参考卷一第 1 章“集合、关系与映射”）。第二个等号成立，是因为 $A_k = A_{k-1} \cup \{v_k\}$ 且 $v_k \notin A_{k-1}$，每一项正是一个有定义的边际增益。下一节判别性质、第 9.1节分析贪心算法，用的都是这个式子。

在传感器实例中取 $A = \varnothing$、$B = \{a, b, c\}$。按 a, c, b 的次序加入：a 覆盖 10 格，c 补上第 7、8 列的 4 格，b 只补上第 1 列的 2 格，增益依次是 $10, 4, 2$。按 b, c, a 的次序加入：b 覆盖 8 格，c 与 b 不重叠，补上 8 格，此时 16 格已全部覆盖，a 的增益为 0，增益依次是 $8, 8, 0$。两种次序的总和都是 $F(\{a, b, c\}) = 16$：总变化由两端的集合决定，单步增益则取决于走哪条路线。

单步增益的大小也不能说明一个元素是否值得选。在空集上，a 的增益 10 是四个传感器中最大的，但只能选两个传感器时，最好的组合是覆盖全部 16 格的 $\{b, c\}$，并不包含 a。第 9.1节会具体比较这两者。

8.2 集合函数的性质

上一节的三个函数在加入一个元素时表现不同：加入传感器不会减少覆盖，但新增的覆盖可能越来越少；加入一项成本总是增加固定的数额；把一个顶点移到 S 一侧，割权可能增加也可能减少。这三种表现分别对应单调性、模性和次模性。

三者都可以用边际增益判别，但各自回答一个不同的问题：新增元素会不会降低评价，每个元素的贡献是否固定，收益是否随已选集合增大而递减。所以一个函数可以具备其中任意几项而缺少其他几项，本节最后用反例和对照表核对这一点。

8.2.1 单调性

多装一个传感器，覆盖的格子只会增加或保持不变。把这一性质写成子集之间的比较，就是下面的定义。

> **定义 8.3　单调非减**　若对任意 $A \subseteq B \subseteq V$ 都有 $F(A) \le F(B)$，称集合函数 F **单调非减**（monotone nondecreasing），简称单调。

定义比较的是任意两个有包含关系的子集，直接检查要考虑很多对。借助式 (8.1)，可以化成只检查单个元素的条件：F 单调，当且仅当对每个 $S \subseteq V$ 和每个 $v \notin S$ 都有 $\Delta(v \mid S) \ge 0$。

必要性只需取 $A = S$、$B = S \cup \{v\}$。充分性：对任意 $A \subseteq B$，式 (8.1)把 $F(B) - F(A)$ 写成若干个边际增益之和，每一项都非负，所以 $F(A) \le F(B)$。

覆盖量是单调的，因为加入 v 后新增的格子数 $\Delta(v \mid S) = |C_v \setminus \bigcup_{u \in S} C_u|$ 不会是负数。

但目标中含有覆盖量，并不保证整个目标单调。把覆盖量减去安装成本，得到净收益 $G(S) = F(S) - c(S)$。$G(\{b\}) = 8 - 3 = 5$，而 $G(\{b, d\}) = 8 - 4 = 4$：d 覆盖的那一格已被 b 覆盖，加入 d 不增加覆盖，却多花 1 个单位成本，所以净收益下降。

非负性与归一化　函数值全为非负，也不能推出单调。在二元底集 $\{1, 2\}$ 上取

$$F(\varnothing) = 1, \quad F(\{1\}) = 3, \quad F(\{2\}) = 2, \quad F(\{1, 2\}) = 2,$$

四个值都是正数，但 $\Delta(2 \mid \{1\}) = 2 - 3 = -1 < 0$，所以它不单调。

反过来，单调也不能推出 $F(\varnothing) = 0$：常数函数 $F \equiv 5$ 是单调的。$F(\varnothing) = 0$ 称为归一化，它是一项约定，不是函数的固有性质：对任意集合函数，减去常数 $F(\varnothing)$ 就得到归一化的函数，而所有边际增益保持不变。归一化且单调的函数必然处处非负，因为 $F(S) \ge F(\varnothing) = 0$。

8.2.2 模性

总成本 $c(S)$ 具有最简单的结构：每个元素贡献一个固定的数，彼此互不影响。一般地，取常数 c_0 和每个元素的权 w_v，令

$$F(S) = c_0 + \sum_{v \in S} w_v. \tag{8.2}$$

这时对任意 S 和 $v \notin S$，$\Delta(v \mid S) = w_v$，增益与已选集合无关。由上一小节的判别条件，这样的 F 单调当且仅当每个 $w_v \geq 0$；常数 c_0 不影响任何边际增益。

给定一个集合函数，要判断它是否具有这种固定贡献的形式，可以逐个检查边际增益是否与 S 无关；更紧凑的办法是检查一个只涉及四个函数值的等式。

对形如式 (8.2)的函数和任意两个子集 A, B，比较 $F(A) + F(B)$ 与 $F(A \cup B) + F(A \cap B)$。两边都含两个常数 c_0，只需逐个元素比较权重被计了几次。对一个元素 v：若 v 同时属于 A 和 B，则 w_v 在左边计两次，在右边的并集与交集中也各计一次；若 v 只属于其中一个，则 w_v 在两边都只计一次；若 v 都不属于，则两边都不计。所以

$$F(A) + F(B) = F(A \cup B) + F(A \cap B). \tag{8.3}$$

对所有 $A, B \subseteq V$ 满足式 (8.3)的集合函数称为**模函数**（modular function）。

反过来，模等式也足以恢复固定权重的表示，所以可以用它判断元素之间有没有交互。设 F 是模函数，取 $S \neq \varnothing$ 与 $v \in S$，在式 (8.3)中令 $A = S \setminus \{v\}$、$B = \{v\}$，这时 $A \cup B = S$，$A \cap B = \varnothing$，得到

$$F(S) = F(S \setminus \{v\}) + \big(F(\{v\}) - F(\varnothing)\big).$$

对 $|S|$ 归纳，就得到 $F(S) = F(\varnothing) + \sum_{v \in S}\big(F(\{v\}) - F(\varnothing)\big)$，即式 (8.2)取 $c_0 = F(\varnothing)$、$w_v = F(\{v\}) - F(\varnothing)$。所以模函数恰好就是固定贡献形式的函数，权重可以直接从空集和单元素集的值读出。

覆盖量不是模函数。取 $A = \{a\}$、$B = \{b\}$，$F(\{a\}) + F(\{b\}) = 18$，而 $F(\{a, b\}) + F(\varnothing) = 12$，模等式不成立；差值 6 正是第 2 至 4 列被重复计数的格子。总成本则满足模等式，例如 $c(\{a\}) + c(\{b\}) = 7 = c(\{a, b\}) + c(\varnothing)$。

8.2.3 次模性

次模性说的是收益递减：已经拥有的越多，再添一样东西带来的好处越少。覆盖量正是这样：已覆盖的格子越多，新传感器能补上的格子越少。图 8.3中，$\Delta(c \mid \varnothing) = 8$，$\Delta(c \mid \{a\}) = 4$；再加入 b 之后，$\Delta(c \mid \{a, b\}) = 4$，

没有继续变小，但也没有变大。把“不会变大”作为要求，就得到边际收益递减的条件。

定义 8.4　次模函数　若对任意 $A \subseteq B \subseteq V$ 和任意 $v \in V \setminus B$ 都有

$$\Delta(v \mid A) \geq \Delta(v \mid B), \tag{8.4}$$

称集合函数 F 是**次模**函数（submodular function）。

条件中的 v 不属于 B，从而也不属于 A，两个增益都有定义。

式 (8.4)要对每一对嵌套集合和每个外部元素检查。次模性还有一种不出现边际增益的写法，它和模等式形式相同，只是把等号换成不等号。

定理 8.5　次模性的两种刻画　设 V 是有限集合，F 是 V 上的集合函数。F 满足式 (8.4)，当且仅当对任意 $X, Y \subseteq V$ 都有

$$F(X) + F(Y) \geq F(X \cup Y) + F(X \cap Y). \tag{8.5}$$

证明　先设式 (8.5)成立。给定 $A \subseteq B$ 和 $v \notin B$，取 $X = A \cup \{v\}$、$Y = B$。由 $A \subseteq B$ 得 $X \cup Y = B \cup \{v\}$；由 $v \notin B$ 得 $X \cap Y = A$。代入式 (8.5)，

$$F(A \cup \{v\}) + F(B) \geq F(B \cup \{v\}) + F(A),$$

移项即 $\Delta(v \mid A) \geq \Delta(v \mid B)$。

再设式 (8.4)成立。给定 X, Y，记 $C = X \cap Y$，把 $Y \setminus X$ 中的元素按任意次序排成 $y_1, \dots, y_r$。分别从 X 和从 C 出发逐个加入这些元素，由式 (8.1)，

$$F(X \cup Y) - F(X) = \sum_{k=1}^{r} \Delta\big(y_k \mid X \cup \{y_1, \dots, y_{k-1}\}\big),$$

$$F(Y) - F(C) = \sum_{k=1}^{r} \Delta\big(y_k \mid C \cup \{y_1, \dots, y_{k-1}\}\big).$$

这里用到 $X \cup (Y \setminus X) = X \cup Y$ 与 $C \cup (Y \setminus X) = Y$。对每个 k，$C \cup \{y_1, \dots, y_{k-1}\} \subseteq X \cup \{y_1, \dots, y_{k-1}\}$，且 y_k 不属于后者（$y_k \notin X$，并且与前面的 y_j 不同）。由式 (8.4)，第二个和的每一项不小于第一个和的对应项，所以 $F(X \cup Y) - F(X) \leq F(Y) - F(C)$，移项即得式 (8.5)。 ■

两种刻画用途不同：式 (8.4)直接对应“收益递减”的直观，适合分析逐个加入元素的算法；式 (8.5)只涉及四个函数值，适合验证具体函数。

两种刻画也给出构造次模函数的办法。模函数在式 (8.5)中处处取等号，所以是次模的。若干个函数各自满足式 (8.4)时，把这些不等式逐项相加，可知它们的和也满足；所以次模函数之和仍是次模函数，次模函数加上模函数也仍是次模函数。

覆盖函数 覆盖量是次模的。设 $A \subseteq B$，$v \notin B$。v 相对于 S 的增益是 C_v 中尚未被 S 覆盖的格子数，即 $\Delta(v \mid S) = |C_v \setminus U(S)|$，其中 $U(S) = \bigcup_{u \in S} C_u$。由 $A \subseteq B$ 得 $U(A) \subseteq U(B)$，于是 $C_v \setminus U(B) \subseteq C_v \setminus U(A)$，取元素个数就得到式 (8.4)。这个论证不依赖格子的具体排列，只用到“增益是 C_v 中尚未被覆盖的部分”。所以任何“覆盖了多少个不同对象”的函数都是次模的，这样的函数称为**覆盖函数**（coverage function）。

覆盖函数还让四值不等式有一个逐格的读法。$F(X) + F(Y)$ 对每个格子数两次：X 覆盖它就计 1，Y 覆盖它再计 1；$F(X \cup Y) + F(X \cap Y)$ 则按 $X \cup Y$ 与 $X \cap Y$ 是否覆盖它来数。

固定一个格子，记 $p, q, r \in \{0, 1\}$ 分别表示 X、Y、$X \cap Y$ 是否覆盖它。$X \cup Y$ 覆盖它当且仅当 p, q 中至少一个为 1，所以左边计 $p + q$，右边计 $\max\{p, q\} + r$，两者之差是 $\min\{p, q\} - r$。$X \cap Y$ 中的传感器同时属于 X 和 Y，它覆盖的格子 X 与 Y 都覆盖，所以 $r \le \min\{p, q\}$，每个格子的差都不小于 0。差为 1 的格子恰好是 X 与 Y 都覆盖、而公共部分 $X \cap Y$ 不覆盖的格子：两组各用自己的传感器覆盖了同一格。

图 8.4取 $X = \{a, b\}$、$Y = \{b, c\}$。第 1 至 4 列由公共的 b 覆盖，两边都计 2；第 7、8 列只有 Y 覆盖，两边都计 1；第 5、6 列由 X 中的 a 和 Y 中的 c 分别覆盖，b 不覆盖它们，左边计 2 而右边计 1。于是 $F(X) + F(Y) = 12 + 16 = 28$，$F(X \cup Y) + F(X \cap Y) = 16 + 8 = 24$，差 4 正是这 4 个格子。

换两组集合对照。若改取 $Y = \{a, c\}$，X 与 Y 都覆盖的第 2 至 6 列全被公共的 a 覆盖，两边相等：$12 + 14 = 16 + 10$。取 $X = \{a\}$、$Y = \{b\}$ 时公共部分为空，差 $18 - 12 = 6$ 就是上一小节第 2 至 4 列的重叠格数。

概括地说，两组分别评价时重复计入的部分，不少于公共部分自身就有的部分。一般的次模函数没有格子可数，但定理 8.5的证明给出同样的含义：把 $Y \setminus X$ 中的元素加到较小的 $X \cap Y$ 上，收益不少于把它们加到较大的 X 上。

割权的次模性 割权按边累加，所以先看一条权为 w 的边 $\{i, j\}$ 单独贡献的函数：它在 S 恰含 i, j 之一时取 w，否则取 0。对不是端点的元素，这条边的贡献不随它的加入而改变，增益为 0。加入端点 i 时，若 j 已在集合中，这条边从跨越两侧变为两端同侧，增益为 $-w$；若 j 不在集合中，这条边从两端同侧变为跨越两侧，增益为 $+w$。

	1	2	3	4	5	6	7	8	
(a) 分别评价	2	2	2	2	2	2	1	1	$F(X)+F(Y)=28$
	2	2	2	2	2	2	1	1	
(b) 并与交	2	2	2	2	1	1	1	1	$F(X\cup Y)+F(X\cap Y)=24$
	2	2	2	2	1	1	1	1	
列号	1	2	3	4	5	6	7	8	

图 8.4　覆盖函数上四值不等式的逐格计数，取 $X=\{a,b\}$、$Y=\{b,c\}$。(a) 每格的数字是 X 与 Y 中覆盖它的组数，合计 $F(X)+F(Y)$；(b) 每格的数字是 $X\cup Y$ 与 $X\cap Y=\{b\}$ 中覆盖它的组数，合计 $F(X\cup Y)+F(X\cap Y)$。蓝底的第 1 至 4 列由公共部分覆盖，两边相同；黄底的第 5、6 列被 X 的 a 与 Y 的 c 分别覆盖而不被 b 覆盖，左边多计一次，四格合计正是差值 4。

现在取 $A\subseteq B$、$i\notin B$，按 j 的归属分三种情形比较两个增益，见图 8.5。若 $j\in A$，则 j 也在 B 中，两个增益都是 $-w$；若 $j\in B\setminus A$，则 $\Delta(i\mid A)=+w$ 而 $\Delta(i\mid B)=-w$；若 $j\notin B$，两个增益都是 $+w$。三种情形都满足 $\Delta(i\mid A)\geq\Delta(i\mid B)$；待加入的元素不是这条边的端点时，两个增益都是 0，不等式也成立。所以单条边的贡献是次模的。

由次模函数之和仍是次模函数，cut 作为各边贡献之和也是次模的。边权非负在这里是必要的：若某条边的权为负，第二种情形的不等号就反向了。

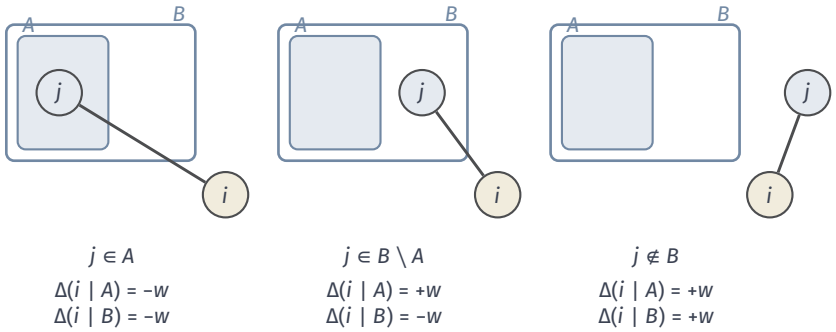


图 8.5　割权函数次模性的逐边验证。固定一条权为 $w\geq 0$ 的边 $\{i,j\}$，已选集合 $A\subseteq B$（内框为 A，外框为 B），待加入的端点 i（黄色）不在 B 中。按另一端点 j 的位置分三种情形：$j\in A$ 时两个增益都是 $-w$；$j\in B\setminus A$ 时 A 上增益为 $+w$、B 上增益为 $-w$；$j\notin B$ 时两个增益都是 $+w$。每种情形都有 $\Delta(i\mid A)\geq\Delta(i\mid B)$。

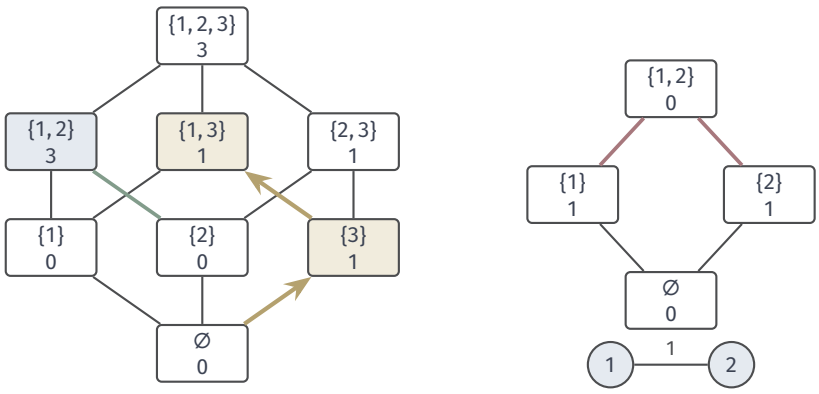
8.2.4　性质之间的关系

次模与单调互不蕴含。割权常常随集合增大而下降：$\mathrm{cut}(\varnothing)=\mathrm{cut}(V)=0$，而中间的子集可以有正的割权。图 8.6(b) 是只有一条权 1 的边的情形，割权依次为 $0,1,1,0$，从单元素集到 $\{1,2\}$ 时下降，所以次模函数不必单调。

反方向，图 8.6(a) 的函数

$$F(S) = \max\{3\,\mathbb{I}\{\{1,2\} \subseteq S\},\ \mathbb{I}\{3 \in S\}\}$$

是单调的，但 $\Delta(2 \mid \varnothing) = 0 < 3 = \Delta(2 \mid \{1\})$，违反式 (8.4)，所以单调函数也不必次模。



(a) 单调但非次模　　(b) 次模但非单调

图 8.6　单调性与次模性互不蕴含。每个框上行是子集，下行是函数值，连线连接只差一个元素的子集。(a) $F(S) = \max\{3\,\mathbb{I}\{\{1,2\} \subseteq S\}, \mathbb{I}\{3 \in S\}\}$：沿任一连线向上函数值不减，但绿色连线 $\{2\} \to \{1,2\}$ 的增益 3 大于从空集加入 2 的增益 0；黄色箭头是第 9.1节贪心选择的路径，蓝色框是两元素时的最优集合。(b) 单条权 1 的边上的割权：次模，但红色连线上函数值从 1 降到 0。

超模函数　把式 (8.4)的不等号反过来，即要求 $\Delta(v \mid A) \le \Delta(v \mid B)$，得到的函数称为**超模**函数（supermodular function）；等价地，F 超模当且仅当 $-F$ 次模。

超模描述的是协同：两个部件只有同时具备才能工作时，单独加入任何一个的增益都是 0，在另一个已经存在时加入，增益才是正的。例如 $F(S) = \mathbb{I}\{\{1,2\} \subseteq S\}$ 是超模的。

图 8.6(a) 的函数既不是次模也不是超模：元素 1, 2 之间是协同的，而元素 3 与 $\{1,2\}$ 之间是替代的，$\Delta(3 \mid \varnothing) = 1 > 0 = \Delta(3 \mid \{1,2\})$。模函数的增益恒为 w_v，与已选集合无关，同时满足两个方向的不等式，所以它既是次模的也是超模的。

但同时次模和超模并不带来单调：由第 8.2.2小节，模函数是否单调只取决于权重的符号。例如二元底集上取 $c_0 = 0$、$w_1 = 1$、$w_2 = -1$，则 $F(\{1,2\}) = 0 < 1 = F(\{1\})$，这个模函数既次模又超模，却不单调。

性质对照　表 8.1汇总本节的函数（割权取图 8.6(b) 的单边图）。

表中尚未直接核对的几项如下：

- 覆盖量：$\Delta(c \mid \varnothing) = 8 > 4 = \Delta(c \mid \{a\})$，增益随已选集合变小，所以不是超模的。
- 单边割权：$\Delta(1 \mid \varnothing) = 1 > -1 = \Delta(1 \mid \{2\})$，所以不是超模的；$\mathrm{cut}(\{1\}) + \mathrm{cut}(\{2\}) = 2 \neq 0 = \mathrm{cut}(\{1,2\}) + \mathrm{cut}(\varnothing)$，所以不是模函数。
- 净收益：它是次模函数 F 加上模函数 $-c$，仍为次模。若它是模函数或超模函数，再加回模函数 c，F 也会是模函数或超模函数，与前两行矛盾；所以它同 F 一样既不是模函数也不是超模的。
- $\mathbb{I}\{\{1,2\} \subseteq S\}$：加入一个元素的增益只在另一元素已选时为 1，随已选集合只增不减，所以是超模的；$\Delta(2 \mid \varnothing) = 0 < 1 = \Delta(2 \mid \{1\})$ 说明它不是次模的。

四种性质可以各种方式组合出现。所以后两章的算法保证都同时写明目标方向、函数是否单调、是否次模以及约束；只凭“次模”二字，不能决定采用哪种方法。

表 8.1　本节各函数的性质。“是”“否”均已在正文中由定义、判别式或反例核对；模函数同时满足次模与超模的不等式。

函数	单调	模	次模	超模
总成本 c	是	是	是	是
覆盖量 F	是	否	是	否
割权 cut	否	否	是	否
净收益 $F - c$	否	否	是	否
$\mathbb{I}\{\{1,2\} \subseteq S\}$	是	否	否	是
图 8.6(a) 的函数	是	否	否	否

8.3　子集累计与 Möbius 反演

设想每个课程组合有一个总得分：单门课各有自己的贡献，某两门课同时选又会产生额外的配合效果，配合也可能是负的。能观察到的是每个组合的总得分，想知道的却是每个组合**自身**贡献了多少。

卷一计数中的容斥原理由各交集的大小恢复并集的大小，这里做的是同一类“由累计量恢复局部量”的事，只是对象换成了一般的数值。本节只用有限求和与包含关系，不依赖上一节的单调或次模假设，也不是后面选择算法的前提。

8.3.1　子集格与累计

累计沿包含关系进行，所以先把包含关系画出来。包含关系是幂集上的偏序：每个集合包含自身；$S \subseteq T$ 与 $T \subseteq S$ 同时成立时 $S = T$；包含关系可以传递。有限集合 V 的全部子集连同包含关系称为 V 的**子集格**（subset lattice）：两个子集的交是它们最大的公共子集，并是包含两者的最小集合。

图 8.7按元素个数自下而上分层画出 $V_0 = \{1,2,3,4\}$ 的子集格，每条连线连接只差一个元素的两个子集，沿连线向上一步就是加入一个元素。图 8.2中的立方体正是三元底集的子集格。

从 S 出发沿连线一路向下能到达的子集，恰好是 S 的全部子集，例如从 $\{1,3\}$ 向下只能到达 $\{1\}$、$\{3\}$ 和空集。累计值就沿这些向下的路径收集贡献。

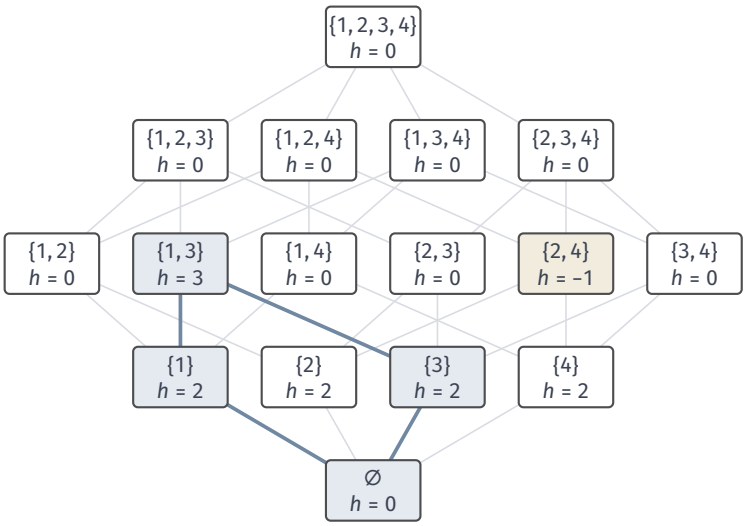


图 8.7　四元素集合 $V_0 = \{1,2,3,4\}$ 的子集格与各子集自身的贡献 h。子集按元素个数自下而上分层，连线连接只差一个元素的两个子集；框内上行为子集，下行为教学设定的贡献。蓝色标出 $\{1,3\}$ 及其全部子集，它们的贡献相加得到累计值 $g(\{1,3\}) = 0+2+2+3 = 7$；黄色框 $\{2,4\}$ 的自身贡献为负。

自身贡献与累计值　给每个子集 T 指定一个实数 $h(T)$，表示 T 作为整体自身带来的贡献：单元素集的贡献是单门课的得分，二元集的贡献是两门课在各自得分之外的配合效果，依此类推。S 的累计值收集它的全部子集的贡献：

$$g(S) = \sum_{T \subseteq S} h(T). \tag{8.6}$$

g 与 h 都是 V 上的集合函数。累计不只相加单元素的贡献，S 内部更大子集的配合贡献也计入其中。

图 8.7的教学设定让四个单项各贡献 2，$\{1,3\}$ 另有配合贡献 3，$\{2,4\}$ 另有配合贡献 -1，其余子集（包括空集和全部三元、四元子集）的贡献为 0。于是

$$g(\{1,3\}) = h(\varnothing) + h(\{1\}) + h(\{3\}) + h(\{1,3\}) = 0+2+2+3 = 7,$$

而两个单项的累计值之和只有 $g(\{1\}) + g(\{3\}) = 4$，多出的 3 正是 $\{1,3\}$ 自身的贡献。

反演处理相反方向的问题：已知全部 2^n 个累计值 $g(S)$，求全部 2^n 个自身贡献 $h(S)$，使它们按式 (8.6)累计后恰好给出已知的 g。需要回答的是这样的 h 是否存在、是否唯一，以及怎样算出来。

8.3.2　Möbius 反演

先看一列数的情形。若 $g_k = h_1 + \cdots + h_k$ 是前缀和，恢复 h_k 只需相邻两项相减：$h_k = g_k - g_{k-1}$。子集格上的累计也是一种“前缀和”，只是 S 的下方不止一个紧邻的子集，而有 $|S|$ 个 $S \setminus \{i\}$，每个元素给出一个相减的方向。只沿一个方向相减还不够，各个方向的差会把某些贡献重复减去，需要再加回来。

两元素的情形可以完整写出。$g(\{1,3\})$ 包含 $h(\varnothing)$、$h(\{1\})$、$h(\{3\})$ 和 $h(\{1,3\})$ 四项。为了只留下 $h(\{1,3\})$，先减去 $g(\{1\}) = h(\varnothing)+h(\{1\})$ 和 $g(\{3\}) = h(\varnothing)+h(\{3\})$，两个单项贡献被去掉，但 $h(\varnothing)$ 被多减了一次，再加回 $g(\varnothing) = h(\varnothing)$，剩下的恰是

$$h(\{1,3\}) = g(\{1,3\}) - g(\{1\}) - g(\{3\}) + g(\varnothing).$$

表 8.2逐项记录四个累计值各含哪些贡献、带什么符号：按列相加，$h(\varnothing)$、$h(\{1\})$、$h(\{3\})$ 的系数都抵消为 0，只有 $h(\{1,3\})$ 留下系数 1。

同一个式子也是两次相邻差：

$$h(\{1,3\}) = \big(g(\{1,3\}) - g(\{3\})\big) - \big(g(\{1\}) - g(\varnothing)\big),$$

两个括号各沿“加入元素 1”的方向相减，括号之间再沿“加入元素 3”的方向相减。展开后，每个 $g(T)$ 的符号由 T 比 S 少几个元素决定，少一个变一次号。一般情形的公式正是这样，称为Möbius 反演（Möbius inversion）。

表 8.2　$h(\{1,3\}) = g(\{1,3\}) - g(\{1\}) - g(\{3\}) + g(\varnothing)$ 中各项的抵消。每行把一个累计值按式 (8.6)展开，记下各贡献的系数（已乘上该累计值前的符号），空白为 0；末行按列相加。

	$h(\varnothing)$	$h(\{1\})$	$h(\{3\})$	$h(\{1,3\})$
$+g(\{1,3\})$	$+1$	$+1$	$+1$	$+1$
$-g(\{1\})$	-1	-1		
$-g(\{3\})$	-1		-1	
$+g(\varnothing)$	$+1$			
合计	0	0	0	1

定理 8.6　子集格上的 Möbius 反演　设 V 为有限集合，$g, h : \mathcal{P}(V) \to \mathbb{R}$。若对每个 $S \subseteq V$ 式 (8.6)成立，则对每个 $S \subseteq V$ 有

$$h(S) = \sum_{T \subseteq S} (-1)^{|S|-|T|} g(T). \tag{8.7}$$

反过来，若对每个 S 式 (8.7)成立，则式 (8.6)也对每个 S 成立。

证明　把式 (8.6)代入式 (8.7)右侧，得到对全部满足 $R \subseteq T \subseteq S$ 的集合对 (R, T) 的双重和。先固定 T 再取 R，与先固定 R 再取 T，遍历的是同一批有限个集合对；有限和只是把这些项全部相加，与分组次序无关，所以可以交换求和次序：

$$\sum_{T \subseteq S} (-1)^{|S|-|T|} \sum_{R \subseteq T} h(R) = \sum_{R \subseteq S} h(R) \sum_{T \colon R \subseteq T \subseteq S} (-1)^{|S|-|T|}.$$

固定 R 后，满足 $R \subseteq T \subseteq S$ 的 T 由 $S \setminus R$ 中哪些元素被加入决定，即 $T = R \cup D$，$D \subseteq S \setminus R$ 任意，且 $|S| - |T| = |S \setminus R| - |D|$。记 $q = |S \setminus R|$，按 $|D| = j$ 分组，大小为 j 的 D 有 $\binom{q}{j}$ 个，所以内层和等于

$$\sum_{j=0}^{q} \binom{q}{j} (-1)^{q-j} = (-1)^q \sum_{j=0}^{q} (-1)^j \binom{q}{j} = \begin{cases} 1, & q = 0, \\ 0, & q \geq 1, \end{cases}$$

最后一步用组合数的交错和：$q \geq 1$ 时，q 元集合中偶数大小的子集与奇数大小的子集一样多（固定一个元素，加入或删去它就把两类一一配对），所以 $\sum_{j=0}^{q} (-1)^j \binom{q}{j} = 0$。

内层和只在 $R = S$ 时为 1，在 R 是 S 的真子集时为 0，所以真子集上的贡献全部抵消，右侧只剩 $h(S)$。表 8.2的各列合计正是 $S = \{1, 3\}$ 时的这些内层和。

反方向同样直接核对。由式 (8.7)给出 h，代入 $\sum_{T \subseteq S} h(T)$ 并交换求和次序，

$$\sum_{T \subseteq S} h(T) = \sum_{R \subseteq S} g(R) \sum_{T \colon R \subseteq T \subseteq S} (-1)^{|T|-|R|}.$$

仍写 $T = R \cup D$，内层和等于 $\sum_{j=0}^{q} (-1)^j \binom{q}{j}$，它在 $R = S$ 时只有 $D = \varnothing$ 一项而等于 1，在 $R \neq S$ 时为 0。所以右侧等于 $g(S)$。 ∎

定理回答了存在与唯一的问题。对任意给定的 g，按式 (8.7)定义的 h 累计后恰好回到 g，所以所求的 h 存在；而任何满足式 (8.6)的 h 都必须等于式 (8.7)的右侧，所以它是唯一的。

式 (8.6)与式 (8.7)是一对互逆的变换：从贡献到累计，再从累计回到贡献，不丢失任何信息。

证明中起作用的是组合数的交错和为零，这与容斥原理相同；区别在于容斥使并集中每个元素恰计一次，反演只保留指定子集自身的贡献。

四元素算例　仍用指示函数 $\mathbb{I}\{\cdot\}$，沿用图 8.7的设定，累计值可以写成

$$g(S) = 2|S| + 3\,\mathbb{I}\{\{1,3\} \subseteq S\} - \mathbb{I}\{\{2,4\} \subseteq S\}.$$

其中 $2|S|$ 收集 S 中各单项的贡献，后两项在 S 包含 $\{1,3\}$ 或 $\{2,4\}$ 时计入相应的配合贡献。表 8.3列出全部 16 个累计值，以及只用这些累计值按式 (8.7)恢复的贡献。

S	$g(S)$	$h(S)$	S	$g(S)$	$h(S)$
$\varnothing$	0	0	$\{2,3\}$	4	0
$\{1\}$	2	2	$\{2,4\}$	3	-1
$\{2\}$	2	2	$\{3,4\}$	4	0
$\{3\}$	2	2	$\{1,2,3\}$	9	0
$\{4\}$	2	2	$\{1,2,4\}$	5	0
$\{1,2\}$	4	0	$\{1,3,4\}$	9	0
$\{1,3\}$	7	3	$\{2,3,4\}$	5	0
$\{1,4\}$	4	0	V_0	10	0

表 8.3　四元素子集格上的累计值 g 与由式 (8.7)恢复的自身贡献 h。恢复结果与教学设定一致：四个单项各为 2，$\{1,3\}$ 为 3，$\{2,4\}$ 为 -1，其余子集（包括全部三元子集和 V_0）均为 0。

逐个套用公式，两个二元子集的贡献按前面的四项公式恢复：

$$h(\{1,3\}) = 7 - 2 - 2 + 0 = 3, \qquad h(\{2,4\}) = 3 - 2 - 2 + 0 = -1.$$

三元子集要用到八个累计值。对 $S = \{1,2,3\}$，按子集大小依次取三元、二元、一元和空集项，

$$h(\{1,2,3\}) = 9 - (4 + 7 + 4) + (2 + 2 + 2) - 0 = 0.$$

虽然 $\{1,2,3\}$ 包含有配合效果的 $\{1,3\}$，它的累计值 9 也大于三个单项之和 6，但多出的 3 已经由 $h(\{1,3\})$ 解释，三门课之间没有新的整体效果，所以三元贡献为 0。

$\{1,3,4\}$ 也是如此：$g = 9$，恢复出 $h = 0$。其余两个三元子集同样得到 0，例如 $h(\{2,3,4\}) = 5 - (4 + 3 + 4) + (2 + 2 + 2) - 0 = 0$，其中 -1 的配合贡献已由 $h(\{2,4\})$ 解释。反演还允许贡献为负，$h(\{2,4\}) = -1$ 就是一例。

8.3.3 反演的计算

8.3.3.1 问题与思路

输入是全部 2^n 个累计值，输出是全部 2^n 个自身贡献。直接套用式 (8.7)，计算 $h(S)$ 要读取 S 的全部 $2^{|S|}$ 个子集上的累计值。

另一种做法是由式 (8.6)移项逐个求解：

$$h(S) = g(S) - \sum_{T \subsetneq S} h(T).$$

从 $h(\varnothing) = g(\varnothing)$ 出发，按元素个数从小到大计算，算到 S 时它的全部真子集的 h 都已求出。在四元素算例中，先得到 $h(\varnothing) = 0$ 和单项贡献 2，再得 $h(\{1,3\}) = 7 - (0 + 2 + 2) = 3$ 等二元贡献，最后 $h(\{1,2,3\}) = 9 - (0 + 2 + 2 + 2 + 0 + 3 + 0) = 0$。

这种递推同样要对每个 S 遍历它的全部子集。k 元子集有 $\binom{n}{k}$ 个，各有 2^k 个子集，所以两种做法读取的项数都是 $\sum_{k=0}^{n} \binom{n}{k} 2^k = 3^n$（二项式定理）。两种做法的共同瓶颈是每个 S 都要单独遍历它的全部子集。

上一小节把单个值 $h(\{1,3\})$ 写成了两次相邻差，这提示可以按元素而不是按子集组织计算。式 (8.7)中的符号可以按元素拆开：对 $T \subseteq S$，$(-1)^{|S|-|T|}$ 是 $S \setminus T$ 中每个元素各贡献一个 -1 的乘积。于是交错和可以一个元素一个元素地做：先对全部子集沿“加入元素 1”的方向取相邻差，再在结果上沿元素 2 的方向取差，依此类推。

每个元素只处理一轮，一轮对 2^{n-1} 对相邻子集各做一次减法，不再为每个 S 单独遍历它的子集。这种做法称为逐元素差分。

8.3.3.2 算法实现

元素的编号 $1, \ldots, n$ 与子集在数组中的位置在开始前确定，运行中不变，不算状态。

子集数组的数据结构 数组以子集为下标。沿用第 8.1.2小节的指示向量，把 S 的指示向量 $(x_1, \ldots, x_n)$ 读成二进制数 $\sum_{i=1}^{n} x_i 2^{i-1}$，作为 S 在长度为 2^n 的数组中的位置。S 含元素 i，当且仅当这个数从低位数起的第 i 位为 1；这时 $S \setminus \{i\}$ 的位置是它减去 2^{i-1}。例如 $n = 3$ 时 $\{1,3\}$ 的位置是 $101_2 = 5$，$\{1,3\} \setminus \{1\} = \{3\}$ 的位置是 $5 - 1 = 4$。依次扫描位置 $0, 1, \ldots, 2^n - 1$，挑出第 i 位为 1 的位置，就把含 i 的子集各访问一次，每次用一次减法找到 $S \setminus \{i\}$。

逐元素差分的实现 算法维护两项状态：

- 数组 $a(\cdot)$：以子集为下标，长度为 2^n，初值 $a(S) = g(S)$；

- 当前元素 i：本轮处理的元素，初值为 1。

第 i 轮按子集是否含 i 分别处理，处理完令 $i \leftarrow i+1$：

- 含 i 的子集 S：令 $a(S) \leftarrow a(S) - a(S \setminus \{i\})$；
- 不含 i 的子集：保持不变。

本轮读取的 $a(S \setminus \{i\})$ 所在的子集不含 i，本轮不会被修改，所以更新可以原地进行，不需要另存上一轮的数组。处理完元素 n 后停止，输出 a。在子集格上看，第 i 轮只沿“加入元素 i”方向的连线做差，图 8.2中同色的一组边就是一轮读取的方向。

算法 8.1　逐元素差分

输入：底集 $V = \{1, \dots, n\}$ 上的全部累计值 $g(S)$。输出：全部自身贡献 $h(S)$。

1. 对每个 $S \subseteq V$ 置 $a(S) = g(S)$；令 $i = 1$。
2. 对每个含 i 的子集 S，置 $a(S) = a(S) - a(S \setminus \{i\})$；不含 i 的子集不变。
3. 若 $i < n$，令 $i \leftarrow i+1$ 并回到第 2 步；否则输出 $h(S) = a(S)$。

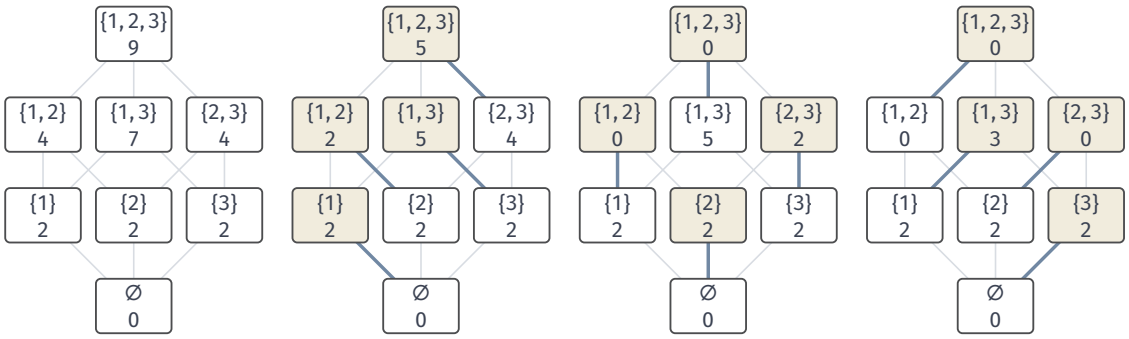
执行过程　先在三元子集格 $\{1,2,3\}$ 上执行。四元素设定限制到 $\{1,2,3\}$ 的子集上，仍是同一组累计值与贡献，因为 $g(S)$ 和 $h(S)$ 只读取 S 的子集。初始数组就是累计值：$a(\varnothing) = 0$，三个单元素集为 2，$a(\{1,2\}) = 4$，$a(\{1,3\}) = 7$，$a(\{2,3\}) = 4$，$a(\{1,2,3\}) = 9$。每轮只改动含被处理元素的四个子集：

- 处理元素 1：$\{1\}$ 为 $2-0=2$，$\{1,2\}$ 为 $4-2=2$，$\{1,3\}$ 为 $7-2=5$，$\{1,2,3\}$ 为 $9-4=5$。
- 处理元素 2：$\{2\}$ 为 $2-0=2$，$\{1,2\}$ 为 $2-2=0$，$\{2,3\}$ 为 $4-2=2$，$\{1,2,3\}$ 为 $5-5=0$。
- 处理元素 3：$\{3\}$ 为 $2-0=2$，$\{1,3\}$ 为 $5-2=3$，$\{2,3\}$ 为 $2-2=0$，$\{1,2,3\}$ 为 $0-0=0$。

最终数组就是 h：单项各为 2，$h(\{1,3\}) = 3$，其余为 0。

$\{1,3\}$ 的值 $7 \to 5 \to 5 \to 3$ 正是上一小节的两次相邻差：处理元素 1 时减去 $g(\{3\})$，处理元素 3 时减去已经差分过的 $a(\{1\}) = g(\{1\}) - g(\varnothing)$。图 8.8画出三轮的状态。

四元素算例照样做四轮，每轮改动含 i 的 8 个子集，最后的数组与表 8.3中的 h 逐项一致。例如 $\{2,4\}$ 依次为 $3, 3, 1, 1, -1$：处理元素 2 时减去 $a(\{4\}) = 2$，处理元素 4 时减去当前的 $a(\{2\}) = 2$。中间各轮的数组只是部分差分，只有最后一轮之后才是 h。



(a) 初始：存储 g　(b) 处理元素 1 后　(c) 处理元素 2 后　(d) 处理元素 3 后，即 h

图 8.8　逐元素差分在三元子集格 $\{1,2,3\}$ 上的状态变化，累计值取本节四元素设定在 $\{1,2,3\}$ 的子集上的值。框内下行是数组 a 的当前值。每一轮中，含被处理元素 i 的子集（黄色）减去下方 $S\setminus\{i\}$ 处的当前值，蓝色粗线连接这一轮读取的两个子集。三轮之后，数组中存的恰是自身贡献 h。

8.3.3.3 算法分析

正确性　上面按元素拆分符号的说法，可以由一个不变式精确表达。记 $[i]=\{1,\dots,i\}$。断言：处理完元素 $1,\dots,i$ 后，

$$a(S)=\sum_{\substack{T\subseteq S\\ T\setminus[i]=S\setminus[i]}}(-1)^{|S|-|T|}g(T),$$

即只对那些与 S 在元素 $i+1,\dots,n$ 上一致的子集 T 做交错求和。$i=0$ 时条件迫使 $T=S$，右侧就是 $g(S)$，与初始化一致。

设断言对 i 成立，看第 $i+1$ 轮。若 $i+1\notin S$，$a(S)$ 不变；而 S 的子集都不含 $i+1$，“在 $i+1,\dots,n$ 上一致”与“在 $i+2,\dots,n$ 上一致”是同一个条件，断言对 $i+1$ 成立。若 $i+1\in S$，新值是 $a(S)-a(S\setminus\{i+1\})$。前一项求和的 T 与 S 在 $i+1,\dots,n$ 上一致，因而含 $i+1$；后一项求和的 T 与 $S\setminus\{i+1\}$ 一致，因而不含 $i+1$，符号 $(-1)^{|S|-1-|T|}$ 再乘以 -1 变成 $(-1)^{|S|-|T|}$。两组 T 合起来，恰是与 S 在 $i+2,\dots,n$ 上一致的全部子集，断言对 $i+1$ 成立。

取 $i=n$，条件不再限制 T，右侧正是式 (8.7)，所以输出就是 h。

复杂度　逐元素差分每轮做 2^{n-1} 次减法，共 $n2^{n-1}$ 次；按上面的数据结构，每次找到 $S\setminus\{i\}$ 只需一次整数减法，原地更新只用长度为 2^n 的数组。逐个套用公式与按子集大小递推都需要 3^n 次运算，四元素算例中是 32 次对 81 次。

不过输入本身已有 2^n 个数，三种方式都随 n 指数增长，单个递推式写起来简短并不意味着问题便宜。

累计方向　把每一轮的减号换成加号、从 $a = h$ 出发做 n 轮，同样的归纳（不变式中的符号 $(-1)^{|S|-|T|}$ 换成 1，g 换成 h）说明得到的是累计值 g。例如 $\{1,3\}$ 由 3 经 $3+2=5$、$5+2=7$ 回到 $g(\{1,3\})$。所以累计和反演可以用同一套逐元素扫描完成，开销也相同。

8.3.4　有限偏序集上的推广

反演公式中的系数 $(-1)^{|S|-|T|}$ 只依赖两个子集的相对位置。同样的问题可以在任意有限偏序集 $(X,\preceq)$ 上提出：给每个元素 y 一个数 $h(y)$，沿下方累计得 $g(x)=\sum_{y\preceq x}h(y)$，再问怎样由 g 恢复 h。这时系数不再一律是 ±1，而由下面的函数确定。

定义 8.7　Möbius 函数　设 $(X,\preceq)$ 是有限偏序集，记 $y\prec x$ 表示 $y\preceq x$ 且 $y\neq x$。偏序集的Möbius 函数（Möbius function）μ 对满足 $y\preceq x$ 的元素对定义为

$$\mu(y,y)=1,\qquad \mu(y,x)=-\sum_{z\colon y\preceq z\prec x}\mu(y,z)\quad (y\prec x),$$

对不满足 $y\preceq x$ 的元素对取 $\mu(y,x)=0$。

定义固定区间的下端 y，从对角值 $\mu(y,y)=1$ 出发向上递推：算 $\mu(y,x)$ 时只用到区间 $\{z:y\preceq z\prec x\}$ 上已经算出的值，而有限偏序集保证递推在有限步内完成。递推式移项后写成

$$\sum_{z\colon y\preceq z\preceq x}\mu(y,z)=0\qquad (y\prec x),\tag{8.8}$$

即每个非平凡区间上的系数之和为零；$y=x$ 时这个和是 $\mu(y,y)=1$。

这一抵消性质足以完成反演,论证分两步。先验证按 $\tilde h(x)=\sum_{y\preceq x}\mu(y,x)g(y)$ 定义的函数累计后回到 g。对任意 $s\in X$，交换有限和的次序，

$$\sum_{x\preceq s}\tilde h(x)=\sum_{x\preceq s}\sum_{y\preceq x}\mu(y,x)g(y)=\sum_{y\preceq s}g(y)\sum_{x\colon y\preceq x\preceq s}\mu(y,x).$$

由式 (8.8)，内层和在 $y\prec s$ 时为 0，在 $y=s$ 时为 1，所以右侧等于 $g(s)$。

再说明满足累计式的 h 只有一个。累计式可以写成 $h(x)=g(x)-\sum_{y\prec x}h(y)$，右侧只用到严格位于 x 下方的元素。把有限偏序集的元素排成一列，使每个元素都排在它上方的元素之前（例如反复取出当前剩余元素中的一个极小元），按这个次序逐个计算，h 的每个值都被 g 唯一确定。$\tilde h$ 也满足累计式，所以 $h=\tilde h$，即

$$h(x)=\sum_{y\preceq x}\mu(y,x)g(y).$$

在子集格上，$\mu(T,S)=(-1)^{|S|-|T|}$ 满足这个递推：对 $T\subsetneq S$，记 $q=|S\setminus T|\geq 1$，

$$\sum_{R:\,T\subseteq R\subseteq S}(-1)^{|R|-|T|}=\sum_{j=0}^{q}(-1)^j\binom{q}{j}=0.$$

由于递推从 $\mu(T,T)=1$ 出发唯一确定全部值，子集格上的 Möbius 函数就是 $(-1)^{|S|-|T|}$，只取 ±1，式 (8.7)是一般反演式的特例。

整除偏序 图 8.9是 12 的六个正因数在整除关系下的偏序。从 $\mu(1,1)=1$ 出发，$\mu(1,2)=\mu(1,3)=-1$；区间 $\{1,2,4\}$ 是一条链，$\mu(1,4)=-(1-1)=0$；区间 $\{1,2,3,6\}$ 与两元素的子集格形状相同，$\mu(1,6)=-(1-1-1)=1$；最后 $\mu(1,12)=-(1-1-1+0+1)=0$。系数出现了 0，说明一般偏序上的反演确实不同于子集格上的交错加减。

本小节只作为有限组合结构的补充，不涉及无限偏序和收敛问题，后面的选择算法也不以它为前提。

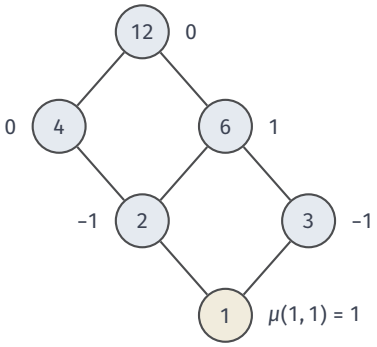


图 8.9 12 的正因数在整除关系下的偏序，连线连接直接相邻的两个因数，下方整除上方。节点旁的数是 $\mu(1,x)$，由 $\mu(1,1)=1$（黄色节点）自下而上递推得到。链 1 | 2 | 4 上 $\mu(1,4)=0$，与子集格同形的区间 $\{1,2,3,6\}$ 上 $\mu(1,6)=1$。

8.4 本章小结

本章从传感器覆盖出发，把对整组选择的评价写成有限底集上的集合函数。底集可以是传感器、特征、顶点或边，不必带图结构。n 元底集的完整取值表有 2^n 项，所以函数通常由求值规则给出，算法的代价要分开计入求值次数和每次求值的开销。指示向量把子集换成零一向量，只换了索引方式，不带来可加性或局部分解。边际增益 $\Delta(v\mid S)$ 是后面一切判别的基本量：它依赖已选集合，沿任一加入次序累加，总和等于两端函数值之差。

用边际增益可以分别回答三个问题。单调性问新增元素会不会降低评价，等价于全部边际增益非负；模性问每个元素的贡献是否固定，等价于四值等式，模函数恰是常数加固定权重之和；次模性问收益是否随已选集

合增大而递减，在有限底集上与四值不等式等价。覆盖量与非负边权的割权是次模的典型例子，协同效应则给出超模函数。

这些性质互不蕴含。割权次模而不单调；$\max\{3\,\mathbb{I}\{\{1,2\}\subseteq S\},\mathbb{I}\{3\in S\}\}$ 中元素 $1,2$ 只有同时选中才有大价值，它单调而不次模；模函数同时次模和超模，却可能因为负权重而不单调。归一化只是减去常数 $F(\varnothing)$ 的约定，不改变任何边际增益。所以后面引用某种算法保证时，要逐项写明它需要哪些性质，不能只凭“次模”判断。

子集累计与 Möbius 反演回答另一个问题：由全部累计值恢复每个子集自身的贡献。反演按元素个数之差交替加减，依靠的是组合数交错和为零的抵消，它与累计互为逆变换，结果可以为负。这一问题只依赖有限求和与包含关系，不需要单调或次模。计算上，逐个套用公式和按子集大小递推都要 3^n 次运算，逐元素差分降到 $n2^{n-1}$ 次，但仍随底集大小指数增长。一般有限偏序集上，Möbius 函数由区间递推确定，子集格上的交替符号是它的特例。

下一章利用本章的边际增益和次模性，在基数或预算约束下最大化集合函数；第 10章再把一类可分解为一元项与成对项的次模函数化为割，精确求最小值。

第 9 章

约束下的子集选择

上一章的传感器实例里，四个传感器合起来能覆盖全部 16 格。实际中名额或经费有限：只能装两个传感器时该装哪两个？各传感器价格不同、总经费固定时又该怎样选？这类问题在有限底集上最大化一个集合函数，同时限制所选子集的规模或总成本。小实例可以枚举全部可行子集，底集稍大时枚举就不可行，于是需要只看少量子集、又能说明结果离最优有多远的方法。

本章研究逐轮加入元素的贪心规则，分基数约束和预算约束两节。第 9.1节在“至多选 k 个”的限制下每轮加入边际增益最大的元素，证明目标归一化、单调且次模时，所得值不低于最优值的 $1-(1-1/k)^k$ 倍；同一节还说明次模性怎样让旧增益成为当前增益的上界，从而得到减少求值次数的惰性贪心。第 9.2节让各元素占用不同的预算，把比较依据换成单位成本的增益，并用反例说明这一规则不能继承基数约束下的保证。

两节都要分清三件事：贪心规则本身对任何集合函数都能执行；惰性实现与普通贪心选出同样元素要靠次模性；比例保证则同时依赖归一化、单调、次模和约束的形式。本章使用上一章的边际增益、逐个加入的累计式和次模性的两种刻画。

9.1 基数约束下的子集选择

回到传感器。只允许安装 k 个传感器时，应当选哪几个？枚举很快变得不可行，于是改为逐轮加入当前增益最大的元素。本节先给出这一规则及其实现，再证明目标归一化、单调且次模时所得值离最优值不远，最后利用次模性跳过大部分增益的重算。

9.1.1 问题与思路

设 V 是 n 元底集，F 是 V 上的集合函数，整数 k 满足 $0 \le k \le n$。要求的是

$$\max_{S \subseteq V,\ |S| \le k} F(S). \tag{9.1}$$

“至多选 k 个”称为**基数约束**（cardinality constraint）。F 通过求值规则给出：算法可以对任意子集询问 F 的值，每次询问计一次求值。计算成本因此同时取决于求值次数和每次求值的开销。算法输出一个至多含 k 个元素的集合，希望它的函数值尽量接近最优值。

对单调函数，“至多 k 个”与“恰好 k 个”给出同样的最优值。若某个最优集合不足 k 个元素，任意补入元素使它恰有 k 个，由单调性函数值不会下降，补足后的集合仍是最优的。非单调函数不能这样处理：补入的元素可能降低目标值，所以“至多”与“恰好”是两个不同的问题。

小底集可以枚举全部可行子集，所得的最优值是衡量其他方法的基准。在传感器实例中取 $k=2$，可行子集有 $\binom{4}{0}+\binom{4}{1}+\binom{4}{2}=11$ 个。逐个读出覆盖量（第 8.1.1小节），最优值是 16，最优集合唯一，为 $\{b,c\}$；单元素中最好的是 $\{a\}$，覆盖 10 格。

一般地，最优值是唯一的，最优集合却可能有多个并列；$k=3$ 时，$\{b,c\}$、$\{a,b,c\}$、$\{b,c,d\}$ 都覆盖全部 16 个格子，按“至多 3 个”的约束三者都是最优集合。

但枚举在规模稍大时就不可行。即使一次求值很便宜，$n=100$、$k=10$ 时，仅恰有 10 个元素的子集就有 $\binom{100}{10}\approx 1.7\times 10^{13}$ 个。

只看少量子集的办法是逐步构造：从空集出发，每一轮在尚未选中的元素里挑当前增益最大的那个加入，选满 k 个为止。由式 (8.1)，最终集合的函数值等于 $F(\varnothing)$ 加上各轮增益之和，每一轮都让其中一项尽量大。

增益必须相对于已经选好的集合重新计算，只按初始增益排序一次是不够的。传感器实例中初始增益的次序是 a 为 10，然后 b,c 并列为 8；按编号取前两个得到 $\{a,b\}$，只覆盖 12 格。选了 a 之后，b 的增益从 8 降到 2，c 的增益从 8 降到 4，排序已经改变。逐轮重算反映的正是已选元素造成的重叠。

9.1.2 算法实现

底集及其编号、整数 k 和求值规则都在运行前给定，运行中不变，不算状态。算法维护三项状态：

- 已选集合 S：初值为空集；
- 当前值 $F(S)$：初值由一次求值得到 $F(\varnothing)$，此后不另行求值，每轮加上选中元素的增益；
- 本轮增益：每个未选元素 v 相对于当前 S 的增益 $\Delta(v\mid S)$，每轮开始时全部重新计算，不跨轮保留。

初始化只需一次求值，得到 $F(\varnothing)$。每一轮做三件事：

- 求增益：对每个 $v \in V \setminus S$ 求值 $F(S \cup \{v\})$，减去记录的当前值，得到 $\Delta(v \mid S)$；
- 选择：取增益最大的元素 v，增益相同时取编号最小者；
- 更新：令 $F(S) \leftarrow F(S) + \Delta(v \mid S)$，再令 $S \leftarrow S \cup \{v\}$。

$|S| = k$ 时停止，输出 S。记第 t 轮选中的元素为 v_t、结束时的集合为 S_t，于是 $S_0 = \varnothing$，$S_t = S_{t-1} \cup \{v_t\}$。平局规则必须事先规定，否则同一输入可能得到不同输出，结果无法复核。

停止条件还有一种常用的变形：某一轮全部未选元素的增益都是 0 时提前停止，输出当前集合 S。目标单调且次模时，这样做不损失函数值。对任意 $T \subseteq V \setminus S$，把 T 的元素逐个加入 S，由式 (8.1)和次模性，每一步的增益不超过该元素相对于 S 的增益 0，所以 $F(S \cup T) \le F(S)$；单调性又给出 $F(S \cup T) \ge F(S)$，于是继续选满 k 个只会保持当前值。

只有单调性时，提前停止可能错过价值。单个元素的增益都是 0，并不排除两个元素一起加入才产生价值。第 9.1.3小节末尾的单调非次模函数 $F(S) = \max\{3\,\mathbb{I}\{\{1,2\} \subseteq S\}, \mathbb{I}\{3 \in S\}\}$ 取 $k = 3$ 时，第 1 轮选 3，第 2 轮元素 $1, 2$ 相对于 $\{3\}$ 的增益都是 0：提前停止得到 1；继续选满则依次加入 1、2，最后一步增益为 3，得到 $F(\{1,2,3\}) = 3$。下面的算法与分析按选满 k 个书写。

算法 9.1　基数约束下的边际增益贪心

输入：有限底集 V（元素已编号），整数 $0 \le k \le |V|$，单调非减集合函数 F 的求值规则。输出：至多含 k 个元素的集合。

1. 令 $S_0 = \varnothing$，记录 $F(S_0)$。
2. 对 $t = 1, \dots, k$：对每个 $v \in V \setminus S_{t-1}$，求值 $F(S_{t-1} \cup \{v\})$，得到 $\Delta(v \mid S_{t-1})$；取增益最大的 v_t，增益相同时取编号最小者；令 $S_t = S_{t-1} \cup \{v_t\}$，$F(S_t) = F(S_{t-1}) + \Delta(v_t \mid S_{t-1})$。
3. 输出 S_k。

执行过程　在传感器实例上取 $k = 2$，按字母顺序编号。初始 $S_0 = \varnothing$，$F(S_0) = 0$。

- 第 1 轮：增益就是各传感器单独的覆盖量，a, b, c, d 依次为 $10, 8, 8, 1$。选 a，$S_1 = \{a\}$，$F(S_1) = 10$。
- 第 2 轮：相对于 $\{a\}$ 重新计算。b 只补上第 1 列的 2 格，c 补上第 7、8 列的 4 格，d 补上左上角 1 格，增益依次为 $2, 4, 1$。选 c，$S_2 = \{a, c\}$，$F(S_2) = 14$。

此时 $|S| = k$，停止。图 9.1把两轮增益画成条形，并与枚举最优集合 $\{b, c\}$（16 格）比较：贪心集合漏掉了第 1 列的两格。

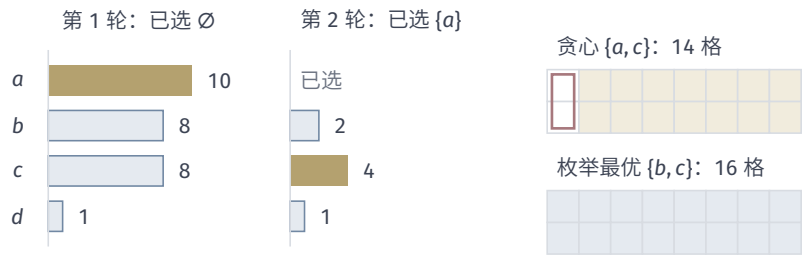


图 9.1　传感器实例上 $k = 2$ 的边际增益贪心。左侧两栏是两轮中各候选元素的当前增益，黄色条为该轮选中的元素；第 2 轮的增益相对于已选的 $\{a\}$ 计算。右侧比较两个覆盖结果：贪心集合 $\{a, c\}$（黄色）漏掉第 1 列两格（红框），枚举最优集合 $\{b, c\}$（蓝色）覆盖全部 16 格。

9.1.3　算法分析

可行性　算法每一轮恰好加入一个尚未选中的元素，执行 k 轮，所以输出恰有 k 个元素，满足基数约束；由于 $k \le n$，每一轮都有候选元素可选。F 单调时每一轮的增益都非负，选满 k 个不会使目标下降。

近似保证　实例中贪心的 14 与最优的 16 相差不多。这不是巧合，可以从“差距”的角度看清原因。记最优值与当前值之差为差距，第 1 轮开始时差距是 $16 - 0 = 16$。

最优集合 $O = \{b, c\}$ 只有两个元素，把它们加入当前集合能补上全部差距；它们相对于当前集合的增益分别是 8 和 8，合计不少于差距，所以其中较大的一个至少补上差距的一半。贪心选的是全体元素中增益最大的，当然不比它差，实际选中的 a 补上了 10。

第 2 轮开始时差距剩 6，b, c 相对于 $\{a\}$ 的增益是 2 和 4，合计仍不少于 6，较大者至少是 3，贪心选中的 c 补上 4，差距降为 2。图 9.2画出这两轮比较。

一般情形的论证与此相同。最优集合至多有 k 个元素，它们合起来能补上全部差距；由次模性，每个元素相对于当前集合的增益不小于它在“合起来补”时的那一份，所以这些增益之和不少于差距，其中最大者至少是差距的 $1/k$。

贪心每轮至少补上剩余差距的 $1/k$，差距每轮至多保留原来的 $1 - 1/k$，经过 k 轮至多剩下 $(1 - 1/k)^k$，不超过 e^{-1}。这就像每期至少偿还剩余欠款的 $1/k$，欠款按几何级数下降。

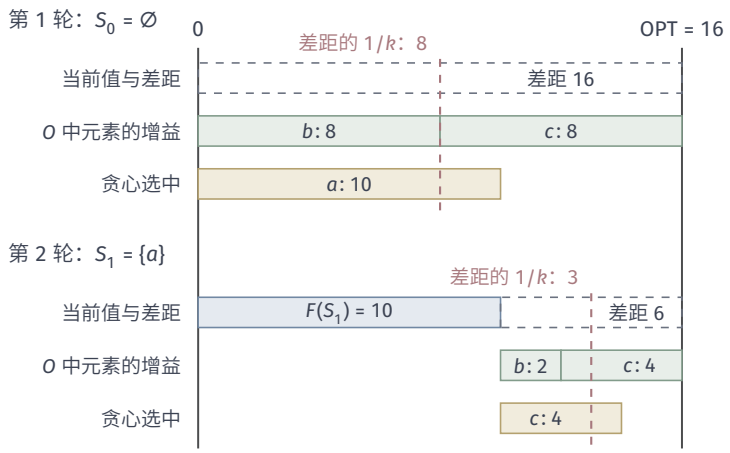


图 9.2 贪心每轮至少补上差距的 $1/k$（传感器实例，$k = 2$，最优集合 $O = \{b, c\}$）。每轮上行是当前值（蓝色）与到最优值 16 的差距（虚线框）；中行是 O 中元素相对于当前集合的增益（绿色），它们首尾相接至少覆盖整个差距；红色虚线标出差距的 $1/k$，O 中至少有一个元素的增益越过它；下行是贪心选中元素的增益（黄色），它不小于 O 中任何元素的增益。

定理 9.1 贪心的近似保证 设 F 是有限底集 V 上的集合函数，满足 $F(\varnothing) = 0$、单调非减且次模；整数 $1 \le k \le |V|$。记 OPT 为问题 (9.1)的最优值，S_ℓ 为算法 9.1执行 ℓ 轮后的集合（$0 \le \ell \le k$）。则

$$F(S_\ell) \ge \Big(1 - \big(1 - \tfrac{1}{k}\big)^{\ell}\Big)\mathrm{OPT}.$$

特别地，$F(S_k) \ge \big(1 - (1 - 1/k)^k\big)\mathrm{OPT} \ge (1 - e^{-1})\mathrm{OPT}$。

证明 固定一个最优集合 O，$|O| \le k$，$F(O) = \mathrm{OPT}$。考察第 t 轮开始时的集合 S_{t-1}（$1 \le t \le k$），先用次模性把它与最优值的差距上界为若干个边际增益之和。把 $O \setminus S_{t-1}$ 中的元素排成 $o_1, \dots, o_r$，由单调性和式 (8.1)，

$$\mathrm{OPT} = F(O) \le F(O \cup S_{t-1}) = F(S_{t-1}) + \sum_{j=1}^{r} \Delta\big(o_j \mid S_{t-1} \cup \{o_1, \dots, o_{j-1}\}\big).$$

每个 o_j 都不在 $S_{t-1} \cup \{o_1, \dots, o_{j-1}\}$ 中，由式 (8.4)，第 j 项不超过 $\Delta(o_j \mid S_{t-1})$。而贪心在第 t 轮选的是增益最大的元素，所以 $\Delta(o_j \mid S_{t-1}) \le \Delta(v_t \mid S_{t-1}) = F(S_t) - F(S_{t-1})$。又 $r \le |O| \le k$，且每一项增益非负，于是

$$\mathrm{OPT} - F(S_{t-1}) \le k\big(F(S_t) - F(S_{t-1})\big).$$

也就是说，最优集合的至多 k 个元素合起来能补上的差距，至少有 $1/k$ 能被当前最好的单个元素补上。

记差距 $\delta_t = \mathrm{OPT} - F(S_t)$。上式可写成 $\delta_{t-1} \le k(\delta_{t-1} - \delta_t)$，即

$$\delta_t \le \Big(1 - \frac{1}{k}\Big)\delta_{t-1}, \qquad t = 1, \dots, k.$$

由 $F(\varnothing) = 0$ 得 $\delta_0 = \mathrm{OPT}$，连续使用 ℓ 次得 $\delta_\ell \le (1 - 1/k)^\ell \,\mathrm{OPT}$，移项即为所求。

最后比较 $(1-1/k)^k$ 与 e^{-1}。函数 $\psi(x) = e^{-x} - (1-x)$ 满足 $\psi(0) = 0$，导数 $\psi'(x) = 1 - e^{-x}$ 在 $x > 0$ 时为正，所以 ψ 在 $[0, +\infty)$ 上单调递增：闭区间上连续、内部导数为正的函数严格递增，这是微分中值定理的推论（可参考卷一第 6 章“一元函数的微分”）。从而 $0 \le 1 - x \le e^{-x}$ 对 $0 \le x \le 1$ 成立。取 $x = 1/k$ 再求 k 次幂，得到 $(1 - 1/k)^k \le e^{-1}$。 ■

证明中三个条件各有作用。归一化保证差距从 OPT 开始；若 $F(\varnothing) \neq 0$，给 F 加上一个常数会改变 $F(S_k)$ 与 OPT 的比值，比例式就失去意义。单调性用在 $F(O) \le F(O \cup S_{t-1})$ 和增益非负两处，次模性用在把 O 中元素的增益统一换成相对于 S_{t-1} 的增益。

$k = 1$ 时界为 1，贪心选出的就是最好的单元素；$k = 2$ 时界为 $3/4$。在传感器实例中逐轮核对：$\ell = 1$ 时界为 $\frac{1}{2} \times 16 = 8$，而 $F(S_1) = 10$；$\ell = 2$ 时界为 $\frac{3}{4} \times 16 = 12$，而 $F(S_2) = 14$，两轮都与定理一致。$1 - e^{-1} \approx 0.632$ 是对一切 k 都成立的比例，但它只是近似保证，贪心并不保证精确最优。

复杂度　第 t 轮要对 $n - t + 1$ 个未选元素各求值一次，总共

$$\sum_{t=1}^{k} (n - t + 1) \le nk$$

次求值，再加上开始时的一次 $F(\varnothing)$。传感器实例 $n = 4, k = 2$，共 $1+4+3 = 8$ 次。若一次求值的代价为 τ（例如覆盖量需要合并若干覆盖范围），总代价是 $O(nk\tau)$，选最大值等其余操作只占 $O(nk)$。与枚举相比，nk 次求值随 n, k 多项式增长，而可行子集的个数随 k 迅速增长。

适用范围　撤去次模性，保证就不再成立。图 8.6(a) 的函数 $F(S) = \max\{3\,\mathbb{I}\{\{1,2\} \subseteq S\}, \mathbb{I}\{3 \in S\}\}$ 单调、归一化，但不是次模的。取 $k = 2$：第 1 轮三个增益依次为 $0, 0, 1$，选 3；第 2 轮相对于 $\{3\}$，加入 1 或 2 的增益都是 0，按编号选 1，得到 $F(\{1,3\}) = 1$。最优集合是 $\{1,2\}$，$\mathrm{OPT} = 3$。

元素 1, 2 只有同时选中才有价值，每一轮单独看它们的增益都是零，贪心永远看不到这个互补组合。对照上面的差距论证，这里 $O = \{1,2\}$ 合起

来能补上差距 3，各自的增益却都是 0，“各元素增益之和不少于差距”这一步失效。把 3 换成任意大的 M，比例 $1/M$ 可以任意小，所以没有次模性就没有固定比例的保证。

撤去单调性同样会造成损失。图 8.6(b) 的单边割权是次模的，但不单调。取 $k=2$ 并按算法 9.1强制选满：第 1 轮选 1（增益 1），第 2 轮只剩 2，增益为 -1，仍须加入，结果 $\mathrm{cut}(\{1,2\})=0$，而至多选两个时的最优值是 1。在增益为负时停止可以避免这个例子中的损失，但对一般的非单调次模函数，这样修改后的贪心仍没有定理 9.1那样的保证，需要另外的算法与分析，本节不展开。

所以定理 9.1只适用于在基数约束下最大化归一化、单调、次模的集合函数，在这一范围内，贪心用至多 $nk+1$ 次求值换来比例保证。贪心规则本身对任何集合函数都能执行，但目标、约束或函数性质改变时，保证都需要重新论证；下一节的预算约束就是一例。

9.1.4 惰性贪心

9.1.4.1 问题与思路

普通贪心每轮都重算全部未选元素的增益，其中许多元素的旧增益已经远低于领先者，本轮不可能被选中。次模性提供了一个省力的办法：已选集合只会变大，由式 (8.4)，元素的增益只会变小，所以早先算出的增益是它当前增益的上界。若某个元素刚刚重算出的增益不小于其他所有元素的旧增益，它就不小于这些元素的当前增益，可以直接选中，其余元素不必重算。

在实例的第 2 轮，三个候选的旧增益是 b:8、c:8、d:1。先重算 b，得 2，它不再领先；再重算 c，得 4，不小于 b 的新值 2 与 d 的旧值 1。d 的当前增益不会超过 1，不必重算，选 c。这一轮只做了 2 次求值，选择与普通贪心相同。按这种方式缓存旧增益、只在必要时重算的实现称为**惰性贪心**（lazy greedy）。

9.1.4.2 算法实现

缓存增益的优先队列 每轮都要找出缓存增益最大的元素，并在某个元素的增益重算后把它放回合适的位置，这由以缓存增益为键的最大优先队列完成。每个未选元素 v 在队列中恰有一个条目 (v, g_v, r_v)：g_v 是缓存的增益，r_v 是算出它的轮次，即 $g_v=\Delta(v\mid S_{r_v-1})$。键的次序按 g_v 从大到小，g_v 相同时编号小者在前，与普通贪心的平局规则一致。用到的操作只有两个：取出最前的条目，以及插入一个条目。用二叉堆实现时，两者都需要 $O(\log n)$ 次比较。轮次 r_v 用来判断缓存是否过期：r_v 等于当前轮次时，g_v 就是相对于当前集合的增益。

惰性贪心的实现　与算法 9.1相同，底集、k 和求值规则在运行前给定；此外要求 F 是次模的，否则旧增益不是上界，下面的选择可能与普通贪心不同。算法维护四项状态：

- 已选集合 S：初值为空集；
- 当前值 $F(S)$：初值为一次求值得到的 $F(\varnothing)$；
- 当前轮次 t：正在进行的轮次，初值为 1；
- 优先队列：每个未选元素一个条目 (v, g_v, r_v)，初值 $g_v = \Delta(v \mid \varnothing)$、$r_v = 1$，由 n 次求值得到。

初始化共需 $1 + n$ 次求值，第 1 轮的缓存因此全部是新的。每一步取出队列最前的条目 (u, g_u, r_u)，按两种情形处理：

- 若 $r_u = t$，缓存是新的：选中 u，令 $S \leftarrow S \cup \{u\}$，$F(S) \leftarrow F(S) + g_u$，$t \leftarrow t + 1$；
- 若 $r_u < t$，缓存已过期：求值一次，令 $g_u \leftarrow \Delta(u \mid S)$、$r_u \leftarrow t$，把条目放回队列。

$|S| = k$ 时停止，输出 S。

算法 9.2　惰性贪心

输入：有限底集 V（元素已编号），整数 $0 \le k \le |V|$，单调非减且次模的集合函数 F 的求值规则。输出：与算法 9.1相同的集合。

1. 令 $S = \varnothing$，记录 $F(\varnothing)$，$t = 1$；对每个 $v \in V$ 求 $g_v = \Delta(v \mid \varnothing)$，令 $r_v = 1$，把 (v, g_v, r_v) 插入队列。
2. 当 $|S| < k$ 时，取出队列最前的条目 (u, g_u, r_u)。
3. 若 $r_u = t$，令 $S \leftarrow S \cup \{u\}$，$F(S) \leftarrow F(S) + g_u$，$t \leftarrow t + 1$；否则求 $g_u = \Delta(u \mid S)$，令 $r_u = t$，把条目放回队列。回到第 2 步。
4. 输出 S。

执行过程　为了看到旧缓存跨越两轮的情形，在传感器实例上取 $k = 3$。初始化求值 $F(\varnothing) = 0$ 与四个单元素增益，队列为 a:10、b:8、c:8、d:1，轮次都是 1。表 9.1逐步列出取出的条目、所做的处理和处理后的队列。

- 第 1 轮：取出 a，缓存是新的，直接选中，$S = \{a\}$，$F = 10$。
- 第 2 轮：取出 b，缓存来自第 1 轮，重算得 2 后放回；取出 c，重算得 4 后放回，排到最前；再取出 c，缓存是新的，选中，$S = \{a, c\}$，$F = 14$。d 的旧值 1 始终排在后面，没有重算。
- 第 3 轮：取出 b，它的缓存 2 来自第 2 轮，已过期，重算仍得 2 后放回；再取出 b，选中，$S = \{a, b, c\}$，$F = 16$。

此时 $|S| = 3$，停止。三轮的选择与普通贪心逐轮重算的结果一致，求值共 $1 + 4 + 2 + 1 = 8$ 次，普通贪心需要 $1 + 4 + 3 + 2 = 10$ 次。

表 9.1 传感器实例上 $k=3$ 的惰性贪心。每行是一次取出："重算"表示缓存过期而求值一次后放回，"选中"表示缓存是新的；队列按键的次序列出，每项写元素、缓存增益与算出它的轮次。

轮	取出	处理	处理后的队列
初始	—	求值 5 次	$a(10,1)$，$b(8,1)$，$c(8,1)$，$d(1,1)$
1	$a(10,1)$	选中	$b(8,1)$，$c(8,1)$，$d(1,1)$
2	$b(8,1)$	重算得 2	$c(8,1)$，$b(2,2)$，$d(1,1)$
2	$c(8,1)$	重算得 4	$c(4,2)$，$b(2,2)$，$d(1,1)$
2	$c(4,2)$	选中	$b(2,2)$，$d(1,1)$
3	$b(2,2)$	重算得 2	$b(2,3)$，$d(1,1)$
3	$b(2,3)$	选中	$d(1,1)$

9.1.4.3 算法分析

正确性 惰性贪心每轮选出的元素与普通贪心按同一平局规则选出的相同，从而定理 9.1的保证原样适用。证明对轮次归纳：设前 $t-1$ 轮两者选出相同元素，当前集合都是 S_{t-1}。

在第 t 轮，设取出的条目 (u, g_u, t) 缓存是新的，则 $g_u = \Delta(u \mid S_{t-1})$。对队列中任一其他元素 v，它的缓存 $g_v = \Delta(v \mid S_{r_v-1})$ 是在更小的集合 $S_{r_v-1} \subseteq S_{t-1}$ 上算出的，由式 (8.4)，$g_v \geq \Delta(v \mid S_{t-1})$。$u$ 排在 v 前面，所以 $\Delta(u \mid S_{t-1}) \geq g_v \geq \Delta(v \mid S_{t-1})$。若某个 v 的当前增益与 u 相等，则 g_v 也与 g_u 相等，u 排在前面说明它的编号更小。所以 u 正是普通贪心第 t 轮的选择。

每一轮都会结束：每次重算都把一个过期条目变成新的，一轮内至多重算 $n-t+1$ 次，之后队列最前的条目必定是新的而被选中。

这里的正确性只用到次模性，没有用到单调性和归一化；它们是近似保证需要的条件。对一般函数，旧增益不是上界，排在后面的元素的当前增益可能已经超过选中的元素，因此必须每轮全部重算。

复杂度 初始化需要 $1+n$ 次求值。第 $t \geq 2$ 轮的重算次数不超过未选元素个数 $n-t+1$，所以最坏情形的求值次数与普通贪心相同；省下多少取决于增益下降的快慢，没有一般的保证。每次重算伴随一次取出与一次插入，每轮再有一次取出用于选中，每次队列操作需要 $O(\log n)$ 次比较，最坏总开销为 $O(nk \log n)$ 次比较，另加初始插入的 $O(n \log n)$ 次比较。队列存放 n 个条目，额外空间为 $O(n)$。一次求值的代价 τ 通常远大于 $O(\log n)$，这时省下的求值次数决定了实际运行时间。

9.2 预算约束下的子集选择

基数约束让每个元素占用一个名额。实际中不同传感器的价格不同，限制的往往是总花费而不是个数。逐轮贪心仍然可用，但比较的对象要改成单

位成本的增益。本节说明这一改动的理由与实现，再用反例说明上一节的近似保证不能直接搬到预算约束下。

9.2.1 问题与思路

给每个元素 $v \in V$ 指定成本 c_v，再给定总预算 $B \geq 0$。要求的是

$$\max_{S \subseteq V} F(S) \qquad \text{满足} \quad \sum_{v \in S} c_v \leq B. \tag{9.2}$$

满足成本条件的子集称为预算可行的，这一条件称为**预算约束**（budget constraint）。目标函数 F 与上一节相同，改变的只是允许的子集。所有成本都取 1、预算取 k 时，预算约束就是基数约束，所以本节的问题是上一节的推广。

成本为零的元素需要先单独处理。零成本元素不占预算，若 F 单调，把它们全部加入不会降低目标，可以在开始时一次放进集合；若 F 不单调，它们是否值得加入要看增益的正负。以下设所有成本为正。另外，成本超过 B 的元素不可能出现在任何可行集合中，可以预先删去。

在传感器实例中取第 8.1.1小节的成本 $c_a = 4$、$c_b = 3$、$c_c = 4$、$c_d = 1$，预算 $B = 7$。预算可行的子集共有 10 个：空集、四个单元素集，以及 $\{a, b\}$（成本 7）、$\{a, d\}$（5）、$\{b, c\}$（7）、$\{b, d\}$（4）、$\{c, d\}$（5）；三元及以上的子集成本都超过 7。枚举得最优集合为 $\{b, c\}$，覆盖 16 格。

与 $k = 2$ 的基数约束相比，$\{a, c\}$ 成本为 8，现在不可行了；而 $\{a, b\}$ 在两种约束下都可行。选择的个数不再直接决定资源消耗，每次加入元素前都要检查剩余预算。

把上一节的贪心原样搬过来，就是每轮在付得起的元素里选增益最大的。这样做可能偏爱昂贵的元素：一个元素增益大，可能只是因为它贵，而它占去的预算本可以换来更多增益。预算是按成本消耗的，所以更合理的比较是每单位成本带来多少增益，即比值 $\Delta(v \mid S)/c_v$。

这样得到单位成本规则：每轮选比值最大且付得起的元素，直到剩余预算放不下任何元素，或者付得起的元素都不再带来增益。

9.2.2 算法实现

成本 c_v、预算 B 和求值规则在运行前给定，运行中不变，不算状态。与上一节相比，算法多记录剩余预算，并且每轮只在付得起的元素中比较。算法维护四项状态：

- 已选集合 S：初值为空集；
- 剩余预算 $R = B - \sum_{v \in S} c_v$：初值为 B；

- 当前值 $F(S)$：初值由一次求值得到 $F(\varnothing)$，此后每轮加上选中元素的增益；
- 本轮增益：每个付得起的未选元素 v 相对于当前 S 的增益 $\Delta(v \mid S)$，每轮重新计算。

初始化令 $S=\varnothing$、$R=B$，并求值一次得到 $F(\varnothing)$。每一轮做四件事：

- 列候选：记 $Q=\{v\in V\setminus S:\ c_v\le R\}$ 为付得起的未选元素；若 Q 为空，停止；
- 求增益：对每个 $v\in Q$ 求值，得到 $\Delta(v\mid S)$；若这些增益全为零，停止；
- 选择：取比值 $\Delta(v\mid S)/c_v$ 最大的 v，比值相同时取编号最小者；
- 更新：令 $F(S)\leftarrow F(S)+\Delta(v\mid S)$，再令 $S\leftarrow S\cup\{v\}$，$R\leftarrow R-c_v$。

停止条件因此有两个：剩余预算放不下任何未选元素，或者付得起的元素都不再增加目标。

算法 9.3　预算约束下的单位成本贪心

输入：有限底集 V（元素已编号），正成本 c_v，预算 $B\ge 0$，单调非减集合函数 F 的求值规则。输出：预算可行的集合。

1. 令 $S=\varnothing$，$R=B$，记录 $F(\varnothing)$。
2. 记候选集合 $Q=\{v\in V\setminus S:\ c_v\le R\}$。若 Q 为空，停止。
3. 对每个 $v\in Q$ 计算 $\Delta(v\mid S)$。若这些增益全为零，停止；否则取比值 $\Delta(v\mid S)/c_v$ 最大的 v，比值相同时取编号最小者。
4. 令 $S\leftarrow S\cup\{v\}$，$R\leftarrow R-c_v$，回到第 2 步。

执行过程　在 $B=7$ 的实例上，表 9.2逐轮记录状态。

- 第 1 轮：$R=7$，四个元素都付得起，增益是各自的覆盖量，比值为 a:$10/4=2.5$，b:$8/3\approx 2.67$，c:$8/4=2$，d:$1/1=1$。选 b，$R=4$。
- 第 2 轮：$Q=\{a,c,d\}$，增益相对于 $\{b\}$ 重新计算。a 只补上第 5、6 列的 4 格，比值 1；c 与 b 不重叠，增益 8，比值 2；d 那一格已被 b 覆盖，增益 0。选 c，$R=0$。
- 第 3 轮：没有成本不超过 0 的元素，Q 为空，停止。

输出 $\{b,c\}$，覆盖 16 格，恰为枚举最优值。

表 9.2　$B=7$ 时单位成本贪心的状态表（成本 $c_a=4$、$c_b=3$、$c_c=4$、$c_d=1$）。每格写增益，括号内为增益与成本之比；“—”表示已选或付不起；最后一列是本轮选中的元素与更新后的剩余预算。

轮	S	R	a	b	c	d	选中，新 R
1	$\varnothing$	7	10 (2.5)	8 (2.67)	8 (2)	1 (1)	b，4
2	$\{b\}$	4	4 (1)	—	8 (2)	0 (0)	c，0
3	$\{b,c\}$	0	—	—	—	—	Q 为空，停止

作为对照，按总增益最大选择的规则在第 1 轮选 a（增益 10），剩余预算 3；第 2 轮付得起的有 b（增益 2）和 d（增益 1），c 的成本 4 超出剩余预算，选 b，剩余预算 0，停止，覆盖 12 格。图 9.3把两种规则的选择画在同一条预算上：昂贵的 a 一开始就占去一半以上的预算，之后只剩小元素可选。

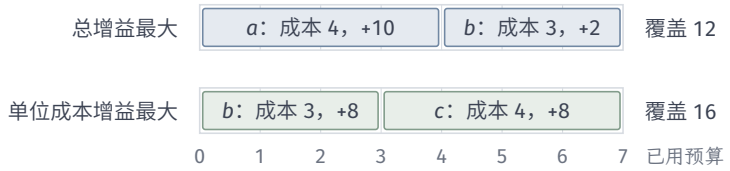


图 9.3　预算 $B = 7$ 下两种逐轮规则的选择结果（成本 $c_a = 4$、$c_b = 3$、$c_c = 4$、$c_d = 1$）。每条横条代表 7 个单位的预算，色块的宽度是所选元素的成本，块内写出选择时的增益。按总增益先选昂贵的 a，剩余预算只够再选增益 2 的 b，覆盖 12 格；按单位成本先选 b，再选 c，覆盖 16 格。

最佳单元素比较　单位成本规则也有自己的偏差：它可能先选几个便宜的小元素，把预算切碎，使一个昂贵但价值很高的元素再也放不进去。

一种简单的补救是在算法结束后增加一步：计算每个可行单元素的值 $F(\{v\})$（$c_v \le B$），输出 S 与最佳单元素中函数值较大的一个。这只需另外 n 次求值。在 $B = 7$ 的实例中，最佳单元素是 $\{a\}$，覆盖 10 格，小于 16，输出仍是 $\{b, c\}$。这一检查只是一个具体策略；本节没有给出它的分析，因此不附加任何近似比例。

9.2.3　算法分析

可行性　算法始终保持 $R = B - \sum_{v \in S} c_v \ge 0$：初始时 $R = B \ge 0$；每轮只从 Q 中选元素，而 Q 中元素的成本不超过 R，减去后 R 仍非负。所以输出总是预算可行的。

停止性　每轮加入一个新元素，至多执行 n 轮就会停止。停止时有两种原因：没有付得起的元素，或者付得起的元素都不再增加目标；对单调函数，后一种情形下再加入任何一个付得起的元素都不会提高函数值；若函数还是次模的，按上一节提前停止的论证，一次加入若干个付得起的元素也不会提高函数值。

复杂度　第 t 轮的候选元素不超过 $n - t + 1$ 个，所以求值次数不超过 $n + (n-1) + \cdots + 1 = n(n+1)/2$，再加开始时的一次 $F(\varnothing)$；附加最佳单元素比较时另加 n 次。上面的实例做了 $1 + 4 + 3 = 8$ 次求值。若一次求值的代价为 τ，总代价为 $O(n^2 \tau)$。

适用范围 单位成本规则在上面的实例中表现很好，但它本身没有任何固定比例的保证。

> **示例 9.2 比值规则的失效** 底集 $\{u, w\}$，预算 $B = 100$。u 的成本为 1、价值为 2，w 的成本为 100、价值为 100，目标取模函数 $F(S) = \sum_{v \in S}(\text{价值})$。各价值非负，所以它归一化、单调，也是次模的。
>
> 比值规则第 1 轮比较 $2/1 = 2$ 与 $100/100 = 1$，选 u；剩余预算 $99 < 100$，w 付不起，算法停止，得 $F(\{u\}) = 2$。最优选择是 $\{w\}$，$F = 100$。见图 9.4。
>
> 把 w 的成本、价值与预算同时换成任意大的 M，比值规则仍先选 u，得到 2 而最优值为 M，比例 $2/M$ 可以任意小。

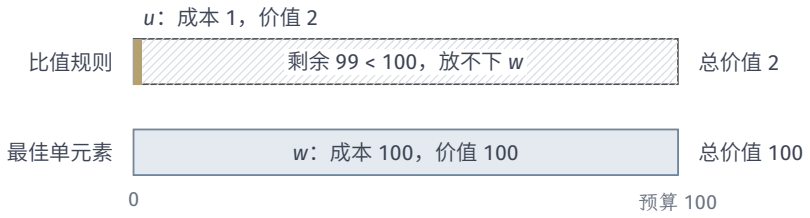


图 9.4 比值规则的失效例。横条长度代表预算 100。上行：比值规则先选单位价值较高的 u（黄色，成本 1，为便于观察略加宽），剩余 99 个单位（斜线）放不下成本 100 的 w，总价值 2。下行：最佳单元素 w 恰好用满预算，总价值 100。

这个例子的目标是模函数，从而归一化、单调且次模，满足上一节定理的全部函数条件。比值规则仍不能继承那里的保证，原因在于上一节差距论证的关键一步没有对应物。

那里最优集合的至多 k 个元素在每一轮都仍然可选，所以当前最好的元素至少补上差距的 $1/k$。预算约束下，选进一个便宜元素就可能让最优集合中的元素再也付不起：例中选了 u 以后，差距是 $100 - 2 = 98$，而最优集合唯一的元素 w 已经放不进剩余的 99 个单位，可选的元素一个也没有，差距一点也补不上。

最佳单元素比较能修复这个例子，但修复一个例子不构成一般保证；有严格保证的预算选择算法需要另外的枚举或修正机制及其证明，本书不展开。

结果检验 小实例可以枚举全部预算可行子集，核对总成本、函数值以及与最优值的差距：$B = 7$ 时比值规则的 $\{b, c\}$ 与枚举最优一致，总增益规则的 $\{a, b\}$ 差 4 格。

大实例无法枚举时，至少应报告剩余预算、最终目标值、最佳单元素的值和总求值次数；若逐轮规则的结果不如最佳单元素，往往就是出现了示例 9.2那样预算被小元素切碎的情形。

与基数约束贪心的比较　两种贪心的状态与更新几乎相同，差别只在选择依据换成单位成本的增益，停止条件换成剩余预算放不下有增益的元素。所有成本都取 1、预算取 k 时，单位成本规则除了增益全为零时提前停止以外，就是上一节的贪心。

对同样的归一化单调次模目标，基数约束下有 $1-(1-1/k)^k$ 的保证，预算约束下简单的比值规则却可以任意差：约束的形式本身就决定算法能否得到保证。

9.3 本章小结

本章回答了名额或经费有限时怎样选子集。两种贪心有相同的骨架：从空集出发，逐轮维护已选集合与当前值，相对于当前集合重算增益，加入一个元素。基数约束下比较的是边际增益本身，预算约束下比较的是单位成本的增益，并且只在付得起的元素中比较。两者都只需多项式次求值，分别至多 $nk+1$ 次和 $n(n+1)/2+1$ 次，总代价还要乘以每次求值的开销。

贪心规则本身对任何集合函数都能执行，但结果有没有保证取决于目标性质和约束。基数约束下最大化归一化、单调、次模的函数时，最优集合的至多 k 个元素在每一轮都仍然可选；由次模性，当前最好的元素至少补上剩余差距的 $1/k$，差距按几何级数缩小，贪心值不低于最优值的 $1-(1-1/k)^k$ 倍，从而不低于 $1-e^{-1}$ 倍。撤去次模性，贪心看不到只有同时选中才有价值的互补组合；撤去单调性，强制选满会降低目标。两种情形下比例保证都不再成立。

惰性贪心是同一规则的另一种实现。次模性使旧增益成为当前增益的上界，优先队列据此只重算排在最前的元素；在一致的平局规则下，它与普通贪心逐轮选出相同的元素，所以继承同样的保证。它的正确性只需要次模性，节省多少求值没有一般的保证，最坏情形与普通贪心相同。

预算约束改变了可行性，也改变了分析。单位成本规则在传感器实例中找到最优解，却在成本与价值分别为 $(1,2)$ 与 $(100,100)$ 的例子中可以任意差：选进便宜元素以后，最优集合的元素可能再也放不进剩余预算，差距论证中“最优集合的元素仍可选”这一步失效。最佳单元素比较能修复这个例子，但本章没有证明它的一般比例。使用任何一种选择方法之前，都要写明目标方向、约束形式、函数是否归一化、单调和次模。

下一章转向最小化方向，没有基数或预算约束，但要求目标由一元项和成对项组成；在相应的次模条件下，最小割给出精确解。

第 10 章

成对二值函数的最小化

给一张图的每个顶点贴上“前景”或“背景”两种标签：每个顶点各自有偏好，相邻顶点又希望标签一致，代价写成每个顶点的成本加上每条边的成本。这样的目标是一个二值配置的函数，n 个顶点有 2^n 种配置，逐一枚举很快不可行。本章要回答：这类只由一元项和成对项组成的二值目标，什么时候可以不枚举而精确求出最小值，怎样求，以及结果怎样核对。

上一章在约束下最大化集合函数，贪心只给出有比例保证的近似解，保证依赖单调、次模和约束形式。本章换一个方向：目标是最小化，没有基数或预算约束，但函数必须具有一元加成对的结构。第 10.1节给出问题，并说明逐点选择为什么不够。第 10.2节说明割容量能表示哪些成本，导出成对项的次模条件，并证明条件成立时每个配置的成本等于一个常数加上对应割的容量。

后两节把这一表示变成算法并加以检验。第 10.3节逐项合并一元成本、移出常数、构造非负容量网络，调用第 5章的最大流算法，再由割的两侧读出配置。第 10.4节用割表示与最大流最小割定理证明结果全局最优，给出复杂度和独立核对的办法，并说明非次模成对项、高阶项和附加约束为什么不能直接处理。本章用到第 8章的指示向量与次模不等式，以及第 5章的 s–t 割、最大流最小割定理和 Edmonds–Karp 算法。

10.1 问题与思路

给底集 V 的每个元素 i 一个二值变量 $x_i \in \{0,1\}$，$x_i = 1$ 表示选中 i，于是配置 $\boldsymbol{x} \in \{0,1\}^V$ 就是第 8.1.2小节的指示向量。元素之间的交互用无向图 $\mathcal{G} = (V, E)$ 记录：只有 $\{i, j\} \in E$ 的两个元素之间有成对项。这张交互图只记录哪些变量共同出现在成本项中；底集不必来自某张事先给定的图，交互图由目标的写法决定。目标是

$$\min_{\boldsymbol{x} \in \{0,1\}^V} J(\boldsymbol{x}), \qquad J(\boldsymbol{x}) = \sum_{i \in V} \phi_i(x_i) + \sum_{\{i,j\} \in E} \phi_{i,j}(x_i, x_j). \tag{10.1}$$

一元项ϕ_i 给出元素 i 两种状态各自的成本，只有两个数 $\phi_i(0), \phi_i(1)$；**成对项**$\phi_{i,j}$ 给出 i, j 四种联合状态的成本，例如两者取值不同时的不一致代价，或两者同时选中的配合收益（以负成本表示）。为了让 $\phi_{i,j}(a, b)$ 的含义确定，对每条边固定一个次序，约定 a 是 x_i、b 是 x_j 的值。

本章只考虑有限实值成本。输入是这些成本表，输出是一个使 J 最小的配置 $\hat{\boldsymbol{x}}$ 及其目标值。

通过 $S = \{i : x_i = 1\}$，J 也是 V 上的集合函数。但并不是每个集合函数都能写成式 (10.1)的形式：例如只在三个元素同时选中时才产生的成本，不能拆成两两之间的成对项。本章的方法只适用于已经具有这种一元加成对结构的目标。

成对项会改变各元素的最优取值。取两个变量，$\phi_1(0) = 0$、$\phi_1(1) = 1$，$\phi_2(0) = 2$、$\phi_2(1) = 0$，成对项为

$$\phi_{1,2}(0,0) = 0, \quad \phi_{1,2}(0,1) = 3, \quad \phi_{1,2}(1,0) = 5, \quad \phi_{1,2}(1,1) = 0.$$

图 10.1列出四种配置的成本。若只看一元项，元素 1 偏好 0，元素 2 偏好 1，逐点选择得到 $(0, 1)$，成本 $0 + 0 + 3 = 3$。但成对项惩罚两者取值不同，整体最优是 $(1, 1)$，成本 $1 + 0 + 0 = 1$：让元素 1 放弃自己的偏好只多付 1，却省下了 3 的不一致代价。

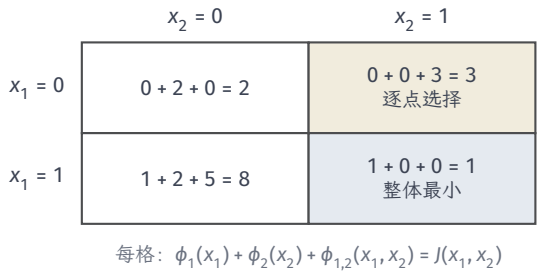


图 10.1　两变量成对二值函数的四种配置。行是 x_1，列是 x_2，每格依次写出 $\phi_1(x_1)$、$\phi_2(x_2)$、$\phi_{1,2}(x_1, x_2)$ 与总成本 J。逐点按一元项选择得到黄色格 $(0, 1)$，成本 3；整体最小是蓝色格 $(1, 1)$，成本 1。

两个变量可以枚举，n 个变量则有 2^n 种配置。避开枚举的办法是借助割：在顶点为 $V \cup \{s, t\}$ 的网络中，一个 s–t 割由 V 中哪些元素与 s 同侧决定；规定源侧取 0、汇侧取 1，割就成了配置。

如果能选非负的弧容量，使每个配置的割容量都等于 $J(\boldsymbol{x})$ 减去同一个常数，最小割对应的配置就是 J 的最小点，2^n 种配置的枚举变成一次最大流计算。困难在于割容量只能累加非负容量，而成本可以为负，成对项的四个值也未必能用弧表示。

10.2 次模条件与割表示

割容量只累加从源侧指向汇侧的弧。按“源侧取 0、汇侧取 1”的约定，每条弧都像一笔只在特定情形下收取的费用。弧 $s \to i$ 在 i 位于汇侧、即 $x_i = 1$ 时被割断，相当于对 $x_i = 1$ 收费；弧 $i \to t$ 在 $x_i = 0$ 时被割断，相当于对 $x_i = 0$ 收费；边 $\{i, j\}$ 上方向相反、容量相同的两条弧 $i \to j$ 与 $j \to i$，在 $x_i \neq x_j$ 时恰有一条从源侧指向汇侧，相当于对两者取值不同收费。容量非负，这三类费用也都必须非负。所以要做的是把 J 改写成一个常数加上这三类非负费用；常数与配置无关，不影响哪个配置最小。

一元项只有两个值，减去其中较小者就成为非负的单变量费用，总能改写。困难在成对项。

成对项有四个值，其中三个自由度可以吸收：一个常数，以及只依赖 x_i 或只依赖 x_j 的部分，后两者可以并入一元项。剩下的一个自由度衡量两个变量的相互作用，即两种取值不同的配置比两种取值相同的配置合计多付多少。它只能由“取值不同时收费”的两条弧表示，因而必须非负。

把这一要求写成四个值的关系，就是对每条边 $\{i, j\} \in E$ 成立

$$\phi_{i,j}(0,0) + \phi_{i,j}(1,1) \le \phi_{i,j}(0,1) + \phi_{i,j}(1,0). \tag{10.2}$$

它正是两元素底集 $\{i, j\}$ 上的次模不等式：把 $\phi_{i,j}$ 看成 $\{i, j\}$ 上的集合函数，在式 (8.5)中取 $X = \{i\}$、$Y = \{j\}$，左边 $F(X)+F(Y)$ 对应 $\phi_{i,j}(1,0)+\phi_{i,j}(0,1)$，右边 $F(X\cup Y)+F(X\cap Y)$ 对应 $\phi_{i,j}(1,1)+\phi_{i,j}(0,0)$。上面的例子中 $0+0 \le 3+5$，条件成立。

这个条件也让整个 J 成为次模的。把成对项看成 V 上的集合函数 $S \mapsto \phi_{i,j}(\mathbb{I}\{i \in S\}, \mathbb{I}\{j \in S\})$，它只读取 $S \cap \{i, j\}$；加入 i 或 j 以外的元素增益为 0，加入 i 时的增益只取决于 j 是否已选，式 (10.2)正是说 j 已选时这个增益不更大，所以它满足边际递减。一元项之和是模函数。由次模函数之和仍为次模，满足条件时 J 作为集合函数是次模的。

条件成立时，可吸收的三个自由度与相互作用能够显式分开。

引理 10.1　成对项的分解　设 $\phi_{i,j}$ 满足式 (10.2)，简记 $p_{a,b} = \phi_{i,j}(a, b)$，令

$$w_{i,j} = \frac{p_{0,1} + p_{1,0} - p_{0,0} - p_{1,1}}{2},$$

$$\alpha_i = p_{1,0} - p_{0,0} - w_{i,j}, \qquad \alpha_j = p_{0,1} - p_{0,0} - w_{i,j}.$$

则 $w_{i,j} \ge 0$，并且对四种配置都有

$$\phi_{i,j}(x_i, x_j) = p_{0,0} + \alpha_i x_i + \alpha_j x_j + w_{i,j}\,|x_i - x_j|. \tag{10.3}$$

证明　$w_{i,j} \geq 0$ 就是式 (10.2)移项除以 2。逐个代入四种配置核对右边：$(0,0)$ 时为 $p_{0,0}$；$(1,0)$ 时为 $p_{0,0}+\alpha_i+w_{i,j}=p_{1,0}$；$(0,1)$ 时为 $p_{0,0}+\alpha_j+w_{i,j}=p_{0,1}$；$(1,1)$ 时为 $p_{0,0}+\alpha_i+\alpha_j=p_{1,0}+p_{0,1}-p_{0,0}-2w_{i,j}=p_{1,1}$，最后一步代入 $w_{i,j}$ 的定义。 ■

在例子中，$w_{1,2}=(3+5-0-0)/2=4$，$\alpha_1=5-0-4=1$，$\alpha_2=3-0-4=-1$。

图 10.2把成对项的 2×2 成本表拆成四张表之和：常数表全为 0；$\alpha_1 x_1$ 只随行变化，$\alpha_2 x_2$ 只随列变化，它们稍后并入一元项；剩下的不一致惩罚只在两个非对角格上取 4，正好由容量为 4 的一对反向弧表示。

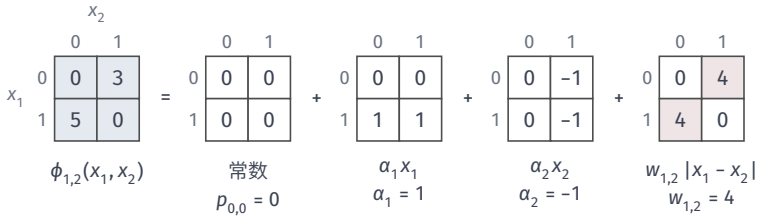


图 10.2　两变量例子中成对项的分解（引理 10.1）。每张 2×2 表的行是 x_1、列是 x_2。左边的成本表等于常数 $p_{0,0}$、只随 x_1 变化的 $\alpha_1 x_1$、只随 x_2 变化的 $\alpha_2 x_2$ 与不一致惩罚 $w_{1,2}|x_1-x_2|$ 之和，逐格相加可以核对。只有最后一张表（红色格）需要用弧 $1\to 2$ 与 $2\to 1$ 表示，次模条件保证它的值 $w_{1,2}$ 非负。

一元项的合并与移位　对每条边作上述分解，把常数 $\phi_{i,j}(0,0)$ 累加起来，把线性部分 $\alpha_i x_i$、$\alpha_j x_j$ 并入相应元素的一元项。记合并后的一元项为 ψ_i，即 $\psi_i(0)=\phi_i(0)$，$\psi_i(1)=\phi_i(1)$ 加上与 i 关联的各边分解出的 α_i。注意 α_i 随边而变，同一个 i 在不同的边上分解出的数一般不同，要逐边累加。

ψ_i 可能取负值，而容量必须非负，所以再令 $m_i=\min\{\psi_i(0),\psi_i(1)\}$，$\tilde{\psi}_i=\psi_i-m_i$，两种状态的成本都减去同一个数，$\tilde{\psi}_i$ 的两个值非负且至少一个为零。移出的数另行累计成常数

$$K=\sum_{\{i,j\}\in E}\phi_{i,j}(0,0)+\sum_{i\in V}m_i.$$

于是对每个配置都有

$$J(\boldsymbol{x})=K+\sum_{i\in V}\tilde{\psi}_i(x_i)+\sum_{\{i,j\}\in E}w_{i,j}\,|x_i-x_j|, \tag{10.4}$$

右边除常数 K 外各项都非负。

在例子中，$\psi_1=(0,\,1+1)=(0,2)$，$\psi_2=(2,\,0-1)=(2,-1)$；$m_1=0$，$m_2=-1$，所以 $\tilde{\psi}_1=(0,2)$，$\tilde{\psi}_2=(3,0)$，$K=0+0+(-1)=-1$。

网络构造 由式 (10.4)构造有向网络：顶点为 $V \cup \{s,t\}$；对每个 $i \in V$，建弧 $s \to i$，容量 $\tilde{\psi}_i(1)$，以及弧 $i \to t$，容量 $\tilde{\psi}_i(0)$；对每条边 $\{i,j\} \in E$，建方向相反的两条弧 $i \to j$ 与 $j \to i$，容量都是 $w_{i,j}$。容量为零的弧可以省略。配置 $\boldsymbol{x}$ 对应 s–t 割 $(S_{\boldsymbol{x}}, \bar{S}_{\boldsymbol{x}})$，其中 $S_{\boldsymbol{x}} = \{s\} \cup \{i : x_i = 0\}$；这是第 5.1节意义下的 s–t 割，割容量只累计从源侧跨向汇侧的弧。用最小割求解这类问题的方法常称为图割（graph cut）。

定理 10.2 成对二值函数的割表示 设式 (10.1)中的成本都是有限实数，且每个成对项满足式 (10.2)。则在上面构造的网络中，配置与 s–t 割一一对应，并且每个配置 $\boldsymbol{x}$ 对应的割容量等于 $J(\boldsymbol{x}) - K$。因此最小割对应的配置是 J 的最小点。

证明 任一 s–t 割的源侧都含 s、不含 t，由 V 中哪些元素在源侧唯一确定，所以割与配置一一对应。

割容量只累计从源侧指向汇侧的弧，逐类计算。若 $x_i = 0$，i 在源侧，弧 $i \to t$ 跨出，计入 $\tilde{\psi}_i(0)$，弧 $s \to i$ 两端都在源侧，不计入；若 $x_i = 1$，i 在汇侧，弧 $s \to i$ 跨出，计入 $\tilde{\psi}_i(1)$，弧 $i \to t$ 两端都在汇侧，不计入。两种情形合起来恰好计入 $\tilde{\psi}_i(x_i)$。

对边 $\{i,j\}$，若 $x_i = x_j$，两条弧都不跨越；若 $x_i \neq x_j$，两条反向弧中恰有一条从源侧指向汇侧，计入一次 $w_{i,j}$。合起来恰好计入 $w_{i,j}|x_i - x_j|$。

所以割容量等于式 (10.4)右边去掉 K，即 $J(\boldsymbol{x}) - K$。K 与配置无关，最小化割容量与最小化 J 是同一件事。 ■

在例子中，网络有弧 $s \to 1$（容量 2）、$2 \to t$（容量 3）、$1 \to 2$ 与 $2 \to 1$（容量都是 4），弧 $1 \to t$ 与 $s \to 2$ 容量为零。表 10.1逐个核对四种配置：割容量都比 J 大 1，即 $-K$。

配置 $\boldsymbol{x}$	源侧	跨出的正容量弧	割容量 / $J(\boldsymbol{x})$
$(0,0)$	$\{s,1,2\}$	$2 \to t$	3 / 2
$(0,1)$	$\{s,1\}$	$1 \to 2$	4 / 3
$(1,0)$	$\{s,2\}$	$s \to 1$，$2 \to t$，$2 \to 1$	9 / 8
$(1,1)$	$\{s\}$	$s \to 1$	2 / 1

表 10.1 两变量例子中四种配置的割容量与原目标值。源侧元素取 0、汇侧元素取 1；每行割容量都等于 $J(\boldsymbol{x}) + 1$，相差的常数是 $-K = 1$，所以割容量最小的 $(1,1)$ 也是 J 的最小点。

10.3　算法实现

交互图、一元成本表和成对成本表在运行前给定，运行中不变，不算状态。计算分三个阶段：构造阶段把目标改写成网络，求解阶段求最小割，读出阶段由割恢复配置并按原成本表复核。构造阶段维护三项状态：

- 合并一元项 ψ_i：每个元素两个数 $\psi_i(0),\psi_i(1)$，初值为 $\phi_i(0),\phi_i(1)$；
- 常数 K：初值为 0；
- 不一致惩罚 $w_{i,j}$：每条边一个数，处理该边时算出。

构造按三步进行：

- 逐边分解：对每条边 $\{i,j\}$ 先检查式 (10.2)，不成立就报告无法用本方法处理并停止；成立时按引理 10.1算出 $w_{i,j},\alpha_i,\alpha_j$，令 $K\leftarrow K+\phi_{i,j}(0,0)$，$\psi_i(1)\leftarrow\psi_i(1)+\alpha_i$，$\psi_j(1)\leftarrow\psi_j(1)+\alpha_j$。
- 逐点移位：全部边处理完以后，对每个 i 令 $m_i=\min\{\psi_i(0),\psi_i(1)\}$，$\tilde{\psi}_i=\psi_i-m_i$，$K\leftarrow K+m_i$。移位必须等所有边的线性部分都并入以后进行，否则 m_i 取的不是最终一元项的较小值。
- 建弧：对每个 i 建弧 $s\to i$（容量 $\tilde{\psi}_i(1)$）与 $i\to t$（容量 $\tilde{\psi}_i(0)$），对每条边建两条反向弧，容量都是 $w_{i,j}$；容量为零的弧省略。

求解阶段在建好的网络上运行第 5.3节的 Edmonds–Karp 算法。它维护流与残量网络，每次用广度优先遍历找一条弧数最少的增广路径，沿它推送瓶颈流量，直到 t 从 s 不可达。这里所有容量都已非负，满足该算法与最大流最小割定理的前提。

停止后进入读出阶段，做三件事：

- 读割：残量网络中从 s 可达的顶点集合 S^* 是一个最小割的源侧；
- 读配置：S^* 中的元素取 0，其余元素取 1，得到 $\hat{\boldsymbol{x}}$；最小值为最大流值 Φ 加上 K；
- 复核：按原始成本表直接计算 $J(\hat{\boldsymbol{x}})$，它应等于 $\Phi+K$。不相等说明分解、移位或建弧的某一步出了错。

算法 10.1　成对二值函数的图割最小化

输入：交互图 $\mathcal{G}=(V,E)$，一元成本表 ϕ_i，满足式 (10.2)的成对成本表 $\phi_{i,j}$。
输出：J 的一个最小点 $\hat{\boldsymbol{x}}$ 及最小值。

1. 令 $\psi_i=\phi_i$（$i\in V$），$K=0$。
2. 对每条边 $\{i,j\}$：若式 (10.2)不成立，报告无法处理并停止；否则按引理 10.1算出 $w_{i,j},\alpha_i,\alpha_j$；$K\leftarrow K+\phi_{i,j}(0,0)$，$\psi_i(1)\leftarrow\psi_i(1)+\alpha_i$，$\psi_j(1)\leftarrow\psi_j(1)+\alpha_j$。
3. 对每个 i：$m_i=\min\{\psi_i(0),\psi_i(1)\}$，$\tilde{\psi}_i=\psi_i-m_i$，$K\leftarrow K+m_i$。
4. 建网络：弧 $s\to i$ 容量 $\tilde{\psi}_i(1)$，弧 $i\to t$ 容量 $\tilde{\psi}_i(0)$，每条边 $\{i,j\}$ 建两条反向弧，容量都为 $w_{i,j}$。

5. 用 Edmonds–Karp 算法求最大流值 Φ；取残量网络中从 s 可达的顶点集合 S^{*}。
6. 令 $\hat{x}_i = 0$（$i \in S^{*}$），$\hat{x}_i = 1$（$i \notin S^{*}$）；按原成本表计算 $J(\hat{\boldsymbol{x}})$，核对它等于 $\Phi + K$。
7. 输出 $\hat{\boldsymbol{x}}$ 与最小值 $\Phi + K$。

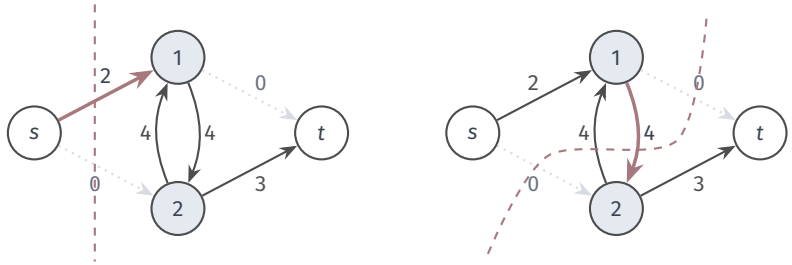
执行过程　在两变量例子上，构造阶段的状态依次变化如下。

- 初始：$\psi_1 = (0, 1)$，$\psi_2 = (2, 0)$，$K = 0$（括号内依次为取 0 与取 1 的值）。
- 分解边 $\{1, 2\}$：条件 $0 + 0 \le 3 + 5$ 成立，$w_{1,2} = 4$，$\alpha_1 = 1$，$\alpha_2 = -1$；常数 $\phi_{1,2}(0, 0) = 0$ 计入 K，于是 $\psi_1 = (0, 2)$，$\psi_2 = (2, -1)$。
- 移位：$m_1 = 0$、$m_2 = -1$，得到 $\tilde{\psi}_1 = (0, 2)$，$\tilde{\psi}_2 = (3, 0)$，$K = -1$。
- 建弧：$s \to 1$（容量 2）、$2 \to t$（容量 3）、$1 \to 2$ 与 $2 \to 1$（容量都是 4）；$1 \to t$ 与 $s \to 2$ 容量为零，省略。

求解阶段从零流出发，残量网络就是原网络。

- 第 1 次遍历：从 s 经 $s \to 1$ 到 1，经 $1 \to 2$ 到 2，经 $2 \to t$ 到 t，找到增广路径 $s \to 1 \to 2 \to t$，瓶颈为 $\min\{2, 4, 3\} = 2$。推送 2 个单位后，残量 $s \to 1$ 为 0，$1 \to 2$ 为 2，$2 \to t$ 为 1，反向残量 $1 \to s$、$2 \to 1$、$t \to 2$ 相应增加。
- 第 2 次遍历：$s \to 1$ 的残量为零，$s \to 2$ 本来不存在，s 没有任何正残量的出弧，可达集合 $S^{*} = \{s\}$，t 不可达，停止，流值 $\Phi = 2$。
- 读出：两个元素都在汇侧，$\hat{\boldsymbol{x}} = (1, 1)$，最小值 $\Phi + K = 2 - 1 = 1$。按原成本表复核，$J(1, 1) = \phi_1(1) + \phi_2(1) + \phi_{1,2}(1, 1) = 1 + 0 + 0 = 1$，两者相等。

结果与图 10.1中的枚举一致。图 10.3把这个最小割与配置 $(0, 1)$ 对应的割并排比较。



(a) 源侧 {s}，**x** = (1, 1)
割容量 2 = J(**x**) + 1

(b) 源侧 {s, 1}，**x** = (0, 1)
割容量 4 = J(**x**) + 1

图 10.3　两变量例子的 s–t 网络。弧旁数字是容量，灰色点线是容量为零的弧；红色虚线分开源侧与汇侧，红色粗弧是跨出割的正容量弧。源侧元素取 0、汇侧元素取 1。(a) 最小割，源侧 $\{s\}$，只割断 $s \to 1$，容量 2，对应 $(1, 1)$；(b) 源侧 $\{s, 1\}$，割断 $1 \to 2$，容量 4，对应逐点选择得到的 $(0, 1)$。两种情况的割容量都等于 $J(\boldsymbol{x}) + 1$。

10.4　算法分析

正确性　正确性由两部分合成。由定理 10.2，每个配置的 J 值等于对应割的容量加 K。由第 5章的最大流最小割定理（定理 5.8）及增广路径判据（定理 5.5），在非负容量的网络上，算法停止时残量网络中从 s 可达的集合给出一个容量等于最大流值 Φ 的割，而任何割的容量都不小于 Φ。两者合起来，$\hat{\boldsymbol{x}}$ 对应的割容量最小，所以 $\hat{\boldsymbol{x}}$ 是 J 的全局最小点，最小值为 $\Phi + K$。

结果检验　结果可以独立核对，不必相信算法实现本身。需要逐项检查四件事：

- 原始成本：按原成本表直接计算 $J(\hat{\boldsymbol{x}})$；
- 常数移位：重新累加各边的 $\phi_{i,j}(0,0)$ 与各点的 m_i，核对 K；并逐点核对 $\tilde{\psi}_i(0), \tilde{\psi}_i(1) \geq 0$、各边 $w_{i,j} \geq 0$；
- 流的可行性：每条弧上 $0 \leq$ 流量 $\leq$ 容量，每个元素顶点流入等于流出，并算出流值 Φ；
- 割容量：按跨出 S^* 的弧重新求和，核对它等于 Φ。

四项都通过时，由流值不超过割容量（引理 5.2），可行流的流值 Φ 是一切割容量的下界，从而 $\Phi + K$ 是一切配置成本的下界；而 $J(\hat{\boldsymbol{x}}) = \Phi + K$ 说明 $\hat{\boldsymbol{x}}$ 达到了这个下界。流与割在这里共同充当最优性证书。

复杂度　网络有 $n+2$ 个顶点，弧数 $a \leq 2n + 2m$，其中 $m = |E|$；分解、合并一元项、移位和建弧都只需扫描一遍元素与边，用时 $O(n+m)$，存储也是 $O(n+m)$。主要开销在最大流：按第 5章的分析，Edmonds–Karp 算法需要 $O(na^2)$ 次算术与比较操作，代入 $a = O(n+m)$ 即 $O(n(n+m)^2)$；换用第 5.4节的 Dinic 算法，界为 $O(n^2 a) = O(n^2(n+m))$。最后读出可达集合只需一次遍历，按原成本表复核 $J(\hat{\boldsymbol{x}})$ 用时 $O(n+m)$。

与枚举 2^n 种配置相比，这是多项式规模的计算。这里的界按顶点数与弧数计，把每次加减比较算作一步，与容量的数值无关。

适用范围　若某个成对项违反式 (10.2)，引理给出的 $w_{i,j}$ 为负：这个成对项不是在惩罚取值不同，而是在奖励取值不同，而弧只能收费，不能付奖励，网络中就会出现负容量的弧。第 5章的流值与割容量的关系建立在非负容量之上，负容量时不能直接使用。

例如奖励两个变量取值不同的成对项 $\phi_{i,j}(0,1) = \phi_{i,j}(1,0) = 0$、$\phi_{i,j}(0,0) = \phi_{i,j}(1,1) = 1$ 就违反条件：$1 + 1 > 0 + 0$，算出 $w_{i,j} = -1$。这时的最小化一般不能化为一次最小割。所以算法 10.1把条件写进了输入要求，构造阶段逐边检查它，不成立就报告失败，不会建出带负容量的网络。

一元项的符号则不构成限制：无论 ψ_i 取什么有限值，减去较小者后总是非负，移出的数进入常数 K。所以容量非负只需要两件事同时成立：成对项满足次模条件，以及移位在全部边的线性部分并入之后进行。

即使每个成对项都满足条件，附加约束也会破坏这一表示。割可以任意选择源侧，所以全部 2^n 个配置都对应某个割；“恰好选 k 个”这样的数量限制只允许其中一部分配置，而最小割不受这种限制，可能落在不允许的配置上。同样不能直接套用的还有：含三个及以上变量的高阶项，以及每个变量有三种以上取值的问题。它们需要另行建模与求解；一般次模函数的最小化另有专门的算法，超出本书范围。

与上一章的比较 本章与上一章都在子集上做选择，但条件和结论不同。上一章在基数或预算约束下最大化单调次模函数，算法逐轮维护已选集合，得到的是近似解，保证依赖单调性、次模性和约束的形式。本章最小化可以写成一元项加满足式 (10.2)的成对项的函数，没有额外约束，算法一次性建出网络再求最小割，得到的是精确解，依据是割表示，并附带可以独立核对的证书。

10.5 本章小结

本章回答了成对二值目标什么时候能不经枚举而精确最小化。目标由一元项与成对项组成，配置是 2^n 个零一向量；逐点按一元项选择会忽略成对项，一般得不到最小值。把配置看成 s–t 割的两侧（源侧取 0、汇侧取 1）后，割的每条弧都相当于一笔只在特定取值下收取的非负费用：$s \to i$ 对 $x_i = 1$ 收费，$i \to t$ 对 $x_i = 0$ 收费，一对反向弧对两者取值不同收费。

能否把目标写成这些费用之和，逐项看就清楚了。一元项减去较小值后总是非负，移出的数进入常数。成对项的四个值中，常数和只依赖单个变量的部分可以分别移出和并入一元项，剩下的相互作用 $w_{i,j}$ 必须是取值不同时的非负惩罚，这正是成对项的次模条件 $\phi_{i,j}(0,0) + \phi_{i,j}(1,1) \le \phi_{i,j}(0,1) + \phi_{i,j}(1,0)$。条件成立时 J 作为集合函数是次模的，并且每个配置的成本等于常数 K 加对应割的容量。

算法因此分三步：逐边分解并合并一元项，全部边处理完后逐点移位得到非负容量，再求最大流并读出残量网络中从源点可达的一侧。最大流最小割定理保证读出的配置全局最优，最小值为流值加 K。核对原始成本、常数移位、流的可行性和割容量，就能独立确认结果，不必相信算法实现。计算量是一次最大流，与枚举 2^n 种配置相比是多项式规模。

这一方法的边界也由割表示决定。成对项违反次模条件时会出现负容量，不能直接求最小割；附加“恰好选 k 个”等配置约束时，最小割可能落在不允许的配置上；含三个以上变量的高阶项或多值变量也不在这一表

示之内。与上一章相比，本章得到的是精确解而不是近似解，代价是对目标结构提出了更强的要求。

许多问题的变量有更多取值，局部成本读取三个变量，或者禁止某些组合；但整体函数常常仍是只读取少数变量的局部项之和。第 4部分研究怎样借助记录读取关系的依赖图，把对全部配置的枚举化为沿图传递的小规模表格计算。

第四部分

图上的动态规划

Dynamic Programming on Graphs

阅读指引

许多有限状态问题的整体成本是若干局部项之和，每一项只读取少数几个变量：变量可以有多种取值，一项成本可以同时读取三个变量，某些组合可以直接被禁止。逐个检查全部联合取值的开销随变量数指数增长。本部分利用“每项只读少数变量”这一结构精确求出最小成本和最优配置，并顺带回答保留部分变量时的条件最小成本、可行配置的个数和有限加权总和。

贯穿三章的是同一个动作：固定一组接口变量，把只在一侧出现的变量汇总成一张以接口取值为索引的表。第 11章写清局部函数、作用域和几种输出，在链和树上建立接口递推、消息与回溯，再用一个四环说明依赖图含环时接口必须同时保留多个变量。第 12章只处理结构：按消元次序模拟消元得到的消元簇，与树分解的袋和接口是同一结构的两种描述，最小诱导宽度等于树宽；构造策略包括贪心选序、弦图的完美消元序和借最小生成树构造团树。第 13章在构建好的结构上统一完成最小成本、带保留变量的最小成本、计数和求和的计算，证明正确性，并分析时间与存储。

阅读时要分清三种对象：局部函数表记录数值，原始图记录哪些变量共同出现，袋树记录计算怎样组织；原始图的边与袋树的边含义不同，共享变量可以出现在多个袋中，同一个局部项却只能计一次。还要分清构建与计算：构建只用作用域，计算才用数值；某个次序或分解的宽度只是树宽的上界。最后，取最小与求和各自可以换序，二者混用时不能交换。

第 11 章

局部函数与树上动态规划

第 10章把每个顶点的“选或不选”写成二值变量，在成对成本满足次模条件时用最小割求出整体最优。许多问题不满足那里的条件：变量可以有三种以上的取值，一项成本可以同时读取三个变量，还可能直接禁止某些组合。不过这些问题常常保留另一种结构：整体成本是许多局部项之和，每一项只读取少数几个变量。

本章要回答三个问题。第一，这类问题的输入和输出究竟是什么，哪些结构信息可以用图记录。第二，依赖结构是一条链或一棵树时，怎样利用“每项只读少数变量”避免逐个检查全部联合取值，并且不但求出最小成本，还恢复达到它的取值。第三，依赖结构含环时，链和树上的做法缺了什么。

第 11.1节固定对象：有限状态的变量、局部函数及其作用域、记录读取关系的几种图，以及最小成本、保留变量的成本表、计数和加权求和这几种输出。第 11.2节在链和树上建立本部分反复使用的基本动作：固定一个接口变量，把只在一侧出现的变量汇总成一张以接口取值为索引的表，再沿保存的选择回溯。第 11.3节用一个四环说明，环上的接口必须同时保留多个变量，由此提出后两章要完成的构建与计算任务。

树上的计算要用到第 1.4节的选根、父点与子树，以及第 2章由深度优先遍历得到的后序。本部分记变量个数为 n，每个变量至多有 q 个状态，局部函数的个数为 $|A|$；统计计算量时数的是表项的加法、比较等基本运算，读取稠密表的一项按常数时间计。

11.1 局部函数与计算目标

本部分的问题都从一组变量的联合取值出发：每个变量从有限个状态中取一个，整组取值按某种规则得到成本，或者被判为不允许。本节先写清这组取值和它的成本由什么组成，再说明怎样用图记录“哪些变量出现在同一条规则里”，最后列出要计算的几种输出。

需要始终分开的是两类信息。图只记录读取关系，即哪条规则读取哪些变量；局部函数表才记录具体数值。二者共同确定一个计算问题，不能互相替代。

11.1.1　有限状态与配置

考虑三台相邻的设备，每台可以关（记为 0）或开（记为 1）。用 x_1, x_2, x_3 分别表示三台设备的状态，一次联合取值 (x_1, x_2, x_3) 就确定了全部设备的状态。三台设备共有 $2^3 = 8$ 种联合取值：

$$(0,0,0),\ (0,0,1),\ (0,1,0),\ (0,1,1),\ (1,0,0),\ (1,0,1),\ (1,1,0),\ (1,1,1).$$

一般地，设变量的下标集合为 $V = \{1, \dots, n\}$，给每个变量 x_i 指定一个非空有限的**状态集合** $\mathcal{X}_i$。全部变量的一次联合取值 $\boldsymbol{x} = (x_1, \dots, x_n)$ 称为一个**配置**（configuration），全部配置组成笛卡尔积

$$\mathcal{X} = \prod_{i \in V} \mathcal{X}_i.$$

对下标子集 $S \subseteq V$，记 $\boldsymbol{x}_S = (x_i)_{i \in S}$ 为配置在 S 上的部分，称为 S 上的部分配置，即只保留 S 中下标的那些分量。后面固定一组变量的取值、或者只为一部分变量列表时，都用这个记号。

由乘法原理，配置的数目是 $|\mathcal{X}| = \prod_{i \in V} |\mathcal{X}_i|$，乘法原理可参考卷一第 2 章“计数”。若每个变量至多有 q 个状态，配置数可以达到 q^n。三台设备时只有 8 种，三十个二值变量就已超过十亿种。

本部分研究的是给整组配置赋值的函数 $F : \mathcal{X} \to \mathbb{R} \cup \{+\infty\}$，它的一次求值要读入全部变量的状态。它与图上的顶点函数不同：顶点函数给每个顶点单独赋一个数，每个值只对应一个位置，这类函数可参考卷九第 9 章“图上函数与图算子”。

二值情形下，配置 $\boldsymbol{x} \in \{0,1\}^n$ 与子集 $\{i : x_i = 1\}$ 一一对应，所以第 8.1节的集合函数也可以看成二值配置的函数。一般有限状态则允许每个变量有多种标签，例如三档取值 $\{0,1,2\}$。这种对应只是换了一种表示，并不意味着任意整体函数都容易求最小值；能否高效计算，取决于下面的局部结构。

11.1.2 局部函数与作用域

回到三台设备。假设相邻两台设备状态不同时会产生 1 个单位的切换成本，状态相同时没有成本。设备 1, 2 之间的成本只取决于 x_1, x_2，设备 2, 3 之间的成本只取决于 x_2, x_3，于是总成本可以写成

$$F(x_1, x_2, x_3) = \phi_{1,2}(x_1, x_2) + \phi_{2,3}(x_2, x_3), \qquad \phi_{i,j}(a, b) = \begin{cases} 0, & a = b, \\ 1, & a \neq b. \end{cases}$$

图 11.1左侧列出了全部 8 种配置及其总成本：$(0,0,0)$ 和 $(1,1,1)$ 成本为 0，$(0,1,0)$ 和 $(1,0,1)$ 成本为 2，其余为 1。右侧显示，计算任何一个配置的总成本时，$\phi_{1,2}$ 从不需要看 x_3，$\phi_{2,3}$ 也从不需要看 x_1。

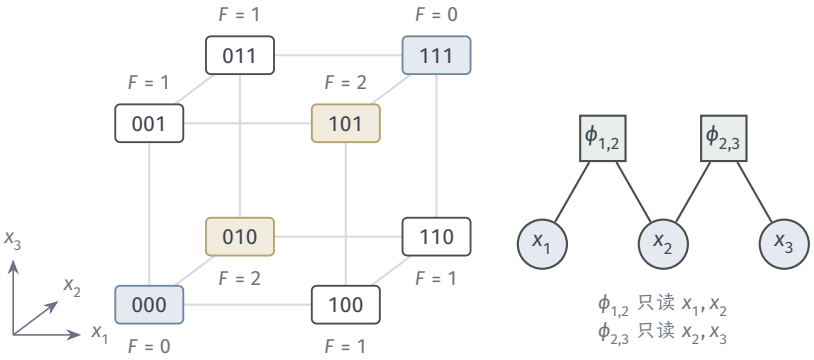


图 11.1　配置空间与局部作用域。左：三个二值变量的 8 种配置排在立方体的顶点上，顶点旁的数是总成本 $F = \phi_{1,2} + \phi_{2,3}$，其中相邻变量取值不同时成本为 1；蓝色顶点成本为 0，黄色顶点成本为 2。右：圆点表示变量，方块表示局部函数，连线表示该函数读取的变量。每个局部函数只读取两个变量，但它们的和仍然对全部 8 种配置给出成本。

把这个例子一般化。设有一组局部项，用下标 $a \in A$ 区分，每一项 ϕ_a 只读取下标集合 $S_a \subseteq V$ 中的变量，总成本为

$$F(\boldsymbol{x}) = \sum_{a \in A} \phi_a(\boldsymbol{x}_{S_a}), \qquad \boldsymbol{x} \in \mathcal{X}. \tag{11.1}$$

这里的 ϕ_a 称为**局部函数**（local function），也常称为**因子**（factor）；它读取的下标集合 S_a 称为 ϕ_a 的**作用域**（scope）。

“局部”限定的是读取范围，不是精度：ϕ_a 精确地给出 $\boldsymbol{x}_{S_a}$ 每种取值的成本，并不是对某个整体函数的近似。作用域也不限于两个变量，三元、四元的局部函数同样允许；一元项 $\phi_i(x_i)$ 是作用域为 $\{i\}$ 的特例，作用域为空集的项是一个常数。

局部函数的取值允许为 $+\infty$，用来表示禁止的组合。例如设备 1, 2 不能同时打开时，可以令 $\phi_{1,2}(1,1) = +\infty$。本部分固定如下运算约定：$+\infty$ 加

任何有限数或 $+\infty$ 仍得 $+\infty$；比较大小时 $+\infty$ 大于每个实数，因而有限集合中只要有一个有限数，最小值就是有限的。这样，只要某一项禁止，整个配置的总成本就是 $+\infty$。

总成本有限的配置称为**可行配置**。同一个式 (11.1)因此既表达偏好，也表达约束。若最小值为 $+\infty$，说明每个配置都被某项禁止，所以求最小值同时回答了是否存在可行配置。

式 (11.1)的分解由问题给出，不是自动成立的性质。任意函数 F 都可以写成作用域为 V 的单个“局部”函数，但这时局部函数表就有 $|\mathcal{X}|$ 项，与直接枚举没有区别。本部分始终假定分解已经给定，并且各作用域都较小；从一个任意给定的整体函数中寻找小作用域分解，是另一个问题。

11.1.3 依赖图与因子图

要利用“某一项不读某个变量”，首先要知道哪些变量出现在同一个局部函数里。这是一种纯粹的结构信息，可以用几种图来记录，它们保留和丢失的内容各不相同。

只有成对局部项时，可以以变量为顶点，两个变量出现在同一个成对项中就连一条边。三台设备的例子得到路径 $1-2-3$。这样的图记录的是函数的依赖关系，不是变量的取值本身；称“依赖结构是一条链”或“是一棵树”，指的就是这个依赖图是路径或树。

三元及更高阶的局部项无法用一条边表示。这时把变量和局部函数分成两类节点：每个变量一个圆形节点，每个局部函数一个方形节点；变量属于某个局部函数的作用域时，在两者之间连边。这样得到的二部图称为**因子图**（factor graph）。因子图保留了每个局部函数的身份：两个不同的局部函数即使作用域相同，也是两个不同的方形节点。图 11.1右侧就是三台设备例子的因子图。

原始图与超图　看一个含三元项的例子，后文称为**三元作用域例子**：四个二值变量 $x_1,\dots,x_4$，两个局部函数 $\phi_A(x_1,x_2,x_3)$ 与 $\phi_B(x_2,x_3,x_4)$，作用域为 $S_A=\{1,2,3\}$ 和 $S_B=\{2,3,4\}$。它们的数值在下一小节给出，这里只看结构。

除因子图外，还有两种常用的结构表示。第一种不保留函数节点，把每个作用域内的变量两两连边，得到的图称为**原始关系图**（primal graph），简称原始图。这里原始图有边 $\{1,2\},\{1,3\},\{2,3\},\{2,4\},\{3,4\}$，唯独缺少 $\{1,4\}$，因为变量 1 和 4 从不出现在同一个局部函数中。每个作用域在原始图中都是两两相邻的顶点组。

第二种允许一条“边”包含任意多个顶点：每个作用域记为一条**超边**（hyperedge），由此得到**超图**（hypergraph）。这里超图有两条超边 $\{1,2,3\}$ 和 $\{2,3,4\}$。图 11.2并排画出了三种表示。

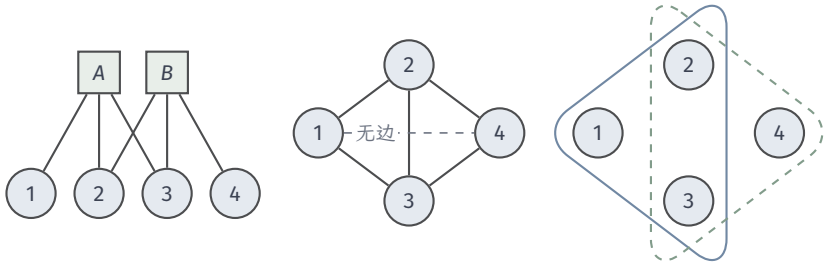


图 11.2　作用域 $\{1,2,3\}$ 与 $\{2,3,4\}$ 的三种结构表示。左：因子图，方块 A、B 是两个局部函数，连线表示读取关系。中：原始关系图，把每个作用域内的变量两两相连，只缺边 $\{1,4\}$。右：超图，蓝色实线与绿色虚线围出的区域各是一条超边。若把两个三元函数换成五条边上的五个成对函数，原始图完全相同，而因子图和超图都会改变。

原始图丢失的信息　原始图丢失了作用域的分组：若把 ϕ_A,ϕ_B 换成定义在五条边上的五个成对局部函数，原始图不变，因子图和超图却不同。

更重要的是，相同的两两连接不能恢复函数表。设另有两个作用域同为 $\{1,2,3\}$ 与 $\{2,3,4\}$ 的局部函数：$\chi_A(x_1,x_2,x_3)$ 在 $x_1+x_2+x_3$ 为偶数时取 0、为奇数时取 $+\infty$，χ_B 对 $x_2+x_3+x_4$ 作同样的规定。χ_A 允许的取值恰为 $(0,0,0),(0,1,1),(1,0,1),(1,1,0)$，其中任意两个变量的四种取值组合都出现过。

因此，若只保留“每对变量允许哪些取值”的成对规则，这些规则什么也不禁止，16 种配置都会被放行。而原问题中 x_2,x_3 确定后，x_1 和 x_4 都被奇偶条件唯一确定，只有 4 种可行配置。三元规则不是成对规则的叠加。

超图保住了作用域的分组，但和原始图一样不保存局部函数的数值；因子图保住了函数身份，也仍需另外存储每个函数的取值表。第 12章构建计算结构时以原始图为依据，因为原始图上的边已经足以说明哪些变量必须同时出现，并且每个作用域都能整体放进构建出的某个计算单元（第 12.3.1小节）。计算时则始终使用完整的局部函数，不把它们拆成成对规则。

11.1.4 计算任务与输出

本部分的输入由两部分组成：每个变量的有限状态集合 $\mathcal{X}_i$；一组局部函数 ϕ_a，每个以作用域 S_a 和一张取值表给出。取值表对 $\boldsymbol{x}_{S_a}$ 的每种取值列

出一个属于 $\mathbb{R}\cup\{+\infty\}$ 的数，共 $\prod_{i\in S_a}|\mathcal{X}_i|$ 项。在这同一份输入上，可以提出几种不同的计算任务。

最基本的任务是最小成本：

1. 求最小成本 $F^\star=\min_{\boldsymbol{x}\in\mathcal{X}}F(\boldsymbol{x})$；
2. 若 $F^\star<+\infty$，给出一个最优配置 $\boldsymbol{x}^\star$，即满足 $F(\boldsymbol{x}^\star)=F^\star$ 的配置；若 $F^\star=+\infty$，报告不存在可行配置。

有时不想把全部变量都优化掉，而要保留一组变量 $R\subseteq V$，对它们的每种取值给出其余变量取最优时的成本，即一张以 $\boldsymbol{x}_R$ 为索引的表

$$F_R(\boldsymbol{x}_R)=\min_{\boldsymbol{x}_{V\setminus R}}F(\boldsymbol{x}_R,\boldsymbol{x}_{V\setminus R}).$$

这张表回答“若把这几个变量固定成某组值，最好能做到多少”。它的输出本身有 $\prod_{i\in R}|\mathcal{X}_i|$ 项，$R=\varnothing$ 时退化为单个数 $F^\star$。

另外两种任务不再挑选一个配置，而是对全部配置汇总。第一种是计数：可行配置有多少个。第二种是加权求和：给每个局部函数一张非负权重表，把配置的权重定义为各局部权重之积，求全部配置的权重总和。这两种任务的组合方式（相乘）和汇总方式（求和）都与最小成本不同，第 13.4节统一展开。

三元作用域例子的数值　三元作用域例子的总成本为 $F=\phi_A+\phi_B$，两张表由表 11.1给出，其中 $+\infty$ 表示禁止的组合。

表 11.1　三元作用域例子的两个局部函数。第一列的三位二进制串对 ϕ_A 表示 (x_1,x_2,x_3)，对 ϕ_B 表示 (x_2,x_3,x_4)。

取值	000	001	010	011	100	101	110	111
$\phi_A(x_1,x_2,x_3)$	1	0	2	$+\infty$	0	2	1	2
$\phi_B(x_2,x_3,x_4)$	2	1	0	$+\infty$	$+\infty$	0	1	2

最直接的做法是按字典序逐个检查全部配置，保留目前最小的成本 $\hat{F}$（初始为 $+\infty$）及其配置，遇到更小的成本就替换，全部检查完后停止。在这个例子上，枚举 16 个配置得 $F^\star=0$，唯一的最优配置为 $\boldsymbol{x}^\star=(0,0,1,0)$，可行配置共 10 个；后面用这一例子演示消元时，都用这一结果核对。

枚举总是正确，但要读 $|\mathcal{X}|\cdot|A|$ 次表，各变量至多 q 个状态时可达 $|A|\,q^n$。它把每个配置当作整体，没有利用“某一项不读某个变量”这一信息。下一节从链和树开始利用这一信息。

11.2 链与树上的接口递推

依赖图没有环、局部项至多读取两个变量时，可以把变量分成只通过一个变量相连的两侧。本节先在小例子上看清“固定这个变量、分别处理两侧”为什么可行，以及要保留什么；再把这一动作在链上从一端推到另一端，在树上从叶推到根，每一步的接口都是单个变量。带环的情形留到下一节。

11.2.1 固定接口后的子问题

接口变量 回到第 11.1.2小节的三台设备。若先把 x_2 固定为某个值，$\phi_{1,2}$ 只剩 x_1 可变，$\phi_{2,3}$ 只剩 x_3 可变，两项不再共享任何变量，于是 x_1 和 x_3 可以分别选择：

$$\min_{x_1,x_2,x_3} F = \min_{x_2}\Big[\min_{x_1}\phi_{1,2}(x_1,x_2) + \min_{x_3}\phi_{2,3}(x_2,x_3)\Big].$$

对 x_2 的每个取值，两个内层最小值各只比较两个数。

这里 x_2 起着**接口**（interface）的作用：它把其余变量分成两侧，一侧的选择只通过 x_2 影响另一侧。好比两个房间之间只有一扇门，只要知道门开着还是关着，每个房间里怎样安排最好就可以各自决定。于是对接口的每种取值，先把一侧最好的成本算出来，记成一张以接口取值为索引的表，另一侧只需查表。

本部分反复使用的就是这个动作：固定一组接口变量，把只在一侧出现的变量汇总掉，留下一张以接口取值为索引的表。链和树上，接口可以沿依赖结构依次找到；一般依赖结构上接口要包含多个变量，怎样找到它们是第 12章的问题。

成对成本的模型 本节只考虑一元项和成对项，并假定成对项组成的依赖图没有环。这时总成本为

$$F(\boldsymbol{x}) = \sum_{i\in V}\phi_i(x_i) + \sum_{\{u,v\}\in E}\phi_{u,v}(x_u,x_v), \tag{11.2}$$

其中 $\mathcal{G}=(V,E)$ 是依赖图，$\phi_{u,v}(x_u,x_v)$ 与 $\phi_{v,u}(x_v,x_u)$ 指同一个局部函数。没有一元项的变量可以取 $\phi_i\equiv 0$。输入是各变量的状态集合、一元项表和每条边上的成对项表；输出是最小成本 $F^\star$ 和一个最优配置，$F^\star=+\infty$ 时报告不可行。

三变量链　先看三个变量的链 $1-2-3$，每个变量取三档 $\{0,1,2\}$，总成本 $F=\phi_{1,2}(x_1,x_2)+\phi_{2,3}(x_2,x_3)$。两张局部函数表按行列给出：

$$[\phi_{1,2}(x_1,x_2)]=\begin{bmatrix}4&1&3\\2&3&5\\3&4&2\end{bmatrix},\qquad [\phi_{2,3}(x_2,x_3)]=\begin{bmatrix}1&3&2\\4&3&5\\2&2&4\end{bmatrix},$$

左表的行对应 $x_1=0,1,2$、列对应 $x_2=0,1,2$，右表的行对应 x_2、列对应 x_3。直接枚举要计算 $3^3=27$ 个配置的成本。

x_1 只出现在 $\phi_{1,2}$ 中。若先把 x_2 固定为某个值，x_1 的选择就不再影响 $\phi_{2,3}$，只需在左表的这一列中取最小。对 $x_2=0,1,2$ 分别取列最小值，得到一张只依赖 x_2 的表

$$m_{1\to 2}(x_2)=\min_{x_1}\phi_{1,2}(x_1,x_2)=(2,\ 1,\ 2),$$

达到最小值的 x_1 依次为 $1,0,2$。这张表概括了链左端的全部信息：无论右侧怎样取值，只要 x_2 给定，左端最好的做法和它的成本都已确定。

剩下只需在 x_2,x_3 上求

$$\min_{x_2,x_3}\bigl[m_{1\to 2}(x_2)+\phi_{2,3}(x_2,x_3)\bigr].$$

对 $x_2=0,1,2$，括号内对 x_3 的最小值依次为 $2+1=3$、$1+3=4$、$2+2=4$，所以最优成本为 3，在 $x_2=0,x_3=0$ 时达到，再查表得 $x_1=1$。整个过程计算了 $9+9=18$ 个候选值，而不是 27 个配置；链越长，节省越多。

为什么要保留整张表　左端交给右侧的必须是一张以接口取值为索引的表，而不只是左端的一个最优值。左表的最小值是 1，在 $(x_1,x_2)=(0,1)$ 处达到；若左端只报告“我最少付 1，此时 $x_2=1$”，右侧只能在 $x_2=1$ 下选 x_3，得到 $1+\min(4,3,5)=4$。而整体最优配置 $(1,0,0)$ 中左端付的是 $\phi_{1,2}(1,0)=2$，比左端自己的最优值大，却让右端只付 1。左端事先无法知道右端偏好哪个 x_2，所以要对接口的每个取值都给出条件最小成本，把选择 x_2 的决定留到两侧的成本都已知之后。

最小值的三条规则　这样分步的依据是有限集合上最小值的三条规则。设 y,z 分别在有限集合中取值，h、g 取值于 $\mathbb{R}\cup\{+\infty\}$，$c$ 与 y 无关，则

$$\min_{y,z}h(y,z)=\min_{z}\min_{y}h(y,z),\qquad \min_{y}\bigl[g(y)+c\bigr]=\min_{y}g(y)+c,\quad (11.3)$$

并且对取值于互不相交变量组 $y_1,\dots,y_k$ 的函数，$\min_{y_1,\dots,y_k}\sum_r g_r(y_r)=\sum_r\min_{y_r}g_r(y_r)$。

第一条成立，因为两边都是同一个有限数集中的最小元。第二条成立，因为加上同一个 c 不改变比较结果。第三条成立，因为各 y_r 可以分别取到各自的最小点，而任何选择都不会使某一项小于它的最小值。c 或某项为 $+\infty$ 时，按第 11.1.2小节的运算约定，三条规则依然成立。

在三变量链上，$\phi_{2,3}$ 对 x_1 而言是固定加项，由第一、二条，$\min_{x_1}$ 可以移到 $\phi_{1,2}$ 上。

11.2.2 链上的递推与回溯

11.2.2.1 问题与思路

现在设链为 $1-2-\cdots-n$，总成本含一元项 $\phi_k(x_k)$ 和相邻成对项 $\phi_{k-1,k}(x_{k-1},x_k)$。三变量链上的汇总可以从左端开始一步一步向右推进，每一步把接口右移一个变量。

推进到第 k 个变量时，左边的 $x_1,\dots,x_{k-1}$ 已经汇总掉，留下一张以 x_k 为索引的表 $D_k(x_k)$：x_k 取某个状态时，前 k 个变量的局部项

$$\sum_{i=1}^{k}\phi_i(x_i)+\sum_{i=2}^{k}\phi_{i-1,i}(x_{i-1},x_i)$$

对 $x_1,\dots,x_{k-1}$ 求最小值的结果，称为前 k 个变量的最小部分成本。这里 x_k 是已处理部分与其余部分之间的接口，D_k 就是上一小节所说的条件最小成本表。

只有第一个变量时没有需要汇总的变量，$D_1(x_1)=\phi_1(x_1)$。从 D_{k-1} 得到 D_k 只需再汇总一个变量 x_{k-1}：对 x_k 的每个状态，把前 $k-1$ 个变量的最小部分成本加上连接两者的成对项，对 x_{k-1} 取最小值，再加上 x_k 自己的一元项，即

$$\begin{aligned} D_1(x_1)&=\phi_1(x_1),\\ D_k(x_k)&=\phi_k(x_k)+\min_{x_{k-1}\in\mathcal{X}_{k-1}}\left[D_{k-1}(x_{k-1})+\phi_{k-1,k}(x_{k-1},x_k)\right],\quad k=2,\dots,n. \end{aligned}\tag{11.4}$$

整条链的最小成本是 $\min_{x_n}D_n(x_n)$。这种把问题按顺序拆开、为每个子问题保存最优值表并由前一张表算出后一张表的做法称为**动态规划**（dynamic programming）。

最小值只告诉我们最小成本是多少，不直接给出达到它的配置。取最小值时，“是哪一个 x_{k-1} 达到最小”被丢掉了；若只保存 D_k，事后只能重新枚举。所以每算一个 $D_k(x_k)$，都顺手记下达到最小值的 x_{k-1}，组成**回溯表** $p_k(x_k)$。

算完以后从末端出发：x_n 取使 D_n 最小的状态，再由 $p_n(x_n)$ 读出 x_{n-1}，逐个向左读到 x_1。并列最优时需要事先固定选择规则，本节约定取编号最小的状态。

11.2.2.2 算法实现

链的次序、各变量的状态集合和局部函数表都由输入给定，运行中不变。算法维护三项状态：

- 最小部分成本表 D_k，$k = 1, \dots, n$：每张以 x_k 为索引，至多 q 项；初始只有 $D_1 = \phi_1$；
- 回溯表 p_k，$k = 2, \dots, n$：$p_k(x_k)$ 是达到 $D_k(x_k)$ 时 x_{k-1} 的取值；初始为空；
- 轮次 k：已经算出的最后一张表的下标，初值为 1。

此后每一轮令 $k \leftarrow k + 1$，对 x_k 的每个状态 s 做两件事：

- 列候选：对 x_{k-1} 的每个状态 r，计算 $D_{k-1}(r) + \phi_{k-1,k}(r, s)$，即式 (11.4)方括号中的值；
- 取最小：令 $D_k(s)$ 等于一元项 $\phi_k(s)$ 加上最小的候选值，$p_k(s)$ 为达到它的 r，并列时取编号最小者。

第 n 轮之后递推停止。若 $\min_{x_n} D_n(x_n) = +\infty$，报告不可行；否则按下面两步恢复配置：

- 定末端：x_n 取使 D_n 最小的状态，并列取编号最小者；
- 逆向读取：对 $k = n, \dots, 2$，令 $x_{k-1} = p_k(x_k)$。

算法 11.1　链上的最小成本递推

输入：链上各变量的有限状态集合、一元项表和相邻成对项表。输出：最小成本和一个最优配置。

1. 令 $D_1(x_1) = \phi_1(x_1)$。
2. 对 $k = 2, \dots, n$ 和每个 x_k，计算式 (11.4)方括号中的全部候选值，取最小值记为 $D_k(x_k)$，并把达到最小值的 x_{k-1}（并列时取编号最小者）记为 $p_k(x_k)$。
3. 若 $\min_{x_n} D_n(x_n) = +\infty$，报告不可行并停止。否则取使 D_n 最小的 x_n（并列取编号最小者）；再对 $k = n, \dots, 2$ 令 $x_{k-1} = p_k(x_k)$。

执行过程　取 $n = 4$ 的二值链，一元项为 $\phi_1 = (0, 3)$、$\phi_2 = (2, 0)$、$\phi_3 = (3, 0)$、$\phi_4 = (1, 2)$，括号内依次是取 0 和取 1 的成本；相邻成对项在两端取值相同时为 0，不同时为 2。表 11.2列出各轮的候选与结果，逐轮看：

- 第 1 轮：$D_1 = \phi_1 = (0, 3)$。

- 第 2 轮：$x_2=0$ 的候选为 $D_1(0)+0=0$ 与 $D_1(1)+2=5$，加一元项 2 得 $D_2(0)=2$；$x_2=1$ 的候选为 $D_1(0)+2=2$ 与 $D_1(1)+0=3$，加一元项 0 得 $D_2(1)=2$。两处都由 $x_1=0$ 达到，$p_2=(0,0)$。
- 第 3 轮：$x_3=0$ 的候选为 $2,4$，加 3 得 5；$x_3=1$ 的候选为 $4,2$，加 0 得 2。$D_3=(5,2)$，$p_3=(0,1)$。
- 第 4 轮：$x_4=0$ 的候选为 $5,4$，加 1 得 5；$x_4=1$ 的候选为 $7,2$，加 2 得 4。$D_4=(5,4)$，$p_4=(1,1)$。

第 4 轮后停止。$\min D_4=\min(5,4)=4$，在 $x_4=1$ 处达到；沿回溯表逆向读取，$x_3=p_4(1)=1$，$x_2=p_3(1)=1$，$x_1=p_2(1)=0$，得到最优配置 $(0,1,1,1)$。图 11.3把同一过程画成每一轮的表和每个状态选中的前驱。

表 11.2　四变量二值链上的逐轮计算。“候选”一栏是式 (11.4)方括号中对 $x_{k-1}=0,1$ 的两个值；D_k 等于一元项加上候选中的较小者，p_k 是达到较小者的 x_{k-1}。

		$x_k=0$			$x_k=1$		
k	ϕ_k	候选	$D_k(0)$	$p_k(0)$	候选	$D_k(1)$	$p_k(1)$
1	$(0,3)$	—	0	—	—	3	—
2	$(2,0)$	0, 5	2	0	2, 3	2	0
3	$(3,0)$	2, 4	5	0	4, 2	2	1
4	$(1,2)$	5, 4	5	1	7, 2	4	1

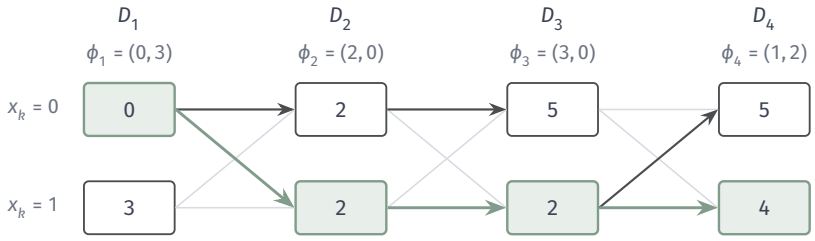


图 11.3　四变量二值链上的递推表。每列是一个变量，上下两格分别是 $D_k(0)$ 与 $D_k(1)$，列上方的小字为一元项。两列之间的灰色线表示全部候选，箭头从每个状态选中的前驱指向该状态，即回溯表 p_k 记录的选择。末列中 $D_4(1)=4$ 最小，沿绿色箭头逆向读取得到最优配置 $(0,1,1,1)$。逐点选取一元项最小的状态会得到 $(0,1,1,0)$，成本为 5。

11.2.2.3 算法分析

正确性　递推的正确性在于每张表 D_k 确实是前 k 个变量的最小部分成本。对 k 归纳。

$k=1$ 时没有被汇总的变量,结论就是定义。设结论对 $k-1$ 成立。前 k 个变量的部分成本等于前 $k-1$ 个变量的部分成本加上 $\phi_{k-1,k}(x_{k-1},x_k)+\phi_k(x_k)$，后两项不含 $x_1,\dots,x_{k-2}$。按式 (11.3)的第一条，对 $x_1,\dots,x_{k-1}$ 求最小值可以

先对 $x_1,\dots,x_{k-2}$、再对 x_{k-1} 进行；按第二条，先求的那一步可以越过后两项，只作用在前 $k-1$ 个变量的部分成本上，由归纳假设得到 $D_{k-1}(x_{k-1})$。

再对 x_{k-1} 求最小值，并把与 x_{k-1} 无关的 $\phi_k(x_k)$ 移出，就得到式 (11.4)。取 $k=n$，前 n 个变量的部分成本就是 F，所以 $\min_{x_n} D_n(x_n)=F^\star$。

回溯得到的配置达到这个最小值。从末端起逐步向左，每一步选的 $x_{k-1}=p_k(x_k)$ 都达到式 (11.4)中的最小值，于是读出的 $x_1,\dots,x_k$ 使前 k 个变量的部分成本恰好等于 $D_k(x_k)$；读到 x_1 时，整个配置的成本等于 $D_n(x_n)=F^\star$。这一论证要求 $F^\star$ 有限；$F^\star=+\infty$ 时每个配置都“达到”最小值，所以算法报告不可行，而不输出配置。

与逐点选择的比较　成对项正是逐点选择失效的原因。让每个变量只看自己的一元项、各自取最小，在上面的二值链中会选 $(0,1,1,0)$：一元项合计为 1，但两处取值变化要付 4，总成本为 5。递推得到的 $(0,1,1,1)$ 在 x_4 上多付了 1 个单位的一元成本，却省掉了最后一处变化，总成本为 4。D_k 之所以能避免这种短视，是因为它对 x_k 的每个状态都保留了“左侧最好能做到多少”，等后面的变量看过成对项以后再决定 x_k 取哪个状态。

复杂度　设每个变量至多有 q 个状态。每一轮为 x_k 的每个状态比较至多 q 个候选，共至多 q^2 次候选计算，整条链约 nq^2 次；D_k 与 p_k 各至多 q 项，共约 $2nq$ 项。若只要最优值而不恢复配置，任一时刻只需保留上一轮的表。与 q^n 个配置的枚举相比，开销从指数降为线性于 n。

11.2.3　树上的消息递推

11.2.3.1　问题与思路

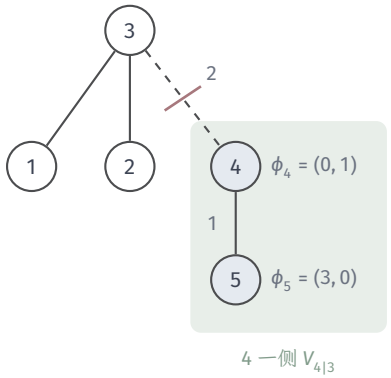
树没有链那样的天然顺序，但选定一个根 ρ 以后，每个非根顶点都有唯一的父点（第 1.4节）。链上的 D_{k-1} 概括了 x_k 左侧的全部变量；树上一个顶点可能有多个子点，每个子点下面挂着一棵子树，需要分别概括后再合并。

对树的一条边 $\{u,v\}$，删去它以后树分成两部分，记含 u 的那部分顶点集为 $V_{u|v}$，称为 u 一侧。u 一侧的变量只通过这条边与其余部分发生联系，x_v 就是这一侧与其余部分之间的接口：只要 x_v 给定，u 一侧的变量最好怎样取值、最少要付多少成本就确定了。

把这个“最少要付多少”对 x_v 的每个状态列出来，就得到一张报价单：“你取 0，我这边最少付多少；你取 1，最少付多少”。v 拿到报价单，不必知道 u 一侧有哪些变量、怎样取值，就能在决定自己的状态时把那一侧的成本算进去。

以图 11.4的五顶点树为例，删去边 $\{3,4\}$ 后，4 一侧是 $\{4,5\}$。$x_3=0$ 时，这一侧最便宜的取法是 $(x_4,x_5)=(0,1)$：一元项 $\phi_4(0)+\phi_5(1)=0$，边 $\{3,4\}$

两端相同不付费，边 $\{4,5\}$ 两端不同付 1，合计 1。$x_3 = 1$ 时最便宜的是 $(1,1)$，只付一元项 $\phi_4(1) = 1$。于是 4 向 3 报出的价目是 $(1,1)$。



消息 $m_{4\to 3}$：4 一侧的最低成本

x_3	最低成本	(x_4, x_5)
0	1	$(0,1)$
	0 + 0 + 0 + 1	
1	1	$(1,1)$
	1 + 0 + 0 + 0	

图 11.4　消息是一侧的报价单。左：删去边 $\{3,4\}$（虚线，旁边的 2 是两端取值不同时的成本），绿色区域是 4 一侧 $V_{4|3} = \{4,5\}$，边 $\{4,5\}$ 两端不同时成本为 1。右：x_3 取各状态时，4 一侧的最低成本及达到它的取值；小字依次是 ϕ_4、ϕ_5、边 $\{3,4\}$、边 $\{4,5\}$ 的成本。回溯表 $b_{4\to 3}$ 只记录其中 x_4 的取值 $(0,1)$，x_5 由 $b_{5\to 4}$ 给出。

这张报价单称为从 u 发往 v 的**消息**（message）。它可以逐层算出：u 一侧除 u 本身以外，按 u 的其他邻居 r 分成几块，每块的最低成本已由消息 $m_{r\to u}$ 报给 u；u 把自己的一元项、各块报来的价目和边 $\{u,v\}$ 上的成对项相加，再替自己选最好的状态，即

$$m_{u\to v}(x_v) = \min_{x_u \in \mathcal{X}_u} \Big[\phi_u(x_u) + \phi_{u,v}(x_u, x_v) + \sum_{r \in N(u)\setminus\{v\}} m_{r\to u}(x_u) \Big], \tag{11.5}$$

其中 $N(u)$ 是 u 在依赖图中的邻居集合。消息等于 u 一侧在 x_v 固定时的最低成本，这一含义在第 11.2.3.3小节证明。叶顶点只有一个邻居，求和为空，消息只由它自己的一元项和边上的成对项算出。链上的 $D_k(x_k)$ 正是 $\phi_k(x_k) + m_{k-1\to k}(x_k)$。

式 (11.5)右端用到 u 的其他邻居发来的消息，所以要从叶开始算，每个顶点等全部子点报完价再向父点报价；全部报到根以后，根把一元项与各条入消息相加，就得到整棵树在根取各状态时的最低成本。

与链上一样，取最小值丢掉了达到者，所以算每条消息时同时为接收端的每个状态 x_v 记下达到最小值的 x_u，组成回溯表 $b_{u\to v}(x_v)$。根选定状态以后，每个子点只需按父点的取值查自己的回溯表，就能沿树向外恢复整个配置。

11.2.3.2 算法实现

计算开始前任选根 ρ，求出每个顶点的父点 $p(u)$ 和一个后序，即先列出全部子点、再列出父点的顶点序列，可由一次深度优先遍历的完成次序得到。根、父点和后序只规定计算次序，运行中不变，与局部函数表一样不算状态。算法维护四项状态：

- 消息表 $m_{u\to p(u)}$，每个非根顶点 u 一张，以父点的取值 $x_{p(u)}$ 为索引；初始均未计算；
- 回溯表 $b_{u\to p(u)}$，与消息表一一对应，记录达到最小值的 x_u；初始为空；
- 根处的合并值 $g_\rho(x_\rho)$：根取各状态时整棵树的最低成本；向上一遍结束后才算出；
- 配置 $\boldsymbol{x}$：初始各变量均未赋值，回溯时逐个写入。

向上一遍按后序依次处理非根顶点 u，记 $v = p(u)$。此时 u 的全部子点都已处理，消息 $m_{r\to u}$ 都已算出。每处理一个顶点做三件事：

- 汇总入消息：对 x_u 的每个状态，计算 $\phi_u(x_u)+\sum_{r\in N(u)\setminus\{v\}} m_{r\to u}(x_u)$，即 u 及其子树在 x_u 固定时的最低成本；
- 列候选：对 x_v 的每个状态和 x_u 的每个状态，在上一步的值上加 $\phi_{u,v}(x_u,x_v)$，得到式 (11.5)方括号中的值；
- 取最小：对 x_v 的每个状态，把最小的候选值记为 $m_{u\to v}(x_v)$，达到它的 x_u（并列取编号最小者）记为 $b_{u\to v}(x_v)$。

全部非根顶点处理完后，在根处合并：

$$g_\rho(x_\rho) = \phi_\rho(x_\rho) + \sum_{r\in N(\rho)} m_{r\to\rho}(x_\rho),$$

最小成本为 $\min_{x_\rho} g_\rho(x_\rho)$。若它为 $+\infty$，报告不可行并停止；否则向外一遍恢复配置：

- 定根：x_ρ 取使 g_ρ 最小的状态，并列取编号最小者；
- 查表：按前序（后序的逆序，父点总在子点之前）处理非根顶点 u，令 $x_u = b_{u\to p(u)}(x_{p(u)})$。轮到 u 时父点已经赋值，所以查表总能进行。

每个非根顶点恰好赋值一次，全部变量赋值后停止。

算法 11.2　树上的消息递推

输入：依赖树上各变量的有限状态集合、一元项表和各边的成对项表。输出：最小成本和一个最优配置，或不可行报告。

1. 任选根 ρ，求出父点 $p(u)$ 与一个后序。
2. 按后序处理每个非根顶点 u，记 $v = p(u)$：对 x_v 的每个状态，枚举 x_u 的全部状态，按式 (11.5)计算候选值，取最小者记为 $m_{u\to v}(x_v)$，达到它的 x_u（并列取编号最小者）记为 $b_{u\to v}(x_v)$。

3. 计算 $g_\rho(x_\rho)=\phi_\rho(x_\rho)+\sum_{r\in N(\rho)} m_{r\to\rho}(x_\rho)$。若 $\min_{x_\rho} g_\rho(x_\rho)=+\infty$，报告不可行并停止。
4. 取使 g_ρ 最小的 x_ρ（并列取编号最小者）；按前序对每个非根顶点 u 令 $x_u=b_{u\to p(u)}(x_{p(u)})$。

执行过程 沿用图 11.4的树：边为 $\{1,3\},\{2,3\},\{3,4\},\{4,5\}$，以 3 为根；二值一元项为 $\phi_1=(0,2)$、$\phi_2=(2,0)$、$\phi_3=(1,0)$、$\phi_4=(0,1)$、$\phi_5=(3,0)$；成对项在两端相同时为 0，不同时分别为 $1,1,2,1$（按上面边的顺序）。父点为 $p(1)=p(2)=p(4)=3$、$p(5)=4$，取后序 $1,2,5,4,3$。向上一遍依次处理四个非根顶点，表 11.3列出每条消息的候选：

- 处理 1：叶顶点，汇总值就是 $\phi_1=(0,2)$。$x_3=0$ 的候选为 $0+0$ 与 $2+1$，$x_3=1$ 的候选为 $0+1$ 与 $2+0$，得 $m_{1\to3}=(0,1)$，两处都由 $x_1=0$ 达到，$b_{1\to3}=(0,0)$。
- 处理 2：汇总值为 $\phi_2=(2,0)$，得 $m_{2\to3}=(1,0)$，$b_{2\to3}=(1,1)$：无论 x_3 取什么，x_2 都取 1 更便宜。
- 处理 5：汇总值为 $\phi_5=(3,0)$，得 $m_{5\to4}=(1,0)$，$b_{5\to4}=(1,1)$。
- 处理 4：子点 5 的消息已经算出，汇总值为 $\phi_4+m_{5\to4}=(0+1,\,1+0)=(1,1)$。$x_3=0$ 的候选为 $1+0$ 与 $1+2$，$x_3=1$ 的候选为 $1+2$ 与 $1+0$，得 $m_{4\to3}=(1,1)$，$b_{4\to3}=(0,1)$，与图 11.4右侧的报价一致。
- 根处合并：$g_3(0)=1+0+1+1=3$，$g_3(1)=0+1+0+1=2$。最小成本为 2。

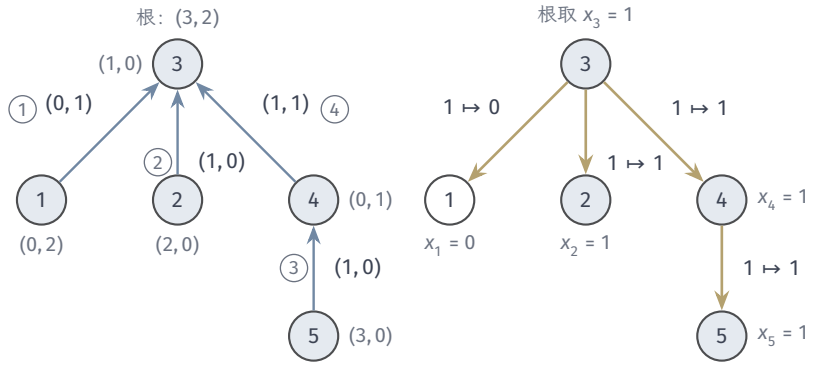
向外一遍从根开始，查表 11.3中接收端取 1 的回溯列：

- 定根：$g_3=(3,2)$，取 $x_3=1$；
- 按前序 $3,4,5,2,1$ 查表：$x_4=b_{4\to3}(1)=1$，$x_5=b_{5\to4}(1)=1$，$x_2=b_{2\to3}(1)=1$，$x_1=b_{1\to3}(1)=0$。

恢复的配置为 $(x_1,\dots,x_5)=(0,1,1,1,1)$。整个计算只算了 4 条消息，每条比较 4 个候选，而枚举需要 $2^5=32$ 个配置。图 11.5画出了两遍的方向与次序。

表 11.3 分叉树上按后序计算的消息。“候选”一栏是式 (11.5)方括号中对 $x_u=0,1$ 的两个值，每个值依次由一元项、边上成对项和子点消息相加；消息值取较小者，回溯表记录达到较小者的 x_u。

	接收端取 0			接收端取 1		
消息	候选	值	回溯	候选	值	回溯
$m_{1\to3}$	$0+0,\ 2+1$	0	0	$0+1,\ 2+0$	1	0
$m_{2\to3}$	$2+0,\ 0+1$	1	1	$2+1,\ 0+0$	0	1
$m_{5\to4}$	$3+0,\ 0+1$	1	1	$3+1,\ 0+0$	0	1
$m_{4\to3}$	$0+0+1,\ 1+2+0$	1	0	$0+2+1,\ 1+0+0$	1	1



(a) 自叶向根的消息　　(b) 自根向外的回溯

图 11.5　分叉树上的消息与回溯。左：以 3 为根，按后序 1, 2, 5, 4 发送消息；顶点旁的灰色二元组是一元项，箭头旁的二元组是消息表在接收端取 0、1 时的值，小圆圈中的序号是发送顺序；根合并后得到 $(3, 2)$。右：根取 $x_3 = 1$ 后沿树向外查回溯表，黄色箭头旁的“$1 \mapsto 0$”表示父点取 1 时子点的最优选择为 0；蓝色顶点取 1，白色顶点取 0，恢复的配置 $(0, 1, 1, 1, 1)$ 成本为 2。

把恢复的配置代回全部原始局部项，可以不依赖消息计算而独立核对结果。一元项依次为 $0, 0, 0, 1, 0$，合计 1；成对项中只有边 $\{1, 3\}$ 两端不同，贡献 1；总成本为 2，与根处的值一致，而且每个局部项只计了一次。

11.2.3.3 算法分析

正确性　树上递推的正确性归结为一件事：每条消息确实是发送侧在接收端取值固定时的最低成本，即前面所说的报价单。

定理 11.1　树上消息的含义　设依赖图 $\mathcal{G} = (V, E)$ 是有限树，总成本由式 (11.2)给出。对任意边 $\{u, v\}$，记 $F_{u|v}$ 为 u 一侧的部分成本：$V_{u|v}$ 中各顶点的一元项、两端都在 $V_{u|v}$ 中的成对项，再加上边 $\{u, v\}$ 上的 $\phi_{u,v}$。则对每个 x_v，

$$m_{u\to v}(x_v) = \min_{\boldsymbol{x}_{V_{u|v}}} F_{u|v}(\boldsymbol{x}_{V_{u|v}}, x_v).$$

进一步，对任意根 ρ，$\min_{\boldsymbol{x}} F(\boldsymbol{x}) = \min_{x_\rho}\big[\phi_\rho(x_\rho) + \sum_{r\in N(\rho)} m_{r\to\rho}(x_\rho)\big]$。

证明 对 $|V_{u|v}|$ 归纳。删去 u 以后，$V_{u|v} \setminus \{u\}$ 按 u 的其他邻居 $r \in N(u) \setminus \{v\}$ 分成互不相交的部分 $V_{r|u}$。由树中路径的唯一性，不同部分之间没有边，否则会与 u 形成环。因此 $F_{u|v}$ 可以拆成

$$F_{u|v} = \phi_u(x_u) + \phi_{u,v}(x_u, x_v) + \sum_{r \in N(u)\setminus\{v\}} F_{r|u},$$

其中每个 $F_{r|u}$ 只依赖 x_u 与 $\boldsymbol{x}_{V_{r|u}}$，并且每个局部项恰好出现在其中一处。固定 x_u 后，各 $\boldsymbol{x}_{V_{r|u}}$ 是互不相交的变量组，由式 (11.3)后的第三条规则，对它们求最小值得到 $\sum_r \min F_{r|u}$；由归纳假设，这个和等于 $\sum_r m_{r\to u}(x_u)$。最后对 x_u 求最小值，就得到式 (11.5)。叶顶点是归纳的起点，此时求和为空，结论直接由定义得出。根处的等式用同样的拆分得到：$V \setminus \{\rho\}$ 按 ρ 的邻居分成互不相交的部分，每部分的最小成本是一条入消息。 ■

链是树的特例：以 n 为根时 $D_k(x_k) = \phi_k(x_k) + m_{k-1\to k}(x_k)$，定理再次给出 D_k 的含义。

回溯得到的配置也确实达到最优值。对子树大小归纳可知：若 x_v 已经确定，按回溯表给 u 一侧赋值后，u 一侧的部分成本 $F_{u|v}$ 恰好等于 $m_{u\to v}(x_v)$，因为每一步选的 x_u 都达到式 (11.5)中的最小值，而子树内部又由归纳假设达到各条入消息的值。在根处合并后，总成本就等于根处的最小合并值。

这一论证要求根处最小值有限；为 $+\infty$ 时任何配置都“达到”它，所以算法在这种情况下报告不可行，而不把回溯结果当作最优解输出。

回溯表只恢复一个最优配置。若要列出全部并列最优的配置，需要在每个状态记录全部最优选择，而这类配置的数目本身可能随 n 指数增长。

复杂度 设每个变量至多有 q 个状态。处理非根顶点 u 时，汇总入消息需要对 x_u 的至多 q 个状态各加 $|N(u)| - 1$ 项，列候选与取最小共需至多 q^2 次加法和比较；对全部顶点求和，利用树中 $\sum_u |N(u)| = 2(n-1)$，向上一遍共需 $O\big(\sum_u |N(u)|q + nq^2\big) = O(nq^2)$ 次运算。根处合并需要 $O(|N(\rho)|q)$ 次，回溯只查 $n-1$ 次表。存储为 $n-1$ 张消息表和同样多的回溯表，每张至多 q 项。与 q^n 个配置的枚举相比，开销从指数降为线性于 n，代价是要为每条边保存一张表。

适用范围 本节的递推依赖两个条件。一是依赖图无环：定理的证明用到“删去 u 后各部分之间没有边”。二是局部项至多读取两个变量，使接口总是单个变量。依赖图含环时，删去一条边不再把变量分成两侧，单个变量不足以充当接口；下一节在四环上具体看这一点。

11.2.3.4 各顶点的最小成本表

有时需要的不只是一个最优配置，而是每个顶点 v 取各状态 s 时整体能达到的最小成本 $\min_{\boldsymbol{x}:\, x_v=s} F(\boldsymbol{x})$，例如要知道某个变量换一个状态会多付多少。对根而言，这正是合并值 g_ρ；对其他顶点，向上一遍只收集了来自子树一侧的消息，还缺父点一侧的报价。

所以再按前序从根向叶发送一遍消息，同样使用式 (11.5)：父点 v 向子点 u 发出的 $m_{v\to u}$，汇总的是 v 的一元项、v 的父点发来的消息和 v 的其他子点发来的消息。到达 u 时，u 的全部邻居都已向它发过消息，把一元项与全部入消息相加，由定理 11.1（以 u 为根）就得到 u 的最小成本表。两遍共发送 $2(n-1)$ 条消息，开销仍为 $O(nq^2)$。

在上例中，

$$m_{3\to 1}(x_1)=\min_{x_3}\bigl[\phi_3(x_3)+\phi_{1,3}(x_1,x_3)+m_{2\to 3}(x_3)+m_{4\to 3}(x_3)\bigr]=(2,1),$$

其中 $x_3=0,1$ 时一元项与两条入消息之和分别为 $1+1+1=3$ 与 $0+0+1=1$，再加上边 $\{1,3\}$ 的成本取最小即得。于是 x_1 取 0、1 时的最小总成本为 $\phi_1+m_{3\to 1}=(2,3)$。同样可得其余顶点的表：x_2 为 $(4,2)$，x_3 为 $(3,2)$，x_4 为 $(3,2)$，x_5 为 $(5,2)$。每张表的最小值都等于整体最优值 2，并且都在最优配置 $(0,1,1,1,1)$ 的相应分量处达到。

11.3 从树到一般图

上一节的每一步都依赖一个事实：删去依赖树的一条边，变量就分成两侧，两侧只通过一个变量相连。依赖图含环时，这个事实不再成立。本节先说明环怎样改变接口，再用一个四环算例把变化落到具体数值上，最后说明一般图上的动态规划要事先构建什么。

11.3.1 环带来的接口变化

考虑四个变量排成的环 $1-2-3-4-1$，每条边上有一个成对成本。删去边 $\{1,2\}$ 后，剩下的是路径 $2-3-4-1$，仍然连通：x_1 不直接与 x_2 相连，却可以经由 x_4 和 x_3 影响 x_2 的最优选择。因此，沿一条边传递一张只以 x_2 为索引的报价单，无法概括 x_1 一侧，因为根本不存在只经过 x_2 与其余部分相连的“x_1 一侧”。

链和树上的做法可以换一种说法，使它不依赖“一侧”。叶顶点 u 只出现在 ϕ_u 和 $\phi_{u,p(u)}$ 两项中；计算消息 $m_{u\to p(u)}$，就是把这两项相加后对 x_u 取最小，然后用得到的表替换这两项。这个动作对任意局部函数集合都有意义。

设当前有一组函数，要汇总变量 x_i。先找出当前全部作用域含 i 的函数，记为 $\mathcal{F}_i$；它们的作用域之并去掉 i，记为 S_i。不含 i 的函数与 x_i 无关，可以留在最小化之外；含 i 的函数则必须一起处理。于是对 $\boldsymbol{x}_{S_i}$ 的每种取值，把 $\mathcal{F}_i$ 中的函数相加，再对 x_i 取最小，得到一个新函数

$$\psi_i(\boldsymbol{x}_{S_i}) = \min_{x_i \in \mathcal{X}_i} \sum_{f \in \mathcal{F}_i} f. \tag{11.6}$$

然后从当前函数集合中删去 $\mathcal{F}_i$，加入 ψ_i；不含 i 的函数保持原样。这一操作称为消去 x_i，按某个次序逐个消去变量的方法称为**变量消元**（variable elimination）。

消去 x_i 可以理解为替 x_i 提前做好决定。x_i 的好坏只取决于 S_i 中那些与它同在某个函数里的变量；对这些变量的每一种取值，先算出 x_i 最好取什么、最少付多少，写成一张以 $\boldsymbol{x}_{S_i}$ 为索引的表，此后其余变量只需查这张表，不必再关心 x_i。由式 (11.3)的前两条，不含 i 的函数可以移出对 x_i 的最小化，所以替换前后，剩余变量每种取值下的最小成本不变；第 13.2.5小节给出完整证明。

在树上消去叶顶点 u 时，$S_u = \{p(u)\}$，新函数就是消息 $m_{u \to p(u)}$，接口仍是单个变量。在四环上消去 x_1，情况不同：含 1 的函数是 $\phi_{1,2}$ 与 $\phi_{1,4}$，所以 $S_1 = \{2, 4\}$，新函数 $\psi_1(x_2, x_4)$ 同时读取 x_2 和 x_4。在原来的环上，2 与 4 并不相邻；消去 x_1 以后，它们通过 ψ_1 直接耦合在一起。

能否不要这张二元表，而给 2 和 4 各发一张单变量表？分别计算

$$g(x_2) = \min_{x_1} \phi_{1,2}(x_1, x_2), \qquad h(x_4) = \min_{x_1} \phi_{1,4}(x_1, x_4)$$

相当于允许两项各自为 x_1 挑选取值。由于每个 x_1 都满足 $\phi_{1,2} + \phi_{1,4} \geq g(x_2) + h(x_4)$，总有 $g(x_2) + h(x_4) \leq \psi_1(x_2, x_4)$，但两边一般不相等：$x_1$ 只有一个，它在两项中必须取同一个值。下一小节的数值会说明，这个差异不仅是这一种拆法的缺陷，而且任何单变量表之和都无法代替 ψ_1。

还有一种看似自然的做法，是在含环的依赖图上照搬式 (11.5)，让各顶点反复交换消息。这时同一部分成本会沿环被重复计入，上一节的正确性证明不再适用。这类迭代是否收敛、收敛到什么，需要另外分析，属于后续概率推断内容中的近似方法；本部分只讨论精确计算。

11.3.2 四环算例

本小节固定一个具体问题，后两章沿用它，分别演示两种结构构建和统一求值，避免更换例子掩盖它们之间的对应关系。四个二值变量 $x_1, \ldots, x_4$ 排成环 $1 - 2 - 3 - 4 - 1$，四条边上各有一个成对成本，没有一元项：

$$F(\boldsymbol{x}) = \phi_{1,2}(x_1, x_2) + \phi_{2,3}(x_2, x_3) + \phi_{3,4}(x_3, x_4) + \phi_{1,4}(x_1, x_4). \tag{11.7}$$

四张表由表 11.4给出，每张表的行对应作用域中编号较小的变量。这些数值是教学示例：$\phi_{3,4}(1,1)=+\infty$ 禁止 x_3,x_4 同时取 1，$\phi_{1,4}(0,1)=+\infty$ 禁止 $x_1=0$ 而 $x_4=1$。

表 11.4　四环算例的局部成本。每张表的行对应作用域中编号较小的变量取 $0,1$，列对应编号较大的变量取 $0,1$；例如 $\phi_{1,4}(0,1)=+\infty$。

$\phi_{1,2}(x_1,x_2)$	$\phi_{2,3}(x_2,x_3)$	$\phi_{3,4}(x_3,x_4)$	$\phi_{1,4}(x_1,x_4)$
$\begin{bmatrix}3&0\\1&2\end{bmatrix}$	$\begin{bmatrix}1&3\\1&1\end{bmatrix}$	$\begin{bmatrix}0&1\\0&+\infty\end{bmatrix}$	$\begin{bmatrix}3&+\infty\\3&0\end{bmatrix}$

枚举基准　先用枚举给出核验基准。表 11.5按字典序列出全部 16 个配置的总成本。其中 6 个配置被禁止：$x_4=1$ 而 $x_1=0$ 的四个，以及 $x_3=x_4=1$、$x_1=1$ 的两个。其余 10 个可行。最小成本为 $F^\star=3$，唯一的最优配置是 $\boldsymbol{x}^\star=(1,0,0,1)$，此时四项依次为 $\phi_{1,2}(1,0)=1$、$\phi_{2,3}(0,0)=1$、$\phi_{3,4}(0,1)=1$、$\phi_{1,4}(1,1)=0$。

表 11.5　四环算例全部配置的总成本。配置按 (x_1,x_2,x_3,x_4) 的字典序排列，前两行是 $x_1=0$ 的八个配置，后两行是 $x_1=1$ 的八个配置。

0000	0001	0010	0011	0100	0101	0110	0111
7	$+\infty$	9	$+\infty$	4	$+\infty$	4	$+\infty$
1000	1001	1010	1011	1100	1101	1110	1111
5	3	7	$+\infty$	6	4	6	$+\infty$

消去 x_1　按式 (11.6)消去 x_1。对 (x_2,x_4) 的每种取值，比较 $x_1=0,1$ 两个候选 $\phi_{1,2}(x_1,x_2)+\phi_{1,4}(x_1,x_4)$：

$$\psi_1(0,0)=\min(3+3,\ 1+3)=4,\qquad \psi_1(0,1)=\min(3+\infty,\ 1+0)=1,$$
$$\psi_1(1,0)=\min(0+3,\ 2+3)=3,\qquad \psi_1(1,1)=\min(0+\infty,\ 2+0)=2,$$

达到最小值的 x_1 依次为 $1,1,0,1$。

与之比较，分别取最小的单变量表是 $g(x_2)=\min_{x_1}\phi_{1,2}(x_1,x_2)=(1,0)$ 与 $h(x_4)=\min_{x_1}\phi_{1,4}(x_1,x_4)=(3,0)$。它们的和在 $(x_2,x_4)=(1,1)$ 处为 0，而 $\psi_1(1,1)=2$：$g(1)=0$ 要求 $x_1=0$，$h(1)=0$ 要求 $x_1=1$，两项选的 x_1 互相矛盾。

更进一步，任何形如 $g'(x_2)+h'(x_4)$ 的有限函数都满足

$$\big[g'(0)+h'(0)\big]+\big[g'(1)+h'(1)\big]=\big[g'(0)+h'(1)\big]+\big[g'(1)+h'(0)\big],$$

而 $\psi_1(0,0)+\psi_1(1,1)=6\neq 4=\psi_1(0,1)+\psi_1(1,0)$。所以 ψ_1 不能拆成两张单变量表之和，消去 x_1 以后的接口必须是 $\{2,4\}$ 这两个变量的联合取值。

完成求值 消去 x_1 后，剩下的函数是 $\psi_1(x_2,x_4)$、$\phi_{2,3}(x_2,x_3)$ 与 $\phi_{3,4}(x_3,x_4)$，只涉及 x_2,x_3,x_4 三个变量。把三项相加，按 (x_2,x_3,x_4) 的字典序得到

$$(5,\ 3,\ 7,\ +\infty,\ 4,\ 4,\ 4,\ +\infty),$$

最小值 3 在 $(x_2,x_3,x_4)=(0,0,1)$ 处唯一达到。再查 x_1 的选择：$(x_2,x_4)=(0,1)$ 时 $x_1=1$。恢复的配置 $(1,0,0,1)$ 与枚举结果一致。

这里同时处理的变量最多是三个：消去 x_1 时要看 $\{1,2,4\}$ 的全部 8 种取值，最后又要看 $\{2,3,4\}$ 的 8 种取值。第 13章会把这一计算写成统一的递推，并用同一组数据检验最小成本、保留变量的成本表、计数和加权求和。

11.3.3 一般图动态规划的构建目标

四环算例中的计算可以看成两个局部子问题。第一个子问题负责变量组 $\{1,2,4\}$，固定接口 (x_2,x_4) 后汇总内部变量 x_1，输出表 ψ_1；第二个子问题负责 $\{2,3,4\}$，读入 ψ_1 和其余两项，汇总全部剩余变量。链和树上的递推也是这样一族子问题，只是每个接口恰好是单个变量。

一般图上的动态规划要先构建这样一族子问题，它必须满足三项要求：

1. 每个局部项都放进某个子问题，该子问题处理的变量包含它的整个作用域，并且每项只计一次；
2. 摘要足够：一个子问题的内部变量与尚未处理部分的全部耦合都经过接口，所以以接口为索引的表不会丢失信息；
3. 依赖可以排序：每个子问题只使用此前已完成的子问题的输出，最后汇总成整体结果。

这三项都只涉及作用域和变量组，不涉及函数的具体数值，所以构建可以在求值之前单独完成。

树上递推正是满足这些要求的特例。把依赖树的一元项和成对项作为局部函数，按后序消去非根顶点：轮到 u 时，它的子点都已消去，子点留下的新函数与 ϕ_u、$\phi_{u,p(u)}$ 是仅有的含 u 的函数，所以 $S_u=\{p(u)\}$，新函数 ψ_u 就是消息 $m_{u\to p(u)}$，达到最小值的 x_u 就是回溯表 $b_{u\to p(u)}$。

构建这族子问题有两种描述方式。变量消元用一个处理次序生成它们：每一步处理的变量组是当前变量及其 S_i，新函数交给后面第一个用到它的步骤。树分解则把变量组（称为袋）和它们之间的接口显式组织成一棵树。第 12章说明这两种构建怎样互相转换、怎样衡量其规模、怎样构造；第 13章在构建好的结构上完成数值计算。四环算例已经显示，构建的质量体现为最多同时处理多少个变量，这里是三个。

11.4　本章小结

本章的问题由局部函数组成：每个变量取有限个状态，整体成本是若干局部项之和，每一项只读取作用域中的少数变量，取值 $+\infty$ 表示禁止的组合。同一份输入可以提出不同的输出要求：最小成本与一个最优配置、保留一组变量后的条件最小成本表、可行配置的个数，以及按局部权重之积定义的加权总和。枚举总能回答这些问题，但开销随变量数指数增长。

记录结构的几种图各有分工。原始图记录哪些变量共同出现，超图保留作用域的分组，因子图保留每个局部函数的身份；三者都不保存数值，原始图也不能恢复高阶规则，奇偶约束的例子说明三元规则不是成对规则的叠加。计算时始终使用完整的局部函数表。

链和树上之所以能精确递推，是因为删去依赖树的一条边就把变量分成两侧，两侧只通过一个接口变量相连。固定接口取值后，一侧的内部变量可以单独优化，结果写成以接口取值为索引的条件最小成本表；必须保留整张表，而不是一侧自己的最优值，因为接口最终取什么要等另一侧的成本也已知后才能决定。链上的递推、树上的消息和回溯表都是这一动作的具体形式，正确性来自有限集合上最小值的分步、移项和分离规则，开销对变量数是线性的。

依赖图含环时，单个变量不再能分隔两侧。消去一个变量会把它的邻居通过新函数耦合起来；四环算例中，消去 x_1 得到的 $\psi_1(x_2, x_4)$ 不能拆成两张单变量表之和，接口必须同时保留两个变量。因此，一般图上的动态规划要先构建一族局部子问题，使每个局部项有处可放、接口足以概括内部变量、子问题的依赖可以排序；变量消元与树分解是描述这一构建的两种方式，分别在下一章建立并证明等价，数值计算统一放在第 13章。

第 12 章

一般图上动态规划的构建

上一章的四环算例说明，依赖图含环时，消去一个变量会把它的邻居耦合起来，接口必须同时保留多个变量。于是在计算任何数值之前，要先回答：应当把哪些变量放在一起处理，每个子问题以哪些变量为接口，子问题之间按什么次序传递结果，才能保证每个局部项有处可放、每张摘要表不丢失信息？

本章只处理这个结构问题，输入是各局部函数的作用域，输出是一个可以核验的计算结构；具体数值表的计算留给下一章。第 12.1节写清输出要求，引入袋、接口和树分解，并说明保留变量的查询对结构提出什么额外要求。

随后两节从两个方向构建同一种结构。第 12.2节按一个消元次序模拟消元，用填充边记录新产生的耦合，把每一步同时处理的变量连成一棵树；第 12.3节从一个给定的树分解出发，证明每个局部项都能放进某个袋、接口确实分隔两侧，并从叶袋读出一个相容的消元次序。

第 12.4节把两个方向合起来：消元得到的中间函数对应袋之间的接口消息，最小诱导宽度等于树宽，并在四环上把双向转换逐步做一遍。第 12.5节讨论怎样实际得到好的结构：贪心选择消元次序、利用弦图的完美消元序、借第 4章的最大权生成树构造团树，以及怎样检验构建结果。

本章依赖上一章的消元操作式 (11.6)与四环算例，以及第 1.4节关于树的基本事实：树中两点之间的路径唯一，选根后每个非根节点有唯一的父节点。

12.1 构建问题与输出要求

第 11.3.3小节提出了一般图上动态规划的三项要求：每个局部项有处可放，接口足以概括内部变量，子问题的依赖可以排序。本节把这些要求写成可以核验的定义。先固定构建的输入，再引入袋与接口，用树分解刻画合法的袋树，并说明每个子问题的接口与内部变量；最后说明保留变量的查询对结构提出什么额外要求。

12.1.1　作用域与依赖图

构建的输入只是变量集合 V 和各局部函数的作用域 S_a，$a \in A$，不含任何函数值。由它们得到第 11.1.3小节的原始图 $\mathcal{G} = (V, E)$：以全部变量为顶点，两个变量同属某个作用域时连一条边。每个作用域 S_a 在 $\mathcal{G}$ 中两两相邻。

顶点集取全部变量，包括不出现在任何局部函数中的孤立变量。这样的变量不带来成本，但最优配置仍要给它赋值，计数时也要累计它的全部状态，所以它仍须出现在结构中。四环算例的作用域就是四条边，原始图就是环 $1-2-3-4-1$ 本身；三元作用域例子的原始图有五条边，只缺 $\{1,4\}$。

构建只使用作用域，所得结构对作用域相同而数值任意的全部问题都适用。某组特定数值可能恰好使一张中间表还能进一步分解（第 12.2.1小节给出一例），但构建不利用这种巧合：一般数值下必须保留的耦合，构建时一律保留。数值上的进一步化简要回到函数表本身判断，不属于本章的结构问题。

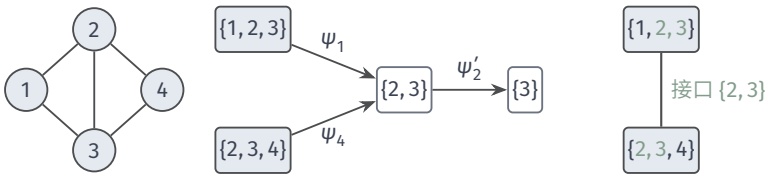
12.1.2　袋与接口

回到四环算例。第 11.3.2小节的计算先同时看 $\{1,2,4\}$，产生以 $\{2,4\}$ 为索引的表 ψ_1；再同时看 $\{2,3,4\}$，读入 ψ_1 完成计算。三元作用域例子也一样：若按次序 $(1,4,2,3)$ 消元，消去 x_1 时要同时处理 $\{1\} \cup S_1 = \{1,2,3\}$，消去 x_4 时处理 $\{2,3,4\}$，随后依次处理 $\{2,3\}$ 和 $\{3\}$。每一步产生的新函数会在后面某一步被用到：ψ_1 与 ψ_4 都在消去 x_2 时被合并，消去 x_2 得到的新函数在消去 x_3 时被使用。

把每一步同时处理的变量集合看成一个节点，按“新函数交给哪一步使用”连接这些节点，就得到图 12.1中间的一棵树。这棵树只记录了哪些变量要一起处理、表交给谁，与 ϕ_A, ϕ_B 的具体数值无关，换一组作用域相同的函数，它照样适用。

准备放进同一张表里同时索引的变量集合称为**袋**（bag），可以把它想成计算时一次摆上桌面的变量：对它们的每一种联合取值各记一个数。袋组成的树称为袋树。袋树的节点是计算单元，袋树的边表示两个计算单元之间传递一张表；它与原始图中连接两个变量的边含义不同。不同的袋可以有公共变量，例如 $\{1,2,3\}$ 与 $\{2,3,4\}$ 都含 $2,3$。

袋与原始图中的团有关，但不必是团。原始图中两两相邻的顶点集合称为**团**（clique）。三元作用域例子中 $\{1,2,3\}$ 是团，$\{1,2,4\}$ 不是，因为缺边 $\{1,4\}$。四环算例中第一个袋 $\{1,2,4\}$ 也不是原始图的团：2 与 4 在环上不相邻，消去 x_1 后才通过 ψ_1 耦合。所以袋可以重叠，也可以含有原图中不相邻的变量。



(a) 原始图 (b) 消元步骤的袋 (c) 并入小袋后

图 12.1 从消元步骤得到袋。左：三元作用域例子的原始图。中：按次序 $(1, 4, 2, 3)$ 消元时，每一步同时处理的变量集合构成一个袋，箭头表示新函数交给哪一步使用。右：$\{2, 3\}$ 和 $\{3\}$ 已包含在相邻的大袋中，并入后只剩两个袋，它们的交集 $\{2, 3\}$（绿色）就是接口。

接口与一致性 在三元作用域例子中，袋 $\{2, 3\}$ 和 $\{3\}$ 都已包含在相邻的大袋中，可以并入，最后只剩两个袋 $\{1, 2, 3\}$ 与 $\{2, 3, 4\}$，如图 12.1右侧。相邻两袋的交集称为它们的接口，这里是 $\{2, 3\}$；四环算例中两个袋的接口是 $\{2, 4\}$。这与第 11.2.1小节的接口变量含义相同，只是现在接口可以含多个变量。

接口上的变量在两个袋里是同一个变量，不是两个副本。左袋在汇总 x_1 时必须对 (x_2, x_3) 的每种取值分别给出结果，右袋使用这些结果时也必须按同样的 (x_2, x_3) 取值来读。左袋交出的正是一张以接口为索引的表。

两个袋之间只交换这样一张表，所以一个变量的取值也只能经接口从一个袋传到另一个袋，这就限制了袋的连接方式。若某个变量出现在两个不相邻的袋中，而连接它们的袋树路径上有某个袋不含这个变量，那么那个袋发出和收到的表都不以这个变量为索引，两侧便无法约定它取同一个值。另外，袋还必须能容纳所有局部函数，否则有的成本项无处计算。这两项要求合起来，就是下面的定义。

12.1.3 树分解与树宽

在结构上，容纳局部函数的要求落实为每个变量和原始图的每条边都被某个袋包含；保持一致的要求就是上面的连接要求。

定义 12.1 树分解 设 $\mathcal{G} = (V, E)$ 是有限无向图。$\mathcal{G}$ 的一个树分解（tree decomposition）由一棵有限树 $\mathcal{T}$（节点集记为 T）和每个节点 $t \in T$ 上的子集 $B_t \subseteq V$ 组成，满足：

1. 顶点覆盖：每个顶点 $v \in V$ 至少属于一个袋；
2. 边覆盖：对每条边 $\{u, v\} \in E$，存在某个袋同时含有 u 与 v；
3. 运行交集：对每个顶点 $v \in V$，含 v 的节点集合 $T_v = \{t \in T : v \in B_t\}$ 在 $\mathcal{T}$ 中连通。

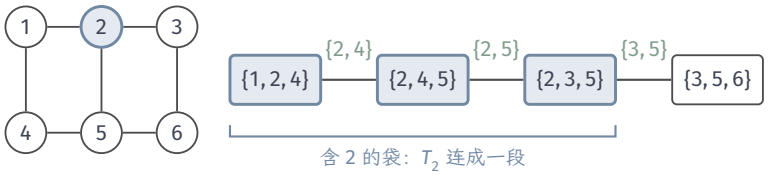
> 第三个条件称为**运行交集性质**（running intersection property）。

运行交集等价于：若 v 同时属于 B_s 和 B_t，则 $\mathcal{T}$ 中从 s 到 t 的路径上每个袋都含 v。因为树中两点之间只有一条路径，T_v 连通恰好意味着这条路径留在 T_v 中。用日常的话说，每个变量在袋树上占据一块连成一片的地盘；沿袋树行走，一个变量一旦离开就不会再出现。

梯子形图的分解　图 12.2中的梯子形图有两排顶点 $1,2,3$ 与 $4,5,6$，每排相邻顶点之间和同一列的两个顶点之间各有一条边。四个袋 $\{1,2,4\},\{2,4,5\},\{2,3,5\},\{3,5,6\}$ 连成一条路径。

可以逐条核对定义：每个顶点都在某个袋中；七条边中，$\{1,2\},\{1,4\}$ 在第一个袋，$\{4,5\},\{2,5\}$ 在第二个袋，$\{2,3\}$ 在第三个袋，$\{5,6\},\{3,6\}$ 在第四个袋；含 2 的袋是前三个，含 5 的是后三个，含 4 的是前两个，含 3 的是后两个，都连成一段。

这个分解像一个窄窗口从左向右扫过梯子：每个袋是窗口某一时刻看到的顶点，相邻两袋的接口是窗口移动时必须记住的顶点。从第一个袋移到第二个袋时，1 已经处理完毕，可以忘掉，只需记住 $\{2,4\}$；之后 4 也处理完毕，只需记住 $\{2,5\}$。运行交集保证被忘掉的顶点不会在后面的袋里再次出现，否则它的取值早已无从查找。梯子再长，窗口也只需同时看三个顶点；第 11.2.2小节的链上递推是同样的扫描，只是每次只需记住一个变量。



(a) 梯子形图　　(b) 宽度为 2 的树分解

图 12.2　梯子形图的树分解。(a) 两排顶点由三条横档相连。(b) 四个袋连成一条路径，像窗口自左向右扫过梯子；边上的绿色集合是相邻两袋的接口，即窗口移动时要记住的顶点。蓝色的袋都含顶点 2，它们连成一段，满足运行交集。

运行交集的作用　只覆盖顶点和边还不够。图 12.3比较路径 $1-2-3-4$ 上的两种袋连接。两种连接使用同样的三个袋 $\{1,2\},\{2,3\},\{3,4\}$，都覆盖了全部顶点和边。

上面一种按 $\{1,2\}-\{2,3\}-\{3,4\}$ 连接，含 2 的两个袋相邻，含 3 的两个袋也相邻，是树分解。下面一种按 $\{1,2\}-\{3,4\}-\{2,3\}$ 连接，含 2 的两个袋被不含 2 的袋 $\{3,4\}$ 隔开。若沿这棵树计算，左端的袋可以选 $x_2=0$，右

端的袋可以选 $x_2 = 1$，中间的袋不记录 x_2，无从发现矛盾；拼起来的结果不对应任何一个配置。

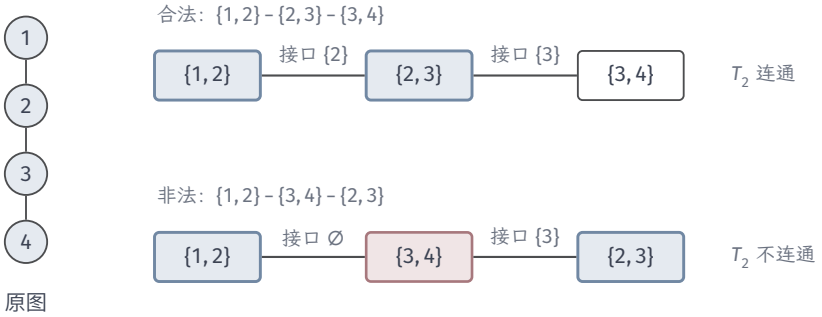


图 12.3　路径 $1-2-3-4$ 上的合法与非法袋连接。两种连接使用相同的三个袋，都覆盖全部顶点和边。上：含 2 的袋（蓝色）彼此相邻，含 3 的袋也相邻，满足运行交集。下：含 2 的两个袋之间隔着不含 2 的袋 $\{3,4\}$（红色），T_2 不连通，不是树分解。

宽度与树宽　袋越大，袋表越大。为了用一个数概括一次分解的规模，定义树分解的宽度为 $\max_{t\in T} |B_t| - 1$，即最大袋的大小减一；图 $\mathcal{G}$ 的**树宽**（treewidth）是它的全部树分解宽度的最小值，记为 $\mathrm{tw}(\mathcal{G})$。减一是一种约定，使得至少含一条边的树宽度恰为一。

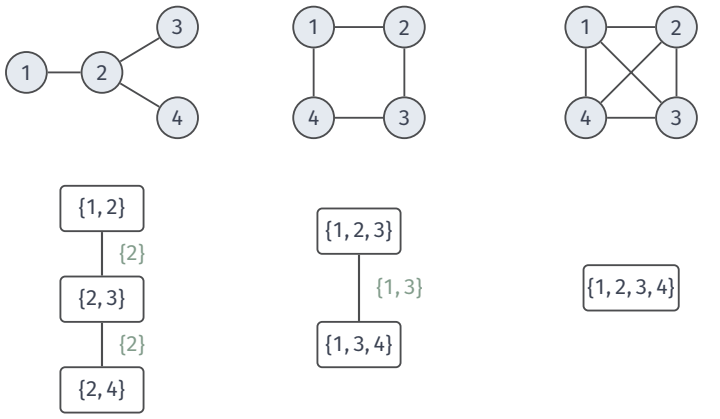
把整个图放进一个袋总是合法的树分解，宽度为 $|V| - 1$，所以树宽总有定义，问题在于能否用更小的袋。直观地说，树宽回答的是：把这个图压成一棵树的形状时，每个树节点至少要装下多少个顶点。图越像树，需要的袋越小。

确定一个图的树宽需要两步：构造一个分解给出上界，再证明不存在更小宽度的分解给出下界。下面先给出几个小图的上界构造，图 12.4画出了相应的分解；下界的证明要用到第 12.3.1小节的团容纳引理，在那里补上。

单个顶点的图只需一个袋 $\{1\}$，宽度为 0，任何没有边的图都如此。至少含一条边的树有宽度为 1 的分解：任选根 ρ，为每个非根顶点 u 建袋 $\{u, p(u)\}$，其中 $p(u)$ 是父点；若 $p(u)$ 不是根，就把这个袋连到 $p(u)$ 自己的袋上，根的各子点的袋则依次连成一条链。这样，非根顶点 v 出现在自己的袋和各子点的袋中，后者都与前者相邻；根只出现在各子点的袋中，它们连成链，所以运行交集成立。

四个顶点的环 $1-2-3-4-1$ 有宽度为 2 的分解：两个袋 $\{1,2,3\}$ 与 $\{1,3,4\}$ 以一条边相连，接口为 $\{1,3\}$；四环算例使用的 $\{1,2,4\}$ 与 $\{2,3,4\}$ 也是一个宽度为 2 的分解，接口为 $\{2,4\}$。图 12.2的梯子有宽度为 2 的分解。四个顶点两两相邻的完全图只用一个袋 $\{1,2,3,4\}$，宽度为 3。

某一次构造得到宽度 k，只说明 $\mathrm{tw}(\mathcal{G}) \le k$；要断言树宽恰为 k，还需要证明不存在更小宽度的分解。



(a) 树：树宽 1 (b) 四环：树宽 2 (c) 完全图：树宽 3

图 12.4 三个四顶点图及其宽度最小的树分解，绿色集合是接口。(a) 树：每条边一个袋，宽度为 1。(b) 四环：任何分解都有一个袋含三个顶点，宽度为 2。(c) 完全图：全部顶点必须同在一个袋中，宽度为 3。

若每个变量至多有 q 个状态，宽度为 k 的分解中，每个袋至多有 $k+1$ 个变量，稠密存储的袋表至多 q^{k+1} 项。这是第 13章中主要的枚举规模。实际的总开销还包括袋的个数、接口的大小和每次读取局部函数的成本，宽度只控制其中随状态数指数增长的那一部分，第 13.5节再作完整分析。

寻找树宽最小的分解与使用一个已经给定的分解是两项不同的任务。前者在一般图上是困难的组合问题；后者只需按定义核验三个条件，再沿树计算。第 12.5节给出的构造方法都只保证得到合法的分解和宽度上界。

12.1.4 接口与内部变量

一个树分解还要选定根，才能确定子问题的依赖次序。设已给袋树选根 r，每个非根节点 u 有父节点 $p(u)$。称

$$S_u = B_u \cap B_{p(u)}$$

为 u 的接口，$I_u = B_u \setminus S_u$ 为 u 的**内部变量**（internal variable）。根没有父节点，约定 $S_r = \varnothing$、$I_r = B_r$。

节点 u 对应的子问题是：固定接口取值 $\boldsymbol{x}_{S_u}$，汇总内部变量 $\boldsymbol{x}_{I_u}$ 以及 u 下方各子问题已经汇总的内容，输出一张以 $\boldsymbol{x}_{S_u}$ 为索引的表，交给父节点。接口为空时，这张表只有一项，是一个数。四环算例以 $\{2,3,4\}$ 为根时，子节点 $\{1,2,4\}$ 的接口为 $\{2,4\}$，内部变量为 $\{1\}$；根的内部变量为 $\{2,3,4\}$。

为了让每个变量恰好被汇总一次，每个变量应当恰好是一个节点的内部变量。下面的引理说明运行交集保证了这一点，并指出是哪个节点。

引理 12.2 每个变量恰在一个袋中作内部变量 设 $(\mathcal{T},\{B_t\}_{t\in T})$ 是 $\mathcal{G}$ 的树分解，袋树以 r 为根。对每个 $v\in V$，T_v 中有唯一一个离根最近的节点，记为 $\mathrm{top}(v)$，T_v 的每个节点都是它的后代（含它本身）；并且 $v\in I_u$ 当且仅当 $u=\mathrm{top}(v)$。

证明 由顶点覆盖，T_v 非空。取 T_v 中深度最小的一个节点 t。若 $y\in T_v$ 不是 t 的后代，从 y 到 t 的树上路径要先向上经过 y 与 t 的最近公共祖先 z，而 z 比 t 更靠近根；由 T_v 连通，这条路径留在 T_v 中，于是 $z\in T_v$，与 t 的选取矛盾。所以 T_v 的节点都是 t 的后代，深度最小的节点也只有 t 一个，记 $\mathrm{top}(v)=t$。

若 $u=\mathrm{top}(v)$ 是根，则 $v\in B_r=I_r$。若它不是根，其父节点不是它的后代，因而不在 T_v 中，即 $v\notin B_{p(u)}$，所以 $v\in I_u$。反过来，若 $u\in T_v$ 而 $u\neq\mathrm{top}(v)$，则 u 是 $\mathrm{top}(v)$ 的真后代，从 u 到 $\mathrm{top}(v)$ 的路径第一步就到达 $p(u)$；由运行交集，$p(u)\in T_v$，所以 $v\in S_u$，不是 u 的内部变量。 ■

由此，构建的结果应当记录每个节点的袋 B_u、父节点 $p(u)$、接口 S_u 和内部变量 I_u。这些都是结构信息；节点上要汇总哪些局部函数、汇总出的表取什么值，在第 13章求值时确定。

12.1.5 保留变量的结构要求

第 11.1.4小节的另一种输出是保留集 $R\subseteq V$ 上的条件最小成本表 $F_R(\boldsymbol{x}_R)$。这张表以 $\boldsymbol{x}_R$ 的联合取值为索引，所以必须有某个计算单元同时看到 R 中的全部变量，并且在那之前不能把 R 中的任何变量汇总掉。

结构上采用两条要求。第一，在原始图中把 R 补成团，即在 R 的任意两个不相邻的变量之间加边，得到图 $\mathcal{G}_R$，再为 $\mathcal{G}_R$ 构建结构。第二，选取的根袋包含 R。由第 12.3.1小节的团容纳引理，$\mathcal{G}_R$ 的任一树分解都有某个袋包含 R，可以选它作根。

根袋包含 R 以后，R 中每个变量 v 的 $\mathrm{top}(v)$ 都是根，由引理 12.2，它们不会成为任何非根节点的内部变量，因而不会提前被汇总。根处只汇总 $B_r\setminus R$，留下以 $\boldsymbol{x}_R$ 为索引的表。用变量消元描述时，这相当于只消去 $V\setminus R$ 中的变量，最后保留 R（第 12.2.3小节）。

补边可能增大宽度。路径 $1-2-3$ 的树宽为 1，若 $R=\{1,3\}$，补边后得到三角形，树宽变为 2。所以带保留变量的任务，其宽度要按 $\mathcal{G}_R$ 计算，不能直接沿用不带查询时的最小树宽；输出表本身还有 $\prod_{i\in R}|\mathcal{X}_i|$ 项。

在四环算例中取 $R = \{1,3\}$。1 与 3 在环上不相邻，补边 $\{1,3\}$ 后，$\{1,2,3\},\{1,3,4\}$ 构成宽度为 2 的分解，两个袋都含 R，任选一个作根即可。原来使用的分解 $\{1,2,4\}-\{2,3,4\}$ 则没有袋同时含 1 和 3，不能直接回答这个查询。两个边界情形是：$R=\varnothing$ 时不加任何要求；$R=V$ 时 $\mathcal{G}_R$ 是完全图，任何分解都有一个袋含全部变量，宽度为 $n-1$，这与输出表本身就要列出全部配置一致。

12.2 由变量消元构建动态规划

第 11.3.1小节的消元操作每一步都产生一个新函数，它的作用域由当前所有含该变量的函数共同决定。本节说明，这些作用域可以只在原始图上模拟消元就预先算出，不需要任何函数值。按一个次序模拟一遍，就得到每一步同时处理的变量组和新函数交给哪一步使用，把它们连起来就是一个树分解。

12.2.1 消元次序与填充图

变量消元需要事先给定一个**消元次序**（elimination order）π，即全部要消去的变量的一个排列。消去一个变量时，新函数 ψ_i 的作用域 S_i 由所有含 i 的函数共同决定。在原始图上看，S_i 恰好是 i 在当前原始图中的邻居集合，因为两个变量相邻当且仅当它们同在某个当前函数的作用域中。新函数读取这些邻居的联合取值，所以消元后的原始图中，这些邻居两两相邻。

一次消元的图操作　在三元作用域例子中先消去 x_2，就能看到这一点。ϕ_A 与 ϕ_B 都含 2，$S_2=\{1,3\}\cup\{3,4\}=\{1,3,4\}$，新函数

$$\psi_2(x_1,x_3,x_4)=\min_{x_2}\bigl[\phi_A(x_1,x_2,x_3)+\phi_B(x_2,x_3,x_4)\bigr]$$

同时读取 x_1,x_3,x_4。按 (x_1,x_3,x_4) 的字典序，它的八个值为

$$\psi_2=(3,\ 2,\ 0,\ +\infty,\ 2,\ 1,\ 2,\ 4),$$

例如 $(x_1,x_3,x_4)=(0,0,0)$ 处的两个候选为 $\phi_A(000)+\phi_B(000)=3$ 和 $\phi_A(010)+\phi_B(100)=+\infty$。原始图中原来没有边 $\{1,4\}$，消元后必须补上，如图 12.5所示。这样为记录新函数的联合依赖而补入的边称为**填充边**（fill-in edge）。

填充边不表示原问题增加了约束，而是把原来经过 x_2 的间接联系直接记下来。看 $x_3=1$ 的情形：$x_1=0$ 时，$\phi_A(0,1,1)=+\infty$ 迫使 $x_2=0$，而 $\phi_B(0,1,1)=+\infty$ 又禁止 $x_4=1$；$x_1=1$ 时 x_2 可以取 1，$x_4=1$ 仍然可行。原问题中 x_1 通过 x_2 牵制 x_4。

消去 x_2 以后，这种牵制只能由 ψ_2 自己记录：上面的值中 $\psi_2(0,1,1)=+\infty$，而 $\psi_2(0,1,0)=0$、$\psi_2(1,1,1)=4$ 都有限。若 ψ_2 能拆成一个 (x_1,x_3) 的函数与一个 (x_3,x_4) 的函数之和，固定 $x_3=1$ 后就有 $\psi_2(x_1,1,x_4)=g(x_1)+h(x_4)$；由 $g(0)+h(1)=+\infty$ 知 $g(0)$ 或 $h(1)$ 为 $+\infty$，与 $g(0)+h(0)$、$g(1)+h(1)$ 都有限矛盾。所以这张表必须同时以 x_1 和 x_4 为索引，填充边 $\{1,4\}$ 反映的正是这一点。四环算例中消去 x_1 补入的边 $\{2,4\}$ 也是如此（第 11.3.2小节）。

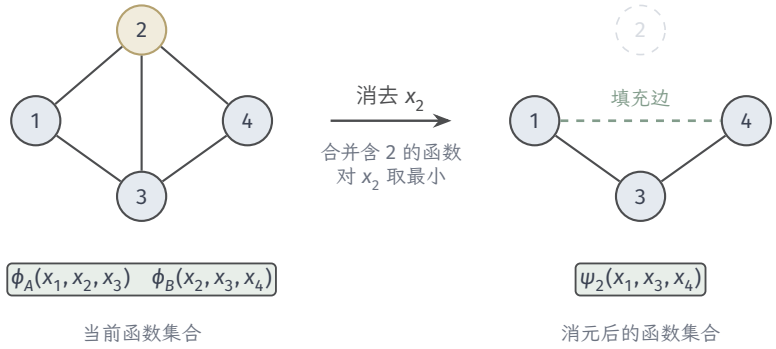


图 12.5 消去 x_2 时的函数更新与图更新。左：消元前，黄色的 x_2 同时属于 ϕ_A 与 ϕ_B 的作用域，它的邻居是 $1,3,4$。右：消元后，两个函数被新函数 $\psi_2(x_1,x_3,x_4)$ 取代，x_2 及其关联边被删除，邻居 $1,3,4$ 两两相邻，其中绿色虚线 $\{1,4\}$ 是填充边。

一般地，消元在原始图上对应一个纯粹的结构操作：把 i 的当前邻居两两连边，然后删去 i 及其关联边。不含 i 的函数提供的边保持不变；含 i 的函数在剩余变量之间提供的边都落在 S_i 内，已被补成的团覆盖；新函数 ψ_i 提供的边恰是 S_i 内的全部点对。因此，消元后函数集合的原始图恰好是对原始图执行这一操作的结果。对消元步数归纳，每一步的当前图都是当前函数集合的原始图，于是只看这个图操作，就能在不计算任何数值的情况下预先知道每一步的 S_i。

填充图 按次序 π 执行全部消元步骤，把途中补入的所有填充边加入原始图，得到的图称为 π 的**填充图**（filled graph），记为 $\mathcal{G}_\pi^+$。它记录了这次消元中出现过的全部耦合：原始边来自局部函数，填充边来自中间函数。四环算例按次序 $(1,2,3,4)$ 消元时只补入 $\{2,4\}$，填充图是环加上弦 $\{2,4\}$。第 12.5.2小节会说明填充图总是弦图。

作用域的结构上界 图操作只预测新函数可能同时读取哪些变量，不计算新函数的取值，而对特殊的局部函数，新函数可能仍能进一步分解。例如，若 $\phi_A(x_1,x_2,x_3)=g(x_1)+h(x_2,x_3)$，则

$$\psi_2(x_1,x_3,x_4)=g(x_1)+\min_{x_2}\bigl[h(x_2,x_3)+\phi_B(x_2,x_3,x_4)\bigr],$$

它是一个 x_1 的函数与一个 (x_3, x_4) 的函数之和，实际上并不需要同时索引 x_1 与 x_4。但这时 ϕ_A 本可以作为两个作用域更小的局部函数输入；只给作用域 $\{1,2,3\}$ 时，填充边不知道这种数值巧合。所以补边图给出的是作用域给定、数值任意时的作用域上界，这正是第 12.1.1小节所说的结构记账。

12.2.2 消元簇与中间作用域

设消元次序为 $\pi = (v_1, \dots, v_n)$。消去 v_i 时，记它在当前图中的邻居集合为 S_i，也记作 $N_\pi^+(v_i)$；上标 $+$ 表示这些邻居都排在 v_i 之后，因为排在前面的顶点已从当前图中删去。称

$$B_i = \{v_i\} \cup S_i$$

为第 i 步的**消元簇**（elimination cluster）；以顶点本身作下标时，也写作 B_{v_i}、S_{v_i}。它是这一步同时处理的全部变量：计算新函数 ψ_i 要枚举 $\boldsymbol{x}_{B_i}$ 的全部取值，而 ψ_i 的作用域是 S_i。

消元簇有两条结构性质。第一，B_i 在填充图中是团：S_i 在第 i 步被补成团，v_i 与 S_i 中每个顶点相邻。第二，原始图中排在 v_i 之后的邻居都在 S_i 中，因为在处理 v_i 之前只删去过已处理的顶点及其关联边。

消元次序 π 下的最大剩余邻居数 $\max_i |S_i|$ 称为 π 的**诱导宽度**（induced width），它等于最大消元簇的大小减一。诱导宽度决定了这次消元中最多同时处理多少个变量。

在四环算例中按 $(1,2,3,4)$ 消元：消去 1 时 $S_1 = \{2,4\}$，补入 $\{2,4\}$，消元簇为 $\{1,2,4\}$；消去 2 时当前邻居为 $\{3,4\}$，其中 4 来自刚补入的填充边，消元簇为 $\{2,3,4\}$；随后依次得到 $\{3,4\}$ 和 $\{4\}$。诱导宽度为 2。三元作用域例子按 $(1,4,2,3)$ 消元，消元簇依次为 $\{1,2,3\}, \{2,3,4\}, \{2,3\}, \{3\}$，诱导宽度也是 2。

12.2.3 消元森林的构造

12.2.3.1 问题与思路

输入是非空有限无向图 $\mathcal{G} = (V, E)$（通常是局部函数的原始图）和一个消元次序；若有保留集 R，图取补边后的 $\mathcal{G}_R$，次序只排列 $V \setminus R$。要求输出一个树分解，其袋是各步的消元簇，宽度等于诱导宽度。

第 12.1.2小节已经从消元步骤读出过袋，这里把那个做法变成算法。消去 v_i 时同时处理的变量构成袋 B_i。新函数 ψ_i 的作用域是 S_i，它要等到 S_i 中第一个被消去的变量才会被用到，所以 B_i 应当连到那个变量的袋上。这正是图 12.1中按“新函数交给哪一步使用”连接袋的做法。

父袋容纳剩余作用域 这种连接之所以合适，是因为父袋装得下整个 S_i。设 v_j 是 S_i 中最早被消去的变量。第 i 步把 S_i 补成团；此后直到处理 v_j 之前，S_i 中没有顶点被删去，因为它们都不早于 v_j。所以处理 v_j 时，$S_i \setminus \{v_j\}$ 中的顶点都是 v_j 的当前邻居，即 $S_i \setminus \{v_j\} \subseteq S_j$，从而

$$S_i \subseteq B_j.$$

也就是说，ψ_i 交给的那一步恰好会枚举 ψ_i 所需的全部变量。

保留变量与分量 有保留集 R 时，R 中的变量不被消去，所有袋最后汇到一个根袋 $B_R = R$。若 S_i 含待消去的变量，父袋仍取其中最早被消去者的袋，上面的论证不变，因为 R 中的变量从不删除。若 S_i 非空且全在 R 内，ψ_i 只读取保留变量，就把 B_i 连到根袋 B_R。

若 $S_i = \varnothing$，ψ_i 是一个常数，B_i 没有父袋，成为一个分量的根。原图不连通时会出现多个这样的根；再用任意边把这些根（以及根袋 B_R，若有）连成一棵树，这些边的接口为空，只用来把各分量的常数汇总起来。

12.2.3.2 算法实现

原图和次序在运行中不变；事先为每个顶点记下它在次序中的位置，以便比较先后。算法维护三项状态：

- 当前图 $\mathcal{H}$：用邻接集合存储，初值为 $\mathcal{G}$（有保留集时为 $\mathcal{G}_R$）的副本；
- 袋表：为每个已处理的顶点 v 记录袋 B_v，初值为空；
- 父袋表：为每个袋记录它在袋树中的父袋，初值为空。

按次序逐个处理顶点，处理 v 时依次做四件事：

- 读邻居：读出 v 在当前图 $\mathcal{H}$ 中的邻居集合 $N_\pi^+(v)$；
- 记袋：记录 $B_v = \{v\} \cup N_\pi^+(v)$；
- 补团：把 $N_\pi^+(v)$ 中尚不相邻的顶点两两连边，这些边就是填充边；
- 删点：从 $\mathcal{H}$ 中删去 v 及其关联边。

这正是第 12.2.1小节中消元在原始图上的结构操作。关键在于每一轮都要从更新后的图读取邻居，而不是从原图读取。全部待消去的顶点处理完后，再连接袋：

- 对每个 $N_\pi^+(v)$ 含待消去顶点的 v，令 $p(v)$ 为其中在次序里最早的顶点，把 $B_{p(v)}$ 记为 B_v 的父袋；
- 对每个 $N_\pi^+(v)$ 非空且包含于 R 的 v，把根袋 B_R 记为父袋；
- $N_\pi^+(v)$ 为空的袋没有父袋，作为分量的根；若有多个根（连同 B_R），用任意边把它们连成一棵树，这些边的接口为空。

不加空接口边时，这些袋和父袋连接按分量组成一片森林，称为 π 的**消元森林**（elimination forest）；加上空接口边后，它成为一棵袋树。

算法 12.1　由消元次序构造树分解

输入：非空有限无向图 $\mathcal{G}$，可选的保留集 R，以及 $V \setminus R$ 的一个消元次序 π。
输出：$\mathcal{G}_R$ 的树分解，有保留集时根袋为 R。

1. 令 $\mathcal{H}$ 为 $\mathcal{G}_R$ 的副本（$R = \varnothing$ 时即 $\mathcal{G}$），记下每个顶点在 π 中的位置。
2. 按 π 依次取顶点 v：读出 v 在 $\mathcal{H}$ 中的邻居集合 $N_\pi^+(v)$，记录 $B_v = \{v\} \cup N_\pi^+(v)$；把 $N_\pi^+(v)$ 中不相邻的顶点两两连边；从 $\mathcal{H}$ 中删去 v。
3. 若 $R \neq \varnothing$，建立根袋 $B_R = R$。
4. 对每个 v：若 $N_\pi^+(v)$ 含待消去的顶点，取其中位置最早的 $p(v)$，连接 B_v 与 $B_{p(v)}$；若 $N_\pi^+(v)$ 非空且包含于 R，连接 B_v 与 B_R；若 $N_\pi^+(v) = \varnothing$，B_v 作为一个根。
5. 若有多个根（连同 B_R），用任意边把它们连成一棵树。

执行过程　图 12.6的例子有边 $\{1,2\},\{1,4\},\{2,3\},\{3,4\},\{3,5\},\{4,5\}$，不保留变量，次序为 $(1,2,5,3,4)$，表 12.1逐轮列出状态的变化。

- 处理 1：邻居为 $\{2,4\}$，二者不相邻，补入填充边 $\{2,4\}$，袋为 $\{1,2,4\}$。
- 处理 2：当前邻居 $\{3,4\}$ 中的 4 来自上一轮补入的填充边，袋为 $\{2,3,4\}$。若误从原图读取，袋就成了 $\{2,3\}$，漏掉了消去 x_1 后新函数同时读取的 x_2, x_4。
- 处理 5：邻居 $\{3,4\}$ 已经相邻，不补边，袋为 $\{3,4,5\}$。
- 处理 3 与 4：分别得到袋 $\{3,4\}$ 与 $\{4\}$，不再补边。

连接时，B_1 的后续邻居中最早的是 2，连到 B_2；B_2 与 B_5 的后续邻居都是 $\{3,4\}$，连到 B_3；B_3 连到 B_4；B_4 没有后续邻居，作为根。得到以 $B_4 = \{4\}$ 为根的袋树，最大袋含 3 个顶点，宽度为 2。

表 12.1　按次序 $(1,2,5,3,4)$ 构造树分解时每一轮的状态。“当前邻居”从上一轮更新后的图读取；父袋在全部轮次结束后按“后续邻居中次序最早者”确定，最后一列是与父袋的接口。

轮	处理	当前邻居	填充边	袋	父袋	接口
1	1	$\{2,4\}$	$\{2,4\}$	$\{1,2,4\}$	B_2	$\{2,4\}$
2	2	$\{3,4\}$	无	$\{2,3,4\}$	B_3	$\{3,4\}$
3	5	$\{3,4\}$	无	$\{3,4,5\}$	B_3	$\{3,4\}$
4	3	$\{4\}$	无	$\{3,4\}$	B_4	$\{4\}$
5	4	$\varnothing$	无	$\{4\}$	根	—

表中每个非根袋与父袋的接口恰好等于它的后续邻居集合。这不是巧合：父袋容纳 S_v，而 v 本身比父袋的顶点都早被处理，不在父袋中，所以 $B_v \cap B_{p(v)} = S_v$。第 12.4.1小节会利用这一点把中间函数与袋之间的消息对应起来。

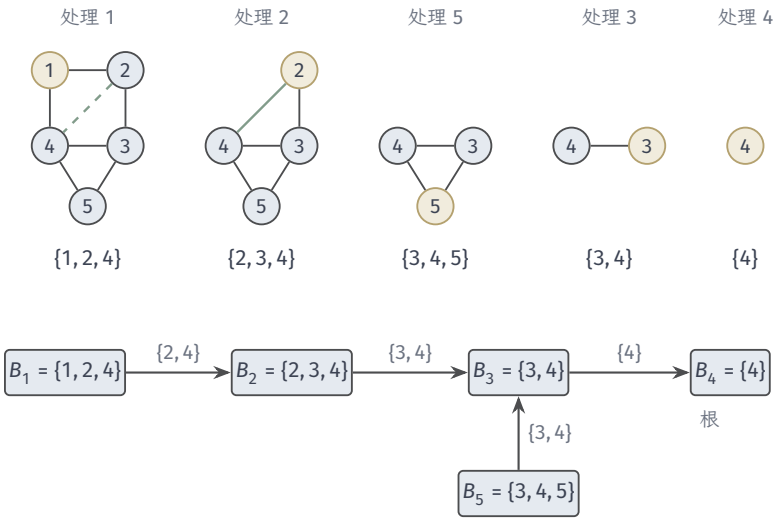


图 12.6 由消元次序 $(1, 2, 5, 3, 4)$ 构造树分解。上行：每一轮的当前图，黄色为正在处理的顶点，绿色虚线为本轮补入的填充边，下一轮中它以绿色实线留在当前图里；图下标出记录的袋。下：按“连到最早被处理的后续邻居”连接父袋得到的袋树，箭头指向父袋，边旁标出接口。

12.2.4 从消元簇得到树分解

下面的定理说明上述构造总是给出树分解，从而把消元过程中生成的依赖正式转成树分解，而不是另设一套独立的构建方法。

定理 12.3 消元次序产生树分解 设 $\mathcal{G}$ 是非空有限无向图，$R \subseteq V$，π 是 $V \setminus R$ 的消元次序。按算法 12.1得到的袋树是 $\mathcal{G}_R$ 的树分解，有保留集时根袋 $B_R = R$；其宽度为 π 的诱导宽度与 $|R| - 1$ 中的较大者，无保留集时就是诱导宽度。特别地，$\mathcal{G}$ 的树宽不超过任何完整消元次序的诱导宽度。

证明 先说明袋树是树。每个非根袋恰有一条通向父袋的边。父袋若为 $B_{p(v)}$，则 $p(v)$ 在次序中排在 v 之后，沿父边前进时次序位置严格增大；父袋若为 B_R，则到达根袋。所以沿父边前进不会回到出发点，不形成环，每条父链都在有限步后到达一个根。各根之间再用树边连接，整体连通且无环。

顶点覆盖：待消去的 v 属于 B_v，R 中的顶点属于 B_R。边覆盖：设 $\{u, v\}$ 是 $\mathcal{G}_R$ 的边。若两端都在 R 中，它被 B_R 覆盖。否则设 u 是两端中先被处理的待消去顶点；在处理 u 之前，图中只删去过已处理的顶点及其关联边，所以边 $\{u, v\}$ 仍在，$v \in N_\pi^+(u)$，于是 B_u 同时含 u, v。

运行交集：设 $z \in B_v$ 且 $z \neq v$，只需证明从 B_v 沿父链向上的每个袋都含 z，直到到达 B_z（z 待消去时）或 B_R（$z \in R$ 时）。这样含 z 的每个袋都与 B_z 或 B_R 由一条全部含 z 的路径相连，T_z 连通。由 $z \in N_\pi^+(v) = S_v$ 和上一小节的“父袋容纳剩余作用域”，父袋包含 S_v，所以含 z；若父袋是 $B_{p(v)}$ 且 $p(v) \neq z$，就对 $p(v)$ 重复同样的论证。次序位置严格增大，最终到达 B_z 或 B_R。

宽度：待消去顶点的袋大小为 $|S_v| + 1$，根袋大小为 $|R|$，所以宽度如定理所述。最后，无保留集时树宽是全部树分解宽度的最小值，所以不超过诱导宽度。 ■

证明中的边覆盖论证同样适用于填充边，因此所得袋树也是填充图 $\mathcal{G}_\pi^+$ 的树分解。另一个直接推论是：每个局部函数的作用域 S_a 都整体落在某个袋中。取 S_a 中最先被处理的待消去变量 v，S_a 的其余变量在原始图中都与 v 相邻且排在 v 之后（或属于 R），所以都在 S_v 中，$S_a \subseteq B_v$；若 $S_a \subseteq R$，则 $S_a \subseteq B_R$。第 13.1.1小节据此为局部项指定归属。

结构操作的代价　设处理第 t 个顶点时它有 d_t 个当前邻居，补边要检查 $\sum_t d_t(d_t-1)/2$ 对顶点，记录的袋共含 $\sum_t (d_t+1)$ 个顶点。用支持常数时间查边和加边的邻接集合实现时，时间为 $O\big(n + m + \sum_t d_t^2\big)$，其中 m 是原始图的边数。这只是结构操作的开销；在袋上计算数值表时，开销随袋大小指数增长，第 13.5节另行计算。

12.3　由树分解构建动态规划

上一节从消元次序出发得到树分解。本节反过来，从一个给定的树分解出发，直接规定局部子问题。需要证明的有两件事：每个局部项都能整体放进某个袋，这样它才有处可放；相邻两袋的接口确实把两侧的变量隔开，这样以接口为索引的摘要才足够。最后从叶袋向内处理变量，读出一个与分解相容的消元次序，说明树分解既能作为构建的输入，也能作为消元结果的显式表示。

12.3.1　包含局部作用域的袋

计算时每个局部函数要整个放进一个袋，因为袋表需要同时索引它的全部变量。一个作用域 S_a 在原始图上是团：其中任意两个变量同在 ϕ_a 中，因而相邻。但树分解的定义只要求每条边被某个袋覆盖，三个变量两两成边时，三条边似乎可能分别落在三个不同的袋中。下面先证明一个关于子树的一般事实，再由它推出这种情况不会发生。

把每个顶点 v 的出现位置 T_v 看成袋树中的一棵连通子树。若 u,v 相邻，边覆盖给出一个同时含 u,v 的袋，于是 T_u 与 T_v 相交。对一个团 C，各 T_v（$v\in C$）两两相交；要证明的是它们有一个公共节点。

引理 12.4　两两相交的子树有公共节点　设 $\mathcal{T}$ 是有限树，$T_1,\dots,T_k$（$k\ge 1$）是它的非空连通子树，并且两两相交。则存在节点同时属于全部 T_j。

证明　任取 $\mathcal{T}$ 的一个节点作为根。与引理 12.2的证明相同，每个 T_j 有唯一一个离根最近的节点，记为 r_j，并且 T_j 的每个节点都是 r_j 的后代（含 r_j 本身）。取 j^* 使 r_{j^*} 的深度在 $r_1,\dots,r_k$ 中最大，下面证明 r_{j^*} 属于每个 T_l。

$l=j^*$ 时结论由定义成立。$l\neq j^*$ 时，$T_l\cap T_{j^*}$ 中有某个节点 x；$x\in T_{j^*}$，因而 x 是 r_{j^*} 的后代。考察 T_l 中从 x 到 r_l 的路径，它留在 T_l 中。若 r_l 不是 r_{j^*} 的后代，这条路径从 r_{j^*} 下方的子树走到外部，必须经过 r_{j^*}，因为这棵子树与外部只通过 r_{j^*} 与其父节点之间的边相连；于是 $r_{j^*}\in T_l$。若 r_l 是 r_{j^*} 的后代，则 r_l 的深度不小于 r_{j^*} 的深度；而按 j^* 的选取，r_l 的深度又不大于 r_{j^*} 的深度，所以 $r_l=r_{j^*}$，同样有 $r_{j^*}\in T_l$。 ■

引理 12.5　团被某个袋容纳　设 $(\mathcal{T},\{B_t\}_{t\in T})$ 是 $\mathcal{G}$ 的树分解，$C\subseteq V$ 是 $\mathcal{G}$ 中的非空团，则存在节点 $t\in T$ 使 $C\subseteq B_t$。

证明　由顶点覆盖与运行交集，每个 T_v（$v\in C$）是非空连通子树。C 中任意两个顶点 u,v 相邻，由边覆盖，$T_u\cap T_v\neq\varnothing$。由引理 12.4，存在节点 t 属于全部 T_v，即 $C\subseteq B_t$。 ■

图 12.7画出了证明中的几棵子树：每个变量的出现位置向上延伸到自己的最高节点，最高节点最深的那个变量，它的最高节点被其余各子树“经过”，因而就是要找的袋。

由引理 12.5，原始图的任一树分解中，每个局部函数的完整作用域都被某个袋整体容纳。因此在原始图上构造树分解就足够了，不需要额外记录高阶作用域；第 11.1节所说的“结构以原始图为依据、计算保留完整函数”在这里得到保证。要找出容纳 S_a 的袋，只需在 T_v（$v\in S_a$）中最小的一个里逐个检查袋是否包含 S_a。同样，第 12.1.5小节补成团的保留集 R 也一定被某个袋容纳。

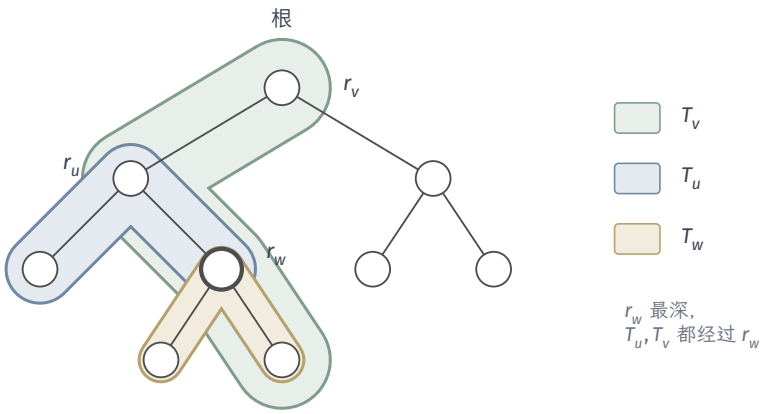


图 12.7 团 $\{u, v, w\}$ 被某个袋容纳的论证。袋树以顶部节点为根，三种颜色的区域分别是 T_u、T_v、T_w，标签 r_u, r_v, r_w 标出各自离根最近的节点。r_w 最深；T_u、T_v 都要与 T_w 相交，而它们的最高节点不比 r_w 深，所以必须经过 r_w。粗圈节点 r_w 的袋同时含 u, v, w。

树宽的下界 引理 12.4也给出第 12.1.3小节几个例子的下界。一条边要求某个袋含两个顶点，所以含边的图宽度不能小于 1；至少含一条边的树因而树宽恰为 1。

四个顶点的环 $1-2-3-4-1$ 没有宽度为 1 的树分解。给任一树分解选根，取环上的顶点 w 使 T_w 的最高节点最深。w 在环上的两个邻居 u, v 各自与 w 相邻，于是 T_u、T_v 都与 T_w 相交；引理证明中的论证表明，T_w 的最高节点同时属于 T_u 与 T_v。这个袋含 u, v, w 三个顶点，所以宽度至少为 2，四环的树宽恰为 2。

这一论证对任何环都适用：含环的图树宽至少为 2。因此树宽为 1 的图恰好是至少含一条边的森林。森林的各个分量分别按树的方法分解，再用任意的边把各分量的袋树连成一棵树；不同分量没有公共顶点，这些连接不影响运行交集。图 12.2的梯子含环，又有宽度为 2 的分解，所以树宽恰为 2；按同样的方式扫描，任意长的梯子树宽都是 2。

四个顶点两两相邻的完全图恰好相反。整个顶点集是一个团，由引理 12.5，任何树分解都有一个袋装下全部四个顶点，宽度至少为 3，所以树宽为 3。同样，n 个顶点的完全图树宽为 $n-1$，树分解对它没有任何节省。

可见树宽衡量的是图与树的接近程度：一条边需要两个变量同时出现，一个环迫使某处至少三个变量同时出现，而一个大团迫使它的全部变量同时出现。

12.3.2 接口的分隔性质

树上消息的正确性依赖于一件事：删去一条边后，两侧除了接收变量以外没有共同的变量。袋树上，这件事由运行交集保证。删去袋树边 $\{s,t\}$ 后，袋树分成两部分；记 s 一侧全部袋中出现的变量为 $V_{s|t}$，t 一侧为 $V_{t|s}$，接口为 $B_s \cap B_t$。

若某个变量 v 在两侧都出现，从它在 s 侧的某个出现位置到 t 侧的某个出现位置，袋树路径必须经过边 $\{s,t\}$；由运行交集，B_s 与 B_t 都含 v。所以

$$V_{s|t} \cap V_{t|s} \subseteq B_s \cap B_t, \tag{12.1}$$

即接口把两侧的变量隔开：固定接口取值后，两侧的其余变量互不重叠。

只有变量不重叠还不够，还要确认两侧没有被某个局部项直接连在一起，否则两侧的其余变量仍不能分别优化。下面的引理说明，每个局部项都整体落在某一侧。

引理 12.6　接口分隔两侧　设 $(\mathcal{T},\{B_t\}_{t\in T})$ 是原始图的树分解，$\{s,t\}$ 是袋树的边。则式 (12.1)成立；并且对每个局部函数 ϕ_a，$S_a \subseteq V_{s|t}$ 或 $S_a \subseteq V_{t|s}$。因此，任何局部项都不同时读取 $V_{s|t}\setminus B_t$ 中的变量与 $V_{t|s}\setminus B_s$ 中的变量。

证明　式 (12.1)已在上面证明。由引理 12.5，S_a 被某个袋 B_x 容纳，x 在 s 一侧或 t 一侧，于是 $S_a \subseteq V_{s|t}$ 或 $S_a \subseteq V_{t|s}$。若某个 ϕ_a 同时读取 $y \in V_{s|t}\setminus B_t$ 与 $z \in V_{t|s}\setminus B_s$，不妨设 $S_a \subseteq V_{s|t}$，则 $z \in V_{s|t}\cap V_{t|s} \subseteq B_s$，与 $z \notin B_s$ 矛盾；另一种情形同理。 ■

在选了根的袋树中，取 $t = p(u)$。u 下方各袋（含 u 本身）中出现而不在接口 S_u 中的变量，由式 (12.1)不会出现在其余袋中，它们正是 u 下方各节点的内部变量之并。引理 12.6说明，固定 $\boldsymbol{x}_{S_u}$ 后，这些变量与其余变量之间没有遗漏的直接耦合，所以 u 下方可以单独摘要成一张以 $\boldsymbol{x}_{S_u}$ 为索引的表。这正是第 11.3.3小节的第二项要求；第 13.2.5小节用它证明数值递推的正确性。

12.3.3 从叶袋得到消元序

给定一个宽度为 w 的树分解，可以反过来读出一个诱导宽度不超过 w 的消元次序。思路是从叶袋开始：叶袋中不在父袋里的变量只出现在这个叶袋中，消去它们时，它们的邻居都在叶袋里，产生的填充边也都落在叶袋里。

算法　输入是图 G 的一个树分解，选定一个根；输出是全部顶点的一个消元次序。维护当前图（初值为 G）、当前袋树（初值为输入分解）和已输出的次序（初值为空）。当前袋树不只一个节点时，反复处理一个非根叶节点 ℓ，记其父节点为 p：

- 若 $B_\ell \subseteq B_p$，直接从袋树中删去 ℓ；
- 否则按任意次序依次消去 $B_\ell \setminus B_p$ 中的变量：把每个变量追加到次序末尾，在当前图中把它的邻居补成团并删去它，同时把它从 B_ℓ 中删去；此时 $B_\ell \subseteq B_p$，再删去 ℓ。

最后只剩根袋，按任意次序消去其中剩余的变量。

算法 12.2　由树分解读出消元次序

输入：有限无向图 G 的树分解，以及选定的根。输出：G 的一个消元次序。

1. 令当前图为 G 的副本，当前袋树为输入的分解，次序为空。
2. 若当前袋树只剩根，转第 4 步。否则取一个非根叶节点 ℓ 及其父节点 p。
3. 依次消去 $B_\ell \setminus B_p$ 中的变量（追加到次序、补团、从当前图和 B_ℓ 中删去），然后删去节点 ℓ，回到第 2 步。
4. 按任意次序消去根袋中剩余的变量，输出次序。

正确性　算法维持一个不变量：当前袋树（各袋只保留尚未消去的变量）是当前图的树分解。开始时它就是输入分解。

设叶节点 ℓ 中有变量 $v \notin B_p$。由运行交集，T_v 连通，而 ℓ 在袋树中只与 p 相邻、$p \notin T_v$，所以 $T_v = \{\ell\}$：v 只出现在 B_ℓ 中。由边覆盖，v 在当前图中的每条边都被某个含 v 的袋覆盖，只能是 B_ℓ，所以 v 的当前邻居全在 B_ℓ 中，个数至多 $|B_\ell| - 1 \le w$。消去 v 补入的填充边两端都在 B_ℓ 中，被 B_ℓ 覆盖；从 B_ℓ 中删去 v 后，三个条件对新的当前图仍然成立。

$B_\ell \subseteq B_p$ 时删去 ℓ 也保持不变量：ℓ 覆盖的顶点和边都被 B_p 覆盖；含某个变量的节点集合去掉一个叶节点后仍然连通。最后剩下根袋时，当前图的全部顶点都在根袋中，每个顶点的当前邻居至多 $|B_r| - 1 \le w$ 个。

所以每个顶点被消去时，剩余邻居数都不超过 w，得到的消元次序诱导宽度不超过分解的宽度。

执行过程　以图 12.2的梯子为例，以 $\{3,5,6\}$ 为根。叶袋 $\{1,2,4\}$ 的父袋是 $\{2,4,5\}$，消去 1：它的当前邻居为 $\{2,4\}$，补入 $\{2,4\}$；剩余的 $\{2,4\}$ 包含于父袋，删去该叶。接着 $\{2,4,5\}$ 成为叶，消去 4：当前邻居为 $\{2,5\}$，已相邻。再处理 $\{2,3,5\}$，消去 2：当前邻居为 $\{3,5\}$，补入 $\{3,5\}$。最后在根袋中依次消去 3, 5, 6。

得到的次序为 $(1,4,2,3,5,6)$，各步剩余邻居数依次为 $2,2,2,2,1,0$，诱导宽度为 2，等于分解的宽度。每一步补入的填充边都落在当时的叶袋里，正如正确性论证所说。

12.4 两种构建的等价关系

前两节分别从消元次序和树分解出发，构建出同一类结构。本节把两种表示逐项对照：消元簇对应袋，剩余作用域对应接口，中间函数对应向父袋传递的消息。比较规模时，要区分某一个构建的宽度与全部构建中的最优值；后者在两种表示下相等。最后在四环上把双向转换逐步做一遍。

12.4.1 中间函数与接口消息的对应

在算法 12.1得到的袋树中，取消元森林的根为根。每个非根袋 $B_i=\{v_i\}\cup S_i$ 与父袋的接口恰为 S_i：第 12.2.3小节已说明父袋包含 S_i；另一方面，父袋若是 $B_j=\{v_j\}\cup S_j$，其中的变量在处理 v_j 时都还在当前图中，而 v_i 此前已被删去，所以 $v_i\notin B_j$，父袋若是根袋 B_R，也因 $v_i\notin R$ 而不含 v_i。所以 B_i 的内部变量只有 v_i 一个，这与引理 12.2一致：v_i 所在的最高节点就是 B_i。

于是两种表示中的对象逐项对应：

- 第 i 步的消元簇 B_i 对应袋树的一个节点；
- 中间函数 ψ_i 的作用域 S_i 对应该节点与父节点的接口；
- 消去 v_i 对应汇总该节点的内部变量；
- “ψ_i 在消去 S_i 中最早的变量时被使用”对应“消息从子节点发往父节点”。

第 13.1.3小节在相同的局部项归属下证明，按消元次序算出的 ψ_i 与沿袋树算出的消息逐项相等。

反过来，算法 12.2从树分解读出消元次序时，处理叶袋 ℓ 就是一次消息传递：消去 $B_\ell\setminus B_p$ 中的变量，留下以 $B_\ell\cap B_p$ 为作用域的结果交给父袋。一个袋可以有多个内部变量，相当于把连续的几步消元合并在一个计算单元里完成。

对应关系不要求逐表相同。把相邻的袋合并、把一个局部项从一个容纳它的袋移到另一个，都会改变中间表；只要结果仍是合法的树分解、每个局部项的作用域被所归的袋容纳、每项恰好归入一处，最终的最小成本、条件最小成本表、计数和加权总和都不变。这一点在第 13章随各任务的正确性一起证明。

两种表示的组织差别　两种做法的差别在于组织方式。变量消元边走边决定新函数交给谁，结构与数值交织在一起；树分解先把结构确定下来并核验，同一个分解可以用于作用域相同而数值不同的多组局部函数，也便于

在不同的根下重复使用，例如第 11.2.3.4小节那样再发一遍反向消息，得到每个袋上的表。

12.4.2　树宽与最小诱导宽度

定理 12.3给出一个方向：每个消元次序都产生宽度等于其诱导宽度的树分解。第 12.3.3小节给出另一个方向：每个宽度为 w 的树分解都读出诱导宽度不超过 w 的消元次序。两个方向合起来，得到下面的等式。

定理 12.7　树宽等于最小诱导宽度　设 $\mathcal{G}$ 是非空有限无向图。则

$$\operatorname{tw}(\mathcal{G}) = \min_{\pi} \max_{v} |N_{\pi}^{+}(v)|,$$

其中 π 取遍全部顶点的消元次序，$N_{\pi}^{+}(v)$ 是按 π 消元时 v 的剩余邻居集合。

证明　记右端为 w^{*}，它是有限个次序上的最小值，因而可以达到。由定理 12.3，诱导宽度为 w^{*} 的次序给出宽度为 w^{*} 的树分解，所以 $\operatorname{tw}(\mathcal{G}) \le w^{*}$。反过来，取宽度为 $\operatorname{tw}(\mathcal{G})$ 的树分解，由算法 12.2及其正确性论证，得到诱导宽度不超过 $\operatorname{tw}(\mathcal{G})$ 的次序，所以 $w^{*} \le \operatorname{tw}(\mathcal{G})$。 ■

这个等式说的是两种表示的最优值相等，不是两种对象一一对应。同一个树分解按不同的根、不同的叶袋处理次序可以读出许多消元次序；不同的消元次序也可能给出合并后相同的袋树。等式的用途是：寻找好的结构时，可以在消元次序上搜索，也可以在树分解上搜索，二者的最优规模相同。对某一个具体次序或分解，它的宽度只是树宽的上界。

12.4.3　四环上的双向构建

现在在四环算例上把两个方向逐步做一遍。原始图是环 $1-2-3-4-1$。

由消元次序到袋树　按次序 $(1,2,3,4)$ 执行算法 12.1，表 12.2列出每一轮的状态。消去 1 时补入填充边 $\{2,4\}$；消去 2 时当前邻居为 $\{3,4\}$，已经相邻；之后不再补边。

得到的袋树是一条路径 $\{1,2,4\}-\{2,3,4\}-\{3,4\}-\{4\}$，宽度为 2。后两个袋包含于 $\{2,3,4\}$，把它们并入后只剩 $\{1,2,4\}-\{2,3,4\}$，接口为 $\{2,4\}$，这就是第 11.3.2小节计算时使用的两个子问题。可以逐条核对定义：四条环边 $\{1,2\},\{1,4\}$ 在第一个袋，$\{2,3\},\{3,4\}$ 在第二个袋；含 2 或 4 的袋相邻，含 1 或 3 的袋只有一个。

表 12.2 四环上按次序 $(1,2,3,4)$ 构建袋树。“当前邻居”从上一轮更新后的图读取；父袋按“后续邻居中次序最早者”确定，最后一列是与父袋的接口，它恰好等于当前邻居集合。

轮	消去	当前邻居 S_i	填充边	消元簇 B_i	父袋	接口
1	1	$\{2,4\}$	$\{2,4\}$	$\{1,2,4\}$	B_2	$\{2,4\}$
2	2	$\{3,4\}$	无	$\{2,3,4\}$	B_3	$\{3,4\}$
3	3	$\{4\}$	无	$\{3,4\}$	B_4	$\{4\}$
4	4	$\varnothing$	无	$\{4\}$	根	—

由袋树到消元次序 反过来，从分解 $\{1,2,4\}-\{2,3,4\}$ 出发执行算法 12.2。以 $\{2,3,4\}$ 为根时，叶袋 $\{1,2,4\}$ 中不在父袋里的变量是 1，先消去 1，再在根袋中按 $2,3,4$ 消去，读回次序 $(1,2,3,4)$。

以 $\{1,2,4\}$ 为根时，叶袋变成 $\{2,3,4\}$，不在父袋里的变量是 3。消去 3 时它在环上的当前邻居是 $\{2,4\}$，补入 $\{2,4\}$，这条填充边落在叶袋里；随后在根袋中按 $1,2,4$ 消去，得到次序 $(3,1,2,4)$，各步剩余邻居数为 $2,2,1,0$。两个次序补入的是同一条填充边，诱导宽度都是 2。

两种表示的对照 两种表示中，最多同时处理的变量都是三个：消元簇 $\{1,2,4\}$、$\{2,3,4\}$ 就是两个袋。由定理 12.7和第 12.3.1小节的下界，四环的树宽为 2，所以四环上任何消元次序的诱导宽度都至少是 2，这里的两个次序已经最优。

另一种同样宽度的分解是 $\{1,2,3\}-\{1,3,4\}$，接口为 $\{1,3\}$，它对应先消去 2 或 4、补入 $\{1,3\}$ 的次序。第 12.1.5小节的保留集 $R=\{1,3\}$ 要求的正是这种结构：在补边 $\{1,3\}$ 后的图上只消去 $V\setminus R$，按次序 $(2,4)$ 执行算法 12.1，消去 2 时 $S_2=\{1,3\}\subseteq R$，消去 4 时 $S_4=\{1,3\}\subseteq R$，两个消元簇 $\{1,2,3\}$ 与 $\{1,3,4\}$ 都连到根袋 $B_R=\{1,3\}$。第 13.3.6小节在这个结构上计算条件最小成本表。

12.5 构造策略

前几节说明了消元次序与树分解怎样互相转换，但没有说明怎样得到规模小的结构。本节在已经建立的等价关系上讨论实际构造：用局部规则选择消元次序；在弦图上利用不需补边的完美消元序；把弦图的极大团借第 4章的最大权生成树连成团树；最后逐项检验构建结果。本节的算法只做结构操作，输出袋和袋之间的连接，不计算数值表；输出中实际的袋大小是第 13.5节代价分析的输入。

12.5.1 消元次序的选择

12.5.1.1 次序对消元簇的影响

由定理 12.3，任何消元次序都给出合法的结构，但不同次序产生的消元簇可以相差很多。在三元作用域例子上比较两种次序：$(1,4,2,3)$ 与先消 x_2 的 $(2,1,3,4)$。前者每一步被消去的顶点的邻居都已两两相邻，不产生填充边，消元簇依次为 $\{1,2,3\},\{2,3,4\},\{2,3\},\{3\}$。后者第一步要处理 $\{1,2,3,4\}$ 并补入 $\{1,4\}$，此后的消元簇为 $\{1,3,4\},\{3,4\},\{4\}$。表 12.3列出了两种次序每一步的作用域和规模，图 12.8画出了对应的图变化。

表 12.3 三元作用域例子上两种消元次序的逐步规模。“枚举”指计算新函数时要比较的候选组合数，即消元簇上的配置数；“输出”指新函数表的项数，回溯表的项数与之相同。

次序 $(1,4,2,3)$				次序 $(2,1,3,4)$			
消去	新作用域	枚举	输出	消去	新作用域	枚举	输出
x_1	$\{2,3\}$	8	4	x_2	$\{1,3,4\}$	16	8
x_4	$\{2,3\}$	8	4	x_1	$\{3,4\}$	8	4
x_2	$\{3\}$	4	2	x_3	$\{4\}$	4	2
x_3	$\varnothing$	2	1	x_4	$\varnothing$	2	1
合计		22	11	合计		30	15

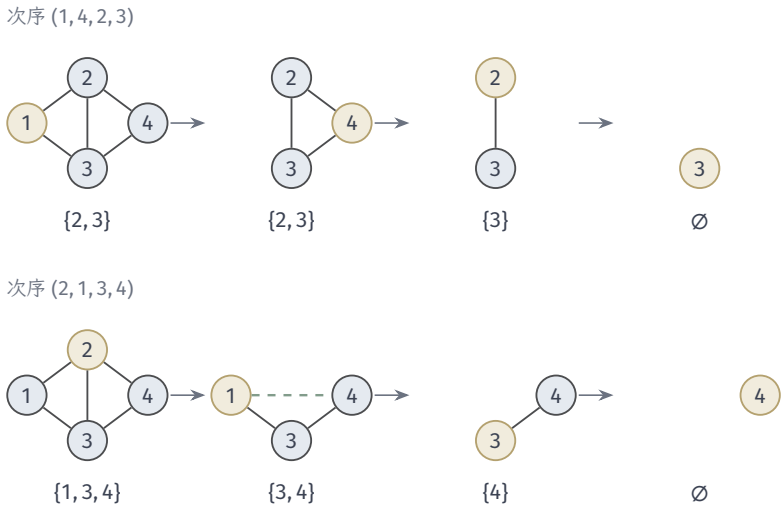


图 12.8 两种消元次序下原始图的变化。上行按 $(1,4,2,3)$ 消元，每一步被消去的黄色顶点的邻居都已两两相邻，不产生填充边，新函数至多读取两个变量。下行按 $(2,1,3,4)$ 消元，消去 2 时要补入填充边 $\{1,4\}$（第二个图中的绿色虚线），新函数读取三个变量。每个图下方标出消去黄色顶点后新函数的作用域。

第一种次序的最大中间表只有 4 项，第二种有 8 项。在这个小例子里差别不大；但若每个变量有 q 个状态，读取 r 个变量的表就有 q^r 项，多读取一个变量意味着表扩大 q 倍。

次序选得不好时，即使在树上也会产生大表：在第 11.2.3小节的分叉树上先消去中心 3，它的邻居 $1,2,4$ 两两补边，新函数要读取三个变量；而按后序消元时不产生任何填充边。两种次序算出的最小值相同（第 13.2.5小节），差别只在中间表的规模。

12.5.1.2 贪心规则

定理 12.7把找好的树分解归结为找诱导宽度小的消元次序。在一般图上找诱导宽度最小的次序是困难的组合问题，实践中常用局部规则逐轮挑选顶点，得到一个可用但不保证最优的次序。

消去一个顶点，就是把它的全部当前邻居拉进同一个袋，并让它们两两相邻。代价因此体现在两处：它的当前邻居越多，这一步的袋越大；邻居之间缺的边越多，要补的填充边越多，剩下的图越稠密，以后的袋也可能越大。据此有两条常见规则：**最小度启发式**（min-degree），选当前邻居最少的顶点；**最小填充启发式**（min-fill），选消去时需要补边最少的顶点，即当前邻域中缺少的边数最少者。两条规则都只看当前这一步。

算法实现 原图和评分规则在运行中不变。算法维护三项状态：

- 当前图：初值为原图的副本；
- 已输出的次序：初值为空；
- 各轮的袋和填充边：初值为空。

每一轮依次做三件事：

- 评分：在当前图上为每个剩余顶点计算评分，即当前度数或当前缺边数；
- 选择：取评分最小者，评分相同时取编号最小者，记为 v；
- 消去：记录袋 $\{v\}\cup N(v)$，其中 $N(v)$ 是 v 的当前邻居，把 $N(v)$ 补成团并记录填充边，删去 v，把 v 追加到次序末尾。

当前图为空时停止。

算法 12.3 贪心选择消元次序

输入：非空有限无向图和一条评分规则（当前度数或当前缺边数）。输出：消元次序、各轮的袋和填充边。

1. 对当前图中的每个顶点按规则计算评分，取评分最小者，评分相同时取编号最小者，记为 v。
2. 记录袋 $\{v\}\cup N(v)$，把 $N(v)$ 补成团，删去 v，并把 v 追加到次序末尾。
3. 若图非空，回到第 1 步，并在更新后的图上重新计算评分。

评分必须在每轮更新后的图上重算。若只按初始度数排一次序，或用原图的缺边数代替当前缺边数，得到的就不是这两条规则，因为补边会改变其他顶点的度数和邻域。

执行过程　图 12.9的图有六个顶点和边 $\{1,3\},\{1,4\},\{1,5\},\{1,6\},\{2,3\},\{2,4\},\{2,6\},\{3,5\},\{4,5\},\{5,6\}$。初始度数依次为 $4,3,3,3,4,3$；缺边数依次为 $3,3,2,2,3,2$，例如顶点 2 的邻居 $3,4,6$ 两两不相邻，缺 3 条边，而顶点 3 的邻居 $1,2,5$ 中只缺 $\{1,2\}$ 与 $\{2,5\}$。表 12.4并列两条规则的各轮结果。

最小度规则的执行如下：

- 第 1 轮：度数为 3 的有 $2,3,4,6$，按编号选 2，袋为 $\{2,3,4,6\}$，补入 $\{3,4\},\{3,6\},\{4,6\}$。
- 第 2 轮：剩下的五个顶点因此两两相邻，度数都是 4，按编号选 1，袋为 $\{1,3,4,5,6\}$，宽度升到 4。
- 第 3 至 6 轮：剩余图始终是完全图，依次选 $3,4,5,6$，不再补边。

最小填充规则的执行如下：

- 第 1 轮：缺边数为 2 的有 $3,4,6$，按编号选 3，袋为 $\{1,2,3,5\}$，补入 $\{1,2\}$ 与 $\{2,5\}$。
- 第 2 轮：4 的邻居 $1,2,5$ 已两两相邻，缺边数为 0；6 的缺边数也是 0，按编号选 4，袋为 $\{1,2,4,5\}$。
- 第 3 至 6 轮：剩下的四个顶点两两相邻，依次选 $1,2,5,6$，不再补边。

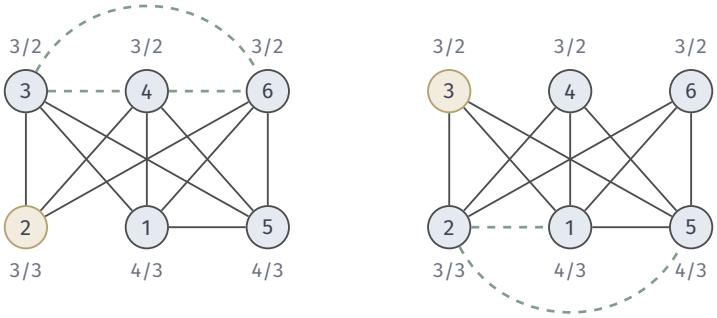
最大袋含 4 个顶点，宽度为 3，比最小度规则少 1。

表 12.4　两条贪心规则在图 12.9的六顶点图上的各轮结果。“选中”列括号内是该顶点当轮的评分（最小度规则为当前度数，最小填充规则为当前缺边数），“补边”列是该轮补入的填充边条数。

	最小度			最小填充		
轮	选中	袋	补边	选中	袋	补边
1	2（3）	$\{2,3,4,6\}$	3	3（2）	$\{1,2,3,5\}$	2
2	1（4）	$\{1,3,4,5,6\}$	0	4（0）	$\{1,2,4,5\}$	0
3	3（3）	$\{3,4,5,6\}$	0	1（0）	$\{1,2,5,6\}$	0
4	4（2）	$\{4,5,6\}$	0	2（0）	$\{2,5,6\}$	0
5	5（1）	$\{5,6\}$	0	5（0）	$\{5,6\}$	0
6	6（0）	$\{6\}$	0	6（0）	$\{6\}$	0
宽度		4			3	

12.5.1.3　贪心规则的分析

启发式的局限　两种规则都只看一步：度数小不代表补边少，补边少也不保证以后的袋小。上例中最小度规则受平局规则影响，第一步补的三条边把剩余图变成了完全图，得到的宽度比最小填充多 1。穷举全部 720 个消



(a) 最小度：选 2，补 3 条边

最终宽度 4

(b) 最小填充：选 3，补 2 条边

最终宽度 3

图 12.9 最小度规则与最小填充规则的第一步。每个顶点旁的“d/f”是当前度数与当前缺边数。左：最小度规则在度数为 3 的顶点中按编号选 2（黄色），要补三条绿色虚线，最终宽度为 4。右：最小填充规则选缺边最少的 3（黄色），只补两条边，最终宽度为 3。

元次序可以核对，这个图上任何次序的诱导宽度都不小于 3，所以最小填充在这里恰好达到了最好的次序。换一个图，最小填充同样可能不是最优。因此报告结果时应分开写出构造的宽度、填充边数和结构操作次数，并把宽度理解为树宽的上界，而不是树宽本身。

宽度按变量个数计，而表的项数按状态数之积计。各变量状态数不同时，诱导宽度相同的两个次序，其最大消元簇的配置数可以相差很多；这时可以把评分换成消元簇上的状态数之积，或者构造后直接比较各簇的实际配置数。第 13.5.1小节按实际乘积比较候选结构。

复杂度 直接实现时每轮要为全部剩余顶点重算评分。最小度规则的评分只需读出度数；最小填充规则要检查每个顶点的每对邻居是否相邻，一轮的检查量可达 $\sum_{v} |N(v)|^2$。用缓存时只重算受影响的顶点：加入边 $\{a, b\}$ 只改变 a、b 及其公共邻居的评分，删去 v 只改变 v 的邻居的评分。

12.5.2 弦图与完美消元序

填充边会使袋变大，最理想的情形是全程不需要补边。处理顶点 v 时不需要补边，当且仅当 v 的当前邻居已经两两相邻。当前邻居构成团的顶点称为**单纯顶点**（simplicial vertex）；没有邻居或只有一个邻居的顶点也算单纯顶点。若一个消元次序使每个顶点在被处理时都是当前图中的单纯顶点，就称它为**完美消元序**（perfect elimination ordering）。沿完美消元序消元不产生任何填充边，当前图始终是原图在剩余顶点上的诱导子图。

弦图　不是每个图都有完美消元序。四个顶点的环 $1-2-3-4-1$ 中，每个顶点的两个邻居都不相邻：1 与 3 的邻居都是 $\{2,4\}$，缺边 $\{2,4\}$；2 与 4 的邻居都是 $\{1,3\}$，缺边 $\{1,3\}$。所以第一步就找不到单纯顶点。补入边 $\{1,3\}$ 后，顶点 2 的邻居 $\{1,3\}$ 相邻，可以先消去；剩下的三角形 $1,3,4$ 中每个顶点都单纯，于是 $(2,1,3,4)$ 是完美消元序。图 12.10画出了这一对比。

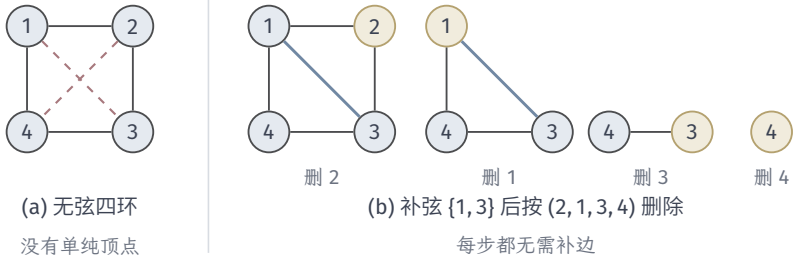


图 12.10　无弦四环与补弦后的图。左：四环中每个顶点的两个邻居都不相邻（红色虚线为所缺的边），没有单纯顶点。右：补入弦 $\{1,3\}$（蓝色）后，按 $(2,1,3,4)$ 逐个删除，每一步被删除的黄色顶点的剩余邻居都已相邻，全程无填充。

四环的问题出在“长度至少为四而没有捷径的环”。消去这种环上的任何一个顶点，它在环上的两个邻居都不相邻，必须补一条边把它们直接连起来；三角形则没有这个问题，因为两个邻居本来就相邻。在长度至少为 4 的环上，连接两个在环上不相邻的顶点的边称为该环的**弦**（chord），也就是一条捷径。每个长度至少为 4 的环都有弦的有限无向图称为**弦图**（chordal graph）。

树没有环，是弦图；补入 $\{1,3\}$ 后的四环是弦图；无弦四环不是。给一般图补边使之成为弦图称为**弦化**（chordal completion）。

定理 12.8　弦图与完美消元序　有限无向图是弦图，当且仅当它有完美消元序。

证明　先证有完美消元序的图是弦图。设图中有长度至少为 4 且无弦的环，考察环上最先被处理的顶点 v。v 在环上的两个邻居 u,w 此时都还在图中，并且与 v 相邻。v 是单纯顶点，所以 u,w 相邻；而沿完美消元序处理时从不补边，$\{u,w\}$ 必是原图的边。环长至少为 4，u,w 在环上不相邻，于是 $\{u,w\}$ 是这个环的弦，矛盾。

反方向的论证路线如下。第一步是一个引理：每个非空有限弦图都有单纯顶点，若它不是完全图，还有两个不相邻的单纯顶点。这一步需要分析极小分隔集，证明极小分隔集在弦图中都是团，本书不展开。第二步，弦图的诱导子图仍是弦图，因为诱导子图中的无弦环

> 在原图中也无弦。于是可以反复删去一个单纯顶点，余下的图仍是弦图、仍有单纯顶点，直到删完，删除的次序就是完美消元序。 ■

填充图与弦化 一般图经一次完整消元得到的填充图 $\mathcal{G}_\pi^+$（第 12.2.1小节）总是弦图，并且 π 就是它的完美消元序。要验证这一点，只需说明在 $\mathcal{G}_\pi^+$ 中，v 排在它之后的邻居恰是消元时的剩余邻居 $N_\pi^+(v)$：$\mathcal{G}_\pi^+$ 中连接 v 与某个较晚顶点 w 的边，若是原始边，处理 v 时仍在；若是填充边，它在处理某个更早的顶点时补入，那时 v 还没有被删去，所以处理 v 时这条边也在。于是这些后续邻居在处理 v 时已被补成团，v 在 $\mathcal{G}_\pi^+$ 中按 π 处理时总是单纯顶点。由定理 12.8，$\mathcal{G}_\pi^+$ 是弦图，第 12.2.3小节的构造实际上隐含了一次弦化。

反过来，设 $\mathcal{H}$ 是包含 $\mathcal{G}$ 的弦图（顶点相同、边更多），σ 是 $\mathcal{H}$ 的完美消元序。按 σ 对 $\mathcal{G}$ 消元，对步数归纳可知当前图始终是 $\mathcal{H}$ 在剩余顶点上的诱导子图的子图：处理 v 时，它的当前邻居包含在 $\mathcal{H}$ 中 v 的后续邻居里，后者在 $\mathcal{H}$ 中成团，所以补入的填充边都是 $\mathcal{H}$ 的边。因此 $\mathcal{G}_\sigma^+ \subseteq \mathcal{H}$，每个消元簇都含于 $\mathcal{H}$ 的某个团，诱导宽度不超过 $\mathcal{H}$ 的最大团大小减一。

这样，消元次序与弦化相互对应：选一个次序就选定了一个弦化，选一个弦化及其完美消元序也就得到一个宽度不更大的次序。四环算例按 $(1,2,3,4)$ 消元，填充图是环加上弦 $\{2,4\}$，$(1,2,3,4)$ 正是它的完美消元序；按 $(2,1,3,4)$ 消元则对应图 12.10中补弦 $\{1,3\}$ 的弦化。

12.5.3 完美消元序的构造与核验

12.5.3.1 问题与思路

定理 12.8只说明弦图有完美消元序，没有给出求法。输入是一个有限无向图，输出是它的一个完美消元序，或者“不是弦图”的报告及证据。定理证明的反方向已经提示了做法：反复找一个单纯顶点删去。弦图的每个诱导子图仍有单纯顶点，所以对弦图这一过程不会卡住；非弦图没有完美消元序，过程必然在某一轮找不到单纯顶点，这时剩下的图本身就是证据。

12.5.3.2 算法实现

输入图在运行中不变。算法维护两项状态：

- 当前图：初值为输入图的副本；
- 已输出的次序：初值为空。

每一轮依次做两件事：

- 检查：按编号逐个检查当前图中的顶点，枚举它的每对邻居看是否相邻，遇到第一个单纯顶点就停下；

- 删除：把找到的单纯顶点从当前图中删去，追加到次序末尾。

当前图为空时停止，输出的次序就是完美消元序。若非空的当前图中找不到单纯顶点，就停止并报告输入不是弦图，返回这个当前图作为证据，而不暗中补边。

算法 12.4　逐次删除单纯顶点

输入：有限无向图。输出：完美消元序，或“不是弦图”的报告及当前图。

1. 令当前图为输入图的副本，次序为空。
2. 若当前图为空，输出次序并停止。
3. 按编号检查当前图的顶点，找出第一个邻居两两相邻的顶点 v；若不存在，报告输入不是弦图，返回当前图并停止。
4. 从当前图中删去 v，把 v 追加到次序末尾，回到第 2 步。

执行过程　在补弦后的四环上（边 $\{1,2\},\{2,3\},\{3,4\},\{1,4\},\{1,3\}$）：

- 第 1 轮：顶点 1 的邻居 $\{2,3,4\}$ 缺边 $\{2,4\}$，跳过；顶点 2 的邻居 $\{1,3\}$ 相邻，选中并删去。
- 第 2 轮：剩下三角形 $1,3,4$，顶点 1 的邻居 $\{3,4\}$ 相邻，选中。
- 第 3、4 轮：依次选中 3 与 4。

输出 $(2,1,3,4)$。在无弦四环上，第 1 轮四个顶点都不单纯，过程失败并返回整个四环。

12.5.3.3　算法分析

正确性　成功时，每个顶点在被删去时都是当前图的单纯顶点，而删点不补边，当前图始终是输入图的诱导子图，所以输出的次序是完美消元序。对弦图，由定理 12.8证明中的引理与诱导子图的封闭性，每一轮的当前图都有单纯顶点，过程不会中途失败；对非弦图，若过程成功就得到一个完美消元序，与定理矛盾，所以必然在某一轮失败。

复杂度　直接实现每轮最多检查 n 个顶点，检查顶点 v 需要 $O(|N(v)|^2)$ 次查边，总检查量最坏可达 $O(n^4)$。存在线性时间的专门算法，这里不展开。

12.5.3.4　完美消元序的核验

构造得到的次序应当独立核验，来自别处的次序更应如此。给定原图和一个次序，按下面的步骤检查：

- 确认次序恰好包含每个顶点一次，并为每个顶点记下次序位置；
- 对每个 v 取出原图中排在它之后的邻居，检查它们是否两两相邻。

全部通过时次序是完美消元序，因为在不补边的前提下，处理 v 时的当前邻居恰好就是这些后续邻居；发现缺边时，就把 v 和那对不相邻的邻居作为失败证据返回。例如把 $(2,1,3,4)$ 用于无弦四环，第一个顶点 2 的后续邻居 $\{1,3\}$ 不相邻，立即被拒绝；用于补弦后的图则通过。

核验只需 $O(n+m)$ 再加 $\sum_v |N_\pi^+(v)|^2$ 次查边，比构造便宜得多，所以即使次序来自别处，也值得先核验再使用。

12.5.4 团树的构造

12.5.4.1 问题与思路

弦图上可以直接用团作袋，这需要区分两种“最大”。不能再加入任何其他顶点而保持两两相邻的团称为**极大团**（maximal clique），这是按集合包含判断的；在所有团中顶点数最多的称为**最大团**（maximum clique）。例如三角形 $1,2,3$ 外加一条边 $\{3,4\}$，极大团是 $\{1,2,3\}$ 和 $\{3,4\}$；后者不能再扩大，但只有两个顶点，不是最大团。以弦图的全部极大团为袋、并满足运行交集的树分解称为**团树**（junction tree，也称 clique tree）。团树中每个袋都是团，没有因填充而多出的变量，相邻两袋的交集就是它们共享的变量。

给定一个弦图，构造团树要做两件事：得到全部极大团，再把它们连成满足运行交集的树。极大团可以从完美消元序中读出。设 π 是完美消元序，候选集合 $K_v = \{v\} \cup N_\pi^+(v)$ 就是第 12.2.3小节构造中的袋；v 被处理时是单纯顶点，所以 K_v 是团，极大团就藏在这些候选中。

连接时要选一棵树，使每个变量所在的团连成一片。可以按变量数边：一个变量属于 r 个团时，树中两端都含它的边至多有 $r-1$ 条，再多就会在这 r 个团之间成环；恰有 $r-1$ 条时，这 r 个团才连成一片。反过来看一条树边 $\{C,D\}$，它为 $C \cap D$ 中的每个变量各计一条，所以它的贡献是 $|C \cap D|$。以交集大小为边权，一棵生成树的总权就等于各变量的这种边数之和，不超过各变量的 $r-1$ 之和；达到这个上限，恰好意味着每个变量的团都连成了一片。这样，连接团的问题变成了最大权生成树问题。把每条边的权取负，就可以直接调用第 4章的最小生成树算法，那里的割性质保证 Kruskal 与 Prim 方法给出最优树。

12.5.4.2 算法实现

输入是弦图 $\mathcal{H}$，先用第 12.5.3小节的方法求出并核验它的完美消元序 π；此后 $\mathcal{H}$ 与 π 都不再改变。算法维护三项状态：

- 团表：初值为全部候选 K_v，处理后只剩极大团；
- 候选边表：团两两之间的边及其权 $w(C,D) = |C \cap D|$，按权从大到小排列；

- 并查集与已选树边：并查集初值为每个团各成一组，已选树边初值为空。

计算分三步：

- 求极大团：对每个顶点生成候选 K_v，删去重复的集合和被其他候选严格包含的集合，剩下的就是全部极大团。
- 连接：在以极大团为顶点的完全图上，按权从大到小考察候选边，两端不在同一组时加入树边并合并两组，否则跳过；这正是把权取负后调用 Kruskal 方法：负权的最小生成树就是原权的最大权生成树。也可以改用 Prim 方法。
- 核验：对输出的树逐个顶点检查运行交集。

已选树边达到团数减一时，连接停止。

算法 12.5　由弦图构造团树

输入：有限弦图 $\mathcal{H}$。输出：$\mathcal{H}$ 的团树。

1. 求完美消元序 π 并核验；对每个顶点 v 生成候选 $K_v = \{v\} \cup N_\pi^+(v)$。
2. 删去重复的候选和被其他候选严格包含的候选，得到极大团 $\mathcal{C}$。
3. 对 $\mathcal{C}$ 中每对团计算 $w(C, D) = |C \cap D|$，按权从大到小考察，用并查集判断是否成环，不成环时加入树边，直到树边数为 $|\mathcal{C}| - 1$。
4. 对每个顶点检查含它的团在树中是否连通。

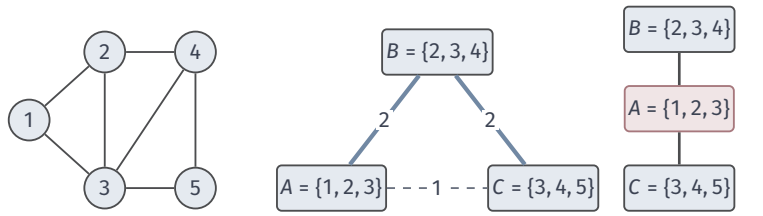
执行过程　图 12.11的弦图有边 $\{1,2\},\{1,3\},\{2,3\},\{2,4\},\{3,4\},\{3,5\},\{4,5\}$。

- 完美消元序：取 $(1,2,5,3,4)$ 并核验，各顶点在次序中靠后的邻居分别为 $\{2,3\}$、$\{3,4\}$、$\{3,4\}$、$\{4\}$、$\varnothing$，都两两相邻。
- 求极大团：候选为 $K_1 = \{1,2,3\}$、$K_2 = \{2,3,4\}$、$K_5 = \{3,4,5\}$、$K_3 = \{3,4\}$、$K_4 = \{4\}$；K_3 被 K_2 包含，K_4 被 K_3 包含，删去后得到三个极大团 $A = \{1,2,3\}$、$B = \{2,3,4\}$、$C = \{3,4,5\}$。
- 计算权：$w(A,B) = |\{2,3\}| = 2$、$w(B,C) = |\{3,4\}| = 2$、$w(A,C) = |\{3\}| = 1$。
- 连接：先加入 $A - B$，再加入 $B - C$，此时三个团已连通，$A - C$ 会成环而被跳过。得到的最大权生成树是 $A - B - C$，总权为 4。
- 核验：含 3 的团 A, B, C 沿 $A - B - C$ 连通，含 2 的团 A, B 相邻，含 4 的团 B, C 相邻，运行交集成立。

若改选 $A - B$ 与 $A - C$，总权为 3，此时含 4 的两个团 B, C 之间隔着不含 4 的 A，运行交集被破坏，如图 12.11右侧所示。

12.5.4.3 算法分析

正确性　先说明候选集合给出全部极大团。每个 K_v 都是团。反过来，任一团 C 都包含在某个 K_v 中：取 C 中最先被处理的顶点 v，C 的其余顶点



(a) 弦图　(b) 最大权生成树，总权 4　(c) 总权 3，含 4 的 B, C 被隔开

图 12.11　用交集权重的最大权生成树连接极大团。左：弦图。中：三个极大团组成的候选完全图，边上的数是交集大小，蓝色粗线是总权为 4 的最大权生成树 $A - B - C$，虚线是未被选中的候选边。右：总权为 3 的另一棵生成树 $B - A - C$，含 4 的团 B, C 被不含 4 的 A（红色）隔开，不满足运行交集。

都在 v 之后且与 v 相邻；沿完美消元序消元不补边也不删去尚未处理的顶点之间的边，它们在处理 v 时仍是 v 的当前邻居，所以都属于 $N_{\pi}^{+}(v)$。因此每个极大团都等于某个 K_v（它包含于某个团 K_v，又是极大的），而不被其他候选严格包含的候选都是极大团。这也说明弦图的极大团数不超过顶点数 n。

上面的例子中，满足运行交集的树恰好是总权最大的树。下面的定理把思路中的计数写成证明，说明这一对应是一般的。

定理 12.9　团树与最大权生成树　设 $\mathcal{H}$ 是非空有限弦图，$\mathcal{C}$ 是它的全部极大团。在以 $\mathcal{C}$ 为顶点的完全图上令 $w(C, D) = |C \cap D|$。则一棵生成树满足运行交集，当且仅当它的总权最大。

证明　先给出任意生成树的总权上界。对生成树 $\mathcal{T}'$，把每条边的权按共享变量拆开计数：

$$\sum_{\{C,D\} \in \mathcal{T}'} |C \cap D| = \sum_{v} e_v,$$

其中 e_v 是两端都含 v 的树边条数。设 v 属于 r_v 个极大团，两端都含 v 的树边正是 $\mathcal{T}'$ 在这 r_v 个节点上的诱导子图的边；树的诱导子图没有环，是 r_v 个节点上的森林，所以 $e_v \le r_v - 1$，且等号成立当且仅当这个诱导子图连通，即含 v 的团在 $\mathcal{T}'$ 中连通。于是总权不超过 $\sum_v (r_v - 1)$，等号成立当且仅当对每个 v 都满足运行交集。

剩下要证明确实存在满足运行交集的生成树，即团树存在；那时最大总权恰为 $\sum_v (r_v - 1)$，最大权生成树就是满足运行交集的生成树。取一个完美消元序，按定理 12.3的方法以 K_v 为袋得到树分解。

> 若某个袋包含于相邻的袋，就把这条袋树边收缩，只保留较大的袋：顶点覆盖和边覆盖不受影响；对运行交集，被收缩的较小袋中的变量都在较大袋中，收缩一条两端都含 v 的边不破坏 T_v 的连通性，而不含 v 的较小袋本来不在 T_v 中，收缩后 T_v 也不变。
>
> 反复收缩，直到没有相邻两袋有包含关系。这时任何两个袋都没有包含关系：若 $B_s \subseteq B_t$ 而 s,t 不相邻，取路径上与 s 相邻的节点 s'，由运行交集 B_s 的每个变量都在 $B_{s'}$ 中，与“相邻袋无包含关系”矛盾。各袋都是团且互不包含；每个极大团都被某个袋容纳（引理 12.5），而袋本身是团，所以二者相等；每个袋也都是某个极大团，否则它会被那个极大团所在的另一个袋严格包含。于是剩下的袋恰好是全部极大团，各出现一次，团树存在。 ■

在上例中，$r_1=1$、$r_2=2$、$r_3=3$、$r_4=2$、$r_5=1$，上界 $\sum_v (r_v-1)=4$，正好等于 $A-B-C$ 的总权。不连通的弦图中，不同分量的团之间交集为空，零权边只用于把各分量的团树连成一棵树，不影响上界。上面的计数也给出了最后一步核验的做法：检查每个 e_v 是否等于 r_v-1。

复杂度　设极大团有 $p \le n$ 个，候选完全图有 $p(p-1)/2$ 条边，每条边的权需要一次集合求交，排序与并查集的开销按第 4章的分析计算。求完美消元序和候选集合的开销见第 12.5.3小节与第 12.2.3小节。

适用范围　定理要求输入是弦图，并且使用全部极大团。对无弦四环，极大团是四条边 $\{1,2\},\{2,3\},\{3,4\},\{1,4\}$，每个顶点属于两个团；要让每个顶点的两个团相邻，需要四条树边，而四个节点的树只有三条边，所以任何生成树都不满足运行交集。一般图应先弦化，例如用某个消元次序补边，再在弦化后的图上构造团树；在原图的极大团上任选一棵树并不能保证得到树分解。选哪一种弦化决定了袋的大小，而团树构造只负责连接给定弦图的团，不负责找最好的弦化。输出后逐个顶点检查运行交集，也是对输入是否为弦图、团列表是否完整的独立核对。

12.5.5　构建结果的检验

无论结构来自消元次序、贪心规则还是团树构造，交给下一章计算之前都应独立检验。检验只读输出的袋树，不依赖它是怎样构造出来的，所以同样适用于来自别处的分解。检验的对象是原始图 $\mathcal{G}$（有保留集时是 $\mathcal{G}_R$）、局部函数的作用域、袋树及选定的根。

树分解的三个条件　核验按定义逐条进行：确认袋树是树，即节点数减一等于边数且连通；扫描各袋得到每个顶点的 T_v，检查顶点覆盖与边覆盖；

对每个顶点 v 数出两端袋都含 v 的袋树边条数 e_v。这些边恰是树在 T_v 上的诱导子图的边，而树的诱导子图是森林，所以 $e_v \le |T_v| - 1$，等号成立当且仅当 T_v 连通。

图 12.3下面的连接中 $|T_2| = 2$ 而 $e_2 = 0$，核验失败，2 就是失败的证据；梯子的分解中 $|T_2| = 3$、$e_2 = 2$，通过。团树构造中的最后一步核验（检查每个 e_v 是否等于 $r_v - 1$）也是这一检查。

作用域、接口与查询　树分解的条件只涉及原始图的边。计算还需要每个局部函数有处可放，所以对每个作用域 S_a 找出一个包含它的袋并记录下来；引理 12.5保证合法的分解总能通过这项检查，找不到时说明分解本身有误或作用域与原始图不符。

选定根以后，为每个非根节点算出接口 $S_u = B_u \cap B_{p(u)}$ 与内部变量 I_u，并核对每个变量恰在一个节点作内部变量（引理 12.2）。有保留集 R 时，另检查根袋包含 R。最后记录实际宽度，以及各袋、各接口上的状态数之积，供第 13.5节估计代价。

检验的代价　设袋树有 N 个节点，各袋大小之和为 $L = \sum_t |B_t|$。建立每个顶点的 T_v 和数出各 e_v 需要扫描各袋及各条袋树边两端的公共变量，时间为 $O(L + n)$ 量级（用以变量为下标的数组标记袋内成员）；边覆盖与作用域容纳的检查对每条边或每个作用域，在最小的 T_v 中逐袋测试包含关系。这些都是关于袋大小的多项式开销，与随状态数指数增长的数值计算相比通常很小，但它们是构建阶段的成本，应与构造本身的成本分开记录，不计入下一章的求值时间。

四环算例的分解 $\{1,2,4\} - \{2,3,4\}$ 以 $\{2,3,4\}$ 为根时，检验结果为：四条边分别被两个袋覆盖；$e_2 = e_4 = 1 = |T_2| - 1 = |T_4| - 1$，$e_1 = e_3 = 0 = |T_1| - 1 = |T_3| - 1$；四个作用域 $\{1,2\},\{1,4\}$ 放进第一个袋，$\{2,3\},\{3,4\}$ 放进根袋；子节点的接口为 $\{2,4\}$、内部变量为 $\{1\}$，根的内部变量为 $\{2,3,4\}$；宽度为 2，每个袋上有 8 种配置，接口上有 4 种。

12.6 本章小结

本章回答的是：在计算任何数值之前，一般图上的动态规划要把哪些变量放在一起处理、以哪些变量为接口、按什么次序传递结果。构建的输入只是作用域，所得结构对作用域相同而数值任意的全部问题都适用；一般数值下必须保留的耦合一律保留，不利用特定数值的巧合。

合法的结构是原始图的树分解。顶点覆盖和边覆盖保证每个变量、每条边有处可放，而由两两相交子树的公共节点可知，每个作用域乃至每个团都被某个袋整体容纳；运行交集保证每个变量在袋树上占据连成一片的地

盘，于是接口把两侧的变量和局部项隔开，以接口为索引的摘要不会丢失信息。选根以后，每个变量恰在它所在的最高节点作内部变量，所以每个变量恰被汇总一次。带保留变量的查询要先把保留集补成团，并选包含它的袋作根，宽度要按补边后的图计算。

变量消元与树分解是同一结构的两种描述。按次序在原始图上模拟消元，填充边记录新产生的耦合，消元簇连到其剩余作用域中最早被消去的变量的袋，就得到树分解，每个非根袋的接口恰是该步中间函数的作用域；反过来，从叶袋向内消去只出现在叶袋中的变量，就读出诱导宽度不超过分解宽度的次序。两个方向合起来，树宽等于最小诱导宽度；但某个具体次序或分解的宽度只是上界，二者也不一一对应。

实际构造时，最小度、最小填充等贪心规则给出可用但不保证最优的次序，状态数不同时还要比较实际的配置数；每个消元次序都隐含一次弦化，弦图有不需补边的完美消元序，可以逐次删除单纯顶点来构造并独立核验；弦图的极大团以交集大小为权、经最大权生成树连成团树。构建结果在交付计算前逐项检验三个条件、作用域容纳、接口与查询根，并记录实际宽度与状态数之积。数值表怎样在这样的结构上计算、怎样恢复配置、代价多大，由下一章回答。

第 13 章

一般图上动态规划的计算

上一章交付的是一个已经核验的结构：袋树、选定的根、每个节点的接口与内部变量，有查询时根袋还包含保留集。本章在这个结构上填写数值表，回答第 11.1.4小节提出的几种任务：最小成本及一个最优配置，保留一组变量时的条件最小成本表，可行配置的个数，以及非负权重之积的总和。对每种任务都要说清输入、输出、递推、正确性，并用算例检验；最后还要回答这些计算实际要花多少时间和存储。

各任务共用同一套组织方式，第 13.1节先把它固定下来：每个局部项恰好归入一个容纳其作用域的袋，按后序由叶到根处理，并说明按消元次序计算与沿袋树计算得到同样的中间函数。

第 13.2节处理最小成本。每个袋把本地成本与子袋消息相加，对内部变量取最小，再把结果沿接口交给父袋；取最小时记下达到值的内部取值，根选定最优状态后沿袋树恢复配置。正确性归结为一个不变量：消息等于固定接口后下方全部局部项之和的最小值。第 13.3节保留一组变量，只在根袋中对其余变量取最小，得到以保留变量为索引的表。

第 13.4节把相加换成相乘、取最小换成求和，计算可行配置数和加权总和，并说明求和与取最小不能随意交换次序。第 13.5节把袋与接口换算成实际的表项数，分析时间、存储及二者的取舍。

本章沿用第 11.3.2小节的四环算例，并用第 11章的三元作用域例子演示消元的逐步过程；两例都可以与枚举结果逐项核对。

13.1 局部项归属与求值顺序

在填写任何数值之前，要先固定两件事：每个局部项由哪个袋负责，以及各袋按什么顺序计算。这两件事对本章所有任务都相同。先把它们固定下来，既能避免重复计入局部项，也能保证比较两种表示时计算的是同一个对象。

13.1.1　局部项的唯一归属

计算前先把每个局部函数 ϕ_a 归入一个袋 $B_{\alpha(a)}$，要求 $S_a \subseteq B_{\alpha(a)}$；由引理 12.5，这样的袋总存在。映射 $\alpha : A \to T$ 称为局部项的归属。记归入节点 u 的局部项集合为 $A_u = \{a : \alpha(a) = u\}$，它们的和

$$\Phi_u(\boldsymbol{x}_{B_u}) = \sum_{a \in A_u} \phi_a(\boldsymbol{x}_{S_a}) \tag{13.1}$$

称为 u 的本地成本。没有局部项归入的袋取 $\Phi_u \equiv 0$（空和为 0），但仍保留袋中的变量，因为它们可能需要经过这个袋来保持一致。计数和求和任务把相加换成相乘，空乘积取 1，第 13.4节再说明。

只归入一次是数值正确性的要求。在三元作用域例子的分解 $\{1,2,3\} - \{2,3,4\}$ 中，若另有成对项 $\phi_{2,3}(x_2, x_3)$，它可以归入任一袋，但两边各计一次就会求出 $F + \phi_{2,3}$ 的最小值。共享变量可以在多个袋中重复出现，同一个成本项却只能计一次；袋之间的交集本身不对应任何成本。计算前应逐项检查分配清单：无遗漏、无重复、作用域包含于所归的袋。

消元表示中的归属　用消元森林表示结构时，归属有一条自然的规则：把 ϕ_a 归入 S_a 中最早被消去的变量所对应的袋。第 12.2.4小节已经说明这个袋包含 S_a。这条规则恰好对应消元的实际过程：ϕ_a 在它的某个变量第一次被消去时被取出。作用域全在保留集 R 内的局部项（包括作用域为空的常数项）在消元中从不被取出，归入根袋；没有保留集时，常数项可归入任一个根袋。

在四环算例的分解 $\{1,2,4\} - \{2,3,4\}$ 中，$\phi_{1,2}$ 与 $\phi_{1,4}$ 只能归入 $\{1,2,4\}$，$\phi_{2,3}$ 与 $\phi_{3,4}$ 只能归入 $\{2,3,4\}$。按次序 $(1,2,3,4)$ 的消元森林来看，$\phi_{3,4}$ 本应归入 $B_3 = \{3,4\}$，这个袋并入 $\{2,3,4\}$ 后归属随之并入。三元作用域例子的分解中，ϕ_A 只能归入 $\{1,2,3\}$，ϕ_B 只能归入 $\{2,3,4\}$。

13.1.2　选根与求值顺序

给已核验的袋树选根 r，确定每个非根节点 u 的父节点 $p(u)$、子节点集合 $\mathrm{ch}(u)$、接口 $S_u = B_u \cap B_{p(u)}$ 与内部变量 $I_u = B_u \setminus S_u$（第 12.1.4小节）。查询任务使用第 12.1.5小节保证覆盖保留集 R 的根袋。

求值按袋树的一个后序进行：先处理全部子节点，再处理父节点，最后处理根。每个非根节点只用到子节点交来的表，处理完后把一张以 $\boldsymbol{x}_{S_u}$ 为索引的表交给父节点。后序可以由一次深度优先遍历的完成次序得到，它与根、父节点一样只规定处理次序，在运行中不变。需要恢复配置时，再按前序（后序的逆序，父节点总在子节点之前）走一遍。

原始图不连通时，各分量的袋树之间只有空接口的边相连。沿空接口传递的表只有一项，是一个数：最小成本任务中它是那个分量的最小成本，计数和求和任务中它是那个分量的总数或总和。根把收到的这些数与本分量的结果组合起来，组合方式随任务而定，在各任务中分别说明。

表的存储与对齐 本地成本、消息和回溯表都是以一组变量为索引的稠密表。为每张表固定变量的排列（例如按编号），各取值按字典序存放，即 $(x_{j_1},\dots,x_{j_k})$ 像多位数一样以最后一个变量为最低位递增。于是一组取值的位置可以按混合进制直接算出，读取一项按常数时间计；本部分例子中的表都按这种次序列出，例如作用域为 $\{2,4\}$ 的表依次对应 $(x_2,x_4)=(0,0),(0,1),(1,0),(1,1)$。

计算时要在袋 B_u 的一组取值 $\boldsymbol{x}_{B_u}$ 上读取 $\phi_a(\boldsymbol{x}_{S_a})$ 或子节点的消息，即从 $\boldsymbol{x}_{B_u}$ 中挑出 S_a 或 S_c 的那几个分量，再算它们在小表中的位置。各变量在 B_u 中的位置事先记下，每次读取就只需几次乘加。接口上的“对齐”指的正是这一步：发送方和接收方按同一组接口变量、同一排列存取消息。

13.1.3 消元与袋树的求值对应

第 12.4.1小节在结构上对应了消元簇与袋、剩余作用域与接口。现在在数值上验证这一对应。沿袋树计算时，非根节点 u 先把本地成本与子节点的消息相加，得到

$$g_u(\boldsymbol{x}_{B_u})=\Phi_u(\boldsymbol{x}_{B_u})+\sum_{c\in\mathrm{ch}(u)} m_{c\to u}(\boldsymbol{x}_{S_c}),$$

再对内部变量取最小,得到发往父节点的消息 $m_{u\to p(u)}(\boldsymbol{x}_{S_u})=\min_{\boldsymbol{x}_{I_u}} g_u(\boldsymbol{x}_{B_u})$；第 13.2节逐项解释这两步。

定理 13.1 消元中间函数等于袋消息 设 $\pi=(v_1,\dots,v_k)$ 是 $V\setminus R$ 的消元次序，袋树由算法 12.1得到并以消元森林的根（有保留集时为 B_R）为根，局部项按“作用域中最早被消去的变量”归属，作用域含于 R 的项归入根袋。则按次序消元时，第 i 步取出的函数恰为 $\{\phi_a:a\in A_{B_i}\}$ 与 $\{\psi_j:B_j$的父袋是$B_i\}$，并且 $\psi_i=m_{B_i\to p(B_i)}$。

证明 消元过程中，当前函数集合里的每个函数，作用域只含尚未消去的变量；它在作用域中第一个变量被消去时被取出，此前一直不动。原始局部项 ϕ_a 因此在 S_a 中最早被消去的变量那一步被取出，这正是它的归属。中间函数 ψ_j 的作用域是 S_j，它在 S_j 中最早被消

去的变量那一步被取出，这正是 B_j 的父袋；若 $S_j \subseteq R$，它从不被取出，最后留在根袋。这就证明了第一个断言。

再对 i 归纳证明 $\psi_i = m_{B_i \to p(B_i)}$。第 i 步取出的函数之和为 Φ_{B_i} 加上各子袋的 ψ_j；由归纳假设，后者等于子袋发来的消息，所以这个和就是 g_{B_i}。B_i 的内部变量只有 v_i（第 12.4.1小节），所以式 (11.6)的 $\psi_i = \min_{x_{v_i}} g_{B_i}$ 正是消息 $m_{B_i \to p(B_i)}$。 ■

证明只用到“先把取出的函数组合起来，再对一个变量汇总”这一结构，没有用到取最小或相加的具体性质；把它们换成求和与相乘，同样的对应在第 13.4节的计数和求和中成立。

合并袋与一次处理多个内部变量　实际使用的袋树常常把消元森林中相邻的袋合并，例如四环算例把 $\{2,3,4\},\{3,4\},\{4\}$ 并成一个根袋。合并后的袋一次处理多个内部变量：沿袋树的消息是对这些变量同时取最小，而按次序消元则是逐个取最小。由式 (11.3)的第一条，对几个变量同时取最小等于逐个取最小，所以合并改变的只是中间表，不改变发往父袋的消息和最终结果。更一般的情形，即任意合法分解、任意满足容纳条件的归属，由第 13.2.5小节的不变量统一证明。

例子　三元作用域例子按次序 $(1,4,2,3)$ 消元，得到的袋树是 $\{1,2,3\} \to \{2,3\} \leftarrow \{2,3,4\}$、$\{2,3\} \to \{3\}$，以 $\{3\}$ 为根。各条消息恰好是第 13.2.4小节表 13.1中的 ψ_1, ψ_4 与消去 x_2 得到的新函数。

第 11.2.3小节的树上递推也是这样的特例：对依赖树选定根，以 $\{u, p(u)\}$ 为袋（第 12.1.3小节宽度为 1 的分解），把 ϕ_u 和 $\phi_{u,p(u)}$ 归入这个袋（根的一元项归入任一个含根的袋），接口就是单个变量 $p(u)$，袋消息退化为顶点之间的消息。

13.2　最小成本

本节的输入是各变量的有限状态集合、局部成本表（取值于 $\mathbb{R} \cup \{+\infty\}$），以及上一节固定的袋树、根、后序和局部项归属；输出是最小成本 $F^\star = \min_{\boldsymbol{x}} F(\boldsymbol{x})$，并在 $F^\star$ 有限时给出一个达到它的配置，在 $F^\star = +\infty$ 时报告没有可行配置。数值递推、达到值的记录、回溯、正确性与检验在本节一起展开。

13.2.1 合并本地成本与子消息

处理节点 u 时，它的子节点都已处理完，各自交来一张以接口为索引的消息表 $m_{c\to u}(\boldsymbol{x}_{S_c})$，$c \in \mathrm{ch}(u)$。第一步是把本地成本与这些消息按同一个袋配置读取并相加：

$$g_u(\boldsymbol{x}_{B_u}) = \Phi_u(\boldsymbol{x}_{B_u}) + \sum_{c\in\mathrm{ch}(u)} m_{c\to u}(\boldsymbol{x}_{S_c}). \tag{13.2}$$

由于 $S_c \subseteq B_u$、$S_a \subseteq B_u$（$a \in A_u$），右端每一项都只读取 $\boldsymbol{x}_{B_u}$ 的一部分分量，按第 13.1.2小节的对齐方式取出即可。g_u 的含义是：袋内变量取定 $\boldsymbol{x}_{B_u}$ 时，u 本地的成本加上各子节点下方最好能做到的成本。

叶节点没有子消息，求和为空，$g_u = \Phi_u$；没有局部项归入的节点 $\Phi_u \equiv 0$，g_u 只由子消息组成。两种空和都取 0，每项原始成本仍恰好计入一次：它在自己所归的节点计入 Φ，此后只以消息的形式向上传递。

在四环算例中，叶节点 $\{1,2,4\}$ 的合并结果是 $g(x_1,x_2,x_4) = \phi_{1,2}(x_1,x_2) + \phi_{1,4}(x_1,x_4)$；根 $\{2,3,4\}$ 的合并结果是 $\phi_{2,3} + \phi_{3,4}$ 加上叶节点交来的消息。

13.2.2 消去接口外变量

第二步对内部变量取最小。记父节点为 $p = p(u)$、接口为 $S_u = B_u \cap B_p$，从 u 发往 p 的消息是

$$m_{u\to p}(\boldsymbol{x}_{S_u}) = \min_{\boldsymbol{x}_{I_u}} g_u(\boldsymbol{x}_{S_u}, \boldsymbol{x}_{I_u}), \qquad I_u = B_u \setminus S_u. \tag{13.3}$$

这与式 (11.5)的结构相同，只是被汇总的不再是单个变量，而是袋内全部非接口变量。

可以把每个袋想成一个只看得见袋内变量的计算者。u 向父节点汇报的是：“若接口取这组值，我这一侧最好能做到多少成本。”父节点不需要知道 u 一侧的其他变量取什么，因为由第 12.3.2小节的分隔性质，这些变量不会在父节点一侧再出现，它们只需在 u 一侧选好即可。“u 一侧最好能做到多少”这一含义在第 13.2.5小节证明。

两种退化情形 接口为空时，消息只有一项，是一个数，但仍须对全部内部配置取最小，不能省去这一步。没有内部变量时（$B_u \subseteq B_p$），式 (13.3)中没有可取最小的变量，消息就是 g_u 本身，作为 $\boldsymbol{x}_{S_u} = \boldsymbol{x}_{B_u}$ 的函数原样交给父节点。

数值消元更新 内部变量只有一个时，式 (13.3)就是式 (11.6)的数值消元更新：消元森林中第 i 个袋的内部变量是 v_i，消息就是 ψ_i（定理 13.1）。

在三元作用域例子中消去 x_1：只有 ϕ_A 含 1，$S_1=\{2,3\}$。对 $(x_2,x_3)=(0,0),(0,1),(1,0),(1,1)$，分别比较 $x_1=0,1$ 两行，得到

$$\psi_1(x_2,x_3)=\big(\min(1,0),\ \min(0,2),\ \min(2,1),\ \min(+\infty,2)\big)=(0,\ 0,\ 1,\ 2),$$

达到最小值的 x_1 依次为 $1,0,1,1$。消元后的函数集合是 $\{\psi_1,\phi_B\}$。

这一步没有改变剩余变量的最优成本,可以逐项核对。例如取 $(x_2,x_3,x_4)=(1,0,1)$，原问题对 x_1 求最小值为 $\min\big(\phi_A(0,1,0)+\phi_B(1,0,1),\ \phi_A(1,1,0)+\phi_B(1,0,1)\big)=\min(2,1)=1$，新集合给出 $\psi_1(1,0)+\phi_B(1,0,1)=1+0=1$，两者相同。第 13.2.5小节证明这种一致对每个剩余配置、每一步消元都成立。

13.2.3 根袋求值

全部非根节点处理完后，根 r 按式 (13.2)合并本地成本与全部子消息，得到 $g_r(\boldsymbol{x}_{B_r})$。根的内部变量是整个 B_r，于是

$$F^\star=\min_{\boldsymbol{x}_{B_r}} g_r(\boldsymbol{x}_{B_r}). \tag{13.4}$$

用消元表示时，这对应消去根袋中剩余的变量，剩下的常数就是最小成本。

原始图不连通时，其他分量的根经空接口交来的消息都是标量，即各分量的最小成本；它们作为常数加进 g_r，所以 $F^\star$ 等于各分量最小成本之和。这与式 (11.3)后的第三条规则一致：互不相交的变量组可以分别取最小再相加。

若 $F^\star=+\infty$，报告没有可行配置，不做恢复；若 $F^\star$ 有限，进入下一小节的恢复步骤。

13.2.4 最优配置的记录与恢复

与链和树上一样，取最小值丢掉了达到者，所以计算消息时要同时记录达到值。对每个接口取值 $\boldsymbol{x}_{S_u}$，若 $m_{u\to p}(\boldsymbol{x}_{S_u})$ 有限，记下一个达到最小值的内部取值 $b_u(\boldsymbol{x}_{S_u})\in\prod_{i\in I_u}\mathcal{X}_i$，称为回溯表；并列时按固定规则选取，本部分约定取字典序最小者。若 $m_{u\to p}(\boldsymbol{x}_{S_u})=+\infty$，这个接口取值下没有可行的内部取值，不生成记录。根处同样记下 g_r 的一个最小点。

恢复按前序进行：

- 根：取 g_r 的最小点（并列时取字典序最小者）作为 $\boldsymbol{x}^\star_{B_r}$；
- 非根节点：轮到 u 时，它的接口变量都在父袋中，父节点已经处理，所以 $\boldsymbol{x}^\star_{S_u}$ 已经取定；查 $b_u(\boldsymbol{x}^\star_{S_u})$，给内部变量 $\boldsymbol{x}^\star_{I_u}$ 赋值。

每个变量恰在一个节点作内部变量（引理 12.2），所以恰被赋值一次，在它出现的所有袋中取同一个值。用消元表示时，按前序恢复就是按消元的逆序回溯：轮到 v_i 时，S_i 中的变量都比 v_i 晚消去，在逆序中已经取定。

两种表示的算法 按消元次序直接计算时，每个函数存为一个作用域和一张稠密表，并为每个变量维护一张索引，列出当前作用域含它的函数，消去 x_i 时据此直接取出 $\mathcal{F}_i$，不必扫描全部函数。算法维护当前函数集合（初值为全部局部函数）、已消去的变量集合和各步的回溯表。

算法 13.1　变量消元求最小成本

输入：有限状态集合、局部函数表、一个消元次序。输出：最小成本，以及一个最优配置或“不可行”。

1. 按次序取下一个变量 x_i，由变量索引取出当前全部含 i 的函数 $\mathcal{F}_i$，确定 S_i。
2. 对 $\boldsymbol{x}_{S_i}$ 的每个取值，枚举 x_i 的全部状态，按式 (11.6)求 ψ_i；同时把达到最小值的 x_i（并列时取编号最小者）记入回溯表 $b_i(\boldsymbol{x}_{S_i})$。
3. 用 ψ_i 替换 $\mathcal{F}_i$，同步更新 S_i 中各变量的索引；重复直到变量全部消去，剩下的常数之和即最小成本。
4. 若最小成本为 $+\infty$，报告不可行；否则按消元的逆序依次令 $x_i = b_i(\boldsymbol{x}_{S_i})$。

算法 13.2　树分解上的最小成本动态规划

输入：局部函数表，原始图的已核验树分解。输出：最小成本，以及一个最优配置或“不可行”。

1. 把每个局部函数归入一个包含其作用域的袋，按式 (13.1)计算各袋本地成本 Φ_u。
2. 选根节点 r，求出每个节点的父节点、接口、内部变量和一个后序。
3. 按后序处理每个非根节点 u：按式 (13.2)合并得 g_u；对接口 S_u 的每种取值，枚举 I_u 的全部取值，按式 (13.3)计算 $m_{u\to p(u)}$，并把达到最小值的内部取值记入 b_u。
4. 合并得 g_r，按式 (13.4)计算最小成本；若为 $+\infty$，报告不可行并停止。
5. 取 g_r 的最小点为根袋变量赋值；按前序处理每个非根节点 u，查 b_u 为 I_u 赋值。

消元的执行过程 在三元作用域例子上按次序 $(1,4,2,3)$ 执行算法 13.1。表 13.1汇总了每一步，图 13.1画出了函数集合的变化。

- 消去 x_1：如上一小节，$\mathcal{F}_1 = \{\phi_A\}$，$S_1 = \{2,3\}$，$\psi_1 = (0,0,1,2)$，$b_1 = (1,0,1,1)$。函数集合变为 $\{\psi_1, \phi_B\}$。

- 消去 x_4：只有 ϕ_B 含 4，$S_4=\{2,3\}$。对 (x_2,x_3) 的四种取值比较 $x_4=0,1$：$\min(2,1)$、$\min(0,+\infty)$、$\min(+\infty,0)$、$\min(1,2)$，得 $\psi_4=(1,0,0,1)$，$b_4=(1,0,1,0)$。函数集合变为 $\{\psi_1,\psi_4\}$。
- 消去 x_2：ψ_1 和 ψ_4 都含 2，要先相加再对 x_2 取最小，$S_2=\{3\}$。$x_3=0$ 时两个候选为 $\psi_1(0,0)+\psi_4(0,0)=1$ 与 $\psi_1(1,0)+\psi_4(1,0)=1$，并列，按约定取 $x_2=0$；$x_3=1$ 时候选为 $0+0=0$ 与 $2+1=3$，取 $x_2=0$。新函数记为 $\psi_2'=(1,0)$，以区别于第 12.2.1小节先消 x_2 时的 ψ_2；$b_2=(0,0)$。
- 消去 x_3：只剩 ψ_2'，$S_3=\varnothing$，$\min(1,0)=0$ 在 $x_3=1$ 处达到，$b_3=1$。函数集合只剩常数 0，即最小成本。

最小成本有限，按逆序 $3,2,4,1$ 回溯：$x_3=b_3=1$；$x_2=b_2(x_3{=}1)=0$；$x_4=b_4(x_2{=}0,x_3{=}1)=0$；$x_1=b_1(x_2{=}0,x_3{=}1)=0$。恢复的配置为 $(0,0,1,0)$，代回表 11.1得 $\phi_A(0,0,1)+\phi_B(0,1,0)=0+0=0$，与枚举结果一致。

表 13.1　按次序 $(1,4,2,3)$ 对三元作用域例子消元。新函数与回溯表的取值按剩余变量的字典序列出：作用域为 $\{2,3\}$ 时依次对应 $(x_2,x_3)=(0,0),(0,1),(1,0),(1,1)$，作用域为 $\{3\}$ 时对应 $x_3=0,1$。

消去	取出 $\mathcal{F}_i$	S_i	新函数 ψ_i	回溯表 b_i	消元后的 $\mathcal{F}$
x_1	ϕ_A	$\{2,3\}$	$(0,0,1,2)$	$(1,0,1,1)$	$\{\psi_1,\phi_B\}$
x_4	ϕ_B	$\{2,3\}$	$(1,0,0,1)$	$(1,0,1,0)$	$\{\psi_1,\psi_4\}$
x_2	ψ_1,ψ_4	$\{3\}$	$(1,0)$	$(0,0)$	$\{\psi_2'\}$
x_3	ψ_2'	$\varnothing$	0	1	$\{0\}$

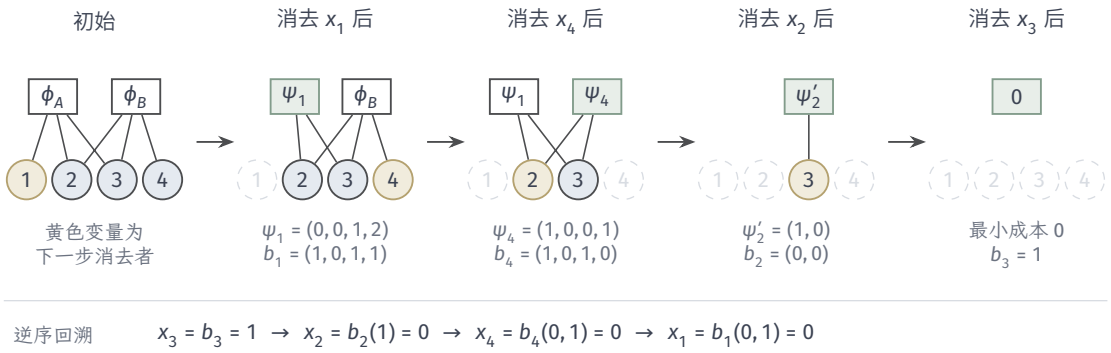


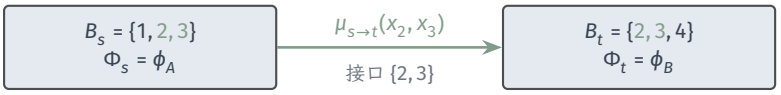
图 13.1　按次序 $(1,4,2,3)$ 消元时函数集合的逐步变化。圆点是变量，方块是当前函数，连线表示函数读取该变量；黄色变量是下一步要消去的变量，绿色方块是刚产生的新函数，虚线圆是已消去的变量。每个面板下方是新函数与回溯表，取值按作用域的字典序列出。底行按消元的逆序查回溯表，恢复最优配置 $(0,0,1,0)$。

袋树上的执行过程　同一个例子也可以直接在两个袋 $B_s=\{1,2,3\}$、$B_t=\{2,3,4\}$ 上执行算法 13.2，取 t 为根，后序为 s,t。本地成本为 $\Phi_s=\phi_A$、$\Phi_t=\phi_B$。

s 是叶节点，没有子消息，内部变量只有 x_1，所以 $m_{s\to t}(x_2,x_3)=\min_{x_1}\phi_A(x_1,x_2,x_3)$。图 13.2逐行列出了这张四项表：接口取值为 $00,01,10,11$

时，消息值依次为 $0,0,1,2$，达到最小值的 x_1 依次为 $1,0,1,1$。它与消去 x_1 得到的 ψ_1 相同。x_1 已经被汇总，不再出现在表中；x_2,x_3 作为接口仍然显式保留，根袋读取这张表时要按同样的 (x_2,x_3) 取值对齐。

根处合并得 $g_t=\phi_B+m_{s\to t}$，按 (x_2,x_3,x_4) 的字典序依次为 $2,\ 1,\ 0,\ +\infty,\ +\infty,\ 1,\ 3,\ 4$，最小值 0 在 $(0,1,0)$ 处达到。回溯时根取 $(x_2,x_3,x_4)=(0,1,0)$，查叶节点的回溯表得 $b_s(0,1)=0$，即 $x_1=0$。恢复的配置 $(0,0,1,0)$ 与枚举和消元的结果相同。



根：$\min(\phi_B+\mu_{s\to t})=0$

(x_2,x_3)	$\phi_A(0,x_2,x_3)$	$\phi_A(1,x_2,x_3)$	$\mu_{s\to t}$	回溯 x_1
$(0,0)$	1	0	0	1
$(0,1)$	0	2	0	0
$(1,0)$	2	1	1	1
$(1,1)$	$+\infty$	2	2	1

图 13.2　两袋之间的消息。左袋 $B_s=\{1,2,3\}$ 只含 ϕ_A，右袋 $B_t=\{2,3,4\}$ 只含 ϕ_B，接口为 $\{2,3\}$（绿色）。下方表格对接口的每种取值比较 $x_1=0,1$ 两个候选，取较小者作为消息值，黄色格是被选中的候选；绿色列是消息值，最后一列是回溯表。根袋 B_t 把消息与 ϕ_B 相加后对 x_2,x_3,x_4 取最小，得到最优值 0。

13.2.5 计算正确性

消息的不变量　正确性归结为一个不变量：每条消息等于固定接口后，下方全部局部项之和的最小值。对非根节点 u，记 D_u 为 u 及其全部后代组成的节点集合，$V_{D_u}=\bigcup_{t\in D_u}B_t$ 为这些袋中出现的变量，$A_{D_u}=\bigcup_{t\in D_u}A_t$ 为归入这些节点的局部项。

定理 13.2　袋消息的含义　设 $(\mathcal{T},\{B_t\}_{t\in T})$ 是原始图的树分解，以 r 为根，每个局部函数恰好归入一个包含其作用域的袋。则对每个非根节点 u 和每个接口取值 $\boldsymbol{x}_{S_u}$，

$$m_{u\to p(u)}(\boldsymbol{x}_{S_u})=\min_{\boldsymbol{x}_{V_{D_u}\setminus S_u}}\ \sum_{a\in A_{D_u}}\phi_a(\boldsymbol{x}_{S_a}).$$

进一步，$\min_{\boldsymbol{x}}F(\boldsymbol{x})=\min_{\boldsymbol{x}_{B_r}}g_r(\boldsymbol{x}_{B_r})$。

证明　对 $|D_u|$ 归纳，叶节点是起点。设 u 的子节点为 $c_1,\dots,c_k$（叶节点时 $k=0$）。D_u 由 u 和各 D_{c_j} 组成，于是 A_{D_u} 分为 A_u 和各 $A_{D_{c_j}}$，每个局部项恰在一处。

先看变量怎样分开。由引理 12.6应用于袋树边 $\{c_j,u\}$，$V_{D_{c_j}}$ 与 D_{c_j} 以外各袋的公共变量都在 $S_{c_j}\subseteq B_u$ 中。所以集合

$$I_u,\qquad V_{D_{c_1}}\setminus B_u,\qquad \dots,\qquad V_{D_{c_k}}\setminus B_u$$

两两不交：$V_{D_{c_j}}\setminus B_u$ 中的变量不在 B_u 里，也不在其他子树中。它们的并恰为 $V_{D_u}\setminus S_u$。另一方面，$A_{D_{c_j}}$ 中函数的变量都在 $V_{D_{c_j}}$ 中，其中属于 B_u 的部分又在 S_{c_j} 中。

现在固定 $\boldsymbol{x}_{B_u}$。各 $A_{D_{c_j}}$ 中函数之和只依赖 $\boldsymbol{x}_{S_{c_j}}$ 与互不相交的变量组 $\boldsymbol{x}_{V_{D_{c_j}}\setminus B_u}$，按式 (11.3)后的第三条规则可以分别求最小；由归纳假设，第 j 项的最小值为 $m_{c_j\to u}(\boldsymbol{x}_{S_{c_j}})$。归入 u 的局部项与这些变量无关，由第二条规则留在最小化之外。于是固定 $\boldsymbol{x}_{B_u}$ 后的最小值是 $g_u(\boldsymbol{x}_{B_u})$。再对 $\boldsymbol{x}_{I_u}$ 求最小值，由第一条规则，就得到式 (13.3)。

根处的等式用同样的拆分得到：由顶点覆盖，全部变量是 B_r 与各 $V_{D_c}\setminus B_r$（$c\in\mathrm{ch}(r)$）的不交并，全部局部项分为归入 r 的与归入各子树的。 ■

证明用到了三项条件：运行交集保证接口隔开两侧；顶点覆盖保证根处的汇总涉及全部变量；每个局部函数恰好归入一个容纳它的袋，保证每一项只计一次。少了任何一项，递推得到的数都可能不等于原问题的最小值。

任意消元次序都正确　按消元次序直接计算时，正确性也可以用一个不变量表述，并且它对任意次序都成立。设已经按某个次序消去了下标集合 Z 中的变量，把剩余变量的一个配置记为 $\boldsymbol{y}=\boldsymbol{x}_{V\setminus Z}$。消元过程保持

$$\sum_{f\in\mathcal{F}} f(\boldsymbol{y})=\min_{\boldsymbol{x}_Z} F(\boldsymbol{x}_Z,\boldsymbol{y})\qquad \text{对每个剩余配置 } \boldsymbol{y}. \tag{13.5}$$

定理 13.3　变量消元的正确性　设各变量的状态集合有限，总成本由式 (11.1)给出。从 $\mathcal{F}=\{\phi_a:a\in A\}$ 出发，按任意次序反复执行式 (11.6)的消元操作，则每一步之后不变量 (13.5)都成立。特别地，消去全部变量后剩下的常数之和等于 $\min_{\boldsymbol{x}}F(\boldsymbol{x})$；只消去 $V\setminus R$ 中的变量时，剩余函数之和是关于 $\boldsymbol{x}_R$ 的表，在每个 $\boldsymbol{x}_R$ 处等于 $\min_{\boldsymbol{x}_{V\setminus R}}F(\boldsymbol{x})$。

证明 开始时 $Z=\varnothing$，当前函数之和就是 F，不变量成立。设消去 x_i 之前不变量对 Z 成立。把当前函数分为含 i 的 $\mathcal{F}_i$ 与其余部分 $\mathcal{F}_{-i}$。固定 $Z\cup\{i\}$ 以外的变量后，$\mathcal{F}_{-i}$ 中各函数的值与 x_i 无关，按式 (11.3)可以移出对 x_i 的最小化：

$$\min_{x_i}\Big[\sum_{f\in\mathcal{F}_{-i}} f+\sum_{f\in\mathcal{F}_i} f\Big]=\sum_{f\in\mathcal{F}_{-i}} f+\min_{x_i}\sum_{f\in\mathcal{F}_i} f=\sum_{f\in\mathcal{F}_{-i}} f+\psi_i .$$

右端正是消元后的函数之和。左端由不变量等于 $\min_{x_i}\min_{\boldsymbol{x}_Z} F$，再由式 (11.3)的第一条等于 $\min_{\boldsymbol{x}_{Z\cup\{i\}}} F$。所以不变量对 $Z\cup\{i\}$ 成立。对消元步数归纳即得结论。 ■

这个定理说明，纯最小化中改变消元次序不改变最小值，但中间表与计算代价可以变化。三元作用域例子按次序 $(2,1,3,4)$ 消元时，第一步得到第 12.2.1小节的 8 项表 ψ_2，其后依次得到 (x_3,x_4) 上的 $(2,1,0,4)$、x_4 上的 $(0,1)$ 和常数 0，逆序回溯同样恢复出 $(0,0,1,0)$。两种次序得到相同的最小值，而最大中间表分别有 4 项和 8 项（表 12.3）。

恢复的配置达到最小值 回溯给出一致的配置：每个变量恰被赋值一次，接口变量在子节点中沿用父节点已取的值。再对 $|D_u|$ 归纳可知，给定接口取值 $\boldsymbol{x}^\star_{S_u}$ 且 $m_{u\to p(u)}(\boldsymbol{x}^\star_{S_u})$ 有限时，按回溯表给 $V_{D_u}\setminus S_u$ 赋值后，A_{D_u} 中局部项之和恰好等于 $m_{u\to p(u)}(\boldsymbol{x}^\star_{S_u})$。这是因为 b_u 选的内部取值达到式 (13.3)的最小值，使 g_u 等于消息；g_u 中的每条子消息又由归纳假设被子树中的赋值达到，而且这些子消息有限，回溯表中有相应的记录。

在根处，所选的 $\boldsymbol{x}^\star_{B_r}$ 使 g_r 等于 $F^\star$，于是整个配置的成本等于 $F^\star$。若 $F^\star=+\infty$，任何配置都“达到”它，所以算法报告没有可行配置，而不把回溯结果当作最优解输出。逆序回溯的正确性是同一论证在消元森林上的特例；若只保存 ψ_i 而不保存回溯表，就无法直接恢复配置。

回溯表只恢复一个最优配置。若要列出全部并列最优的配置，需要在每个接口取值下记录全部最优的内部取值，而这类配置的数目本身可能随 n 指数增长。

13.2.6 四环算例与边界检验

四环算例 在四环算例的分解 $\{1,2,4\}-\{2,3,4\}$ 上执行算法 13.2，以 $\{2,3,4\}$ 为根。归属与第 13.1.1小节相同。

叶节点的接口为 $\{2,4\}$，内部变量为 $\{1\}$。对接口的每种取值比较 $x_1=0,1$ 两个候选 $\phi_{1,2}+\phi_{1,4}$，表 13.2的左半部分列出了结果，消息与第 11.3.2小节的 ψ_1 相同。

根处合并 $g=\phi_{2,3}(x_2,x_3)+\phi_{3,4}(x_3,x_4)+m(x_2,x_4)$，右半部分按 (x_2,x_3,x_4) 的字典序列出。最小值 3 在 $(0,0,1)$ 处唯一达到。回溯时根取 $(x_2,x_3,x_4)=(0,0,1)$，查叶节点的回溯表得 $b(0,1)=1$，恢复 $\boldsymbol{x}^\star=(1,0,0,1)$。这与表 11.5的枚举结果一致；代回四项成本 $1+1+1+0=3$，每项恰计一次。

表 13.2　四环算例的最小成本计算。左：叶节点 $\{1,2,4\}$ 对 $x_1=0,1$ 的两个候选、消息值与回溯表。右：根 $\{2,3,4\}$ 的合并值 $g=\phi_{2,3}+\phi_{3,4}+m$。

(x_2,x_4)	候选	m	b	(x_2,x_3,x_4)	g
$(0,0)$	$6,\ 4$	4	1	$(0,0,0),(0,0,1)$	$5,3$
$(0,1)$	$+\infty,\ 1$	1	1	$(0,1,0),(0,1,1)$	$7,+\infty$
$(1,0)$	$3,\ 5$	3	0	$(1,0,0),(1,0,1)$	$4,4$
$(1,1)$	$+\infty,\ 2$	2	1	$(1,1,0),(1,1,1)$	$4,+\infty$

改用未合并的消元森林，即按次序 $(1,2,3,4)$ 执行算法 13.1，第一步同样得到 ψ_1。消去 x_2 时取出 ψ_1 与 $\phi_{2,3}$，得 $\psi_2(x_3,x_4)=(4,2,4,3)$，回溯表为 $(1,0,1,1)$；消去 x_3 时取出 ψ_2 与 $\phi_{3,4}$，$x_4=0$ 时两个候选 $4+0$ 与 $4+0$ 并列，按约定取 $x_3=0$，$x_4=1$ 时候选为 $2+1$ 与 $3+\infty$，得 $\psi_3(x_4)=(4,3)$；最后消去 x_4 得常数 3。逆序回溯给出 $x_4=1$、$x_3=0$、$x_2=0$、$x_1=1$，结果相同，中间表则与合并袋上的计算不同。

并列最优　把 $\phi_{2,3}(1,0)$ 从 1 改为 0，其余不变。这时 $(0,1,0,0)$、$(1,0,0,1)$、$(1,1,0,1)$ 三个配置的成本都是 3。叶节点的消息不变，根处 g 在 $(0,0,1)$、$(1,0,0)$、$(1,0,1)$ 三处取最小值 3。按字典序规则，根取 $(0,0,1)$，恢复的仍是 $(1,0,0,1)$；若规则改为取字典序最大者，根取 $(1,0,1)$，恢复 $(1,1,0,1)$。两者都是最优配置，固定规则只是让输出可以复现；要得到全部最优配置，须保留全部达到值。

不可行　把四项成本换成约束：$\phi_{1,2},\phi_{2,3},\phi_{3,4}$ 在两端相等时为 0、不等时为 $+\infty$，$\phi_{1,4}$ 在两端不等时为 0、相等时为 $+\infty$。每张表单独看都有可行的组合，但沿环要求 $x_1=x_2=x_3=x_4$ 且 $x_1\neq x_4$，互相矛盾。叶节点的消息为 $m(x_2,x_4)=(+\infty,0,0,+\infty)$：只有 $x_2\neq x_4$ 时存在可行的 x_1。根处 $\phi_{2,3}+\phi_{3,4}$ 要求 $x_2=x_3=x_4$，此时 $m=+\infty$，所以 g 的八项全是 $+\infty$，算法报告不可行，不做恢复。$+\infty$ 按运算约定沿消息传递，不需要特殊处理。

孤立变量与多个分量　在原问题上再加两个分量：一个没有任何局部项的孤立变量 $x_5\in\{0,1,2\}$，以及一条边 $\{6,7\}$ 上的成本 $\phi_{6,7}$，其表的行对应 x_6、列对应 x_7，为 $\begin{bmatrix}2&1\\0&3\end{bmatrix}$。结构上增加袋 $\{5\}$ 和 $\{6,7\}$，各用一条空接口的边连到根 $\{2,3,4\}$。

袋 $\{5\}$ 没有局部项，$g \equiv 0$，对三个状态取最小得标量消息 0，回溯记录 $x_5 = 0$（字典序最小）。袋 $\{6,7\}$ 的接口也为空，消息是 $\min \phi_{6,7} = 0$，在 $(x_6, x_7) = (1,0)$ 处达到。根把这两个标量加进 g，最小成本为 $3+0+0=3$，恢复的配置在原四个变量上不变，并有 $x_5 = 0$、$(x_6, x_7) = (1,0)$。孤立变量虽不影响最小值，仍须出现在某个袋中，才会在恢复时被赋值。

13.3 带保留变量的最小成本

有时不需要把全部变量都优化掉，而是想知道：若把某几个变量固定成某组值，其余变量最好能做到多少。例如要比较一个设备开或关时整体最优成本相差多少，或者要把结果交给另一个只关心这几个变量的计算。本节指定一组暂不消去的变量，求出它们每种固定取值下的最小成本；输出是一张以这些变量的取值为索引的表。

13.3.1 条件最小成本表

设保留集为 $R \subseteq V$。对每个固定的 $\boldsymbol{x}_R$，在其余变量上最小化，得到

$$F_R(\boldsymbol{x}_R) = \min_{\boldsymbol{x}_{V \setminus R}} F(\boldsymbol{x}_R, \boldsymbol{x}_{V \setminus R}). \tag{13.6}$$

这张表称为 R 上的**条件最小成本表**。它有 $\prod_{i \in R} |\mathcal{X}_i|$ 个表项，保留变量是表的索引，不在这里继续取最小；表项为 $+\infty$ 表示把 R 固定成这组值后没有可行配置。

对整张表再取最小，就回到整体最小成本：$\min_{\boldsymbol{x}_R} F_R(\boldsymbol{x}_R) = F^\star$，这由式 (11.3)的第一条得到。两个边界是：$R = \varnothing$ 时表只有一项，就是 $F^\star$；$R = V$ 时不消去任何变量，表就是 F 本身。第 11.2.3.4小节的各顶点最小成本表是 R 为单个变量时的特例。

13.3.2 查询结构与局部项归属

计算使用第 12.1.5小节的查询结构：在原始图中把 R 补成团，为补边后的图 $\mathcal{G}_R$ 构建树分解，并选一个包含 R 的袋作根。用消元表示时，就是只消去 $V \setminus R$，所有剩余作用域含于 R 的袋连到根袋 $B_R = R$（算法 12.1）。

局部项的归属沿用第 13.1.1小节：每项归入一个容纳其作用域的袋。按消元表示，含待消去变量的项归入其中最早消去的变量对应的袋，仅涉及 R 的局部项与常数项归入根袋。

查询引起的补边与宽度变化属于本问题的结构输入。补边后的树宽可能大于原图的树宽，计算代价按实际使用的结构计算，不能直接沿用无保留变量时的最优树宽。

13.3.3　条件最小成本表的计算

非根节点的计算与第 13.2节完全相同：按式 (13.2)合并本地成本与子消息，按式 (13.3)对内部变量取最小，接口变量保留在消息的索引中。由于根袋包含 R，R 中的变量不会是任何非根节点的内部变量（下面第 13.3.5小节），它们若出现在某个非根袋中，必定在该袋的接口里，随消息向上传递。

区别只在根袋。根 r 合并得到 $g_r(\boldsymbol{x}_{B_r})$ 后，只消去 $B_r \setminus R$ 中的变量：

$$F_R(\boldsymbol{x}_R) = \min_{\boldsymbol{x}_{B_r \setminus R}} g_r(\boldsymbol{x}_{B_r}). \tag{13.7}$$

对每个保留配置 $\boldsymbol{x}_R$ 留下一个最小成本，组成整张表。若 $B_r = R$，根处不取最小，$F_R = g_r$。

原始图不连通时，不含 R 的分量经空接口交来标量，即各自的最小成本；它们作为常数加到表的每一项上。用消元表示时，这一计算就是定理 13.3的后半部分：只消去 $V \setminus R$ 中的变量后，剩余函数之和就是 F_R。

13.3.4　条件最优配置的恢复

给定使 $F_R(\boldsymbol{x}_R)$ 有限的保留配置，可以恢复一个条件最优配置，即在 $\boldsymbol{x}_R$ 固定时达到 $F_R(\boldsymbol{x}_R)$ 的配置。做法是在根处对每个 $\boldsymbol{x}_R$ 另记一张回溯表 $b_r(\boldsymbol{x}_R)$，记录达到式 (13.7)最小值的 $\boldsymbol{x}_{B_r \setminus R}$。

恢复时保持 $\boldsymbol{x}_R$ 不变，从根的回溯表读出根袋中其余变量，再按前序沿袋树查各非根节点的回溯表，与第 13.2.4小节相同。表项为 $+\infty$ 时报告这个固定配置不可行，不做恢复；有多个条件最优解时沿用字典序最小的选取规则。

13.3.5　计算正确性

先说明保留变量不会被提前汇总。设 $v \in R$。根袋包含 R，所以 $r \in T_v$，T_v 中离根最近的节点就是 r；由引理 12.2，v 只在根作内部变量。等价地说，含 v 的每个非根袋到根的路径上每个袋都含 v，所以 v 总在这些袋的接口里。

再逐个固定 $\boldsymbol{x}_R$。对固定的 $\boldsymbol{x}_R$，记 $F^{(\boldsymbol{x}_R)}(\boldsymbol{x}_{V \setminus R}) = F(\boldsymbol{x}_R, \boldsymbol{x}_{V \setminus R})$，它仍是局部函数之和：把每个局部函数中 R 的分量代入固定值即可。非根节点的消息在这组 $\boldsymbol{x}_R$ 处的取值，正是对 $F^{(\boldsymbol{x}_R)}$ 执行第 13.2节递推得到的消息，因为 R 中的变量从不在非根节点被取最小，只是作为索引被带着走。由定理 13.2，每条消息等于固定接口后下方局部项之和的最小值；在根处对 $B_r \setminus R$ 取最小，得到 $\min F^{(\boldsymbol{x}_R)} = F_R(\boldsymbol{x}_R)$。

条件最优配置的正确性同样沿用第 13.2.5小节的论证，只是根处的最小点在 $\boldsymbol{x}_R$ 固定的条件下选取；表项为 $+\infty$ 时对应没有可行解。由定理 13.3，

仅改变 $V \setminus R$ 中变量的消去次序时，各表项不变，改变的只是中间表和代价。

13.3.6 条件最小成本表的检验

四环上的查询 在四环算例上取 $R=\{1,3\}$。第 12.4.3小节已在补边 $\{1,3\}$ 后的图上按次序 $(2,4)$ 构建了查询结构：两个袋 $\{1,2,3\}$ 与 $\{1,3,4\}$ 都连到根袋 $\{1,3\}$，接口都是 $\{1,3\}$，内部变量分别是 x_2 与 x_4。按消元规则，$\phi_{1,2},\phi_{2,3}$ 归入 $\{1,2,3\}$，$\phi_{3,4},\phi_{1,4}$ 归入 $\{1,3,4\}$，根袋没有本地成本。

两个子节点的消息为

$$m_2(x_1,x_3)=\min_{x_2}\bigl[\phi_{1,2}(x_1,x_2)+\phi_{2,3}(x_2,x_3)\bigr],$$

$$m_4(x_1,x_3)=\min_{x_4}\bigl[\phi_{3,4}(x_3,x_4)+\phi_{1,4}(x_1,x_4)\bigr].$$

根袋就是 R，不再取最小，$F_R=m_2+m_4$。表 13.3列出了各候选、回溯表与结果。

表 13.3 四环算例在 $R=\{1,3\}$ 上的条件最小成本表。“候选”依次对应内部变量取 $0,1$；b_2、b_4 是达到最小值的 x_2、x_4；最后一列是由回溯表恢复的条件最优配置。

(x_1,x_3)	m_2 的候选	m_2	b_2	m_4 的候选	m_4	b_4	F_R 及条件最优配置
$(0,0)$	$4,\ 1$	1	1	$3,\ +\infty$	3	0	$4,\ (0,1,0,0)$
$(0,1)$	$6,\ 1$	1	1	$3,\ +\infty$	3	0	$4,\ (0,1,1,0)$
$(1,0)$	$2,\ 3$	2	0	$3,\ 1$	1	1	$3,\ (1,0,0,1)$
$(1,1)$	$4,\ 3$	3	1	$3,\ +\infty$	3	0	$6,\ (1,1,1,0)$

逐项用表 11.5核对：$x_1=0,x_3=0$ 的四个配置成本为 $7,+\infty,4,+\infty$，最小 4 在 $(0,1,0,0)$；$x_1=0,x_3=1$ 的四个为 $9,+\infty,4,+\infty$，最小 4 在 $(0,1,1,0)$；$x_1=1,x_3=0$ 的四个为 $5,3,6,4$，最小 3 在 $(1,0,0,1)$；$x_1=1,x_3=1$ 的四个为 $7,+\infty,6,+\infty$，最小 6 在 $(1,1,1,0)$。四个表项及条件最优配置都与表中一致。

对整张表取最小，$\min(4,4,3,6)=3$，在 $(x_1,x_3)=(1,0)$ 处达到，相应的条件最优配置 $(1,0,0,1)$ 就是第 13.2.6小节的整体最优配置，两种计算互相印证。这个查询不能直接用分解 $\{1,2,4\}-\{2,3,4\}$ 回答：那里没有袋同时含 x_1 和 x_3，x_1 会在叶节点被取最小。

两个边界 $R=\varnothing$ 时不保留变量，查询结构就是原来的结构，表只有一项，计算退化为第 13.2节，四环上为 $F^\star=3$。$R=V$ 时不消去任何变量：补边后的图是完全图，根袋包含全部变量，$F_R=g_r$ 就是把全部局部项相加的 16 项表，即表 11.5本身。前者不保留任何变量，后者不消去任何变量，一般的查询介于二者之间，输出表的大小与根袋的宽度也随之介于二者之间。

13.4　计数和求和

最小成本只关心最好的一个配置。另一类问题要对全部配置汇总：可行配置有多少个，或者按局部权重之积给每个配置一个权重，全部权重加起来是多少。本节先用约束指示函数计算可行配置数，再把指示值推广为非负有限权重。两种计算都把“局部项相加、对变量取最小”换成“局部值相乘、对变量求和”，结构、归属和处理次序都不变；变化在于数值的组合与汇总方式，以及输出的含义。

13.4.1　可行配置计数

把每个局部函数换成它的可行性指示

$$\iota_a(\boldsymbol{x}_{S_a}) = \begin{cases} 1, & \phi_a(\boldsymbol{x}_{S_a}) < +\infty, \\ 0, & \phi_a(\boldsymbol{x}_{S_a}) = +\infty. \end{cases}$$

一个配置可行，当且仅当这些指示全为 1，即它们的乘积为 1。于是可行配置数为

$$N = \sum_{\boldsymbol{x} \in \mathcal{X}} \prod_{a \in A} \iota_a(\boldsymbol{x}_{S_a}). \tag{13.8}$$

输入与最小成本问题相同（只用到哪些组合被禁止），输出是一个非负整数。

在袋树上计算时，节点 u 先把归入它的指示值与子节点交来的计数相乘，

$$h_u(\boldsymbol{x}_{B_u}) = \prod_{a \in A_u} \iota_a(\boldsymbol{x}_{S_a}) \cdot \prod_{c \in \mathrm{ch}(u)} n_{c \to u}(\boldsymbol{x}_{S_c}),$$

再对接口外的状态求和，得到发往父节点的计数消息

$$n_{u \to p(u)}(\boldsymbol{x}_{S_u}) = \sum_{\boldsymbol{x}_{I_u}} h_u(\boldsymbol{x}_{S_u}, \boldsymbol{x}_{I_u}).$$

它的含义是：接口取这组值时，u 下方的变量有多少种取值能使下方全部约束满足。

两种空运算的约定与最小成本不同：空乘积取 1，求和从 0 开始。没有局部项归入、也没有子节点的袋，$h_u \equiv 1$，但求和仍要累计内部变量的全部状态。孤立变量 x_v 单独成袋 $\{v\}$ 时，消息是 $\sum_{x_v} 1 = |\mathcal{X}_v|$，不能直接把它的计数设为 1：它不受约束，每个状态都与其余变量的每个可行取值组合成不同的可行配置。

13.4.2 求和

把指示值推广为非负有限的局部权重 $w_a(\boldsymbol{x}_{S_a}) \geq 0$，配置的权重定义为各局部权重之积 $W(\boldsymbol{x}) = \prod_{a \in A} w_a(\boldsymbol{x}_{S_a})$，要求的是全部配置的权重总和

$$Z = \sum_{\boldsymbol{x} \in \mathcal{X}} \prod_{a \in A} w_a(\boldsymbol{x}_{S_a}). \tag{13.9}$$

可行配置计数是取 $w_a = \iota_a$ 的特例。

袋树上的计算与计数相同，只是把指示值换成权重。把本袋权重与子消息的乘积记为

$$h_u(\boldsymbol{x}_{B_u}) = \prod_{a \in A_u} w_a(\boldsymbol{x}_{S_a}) \cdot \prod_{c \in \mathrm{ch}(u)} z_{c \to u}(\boldsymbol{x}_{S_c}),$$

再对接口外状态求和，得到消息 $z_{u \to p(u)}(\boldsymbol{x}_{S_u}) = \sum_{\boldsymbol{x}_{I_u}} h_u(\boldsymbol{x}_{B_u})$；若没有接口外变量，就直接把这个乘积作为消息。消息表照样以接口为索引，但不再需要回溯表：计数和加权总和是对全部配置的汇总，没有“达到它的配置”。

链上这一递推就是矩阵乘法：以 $z_{k-1 \to k}$ 为行向量、以成对权重 $w_{k-1,k}(x_{k-1}, x_k)$ 为矩阵，求和 $\sum_{x_{k-1}} z_{k-1 \to k}(x_{k-1})\, w_{k-1,k}(x_{k-1}, x_k)$ 正是行向量右乘矩阵；一般袋上对多个变量相乘再求和，则是沿指定轴汇总的张量运算。这些运算可参考卷二第 1 章“向量、矩阵与张量的运算”。

13.4.3 无保留变量时的计数与求和汇总

不保留变量时，根 r 合并得到 h_r，再对根袋的全部状态求和：

$$Z = \sum_{\boldsymbol{x}_{B_r}} h_r(\boldsymbol{x}_{B_r}), \tag{13.10}$$

输出一个总计数或加权总和。根袋没有变量时（例如只用来连接各分量的空袋），直接取它的合并值。

原始图不连通时，各分量的根经空接口交来标量，即各分量的总计数或总和；空接口意味着结果是单表项标量，但发出它的节点仍须对自己袋中的状态求和。这些标量作为因子乘进根的合并值，所以非连通分量的结果相乘。这与有限集合的乘法原理一致：各分量的取值互相独立地组合。

三元作用域例子 表 13.4给出三元作用域例子的权重表，权重恰在禁止的组合处为 0。在分解 $\{1,2,3\} - \{2,3,4\}$ 上，表 13.5对接口的每种取值并列三种任务的左袋消息和右袋汇总值，最后一行是根处的结果：最小成本为 0，可行配置有 10 个，权重总和为 19。逐一枚举 16 个配置可以核对这三个数，其中最小成本和可行配置数已在第 11.1.4小节的枚举中得到。

表 13.4 三元作用域例子的非负局部权重。第一列的三位二进制串对 w_A 表示 (x_1, x_2, x_3)，对 w_B 表示 (x_2, x_3, x_4)；权重为 0 的位置恰是表 11.1中成本为 $+\infty$ 的位置。

取值	000	001	010	011	100	101	110	111
w_A	1	2	1	0	2	1	1	1
w_B	1	1	2	0	0	2	1	2

表 13.5 三种任务在同一接口上的消息。最小成本使用 min 与 +，可行计数与加权总和使用 $\sum$ 与 ×。“左”列是从 $\{1,2,3\}$ 发出的消息，对 x_1 汇总；“右”列是根袋 $\{2,3,4\}$ 对 x_4 的汇总；根处对每个接口取值组合两列（最小成本相加，其余相乘），再对接口汇总（最小成本取最小，其余求和）。

	最小成本		可行计数		加权总和	
(x_2, x_3)	左	右	左	右	左	右
$(0,0)$	0	1	2	2	3	2
$(0,1)$	0	0	2	1	3	2
$(1,0)$	1	0	2	1	2	2
$(1,1)$	2	1	1	2	1	3
根处	$\min(1,0,1,3)=0$		$4+2+2+2=10$		$6+6+4+3=19$	

以接口取 $(1,1)$ 的一行为例：$x_1=0$ 的一行被禁止，左袋的最小成本消息为 $\min(+\infty,2)=2$，计数消息为 $0+1=1$，加权消息为 $0+1=1$。三种任务的计算量与表大小相同，但输出含义不同：最小成本是一个配置的值，可以回溯出达到它的配置；计数和加权总和是对全部配置的汇总。

13.4.4 保留变量的累计表

给定保留集 R，使用根袋覆盖 R 的查询结构（第 13.3.2小节），非根节点照常计算，根处只对 $B_r \setminus R$ 求和：

$$Z_R(\boldsymbol{x}_R) = \sum_{\boldsymbol{x}_{B_r \setminus R}} h_r(\boldsymbol{x}_{B_r}). \tag{13.11}$$

结果以 $\boldsymbol{x}_R$ 为索引，逐项表示把 R 固定成这组值时的可行配置数（指示权重）或权重总和。再对所有 $\boldsymbol{x}_R$ 求和，才得到第 13.4.3小节的总量 Z。

与条件最小成本表相比，累计表的每一项不是一个配置的值，而是一组配置的累计；它也不需要回溯表。不含 R 的分量交来的标量作为因子乘到表的每一项上。

13.4.5 计算正确性

求和的三条规则 最小成本递推所用的三条最小值规则，在求和与相乘下有完全对应的形式。设 y, z 在有限集合中取值，h, g 取非负有限值，c 与 y 无关，则

$$\sum_{y,z} h(y,z) = \sum_{z}\sum_{y} h(y,z), \qquad \sum_{y} c\,g(y) = c\sum_{y} g(y),$$

并且对取值于互不相交变量组 $y_1, \dots, y_k$ 的函数，$\sum_{y_1,\dots,y_k} \prod_{j} g_j(y_j) = \prod_{j} \sum_{y_j} g_j(y_j)$。第一条是有限求和的重排；第二条是乘法对加法的分配律；第三条由分配律展开乘积得到，展开后的每一项恰对应各 y_j 的一组取值。

消息的含义 仿照定理 13.2，记号同前，求和任务的消息满足

$$z_{u\to p(u)}(\boldsymbol{x}_{S_u}) = \sum_{\boldsymbol{x}_{V_{D_u}\setminus S_u}} \prod_{a\in A_{D_u}} w_a(\boldsymbol{x}_{S_a}),$$

并且 $Z = \sum_{\boldsymbol{x}_{B_r}} h_r(\boldsymbol{x}_{B_r})$。证明逐字沿用那里的拆分：变量集合 I_u 与各 $V_{D_{c_j}} \setminus B_u$ 两两不交，并恰好组成 $V_{D_u} \setminus S_u$；固定 $\boldsymbol{x}_{B_u}$ 后，各子树的乘积只依赖互不相交的变量组，由第三条规则分别求和，由归纳假设得到各子消息；归入 u 的权重由第二条规则提到求和号外；最后由第一条规则对 $\boldsymbol{x}_{I_u}$ 求和。

这一论证同时说明每个配置恰被累计一次。下方变量的每一组取值，唯一地分解为内部变量的一组取值和各子树中变量的一组取值；分配律展开后，每组取值恰好贡献一个乘积项 $\prod_{a\in A_{D_u}} w_a$。取指示权重时，这个乘积是 1 或 0，消息就是可行取值的个数，根处给出可行配置数。

逐项固定保留变量，同样的论证给出累计表 (13.11)的正确性，全表求和得到总量。固定输入、保留集和输出要求后，仅改变待求和变量的消去次序不改变结果，因为有限求和可以任意重排；定理 13.1的对应关系也照样成立。

共同的代数结构 三种任务共用同一套递推，是因为它们的运算满足同样的代数规则。每种任务都用到两种运算，分工不同：一种把同一配置的各局部项组合成这个配置的值（相加或相乘），另一种把不同取值的结果汇总成一个数（取最小或全部加起来）。消息之所以能在局部提前汇总，靠的是“先组合再汇总”与“先汇总再组合”结果相同，例如 $a + \min(b,c) = \min(a+b, a+c)$、$a(b+c) = ab + ac$：与被汇总变量无关的项 a 可以留到汇总之后再组合。

把这些规则抽象出来。设集合 S 上有两种运算：$\oplus$ 用于跨不同取值的汇总，$\otimes$ 用于在一个配置内组合各局部项。若两种运算都满足结合律和交换律，各有单位元 $\bar{0}$ 与 $\bar{1}$，满足 $\bar{0}\otimes a=\bar{0}$，并且 $\otimes$ 对 $\oplus$ 满足分配律

$$a\otimes(b\oplus c)=(a\otimes b)\oplus(a\otimes c),$$

就称 $(S,\oplus,\otimes,\bar{0},\bar{1})$ 为交换**半环**（semiring）。

最小成本对应 $(\mathbb{R}\cup\{+\infty\},\min,+,+\infty,0)$，此时分配律就是 $a+\min(b,c)=\min(a+b,a+c)$；计数对应自然数上的 $(+,\times,0,1)$；加权求和对应非负实数上的 $(+,\times,0,1)$；只判断是否存在可行配置时，还可以用布尔值上的“或”与“且”。把变量消元和袋消息中的 min 换成 $\oplus$、$+$ 换成 $\otimes$，前面的正确性证明逐字成立，因为证明只用到了结合律、交换律和分配律。半环不要求减法或除法。

13.4.6 求和与取最小的顺序

上面的换序结论只针对同一种汇总：纯最小化中可以任意交换取最小的次序，纯求和中可以任意交换求和的次序。把本节的求和与第 13.2节的取最小放在一起，情况就不同了。

取一张两行两列的成本表 $h(y,z)$，两行分别为 $(0,1)$ 与 $(1,0)$，即 $h(0,0)=h(1,1)=0$、$h(0,1)=h(1,0)=1$。每行先对 z 求和得 1 与 1，再对两行取最小，得到

$$\min_{y}\sum_{z}h(y,z)=\min(1,1)=1.$$

反过来先对每一列在两行之间取最小，两列都得到 0，再求和，得到

$$\sum_{z}\min_{y}h(y,z)=0+0=0.$$

前者要求所有 z 共用同一个 y，后者允许 y 随 z 改变，所以两种顺序回答的是不同的问题。

混合汇总　不同变量混合使用求和与取最小（或取最大）时，须按所求结果确定运算顺序，不能直接沿用纯求和或纯最小化的换序结论。例如对表 $h(x,y)$，$h(0,0)=h(1,1)=4$、$h(0,1)=h(1,0)=0$，有

$$\max_{x}\sum_{y}h(x,y)=4,\qquad\sum_{y}\max_{x}h(x,y)=8.$$

这类混合问题只能按限定的次序消元：被外层汇总的变量必须最后处理，消元次序因而受到约束，所需的中间表可能比单一汇总时大。把非负权重

归一化为概率还要求总和为正且有限；这些概率解释、边缘化查询以及带环图上的近似消息方法，都留到后续的概率推断内容。

13.4.7 四环算例与结果检验

约束指示表与权重表 四环算例的约束指示表由表 11.4读出：只有 $\iota_{3,4}(1,1) = 0$ 与 $\iota_{1,4}(0,1) = 0$，其余指示值都是 1。另取一组非负权重，在被禁止的组合处为 0，其余取 1 或 2，见表 13.6；这些权重是教学示例，与成本表的具体数值无关。

表 13.6 四环算例的非负局部权重。行列约定与表 11.4相同；权重为 0 的位置恰是成本为 $+\infty$ 的位置。

$w_{1,2}(x_1,x_2)$	$w_{2,3}(x_2,x_3)$	$w_{3,4}(x_3,x_4)$	$w_{1,4}(x_1,x_4)$
$\begin{bmatrix}1 & 2\\ 2 & 1\end{bmatrix}$	$\begin{bmatrix}2 & 1\\ 1 & 1\end{bmatrix}$	$\begin{bmatrix}1 & 2\\ 1 & 0\end{bmatrix}$	$\begin{bmatrix}1 & 0\\ 1 & 2\end{bmatrix}$

总量 在分解 $\{1,2,4\} - \{2,3,4\}$ 上以 $\{2,3,4\}$ 为根。叶节点对 x_1 求和：计数消息为

$$n(x_2,x_4) = \sum_{x_1} \iota_{1,2}\,\iota_{1,4} = (2,\ 1,\ 2,\ 1),$$

加权消息为 $z(x_2,x_4) = \sum_{x_1} w_{1,2}\,w_{1,4} = (1\cdot 1 + 2\cdot 1,\ 1\cdot 0 + 2\cdot 2,\ 2\cdot 1 + 1\cdot 1,\ 2\cdot 0 + 1\cdot 2) = (3,4,3,2)$，取值按 (x_2,x_4) 的字典序排列。

根处把消息与 $\phi_{2,3},\phi_{3,4}$ 对应的指示值或权重相乘，按 (x_2,x_3,x_4) 的字典序，计数的合并值为 $(2,1,2,0,2,1,2,0)$，求和得 $N = 10$；加权的合并值为 $(6,16,3,0,3,4,3,0)$，求和得 $Z = 35$。

用表 11.5核对：可行配置恰有 10 个。逐个计算可行配置的权重，按字典序依次为 $2,1,2,2,4,16,2,1,4,1$，总和为 35。例如最优配置 $(1,0,0,1)$ 的权重为 $w_{1,2}(1,0)w_{2,3}(0,0)w_{3,4}(0,1)w_{1,4}(1,1) = 2\cdot 2\cdot 2\cdot 2 = 16$。

保留变量的累计表 在 $R = \{1,3\}$ 的查询结构上，两个子节点分别对 x_2 与 x_4 求和，根袋 $\{1,3\}$ 把两条消息相乘，不再求和。按 (x_1,x_3) 的字典序，计数消息为 $(2,2,2,2)$ 与 $(1,1,2,1)$，累计表为

$$N_R = (2,\ 2,\ 4,\ 2);$$

加权消息为 $(4,3,5,3)$ 与 $(1,1,5,1)$，累计表为

$$Z_R = (4,\ 3,\ 25,\ 3).$$

全表求和分别为 $2+2+4+2=10=N$ 与 $4+3+25+3=35=Z$。逐项核对：例如 $x_1=1,x_3=0$ 时可行配置为 $(1,0,0,0),(1,0,0,1),(1,1,0,0),(1,1,0,1)$，共 4 个，权重为 $4+16+1+4=25$。

结果的解释与边界　计数为零表示没有可行配置：第 13.2.6小节的不可行约束变体中，叶节点的计数消息为 $(0,1,1,0)$，根处要求 $x_2=x_3=x_4$ 时这条消息为 0，于是 $N=0$，与最小成本为 $+\infty$ 一致。一般的加权和为零却不直接表示不可行：权重可以在可行的组合上取 0，例如把 $w_{1,2}$ 全部改为 0，则 $Z=0$，而可行配置仍有 10 个。

第 13.2.6小节加入孤立变量 $x_5\in\{0,1,2\}$ 和边 $\{6,7\}$ 后，两个新分量经空接口交来标量。袋 $\{5\}$ 没有局部项，$h\equiv 1$，计数消息为 3。若 $\phi_{6,7}$ 的四项都有限，袋 $\{6,7\}$ 的计数消息为 4；若再给它权重表 $\begin{bmatrix}1 & 1\\ 2 & 1\end{bmatrix}$，加权消息为 5。根把这些标量乘进合并值，得 $N=10\cdot 3\cdot 4=120$、$Z=35\cdot 3\cdot 5=525$。若错把孤立变量的计数设为 1，结果会少乘一个因子 3。

13.5　计算代价

前几节的计算都是精确的，但精确不等于便宜。本节把已构建的袋和接口换算成实际的状态组合数，分别分析求值的时间、存储及二者的取舍。怎样构造结构由第 12.5节负责；本节依据这里的资源指标比较构建结果，并把构造与核验的成本计入任务的总成本。

13.5.1　状态表的规模

先固定规模变量：变量个数 n，局部项个数 $|A|$，袋树节点数 N，第 i 个变量的状态数 $|\mathcal{X}_i|$，以及 $q=\max_i|\mathcal{X}_i|$。对一组变量 U，记它上面的配置数为

$$|\mathcal{X}_U|=\prod_{i\in U}|\mathcal{X}_i|.$$

计算中出现三类表：袋 u 上的合并表有 $|\mathcal{X}_{B_u}|$ 项，接口消息和回溯表各有 $|\mathcal{X}_{S_u}|$ 项（回溯表的每一项存 $|I_u|$ 个变量的取值），保留变量的输出表有 $|\mathcal{X}_R|$ 项。

在每个变量至多 q 个状态、分解宽度为 w 时，每个袋至多 $w+1$ 个变量，袋表至多 q^{w+1} 项。但这只是上界：接口与输出仍按实际变量计算，状态数不同的变量对乘积的贡献也不同。比较两个候选结构时，应固定同一任务（同一保留集、同一输出要求），并使用实际的乘积，而不是只比较宽度。例如两个结构宽度都是 2，一个的最大袋由三个二值变量组成（8 项），另一个含一个十状态变量（至少 40 项），实际规模相差数倍。

宽度出现在指数上，这是精确计算的主要限制。$q=10$、$w=10$ 时一张袋表就有 10^{11} 项；若查询要求保留很多变量的联合表，输出本身也可能大得不可承受，这时任何精确方法都必须至少写出整张表。

四环算例中，分解 $\{1,2,4\}-\{2,3,4\}$ 的两个袋各有 8 项，接口消息和回溯表各有 4 项；$R=\{1,3\}$ 的查询结构有两个 8 项的袋、一个 4 项的根袋，输出表 4 项。按次序 $(1,2,3,4)$ 不合并时，四个消元簇的配置数为 $8,8,4,2$。

13.5.2 时间代价

约定表查询、加法、乘法和比较各计一次基本运算。处理节点 u 时，要对 $\boldsymbol{x}_{B_u}$ 的每种取值读取归入 u 的每个局部项一次、读取每条子消息一次，把它们组合起来，再参与一次取最小或求和，所以工作量约为 $|\mathcal{X}_{B_u}|\,(|A_u|+|\operatorname{ch}(u)|+1)$。对全部节点求和，利用袋树中 $\sum_u |\operatorname{ch}(u)| = N-1$，求值时间为

$$\sum_{u} |\mathcal{X}_{B_u}|\,\big(|A_u|+|\operatorname{ch}(u)|+1\big) = O\big((|A|+N)\,q^{w+1}\big).$$

单袋的 q^{w+1} 界须结合袋数和逐项工作量才得到总时间。

按消元次序计算时，消去 x_i 要对 $\{i\}\cup S_i$ 的至多 $q^{|S_i|+1}$ 种取值各读取 $\mathcal{F}_i$ 中每个函数一次，时间约为 $|\mathcal{F}_i|\,q^{|S_i|+1}$。每个函数恰好在它的某个变量被消去时被取出一次，原有函数 $|A|$ 个，新函数至多 n 个，所以各步的 $|\mathcal{F}_i|$ 之和不超过 $|A|+n$；按变量维护索引的开销也与函数个数成正比。若诱导宽度为 w，总时间为 $O\big((|A|+n)\,q^{w+1}\big)$，而枚举为 $|A|\,q^{n}$。

除向上的求值外，还要计入结果输出与配置恢复。条件最小成本表或累计表的输出要写出 $|\mathcal{X}_R|$ 项。恢复一个配置时每个节点只查一次回溯表，共 $O\big(\sum_u |B_u|\big)$ 次读写，与求值相比可以忽略。若还需要每个袋上各取值的最小总成本，就像第 11.2.3.4小节那样再按前序发送一遍反向消息，时间约加倍。

结构构造与核验的时间另列，再计入任务总量：模拟消元为 $O\big(n+m+\sum_t d_t^2\big)$（第 12.2.4小节），检验为袋大小的多项式（第 12.5.5小节）。同一结构用于多组数值时，这部分只付一次。

计数中的整数加乘在上面各计一次基本运算。若要考虑位复杂度，还须按数值的位数分析：可行配置数至多为 $\prod_i |\mathcal{X}_i| \le q^{n}$，需要约 $n\log_2 q$ 位，大规模问题中一次加乘的实际代价随位数增长；加权求和若用浮点数，还要另行分析舍入误差。

13.5.3 存储代价

存储要分清对象，并按计算过程中同时保留的对象计算峰值，不能把累计生成的表项总量或单个接口的规模直接当作峰值。计算中要存放的对象有四类：

- 接口消息：每张 $|\mathcal{X}_{S_u}|$ 项，从生成起保留到父节点用完为止；
- 临时袋表：处理节点 u 时的合并表，若整张存放有 $|\mathcal{X}_{B_u}|$ 项；
- 达到值记录：每个非根节点一张回溯表，$|\mathcal{X}_{S_u}|$ 项，每项存 $|I_u|$ 个取值，要保留到恢复结束；
- 输出表：保留变量查询的 $|\mathcal{X}_R|$ 项。

临时袋表不必整张存放。对接口的每个取值，可以逐个枚举内部变量的取值，现场计算 g_u 的一项并立即并入当前的最小值（或累加和），算完一个接口取值就写出消息的一项。这样处理节点 u 时只需 $O(|B_u|)$ 的工作空间，而不是 $|\mathcal{X}_{B_u}|$；代价是对局部项和子消息的读取顺序不再连续，常数因子可能变大。

消息的保留时间取决于后序。只要最优值、不恢复配置时，子节点的消息在父节点处理完后即可释放，所以任一时刻保留的只是“已处理完、父节点尚未处理”的那些节点的消息。链状袋树上同时只有一条消息；一个根下挂很多叶节点时，处理根之前全部叶节点的消息同时存在。峰值是这些同时存在的消息之和，加上当前节点的工作空间，而不是全部消息的总和。

需要恢复配置时，回溯表要保留到前序回溯走完，存储为 $\sum_u |\mathcal{X}_{S_u}| \cdot |I_u|$ 量级，这时可以释放的只有消息。四环算例中，最小成本计算的峰值是一条 4 项消息加根处的工作空间；恢复配置还要保留一张 4 项的回溯表。$R = \{1, 3\}$ 的查询中，处理根之前两条 4 项消息同时存在，输出表另有 4 项。

13.5.4 时间与存储的取舍

固定同一结构和输出要求，可以在三种做法之间取舍：缓存中间结果、在最后一次使用后释放、或者按需重算。

最直接的是全部缓存：保留所有消息和回溯表，向上一遍后既能恢复配置，也能再发一遍反向消息得到每个袋的表，时间最少，存储最多。只需最优值时，在最后一次使用后立即释放消息，存储降到上一小节的峰值，时间不变。

回溯表也可以不存。若保留各条子消息，恢复时对每个节点 u 在已知接口取值下重新枚举 $\boldsymbol{x}_{I_u}$、重算 g_u 并取最小点，就能得到与回溯表相同的选择（并列规则保持一致即可）；额外时间是每个节点 $|\mathcal{X}_{I_u}|$ 量级的重算，存储从回溯表换成消息。若连消息也释放了，恢复时就要重新计算子树的消

息，时间可能成倍增加。释放后仍需使用的信息，必须重算或从另存的记录恢复，不能凭空得到。

是否恢复配置、是否保存整张保留变量表，都会改变记录的保留时机。只要整体最优值时，回溯表根本不必生成；只要条件最小成本表而不要条件最优配置时，根处的回溯表可以省去；只查询少数几个 $\boldsymbol{x}_R$ 而不是整张表时，可以对每个查询值把 $\boldsymbol{x}_R$ 固定后按第 13.2节计算一次，用查询次数乘以单次时间换取不必存放 $|\mathcal{X}_R|$ 项的输出表。哪种做法更合适，取决于结构的实际规模和输出要求，应在固定任务后按上面的资源指标比较。

13.6 本章小结

本章在已核验的结构上完成数值计算。所有任务共用同一套组织：每个局部项恰好归入一个容纳其作用域的袋，按后序由叶到根处理，每个节点合并本地的局部项与子节点交来的表，对内部变量汇总，把以接口为索引的结果交给父节点。按消元次序计算与沿消元森林计算得到同样的中间函数；合并袋、调整归属或改变同类汇总的次序会改变中间表，不改变最终结果。

最小成本问题同时求值和恢复最优配置：合并用加法，汇总取最小，取最小时记下达到值，根选定最优状态后沿前序查回溯表，等价于按消元的逆序回溯；最小值为 $+\infty$ 时报告不可行，并列最优按固定规则选取。带保留变量时，查询结构使根袋覆盖保留集，保留变量从不在非根节点被汇总，根处只对其余变量取最小，输出各固定配置下的最小成本，并可恢复相应的条件最优解。计数和求和把加法换成乘法、取最小换成求和，消息表示下方的可行取值数或权重总和，非连通分量的结果相乘，孤立变量贡献其状态数；它们不需要回溯表。

正确性都来自同一个不变量：消息等于固定接口后下方全部局部项的最优值或累计值。它依赖三项条件：运行交集使接口隔开两侧，顶点覆盖使根处涉及全部变量，唯一归属使每项只计一次；再加上组合与汇总运算的结合律、交换律和分配律。这些规则只保证同一种汇总可以换序，求和与取最小混用时不能交换，四项小表就给出 1 与 0 两个不同结果。

实际代价由袋、接口、各变量的状态数、输出要求与存储安排共同决定。宽度给出袋表 q^{w+1} 的上界，比较结构时应使用实际的状态数之积；求值时间约为各袋配置数乘以逐项工作量之和，结构构造与核验另计；存储按同时保留的消息、回溯表和输出计算峰值，可以用释放和重算换取存储。四环算例在全部任务上与枚举逐项一致。

至此，本部分从局部函数出发，经链和树上的接口递推、一般图上的结构构建，完成了有限状态配置问题的精确计算。图上的另一类问题是顶点上的数值已经给定，研究它们沿边怎样变化，怎样通过图上的线性算子进

行变换和滤波，这类问题所需的内积、二次型与谱分解工具可参考卷九第 9 章“图上函数与图算子”。

参考文献

D. P. Bertsekas, *Auction Algorithms for the Assignment Problem, Chapter 2 of Auction Algorithms for Path Planning, Network Transport, and Reinforcement Learning*, Mar. 15, 2025, URL: https://www.mit.edu/~dimitrib/Ch2_AUCTION.pdf.

D. P. Bertsekas, *Linear Network Optimization: Algorithms and Codes*, MIT Press, 1991.

D. P. Bertsekas, *Network Optimization: Continuous and Discrete Models*, Athena Scientific, 1998.

D. P. Bertsekas, *New Auction Algorithms for the Assignment Problem and Extensions* In Results in Control and Optimization, 2024, vol. 14, pp. 100383.

R. Kyng, *Classical Algorithms for Maximum Flow II, Advanced Graph Algorithms and Optimization, Lecture 11*, ETH Zürich, May 6, 2020, URL: https://kyng.inf.ethz.ch/courses/AGAO20/lectures/lecture11_maxflow-contd.pdf.

NetworkX Developers, *Source Code for networkx.algorithms.flow.networksimplex*, 2026, URL: https://networkx.org/documentation/stable/_modules/networkx/algorithms/flow/networksimplex.html.

R. Sedgewick, K. Wayne, *BipartiteMatching.java*, 2025, URL: https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/BipartiteMatching.java.html.

R. Sedgewick, K. Wayne, *HopcroftKarp.java*, 2025, URL: https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/HopcroftKarp.java.html.

R. Sedgewick, K. Wayne, *Maximum Flow*, June 14, 2017, URL: https://algs4.cs.princeton.edu/64maxflow/.

N. Zeh, *A Faster Implementation of the Hungarian Algorithm*, Dalhousie University, URL: https://web.cs.dal.ca/~nzeh/Teaching/4113/

book/matching/bipartite_perfect_matching/hungarian_fast.html.

N. Zeh, *The Hungarian Algorithm*, Dalhousie University, URL: https://web.cs.dal.ca/~nzeh/Teaching/4113/book/matching/bipartite_perfect_matching/hungarian_initial.html.

索引

一元项，unary term，198
一维 Weisfeiler–Leman 检验,1-dimensional Weisfeiler–Leman test，67
二部图，bipartite graph，118
ε 互补松弛，epsilon-complementary slackness，134
交替树，alternating tree，127
交替路径，alternating path，119
作用域，scope，211
父指针，parent pointer，34
父节点，parent，20
状态集合，state set，210
环，cycle，10
瓶颈容量，bottleneck capacity，104
生成树，spanning tree，21
生成森林，spanning forest，21
相等子图，equality subgraph，126
祖先，ancestor，12
简单图，simple graph，5
紧边，tight edge，126
约化费用，reduced cost，142
编辑距离，edit distance，80
ε 缩放，epsilon-scaling，135
缩点图，condensation graph，55
网络单纯形法，network simplex method，138
自环，self-loop，5
节点势，node potential，142
行走，walk，10
袋，bag，232
覆盖函数，coverage function，169
词典序规则，lexicographic rule，148
诱导子图，induced subgraph，8
诱导宽度，induced width，240
负环，negative cycle，75
超图，hypergraph，213
超模，supermodular，171
超边，hyperedge，213
距离标签，distance label，76
路径，path，10
轻边，light edge，93
边，edge，4
边权，edge weight，6
边表，edge list，22
边际增益，marginal gain，164
运行交集性质，running intersection property，234
连通，connected，13
连通分量，connected component，13
退化换基，degenerate pivot，147
邻域，neighborhood，8
邻接，adjacent，8
邻接矩阵，adjacency matrix，24
邻接表，adjacency list，23
配置，configuration，210
闭行走，closed walk，10
阻塞流，blocking flow，111
集合函数，set function，162
顶点，vertex，4
预算约束，budget constraint，192
颜色细化，color refinement，65

Auction 算法，auction algorithm，131

Ford–Fulkerson 方法，Ford–Fulkerson method，104

Kahn 算法，Kahn’s algorithm，57
KM 算法，Kuhn–Munkres algorithm，126
Kosaraju 算法，Kosaraju’s algorithm，52

Möbius 函数，Möbius function，180
Möbius 反演，Möbius inversion，174

入度，in-degree，8
关联矩阵，incidence matrix，26
内部变量，internal variable，236
出度，out-degree，8
分层网络，level graph，111
分隔，separate，15
分隔集，separator，15
前驱，predecessor，12
割，cut，93
割容量，cut capacity，102
割点，cut vertex，16
动态规划，dynamic programming，217
匈牙利算法，Hungarian algorithm，120
匹配，matching，118
半环，semiring，284
单纯顶点，simplicial vertex，255
单调非减，monotone nondecreasing，166
原始关系图，primal graph，212
反向图，reverse graph，52
发现次序，discovery order，41
变量消元，variable elimination，227
可行标号，feasible labeling，125
可行配置，feasible configuration，212
可达，reachable，11
叶，leaf，18
同构，isomorphic，28
后代，descendant，12
后继，successor，12
回溯表，backpointer table，217
回边，back edge，43
因子，factor，211
因子图，factor graph，212
团，clique，232
团树，junction tree，259
图，graph，4
图不变量，graph invariant，29
图割，Graph cut，201
基数约束，cardinality constraint，184
基本环，fundamental cycle，144
填充图，filled graph，239
填充边，fill-in edge，238
增广路径，augmenting path，104
多源广度优先遍历，multi-source breadth-first search，38
多重集合，multiset，64
子图，subgraph，7
子树，subtree，21
子节点，child，20
子集格，subset lattice，172
完成次序，finishing order，41
完美匹配，perfect matching，118
完美消元序，perfect elimination ordering，255
局部函数，local function，211
局部块，local block，61
平行边，parallel edges，5
并查集，union-find，48
广度优先搜索，breadth-first search，35
底层无向图，underlying undirected graph，6
底集，ground set，162
度，degree，8
弦，chord，256
弦化，chordal completion，256
弦图，chordal graph，256
弧，arc，5
弱连通，weakly connected，14
弱连通分量，weakly connected component，14
强连通，strongly connected，14
强连通分量，strongly connected component，14
当前弧，current arc，112
循环流，circulation，140

惰性贪心，lazy greedy，189
成对项，pairwise term，198
拓扑序，topological order，56
指派问题，assignment problem，125
指示向量，indicator vector，163
接口，interface，215
无向图，undirected graph，5
最大匹配，maximum matching，118
最大团，maximum clique，259
最大流，maximum flow，102
最大流最小割定理，max-flow min-cut theorem，110
最大生成树，maximum spanning tree，93
最小割，minimum cut，106
最小填充启发式，min-fill，253
最小度启发式，min-degree，253
最小生成树，minimum spanning tree，92
最小生成森林，minimum spanning forest，93
最小费用最大流，minimum-cost maximum flow，140
最小费用流，minimum-cost flow，139
最短路，shortest path，75
最短路树，shortest-path tree，93
有向图，directed graph，5

有向无环图，directed acyclic graph，11
有向环，directed cycle，11
有向路径，directed path，11
有根树，rooted tree，20
条件最小成本表，conditional minimum-cost table，277
松弛，relaxation，76
极大匹配，maximal matching，118
极大团，maximal clique，259
树，tree，17
树分解，tree decomposition，233
树基，spanning tree basis，144
树宽，treewidth，235
桥，bridge，16
森林，forest，17
模函数，modular function，167
次模，submodular，168
残量网络，residual network，103
流，flow，101
流值，flow value，102
流守恒，flow conservation，102
流网络，flow network，101
消元森林，elimination forest，241
消元次序，elimination order，238
消元簇，elimination cluster，240
消息，message，221
深度优先搜索，depth-first search，41